Desktop Assist

Code Guide

Morse Micro · 12 October 2026

How the app fits together, and the actual code of every function and method with an explanation. Each part starts on a new page; links within the guide jump to the right place, and each code block's caption links to its lines on GitHub.

Contents

1. Overview
2. Shared code and the preload bridge
3. Main process: startup, IPC and app plumbing
4. JumpCloud sign-in
5. Talking to Claude
6. The bubble: windows, layout, bounce and the state machine
7. The draft, screenshots and safe files
8. The pages: entry points, styles, hooks and views
9. The UI components
10. Claude Desktop: handing questions over
11. Your apps: the JumpCloud apps list
12. Jira and the service desk
13. The Jira and service desk pages

Desktop Assist: Code Guide

A walk through the whole project: what every file is for, and the actual code of every function and method with an explanation of what it does and why. Covers Milestones 0 (Foundation), 1 (Chat with Claude), 2.1 (bubble fixes) and 3 (JumpCloud sign-in), and features 3.1 (questions go to Claude Desktop) and 3.2 (the Apps list, Uninstall in Settings, and fixes), and Milestone 4 (app logos fixed, favourite apps, the IT service desk icon, light or dark mode, and a fix for clicks on the bubble), and Milestone 5 so far (a spinner while an app opens, and Jira and the IT service desk in the panel).

This page is the overview. The code itself is in the code walkthrough, one page per area of the app.

Contents:

1. Background: how an Electron app is put together
2. How the pieces work together
3. Repository root
4. Tenants (branding)
5. Desktop app configuration
6. Code walkthrough
7. Tests (`tests`)

---

1. Background: how an Electron app is put together

An Electron app is a Chromium browser and Node.js bundled together. It runs as several processes, and the code is split the same way:

- **Main process** (`apps/desktop/src/main`): Node.js, one per app. Creates the windows, reads and writes files, owns the tray icon and screen capture, and talks to JumpCloud and Claude (or hands questions to the Claude Desktop app).
- **Renderer** (`apps/desktop/src/renderer`): Chromium, one per window. Draws the UI with HTML, CSS and React. It **cannot** touch files, the network or the operating system.
- **Preload** (`apps/desktop/src/preload`): runs inside each renderer before the page loads, and hands the page a small, safe set of functions (`window.assist`).
- **Shared** (`apps/desktop/src/shared`): bundled into all three. Types and constants every side must agree on.

The renderer and the main process talk over **IPC** (inter-process communication): the renderer sends a named message (a *channel*), and the main process handles it and optionally replies. The main process can also push messages to the renderer, for example each chunk of a streaming Claude reply. Every channel is listed in `src/shared/ipc.ts`.

Why two windows? The bubble and the panel are separate, transparent, always-on-top windows. Both load the same web page; the URL's `?view=bubble` or `?view=panel` decides which UI it shows. Keeping the bubble in its own tiny window means it never resizes (no flicker when the panel opens), and only a 72×72 window moves around during Bounce.

Click-through. The panel window is a large rectangle, but most of it is transparent. So that you can still click whatever is underneath those empty areas, the panel window starts out *ignoring the mouse*. Windows still forwards pointer movement to the page, and while the pointer is over a real piece of UI (anything marked `data-hit`), the page asks the main process to switch mouse input back on. Each time the panel is shown, this is reset from the pointer's actual position. The bubble window is **not** click-through: it's barely bigger than the bubble, and depending on Windows' mouse forwarding there made clicks occasionally fall through the bubble to the window behind it.

Two ways to chat. Each tenant chooses where questions go (`chatApp` in `tenant.json`). With **Claude Desktop** (Morse Micro, since 3.1), the panel is just a text box: asking opens a new chat in the Claude Desktop app with the question typed in, in the person's own Claude account, so it counts against their own usage limit, and (unless the user turns it off) presses Enter there to send it. Desktop Assist then has no sign-in and doesn't call Claude itself. With the **built-in chat**, the conversation happens in the panel, as described next.

The built-in chat: where Claude is called, and how you're signed in. Only the main process talks to JumpCloud and to Claude. You sign in with JumpCloud in your browser; the app keeps the resulting *refresh token* (a long-lived "stay signed in" pass) encrypted on disk. To call Claude it swaps a JumpCloud *ID token* (a short-lived, signed statement of who you are) for a short-lived Claude token, using Anthropic's Workload Identity Federation. There is no API key. The pages only ever learn whether you're signed in and your name, never a token.

The Apps list. If the tenant has an app portal, the card can show the apps in the user's JumpCloud User Portal, either as soon as the panel opens (Morse Micro, with the text box behind an Ask Claude icon) or behind an Apps icon. They come from JumpCloud's "MCP Server for Users", which each person connects to once in the browser (OAuth, with no secret in the app). Again, only the main process talks to JumpCloud, and the pages never see a token.

Jira and the service desk. If the tenant has an Atlassian site, a bell icon opens a Jira page: who else assigned the user an issue or mentioned them lately, and a search of Jira and Confluence; an issue opens in the card, where Claude can be asked about it. With the service desk's request types, the service desk icon opens a page for raising an IT request (finished on the help centre portal) and following the user's own requests. It all goes through Atlassian's Remote MCP server, which each person connects to once in the browser, the same way as the Apps list (the shared code is in `src/main/mcp/`).

---

2. How the pieces work together

Starting up (`src/main/index.ts` → `start()`)

1. Validate `tenant.json`, create the bubble and panel windows (both hidden), load the saved draft from disk, and load preferences.
2. Create the `BubbleController` (the bubble's state machine) and the `ScreenshotService`. Then, depending on the tenant's `chatApp`, either a `ClaudeDesktop` (which hands questions to Claude Desktop) or the built-in chat (`startBuiltInChat()`: the Claude backend, the `ChatSession` and the `AuthManager`). With a `portal`, also `PortalApps` (the Apps list), which starts loading the list in the background. With `atlassian`, also `Atlassian` (Jira), which uses a saved connection if there is one and starts checking notifications; with the service desk's request types, also `ServiceDesk`. Then the IPC handlers and tray.
3. With the built-in chat, **renew the saved sign-in** (`auth.init()`): if there isn't one, or JumpCloud refuses it, the chat box shows **Sign in with JumpCloud** instead of the chat. If JumpCloud can't be reached, the sign-in is kept and a banner offers Retry.

4. Once both pages have painted, `controller.start()` shows the bubble in its saved corner (bottom-right of the main display the first time).

Asking in Claude Desktop (when the tenant's `chatApp` is `claude-desktop`)

1. Enter or **Ask in Claude** → `Panel.askInClaudeDesktop()` → IPC `assist:claude-desktop-ask` with the text → `ClaudeDesktop.ask()`.
2. `ask()` checks there's a question, that it's at most 12,000 characters (Claude Desktop cuts longer ones short), and that Claude Desktop is installed (something opens `claude://` links).
3. Any attached screenshots are copied to the clipboard as one image (`ScreenshotService.forClipboard()` stacks several).
4. The link `claude://claude.ai/new?q=…` opens a new chat in Claude Desktop with the question typed in but not sent. `ask()` returns, and the panel closes and the draft is cleared.
5. In the background (`ClaudeDesktop.finish()`), with **Send in Claude automatically** on and a question typed, `sendInClaude()` runs a hidden PowerShell helper. Using Windows UI Automation, it waits until the focused text box belongs to Claude Desktop and holds exactly the question, then presses Ctrl+V (if there's a screenshot) and Enter, checking again before each key. Claude answers.
6. If it can't send (or the setting is off, or there's only a screenshot), nothing is pressed anywhere else, and a Windows notification says what's left: press Ctrl+V to add the screenshot, or press Enter in Claude. If Claude's box already had other text in it (an unsent draft), nothing is pressed either, and the notification asks the user to check the question. Clicking a notification opens the panel.

Your apps (when the tenant has a `portal`; see Your apps)

1. At startup, `PortalApps` connects to JumpCloud's MCP server with the saved sign-in, calls its `list_applications` tool, fetches the logos and broadcasts the list (`assist:apps-state`).
2. The panel shows the list when it opens (with `"startPage": "apps"`, as for Morse Micro) or when the Apps icon is clicked, and asks for it then (`window.assist.apps.get()` → IPC `assist:apps-get` → `PortalApps.get()`), getting the list already loaded. The **Ask Claude** icon swaps the list for the text box and back; a dot on it means an unsent question is waiting there. Closing the panel (or Esc) goes back to the start page; taking a screenshot leaves the card on the page it was showing.
3. With no saved sign-in, the list offers **Sign in with JumpCloud**: the browser opens JumpCloud's sign-in page and comes back to `127.0.0.1:47622/callback`, and the MCP SDK registers the app, swaps the code for tokens (PKCE, no secret) and saves them encrypted (`jumpcloud-apps.bin`). The panel reopens on the list.
4. Clicking an app → `PortalApps.open()` calls `launch_application` for its sign-in link and opens it in the browser; the panel closes. Meanwhile a spinner turns over the app's logo; if JumpCloud takes more than 15 seconds, the link from the list is used instead.
5. The star on a tile → `settings.setFavoriteApp()` saves the app's ID (`favoriteApps` in `preferences.json`), and the list shows starred apps first (`orderApps()`).

Jira (when the tenant has `atlassian`; see Jira and the service desk)

1. The bell icon swaps the card to the Jira page. Without a saved connection it offers **Connect to Atlassian**: `Atlassian.signIn()` opens Atlassian's sign-in in the browser (through JumpCloud at Morse Micro), which comes back to `127.0.0.1:47623/callback`; the connection is saved encrypted (`atlassian.bin`) and the panel reopens.
2. Once connected, and then every 5 minutes, `Atlassian` searches Jira for issues recently assigned to the user or mentioning them, reads each one's history and comments to see who did it, checks the user's service desk requests, and broadcasts the result (`assist:jira-feed`). What the user has seen is remembered in

`atlassian.json`; new notifications put a dot on the bell, and IT replying to a request gives a Windows notification once.

3. Clicking an issue (from Notifications, Search, My requests, or **Open** beside an issue key typed in the text box) → `Atlassian.issue()` → `IssueView` over the page, with its comments. **Move**, **Comment** and **Log time** each make one tool call and show the issue as it is now. **Ask Claude** builds a question with the issue in it (`issuePrompt()`) and hands it to Claude Desktop, as if typed in the text box.

IT service desk (when the tenant has a `serviceDesk`)

Without request types: headset icon → `Panel.runAction('servicedesk')` → IPC `assist:invoke-action` → the `servicedesk` handler in `src/main/actions.ts` opens the service desk's address from `tenant.json` in the browser, then closes the panel.

With request types (`serviceDesk.requests`, as for Morse Micro), the icon, or **Report to IT** under the text box, opens the service desk page instead. **New request** starts from the text box (its first line as the summary), shows the PC's details and IT's how-to articles, and **Continue on the portal** → IPC `assist:service-desk-raise` → `ServiceDesk.raise()` copies the text box's screenshots to the clipboard, opens the portal's create-request form already filled in (the summary and description go in the address), and closes the panel; the user pastes the screenshot and sends it there. **My requests** lists the user's requests, marked when IT has replied or changed them.

Uninstall

Settings → **Uninstall Desktop Assist** (click twice) → IPC `assist:app-uninstall` → `uninstall()` in `src/main/uninstall.ts` turns off Start with Windows, starts the installer's "Uninstall Desktop Assist.exe" and quits. The data folder and screenshots are kept.

The rest of this section, up to **Typing in the text box**, is the built-in chat.

Signing in

1. **Sign in with JumpCloud** (`SignInPanel`) → `window.assist.auth.signIn()` → IPC `assist:auth-sign-in` → `AuthManager.signIn()`. The status becomes `signing-in` and the panel says to finish in the browser.
2. `listenForRedirect()` starts a tiny web server on `127.0.0.1:47621`, only on this PC. `createOidc().begin()` builds the JumpCloud sign-in address, with a fresh PKCE code, `state` and `nonce`, and the app opens it in your default browser.
3. You sign in at JumpCloud (or you already are), and JumpCloud sends the browser back to `http://127.0.0.1:47621/callback?code=…`. The little server answers "You can close this tab" and shuts down.
4. `finish()` swaps the code (plus the PKCE secret that only this app knows) for tokens, and checks the ID token. The refresh token and your name are saved encrypted (`SecretStore`, Windows DPAPI). The status becomes `signed-in` and the chat box reopens.

Getting Claude access (happens inside the Anthropic SDK, the first time you send a message and then roughly hourly)

1. The SDK needs a Claude token, so it asks `AnthropicBackend`'s identity-token function, which calls `AuthManager.freshIdToken()`.
2. That returns the unused ID token from signing in or starting up, or, if it has been used, refreshes with JumpCloud for a new one (saving JumpCloud's replacement refresh token).
3. The SDK posts the ID token to Anthropic's `/v1/oauth/token`. Anthropic checks it against the federation rule set up in the Claude Console and returns a short-lived Claude token, which the SDK uses for chat requests until shortly before it expires.

Sending a message

1. Enter in the text box → `Panel.send()` → IPC `assist:chat-send` with the text.
2. The main process collects the draft's attached screenshots, shrinks each one for Claude (`ScreenshotService.forClaude()`), and calls `ChatSession.send()`, then clears the draft.
3. `ChatSession` appends your message to the conversation history and starts a reply: `AnthropicBackend.reply()` streams it from the Claude API.
4. As text arrives, `ChatSession` updates the reply and broadcasts it (`assist:chat-message`, batched to at most one update every 50 ms). The page re-renders it as Markdown.
5. When the reply finishes, the whole reply (including Claude's hidden thinking blocks) is appended to the history unchanged, ready to be sent back with the next message.

Stop, errors and Retry

- **Stop** aborts the request. Text that already arrived stays on screen and is added to the history as a plain-text reply (the history is only ever added to, never edited; see `ChatSession`).
- **A temporary error** (offline, overloaded) before any text arrived shows the reason and a **Retry** button, which sends the same conversation again.
- **The sign-in has expired** (JumpCloud refuses the refresh): `AuthManager` signs out, and the chat box shows Sign in with JumpCloud with an explanation. The conversation is kept, and the failed message offers Retry once you're signed back in.
- **Anthropic refuses the swap** (the federation rule doesn't accept you): the message explains that IT needs to set you up.

Typing in the text box

`Composer` textarea → `Panel.changeDraft()` → `window.assist.notes.setText()` → `NotesStore.setText()`. The store keeps the draft in memory and writes `notes.json` about half a second after you stop typing, so an unsent message survives a restart.

Taking a screenshot

Camera icon → `Panel.runAction('screenshot')` → IPC `assist:invoke-action` → the `screenshot` handler in `src/main/actions.ts` → `BubbleController.whileHidden()` hides both windows, waits for Windows to repaint, then `ScreenshotService.capture()` saves the PNG. The windows come back and the page shows a "Screenshot saved" toast. **Attach screenshot** then adds it to the draft as a chip, and it's sent with your next message (with Claude Desktop, put on the clipboard and pasted into Claude).

Bounce

Bounce icon → `BubbleController.startBounce()` hides the panel and runs a 60 fps timer that calls `bounce.step()` and moves the bubble window across all the monitors. Clicking the bubble calls `glideHome()`, which animates it back to its corner of its own display with `bounce.glidePosition()`.

Light or dark mode

Sun or moon icon → `Panel.runAction('theme')` → `window.assist.settings.setTheme()` → IPC `assist:settings-set-theme` → `SettingsService.setTheme()` sets Electron's `nativeTheme.themeSource`, which every page sees as `prefers-color-scheme`, so both windows switch at once, and saves the choice. Desktop Assist is dark until the user switches, whatever Windows uses; `SettingsService.init()` applies the saved mode before any window shows.

Clicking the bubble

`BubbleView` notes when the mouse button goes down, and on the click calls `window.assist.bubbleClick(pressedAt)` → IPC `assist:bubble-click` → `BubbleController.clickBubble(pressedAt)`. Collapsed, it opens the panel; open, it closes it. Both windows can take the keyboard focus (a bubble that couldn't sometimes lost its mouse presses on Windows), so pressing the bubble while the panel is open closes the panel as the button goes down; the press time tells the controller that this click's job is done, and it doesn't reopen.

Dragging the bubble

1. `BubbleView` sees the mouse pressed on the bubble and moved more than 5 pixels → `window.assist.bubbleDragStart()` → `BubbleController.startDrag()`.
2. The controller notes where the bubble was grabbed and, 60 times a second, moves the bubble window to follow the mouse pointer (it reads the real pointer from the main process, which keeps it smooth across monitors with different scaling).
3. On release → `bubbleDragEnd()` → `endDrag()`: the display under the bubble is found, then the nearest of its four corners (`nearestCorner()`), and the bubble glides there.
4. The new anchor (display + corner) is saved to `preferences.json` and broadcast to the panel, which re-lays itself out to open toward the middle of the screen: in a left corner the chat opens to the right, in a top corner the icons drop below the bubble and the chat grows downward.

---

3. Repository root

- `package.json`: The npm *workspaces* root. `apps/*` are the packages (only `apps/desktop` for now; a gateway service can be added beside it later). Its scripts (`dev`, `test`, `lint`, `typecheck`, `dist`) forward to the desktop app; `format` runs Prettier and `docs` refreshes the code walkthrough.
- `package-lock.json`: Exact versions of every installed package, so every machine installs the same thing. Generated by npm; don't edit by hand.
- `.gitignore`: Keeps `node_modules/`, build output (`out/`, `dist/`) and logs out of git.
- `.gitattributes`: Keeps line endings as LF in git and on disk, matching Prettier.
- `.prettierrc.json`: Code formatting rules (no semicolons, single quotes, 100-character lines). Run `npm run format` to apply. Code inside the docs' Markdown is left alone, because it's copied from the source.
- `.prettierignore`: Files Prettier must not touch (build output, the lock file).
- `README.md`: What the app does and how to run, test and build it.
- `docs/PLAN.md`: The plan: tech stack, architecture, milestones, decisions and open questions.
- `docs/code-guide/`: The Code Guide: this overview (`CODE_GUIDE.md`) and the code walkthrough pages (see section 6).
- `docs/CODE_GUIDE.pdf`: The whole Code Guide as one PDF, built from these pages by `npm run docs:pdf`.
- `scripts/code-guide.mjs`: Copies the real code into the walkthrough pages (`npm run docs`), and checks they're up to date (`npm run docs:check`, also run by `npm test`).
- `scripts/code-guide-pdf.mjs`: Builds `docs/CODE_GUIDE.pdf` from the pages (`npm run docs:pdf`).
- `docs/CLAUDE_DESKTOP_SETUP.md`: For IT: what each PC needs so questions open in Claude Desktop (Claude Desktop installed and signed in to the company account), and how to check it works.
- `docs/APPS_SETUP.md`: For IT: turning on JumpCloud's MCP Server for Users for the Apps list, and what each person does to connect.

- `docs/ATLASSIAN_SETUP.md`: For IT: Jira and the service desk: what each person does to connect to Atlassian, what an Atlassian admin may need to allow, the `tenant.json` settings, and troubleshooting.
- `docs/JUMPCLOUD_SETUP.md`: For IT, for the built-in chat only: how to create the JumpCloud app and the Claude Console federation, and which IDs to put in `tenant.json`.

---

4. Tenants (branding)

A *tenant* is one business's branding. The build includes exactly one, chosen by the `TENANT` environment variable (default `morse-micro`). This is what makes the app easy to re-brand later.

- `tenants/README.md`: How tenants work, and how to change the logo.
- `tenants/morse-micro/tenant.json`: `id` (must match the folder name), `companyName`, `appName` (also the name of the screenshots folder), `accentColor` (the highlight colour), `actions` (which icons appear above the bubble, from the bubble upward: for Morse Micro Ask Claude, Jira, the IT service desk, screenshot, light or dark mode, settings, bounce and close), `chatApp` (where questions go: `claude-desktop`, as for Morse Micro, or `built-in`, the default), `startPage` (what the panel opens on: `ask`, the text box, by default, or `apps`, the Apps list, as for Morse Micro, whose `actions` then include `ask` to reach the text box), `portal` for the Apps list (the portal's name, its address, JumpCloud's apps server and the sign-in port), `serviceDesk` for the IT service desk icon (its `https` address, and optionally `requests`: the IT portal's number, its Jira project's key and the request types, to raise requests from the panel), `atlassian` for Jira (the site, the sign-in port, the Confluence spaces with IT's how-to articles, and optionally the MCP server, which defaults to Atlassian's), and for the built-in chat only: optionally `systemPrompt` (extra instructions for Claude), `signIn` (JumpCloud's address, the app's JumpCloud client ID and the sign-in port) and `claudeAccess` (the Claude Console organization, federation rule, service account and optional workspace IDs). The built-in chat needs `signIn` and `claudeAccess`; Claude Desktop needs neither; the Apps list needs `portal`; the service desk icon (`servicedesk` in `actions`) needs `serviceDesk`; the Jira icon (`jira`) needs `atlassian`. None of these are secret. See `docs/CLAUDE_DESKTOP_SETUP.md`, `docs/APPS_SETUP.md`, `docs/ATLASSIAN_SETUP.md` and `docs/JUMPCLOUD_SETUP.md`.
- `tenants/morse-micro/logo.png`: The logo: the Morse Micro "Mμ" mark cut to a circle (512×512, transparent corners). It's the bubble, the tray icon and the `.exe` icon. A tenant can use `logo.svg` for the bubble instead (if both exist, the PNG wins), but then the tray falls back to a plain circle in the accent colour and the `.exe` gets Electron's default icon.

---

5. Desktop app configuration

All in `apps/desktop/`.

- `package.json`: The desktop app's own package. `productName` ("Desktop Assist") becomes the app's name and its `%APPDATA%` folder. `main` points Electron at the built main process. Scripts: `dev`, `build`, `typecheck`, `lint`, `test`, `dist` (build the installer). Every library is a `devDependency` because the build bundles them; the installer ships no `node_modules`. `electron` is pinned to an exact version because the installer builder requires it.
- `electron.vite.config.ts`: Build config for electron-vite, which compiles the three parts (main, preload, renderer). Defines the import shortcuts `@shared` → `src/shared` and `@tenant` → `tenants/<TENANT>`, turns on React and Tailwind for the renderer, and minifies the renderer.

- `electron-builder.cjs`: Installer config: app ID, product name, include only the built `out/` folder, use the tenant's `logo.png` as the `.exe` icon, and build a per-user one-click NSIS installer (`Desktop Assist-Setup-<version>.exe`) into `dist/`. It's JavaScript rather than YAML so the icon can follow `TENANT`. `extraMetadata.name` makes the install folder `desktop-assist` (otherwise it would be named after the npm workspace).
- `tsconfig.json`: Points TypeScript at the two configs below.
- `tsconfig.node.json`: TypeScript settings for code that runs in Node: main, preload, shared, tests and config files. Strict mode on.
- `tsconfig.web.json`: TypeScript settings for the renderer (browser code with React/JSX).
- `vitest.config.ts`: Test runner config: the same `@shared`/`@tenant` shortcuts, run `tests/**/*.test.ts` in Node.
- `eslint.config.mjs`: Lint rules: recommended JavaScript and TypeScript rules everywhere, React Hooks rules for the renderer, and Prettier compatibility.

---

6. Code walkthrough

Each page goes through its files in order. For every function, method, class and type it shows the real code, with a link to the exact lines on GitHub, then explains what it does and why, with notes on the lines that need them. Read them in order the first time; after that, jump to the area you're working on.

1. Shared code and the preload bridge: `src/shared` (sizes and layout constants, the action list, data types, the IPC contract, chip labels and "how long ago", the Claude Desktop download link, the Apps list's order, Jira issue keys) and `src/preload` (the `window.assist` bridge between the pages and the main process).
2. Main process: startup, IPC and app plumbing: `index.ts` (startup and shutdown, and choosing the built-in chat or Claude Desktop), `ipc.ts` (handling requests from the pages), `actions.ts`, `uninstall.ts`, `tenant.ts`, `settings.ts`, the tray icon.
3. JumpCloud sign-in: `auth/` (`AuthManager`, OpenID Connect with JumpCloud, the loopback listener) and `storage/SecretStore.ts`.
4. Talking to Claude: `claude/` (the model settings, `AnthropicBackend`, error handling, `ChatSession`).
5. The bubble: `bubble/` (`BubbleController`, the layout maths, bounce physics, creating the windows).
6. The draft, screenshots and safe files: `notes/NotesStore.ts`, `screenshots/` (including stacking several screenshots into one image for the clipboard), `storage/jsonFile.ts`.
7. The pages: entry points, styles, hooks and views: `src/renderer`: `index.html`, `main.tsx`, `styles.css`, `lib/`, `hooks/`, `BubbleView`, `PanelView`.
8. The UI components: `src/renderer/src/components`: the chat box (with its Open buttons for issue keys and Report to IT), messages, Markdown, sign-in panel, settings menu, action icons, screenshot chips.
9. Claude Desktop: handing questions over: `claudeDesktop.ts` (feature 3.1: opening a question in the Claude Desktop app with a link, the screenshots on the clipboard, the notifications, checking it's installed), `sendInClaude.ts` (sending the question there: a hidden PowerShell helper that uses Windows UI Automation to find Claude's text box, then presses Ctrl+V and Enter) and what's tested.
10. Your apps: the JumpCloud apps list: `apps/` (`portalData.ts` reading JumpCloud's answers, `PortalApps.ts` with the list and opening apps, `logos.ts` fetching the logos), `mcp/` (shared with Jira: `McpAccount.ts` with the OAuth connection and tool calls, `connector.ts` on the MCP SDK, `toolResult.ts` reading a tool's answer) and `AppsList.tsx`, the list in the panel.

11. Jira and the service desk: `atlassian/` (`jiraData.ts` reading Atlassian's answers and writing the queries, `Atlassian.ts` with the connection, the notifications and requests checked in the background, search, and working on an issue) and `serviceDesk.ts` (raising an IT request on the help centre portal).
12. The Jira and service desk pages: `PageParts.tsx` (pieces the pages share), `JiraPage.tsx` (Notifications and Search), `IssueView.tsx` (one issue: Ask Claude, Move, Comment, Log time) and `ServiceDeskPage.tsx` (New request and My requests).

Keeping it current. The code on these pages is not typed by hand: each block is a marker like `` that `npm run docs` fills from the source. After changing code, run `npm run docs`; `npm test` fails if a page shows code that no longer matches. When you add a new function, add a marker and a short explanation for it on the right page. The explanations are written by hand, so check they still hold when behaviour changes. Then `npm run docs:pdf` rebuilds `docs/CODE_GUIDE.pdf`, the whole guide as one PDF.

---

7. Tests (`tests`)

Run with `npm test`. Each file tests code that doesn't need a real window or a real API.

- `layout.test.ts`: Home positions in all four corners (including displays not at the origin), nearest-corner snapping, window sizes, and that the panel window opens into the screen and fits on it in every corner.
- `bounce.test.ts`: Launch direction (away from each corner) and speed, bouncing off every edge, crossing onto the next monitor where two meet (over a taskbar between stacked monitors) and bouncing off an edge with no monitor beyond it, roaming both monitors without leaving them, getting back on screen when a monitor goes, and the glide.
- `bubbleController.test.ts`: The bubble state machine with fake windows, a fake clock and two fake displays: open/close, blur (including staying open when another app takes the focus back just as it opens), clicking the bubble (staying closed when the press on the bubble is what closed the panel, even for a slow click, and reopening straight away after clicking somewhere else, however quick), logging a click ignored while capturing, the saved corner, dragging (follows the mouse, snaps to the nearest corner, onto another display, panel re-placed), bouncing across both displays and gliding back to its own display's corner, screenshots, and display changes (including unplugging the bubble's display).
- `notesStore.test.ts`: Saving the draft, reloading, overtaken writes, damaged files, attachments.
- `screenshotFiles.test.ts`: Screenshot names, never overwriting, finding the newest, the inside-the-folder check.
- `screenshotStack.test.ts`: Stacking screenshots into one image for the clipboard: the width they're scaled to (the narrowest, capped), the order and the grey band between them (checked pixel by pixel), a single image left as it is, and refusing different widths, padded rows or an empty list.
- `format.test.ts`: Chip labels, and "how long ago" (`timeAgo`).
- `tenants.test.ts`: Every tenant folder has a valid `tenant.json` and a logo; `chatApp` defaults to the built-in chat, which needs `signIn` and `claudeAccess`, while Claude Desktop needs neither (and an unknown `chatApp` is refused); the Apps icon needs `portal`, whose address must be `https`; starting on the Apps list needs `portal` and the `ask` action; the service desk icon needs an `https` `serviceDesk` address; request types (at least one, with numeric IDs) reach the panel; the Jira icon needs `atlassian`, which gets Atlassian's MCP server by default; a dev override can change one Atlassian setting without resetting the others; the list of missing sign-in settings.
- `trayIcon.test.ts`: The tray circle.

- `chatSession.test.ts`: The conversation with a scripted fake backend: streaming, images before text, replaying replies unchanged (thinking included), busy/empty/signed-out, Stop with and without text, Retry, an expired sign-in, refusals and length limits, the tool loop, and New conversation ignoring a late reply.
- `authManager.test.ts`: The sign-in with a fake JumpCloud (whose refresh tokens work once, like the real one): not set up, nothing saved, renewing at startup and saving the replacement token, offline and Retry, a refused sign-in, browser sign-in, Cancel, no refresh token, a port in use, handing out a different ID token each time, never refreshing twice at once, skipping an almost-expired ID token, Log out (revoking), and logging out during a renewal.
- `claudeDesktop.test.ts`: Handing questions to Claude Desktop, with fakes that record the order of events: the link (the documented `q`, spaces as `%20`, any text round-trips, no `q` without text), the two notifications' wording, copying screenshots before opening Claude and notifying only when there were some, a screenshot-only question, an empty question, the 12,000-character limit, Claude Desktop not installed, a missing screenshot, and a link or clipboard that fails (logged, no notification). With "Send in Claude" on: sending (pasting first when there's a screenshot) with nothing more said, `ask()` returning while sending carries on, the right notification when Claude never showed the question or the screenshot was pasted but not sent, a helper that couldn't run (logged, then the notification), never sending a screenshot on its own, and sending nothing (with a "check your question" notification) when Claude's box already had other text.
- `portalApps.test.ts`: The Apps list: reading JumpCloud's answers (sorted by name, hidden apps skipped, a bare list or structured content, no list versus an empty one, `https` links only, the launch tool's ID argument, the sign-in link in a launch answer), telling images by their first bytes, fetching logos (`https` images only, judged by their bytes rather than their label, so uploaded logos labelled `application/octet-stream` work and mislabelled HTML doesn't, and not too big), starred apps first (`orderApps`), keeping fetched logos across a refresh, and `PortalApps` against a fake portal: asking to sign in without opening the browser, signing in through the browser and saving the connection, remembering the list until Refresh, opening an app through its launch link, asking for the launch tool only once, using the list's own link when JumpCloud is slow, refusing a sign-in that comes back with the wrong `state`, and Cancel; plus `SavedAuth` as a public client that keeps only what it was given.
- `atlassian.test.ts`: Jira, with made-up answers and a fake Atlassian: issue keys in text, search words that can't break a query, mentions and text in Jira's document format, who assigned an issue (not the user themselves), the comments that mention the user, a request's latest reply from a person (not the user or an automatic one), Confluence results only on the company's site, the user and the site's cloud ID, and the prompt to Claude kept under the limit; then `Atlassian`: asking to connect without opening the browser, connecting and showing who assigned or mentioned the user, remembering what's been seen and not re-reading unchanged issues, telling the user once when IT replies until they look, opening an issue with its comments, moves and portal page, moving, commenting and logging time, searching Jira and Confluence (and finding an issue by its key), asking Claude with the issue's details, and going back to Connect when the connection ends (and forgetting it on Disconnect).
- `serviceDesk.test.ts`: Raising IT requests: the portal's filled-in form address for a request type, the PC details under the description with the link kept short enough, Windows 11 called by its name, opening the form with the screenshot copied to paste, leaving the PC details out when asked, and refusing an empty request, an unknown request type and a missing screenshot.
- `uninstall.test.ts`: Finding the uninstaller next to the app, turning off Start with Windows before starting it and then quitting, doing nothing in a dev run or without an uninstaller, and staying open when it couldn't start.
- `sendInClaude.test.ts`: The PowerShell helper's script, without running it: the question key (whitespace evened out, 60 characters), evening out Unicode spaces and fingerprinting the whole question (any extra text changes the hash), a question full of PowerShell code carried only as base64, no key pressed before Claude's

box has been found to hold exactly the question, Ctrl+V only with a screenshot, the process name (another for tests, but nothing that could break out of its quotes), and reading the outcome from the last line printed.

- `loopback.test.ts`: The browser-return listener: catches `/callback` and stops, 404 for anything else, shows JumpCloud's error safely escaped, Cancel, timeout, and a port in use.
- `secretStore.test.ts`: Encrypted save and load, unreadable files, refusing to save without encryption, clear.
- `claudeHelpers.test.ts`: Error classification (including sign-in errors wrapped by the SDK and failed swaps), what can be retried, the system prompt, and image scaling.
- `codeGuide.test.ts`: The code shown in the walkthrough pages matches the source (runs `npm run docs:check`).
- `diagnostics.test.ts`: The ID token summary (the claims a federation rule checks, never the token), describing errors for the log, writing the log file and starting a new one when it gets large.

Shared code and the preload bridge

The app runs as two kinds of process: the **main process** (Node.js, which owns the windows, files, sign-in and Claude) and a **renderer** for each window (Chromium, which draws the React UI). They talk over **IPC** (inter-process communication): one side sends a message on a named *channel* and the other side listens on it. `src/shared` holds the TypeScript both sides must agree on (window sizes, action names, data shapes and the list of channels), imported as `@shared/...` and bundled into each part. `src/preload/index.ts` is the small script that turns that list of channels into `window.assist`, the only way a page can reach the main process.

`src/shared/geometry.ts`: sizes and positions

Every size and offset the bubble and the panel use, plus a few helpers built on them. The main process uses these numbers to size and place the two windows (`src/main/index.ts` and `src/main/bubble/layout.ts`). The renderer uses the same numbers to place elements inside those windows (`ActionStack.tsx`, `ChatBox.tsx`, `SettingsMenu.tsx`). Because both read one file, a window can never be the wrong size for what is drawn in it.

All values are **DIPs** (device-independent pixels): pixels before Windows display scaling. On a display scaled to 150%, one DIP is 1.5 real pixels. Electron's window and screen functions and the page's CSS pixels both work in DIPs, so one number means the same thing on both sides.

The key idea is that the panel's layout is measured from the corner the bubble sits in, not from the top-left of the window. That way one set of numbers works in all four screen corners.

`Point`, `Size` and `Rect`

src/shared/geometry.ts, lines 1–14

```ts
// Geometry shared by the main process (which sizes and moves windows) and the renderer (which
// positions elements inside them). All values are DIPs (device-independent pixels).

export interface Point {
  x: number
  y: number
}

export interface Size {
  width: number
  height: number
}

export interface Rect extends Point, Size {}
```

Plain shapes for positions and sizes, used all over the main process and the renderer.

- `export interface Rect extends Point, Size {}`: an interface can extend several others. `Rect` adds nothing of its own; it is `x`, `y`, `width` and `height` together, the same shape Electron uses for window bounds, so a `Rect` can be passed straight to `win.setBounds()` (see `src/main/bubble/windows.ts`).

`Corner`, `isLeftCorner()` and `isTopCorner()`

src/shared/geometry.ts, lines 16–20

```ts
/** The screen corner the bubble rests in. The panel always opens toward the screen's middle. */
export type Corner = 'top-left' | 'top-right' | 'bottom-left' | 'bottom-right'

export const isLeftCorner = (corner: Corner): boolean => corner.endsWith('left')

export const isTopCorner = (corner: Corner): boolean => corner.startsWith('top')
```

`Corner` is a *union of string literals*: a value of this type can only be one of those four strings, and TypeScript rejects anything else at compile time. The bubble rests in one corner of a display's work area (the screen minus the taskbar), and everything else opens toward the middle of the screen from there. The two helpers answer "which side is the bubble on?", which is all the layout code needs in order to mirror itself.

- `corner.endsWith('left')`: every name is "vertical-horizontal", so the horizontal side is the end of the string and the vertical side is the start.
- Callers: `homePosition()` and `panelWindowBounds()` in `src/main/bubble/layout.ts`, `launch()` in `src/main/bubble/bounce.ts` (a bounce sets off away from the corner), and `anchored()` in `src/renderer/src/lib/anchor.ts`, which turns an offset into CSS `left`/`right` and `top`/`bottom`.

`UI`

src/shared/geometry.ts, lines 22–40

```ts
export const UI = {
  /** Gap between the bubble and the edges of the work area. */
  edgeMargin: 16,
  bubbleSize: 56,
  /** Transparent margin around the bubble inside its window (room for the shadow and ring). */
  bubblePad: 8,
  /** Space between the bubble and the chat card / action stack. */
  gap: 12,
  /** The chat card. It starts as just the text box and grows away from the bubble to this height. */
  panelWidth: 400,
  panelMaxHeight: 520,
  /** Room beyond the chat card for a toast message, even when the card is at full height. */
  toastRoom: 40,
  actionSize: 40,
  actionGap: 8,
  /** Room for drop shadows along the panel window's far edges. */
  shadowPad: 16,
  settingsMenuWidth: 248,
} as const
```

Every fixed measurement of the UI in one object, in DIPs.

- `as const`: makes the object read-only and keeps each value's exact type (`56` rather than `number`). Nothing can change these numbers at runtime by accident.
- `edgeMargin: 16`: how far the resting bubble sits from the work area's edges (`homePosition()` in `src/main/bubble/layout.ts`).
- `bubblePad: 8`: the bubble's window is a little bigger than the bubble itself, so the drop shadow and the ring around the bubble aren't clipped by the window edge.
- `panelWidth: 400` and `panelMaxHeight: 520`: the chat card's width and its tallest height. `ChatBox.tsx` sets them as the card's CSS `width` and `maxHeight`; the card starts as just the text box and grows as the

conversation fills it.

- `toastRoom: 40`: toasts ("Screenshot saved") appear just outside the card, on the side away from the screen edge. The window keeps this much room for them even when the card is full height.
- `actionSize: 40` and `actionGap: 8`: the round action icons and the space between them (`ActionStack.tsx`).
- `shadowPad: 16`: transparent room along the panel window's far edges so shadows there aren't cut off.

BUBBLE_BOX

`src/shared/geometry.ts`, lines 42–43

```ts
/** The bubble window is the bubble plus its padding on every side. */
export const BUBBLE_BOX = UI.bubbleSize + UI.bubblePad * 2
```

The bubble window's width and height: 56 + 8 + 8 = 72. `start()` in `src/main/index.ts` creates the bubble window at this size, and `bubbleWindowBounds()` in `src/main/bubble/layout.ts` places it. The bubble window only ever moves; it never changes size.

PANEL_LAYOUT

`src/shared/geometry.ts`, lines 45–56

```ts
/**
 * Where things sit inside the panel window. The panel window shares one corner with the bubble
 * window (the corner of the screen the bubble is in), and these offsets are measured from that
 * corner: `X` across, `Y` away from the screen edge. So the same numbers work in every corner.
 */
export const PANEL_LAYOUT = {
  actionsX: UI.bubblePad + (UI.bubbleSize - UI.actionSize) / 2,
  actionsY: UI.bubblePad + UI.bubbleSize + UI.gap,
  chatX: UI.bubblePad + UI.bubbleSize + UI.gap,
  chatY: UI.bubblePad,
  settingsX: UI.bubblePad + (UI.bubbleSize + UI.actionSize) / 2 + UI.actionGap,
} as const
```

Where the three parts of the panel (the action icons, the chat card and the settings menu) sit inside the panel window. The panel window and the bubble window share one corner: in the bottom-right of the screen, their bottom-right corners are in the same place (`panelWindowBounds()` in `src/main/bubble/layout.ts`). Every offset here is measured from that shared corner: `X` across, `Y` away from the screen edge (upward in a bottom corner, downward in a top corner). In the renderer, `anchored()` (`src/renderer/src/lib/anchor.ts`) turns an `(X, Y)` pair into CSS `right`/`bottom`, `left`/`top` and so on, so the same numbers mirror into every corner.

Here is the bottom-right case with four icons (not to scale; the other corners are mirror images):

```
+-----------------------------------------------------+
|  +-----------------------------+                    |
|  |                             |                    |
|  |  chat card                  |        [ icon 4 ]  |
|  |  (grows upward,             |        [ icon 3 ]  |
|  |   at most 520 tall)         |        [ icon 2 ]  |
|  |                             |        [ icon 1 ]  |
|  |  [ text box ]               |                    |
|  +-----------------------------+        ( bubble )  |
+-----------------------------------------------------+
```

- `actionsX: UI.bubblePad + (UI.bubbleSize - UI.actionSize) / 2`: 8 + 8 = 16, which centres the 40-DIP icons on the 56-DIP bubble.
- `actionsY: UI.bubblePad + UI.bubbleSize + UI.gap`: 76. The first icon starts one gap beyond the bubble.
- `chatX: UI.bubblePad + UI.bubbleSize + UI.gap`: also 76. The chat card starts one gap to the side of the bubble, toward the middle of the screen.
- `chatY: UI.bubblePad`: 8, so the card's near edge is level with the bubble's near edge.
- `settingsX`: 8 + 48 + 8 = 64. The first two terms reach the far side of the icon column; one more `actionGap` puts the menu just beside the gear icon. Its other coordinate depends on where the gear is in the stack, so it comes from `actionOffset()` instead.

`actionStackHeight()` and `actionOffset()`

`src/shared/geometry.ts`, lines 58–65

```ts
export function actionStackHeight(actionCount: number): number {
  return actionCount > 0 ? actionCount * UI.actionSize + (actionCount - 1) * UI.actionGap : 0
}

/** Distance from the bubble's edge of the panel window to the near side of the action at `index`. */
export function actionOffset(index: number): number {
  return PANEL_LAYOUT.actionsY + index * (UI.actionSize + UI.actionGap)
}
```

Two small sums over the icon stack.

- `actionCount > 0 ?`: `n` icons have `n - 1` gaps between them. With no icons the formula would give -8, so zero is handled separately. Only `panelWindowSize()` below uses this.
- `PANEL_LAYOUT.actionsY + index * (UI.actionSize + UI.actionGap)`: icon `index` (0 is nearest the bubble) starts this far from the bubble's edge of the window. `PanelView.tsx` passes `actionOffset(settingsIndex)` to `SettingsMenu` as its `offset`, so the menu lines up with the gear wherever the tenant put the gear in its list.

`panelWindowSize()`

`src/shared/geometry.ts`, lines 67–74

```ts
export function panelWindowSize(actionCount: number): Size {
  const width = PANEL_LAYOUT.chatX + UI.panelWidth + UI.shadowPad
  const contentHeight = Math.max(
    PANEL_LAYOUT.chatY + UI.panelMaxHeight + UI.toastRoom,
    PANEL_LAYOUT.actionsY + actionStackHeight(actionCount),
  )
  return { width, height: contentHeight + UI.shadowPad }
}
```

How big the panel window must be to hold everything at its largest. The window is created at this size (`start()` in `src/main/index.ts`) and never resizes when the chat card grows, so there is no flicker. The unused parts are transparent and let clicks through to whatever is underneath (the *click-through* handled by `useClickThrough()` in the renderer).

- `PANEL_LAYOUT.chatX + UI.panelWidth + UI.shadowPad`: 76 + 400 + 16 = 492 wide.
- `Math.max(`: the height is whichever reaches further from the screen edge: the chat card at full height plus room for a toast (8 + 520 + 40 = 568), or the icon stack. With the usual four icons the stack ends at 76 + 184 = 260, so the card decides and the window is 584 tall. The stack only matters with 11 or more icons.

`src/shared/actions.ts`: the action registry

The icons that can appear in the stack beside the bubble (the *actions*), each with a tooltip label and a *kind*. A tenant's `tenant.json` has an `actions` list that picks which ones appear and in what order, nearest the bubble first. There are three kinds:

- `command` actions run in the main process: the IT service desk, screenshot, bounce and close (handlers in `createActionHandlers()`, `src/main/actions.ts`). The service desk is the one exception: when the tenant lists request types, `PanelView.runAction()` opens the service desk page instead (see Jira and the service desk).
- `popover` actions open some UI inside the panel and never leave the renderer: Settings (the settings menu); Ask Claude and Your apps, which switch the card between the text box and the Apps list; and Jira, which swaps in the Jira page. `PanelView.runAction()` handles them.
- `switch` actions flip a setting straight from the icon, without opening anything: light or dark mode. `PanelView.runAction()` handles it too, through the settings channels.

The ids are written once, as a real array, and the types are derived from it. So adding an action is one edit here, and then the compiler points at the rest: `ICONS` in `ActionStack.tsx` is a `Record<ActionId, ...>` (it must have an icon for every action), and `ActionHandlers` in `src/main/actions.ts` is a `Record<CommandActionId, ...>` (every command needs a handler).

`ACTION_IDS` and `ActionId`

src/shared/actions.ts, lines 1–20

```ts
// The action registry: every icon that can appear in the stack above the bubble. A tenant's
// `actions` list chooses which of these show and in what order (from the bubble upward).
//
// - `command` actions run in the main process (see src/main/actions.ts).
// - `popover` actions open UI inside the panel and never reach the main process.
// - `switch` actions flip a setting from the panel (light or dark mode).

export const ACTION_IDS = [
  'ask',
  'apps',
  'jira',
  'servicedesk',
  'screenshot',
  'theme',
  'settings',
  'bounce',
  'close',
] as const

export type ActionId = (typeof ACTION_IDS)[number]
```

The list of every action that exists, and the type of one of its entries.

- `as const`: without it the array's type would be `string[]`. With it, the type is a read-only list of these nine exact strings, which is what lets the next line work.
- `(typeof ACTION_IDS)[number]`: "the type of whatever you get by indexing the array with a number", which is the union of the nine, `'ask' | 'apps' | 'jira' | ... | 'close'`. The type follows the list automatically.
- The array also exists at runtime, which a type doesn't: `TenantSchema` in `src/main/tenant.ts` uses `z.enum(ACTION_IDS)` to reject a `tenant.json` that lists an unknown action. **zod** is a validation library: you

describe a shape (a *schema*) and zod checks, while the app runs, that untrusted data really has that shape. TypeScript types are erased when the code is compiled, so they can't check anything at runtime.

ACTIONS

src/shared/actions.ts, lines 22–33

```
export const ACTIONS = {
  ask: { label: 'Ask Claude', kind: 'popover' },
  apps: { label: 'Your apps', kind: 'popover' },
  jira: { label: 'Jira: notifications and search', kind: 'popover' },
  // A page in the panel when the tenant lists request types (see PanelView); otherwise a link.
  servicedesk: { label: 'IT service desk', kind: 'command' },
  screenshot: { label: 'Take screenshot', kind: 'command' },
  theme: { label: 'Light or dark mode', kind: 'switch' },
  settings: { label: 'Settings', kind: 'popover' },
  bounce: { label: 'Bounce', kind: 'command' },
  close: { label: 'Close Desktop Assist', kind: 'command' },
} as const satisfies Record<ActionId, { label: string; kind: 'command' | 'popover' | 'switch' }>
```

The label and kind of each action. `ActionStack.tsx` uses `label` as each icon's tooltip and screen-reader name, and gives `popover` actions (and the service desk, when it opens a page) an `aria-expanded` attribute so screen readers announce whether the menu or page is open.

- `jira: { label: 'Jira: notifications and search', kind: 'popover' }`: the Jira icon, a bell.
- The comment above `servicedesk`: it stays a `command`, because without request types it just opens the portal in the browser; with them, the panel opens its page instead.
- `satisfies Record<ActionId, { label: string; kind: 'command' | 'popover' | 'switch' }>`: checks that there is exactly one entry per action id, each with a label and a valid kind. A missing, extra or misspelt action is a compile error. Unlike a normal type annotation, `satisfies` only checks: it doesn't change the object's own type.
- `as const`: keeps each `kind` as its exact string (`'command'`), not just `string`. `CommandActionId` below depends on that; a plain `: Record<...>` annotation would have widened every `kind` to `'command' | 'popover'` and lost which is which.

CommandActionId

src/shared/actions.ts, lines 35–37

```
export type CommandActionId = {
  [K in ActionId]: (typeof ACTIONS)[K]['kind'] extends 'command' ? K : never
}[ActionId]
```

The type of just the command actions: `'servicedesk' | 'screenshot' | 'bounce' | 'close'`. It's worked out from `ACTIONS`, so it can't drift out of step with it.

- `[K in ActionId]:`: a *mapped type*. It builds an object type with one property per action id. Each property's type is the id itself if that action's kind is `'command'`, or `never` (the "impossible" type) if not, giving (in part) `{ ask: never; servicedesk: 'servicedesk'; settings: never; close: 'close' }`.
- `}[ActionId]`: looking up every key at once gives the union of the property types. `never` disappears from a union, leaving the four command ids.
- `AssistApi.invokeAction()` in `src/shared/ipc.ts` only accepts these, so a page can't even try to send `settings`, `ask`, `apps` or `jira` to the main process. In `PanelView.runAction()`, the early returns for those

narrow the id to exactly this type, which is why it can then be passed on.

`COMMAND_ACTION_IDS`

`src/shared/actions.ts`, lines 39–41

```ts
export const COMMAND_ACTION_IDS = ACTION_IDS.filter(
  (id): id is CommandActionId => ACTIONS[id].kind === 'command',
)
```

The same set as a real array, for checks at runtime. `registerIpc()` in `src/main/ipc.ts` uses `z.enum(COMMAND_ACTION_IDS)` to check the id that arrives with `assist:invoke-action` before looking up its handler. The main process treats the pages as untrusted, so a compile-time type isn't enough.

- `(id): id is CommandActionId =>`: a *type guard*. It tells TypeScript that when this function returns true, `id` is a `CommandActionId`, so the filtered array is typed `CommandActionId[]` rather than `ActionId[]`.

`src/shared/types.ts`: data shapes

The shapes of the data that travels between the main process and the pages: the bubble's mode, the draft, settings, branding, sign-in status, chat messages, the Apps list, Jira and the service desk, and the answers to requests. The file holds only types, so nothing in it exists at runtime; every file imports it with `import type`.

Two ideas run through it:

- **Plain data only.** Everything is strings, numbers, booleans, arrays and plain objects. Values sent over IPC are copied (Electron serialises them with the browser's *structured clone* algorithm, which drops functions and class methods), and some are also saved as JSON files. That is why timestamps are ISO strings such as `"2026-10-02T14:03:11.000Z"` rather than `Date` objects.
- **Tagged unions.** Several types are "one of these shapes", told apart by a tag field (`state` or `ok`). Checking the tag tells TypeScript which other fields exist: after `if (result.ok)`, it knows `result.notes` is there; otherwise it knows `result.reason` is.

`Mode`

`src/shared/types.ts`, lines 4–10

```ts
/**
 * What the bubble is doing. The main process owns this; renderers only display it.
 * - `dragging`: following the mouse while the user drags it.
 * - `returning`: gliding into its corner after a bounce or a drag.
 * - `capturing`: windows are hidden while a screenshot is taken.
 */
export type Mode = 'collapsed' | 'expanded' | 'dragging' | 'bouncing' | 'returning' | 'capturing'
```

What the bubble is doing right now. Only `BubbleController` (`src/main/bubble/BubbleController.ts`) changes it, through its private `setMode()`, and each change is broadcast to both windows on `assist:mode-changed`. `collapsed` is just the bubble; `expanded` means the panel is open; `bouncing` is the Bounce animation. The panel shows its contents only while the mode is `expanded`, and `BubbleView` styles the bubble differently while `dragging` and `expanded`.

`Attachment` and `Notes`

src/shared/types.ts, lines 12–27

```ts
export interface Attachment {
  id: string
  /** Absolute path of the screenshot file. */
  path: string
  fileName: string
  /** ISO timestamp. */
  addedAt: string
}

/** The unsent draft in the text box. */
export interface Notes {
  text: string
  attachments: Attachment[]
  /** ISO timestamp of the last change, or null if nothing has been typed yet. */
  updatedAt: string | null
}
```

A screenshot attached to the draft (or to a message already sent), and the draft itself. The code calls the draft "notes" throughout (`NotesStore`, `window.assist.notes`). `NotesStore` (`src/main/notes/NotesStore.ts`) owns it and saves it to `notes.json`, so an unsent message survives a restart.

- `id: string`: a random UUID made by `NotesStore.addAttachment()`. The page uses it to remove one attachment (`notes.removeAttachment(id)`).
- `path: string`: where the PNG is on disk. The main process never trusts a path that comes back from a page: `ScreenshotService` checks it is inside the screenshots folder before reading it.
- `fileName: string`: just the file's name, which `screenshotLabel()` (below) turns into the chip's label.
- `updatedAt: string | null`: set by `NotesStore` on every change; `null` for a draft nobody has typed in yet.

`Effort`, `Settings` and `Theme`

src/shared/types.ts, lines 29–48

```ts
/** How hard Claude thinks before answering (the API's `effort`), shown as Fast/Balanced/Thorough. */
export type Effort = 'low' | 'medium' | 'high'

export interface Settings {
  autoStart: boolean
  /** Starting with Windows only works for an installed build, not `npm run dev`. */
  autoStartAvailable: boolean
  screenshotsDir: string
  effort: Effort
  /** With Claude Desktop: send the question there too, not just fill it in. */
  autoSend: boolean
  /** Uninstalling from Settings only works for an installed build, not `npm run dev`. */
  canUninstall: boolean
  /** IDs of the apps the user starred in the Apps list. */
  favoriteApps: string[]
  /** Light or dark mode for Desktop Assist (dark until the user switches). */
  theme: Theme
}

export type Theme = 'dark' | 'light'
```

`Effort` is the response style. It is sent to the Claude API as `output_config.effort` (`src/main/claude/AnthropicBackend.ts`) and shown in the settings menu as Fast, Balanced and Thorough. `Settings` is what the settings menu shows; `SettingsService.get()` in `src/main/settings.ts` builds a fresh one each time.

- `autoStart: boolean`: whether the app starts with Windows. It is read from Windows each time, because the user can also change it in Windows Settings.
- `autoStartAvailable: boolean`: `app.isPackaged`, true only in the installed app. Under `npm run dev` the switch is greyed out with "Only in the installed app".
- `screenshotsDir: string`: the folder screenshots are saved in, opened by "Open screenshots folder".
- `effort: Effort`: the response style, used by the built-in chat only.
- `autoSend: boolean`: used with Claude Desktop only: whether Desktop Assist also sends the question there (presses Ctrl+V and Enter in Claude), or only fills it in. On unless the user turns "Send in Claude automatically" off in the settings menu.
- `canUninstall: boolean`: like `autoStartAvailable`, true only in the installed app, which is the only copy with an uninstaller. Under `npm run dev` the menu's Uninstall item is greyed out.
- `favoriteApps: string[]`: the IDs of the apps the user starred in the Apps list, which it shows first. They're kept with the other preferences, so they survive a restart.
- `theme: Theme`: light or dark mode for every Desktop Assist window, whatever Windows itself uses. Dark until the user switches, with the theme icon beside the bubble.

ChatApp

`src/shared/types.ts`, lines 50–54

```
/**
 * Where conversations happen: in the panel through the Claude API (`built-in`), or handed over to
 * the Claude Desktop app (`claude-desktop`), which uses each person's own Claude account.
 */
export type ChatApp = 'built-in' | 'claude-desktop'
```

Where questions are answered, set per tenant (`chatApp` in `tenant.json`, checked against `CHAT_APPS` in `src/main/tenant.ts`). The two are quite different apps:

- `built-in`: the panel is a chat of its own. The user signs in with JumpCloud, and the main process calls the Claude API, billed to the company's Claude Console account.
- `claude-desktop`: the panel is only a text box. Asking opens the question in the Claude Desktop app (and, unless the user turns it off, sends it there), where the person is signed in with their own Claude account, so it counts against their own usage limit. Desktop Assist has no sign-in of its own then (see Claude Desktop).

`Branding`, `RequestType` and `StartPage`

src/shared/types.ts, lines 56–82

```ts
export interface Branding {
  companyName: string
  appName: string
  accentColor: string
  actions: ActionId[]
  chatApp: ChatApp
  /** The identity provider's name for the Apps list ("JumpCloud"), or null without one. */
  portalName: string | null
  /** What the card shows when the panel opens: the text box, or the Apps list. */
  startPage: StartPage
  /** The company's Atlassian site (Jira links, the Jira page), or null without one. */
  atlassianSite: string | null
  /**
   * Raising IT requests from the panel: the request types offered, the first one for "Report a
   * problem". Null when the service desk icon just opens the portal.
   */
  requestTypes: RequestType[] | null
}

/** A kind of request on the IT help centre portal ("Other IT Issue", "Request new software"). */
export interface RequestType {
  id: string
  name: string
}

/** The two things the card beside the bubble can show. */
export type StartPage = 'ask' | 'apps'
```

The parts of the tenant's `tenant.json` that the pages need. `brandingOf()` in `src/main/tenant.ts` copies just these fields, deliberately leaving out the sign-in settings, the Claude access settings, the portal's addresses, Atlassian's MCP server and the system prompt. `accentColor` becomes the page's accent colour (`useAccentColor()` sets the CSS variable `--color-accent`), and `actions` decides which icons `ActionStack` shows, in order. `chatApp` tells the panel which kind of chat it is: it changes the send button, the settings menu, and whether there's a sign-in at all. `portalName` is the name of the company's app portal ("JumpCloud"), for the Apps list's buttons ("Sign in with JumpCloud"); it's `null` when the tenant has no portal, and then the Apps list never shows. `atlassianSite` is the company's Atlassian site (`https://morsemicro.atlassian.net`), for Jira links and the Jira page, or `null` without one. `requestTypes` lists the request types the service desk page offers, as its comment says; `null` means the service desk icon just opens the portal. `RequestType` is one of them: its ID, as in the portal's addresses, and its name.

`startPage` is what the card shows when the panel opens: the text box (`ask`) or the Apps list (`apps`). `StartPage` is the type of those two. The panel's `view` (what's showing right now) can also be the Jira page or the service desk page (`View` in `PanelView.tsx`).

`SignedInUser` and `AuthStatus`

src/shared/types.ts, lines 84–103

```ts
export interface SignedInUser {
  name?: string
  email?: string
}

/**
 * Whether the user is signed in with JumpCloud.
 * - `unconfigured`: the JumpCloud or Claude access settings in tenant.json aren't filled in yet.
 * - `checking`: renewing a saved sign-in at startup.
 * - `signing-in`: waiting for the user to finish in their browser.
 * - `offline`: a saved sign-in couldn't be renewed because JumpCloud couldn't be reached. It's
 *   kept, and Retry tries again.
 */
export type AuthStatus =
  | { state: 'unconfigured'; missing: string[] }
  | { state: 'checking' }
  | { state: 'signed-out'; message?: string }
  | { state: 'signing-in' }
  | { state: 'signed-in'; user: SignedInUser }
  | { state: 'offline'; user: SignedInUser; message: string }
```

Who is signed in, and where sign-in stands, for the built-in chat. The user signs in with **JumpCloud**, the company's single sign-on service. Only `AuthManager` (`src/main/auth/AuthManager.ts`) sets the status, and every change is broadcast on `assist:auth-status`. The pages only ever learn the state and the user's name and email; no token ever reaches them. With Claude Desktop there is no `AuthManager` and no status at all (`AppState.auth` is `null`, below).

- `name?: string` and `email?: string`: taken from the `name` and `email` claims in JumpCloud's ID token (its signed statement of who signed in). Both are optional because JumpCloud may leave them out.
- `{ state: 'unconfigured'; missing: string[] }`: `missing` lists the `tenant.json` settings still to fill in, by name (from `missingSettings()` in `src/main/tenant.ts`).
- `{ state: 'signed-out'; message?: string }`: the optional message says why, for example that the sign-in expired.
- `{ state: 'offline'; user: SignedInUser; message: string }`: the saved sign-in is kept and the chat stays usable. `PanelView` treats both `signed-in` and `offline` as signed in, and shows the sign-in panel for `unconfigured`, `signed-out` and `signing-in`.

`ChatMessage` and `SendResult`

src/shared/types.ts, lines 105–121

```ts
/** One message as the chat shows it. */
export interface ChatMessage {
  id: string
  role: 'user' | 'assistant'
  text: string
  /** Screenshots sent with a user message. */
  attachments: Attachment[]
  status: 'streaming' | 'done' | 'stopped' | 'error'
  /** Shown under the message, e.g. why it stopped. */
  notice?: string
  /** A failed reply that can be requested again. */
  retryable?: boolean
}

export type SendResult =
  | { ok: true; notes: Notes }
  | { ok: false; reason: 'busy' | 'empty' | 'signed-out' | 'missing-screenshot' }
```

`ChatMessage` is one message as the chat shows it. `ChatSession` (`src/main/claude/ChatSession.ts`) keeps the list and broadcasts every new or changed message on `assist:chat-message`; the page replaces the message with the same `id` (`upsert()` in `useAssistState.ts`). This is only the display copy: the history sent to Claude, including Claude's hidden thinking, is kept separately inside `ChatSession`.

- `status: 'streaming' | 'done' | 'stopped' | 'error'`: `streaming` while text is still arriving; `stopped` means the user pressed Stop.
- `retryable?: boolean`: set on a failed reply that can be asked for again (a temporary error before any text arrived). `ChatSession.retry()` only acts if the last message has it.

`SendResult` is the answer to `window.assist.chat.send()`. On success, `notes` is the now-empty draft, which the panel shows. The reply itself arrives later, through `onChatMessage`.

- `reason: 'busy' | 'empty' | 'signed-out' | 'missing-screenshot'`: `busy` means a reply is still streaming, and `empty` means there was no text and no screenshot. The first three come from `ChatSession.send()`; `missing-screenshot` (an attached file has gone) comes from the `chat-send` handler in `src/main/ipc.ts`.

`AskResult`

src/shared/types.ts, lines 123–129

```ts
/**
 * Handing the draft to Claude Desktop. `screenshots` is how many were copied to the clipboard
 * (as one image) for the user to paste.
 */
export type AskResult =
  | { ok: true; notes: Notes; screenshots: number }
  | { ok: false; reason: 'empty' | 'too-long' | 'not-installed' | 'missing-screenshot' | 'failed' }
```

The answer to `window.assist.claudeDesktop.ask()`, which hands the draft to Claude Desktop instead of the built-in chat. On success, `notes` is the now-empty draft, as with `SendResult`, and Claude Desktop has opened with the question filled in. The failures come from `ClaudeDesktop.ask()` in `src/main/claudeDesktop.ts` (see Claude Desktop).

- `screenshots: number`: how many attached screenshots were put on the clipboard, combined into one image, to be pasted into Claude (by Desktop Assist with "Send in Claude automatically" on, otherwise by the user with Ctrl+V).
- The result doesn't say whether the question was sent in Claude: that happens afterwards, in the background, and if it doesn't work a Windows notification tells the user what's left.
- `'empty'`, `'missing-screenshot'`: the same as for `SendResult`. There's no `busy` or `signed-out`: nothing streams here, and there's no sign-in.
- `'too-long'`: more text than Claude Desktop accepts in a link (12,000 characters here).
- `'not-installed'`: nothing on the PC opens `claude://` links, so Claude Desktop isn't installed.
- `'failed'`: copying the screenshots or opening the link went wrong; the details are in the log.

AppState

`src/shared/types.ts`, lines 131–142

```ts
export interface AppState {
  mode: Mode
  /** The screen corner the bubble rests in; the panel lays itself out to open away from it. */
  corner: Corner
  notes: Notes
  settings: Settings
  branding: Branding
  version: string
  /** Null when chats happen in Claude Desktop, which has its own sign-in. */
  auth: AuthStatus | null
  chat: ChatMessage[]
}
```

Everything a page needs when it first loads, returned by `window.assist.getState()`. It is put together by the `getState` function that `start()` in `src/main/index.ts` passes to `registerIpc()`. After that first load, `useAssistState()` in the renderer keeps `mode`, `corner`, `auth` and `chat` up to date from the `on...` events. `notes` and `settings` only change through the page's own requests, so the panel takes their new values from the replies.

- `version: string`: `app.getVersion()`, shown in the settings menu.
- `auth: AuthStatus | null`: the sign-in status of the built-in chat, or `null` with Claude Desktop, which has its own sign-in. The pages check for `null` before reading `auth.state`.
- `chat: ChatMessage[]`: the built-in chat's messages. With Claude Desktop it's always empty.

The Apps list isn't in `AppState`: the panel asks for it whenever the list shows (`window.assist.apps.get()`), and then keeps up through `onAppsState`. Nor is Atlassian: the panel asks with `atlassian.get()` and `atlassian.feed()`, and keeps up through `onAtlassianState` and `onJiraFeed`.

`PortalApp` and `AppsState`

`src/shared/types.ts`, lines 144–163

```ts
/** One app from the user's JumpCloud User Portal, as the Apps list shows it. */
export interface PortalApp {
  id: string
  name: string
  /** The logo as a data: URL (the pages can't load images from the web), or null. */
  logo: string | null
}

/**
 * The Apps list.
 * - `sign-in`: the user hasn't connected Desktop Assist to their portal yet (or the connection
 *   ended); a button starts it, in the browser.
 * - `signing-in`: waiting for the user to finish in their browser.
 */
export type AppsState =
  | { status: 'loading' }
  | { status: 'sign-in'; message?: string }
  | { status: 'signing-in' }
  | { status: 'ready'; apps: PortalApp[] }
  | { status: 'error'; message: string }
```

The Apps list (see Your apps). `PortalApp` is one app as the list shows it, and `AppsState` is what the list as a whole is doing. `PortalApps` in `src/main/apps/PortalApps.ts` sets the state and broadcasts every change on `assist:apps-state`.

- `id: string`: JumpCloud's ID for the app. The page sends it back to open the app (`apps.open(id)`); the sign-in link itself never reaches the page.
- `logo: string | null`: as the comment says, a `data:` URL. The page's Content-Security-Policy only allows images from the app and `data:` URLs, so the main process fetches each logo and hands it over already encoded.
- `{ status: 'sign-in'; message?: string }`: the user hasn't connected Desktop Assist to their portal yet, or the connection ended. `message` says why when there's a reason to (a sign-in that failed, say).
- `{ status: 'ready'; apps: PortalApp[] }`: the list, sorted by name.
- `{ status: 'error'; message: string }`: a plain-English reason, such as the portal's apps server not being turned on for the company. The list offers Retry.

`UninstallResult`

`src/shared/types.ts`, lines 165–167

```ts
/** Settings → Uninstall. On success the app is already quitting. */
export type UninstallResult =
  { ok: true } | { ok: false; reason: 'not-installed' | 'missing' | 'failed' }
```

The answer to `window.assist.uninstall()` (Settings → Uninstall; see `src/main/uninstall.ts` in Main process: startup, IPC and app plumbing). On success the app is already quitting, so the page has nothing more to do.

- `'not-installed'`: a dev run (`npm run dev`), which has no uninstaller.
- `'missing'`: the uninstaller isn't next to the app's `.exe`.
- `'failed'`: it couldn't be started; the details are in the log.

`AtlassianState`

`src/shared/types.ts`, lines 169–179

```ts
/**
 * The connection to Atlassian (Jira and Confluence), made once in the browser.
 * - `sign-in`: not connected yet (or the connection ended); a button starts it.
 * - `signing-in`: waiting for the user to finish in their browser.
 */
export type AtlassianState =
  | { status: 'loading' }
  | { status: 'sign-in'; message?: string }
  | { status: 'signing-in' }
  | { status: 'connected'; user: { name: string; email?: string } }
  | { status: 'error'; message: string }
```

The connection to Atlassian, as the comment says, which `Atlassian` in `src/main/atlassian/Atlassian.ts` sets and broadcasts on `assist:atlassian-state` (see Jira and the service desk). Like `AppsState`, it's told apart by `status`.

- `{ status: 'sign-in'; message?: string }`: not connected; `message` says why when the connection has ended or the last try failed.
- `{ status: 'connected'; user: { name: string; email?: string } }`: connected, with who as, for the settings menu. The account ID and tokens stay in the main process.
- `{ status: 'error'; message: string }`: a saved connection couldn't be tried (Atlassian couldn't be reached, say). The pages offer Retry.

`StatusCategory`, `IssueStatus`, `JiraNotification` and `ServiceRequest`

src/shared/types.ts, lines 181–215

```ts
/** How far along an issue is, by its status's category in Jira (which sets its colour). */
export type StatusCategory = 'new' | 'indeterminate' | 'done'

export interface IssueStatus {
  name: string
  category: StatusCategory
}

/** Someone else assigned the user an issue, or mentioned them, in the last two weeks. */
export interface JiraNotification {
  id: string
  kind: 'assigned' | 'mentioned'
  key: string
  summary: string
  /** Who did it (their name in Atlassian). */
  by: string
  /** ISO time. */
  at: string
  /** The start of the comment they were mentioned in. */
  excerpt?: string
  unread: boolean
}

/** One of the user's own requests to the IT service desk. */
export interface ServiceRequest {
  key: string
  summary: string
  status: IssueStatus
  /** ISO time of the last change. */
  updated: string
  /** The request on the help centre portal. */
  url: string
  /** What's new since the user last looked ("Mark Duong replied", "Now Resolved"), or null. */
  news: string | null
}
```

- `StatusCategory`, `IssueStatus`: an issue's status, with the category Jira puts it in (to do, in progress or done), which decides its colour.
- `JiraNotification`: one line in the Notifications tab, as the comment says. `kind` says which of the two it is; `by` is who did it; `at` is when, as an ISO time; `excerpt` is the start of the comment, for a mention; and `unread` puts a dot on it.
- `ServiceRequest`: one line in My requests. `news` is what's changed since the user last looked, in words, or `null`.

`FeedList` and `JiraFeed`

src/shared/types.ts, lines 217–228

```ts
/** A list the main process keeps up to date in the background. */
export interface FeedList<T> {
  status: 'loading' | 'ready' | 'error'
  items: T[]
  message?: string
}

/** The Notifications tab and My requests. */
export interface JiraFeed {
  notifications: FeedList<JiraNotification>
  requests: FeedList<ServiceRequest>
}
```

- `FeedList<T>`: a list the main process keeps up to date in the background, with its own status. `<T>` is the kind of item, so the same shape serves both lists. An `error` can still have `items`: the ones from the last check that worked, with `message` saying why it couldn't be updated.
- `JiraFeed`: the two lists, broadcast together on `assist:jira-feed`.

`IssueSummary`, `ConfluencePage` and `SearchResult`

src/shared/types.ts, lines 230–250

```ts
/** An issue in a list of search results. */
export interface IssueSummary {
  key: string
  summary: string
  status: IssueStatus
  type: string | null
  url: string
}

/** A Confluence page in search results or suggestions. */
export interface ConfluencePage {
  id: string
  title: string
  /** A line or two from the page, as plain text. */
  excerpt: string
  space: string | null
  url: string
}

export type SearchResult =
  { ok: true; issues: IssueSummary[]; pages: ConfluencePage[] } | { ok: false; message: string }
```

What the Search tab shows: Jira issues (`IssueSummary`, with the issue's address in Jira as `url`) and Confluence pages (`ConfluencePage`, with a plain-text `excerpt` and the space's name). `SearchResult` is both lists, or a message when the search failed.

`IssueComment`, `IssueTransition`, `IssueDetail` and `IssueResult`

src/shared/types.ts, lines 252–283

```ts
export interface IssueComment {
  id: string
  author: string
  /** ISO time. */
  created: string
  /** Markdown. */
  body: string
}

/** A status the issue can be moved to ("Start progress", "Done"). */
export interface IssueTransition {
  id: string
  name: string
}

/** An issue as the issue view shows it. */
export interface IssueDetail extends IssueSummary {
  assignee: string | null
  reporter: string | null
  created: string
  updated: string
  /** Markdown, shortened if it's long. */
  description: string
  /** The latest comments, oldest first. */
  comments: IssueComment[]
  /** The request on the help centre portal, for service desk requests; null otherwise. */
  portalUrl: string | null
  transitions: IssueTransition[]
}

/** Opening an issue, or changing it (which answers with the issue as it is now). */
export type IssueResult = { ok: true; issue: IssueDetail } | { ok: false; message: string }
```

What the issue view shows.

- `IssueComment`: a comment, its `body` as Markdown.
- `IssueTransition`: as the comment says, a status the issue can move to, with the ID Jira needs to make the move.
- `IssueDetail extends IssueSummary`: everything an `IssueSummary` has, plus the rest. `extends` means it doesn't have to list the summary's fields again.
- `portalUrl`: as the comment says, only for service desk requests, which then get a link to the portal.
- `IssueResult`: as its comment says, the answer to opening an issue and to changing one, which is the issue as it is now.

`IssuePrompt` and `AskAboutIssueResult`

src/shared/types.ts, lines 285–289

```ts
/** What to ask Claude about an issue. */
export type IssuePrompt = 'summarise' | 'reply' | 'next-steps'

/** Asking Claude about an issue: `inChat` when it went to the chat in the panel. */
export type AskAboutIssueResult = { ok: true; inChat: boolean } | { ok: false; message: string }
```

- `IssuePrompt`: the three **Ask Claude** buttons on an issue.
- `AskAboutIssueResult`: the answer. `inChat` is `true` when the question went to the built-in chat, so the panel shows it; with Claude Desktop the panel has closed instead.

`NewRequest` and `RaiseResult`

src/shared/types.ts, lines 291–302

```ts
/** A request filled in on the panel, to finish on the help centre portal. */
export interface NewRequest {
  requestTypeId: string
  summary: string
  details: string
  includePcDetails: boolean
}

/** Opening the portal's form, filled in, with the text box's screenshots on the clipboard. */
export type RaiseResult =
  | { ok: true; screenshots: number }
  | { ok: false; reason: 'empty' | 'missing-screenshot' | 'failed' }
```

The service desk page's form, and the answer to sending it (see `ServiceDesk.raise()`).

- `NewRequest`: the form's fields. The screenshots aren't in it: the main process takes them from its own copy of the draft.
- `{ ok: true; screenshots: number }`: the portal's form is opening in the browser, and this many screenshots are on the clipboard.
- `reason: 'empty' | 'missing-screenshot' | 'failed'`: nothing was written, an attached screenshot has gone from the folder, or the request type wasn't one of the tenant's or the browser couldn't be opened.

`ActionResult` and `AttachResult`

src/shared/types.ts, lines 304–311

```ts
export type ActionResult = { ok: true; message?: string } | { ok: false; message: string }

/**
 * Attaching a screenshot: the newest one not attached yet. `no-more` when every screenshot is
 * already attached.
 */
export type AttachResult =
  { ok: true; notes: Notes } | { ok: false; reason: 'no-screenshots' | 'no-more' }
```

The answers to two more requests.

- `{ ok: true; message?: string } | { ok: false; message: string }`: what a command action returns (`src/main/actions.ts`). `PanelView.runAction()` shows `message` as a toast, styled as an error when `ok` is false. A failure must carry a message; a success may (the screenshot action says "Screenshot saved", Bounce says nothing).
- `reason: 'no-screenshots' | 'no-more'`: why **Attach screenshot** did nothing: the folder has no screenshots, or every one of them is already in the draft (each click attaches the newest one that isn't, going one further back each time).

`src/shared/ipc.ts`: the IPC contract

The agreement between the pages and the main process: the name of every IPC channel (`IPC`) and the exact set of functions a page gets as `window.assist` (`AssistApi`). Three places depend on it. `src/preload/index.ts` implements `AssistApi` by sending on these channels; `registerIpc()` in `src/main/ipc.ts` listens on them; and `src/renderer/src/env.d.ts` tells TypeScript that `window.assist` is an `AssistApi`.

Messages come in three styles:

- **Request and reply**: the page calls `ipcRenderer.invoke()`, the main process answers from an `ipcMain.handle()` handler, and the page gets a promise of that answer. If the handler throws (for example because the input failed its zod check), the promise rejects.
- **Fire and forget**: `ipcRenderer.send()` to an `ipcMain.on()` listener, with no answer. Used for frequent or one-way signals such as each keystroke in the text box. In `AssistApi`, the methods that return `void` are these; the ones that return a `Promise` are requests.
- **Pushed by the main process**: `webContents.send()` to a window, which the page receives through one of the `on...` listeners. These are the last nine channels.

`IPC`

`src/shared/ipc.ts`, lines 29–98

```ts
/** IPC channel names. The preload maps them onto `window.assist`; src/main/ipc.ts handles them. */
export const IPC = {
  getState: 'assist:get-state',
  bubbleClick: 'assist:bubble-click',
  bubbleDragStart: 'assist:bubble-drag-start',
  bubbleDragEnd: 'assist:bubble-drag-end',
  collapse: 'assist:collapse',
  setInteractive: 'assist:set-interactive',
  invokeAction: 'assist:invoke-action',
  openExternal: 'assist:open-external',
  copyText: 'assist:copy-text',
  notesSetText: 'assist:notes-set-text',
  notesAttachLatest: 'assist:notes-attach-latest',
  notesRemoveAttachment: 'assist:notes-remove-attachment',
  notesClear: 'assist:notes-clear',
  screenshotThumbnail: 'assist:screenshot-thumbnail',
  screenshotOpen: 'assist:screenshot-open',
  screenshotsOpenFolder: 'assist:screenshots-open-folder',
  settingsGet: 'assist:settings-get',
  settingsSetAutoStart: 'assist:settings-set-auto-start',
  settingsSetEffort: 'assist:settings-set-effort',
  settingsSetAutoSend: 'assist:settings-set-auto-send',
  settingsSetFavoriteApp: 'assist:settings-set-favorite-app',
  settingsSetTheme: 'assist:settings-set-theme',
  authSignIn: 'assist:auth-sign-in',
  authCancel: 'assist:auth-cancel',
  authSignOut: 'assist:auth-sign-out',
  authRetry: 'assist:auth-retry',
  chatSend: 'assist:chat-send',
  chatStop: 'assist:chat-stop',
  chatRetry: 'assist:chat-retry',
  chatNew: 'assist:chat-new',
  claudeDesktopAsk: 'assist:claude-desktop-ask',
  claudeDesktopInstalled: 'assist:claude-desktop-installed',
  appUninstall: 'assist:app-uninstall',
  appsGet: 'assist:apps-get',
  appsSignIn: 'assist:apps-sign-in',
```

```
  appsCancelSignIn: 'assist:apps-cancel-sign-in',
  appsOpen: 'assist:apps-open',
  appsOpenPortal: 'assist:apps-open-portal',
  atlassianGet: 'assist:atlassian-get',
  atlassianSignIn: 'assist:atlassian-sign-in',
  atlassianCancelSignIn: 'assist:atlassian-cancel-sign-in',
  atlassianSignOut: 'assist:atlassian-sign-out',
  atlassianRetry: 'assist:atlassian-retry',
  atlassianFeed: 'assist:atlassian-feed',
  atlassianMarkNotificationsRead: 'assist:atlassian-mark-notifications-read',
  atlassianMarkRequestsRead: 'assist:atlassian-mark-requests-read',
  atlassianSearch: 'assist:atlassian-search',
  atlassianIssue: 'assist:atlassian-issue',
  atlassianTransition: 'assist:atlassian-transition',
  atlassianComment: 'assist:atlassian-comment',
  atlassianLogWork: 'assist:atlassian-log-work',
  atlassianArticles: 'assist:atlassian-articles',
  atlassianProjectKeys: 'assist:atlassian-project-keys',
  atlassianAsk: 'assist:atlassian-ask',
  serviceDeskPcDetails: 'assist:service-desk-pc-details',
  serviceDeskRaise: 'assist:service-desk-raise',
  serviceDeskOpenMyRequests: 'assist:service-desk-open-my-requests',
  // main → renderer
  modeChanged: 'assist:mode-changed',
  cornerChanged: 'assist:corner-changed',
  clickThroughReset: 'assist:click-through-reset',
  authStatus: 'assist:auth-status',
  chatMessage: 'assist:chat-message',
  chatReset: 'assist:chat-reset',
  appsState: 'assist:apps-state',
  atlassianState: 'assist:atlassian-state',
  jiraFeed: 'assist:jira-feed',
} as const
```

Every channel name in one object, so the preload and the main process can't disagree about a name: a typo is a compile error instead of a message nobody hears.

- `'assist:get-state'`: every name starts with `assist:`, so it can't clash with a channel that Electron or a library might use.
- `notesClear`: empties the draft, for "Clear text box" in the settings menu.
- `settingsSetAutoSend`: the settings menu's "Send in Claude automatically" switch.
- `settingsSetFavoriteApp`: a star on a tile in the Apps list.
- `settingsSetTheme`: the light or dark mode icon.
- `appUninstall`: Settings → Uninstall.
- `appsGet` to `appsOpenPortal`: the Apps list. Like the two chats' channels, they're only answered when the tenant has a portal.
- `atlassianGet` to `atlassianAsk`: Jira and Confluence (see Jira and the service desk), answered only when the tenant has `atlassian`. `serviceDeskPcDetails` to `serviceDeskOpenMyRequests`: the service desk page, answered only when the tenant lists request types.
- `authSignIn` to `chatNew`, and `claudeDesktopAsk` and `claudeDesktopInstalled`: the first group belongs to the built-in chat and the second to Claude Desktop. The main process only answers the group the tenant uses (`registerIpc()` in `src/main/ipc.ts`).
- `// main → renderer`: the channels from here on go the other way. `atlassianState` and `jiraFeed` carry the Atlassian connection and the notifications and requests. `broadcast()` in `start()` (`src/main/index.ts`) sends

them to both windows, except `clickThroughReset`, which `resetClickThrough()` in `src/main/bubble/windows.ts` sends only to the panel.

AssistApi

src/shared/ipc.ts, lines 100–232

```ts
/** The API the preload exposes to renderers as `window.assist`. */
export interface AssistApi {
  getState(): Promise<AppState>
  /** A click on the bubble; `pressedAt` is when the button went down (Date.now()). */
  bubbleClick(pressedAt?: number): void
  /** The bubble was pressed and moved: the main process makes it follow the mouse. */
  bubbleDragStart(): void
  /** Released: the bubble snaps to the nearest corner of the display it's on. */
  bubbleDragEnd(): void
  collapse(): void
  /** Tell the main process whether the pointer is over real UI (true) or a see-through area. */
  setInteractive(interactive: boolean): void
  invokeAction(id: CommandActionId): Promise<ActionResult>
  /** Opens a web (http/https) or email (mailto) link in the default app. */
  openExternal(url: string): Promise<void>
  copyText(text: string): Promise<void>
  notes: {
    setText(text: string): void
    /** Attaches the newest screenshot that isn't attached yet (each click goes one further back). */
    attachLatestScreenshot(): Promise<AttachResult>
    removeAttachment(id: string): Promise<Notes>
    /** Empties the text box and removes the attached screenshots (the files are kept). */
    clear(): Promise<Notes>
  }
  screenshots: {
    /** A small data-URL preview, or null if the file is missing. */
    thumbnail(path: string): Promise<string | null>
    open(path: string): Promise<boolean>
    openFolder(): Promise<void>
  }
  settings: {
    get(): Promise<Settings>
    setAutoStart(enabled: boolean): Promise<Settings>
    setEffort(effort: Effort): Promise<Settings>
    setAutoSend(autoSend: boolean): Promise<Settings>
    /** Stars an app in the Apps list (it moves to the top), or takes its star away. */
    setFavoriteApp(id: string, favorite: boolean): Promise<Settings>
    /** Light or dark mode for all of Desktop Assist; saved. */
    setTheme(theme: Theme): Promise<Settings>
  }
  auth: {
    /** Opens JumpCloud in the browser; progress arrives through `onAuthStatus`. */
    signIn(): Promise<void>
    /** Stops waiting for a sign-in started in the browser. */
    cancel(): Promise<void>
    signOut(): Promise<void>
    /** Tries again to renew a saved sign-in that couldn't reach JumpCloud. */
    retry(): Promise<void>
  }
  claudeDesktop: {
    /**
     * Opens a new chat in Claude Desktop with the draft text filled in, and copies the attached
     * screenshots for the user to paste. Clears the draft and collapses the panel if it worked.
     */
    ask(text: string): Promise<AskResult>
    /** Whether Claude Desktop is installed on this PC (anything handles claude:// links). */
```

```
  isInstalled(): Promise<boolean>
}
/** The built-in chat (only when the tenant's `chatApp` is `built-in`). */
chat: {
  /** Sends the draft text plus the draft's attached screenshots. */
  send(text: string): Promise<SendResult>
  stop(): Promise<void>
  retry(): Promise<void>
  newConversation(): Promise<void>
}
/** The user's JumpCloud portal apps (only when the tenant has a `portal`). */
apps: {
  /** The list as it stands; loads it first if needed, or again with `refresh`. */
  get(refresh?: boolean): Promise<AppsState>
  /** Connects Desktop Assist to the portal, in the browser; progress via onAppsState. */
  signIn(): Promise<void>
  cancelSignIn(): Promise<void>
  /** Opens the app in the default browser (signed in through the portal). */
  open(id: string): Promise<boolean>
  openPortal(): Promise<void>
}
/** Jira and Confluence (only when the tenant has `atlassian`). */
atlassian: {
  get(): Promise<AtlassianState>
  /** Connects to Atlassian in the browser; progress via onAtlassianState. */
  signIn(): Promise<void>
  cancelSignIn(): Promise<void>
  /** Forgets the connection. */
  signOut(): Promise<void>
  /** Tries the saved connection again after an error. */
  retry(): Promise<void>
  /** Notifications and the user's IT requests; checked again first with `refresh`. */
  feed(refresh?: boolean): Promise<JiraFeed>
  markNotificationsRead(): Promise<JiraFeed>
  /** Marks what's new on one request (or all of them) as seen. */
  markRequestsRead(key?: string): Promise<JiraFeed>
  search(query: string): Promise<SearchResult>
  issue(key: string): Promise<IssueResult>
  transition(key: string, transitionId: string): Promise<IssueResult>
  comment(key: string, body: string): Promise<IssueResult>
  /** Jira's format: "30m", "1h 30m", "2d". */
  logWork(key: string, timeSpent: string): Promise<IssueResult>
  /** How-to pages about what the user is typing in a new request. */
  articles(text: string): Promise<ConfluencePage[]>
  /** The Jira projects' keys, or null if they can't be listed (e.g. not connected). */
  projectKeys(): Promise<string[] | null>
  /**
   * Asks Claude about the issue, with its details: in Claude Desktop (the panel closes), or the
   * chat in the panel.
   */
  ask(key: string, prompt: IssuePrompt): Promise<AskAboutIssueResult>
}
/** Raising IT requests (only when the tenant's service desk lists request types). */
serviceDesk: {
  /** What "Include PC details" adds. */
  pcDetails(): Promise<string>
  /**
   * Opens the portal's form for the request, filled in, with the text box's screenshots on the
   * clipboard to paste. Closes the panel if it worked.
   */
  raise(request: NewRequest): Promise<RaiseResult>
  openMyRequests(): Promise<void>
}
/** Starts Desktop Assist's uninstaller and quits (installed copies only). */
uninstall(): Promise<UninstallResult>
```

```
  onModeChanged(callback: (mode: Mode) => void): () => void
  onCornerChanged(callback: (corner: Corner) => void): () => void
  /** The panel was just shown with click-through reset; `pointer` is where the mouse is now. */
  onClickThroughReset(callback: (pointer: Point) => void): () => void
  onAuthStatus(callback: (status: AuthStatus) => void): () => void
  /** A message was added or changed (streamed text arrives this way). */
  onChatMessage(callback: (message: ChatMessage) => void): () => void
  onChatReset(callback: () => void): () => void
  onAppsState(callback: (state: AppsState) => void): () => void
  onAtlassianState(callback: (state: AtlassianState) => void): () => void
  onJiraFeed(callback: (feed: JiraFeed) => void): () => void
}
```

The API a page sees as `window.assist`. Each method maps onto one channel in `IPC`. Related calls are grouped into nested objects (`notes`, `screenshots`, `settings`, `auth`, `claudeDesktop`, `chat`, `apps`, `atlassian`, `serviceDesk`), which is only for readability. Every page gets all of them, but `auth` and `chat` only work with the built-in chat, `claudeDesktop` only with Claude Desktop, `apps` only when the tenant has a portal, `atlassian` only when it has an Atlassian site, and `serviceDesk` only when it lists request types; the panel checks `branding` to know which to call.

- `getState(): Promise<AppState>`: called once when a page loads (`useAssistState()`).
- `bubbleClick(pressedAt?: number): void`: this and the next three are fire-and-forget calls into `BubbleController` (`clickBubble()`, `startDrag()`, `endDrag()`, `collapse()`). `pressedAt` is when the mouse button went down, which tells the main process whether the press itself closed the panel.
- `setInteractive(interactive: boolean): void`: the heart of click-through. `useClickThrough()` sends `true` when the pointer moves onto an element marked `data-hit` and `false` when it leaves; the main process then turns mouse input on or off for that window with `setIgnoreMouseEvents()`.
- `invokeAction(id: CommandActionId): Promise<ActionResult>`: runs a command action; only command ids are allowed (see `CommandActionId`).
- `openExternal(url: string): Promise<void>`: opens a link from one of Claude's replies. The main process only accepts web (`http:`, `https:`) and email (`mailto:`) addresses (`ExternalUrl` in `src/main/ipc.ts`).
- `setText(text: string): void`: sent on every keystroke, so it doesn't wait for an answer. `NotesStore` saves the draft to disk about half a second after typing stops.
- `clear(): Promise<Notes>`: empties the text and removes the attachments from the draft, and returns the empty draft. The screenshot files themselves stay in the folder.
- `thumbnail(path: string): Promise<string | null>`: returns the preview as a *data URL*, the image encoded into a `data:image/...` string. The page can't load files from disk itself; its Content-Security-Policy (`index.html`) only allows images from the app or from data URLs.
- `setEffort(effort: Effort)`, `setAutoSend(autoSend: boolean)`: save the response style (built-in chat) or "Send in Claude automatically" (Claude Desktop), and return the settings as they now are, like the other `settings` calls. `setFavoriteApp(id, favorite)` does the same for a star in the Apps list, and `setTheme(theme)` for light or dark mode.
- `signIn(): Promise<void>`: resolves straight away, because signing in happens in the browser and can take minutes. Progress arrives through `onAuthStatus`.
- `ask(text: string): Promise<AskResult>`: like `chat.send()`, only the text is passed; the main process takes the screenshots from its own copy of the draft. It resolves once Claude Desktop has been asked to open; sending the question there carries on in the main process afterwards, and there's no reply to wait for, because the conversation carries on in Claude Desktop.
- `isInstalled(): Promise<boolean>`: whether anything on the PC opens `claude://` links. The panel asks each time it opens, to warn before anything is typed.

- `send(text: string): Promise<SendResult>`: sends the draft. The promise only says whether it was accepted; Claude's reply streams in through `onChatMessage`.
- `get(refresh?: boolean): Promise<AppsState>`: the Apps list. The first call loads it (which can take a few seconds), later ones return the list already loaded unless `refresh` is true.
- `apps.signIn(): Promise<void>`: like `auth.signIn()`, resolves straight away; connecting happens in the browser and progress arrives through `onAppsState`.
- `open(id: string): Promise<boolean>`: asks the main process to open the app in the browser. `false` means it had no link for it.
- `atlassian.signIn()`, `retry()`: like `apps.signIn()`, resolve straight away; progress arrives through `onAtlassianState`.
- `feed(refresh?: boolean): Promise<JiraFeed>`: the notifications and requests, checked again first with `refresh` (or if the last check is over a minute old). The answer also arrives through `onJiraFeed`, like every other change.
- `transition()`, `comment()`, `logWork()`: each answers with the issue as it is after the change (`IssueResult`).
- `projectKeys()`: the keys of the Jira projects the user can see, so the text box only offers to open real issue keys; `null` when they aren't known.
- `ask(key, prompt)`: as its comment says, asks Claude about an issue with its details, in Claude Desktop or the built-in chat.
- `serviceDesk.raise(request: NewRequest)`: as the comment says, opens the portal's form, filled in. Like `claudeDesktop.ask()`, the screenshots come from the main process's copy of the draft.
- `uninstall(): Promise<UninstallResult>`: starts the uninstaller; the app quits if it worked.
- `onModeChanged(callback: (mode: Mode) => void): () => void`: each `on...` method starts listening and returns a function that stops. That suits React's `useEffect`, whose cleanup step can simply call it (see `useAssistState()`).
- `onClickThroughReset(callback: (pointer: Point) => void): () => void`: `pointer` is the mouse position relative to the panel window, so the page can decide at once whether the pointer is over real UI.

`src/shared/format.ts`: screenshot labels and times

Two small text helpers. `AttachmentChip.tsx` uses `screenshotLabel()` for the label on each screenshot chip in the text box, and the Jira and service desk pages use `timeAgo()` to say when something happened. The file has no Electron or React code in it, and `tests/format.test.ts` tests it directly. It's short, so here it is whole.

`screenshotLabel()` and `timeAgo()`

src/shared/format.ts, lines 1–29

```ts
const SCREENSHOT_NAME =
  /^Screenshot (\d{4})-(\d{2})-(\d{2}) (\d{2})(\d{2})(\d{2})(?: \(\d+\))?\.png$/
const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

/**
 * Short chip label for a screenshot file: "Screenshot 14:03" if it was taken today, otherwise
 * "Screenshot 2 Oct 14:03". Unrecognised names are shown as-is.
 */
export function screenshotLabel(fileName: string, now: Date = new Date()): string {
  const match = SCREENSHOT_NAME.exec(fileName)
  if (!match) return fileName
  const [, year, month, day, hour, minute] = match.map(Number) as number[]
  const time = `${String(hour).padStart(2, '0')}:${String(minute).padStart(2, '0')}`
  const today = year === now.getFullYear() && month === now.getMonth() + 1 && day === now.getDate()
  return today ? `Screenshot ${time}` : `Screenshot ${day} ${MONTHS[(month ?? 1) - 1]} ${time}`
}

/** How long ago something happened, briefly: "just now", "5m ago", "3h ago", "2d ago", "12 Sep". */
export function timeAgo(iso: string, now: Date = new Date()): string {
  const then = new Date(iso)
  const minutes = Math.floor((now.getTime() - then.getTime()) / 60_000)
  if (Number.isNaN(minutes)) return ''
  if (minutes < 1) return 'just now'
  if (minutes < 60) return `${minutes}m ago`
  if (minutes < 24 * 60) return `${Math.floor(minutes / 60)}h ago`
  if (minutes < 7 * 24 * 60) return `${Math.floor(minutes / (24 * 60))}d ago`
  const date = `${then.getDate()} ${MONTHS[then.getMonth()]}`
  return then.getFullYear() === now.getFullYear() ? date : `${date} ${then.getFullYear()}`
}
```

Turns a screenshot's file name into a short label: just the time for today's screenshots, the date and time for older ones.

- `SCREENSHOT_NAME =`: matches the names that `screenshotFileName()` in `src/main/screenshots/files.ts` creates, such as `Screenshot 2026-10-02 140311.png`. The parentheses capture year, month, day, hour, minute and second. `(?: \(\d+\))?` allows an optional `(2)` for a second capture in the same second, without capturing the number.
- `now: Date = new Date()`: a default parameter. The tests pass a fixed date so the result doesn't depend on the day they run.
- `if (!match) return fileName`: any other name is shown as it is.
- `const [, year, month, day, hour, minute] = match.map(Number)`: turns the captured text into numbers. The leading comma skips `match[0]`, the whole name; the seconds are captured but not used.
- `month === now.getMonth() + 1`: JavaScript months count from 0, file names from 1. Both the file name and `now` use the local clock, so "today" is the user's today.
- `(month ?? 1) - 1`: the regex guarantees `month` is there, but this project turns on TypeScript's `noUncheckedIndexedAccess` setting, which treats every array element as possibly missing. The `?? 1` fallback satisfies the compiler.

`timeAgo()`, as its comment says, turns an ISO time into a short "how long ago":

- `Math.floor((now.getTime() - then.getTime()) / 60_000)`: the whole minutes since then. `Math.floor` rounds down, so 90 minutes is "1h ago".

- `if (Number.isNaN(minutes)) return ''`: a time that can't be read gives "not a number", and then nothing is shown.
- "just now" under a minute, then minutes, hours, and days for up to a week.
- After a week, the date (`12 Sep`), with the year only if it isn't this year (`12 Sep 2025`).

`src/shared/claudeDesktop.ts`: where to get Claude Desktop

One constant for the Claude Desktop hand-over. The rest of that feature is in the main process (`src/main/claudeDesktop.ts`, see Claude Desktop), but a page can't import main-process files, so the one value the panel needs lives here.

`CLAUDE_DOWNLOAD_PAGE`

src/shared/claudeDesktop.ts, lines 1–4

```
// Claude Desktop, which questions are handed to when the tenant's `chatApp` is `claude-desktop`.

/** Where people get Claude Desktop when it isn't installed (IT may install it for them instead). */
export const CLAUDE_DOWNLOAD_PAGE = 'https://claude.com/download'
```

Anthropic's download page for Claude Desktop. When the panel finds that Claude Desktop isn't installed, it shows a banner with a **Download** button that opens this page in the browser (`window.assist.openExternal()`, which allows `https:` links). On company PCs IT may install Claude Desktop instead, which is why the banner suggests asking IT first.

`src/shared/apps.ts`: the order of the Apps list

One small function about the Apps list that doesn't need the main process. It's in `src/shared` so `tests/portalApps.test.ts` can test it without a page.

`orderApps()`

src/shared/apps.ts, lines 3–13

```
/**
 * The apps in the order the Apps list shows them: the starred ones first, then the rest, each
 * keeping the order they came in (by name).
 */
export function orderApps(apps: PortalApp[], favorites: string[]): PortalApp[] {
  const starred = new Set(favorites)
  return [
    ...apps.filter((app) => starred.has(app.id)),
    ...apps.filter((app) => !starred.has(app.id)),
  ]
}
```

The order `AppsList` shows the apps in: the starred ones first, then the rest. The main process already sorts the list by name, and both groups keep that order, so the starred apps are alphabetical among themselves too.

- `new Set(favorites)`: a `Set` answers "is this ID starred?" (`has`) without searching the whole list each time.

- `[...apps.filter(...), ...apps.filter(...)]`: the starred apps, then the others, joined into one new list. The list it was given isn't changed.

`src/shared/jira.ts`: issue keys

Jira issue keys, which, as the comment at the top says, both the main process and the panel need: the panel to offer an **Open** button for keys typed in the text box, and the main process to check keys and build links. The file is short, so here it is whole.

`ISSUE_KEY`, `findIssueKeys()`, `isIssueKey` and `browseUrl()`

src/shared/jira.ts, lines 1–17

```
// Jira issue keys, used by the main process and the panel alike.

/** Jira issue keys in text: a project key, a dash and a number ("ISD-1234"). */
const ISSUE_KEY = /\b[A-Z][A-Z0-9_]{1,9}-\d{1,7}\b/g

/** The issue keys in the text, first to last, each once. */
export function findIssueKeys(value: string, limit = 3): string[] {
  return [...new Set(value.match(ISSUE_KEY) ?? [])].slice(0, limit)
}

export const isIssueKey = (value: string): boolean =>
  /^[A-Z][A-Z0-9_]{1,9}-\d{1,7}$/.test(value.trim())

/** The issue's page in Jira. */
export function browseUrl(site: string, key: string): string {
  return `${new URL(site).origin}/browse/${encodeURIComponent(key)}`
}
```

- `ISSUE_KEY`: a key is a project key (a capital letter, then 1 to 9 more capitals, digits or `_`), a dash and a number, such as `ISD-1234`. `\b` at each end means a whole word, so `myISD-12` or `ISD-123x` don't count. `g` finds every match, not just the first.
- `findIssueKeys(value, limit)`: the keys in the text, each once and at most three. `new Set(...)` drops repeats while keeping the first-to-last order, and `[...]` turns it back into a list. `?? []` covers `match()` finding nothing (it returns `null` then).
- `isIssueKey(value)`: whether the whole text is one key. `^` and `$` pin the pattern to the start and end. The main process checks the same pattern (`IssueKey` in `src/main/ipc.ts`).
- `browseUrl(site, key)`: the issue's page in Jira, such as `https://morsemicro.atlassian.net/browse/ISD-1234`. `encodeURIComponent` keeps the key from changing the address, whatever it holds.

`src/preload/index.ts`: the bridge to the page

A *preload script* runs inside each window (bubble and panel) just before the page's own code. This one builds `window.assist` and is the only route from a page to the main process. To see why it exists, look at how the windows are created (`webPreferences` in `src/main/bubble/windows.ts`):

- `contextIsolation: true`: the preload runs in its own JavaScript "world", with its own globals, separate from the page's. The page can't see or tamper with the preload's variables, including its `ipcRenderer`.
- `nodeIntegration: false`: the page has no Node.js at all: no `require`, no file access.

- `sandbox: true`: even the preload gets only a small part of Electron and Node, and can't load other files at runtime. So electron-vite bundles `@shared/ipc` into the built `out/preload/index.js`, which contains its own copy of `IPC`.

contextBridge is Electron's way across the wall between the two worlds: it copies a chosen object into the page's world. The page therefore gets a fixed list of functions, each sending one specific channel with only the arguments it names. It never gets `ipcRenderer` itself, which would let it send anything on any channel.

`subscribe()`

`src/preload/index.ts`, lines 4–10

```ts
function subscribe<T>(channel: string, callback: (payload: T) => void): () => void {
  const listener = (_event: IpcRendererEvent, payload: T) => callback(payload)
  ipcRenderer.on(channel, listener)
  return () => {
    ipcRenderer.removeListener(channel, listener)
  }
}
```

Starts listening on a channel the main process pushes to, and returns a function that stops listening. Every `on...` method in `api` uses it.

- `const listener = (_event: IpcRendererEvent, payload: T) => callback(payload)`: Electron calls a listener with an event object first. That object's `sender` is the `ipcRenderer` itself, so passing it on would hand the page the very thing the isolation hides. The wrapper passes only the payload.
- `ipcRenderer.removeListener(channel, listener)`: removing a listener needs the exact function that was added. The returned closure keeps hold of `listener`, so the page never has to hand its callback back to unsubscribe.

`api`

`src/preload/index.ts`, lines 12–99

```ts
const api: AssistApi = {
  getState: () => ipcRenderer.invoke(IPC.getState),
  bubbleClick: (pressedAt) => ipcRenderer.send(IPC.bubbleClick, pressedAt),
  bubbleDragStart: () => ipcRenderer.send(IPC.bubbleDragStart),
  bubbleDragEnd: () => ipcRenderer.send(IPC.bubbleDragEnd),
  collapse: () => ipcRenderer.send(IPC.collapse),
  setInteractive: (interactive) => ipcRenderer.send(IPC.setInteractive, interactive),
  invokeAction: (id) => ipcRenderer.invoke(IPC.invokeAction, id),
  openExternal: (url) => ipcRenderer.invoke(IPC.openExternal, url),
  copyText: (text) => ipcRenderer.invoke(IPC.copyText, text),
  notes: {
    setText: (text) => ipcRenderer.send(IPC.notesSetText, text),
    attachLatestScreenshot: () => ipcRenderer.invoke(IPC.notesAttachLatest),
    removeAttachment: (id) => ipcRenderer.invoke(IPC.notesRemoveAttachment, id),
    clear: () => ipcRenderer.invoke(IPC.notesClear),
  },
  screenshots: {
    thumbnail: (path) => ipcRenderer.invoke(IPC.screenshotThumbnail, path),
    open: (path) => ipcRenderer.invoke(IPC.screenshotOpen, path),
    openFolder: () => ipcRenderer.invoke(IPC.screenshotsOpenFolder),
  },
  settings: {
    get: () => ipcRenderer.invoke(IPC.settingsGet),
```

```
    setAutoStart: (enabled) => ipcRenderer.invoke(IPC.settingsSetAutoStart, enabled),
    setEffort: (effort) => ipcRenderer.invoke(IPC.settingsSetEffort, effort),
    setAutoSend: (autoSend) => ipcRenderer.invoke(IPC.settingsSetAutoSend, autoSend),
    setTheme: (theme) => ipcRenderer.invoke(IPC.settingsSetTheme, theme),
    setFavoriteApp: (id, favorite) =>
      ipcRenderer.invoke(IPC.settingsSetFavoriteApp, { id, favorite }),
  },
  auth: {
    signIn: () => ipcRenderer.invoke(IPC.authSignIn),
    cancel: () => ipcRenderer.invoke(IPC.authCancel),
    signOut: () => ipcRenderer.invoke(IPC.authSignOut),
    retry: () => ipcRenderer.invoke(IPC.authRetry),
  },
  claudeDesktop: {
    ask: (text) => ipcRenderer.invoke(IPC.claudeDesktopAsk, text),
    isInstalled: () => ipcRenderer.invoke(IPC.claudeDesktopInstalled),
  },
  chat: {
    send: (text) => ipcRenderer.invoke(IPC.chatSend, text),
    stop: () => ipcRenderer.invoke(IPC.chatStop),
    retry: () => ipcRenderer.invoke(IPC.chatRetry),
    newConversation: () => ipcRenderer.invoke(IPC.chatNew),
  },
  apps: {
    get: (refresh) => ipcRenderer.invoke(IPC.appsGet, refresh ?? false),
    signIn: () => ipcRenderer.invoke(IPC.appsSignIn),
    cancelSignIn: () => ipcRenderer.invoke(IPC.appsCancelSignIn),
    open: (id) => ipcRenderer.invoke(IPC.appsOpen, id),
    openPortal: () => ipcRenderer.invoke(IPC.appsOpenPortal),
  },
  atlassian: {
    get: () => ipcRenderer.invoke(IPC.atlassianGet),
    signIn: () => ipcRenderer.invoke(IPC.atlassianSignIn),
    cancelSignIn: () => ipcRenderer.invoke(IPC.atlassianCancelSignIn),
    signOut: () => ipcRenderer.invoke(IPC.atlassianSignOut),
    retry: () => ipcRenderer.invoke(IPC.atlassianRetry),
    feed: (refresh) => ipcRenderer.invoke(IPC.atlassianFeed, refresh ?? false),
    markNotificationsRead: () => ipcRenderer.invoke(IPC.atlassianMarkNotificationsRead),
    markRequestsRead: (key) => ipcRenderer.invoke(IPC.atlassianMarkRequestsRead, key),
    search: (query) => ipcRenderer.invoke(IPC.atlassianSearch, query),
    issue: (key) => ipcRenderer.invoke(IPC.atlassianIssue, key),
    transition: (key, transitionId) =>
      ipcRenderer.invoke(IPC.atlassianTransition, { key, transitionId }),
    comment: (key, body) => ipcRenderer.invoke(IPC.atlassianComment, { key, body }),
    logWork: (key, timeSpent) => ipcRenderer.invoke(IPC.atlassianLogWork, { key, timeSpent }),
    articles: (text) => ipcRenderer.invoke(IPC.atlassianArticles, text),
    projectKeys: () => ipcRenderer.invoke(IPC.atlassianProjectKeys),
    ask: (key, prompt) => ipcRenderer.invoke(IPC.atlassianAsk, { key, prompt }),
  },
  serviceDesk: {
    pcDetails: () => ipcRenderer.invoke(IPC.serviceDeskPcDetails),
    raise: (request) => ipcRenderer.invoke(IPC.serviceDeskRaise, request),
    openMyRequests: () => ipcRenderer.invoke(IPC.serviceDeskOpenMyRequests),
  },
  uninstall: () => ipcRenderer.invoke(IPC.appUninstall),
  onModeChanged: (callback) => subscribe(IPC.modeChanged, callback),
  onCornerChanged: (callback) => subscribe(IPC.cornerChanged, callback),
  onClickThroughReset: (callback) => subscribe(IPC.clickThroughReset, callback),
  onAuthStatus: (callback) => subscribe(IPC.authStatus, callback),
  onChatMessage: (callback) => subscribe(IPC.chatMessage, callback),
  onChatReset: (callback) => subscribe(IPC.chatReset, () => callback()),
  onAppsState: (callback) => subscribe(IPC.appsState, callback),
  onAtlassianState: (callback) => subscribe(IPC.atlassianState, callback),
  onJiraFeed: (callback) => subscribe(IPC.jiraFeed, callback),
}
```

The implementation of `AssistApi`: one line per method, each sending its channel from `IPC`.

- `const api: AssistApi`: the type annotation makes TypeScript check that every method exists with the right parameters, and lets it infer parameter types (for example, `interactive` is a `boolean`). It can't check the answers: `ipcRenderer.invoke()` returns `Promise<any>`, so the main process's handlers must return what `AssistApi` promises by agreement.
- `ipcRenderer.invoke(IPC.getState)`: request and reply. Nothing extra is passed, which matters because the main process checks that no-argument channels really receive no argument (`NoArgs` in `src/main/ipc.ts`).
- `ipcRenderer.send(IPC.bubbleClick, pressedAt)`: fire and forget, as for the other `void` methods, with the press time.
- `subscribe(IPC.chatReset, () => callback())`: the reset carries no data, so the page's callback is called with no arguments, matching its `() => void` type.
- `ipcRenderer.invoke(IPC.appsGet, refresh ?? false)`: `refresh` is optional for the page, but the main process's check for this channel wants a real `true` or `false`, so a missing one is sent as `false`.
- `ipcRenderer.invoke(IPC.settingsSetFavoriteApp, { id, favorite })`: a request carries one argument, checked by one schema in the main process, so the two values go as one object. The Atlassian calls that take two values (`transition`, `comment`, `logWork` and `ask`) do the same, and `atlassian.feed()` sends a missing `refresh` as `false`, like `apps.get()`.

`contextBridge.exposeInMainWorld()`

The last line of the file, `contextBridge.exposeInMainWorld('assist', api)`, publishes `api` to the page as `window.assist` ("main world" is Electron's name for the page's own world). From then on the renderer calls `window.assist.chat.send(...)` and so on, and `src/renderer/src/env.d.ts` declares `window.assist` as an `AssistApi` so those calls are type-checked too.

Main process: startup, IPC and app plumbing

The *main process* is the Node.js side of an Electron app: the single process that owns the windows, the files, the tray icon and the network. This page covers how it starts and how the pages talk to it. `index.ts` creates every service and wires them together, `ipc.ts` answers the pages' requests, and the smaller files hold the problem log, the tenant config, the user's preferences, the commands behind the action icons, uninstalling and the tray icon. The services themselves (sign-in, Claude, the bubble, the draft and screenshots, Claude Desktop, the Apps list, Jira and the service desk) each have their own page.

`src/main/index.ts`: startup and shutdown

The main process's entry point. electron-vite builds it into `out/main/index.js`, and the `main` field of `apps/desktop/package.json` tells Electron to run that file. It works in three stages:

1. **As soon as the file loads**, module-level code picks the data folder and makes sure only one copy of the app is running.
2. **Once Electron is ready**, `start()` creates the windows and every service, connects them with callbacks, registers the IPC handlers and the tray icon, and finally shows the bubble. Where questions go depends on the tenant's `chatApp`: `start()` either sets up the built-in chat (with `startBuiltInChat()`) or hands questions over to the Claude Desktop app (`ClaudeDesktop`, see Claude Desktop). If the tenant has a `portal`, it also sets up the Apps list (`PortalApps`, see Your apps). With `atlassian` it connects to Jira (`Atlassian`), and with the service desk's request types it sets up raising IT requests from the panel (`ServiceDesk`); see Jira and the service desk.
3. **When the app quits**, a `before-quit` handler saves the draft before letting it close.

The key idea is that the services don't know about each other or about Electron's globals. Each one is handed what it needs (callbacks, small adapter objects) when it's created, which is why the tests can drive them with fakes. `index.ts` is the only file that knows about all of them.

Module-level code

src/main/index.ts, lines 61–77

```ts
const SHUTDOWN_TIMEOUT_MS = 2000

/** The installer's product name (electron-builder.cjs): its uninstaller is named after it. */
const PRODUCT_NAME = 'Desktop Assist'

let controller: BubbleController | null = null

let tray: Tray | null = null
```

This runs the moment Electron loads the file, before the app is ready. The block shows only the four declarations. Two `if` statements sit between them (they can't be shown on their own), so they're quoted in the bullets below, in the order they run.

- `SHUTDOWN_TIMEOUT_MS = 2000`: the longest quitting will wait for the draft to be saved (see Quitting below).
- `PRODUCT_NAME = 'Desktop Assist'`: the installer's product name, set in `electron-builder.cjs`. The installer names its uninstaller after it ("Uninstall Desktop Assist.exe"), which is how Settings → Uninstall finds it (see `src/main/uninstall.ts`).

- `if (!app.isPackaged)`: `app.isPackaged` is true for an installed build and false for `npm run dev`. Dev runs move their *userData* folder (Electron's per-app data folder, normally `%APPDATA%\Desktop Assist`) to `Desktop Assist (Dev)`, so they don't share the draft, the saved sign-in or the single-instance lock with an installed copy.
- `process.env['DESKTOP_ASSIST_DEV_USER_DATA'] ||`: the automated tests name a folder of their own in this environment variable, so they never touch your dev data. `app.setPath('userData', …)` has to run before anything reads that path, which is why this is at the very top of the file.
- `let controller`: the bubble controller, kept at module level so the `second-instance` handler below can reach it. It stays `null` until `start()` has created it.
- `let tray`: if JavaScript frees the `Tray` object (garbage collection, which happens once nothing refers to it), its icon can vanish from the notification area. A module-level variable keeps it alive for the whole run.
- `if (!app.requestSingleInstanceLock())`: the *single-instance lock* lets only one copy of the app run per data folder. A second copy fails to get the lock and quits at once (`app.quit()`).
- `app.on('second-instance', () => controller?.expand())`: Electron fires `second-instance` in the copy that's already running when another copy tries to start. So launching the app again (from the Start menu, say) just opens the panel. The `?.` skips the call if `start()` hasn't created the controller yet.
- `app.whenReady().then(start).catch(fail)`: windows, the screen and the tray can only be used once Electron has finished starting up, and `whenReady()` returns a promise for that moment. Then `start()` runs. If anything fails along the way, `fail()` shows the error.
- `.catch(fail) after .then(start)`: as the comment says, the order matters. Written as `.then(start, fail)`, `fail` would only run if `whenReady()` itself failed; an error thrown inside `start()` would go unhandled and no error box would appear. A `.catch()` at the end catches a failure from either step.

`start()`

Builds the whole app once Electron is ready. It's long, so it's shown below in pieces, in source order. Only declarations (`const`, `let`) can be picked out as pieces; the plain statements in between, such as `registerIpc({...})` and the `before-quit` handler, are quoted in the bullets. The link above each piece opens the full source. The built-in chat's own pieces (the Claude backend, the conversation and the JumpCloud sign-in) are in a separate function, `startBuiltInChat()`, which `start()` calls only when the tenant uses it.

The tenant config

`src/main/index.ts`, line 89

```
const tenant = await withDevOverrides(TenantSchema.parse(tenantConfig))
```

- `TenantSchema.parse(tenantConfig)`: `tenantConfig` is the bundled `tenant.json` (the `@tenant` import shortcut points at `tenants/<TENANT>/`). TypeScript only checks it at build time, so it's checked again here against the schema in `tenant.ts`. A bad config throws and the app doesn't start.
- `await withDevOverrides(...)`: in dev runs, a JSON file can change where chats happen and the JumpCloud and Claude access settings (see `withDevOverrides()`). It returns a whole tenant, checked again, so the rest of `start()` simply reads `tenant`. In an installed build it's the bundled config unchanged.
- `Menu.setApplicationMenu(null)`: removes Electron's default menu (File, Edit, View and so on) and the keyboard shortcuts that come with it, such as reload and developer tools.
- `app.setAppUserModelId('com.morsemicro.desktopassist')`: the *AppUserModelID* is how Windows identifies an app, for example to label its notifications (such as the "Screenshot copied" one when a question goes to Claude Desktop). It matches `appId` in `electron-builder.cjs`, the ID the installer registers.

Folders, the draft and preferences

src/main/index.ts, lines 93–104

```
const userData = app.getPath('userData')

const screenshotsDir = join(app.getPath('pictures'), tenant.appName)

const actionCount = tenant.actions.length

// Problems go to %APPDATA%\Desktop Assist\logs, so an installed copy can be diagnosed too.
const log = createLog(join(userData, 'logs', 'desktop-assist.log'))

const notes = new NotesStore(join(userData, 'notes.json'), {
  onError: (error) => log('Saving the text box failed:', error),
})

const settings = new SettingsService(join(userData, 'preferences.json'), screenshotsDir)
```

- `app.getPath('pictures')`: the user's Pictures folder. Screenshots go in a subfolder named after the app (`appName` in `tenant.json`).
- `createLog(join(userData, 'logs', 'desktop-assist.log'))`: the problem log (see `src/main/log.ts`). `log` is a function: `log(message, detail)` appends a line to `%APPDATA%\Desktop Assist\logs\desktop-assist.log` and prints it to the terminal. An installed copy has no terminal, so this file is the only record of what went wrong.
- `new NotesStore(join(userData, 'notes.json'), ...)`: the unsent draft in the text box. It's followed by `await notes.load()`, which reads the saved draft before anything can ask for it. `onError` only logs: a failed save shouldn't interrupt the user (see The draft, screenshots and safe files).
- `new SettingsService(join(userData, 'preferences.json'), screenshotsDir)`: the user's preferences, followed by `await settings.init()`, which loads them and, on an installed build's first run, turns on "Start with Windows" (see `SettingsService.init()`).

The two windows

src/main/index.ts, lines 107–116

```
// The windows are created after the last `await`: from here to registerIpc() below nothing
// waits, so the pages can't ask for the app state before there's anything to answer them.
const bubbleWindow = createOverlayWindow('bubble', { width: BUBBLE_BOX, height: BUBBLE_BOX })

const panelWindow = createOverlayWindow('panel', panelWindowSize(actionCount))

const windows = [bubbleWindow, panelWindow]

// Listen straight away, so a page that finishes loading quickly isn't missed.
const windowsReady = Promise.all(windows.map((win) => once(win, 'ready-to-show')))

const broadcast = (channel: string, payload?: unknown) => {
  for (const win of windows) if (!win.isDestroyed()) win.webContents.send(channel, payload)
}
```

The windows are created only now, after the last `await`, and the comment above them says why. A page starts loading as soon as its window exists, and its first request is for the app state (`getState`). From here down to `registerIpc()`, `start()` never waits, so JavaScript runs it all in one go and can't handle anything else in between, a page's request included. By the time a page's request is handled, its handler has been registered.

- `createOverlayWindow('bubble', ...)`: creates a hidden, frameless, transparent, always-on-top window and starts loading the page into it (`bubble/windows.ts`, see The bubble). The bubble window is just big enough for the bubble; the panel window's size depends on how many action icons the tenant has (`panelWindowSize()`).
- `once(win, 'ready-to-show')`: Node's `once()` turns the next firing of an event into a promise. `ready-to-show` fires when a hidden window's page has painted for the first time. The listeners are attached as soon as the windows exist, because an event that fires before anyone is listening is lost and the promise would never resolve. `start()` waits for it at the very end.
- `broadcast`: sends a message to both pages. `webContents.send(channel, payload)` is how the main process pushes a message to a page (the page listens through the preload's `on...` functions, see Shared code and the preload bridge). `isDestroyed()` skips a window that has already been closed, which happens while quitting.

The bubble controller

`src/main/index.ts`, lines 118–130

```
const bubble = new BubbleController({
  bubble: bubbleSurface(bubbleWindow),
  panel: panelSurface(panelWindow),
  displays: electronDisplays,
  actionCount,
  initialAnchor: settings.bubbleAnchor,
  onDiagnostic: (message) => log(message),
  onModeChange: (mode) => broadcast(IPC.modeChanged, mode),
  onAnchorChange: (anchor) => {
    broadcast(IPC.cornerChanged, anchor.corner)
    void settings.setBubbleAnchor(anchor) // remembered for next time
  },
})
```

`BubbleController` is the bubble's state machine: it decides when the panel opens, where the bubble sits, and moves and shows the windows to match (see The bubble). It never touches Electron directly; everything it needs is passed in here.

- `bubbleSurface(bubbleWindow)`, `panelSurface(panelWindow)`: small adapters (in `bubble/windows.ts`) that give the controller just "set bounds, show, hide, focus" for each window.
- `displays: electronDisplays`: the adapter for Electron's `screen` module, defined further down this file (`electronDisplays`).
- `initialAnchor: settings.bubbleAnchor`: the corner and display the bubble was dragged to last time, or `null` the first time.
- `onModeChange`: every mode change (collapsed, expanded, dragging and so on) is broadcast, so the pages can animate to match.
- `onDiagnostic: (message) => log(message)`: the controller notes bubble clicks it ignored, and a blur it ignored just after opening, in the problem log (see The bubble).
- `onAnchorChange`: when the bubble settles in a new corner, the panel is told (it lays itself out to open away from that corner) and the new anchor is saved. `void` in front of a promise means "start this and don't wait for it"; it also tells the linter the missing `await` is deliberate.
- `controller = bubble` (the next line): makes the controller reachable from the module-level `second-instance` handler.
- `panelWindow.on('blur', () => bubble.panelBlurred())`: `blur` fires when the panel loses keyboard focus, for example when the user clicks another app. The controller then closes the panel (unless it has only just opened).

Screenshots, notifications and the clipboard

src/main/index.ts, lines 134–155

```
const screenshots = new ScreenshotService(screenshotsDir, () =>
  screen.getDisplayMatching(bubbleWindow.getBounds()),
)

/**
 * A Windows notification. Windows shows it in the bottom-right corner, over the bubble when it's
 * there: clicking it does what clicking the bubble would.
 */
const notify = ({ title, body }: { title: string; body: string }) => {
  if (!Notification.isSupported()) return
  const notification = new Notification({ title, body })
  notification.on('click', () => bubble.expand())
  notification.show()
}

/** Screenshots go on the clipboard as one image, for pasting where a link can't carry them. */
const copyScreenshots = async (paths: string[]) => {
  const image = screenshots.forClipboard(paths)
  if (!image) return false
  const png = new Blob([image.toPNG()], { type: 'image/png' })
  await clipboard.write([new ClipboardItem({ 'image/png': png })])
  return true
}
```

- `screen.getDisplayMatching(bubbleWindow.getBounds())`: the screenshot service is given a function rather than a fixed display, so each capture takes the display the bubble is on at that moment.
- `notify`: a Windows notification, shared by three services: Claude Desktop (to say what's left to do in Claude: paste the screenshot, or press Enter), the service desk (the screenshot is ready to paste on the portal's form) and Atlassian (IT replied to a request). The panel can't say these things, because it has closed by then. `if (!Notification.isSupported()) return`: where Windows can't show notifications, nothing is shown.
- `notification.on('click', () => bubble.expand())`: as the comment says, Windows shows notifications in the bottom-right corner of the screen, which is where the bubble usually is. Someone reaching for the bubble can easily click the notification instead, so clicking it opens the panel, as the bubble would.
- `copyScreenshots`: as the comment says, puts the attached screenshots on the clipboard as one image, for pasting where a link can't carry them (Claude Desktop, and the portal's request form). `screenshots.forClipboard(paths)` makes the image (several are stacked; see The draft, screenshots and safe files). `null` means a file is missing, and `return false` lets the caller report `missing-screenshot`.
- `new Blob([image.toPNG()], { type: 'image/png' })`: the image encoded as PNG and wrapped in a `Blob` (a block of bytes labelled with its media type), which is what a clipboard item holds.
- `clipboard.write([new ClipboardItem({ 'image/png': png })])`: puts the image on the Windows clipboard, replacing whatever was there. This is Electron 44's clipboard API, modelled on the browser's; Electron 44 removed the older `clipboard.writeImage()`. `write()` returns a promise, hence the `await`.

The chat: built in, or in Claude Desktop

src/main/index.ts, lines 158–187

```
const builtIn =
  chatApp === 'built-in' && signIn && claudeAccess
    ? startBuiltInChat({
        tenant,
        signIn,
        claudeAccess,
        userData,
        settings,
        log,
        broadcast,
        bubble,
      })
    : null

const claudeDesktop =
  chatApp === 'claude-desktop'
    ? new ClaudeDesktop({
        isInstalled: () => app.getApplicationNameForProtocol(NEW_CHAT_LINK) !== '',
        openLink: (url) => shell.openExternal(url),
        copyScreenshots,
        autoSend: () => settings.get().autoSend,
        sendInClaude: async (question, paste) => {
          const outcome = await sendInClaude({ question, paste })
          // Not a failure as such (the user is told to finish in Claude), but worth knowing.
          if (outcome !== 'sent') log(`Sending in Claude Desktop stopped: ${outcome}`)
          return outcome
        },
        notify,
        onError: (error) => log('Handing a question to Claude Desktop failed:', error),
      })
    : null
```

Where questions go is the tenant's `chatApp` (see `CHAT_APPS`). Exactly one of these two is created and the other stays `null`; everything after this point checks which one exists.

- `const { chatApp, signIn, claudeAccess } = tenant` (the line before): the three settings that decide it. `signIn` and `claudeAccess` are optional in the schema.
- `chatApp === 'built-in' && signIn && claudeAccess`: the schema already refuses a built-in chat without `signIn` and `claudeAccess`, but TypeScript can't see that rule (it's a `.refine()`). Checking them here tells it they exist, so they can be passed on as definite values.
- `startBuiltInChat({`: creates the Claude backend, the conversation and the JumpCloud sign-in, and returns the two that the rest of the app talks to (see `startBuiltInChat()`). It isn't `async`, so nothing here waits, as the comment in The two windows requires.
- `new ClaudeDesktop({`: hands questions over to the Claude Desktop app (see Claude Desktop). `ClaudeDesktop` knows nothing about Electron: the functions given to it here do the Electron and Windows work, which is what lets `tests/claudeDesktop.test.ts` replace them with fakes.
- `app.getApplicationNameForProtocol(NEW_CHAT_LINK) !== ''`: asks Windows which app opens `claude://` links. Claude Desktop registers itself for them when it's installed; with no app registered, Electron returns an empty string.
- `openLink: (url) => shell.openExternal(url)`: hands the `claude://` link to Windows, which opens it in Claude Desktop, the same way it opens a web link in the browser.
- `copyScreenshots`: the helper above; `ClaudeDesktop` reports `missing-screenshot` when it returns `false`.

- `autoSend: () => settings.get().autoSend`: the user's "Send in Claude automatically" setting. A function rather than a value, so each question uses the setting as it is at that moment.
- `sendInClaude: async (question, paste) => {`: presses Ctrl+V and Enter in Claude Desktop once it shows the question (`sendInClaude()` in `sendInClaude.ts`, see Claude Desktop). The wrapper logs any outcome but `sent` ("Sending in Claude Desktop stopped: not-ready", say). As the comment says, that isn't a failure as such: the user gets a notification saying what's left to do. But it's worth having in the log if someone reports that questions aren't being sent.
- `notify`: the helper above, to say what's left to do in Claude.
- `onError: (error) => log('Handing a question to Claude Desktop failed:', error)`: if copying or opening fails, the panel shows a short message; if the sending helper can't run, the user gets a notification. Either way the log gets the details.

The Apps list

`src/main/index.ts`, lines 188–213

```
// Electron's fetch, so a proxy set in Windows is used.
const netFetch = (url: string | URL, init?: RequestInit) => net.fetch(String(url), init)

let appsStatus: AppsState['status'] = 'loading'

const portalApps = portal
  ? new PortalApps({
      appName: tenant.appName,
      redirectPort: portal.redirectPort,
      connector: mcpConnector({
        serverUrl: portal.appsServer,
        client: { name: tenant.appName, version: app.getVersion() },
        fetch: netFetch,
      }),
      store: new SecretStore(join(userData, 'jumpcloud-apps.bin'), safeStorageEncryptor),
      listen: (port) => listenForRedirect(port),
      openBrowser: (url) => shell.openExternal(url),
      fetchLogo: (url) => fetchLogo(netFetch, url),
      onState: (state) => {
        broadcast(IPC.appsState, state)
        // Connecting took the user to their browser; bring the list back once it's done.
        if (appsStatus === 'signing-in' && state.status !== 'signing-in') bubble.expand()
        appsStatus = state.status
      },
      onError: (message, error) => log(message, error),
    })
  : null
```

The apps in the user's JumpCloud User Portal, which the panel opens on or shows behind the Apps icon (see Your apps). It's only set up when the tenant has a `portal`; otherwise `portalApps` is `null` and the Apps channels aren't answered.

- `netFetch`: Electron's `net.fetch()`, in the shape of the standard `fetch()`. Electron's version goes through Chromium's network stack, so a proxy set up in Windows (common on company networks) is used, as it is in the browser. Node's own `fetch` would ignore it. `String(url)` because `net.fetch()` takes the address as text.
- `const { portal, serviceDesk } = tenant` (the line before `appsStatus`): the portal and service desk settings from `tenant.json`, each `undefined` if the tenant has none.
- `let appsStatus`: the list's previous status, for the same reason as `authState` in the built-in chat's sign-in: inside `onState`, the new one has already arrived.

- `mcpConnector({ serverUrl: portal.appsServer, ... })`: the real connection to JumpCloud's apps server, through the MCP SDK (`mcp/connector.ts`). `client` is how Desktop Assist introduces itself to the server: its name and version.
- `new SecretStore(join(userData, 'jumpcloud-apps.bin'), safeStorageEncryptor)`: where the connection to the portal (the app's registration and the user's tokens) is kept between runs, encrypted with Windows DPAPI like the built-in chat's sign-in, in a file of its own.
- `listen: (port) => listenForRedirect(port)`, `openBrowser: (url) => shell.openExternal(url)`: the same local web server and browser opening as the built-in chat's JumpCloud sign-in, on the portal's own port (`portal.redirectPort`).
- `fetchLogo: (url) => fetchLogo(netFetch, url)`: fetches an app's logo and turns it into a `data:` URL for the page.
- `broadcast(IPC.appsState, state)`: every change in the list's state goes to the pages.
- `if (appsStatus === 'signing-in' && state.status !== 'signing-in') bubble.expand()`: connecting sent the user to their browser. When it's finished (or failed), the panel opens again, so they see the list (or why not) without clicking the bubble.
- `onError: (message, error) => log(message, error)`: problems loading the list or connecting go to the problem log; the list shows its own short message.
- `void portalApps?.get()` (the next statement): starts loading the list in the background at startup, so it's ready when the panel first opens, which for Morse Micro is on the Apps list. As the comment says, it never opens the browser: without a saved connection it just settles on the Sign in button. `?.` skips it when there's no portal.

Jira and the service desk

`src/main/index.ts`, lines 218–280

```
const requests = serviceDesk?.requests

const memoryPath = join(userData, 'atlassian.json')

let atlassianStatus: AtlassianState['status'] = 'loading'

const atlassian = tenant.atlassian
  ? new Atlassian({
      appName: tenant.appName,
      site: tenant.atlassian.site,
      redirectPort: tenant.atlassian.redirectPort,
      knowledgeSpaces: tenant.atlassian.knowledgeSpaces,
      requests:
        serviceDesk && requests
          ? {
              helpCentre: serviceDesk.url,
              portalId: requests.portalId,
              projectKey: requests.projectKey,
            }
          : null,
      connector: mcpConnector({
        serverUrl: tenant.atlassian.mcpServer,
        client: { name: tenant.appName, version: app.getVersion() },
        fetch: netFetch,
      }),
      store: new SecretStore(join(userData, 'atlassian.bin'), safeStorageEncryptor),
      memory: {
        load: async () => {
          const read = await readJsonFile(memoryPath, AtlassianMemorySchema)
          return read.status === 'ok' ? read.value : {}
        },
```

```
        save: async (memory) => void (await writeJsonFile(memoryPath, memory)),
      },
      listen: (port) => listenForRedirect(port),
      openBrowser: (url) => shell.openExternal(url),
      onState: (state) => {
        broadcast(IPC.atlassianState, state)
        // Connecting took the user to their browser; bring the panel back once it's done.
        if (atlassianStatus === 'signing-in' && state.status !== 'signing-in') bubble.expand()
        atlassianStatus = state.status
      },
      onFeed: (feed) => broadcast(IPC.jiraFeed, feed),
      notify,
      onError: (message, error) => log(message, error),
    })
  : null

const desk =
  serviceDesk && requests
    ? new ServiceDesk({
        helpCentre: serviceDesk.url,
        portalId: requests.portalId,
        requestTypeIds: requests.requestTypes.map((type) => type.id),
        pcInfo: () => ({
          computerName: hostname(),
          windows: windowsVersion(osVersion(), release()),
          appName: tenant.appName,
          appVersion: app.getVersion(),
        }),
        openLink: (url) => shell.openExternal(url),
        copyScreenshots,
        notify,
        onError: (error) => log('Opening the request form failed:', error),
      })
    : null
```

Jira, Confluence and the user's IT requests, through Atlassian's Remote MCP server, and raising IT requests from the panel (see Jira and the service desk). `atlassian` is only set up when the tenant has `atlassian`, and `desk` only when its service desk lists request types (`serviceDesk.requests`); otherwise each is `null` and its channels aren't answered.

- `const requests = serviceDesk?.requests`: the portal, the Jira project and the request types, or `undefined`.
- `memoryPath`: `atlassian.json` in the app's data folder, where `Atlassian` remembers what the user has already seen.
- `let atlassianStatus`: the connection's previous status, as `appsStatus` is for the Apps list.
- `site`, `redirectPort`, `knowledgeSpaces`: from the tenant's `atlassian` settings. Atlassian's sign-in comes back to a port of its own (47623 for Morse Micro), different from the Apps list's and the built-in chat's.
- `requests: serviceDesk && requests ? { helpCentre: serviceDesk.url, ... } : null`: what My requests needs: the help centre's address, the portal's number and the Jira project's key.
- `mcpConnector({ serverUrl: tenant.atlassian.mcpServer, ... })`: the same MCP SDK connection as the Apps list's, to Atlassian's server instead, also through `netFetch`.
- `new SecretStore(join(userData, 'atlassian.bin'), safeStorageEncryptor)`: the connection to Atlassian (the registration and the user's tokens), encrypted with Windows DPAPI, in a file of its own.
- `memory: { load, save }`: reads `atlassian.json` with `readJsonFile()` and the `AtlassianMemorySchema` (see The draft, screenshots and safe files): a missing or damaged file just gives `{}`, starting afresh. `writeJsonFile()` writes it safely.

- `onState: (state) => {`: broadcasts every change of the connection on `IPC.atlassianState`, and, like the Apps list, opens the panel again when connecting in the browser has finished.
- `onFeed: (feed) => broadcast(IPC.jiraFeed, feed)`: every change in the notifications or requests goes to the pages, which put dots on the icons from it.
- `notify`: the Windows notification above, for IT's replies.
- `void atlassian?.init()` (the next statement): as the comment says, uses a saved connection if there is one, and starts checking notifications. Like the Apps list, it never opens the browser.
- `new ServiceDesk({`: raising IT requests. `requestTypeIds` is the list of the request types' IDs, the only ones it accepts.
- `pcInfo: () => ({ ... })`: the PC's details for **Include PC details**: its name (`hostname()`), the Windows version (`windowsVersion(osVersion(), release())`, from Node's `os` module) and the app's name and version. A function, so they're read when they're needed.
- `copyScreenshots`, `notify`: the helpers above, for the screenshot that's pasted on the portal.

Two statements follow:

- `if (!builtIn) {`, `void rm(join(userData, 'jumpcloud-session.bin'), ...)`: with Claude Desktop, a JumpCloud sign-in saved by the built-in chat (which Morse Micro used until 3.1) is no longer needed, so the file is deleted rather than left on disk.
- `void rm(join(userData, 'claude-api-key.bin'), { force: true }).catch(() => {})`: an earlier version saved a Claude API key in this file. Nothing uses it now, so it's deleted. `force: true` means "no error if it isn't there", and `.catch(() => {})` ignores any other failure. The JumpCloud file above is deleted the same way.

IPC, the tray and display changes

`src/main/index.ts`, line 336

```
const onDisplayChange = () => bubble.displayChanged()
```

Everything else in this part of `start()` is plain statements:

- `registerIpc({`: connects the pages' requests to the services (see `registerIpc()`). It's given both windows (only they may call in), every service, and `builtIn` and `claudeDesktop`, one of them `null`, so it only answers the channels of the chat in use.
- `apps: portal && portalApps ? { list: portalApps, portalUrl: portal.url } : null`: the Apps list and the portal's own address (for "Open the JumpCloud portal"), or `null` without a portal. Checking `portal` again lets TypeScript know `portal.url` exists.
- `atlassian`: Jira and Confluence, or `null`.
- `serviceDesk: serviceDesk && desk ? { desk, myRequestsUrl: myRequestsUrl(serviceDesk.url) } : null`: raising requests, and the portal's page of the user's requests (for "All your requests on the portal"), or `null` without request types.
- `uninstall: () => uninstall({`: Settings → Uninstall (see `uninstall()`). Everything it needs from Electron and Node is passed in: where the running `.exe` is (`app.getPath('exe')`), `existsSync` to check the uninstaller is there, turning off Start with Windows, and starting the uninstaller with `spawn(path, [], { detached: true, stdio: 'ignore' }).unref()`. `detached` runs it as a process of its own, `stdio: 'ignore'` connects nothing to it, and `unref()` tells Node not to wait for it, so it carries on after Desktop Assist has quit.
- `actions: createActionHandlers({`: the commands behind the service desk, Screenshot, Bounce and Close icons (`actions.ts`).

- `openServiceDesk: serviceDesk ? () => shell.openExternal(serviceDesk.url) : null`: opens the service desk's address from `tenant.json` in the browser, or `null` when the tenant has none.
- `getState: () => ({`: builds the snapshot a page asks for when it first loads: the bubble's mode and corner, the draft, the settings, the branding, the app version (`app.getVersion()`, from `package.json`), the sign-in status and the chat so far. After that the page keeps up through the broadcasts above.
- `auth: builtIn?.auth.status ?? null`, `chat: builtIn?.chat.list() ?? []`: `?.` reads on only if `builtIn` isn't `null` (otherwise the whole expression is `undefined`), and `??` gives the value to use instead. So with Claude Desktop there's no sign-in status (`null`) and the chat is always empty: the panel shows just the text box.
- `tray = createTray({`: adds the icon in the notification area. `onOpen` opens the panel and `onQuit` quits (see `createTray()`).
- `screen.on('display-added', onDisplayChange)`: the same handler also runs for `display-removed` and `display-metrics-changed` (a resolution, scaling or taskbar change). `BubbleController.displayChanged()` moves the bubble back into its corner, or onto the main display if its own display was unplugged.

Quitting

src/main/index.ts, lines 341–342

```
// Save the text box before quitting. before-quit fires again after the second app.quit().
let shuttingDown = false
```

The `app.on('before-quit', (event) => {` handler that follows makes sure the draft reaches the disk before the app closes. `before-quit` fires when something calls `app.quit()`: the tray's Quit or the Close icon. It doesn't fire when Windows shuts down or logs off, which is what the `session-end` handler at the end is for.

- `if (shuttingDown) return`: the handler ends by calling `app.quit()` again, which fires `before-quit` a second time. This flag lets that second one through.
- `event.preventDefault()`: cancels this first quit, to make time for the steps below.
- `bubble.dispose()`, `builtIn?.chat.stop()`: stops the bubble's animation timers and, with the built-in chat, aborts a reply that is still streaming.
- `void portalApps?.dispose()`: closes the Apps list's connection to JumpCloud, and stops waiting for a browser sign-in if one is under way. `atlassian?.dispose()` does the same for Atlassian, and stops the 5-minute checks.
- `tray?.destroy()`: removes the tray icon now. Otherwise it can linger in the tray until the mouse passes over it.
- `Promise.race([notes.flush(), timeout])`: waits for the draft to be written, or `SHUTDOWN_TIMEOUT_MS` (2 seconds), whichever comes first, so a stuck disk can't stop the app from closing. `.finally(() => app.quit())` then quits for real.
- `panelWindow.on('session-end', () => notes.flushSync())`: when Windows logs off or shuts down, it doesn't wait for asynchronous work, so `session-end` (a Windows-only window event) saves the draft with a blocking write instead.

Finally, `await windowsReady` waits until both pages have painted, and `bubble.start()` puts the bubble in its corner and shows it. Waiting avoids showing a window before its page has drawn anything.

`startBuiltInChat()`

The built-in chat: Claude in the panel, reached through the Claude API with the user's JumpCloud sign-in. `start()` calls it only when the tenant's `chatApp` is `built-in`; with Claude Desktop none of this is created.

Its one parameter, `deps`, holds what it needs from `start()`: the `tenant` (for the system prompt), its `signIn` and `claudeAccess` settings (passed separately because here they're known to exist), the `userData` folder, the `settings` service, the problem `log`, `broadcast` and the `bubble` controller. The first line takes them all out of `deps` again (`const { tenant, signIn, claudeAccess, ... } = deps`), so the code below reads as it did when it was part of `start()`. It returns a `BuiltInChat`: the `AuthManager` and the `ChatSession`, which `registerIpc()` and `getState` use.

Claude

src/main/index.ts, lines 378–402

```
// Dev runs may point at a local test server; an installed app always talks to Anthropic.
const devBaseUrl = !app.isPackaged ? process.env['ANTHROPIC_BASE_URL'] : undefined

let lastIdToken: IdTokenSummary | { error: string } | null = null

const backend = new AnthropicBackend({
  baseURL: devBaseUrl || API_BASE_URL,
  access: claudeAccess,
  identityToken: async () => {
    const token = await auth.freshIdToken()
    lastIdToken = summarizeIdToken(token) // what the token says, never the token itself
    return token
  },
})

const chat = new ChatSession({
  backend,
  canChat: () => auth.canChat,
  getEffort: () => settings.get().effort,
  system: buildSystemPrompt(tenant),
  onMessage: (message) => broadcast(IPC.chatMessage, message),
  onReset: () => broadcast(IPC.chatReset),
  onError: (error) => {
    log('Claude request failed:', error)
    // Anthropic refused to swap the sign-in: log what the token said, to compare with the rule.
    if (classifyError(error) === 'not-allowed') log('The JumpCloud ID token sent:', lastIdToken)
  },
})
```

- `devBaseUrl`: lets a dev run send Claude requests to a local mock server named in `ANTHROPIC_BASE_URL`. An installed build ignores the variable, so it can't be redirected.
- `let lastIdToken`: a summary of the last JumpCloud ID token handed to Anthropic, kept only so it can be logged if Anthropic refuses it. It's `null` until the first message is sent, and `{ error: string }` if the token couldn't be decoded.
- `identityToken: async () => {`: how `AnthropicBackend` gets a JumpCloud ID token to swap for Claude access (see Talking to Claude). It gets a new token from `auth.freshIdToken()` and hands it on unchanged.
- `const token = await auth.freshIdToken()`: `auth` is only declared further down. That's fine because this function only runs later, when the first message is sent, by which time `auth` exists. Calling it any earlier would throw.
- `lastIdToken = summarizeIdToken(token)`: records what the token says (who issued it, its audience, its lifetime, which claims it has) without the token itself, which could be swapped for Claude access. `summarizeIdToken()` is in `auth/idToken.ts` (see JumpCloud sign-in).
- `canChat: () => auth.canChat`, `getEffort: () => settings.get().effort`: functions rather than values, so `ChatSession` reads the current sign-in and the current Fast/Balanced/Thorough choice each time it sends.

- `system: buildSystemPrompt(tenant)`: the system prompt (Claude's standing instructions), built once from the app name, company name and any extra `systemPrompt` in `tenant.json`.
- `onMessage`, `onReset`: each new or changed chat message, and "New conversation", are broadcast to the pages.
- `log('Claude request failed:', error)`: the chat itself shows a plain-English message; the log gets the full error, including the status code and request ID.
- `if (classifyError(error) === 'not-allowed')`: `classifyError()` (in `claude/errors.ts`) sorts errors by what they mean for the user. `not-allowed` means Anthropic refused to swap the JumpCloud token for Claude access. The token summary is then logged too, so it can be compared with the federation rule in the Claude Console.
- This logging was added to track down a real refusal. It had two causes: `tenant.json` held the claude.ai organization ID instead of the Claude Console one, and the federation rule had no expected audience. Without one, Anthropic requires the token's audience (`aud`) to be `https://api.anthropic.com`, which JumpCloud ID tokens never have; the rule's audience must be the JumpCloud client ID.

Sign-in

src/main/index.ts, lines 404–428

```
const missing = missingSettings(signIn, claudeAccess)

// Plain http is allowed only for a test identity provider on this PC, in dev runs.
const localIssuer = !app.isPackaged && isLoopback(new URL(signIn.issuer))

let authState: AuthStatus['state'] = 'checking'

const auth = new AuthManager({
  oidc: missing.length === 0 ? createOidc(signIn, { allowInsecure: localIssuer }) : null,
  missing,
  store: new SecretStore(join(userData, 'jumpcloud-session.bin'), safeStorageEncryptor),
  redirectPort: signIn.redirectPort,
  listen: (port) => listenForRedirect(port),
  openBrowser: (url) => shell.openExternal(url),
  onStatus: (status) => {
    broadcast(IPC.authStatus, status)
    // Signing in took the user to their browser; bring the chat back once it's done.
    if (status.state === 'signed-in' && authState === 'signing-in') bubble.expand()
    authState = status.state
  },
  onSignedOut: (reason) => {
    backend.reset()
    // Logging out clears the chat. If the sign-in expired, it stays, so after signing back in
    // the user can carry on (Retry resends a message that failed).
    if (reason === 'logout') chat.newConversation()
  },
  onError: (error) => log('JumpCloud sign-in failed:', error),
})
```

`AuthManager` owns the JumpCloud sign-in (see JumpCloud sign-in). JumpCloud is an *OpenID Connect* (OIDC) provider: a standard way for an app to sign users in through the company's identity service in the browser.

- `missingSettings(signIn, claudeAccess)`: the IDs still blank in `tenant.json`. If any are missing, `oidc` is `null` and the chat box lists what IT still has to fill in instead of offering to sign in.
- `localIssuer`: OIDC normally requires `https`. A dev run pointed at a test identity provider on this PC (`isLoopback()`) may use plain `http`; `createOidc(..., { allowInsecure })` passes that on to the OIDC library.

- `let authState`: the previous sign-in state. Inside `onStatus`, `auth.status` already holds the new state, so the old one has to be remembered here.
- `new SecretStore(join(userData, 'jumpcloud-session.bin'), safeStorageEncryptor)`: where the sign-in is kept between runs, encrypted with `safeStorageEncryptor`.
- `listen: (port) => listenForRedirect(port)`: starts the small web server on `127.0.0.1` that the browser returns to after sign-in. `openBrowser: (url) => shell.openExternal(url)` opens the JumpCloud page in the user's default browser.
- `if (status.state === 'signed-in' && authState === 'signing-in') bubble.expand()`: signing in sent the user to their browser. When it completes, the panel opens again so they can carry on chatting. Renewing at startup goes from `checking` to `signed-in`, so it doesn't pop the panel open.
- `backend.reset()`: whenever nobody is signed in any more, `AnthropicBackend` drops its Claude client and the Claude token it holds, so nothing from the old sign-in is reused.
- `if (reason === 'logout') chat.newConversation()`: Log out clears the chat. If the sign-in merely expired, the conversation stays so the user can sign back in and press Retry.
- `onError: (error) => log('JumpCloud sign-in failed:', error)`: a failed sign-in or renewal goes to the problem log; the chat box shows its own short message.

Two statements end the function:

- `void auth.init()`: loads the saved sign-in and renews it with JumpCloud. It isn't awaited, so the windows appear while that happens; the result reaches the pages through `onStatus`. Waiting for it would make this function `async`, and `start()` would then let a page's request in before `registerIpc()` has run.
- `return { auth, chat }`: the two services the IPC handlers and `getState` need. The backend stays inside: only `chat` and `onSignedOut` use it.

safeStorageEncryptor

src/main/index.ts, lines 434–439

```
/** Windows DPAPI through Electron: only this Windows user can decrypt what it encrypts. */
const safeStorageEncryptor: Encryptor = {
  isAvailable: () => safeStorage.isEncryptionAvailable(),
  encrypt: (text) => safeStorage.encryptString(text),
  decrypt: (data) => safeStorage.decryptString(data),
}
```

The encryption `SecretStore` uses for the saved sign-in. `SecretStore` only knows the small `Encryptor` interface (so the tests can give it a fake); this object fills it in with Electron's `safeStorage`.

- `safeStorage`: Electron's API for encrypting strings with the operating system's own protection. On Windows that's *DPAPI*, which ties the encrypted data to the signed-in Windows user, so another user of the same PC can't decrypt it.
- `isAvailable`: `safeStorage` only works once the app is ready. Nothing calls these methods before then, because the `SecretStore` is created inside `startBuiltInChat()`, which `start()` calls.

`withDevOverrides()`

src/main/index.ts, lines 441–463

```
/**
 * Dev runs only: where chats happen, and the sign-in and Claude access settings, can be
 * overridden from a JSON file named by DESKTOP_ASSIST_DEV_CONFIG, to test against a local
 * identity provider.
 */
async function withDevOverrides(tenant: Tenant): Promise<Tenant> {
  const path = !app.isPackaged ? process.env['DESKTOP_ASSIST_DEV_CONFIG'] : undefined
  if (!path) return tenant
  const override = DevOverrideSchema.parse(JSON.parse(await readFile(path, 'utf8')))
  return TenantSchema.parse({
    ...tenant,
    chatApp: override.chatApp ?? tenant.chatApp,
    startPage: override.startPage ?? tenant.startPage,
    portal: override.portal ? { ...tenant.portal, ...override.portal } : tenant.portal,
    atlassian: override.atlassian
      ? { ...tenant.atlassian, ...override.atlassian }
      : tenant.atlassian,
    signIn: override.signIn ? { ...tenant.signIn, ...override.signIn } : tenant.signIn,
    claudeAccess: override.claudeAccess
      ? { ...tenant.claudeAccess, ...override.claudeAccess }
      : tenant.claudeAccess,
  })
}
```

Returns the tenant config to use. In an installed build, or when `DESKTOP_ASSIST_DEV_CONFIG` isn't set, that's simply `tenant.json`. In a dev run, the JSON file named by that variable can replace where chats happen (`chatApp`) and some of the sign-in and Claude access settings, for example to try the built-in chat against a test identity provider, without editing `tenant.json`.

- `return tenant`: no override file, so the tenant is used as it is.
- `DevOverrideSchema.parse(JSON.parse(await readFile(path, 'utf8')))`: the file is checked against `DevOverrideSchema`, in which every field is optional.
- `override.chatApp ?? tenant.chatApp`: the override's choice if it has one, otherwise the tenant's.
- `override.signIn ? { ...tenant.signIn, ...override.signIn } : tenant.signIn`: `...` copies an object's properties, and later ones win, so the override's fields replace the tenant's. The merge only happens when the override has a `signIn`; a Claude Desktop tenant without one keeps having none, rather than an empty object. `portal`, `atlassian` and `claudeAccess` work the same way, so a dev run can point the Apps list or Jira at a test server. `startPage` is replaced like `chatApp`.
- `TenantSchema.parse({`: the result is checked again as a whole tenant, so an override can't produce an invalid config. For example, switching to the built-in chat without sign-in settings fails here, and the app shows the error instead of starting.

`isLoopback()`

src/main/index.ts, lines 465–467

```
function isLoopback(url: URL): boolean {
  return ['127.0.0.1', 'localhost', '[::1]'].includes(url.hostname)
}
```

Whether a URL points at this PC. The three names are the IPv4 address, the usual name and the IPv6 address for "this machine"; the URL parser keeps the square brackets around IPv6 addresses in `hostname`. Used only to decide whether plain `http` is allowed for a dev identity provider.

`toArea` and `electronDisplays`

src/main/index.ts, lines 469–485

```
/** Electron's `screen`, in the shape the bubble controller uses. All coordinates are DIPs. */
const toArea = (display: Electron.Display): DisplayArea => ({
  id: display.id,
  bounds: display.bounds,
  workArea: display.workArea,
})

const electronDisplays: Displays = {
  primary: () => toArea(screen.getPrimaryDisplay()),
  all: () => screen.getAllDisplays().map(toArea),
  byId: (id) => {
    const display = screen.getAllDisplays().find((d) => d.id === id)
    return display && toArea(display)
  },
  nearest: (point) =>
    toArea(screen.getDisplayNearestPoint({ x: Math.round(point.x), y: Math.round(point.y) })),
  cursor: () => screen.getCursorScreenPoint(),
}
```

The adapter between Electron's `screen` module and the `Displays` interface `BubbleController` uses (see The bubble). Keeping Electron behind this small interface is what lets `tests/bubbleController.test.ts` run the controller with two fake displays.

- `toArea`: keeps only what the controller needs from a display: its ID, its bounds (the whole screen) and its work area (the screen minus the taskbar).
- `DIPs`: device-independent pixels. On a display scaled to 150%, one DIP is 1.5 physical pixels. Electron's window positions and display sizes are all in DIPs, so the controller never has to think about scaling.
- `screen.getAllDisplays().map(toArea)`: every connected display, for Bounce, which roams across them all.
- `return display && toArea(display)`: if no display has that ID any more (it was unplugged), this returns `undefined` rather than throwing.
- `Math.round(point.x)`: the bubble's position can be fractional while it animates, but this Electron call takes whole numbers.
- `screen.getCursorScreenPoint()`: where the mouse pointer is, used while dragging the bubble.

These are defined at module level, before the app is ready, but `screen` is only touched when a method is called, which happens after `start()` has begun.

`fail()`

src/main/index.ts, lines 487–491

```
function fail(error: unknown): void {
  console.error(error)
  dialog.showErrorBox('Desktop Assist could not start', String(error))
  app.exit(1)
}
```

Shows a plain error box and exits. It's the `.catch()` of `app.whenReady().then(start)`, so it runs for any error during startup: an invalid `tenant.json`, an unreadable dev override file or preferences file, and so on. Errors after `start()` has finished don't come here.

- `dialog.showErrorBox(...)`: a simple, blocking message box. Unlike most dialogs, Electron allows it before the app is ready.
- `app.exit(1)`: quits immediately with exit code 1, closing any windows already created and without firing `before-quit`. There's nothing to save yet, and an app that half-started would otherwise keep running with nothing on screen.

`src/main/log.ts`: the problem log

A small log of problems, written to `%APPDATA%\Desktop Assist\logs\desktop-assist.log` (in a dev run, the `Desktop Assist (Dev)` folder) and printed to the terminal as well. An installed copy has no terminal, so without this file there would be no way to see why something failed on a user's PC, for example why Anthropic refused their sign-in. `start()` in `index.ts` creates the one log and gives it to the error callbacks of `NotesStore`, `ClaudeDesktop`, `PortalApps`, `Atlassian` and `ServiceDesk`, the uninstaller and the bubble controller's notes, or of `ChatSession` and `AuthManager` with the built-in chat. It's never given tokens, only errors and summaries such as the ID token summary.

`MAX_LOG_BYTES` and `Log`

src/main/log.ts, lines 4–7

```ts
/** When the log passes this size it's moved to `<name>.old` and a new one is started. */
const MAX_LOG_BYTES = 1_000_000

export type Log = (message: string, detail?: unknown) => void
```

- `MAX_LOG_BYTES = 1_000_000`: about 1 MB. Past that, the file is renamed to `desktop-assist.log.old` (replacing any older one) and a new log is started, so the two files together stay around 2 MB at most.
- `type Log = (message: string, detail?: unknown) => void`: the type of a function. Callers pass a short message and, optionally, the thing that went wrong (usually an error).

`createLog()`

src/main/log.ts, lines 9–25

```
/**
 * A small log file for problems (and the terminal, in dev runs), so an installed copy, which has
 * no terminal, can still be diagnosed. Never pass it tokens: only errors and summaries.
 */
export function createLog(path: string): Log {
  return (message, detail) => {
    const line = `${new Date().toISOString()} ${message}${detail === undefined ? '' : `
${describe(detail)}`}`
    console.error(line)
    try {
      mkdirSync(dirname(path), { recursive: true })
      if (sizeOf(path) > MAX_LOG_BYTES) renameSync(path, `${path}.old`)
      appendFileSync(path, `${line}\n`)
    } catch {
      // Logging must never break the app.
    }
  }
}
```

Returns the logging function for one file. The returned function remembers `path` (a *closure*), so callers only ever pass the message and detail.

- `new Date().toISOString()`: each line starts with the time, in UTC (for example `2026-10-08T09:15:00.000Z`), then the message, then the detail as described by `describe()`.
- `console.error(line)`: the same line in the terminal, for dev runs.
- `mkdirSync(dirname(path), { recursive: true })`: creates the `logs` folder if it isn't there. The file calls are synchronous (the `...Sync` versions), which keeps this simple; the log is only written when something goes wrong, so the short pause doesn't matter.
- `if (sizeOf(path) > MAX_LOG_BYTES) renameSync(path, ...)`: checked before every line. Renaming onto an existing `.old` file replaces it.
- `catch`: as the comment says, logging must never break the app. If the disk is full or the file is locked, that line is simply lost (it was still printed to the terminal).

`sizeOf()`

src/main/log.ts, lines 27–33

```
function sizeOf(path: string): number {
  try {
    return statSync(path).size
  } catch {
    return 0
  }
}
```

The log file's size in bytes. `statSync` throws if the file doesn't exist yet, which counts as 0.

`describe()`

src/main/log.ts, lines 35–63

```ts
/** An error with the details that matter (status, request ID, response body, cause), or JSON. */
export function describe(detail: unknown): string {
  if (detail instanceof Error) {
    const extra = detail as Error & {
      status?: unknown
      statusCode?: unknown
      requestID?: unknown
      requestId?: unknown
      body?: unknown
      error?: unknown
    }
    const fields = {
      status: extra.status ?? extra.statusCode,
      requestId: extra.requestID ?? extra.requestId,
      // A refused sign-in swap keeps Anthropic's answer in `body`; other API errors in `error`.
      body: extra.body ?? extra.error,
    }
    const known = Object.entries(fields).filter(([, value]) => value != null)
    const parts = [`${detail.name}: ${detail.message}`]
    if (known.length) parts.push(JSON.stringify(Object.fromEntries(known)))
    if (detail.cause !== undefined) parts.push(`(cause: ${describe(detail.cause)})`)
    return parts.join(' ')
  }
  try {
    return JSON.stringify(detail)
  } catch {
    return String(detail)
  }
}
```

Turns the `detail` into one line of text. Errors need special handling: `JSON.stringify` of an `Error` gives `{}`, because its message and name aren't the kind of property JSON includes.

- `detail as Error & {`: tells TypeScript the error may also have these extra fields, so they can be read. `&` combines two types.
- `extra.status ?? extra.statusCode`, `extra.requestID ?? extra.requestId`: libraries name these differently. The Anthropic SDK's API errors have `status` and `requestID`. Its error for a refused token swap has `statusCode`, `requestId` and `body` (Anthropic's response, which says why it refused). Taking whichever exists covers both. The request ID is what Anthropic support needs to find a request.
- `body: extra.body ?? extra.error`: Anthropic's response to the failed request. The refused-swap error keeps it in `body`; the SDK's other API errors keep it in `error` (for example `{"type":"error","error":{"type":"rate_limit_error",…}}`), so either is logged.
- `value != null`: `!=` (rather than `!==`) is false for both `null` and `undefined`, so fields the error doesn't have are left out.
- `JSON.stringify(Object.fromEntries(known))`: the fields found, as JSON, for example `{"status":401,"requestId":"req_…"}`.
- `describe(detail.cause)`: an error can carry the error that caused it in `cause`. The Anthropic SDK wraps sign-in errors this way, so the cause is described too (and its cause, and so on, since `describe()` calls itself).
- `return JSON.stringify(detail)`: anything that isn't an error, such as the ID token summary, is written as JSON. `JSON.stringify` throws for some values (an object that refers to itself, for example), and `String(detail)` is used instead.

`src/main/ipc.ts`: handling requests from the pages

IPC (inter-process communication) is how the pages, which run in separate renderer processes, ask the main process to do things. The preload script gives each page `window.assist`, whose functions send messages on named channels (listed in `src/shared/ipc.ts`; see Shared code and the preload bridge). This file registers the main process's answer to every channel.

Two rules apply to every request: it must come from one of the app's own two windows, and its argument must pass a *zod schema*. zod is a validation library: a schema describes the shape a value must have (a string of at most so many characters, one of three words, and so on), and `schema.parse(value)` returns the value if it matches or throws if it doesn't. The pages are trusted code, but checking at this boundary means a bug or an injected script in a page can't send the main process something unexpected.

Only the channels of the features in use are answered. With the built-in chat, the sign-in and chat channels have handlers; with Claude Desktop, the two `claudeDesktop...` channels do instead. The Apps list's channels are answered only when the tenant has a portal, Jira's only when it has an Atlassian site, and the service desk page's only when it lists request types.

`BuiltInChat`

`src/main/ipc.ts`, lines 34–38

```
/** The built-in chat and the JumpCloud sign-in it needs. */
export interface BuiltInChat {
  auth: AuthManager
  chat: ChatSession
}
```

The two services of the built-in chat that the handlers call: `AuthManager` (the JumpCloud sign-in, see JumpCloud sign-in) and `ChatSession` (the conversation, see Talking to Claude). `startBuiltInChat()` in `index.ts` creates them and returns them in this shape. Keeping them together means there is no state in which one exists without the other.

`IpcContext`

`src/main/ipc.ts`, lines 40–61

```ts
export interface IpcContext {
  /** Only these windows' renderers may call in. */
  windows: BrowserWindow[]
  controller: BubbleController
  notes: NotesStore
  screenshots: ScreenshotService
  settings: SettingsService
  /** The built-in chat and its sign-in, or null when chats happen in Claude Desktop. */
  builtIn: BuiltInChat | null
  /** Hands questions to Claude Desktop, or null when the chat is built in. */
  claudeDesktop: ClaudeDesktop | null
  /** The Apps list and the portal it comes from, or null when the tenant has no portal. */
  apps: { list: PortalApps; portalUrl: string } | null
  /** Jira and Confluence, or null when the tenant has no Atlassian site. */
  atlassian: Atlassian | null
  /** Raising IT requests and the user's requests page, or null without request types. */
  serviceDesk: { desk: ServiceDesk; myRequestsUrl: string } | null
  actions: ActionHandlers
  /** Settings → Uninstall. */
  uninstall(): UninstallResult
  getState(): AppState
}
```

Everything the handlers need, passed in by `start()` in `index.ts`. The main process's services are created there, so this file only needs their types, not how to build them.

- `windows: BrowserWindow[]`: the bubble and panel windows; requests from any other page are refused.
- `builtIn: BuiltInChat | null`, `claudeDesktop: ClaudeDesktop | null`: where questions go. Exactly one of the two is set, depending on the tenant's `chatApp` (see The chat: built in, or in Claude Desktop).
- `apps: { list: PortalApps; portalUrl: string } | null`: the Apps list, and the portal's address for "Open the JumpCloud portal"; `null` when the tenant has no portal.
- `atlassian: Atlassian | null`, `serviceDesk: { desk: ServiceDesk; myRequestsUrl: string } | null`: as their comments say, Jira and Confluence, and raising IT requests with the address of the user's requests on the portal; each `null` when the tenant doesn't use it.
- `uninstall(): UninstallResult`: Settings → Uninstall, ready-wired in `start()`.
- `getState(): AppState`: builds the full snapshot a page asks for when it first loads (defined in `start()`).

`NoArgs`, `FilePath`, `IssueKey`, `TimeSpent`, `ASK_PROBLEMS` and `ExternalUrl`

src/main/ipc.ts, lines 63–90

```
const NoArgs = z.undefined()

const FilePath = z.string().min(1).max(1024)

const IssueKey = z.string().regex(/^[A-Z][A-Z0-9_]{1,9}-\d{1,7}$/)

/** Jira's time format: weeks, days, hours and minutes ("1h 30m"). */
const TimeSpent = z
  .string()
  .trim()
  .regex(/^(?=.*\d)(\d{1,3}w ?)?(\d{1,3}d ?)?(\d{1,3}h ?)?(\d{1,4}m)?$/)

/** What to say when an issue couldn't be handed to Claude Desktop. */
const ASK_PROBLEMS: Record<Extract<AskResult, { ok: false }>['reason'], string> = {
  empty: 'There was nothing to ask.',
  'too-long': 'That issue is too long to open in Claude.',
  'not-installed': "Claude Desktop isn't installed on this PC. Ask IT to install it.",
  'missing-screenshot': 'An attached screenshot is missing.',
  failed: "Couldn't open Claude Desktop. Try again.",
}

/** Only web and email links may be opened from the chat (no file:, javascript: and so on). */
const ExternalUrl = z
  .string()
  .max(4096)
  .refine((value) => {
    try {
      return ['http:', 'https:', 'mailto:'].includes(new URL(value).protocol)
    } catch {
      return false
    }
  })
```

Schemas used by several channels.

- `z.undefined()`: for channels that take no argument. `ipcRenderer.invoke(channel)` with no argument arrives as `undefined`; anything else is rejected.
- `FilePath`: only checks for a sensible string. The real check, that the file is inside the screenshots folder, is done by `ScreenshotService` (`owns()`).
- `IssueKey`: a Jira issue key such as `ISD-1234`, the same pattern as `isIssueKey()` in `src/shared/jira.ts`. Every Jira channel that takes a key checks it.
- `TimeSpent`: as the comment says, Jira's way of writing time spent, such as `30m` or `1h 30m` (explained with `TIME_SPENT` in The Jira and service desk pages). `.trim()` comes before the check, so spaces around it don't matter.
- `ASK_PROBLEMS`: as the comment says, what to say when asking Claude about an issue couldn't be handed to Claude Desktop, one message for each of `AskResult`'s failure reasons.
- `.refine(...)`: adds a custom check to a schema. Here the string must parse as a URL and use `http:`, `https:` (opened in the browser) or `mailto:` (opened in the email app), so a link in a Claude reply can't open a local file (`file:`), run script (`javascript:`) or launch another program through its own URL scheme. `new URL()` throws for text that isn't a URL, hence the `try`/`catch`.

registerIpc()

src/main/ipc.ts, lines 92–323

```ts
/** Wires every renderer request to the main process. All arguments are validated with zod. */
export function registerIpc(ctx: IpcContext): void {
  const trusted = (sender: WebContents) => ctx.windows.some((w) => w.webContents.id === sender.id)

  function handle<S extends z.ZodType, R>(
    channel: string,
    schema: S,
    run: (arg: z.output<S>) => R | Promise<R>,
  ): void {
    ipcMain.handle(channel, (event: IpcMainInvokeEvent, raw: unknown) => {
      if (!trusted(event.sender)) throw new Error(`Untrusted sender for ${channel}`)
      return run(schema.parse(raw))
    })
  }

  function on<S extends z.ZodType>(
    channel: string,
    schema: S,
    run: (arg: z.output<S>, event: IpcMainEvent) => void,
  ): void {
    ipcMain.on(channel, (event: IpcMainEvent, raw: unknown) => {
      if (!trusted(event.sender)) return
      const parsed = schema.safeParse(raw)
      if (parsed.success) run(parsed.data, event)
    })
  }

  handle(IPC.getState, NoArgs, () => ctx.getState())

  on(IPC.bubbleClick, z.number().finite().optional(), (pressedAt) =>
    ctx.controller.clickBubble(pressedAt),
  )
  on(IPC.bubbleDragStart, NoArgs, () => ctx.controller.startDrag())
  on(IPC.bubbleDragEnd, NoArgs, () => ctx.controller.endDrag())
  on(IPC.collapse, NoArgs, () => ctx.controller.collapse())
  on(IPC.setInteractive, z.boolean(), (interactive, event) => {
    BrowserWindow.fromWebContents(event.sender)?.setIgnoreMouseEvents(!interactive, {
      forward: true,
    })
  })

  handle(IPC.invokeAction, z.enum(COMMAND_ACTION_IDS), (id) => ctx.actions[id]())
  handle(IPC.appUninstall, NoArgs, () => ctx.uninstall())
  handle(IPC.openExternal, ExternalUrl, (url) => shell.openExternal(url))
  handle(IPC.copyText, z.string().max(1_000_000), (text) => clipboard.writeText(text))

  on(IPC.notesSetText, z.string().max(MAX_NOTE_LENGTH), (text) => ctx.notes.setText(text))
  // Each click attaches the newest screenshot that isn't attached yet, so clicking again goes one
  // further back.
  handle(IPC.notesAttachLatest, NoArgs, async (): Promise<AttachResult> => {
    const next = await ctx.screenshots.latest((path) => ctx.notes.isAttached(path))
    if (!next) {
      const anyAttached = ctx.notes.get().attachments.length > 0
      return { ok: false, reason: anyAttached ? 'no-more' : 'no-screenshots' }
    }
    const notes = ctx.notes.addAttachment(next)
    return notes ? { ok: true, notes } : { ok: false, reason: 'no-more' }
  })
  handle(IPC.notesRemoveAttachment, z.string().min(1).max(64), (id) =>
    ctx.notes.removeAttachment(id),
```

```
)
handle(IPC.notesClear, NoArgs, () => ctx.notes.clear())

handle(IPC.screenshotThumbnail, FilePath, (path) => ctx.screenshots.thumbnail(path))
handle(IPC.screenshotOpen, FilePath, (path) => ctx.screenshots.open(path))
handle(IPC.screenshotsOpenFolder, NoArgs, () => ctx.screenshots.openFolder())

handle(IPC.settingsGet, NoArgs, () => ctx.settings.get())
handle(IPC.settingsSetAutoStart, z.boolean(), (enabled) => ctx.settings.setAutoStart(enabled))
handle(IPC.settingsSetEffort, z.enum(['low', 'medium', 'high']), (effort) =>
  ctx.settings.setEffort(effort),
)
handle(IPC.settingsSetAutoSend, z.boolean(), (autoSend) => ctx.settings.setAutoSend(autoSend))
handle(IPC.settingsSetTheme, z.enum(['dark', 'light']), (theme) => ctx.settings.setTheme(theme))
handle(
  IPC.settingsSetFavoriteApp,
  z.object({ id: z.string().min(1).max(200), favorite: z.boolean() }),
  ({ id, favorite }) => ctx.settings.setFavoriteApp(id, favorite),
)

if (ctx.claudeDesktop) registerClaudeDesktop(ctx.claudeDesktop)
if (ctx.apps) registerApps(ctx.apps)

function registerApps({ list, portalUrl }: NonNullable<IpcContext['apps']>): void {
  handle(IPC.appsGet, z.boolean(), (refresh) => list.get(refresh))
  // Connecting happens in the browser and can take minutes, so this returns straight away;
  // progress is broadcast on IPC.appsState. No tokens ever reach the renderer.
  handle(IPC.appsSignIn, NoArgs, () => void list.signIn())
  handle(IPC.appsCancelSignIn, NoArgs, () => list.cancelSignIn())
  handle(IPC.appsOpen, z.string().min(1).max(200), async (id) => {
    const opened = await list.open(id)
    // The browser is coming to the front; get out of its way.
    if (opened) ctx.controller.collapse()
    return opened
  })
  handle(IPC.appsOpenPortal, NoArgs, async () => {
    await shell.openExternal(portalUrl)
    ctx.controller.collapse()
  })
}
if (ctx.builtIn) registerBuiltInChat(ctx.builtIn)
if (ctx.atlassian) registerAtlassian(ctx.atlassian)
if (ctx.serviceDesk) registerServiceDesk(ctx.serviceDesk)

function registerAtlassian(atlassian: Atlassian): void {
  handle(IPC.atlassianGet, NoArgs, () => atlassian.current)
  // Connecting happens in the browser and can take minutes, so this returns straight away;
  // progress is broadcast on IPC.atlassianState. No tokens ever reach the renderer.
  handle(IPC.atlassianSignIn, NoArgs, () => void atlassian.signIn())
  handle(IPC.atlassianCancelSignIn, NoArgs, () => atlassian.cancelSignIn())
  handle(IPC.atlassianSignOut, NoArgs, () => atlassian.signOut())
  handle(IPC.atlassianRetry, NoArgs, () => void atlassian.retry())
  handle(IPC.atlassianFeed, z.boolean(), (refresh) => atlassian.feed(refresh))
  handle(IPC.atlassianMarkNotificationsRead, NoArgs, () => atlassian.markNotificationsRead())
  handle(IPC.atlassianMarkRequestsRead, IssueKey.optional(), (key) =>
    atlassian.markRequestsRead(key),
  )
  handle(IPC.atlassianSearch, z.string().max(200), (query) => atlassian.search(query))
  handle(IPC.atlassianIssue, IssueKey, (key) => atlassian.issue(key))
  handle(
    IPC.atlassianTransition,
    z.object({ key: IssueKey, transitionId: z.string().regex(/^\d{1,10}$/) }),
    ({ key, transitionId }) => atlassian.transition(key, transitionId),
  )
  handle(
```

```
    IPC.atlassianComment,
    z.object({ key: IssueKey, body: z.string().trim().min(1).max(10_000) }),
    ({ key, body }) => atlassian.comment(key, body),
  )
  handle(
    IPC.atlassianLogWork,
    z.object({ key: IssueKey, timeSpent: TimeSpent }),
    ({ key, timeSpent }) => atlassian.logWork(key, timeSpent),
  )
  handle(IPC.atlassianArticles, z.string().max(2000), (text) => atlassian.articles(text))
  handle(IPC.atlassianProjectKeys, NoArgs, () => atlassian.projectKeys())
  handle(
    IPC.atlassianAsk,
    z.object({ key: IssueKey, prompt: z.enum(['summarise', 'reply', 'next-steps']) }),
    async ({ key, prompt }): Promise<AskAboutIssueResult> => {
      const built = await atlassian.prompt(key, prompt, MAX_PROMPT_LENGTH)
      if (!built.ok) return built
      if (ctx.claudeDesktop) {
        const result = await ctx.claudeDesktop.ask(built.prompt, [])
        if (!result.ok) return { ok: false, message: ASK_PROBLEMS[result.reason] }
        // Claude Desktop is coming to the front; get out of its way.
        ctx.controller.collapse()
        return { ok: true, inChat: false }
      }
      if (ctx.builtIn) {
        const status = ctx.builtIn.chat.send(built.prompt, [], [])
        if (status === 'sent') return { ok: true, inChat: true }
        return {
          ok: false,
          message:
            status === 'busy'
              ? 'Claude is still answering. Try again in a moment.'
              : 'Sign in with JumpCloud first.',
        }
      }
      return { ok: false, message: "Asking Claude isn't set up." }
    },
  )
}

function registerServiceDesk({ desk, myRequestsUrl }: NonNullable<IpcContext['serviceDesk']>) {
  handle(IPC.serviceDeskPcDetails, NoArgs, () => desk.pcDetails())
  // The request: the form's fields, plus the screenshots attached to the text box.
  handle(
    IPC.serviceDeskRaise,
    z.object({
      requestTypeId: z.string().regex(/^\d{1,10}$/),
      summary: z.string().max(255),
      details: z.string().max(MAX_NOTE_LENGTH),
      includePcDetails: z.boolean(),
    }),
    async (request): Promise<RaiseResult> => {
      const result = await desk.raise(request, ctx.notes.get().attachments)
      // The browser is coming to the front; get out of its way.
      if (result.ok) ctx.controller.collapse()
      return result
    },
  )
  handle(IPC.serviceDeskOpenMyRequests, NoArgs, async () => {
    await shell.openExternal(myRequestsUrl)
    ctx.controller.collapse()
  })
}

function registerClaudeDesktop(claudeDesktop: ClaudeDesktop): void {
```

```
    handle(IPC.claudeDesktopInstalled, NoArgs, () => claudeDesktop.isInstalled())
    // Hands the draft over: the text from the box, plus the screenshots attached to it.
    handle(
      IPC.claudeDesktopAsk,
      z.string().max(MAX_NOTE_LENGTH),
      async (text): Promise<AskResult> => {
        const result = await claudeDesktop.ask(text, ctx.notes.get().attachments)
        if (!result.ok) return result
        // Claude Desktop is coming to the front; get out of its way.
        ctx.controller.collapse()
        return { ...result, notes: ctx.notes.clear() }
      },
    )
  }

  function registerBuiltInChat({ auth, chat }: BuiltInChat): void {
    // Sign-in happens in the browser and can take minutes, so these return straight away;
    // progress is broadcast on IPC.authStatus. No tokens ever reach the renderer.
    handle(IPC.authSignIn, NoArgs, () => void auth.signIn())
    handle(IPC.authCancel, NoArgs, () => auth.cancelSignIn())
    handle(IPC.authSignOut, NoArgs, () => auth.signOut())
    handle(IPC.authRetry, NoArgs, () => void auth.retry())

    // Sends the draft: the text from the box plus the screenshots attached to it.
    handle(IPC.chatSend, z.string().max(MAX_NOTE_LENGTH), (text): SendResult => {
      if (chat.busy) return { ok: false, reason: 'busy' }
      const draft = ctx.notes.get()
      const images: OutgoingImage[] = []
      for (const attachment of draft.attachments) {
        const image = ctx.screenshots.forClaude(attachment.path)
        if (!image) return { ok: false, reason: 'missing-screenshot' }
        images.push(image)
      }
      const status = chat.send(text, images, draft.attachments)
      if (status !== 'sent') return { ok: false, reason: status }
      return { ok: true, notes: ctx.notes.clear() }
    })
    handle(IPC.chatStop, NoArgs, () => chat.stop())
    handle(IPC.chatRetry, NoArgs, () => chat.retry())
    handle(IPC.chatNew, NoArgs, () => chat.newConversation())
  }
}
```

Registers a handler for every channel the app uses, once, at startup. The first part sets up a sender check and two helpers; the rest is one line (or a few) per channel, grouped by area. The channels of the two kinds of chat are registered by two inner functions, `registerClaudeDesktop()` and `registerBuiltInChat()`, and only one of them runs. The Apps list, Jira and the service desk have inner functions of their own too.

The sender check and the two helpers

- `trusted`: whether a message came from one of the app's two windows. `event.sender` is the *WebContents* (Electron's object for the page inside a window) that sent it, and each one has a unique `id`. The windows already refuse to open other pages or navigate away (`bubble/windows.ts`), so this is a second line of defence.
- `function handle<S extends z.ZodType, R>(`: for request-and-reply channels, which the page calls with `ipcRenderer.invoke()` and gets a promise back. `S` and `R` are *type parameters*: `S` is the schema's type and `R` the result's, so each call is type-checked against its schema.
- `z.output<S>`: the TypeScript type a schema produces, so `run` receives a correctly typed argument.
- `throw new Error(...)`, `schema.parse(raw)`: when a handler throws (an untrusted sender, or an argument that fails the schema), Electron rejects the page's promise with the error message.

- `function on<S extends z.ZodType>(`: for one-way messages, which the page sends with `ipcRenderer.send()` and gets nothing back. These are the frequent ones (clicks, every keystroke, the pointer moving). With no reply to carry an error, a bad message is simply dropped (`safeParse` checks without throwing).

The bubble and the windows

- `handle(IPC.getState, ...)`: the snapshot each page loads at startup.
- `IPC.bubbleClick`, `IPC.bubbleDragStart`, `IPC.bubbleDragEnd`, `IPC.collapse`: forwarded to the matching `BubbleController` methods.
- `z.number().finite().optional()`: the bubble click's press time, a real number (not `NaN` or infinity), or nothing.
- `setIgnoreMouseEvents(!interactive, { forward: true })`: click-through for the panel. While the pointer is over a see-through area, the page asks for mouse input to be ignored, so clicks go to the window underneath; `forward: true` keeps sending pointer movement to the page so it can tell when the pointer comes back over real UI. `BrowserWindow.fromWebContents()` finds the window that sent the message.

Actions, links and the clipboard

- `z.enum(COMMAND_ACTION_IDS)`: only the `command` actions (service desk, Screenshot, Bounce, Close) are accepted. `ctx.actions[id]()` runs the matching handler from `actions.ts` and returns its result, which the panel shows as a toast. The Settings icon is a `popover` action that never leaves the page.
- `shell.openExternal(url)`: opens a link from a Claude reply (or the Claude Desktop download page) in the default browser, after `ExternalUrl` has checked it (an email link opens the email app instead).
- `z.string().max(1_000_000)`: the text of a chat message whose Copy button was pressed, capped at a million characters (`_` is just a digit separator). `clipboard.writeText` puts it on the Windows clipboard.

The draft

- `on(IPC.notesSetText, ...)`: sent on every keystroke, so it's one-way. The length cap matches what `NotesStore` keeps (`MAX_NOTE_LENGTH`).
- `IPC.notesAttachLatest`: attaches the newest screenshot in the folder that isn't attached yet (`ctx.notes.isAttached` tells `latest()` which to pass over), so each click goes one screenshot further back. With nothing left it answers `no-more` if some are attached (every screenshot is), or `no-screenshots` if there are none at all; the page shows either as an error.
- `z.string().min(1).max(64)`: an attachment ID (a UUID, 36 characters).
- `IPC.notesClear`: empties the text box and removes its screenshots (Clear text box in the Settings menu, with Claude Desktop). It returns the empty draft, which the panel shows.

Screenshots and settings

- `IPC.screenshotThumbnail`, `IPC.screenshotOpen`, `IPC.screenshotsOpenFolder`: a small preview for a chip, opening a screenshot in the default image viewer, and opening the folder. The two that take a path refuse one outside the screenshots folder.
- `IPC.settingsGet`, `IPC.settingsSetAutoStart`, `IPC.settingsSetEffort`, `IPC.settingsSetAutoSend`: the Settings menu. Each returns the full, current settings, so the menu always shows what actually took effect. (The menu offers the response style only with the built-in chat, and "Send in Claude automatically" only with Claude Desktop.)
- `IPC.settingsSetTheme`, `z.enum(['dark', 'light'])`: the light or dark mode icon.

- `IPC.settingsSetFavoriteApp`: a star in the Apps list. Its schema, `z.object({ id: ..., favorite: z.boolean() })`, checks an object field by field, and `({ id, favorite })` unpacks it.

Uninstall

- `handle(IPC.appUninstall, NoArgs, () => ctx.uninstall())`: Settings → Uninstall. On success the app is already quitting.

Built-in chat, Claude Desktop, the Apps list, Jira and the service desk

- `if (ctx.claudeDesktop) registerClaudeDesktop(ctx.claudeDesktop)`, `if (ctx.builtIn) registerBuiltInChat(ctx.builtIn)`: `start()` sets exactly one of the two, so only that chat's channels get handlers. If a page called one of the other chat's channels anyway, Electron would reject its promise, because nothing is registered for it.
- `if (ctx.apps) registerApps(ctx.apps)`: the same for the Apps list, when there's a portal. `registerAtlassian()` and `registerServiceDesk()` likewise run only when `start()` passed an `atlassian` or a `serviceDesk`.
- `function registerClaudeDesktop(...)`, `function registerBuiltInChat(...)`: declared inside `registerIpc()`, so they can use `handle()` and `ctx`. The other three too. They're called above the lines that declare them, which works because JavaScript sets up a block's `function` declarations before running it (this is called *hoisting*).

Claude Desktop (`registerClaudeDesktop()`)

- `IPC.claudeDesktopInstalled`: whether Claude Desktop is installed, which the panel asks each time it opens.
- `handle(IPC.claudeDesktopAsk, ...)`: hands the draft to Claude Desktop (see `ClaudeDesktop.ask()`): the text from the box plus the screenshots attached to the draft. The length cap is the draft's own; `ask()` applies Claude Desktop's lower limit itself and answers `too-long`.
- `if (!result.ok) return result`: a failure (`empty`, `too-long`, `not-installed`, `missing-screenshot` or `failed`) goes straight back to the panel, which shows a toast. The draft is kept.
- `ctx.controller.collapse()`: on success Claude Desktop comes to the front with the question, so the panel closes to get out of its way. `ask()` returns as soon as Claude is opening, so this happens straight away; sending the question in Claude carries on in the background.
- `{ ...result, notes: ctx.notes.clear() }`: the outcome (including how many screenshots were copied) plus the emptied draft, so the text box clears, as after sending in the built-in chat.

The Apps list (`registerApps()`)

- `({ list, portalUrl }: NonNullable<IpcContext['apps']>)`: `IpcContext['apps']` is the type of that field, and `NonNullable<...>` the same without `null`, since this only runs when there is a portal.
- `handle(IPC.appsGet, z.boolean(), (refresh) => list.get(refresh))`: the list, loading it if needed. The preload always sends `true` or `false`.
- `() => void list.signIn()`: connecting happens in the browser and can take minutes, so, as with the built-in chat's sign-in, the call returns at once and progress arrives as `IPC.appsState` broadcasts. No token ever reaches the page.
- `z.string().min(1).max(200)`: an app's ID from the list.
- `if (opened) ctx.controller.collapse()`: the app is opening in the browser, which comes to the front, so the panel closes. If there was no link, the panel stays and shows a toast.
- `shell.openExternal(portalUrl)`: "Open the JumpCloud portal" opens the portal's address from `tenant.json` (never one sent by the page), then closes the panel.

Jira (`registerAtlassian()`)

- `() => void atlassian.signIn()`: as the comment says, connecting happens in the browser, so the call returns at once and progress arrives as `IPC.atlassianState` broadcasts. Retry works the same way. No token ever reaches the page.
- `z.boolean()`: `feed`'s `refresh`; the preload always sends `true` or `false`.
- `IssueKey.optional()`: marking one request's news as seen, or all of them without a key.
- `z.string().max(200)`, `z.string().max(2000)`: a search, and the words to look for articles about, each capped.
- `z.object({ key: IssueKey, transitionId: z.string().regex(/^\d{1,10}$/) })`: a move: the issue and the transition's ID, which is a number. The comment's body must have something in it and be at most 10,000 characters; the time spent must pass `TimeSpent`.
- `handle(IPC.atlassianAsk, ...)`: **Ask Claude** on an issue. `z.enum(['summarise', 'reply', 'next-steps'])` accepts only the three buttons' requests.
- `atlassian.prompt(key, prompt, MAX_PROMPT_LENGTH)`: builds the question with the issue in it, kept within Claude Desktop's limit for a question in a link (`MAX_PROMPT_LENGTH`, from `claudeDesktop.ts`). If the issue can't be read, its message goes back to the page.
- `ctx.claudeDesktop.ask(built.prompt, [])`: with Claude Desktop, the question is handed over like one typed in the text box, with no screenshots. A failure becomes a message from `ASK_PROBLEMS`; on success the panel closes, as Claude Desktop comes to the front (`inChat: false`).
- `ctx.builtIn.chat.send(built.prompt, [], [])`: with the built-in chat, the question is sent in the panel's conversation, and `inChat: true` tells the page to show the chat. If Claude is still answering, or nobody is signed in, the page is told so.

The service desk (`registerServiceDesk()`)

- `IPC.serviceDeskPcDetails`: what **Include PC details** adds, for the form to show.
- `handle(IPC.serviceDeskRaise, z.object({ ... }), ...)`: the form's fields: a request type's ID (a number), a summary of at most 255 characters (Jira's limit), details no longer than the draft can be, and whether to include the PC's details.
- `desk.raise(request, ctx.notes.get().attachments)`: as the comment says, the screenshots are the ones attached to the text box, taken from the main process's own copy of the draft.
- `if (result.ok) ctx.controller.collapse()`: the portal's form is opening in the browser, which comes to the front, so the panel closes. The draft is kept, unlike after asking Claude: the request isn't sent until the user sends it on the portal.
- `shell.openExternal(myRequestsUrl)`: "All your requests on the portal", then the panel closes.

Sign-in (`registerBuiltInChat()`)

- `({ auth, chat }: BuiltInChat)`: the parameter is unpacked straight away into its two services, so the handlers below can say `auth` and `chat`.
- `() => void auth.signIn()`: signing in happens in the browser and can take minutes. `void` discards the promise, so the page's call returns at once; progress arrives as `IPC.authStatus` broadcasts. Retry works the same way.
- `auth.cancelSignIn()`: stops waiting for the browser (it closes the local listener).
- No handler returns a token: the pages only ever see the status and the user's name.

Chat (`registerBuiltInChat()`)

- `handle(IPC.chatSend, ...)`: sends the text plus the screenshots attached to the draft.
- `if (chat.busy) return { ok: false, reason: 'busy' }`: checked first so the screenshots aren't processed for nothing while a reply is still streaming.
- `ctx.screenshots.forClaude(attachment.path)`: shrinks each attached screenshot and encodes it as JPEG for Claude. If any file has gone missing, nothing is sent and the panel shows a toast.
- `chat.send(text, images, draft.attachments)`: starts the reply and returns straight away (it streams in through `IPC.chatMessage`). Anything but `'sent'` (`busy`, `empty`, `signed-out`) is passed back as the reason.
- `{ ok: true, notes: ctx.notes.clear() }`: on success the draft is emptied, and the empty draft is returned so the text box clears.
- `IPC.chatStop`, `IPC.chatRetry`, `IPC.chatNew`: Stop, Retry and New conversation, passed straight to `ChatSession`.

`src/main/actions.ts`: the action icons' commands

The action icons above the bubble come from a registry in `src/shared/actions.ts`. Each action is either a `popover` (opens UI inside the panel, handled entirely by the page) or a `command` (runs in the main process). This file has the main-process side of the commands.

`ActionHandlers` and `createActionHandlers()`

src/main/actions.ts, lines 6–47

```ts
export type ActionHandlers = Record<CommandActionId, () => Promise<ActionResult>>

/** Main-process handlers for the `command` actions in the registry (src/shared/actions.ts). */
export function createActionHandlers(deps: {
  controller: BubbleController
  screenshots: ScreenshotService
  /** Opens the company's IT service desk in the browser, or null if the tenant has none. */
  openServiceDesk: (() => Promise<void>) | null
  quit: () => void
}): ActionHandlers {
  return {
    async servicedesk() {
      if (!deps.openServiceDesk) return { ok: false, message: 'No service desk is set up' }
      try {
        await deps.openServiceDesk()
      } catch (error) {
        console.error('Opening the service desk failed', error)
        return { ok: false, message: "Couldn't open the service desk" }
      }
      // The browser is coming to the front; get out of its way.
      deps.controller.collapse()
      return { ok: true }
    },
    async screenshot() {
      try {
        await deps.controller.whileHidden(() => deps.screenshots.capture())
        return { ok: true, message: 'Screenshot saved' }
      } catch (error) {
        console.error('Screenshot failed', error)
        return { ok: false, message: "Couldn't take a screenshot" }
      }
    },
    async bounce() {
      deps.controller.startBounce()
      return { ok: true }
    },
    async close() {
      deps.quit()
      return { ok: true }
    },
  }
}
```

`createActionHandlers()` returns one async function per command action. `start()` in `index.ts` creates them, and `registerIpc()` runs one when the page sends `IPC.invokeAction`. The `{ ok, message }` each returns is shown by the panel as a toast.

- `Record<CommandActionId, () => Promise<ActionResult>>`: `Record<K, V>` is a TypeScript type for an object with a `V` under every key in `K`. `CommandActionId` is the list of command actions (`'servicedesk' | 'screenshot' | 'bounce' | 'close'`), so if a new command is added to the registry, this file won't compile until it has a handler.
- `openServiceDesk: (() => Promise<void>) | null`: a function that opens the service desk, or `null` when the tenant has none. The brackets around the function type are needed so that `| null` applies to the whole of it.
- `async servicedesk()`: the IT service desk icon (a headset). The tenant check means it's only shown when there's a service desk, but the type allows `null`, so that case gets a message too. Otherwise the page opens in

the browser, and `deps.controller.collapse()` closes the panel to get out of the browser's way. A failure to open becomes a toast; the details go to the terminal.

- `deps.controller.whileHidden(() => deps.screenshots.capture())`: hides both windows, waits for Windows to repaint the area under them, takes the screenshot, then brings the panel back. This way the app isn't in its own screenshot.
- `catch (error)`: any failure (including `whileHidden()` refusing because the panel isn't open) becomes a friendly message; the details go to the log.
- `async bounce()`: starts the bubble bouncing around the screen. It's `async` only so that it fits the type: every handler returns a promise.
- `deps.quit()`: `app.quit()`, which goes through the `before-quit` handler in `index.ts`, so the draft is saved first.

`src/main/uninstall.ts`: uninstalling

Settings → **Uninstall Desktop Assist** removes the app from the PC without a trip to Windows Settings. The installer (electron-builder's NSIS installer, see `electron-builder.cjs`) puts an uninstaller next to the app's `.exe`; this file finds it, starts it, and quits so it can delete the app's files. Everything that touches Windows is passed in, so `tests/uninstall.test.ts` checks it with fakes. `start()` in `index.ts` wires it up (see IPC, the tray and display changes), and the settings menu calls it through `IPC.appUninstall`.

`UninstallDeps` and `uninstallerPath()`

src/main/uninstall.ts, lines 4–23

```ts
export interface UninstallDeps {
  /** Only an installed copy has an uninstaller; `npm run dev` doesn't. */
  isPackaged: boolean
  /** The running .exe, which the uninstaller sits next to. */
  exePath: string
  /** The installer's product name: the uninstaller is "Uninstall <productName>.exe". */
  productName: string
  exists(path: string): boolean
  /** Turns "Start with Windows" off, so no startup entry is left pointing at a deleted .exe. */
  stopStartingWithWindows(): void
  /** Starts the uninstaller on its own, so it carries on once this app has quit. */
  runDetached(path: string): void
  quit(): void
  onError?(error: unknown): void
}

/** Where electron-builder's installer puts the uninstaller: next to the app's .exe. */
export function uninstallerPath(exePath: string, productName: string): string {
  return join(dirname(exePath), `Uninstall ${productName}.exe`)
}
```

- `isPackaged`: a dev run has no uninstaller, so there's nothing to do.
- `exePath`: the running `Desktop Assist.exe`, whose folder holds the uninstaller.
- `stopStartingWithWindows()`: as the comment says, so Windows isn't left with a startup entry for an `.exe` that no longer exists.
- `runDetached(path)`: starts the uninstaller as a separate process that keeps running after this one quits.

- `uninstallerPath()`: `join(dirname(exePath), ...)`, the `.exe`'s folder plus `Uninstall Desktop Assist.exe`, the name the installer gives it.

`uninstall()`

src/main/uninstall.ts, lines 25–44

```ts
/**
 * Removes Desktop Assist from this PC: starts its uninstaller (which shows its own progress and
 * takes the app, its Start menu shortcut and its uninstall entry away), then quits so the files
 * can be deleted. The data folder (%APPDATA%\Desktop Assist: preferences, the text box, logs) and
 * the screenshots in Pictures are kept, as with any uninstall from Windows Settings.
 */
export function uninstall(deps: UninstallDeps): UninstallResult {
  if (!deps.isPackaged) return { ok: false, reason: 'not-installed' }
  const uninstaller = uninstallerPath(deps.exePath, deps.productName)
  if (!deps.exists(uninstaller)) return { ok: false, reason: 'missing' }
  try {
    deps.stopStartingWithWindows()
    deps.runDetached(uninstaller)
  } catch (error) {
    deps.onError?.(error)
    return { ok: false, reason: 'failed' }
  }
  deps.quit()
  return { ok: true }
}
```

What Settings → Uninstall runs. As the comment says, the uninstaller does the removing, with its own progress window, and the data folder and the screenshots are kept, as they would be after uninstalling from Windows Settings.

- `if (!deps.isPackaged) return { ok: false, reason: 'not-installed' }`: the menu item is greyed out in a dev run anyway; this is a second guard.
- `if (!deps.exists(uninstaller))`: no uninstaller next to the `.exe` (a copy that wasn't installed with the installer, say). The panel suggests Windows Settings → Apps instead.
- `deps.stopStartingWithWindows()`, `deps.runDetached(uninstaller)`: in that order, inside a `try`, so a failure to start the uninstaller is logged and reported (`failed`) and the app keeps running.
- `deps.quit()`: the app quits (through `before-quit`, which saves the draft), so the uninstaller can delete its files. The uninstaller is already running by then.

`src/main/tenant.ts`: the tenant config

A *tenant* is one company's branding and settings, from `tenants/<id>/tenant.json`. This file defines the shape of that file as zod schemas, plus two small helpers. `start()` in `index.ts` checks the bundled config with `TenantSchema.parse()`, and `tests/tenants.test.ts` checks every tenant folder the same way, so a mistake is caught by the tests before it reaches a build.

Besides the branding, a tenant chooses where questions go (`chatApp`): to the Claude Desktop app, or to the chat built into the panel. Only the built-in chat needs the sign-in settings (`signIn`) and the Claude access settings (`claudeAccess`). A tenant with the IT service desk icon needs the service desk's address (`serviceDesk`), and its

request types to raise requests from the panel (`serviceDesk.requests`). A tenant with the Apps list (as its start page, or behind the Apps icon) also needs its app portal's settings (`portal`), and one with the Jira icon its Atlassian site (`atlassian`).

None of the values here are secret. The client ID and the Claude IDs identify the app; on their own they don't grant access to anything.

`SignInSchema`

src/main/tenant.ts, lines 5–14

```
/**
 * How users sign in: the company's OpenID Connect app (JumpCloud for Morse Micro). The client ID
 * isn't secret; it identifies the app to JumpCloud. Left empty until IT creates the app.
 */
export const SignInSchema = z.object({
  issuer: z.url(),
  clientId: z.string(),
  /** The browser comes back to http://127.0.0.1:<port>/callback after sign-in. */
  redirectPort: z.number().int().min(1024).max(65535),
})
```

How the built-in chat signs users in with JumpCloud (see JumpCloud sign-in).

- `issuer: z.url()`: the identity provider's address. The OIDC library reads the provider's endpoints from it.
- `clientId: z.string()`: allowed to be empty, so a tenant can ship before IT has created the JumpCloud app; `missingSettings()` reports it instead.
- `redirectPort`: the port of the local web server the browser returns to after sign-in. It must match the redirect address registered in JumpCloud. Ports below 1024 are reserved for system services, hence the minimum.

`ClaudeAccessSchema`

src/main/tenant.ts, lines 16–27

```
/**
 * How a signed-in user reaches Claude: Anthropic Workload Identity Federation swaps the user's
 * JumpCloud ID token for a short-lived Claude API token, acting as a service account. None of
 * these IDs are secret. Left empty until set up in the Claude Console.
 */
export const ClaudeAccessSchema = z.object({
  organizationId: z.string(),
  federationRuleId: z.string(),
  serviceAccountId: z.string(),
  /** Only needed when the federation rule covers more than one workspace. */
  workspaceId: z.string().optional(),
})
```

How a signed-in user reaches Claude in the built-in chat. There's no API key: Anthropic's *Workload Identity Federation* accepts the user's JumpCloud ID token and returns a short-lived Claude token for the company's service account, if the federation rule set up in the Claude Console allows it (see Talking to Claude and `docs/JUMPCLOUD_SETUP.md`). These are the IDs that exchange needs. `organizationId` must be the Claude Console organization's ID: the claude.ai organization has a different one, and using it makes Anthropic refuse every sign-in.

`PortalSchema` and `PortalConfig`

src/main/tenant.ts, lines 29–45

```
/**
 * The company's app portal, for the Apps list: JumpCloud's User Portal. `appsServer` is
 * JumpCloud's "MCP Server for Users", which lists the apps a signed-in user can see in the portal
 * and gives each one's sign-in link; each user connects to it once, in the browser (no secret in
 * the app). An admin turns it on in the JumpCloud Admin Portal (Settings → JumpCloud AI).
 */
export const PortalSchema = z.object({
  /** Shown to users: "Sign in with JumpCloud", "Open the JumpCloud portal". */
  name: z.string().min(1),
  /** The User Portal itself, opened by "Open the … portal". */
  url: z.url({ protocol: /^https$/ }),
  appsServer: z.url({ protocol: /^https?$/ }),
  /** The browser comes back to http://127.0.0.1:<port>/callback after connecting. */
  redirectPort: z.number().int().min(1024).max(65535),
})

export type PortalConfig = z.infer<typeof PortalSchema>
```

The company's app portal, for the Apps list (see Your apps). For Morse Micro it's JumpCloud's User Portal, and the list comes from JumpCloud's "MCP Server for Users", which an admin has to turn on (the comment says where).

- `name`: shown to users, in "Sign in with JumpCloud" and "Open the JumpCloud portal".
- `url: z.url({ protocol: /^https$/ })`: the User Portal itself. Only `https` addresses are accepted.
- `appsServer`: the MCP server's address (`https://usermcp.jumpcloud.com/v1`). `http` is allowed too, for a test server in a dev run.
- `redirectPort`: the port of the local web server the browser returns to after connecting, like `SignInSchema`'s. It's a different port (47622, against the sign-in's 47621), so the two can never get in each other's way.
- `PortalConfig`: the type derived from the schema.

`ServiceDeskSchema` and `ServiceDeskConfig`

src/main/tenant.ts, lines 47–72

```ts
/**
 * The company's IT service desk, opened by the `servicedesk` icon. With `requests`, the icon opens
 * a page for raising requests (filled in on the panel, finished on the help centre portal) and, with
 * `atlassian` too, for following the user's own requests.
 */
export const ServiceDeskSchema = z.object({
  /** The help centre (all portals). */
  url: z.url({ protocol: /^https$/ }),
  requests: z
    .object({
      /** The IT portal's number, from its address: …/servicedesk/customer/portal/<id>. */
      portalId: z.string().regex(/^\d+$/),
      /** The Jira project behind the portal, for My requests. */
      projectKey: z.string().regex(/^[A-Z][A-Z0-9_]+$/),
      /**
       * The request types offered, the first one for "Report a problem". Each ID is the number at
       * the end of the type's form address: …/portal/<portalId>/create/<id>.
       */
      requestTypes: z
        .array(z.object({ id: z.string().regex(/^\d+$/), name: z.string().min(1) }))
        .min(1),
    })
    .optional(),
})

export type ServiceDeskConfig = z.infer<typeof ServiceDeskSchema>
```

The company's IT service desk, opened by the `servicedesk` icon: its address, `https` only. For Morse Micro it's the Atlassian help centre.

- `requests`: optional. With it, the icon opens the service desk page in the panel instead, to raise requests there and see the user's own (see Jira and the service desk).
- `portalId: z.string().regex(/^\d+$/)`: the IT portal's number, as the comment says, from its address. Digits only, since it goes into addresses.
- `projectKey`: the Jira project the portal's requests are kept in (`ISD` at Morse Micro), for My requests: a capital letter, then capitals, digits or `_`.
- `requestTypes`: the kinds of request offered, as the comment says, each with the number from its form's address and its name, in the order the form lists them. `.min(1)`: at least one.
- `ServiceDeskConfig`: the TypeScript type, worked out from the schema.

`AtlassianFields`, `AtlassianSchema` and `AtlassianConfig`

src/main/tenant.ts, lines 74–94

```
/**
 * The company's Atlassian site, for the Jira page (notifications, search, issues) and My requests.
 * Each user connects once, in the browser, through Atlassian's Remote MCP server (no secret in the
 * app); at Morse Micro that sign-in goes through JumpCloud.
 */
const AtlassianFields = z.object({
  /** The site, e.g. https://morsemicro.atlassian.net. */
  site: z.url({ protocol: /^https$/ }),
  mcpServer: z.url({ protocol: /^https?$/ }),
  /** The browser comes back to http://127.0.0.1:<port>/callback after connecting. */
  redirectPort: z.number().int().min(1024).max(65535),
  /** Confluence spaces (their keys) searched for how-to articles while raising an IT request. */
  knowledgeSpaces: z.array(z.string().regex(/^[A-Za-z0-9~_]+$/)),
})

export const AtlassianSchema = AtlassianFields.extend({
  mcpServer: AtlassianFields.shape.mcpServer.default('https://mcp.atlassian.com/v1/mcp'),
  knowledgeSpaces: AtlassianFields.shape.knowledgeSpaces.default([]),
})

export type AtlassianConfig = z.infer<typeof AtlassianSchema>
```

The company's Atlassian site, for the Jira page and My requests. As the comment says, each user connects once in the browser, so there's no secret here.

- `site`: the site's address, `https` only (`https://morsemicro.atlassian.net`).
- `mcpServer`: Atlassian's Remote MCP server. `http` is allowed as well as `https`, so a dev run can point it at a test server on the PC.
- `redirectPort`: the port the browser comes back to after connecting (47623 for Morse Micro). Ports below 1024 are reserved on many systems, hence `.min(1024)`.
- `knowledgeSpaces`: as the comment says, the Confluence spaces searched for how-to articles while raising a request (`INFRA` at Morse Micro), by their keys.
- `AtlassianSchema = AtlassianFields.extend({ ... })`: the same fields, with defaults for two of them: `.extend()` replaces those fields, and `AtlassianFields.shape.mcpServer` is the original field's schema, to add `.default(...)` to. A `tenant.json` can leave out the MCP server (Atlassian's own is used) and the knowledge spaces (none). The fields are defined separately so the dev override can use them without the defaults (see `DevOverrideSchema`).
- `AtlassianConfig`: the TypeScript type, with `mcpServer` and `knowledgeSpaces` always there.

`SignInConfig` and `ClaudeAccessConfig`

src/main/tenant.ts, lines 96–97

```
export type SignInConfig = z.infer<typeof SignInSchema>

export type ClaudeAccessConfig = z.infer<typeof ClaudeAccessSchema>
```

The TypeScript types of the two settings blocks. `z.infer<typeof SignInSchema>` derives the type from the schema, so the compile-time type and the run-time check can't drift apart. `oidc.ts` and `AnthropicBackend.ts` use these types.

CHAT_APPS

src/main/tenant.ts, lines 99–107

```
/**
 * Where conversations with Claude happen.
 * - `claude-desktop`: Desktop Assist hands the question to the Claude Desktop app, so it uses the
 *   person's own Claude (Team/Enterprise) account and counts against their own usage limit. No
 *   sign-in in Desktop Assist.
 * - `built-in`: the chat runs in the panel, through the Claude API, after a JumpCloud sign-in
 *   (`signIn` and `claudeAccess`). Usage is billed to the company's Claude Console account.
 */
export const CHAT_APPS = ['built-in', 'claude-desktop'] as const satisfies ChatApp[]
```

The two places a question can go, as the comment explains. With `claude-desktop` the app has no sign-in and sends nothing to Anthropic itself: it opens the question in the Claude Desktop app (see Claude Desktop). With `built-in` the panel is a chat of its own, billed to the company.

- `as const`: makes the list's type the two exact strings, read-only, rather than `string[]`. That's what `z.enum()` needs to produce the `'built-in' | 'claude-desktop'` type.
- `satisfies ChatApp[]`: checks every entry is a `ChatApp` (the type in `src/shared/types.ts` that the pages see) without changing the list's own type. A misspelt entry is a compile error.

START_PAGES

src/main/tenant.ts, lines 109–113

```
/**
 * What the panel opens on: the text box for asking Claude (`ask`), or the Apps list (`apps`). The
 * other one is behind its icon (`ask` or `apps` in `actions`).
 */
export const START_PAGES = ['ask', 'apps'] as const satisfies StartPage[]
```

What the panel opens on, as the comment says. Written the same way as `CHAT_APPS`: `as const` for `z.enum()`, and `satisfies StartPage[]` to check it against the `StartPage` type the pages use. Morse Micro opens on the Apps list, with the text box behind the Ask Claude icon.

TenantSchema and Tenant

src/main/tenant.ts, lines 115–161

```
export const TenantSchema = z
  .object({
    id: z.string().regex(/^[a-z0-9-]+$/),
    companyName: z.string().min(1),
    appName: z.string().min(1),
    accentColor: z.string().regex(/^#[0-9a-fA-F]{6}$/, 'must be a #RRGGBB colour'),
    actions: z
      .array(z.enum(ACTION_IDS))
      .min(1)
      .refine((ids) => new Set(ids).size === ids.length, 'actions must not repeat'),
    chatApp: z.enum(CHAT_APPS).default('built-in'),
    startPage: z.enum(START_PAGES).default('ask'),
    /** Extra instructions for Claude, added to the built-in chat's system prompt. */
    systemPrompt: z.string().max(8000).optional(),
    /** Needed for the Apps action. */
    portal: PortalSchema.optional(),
    /** Needed for the service desk icon. */
```

```
    serviceDesk: ServiceDeskSchema.optional(),
    /** Needed for the Jira icon. */
    atlassian: AtlassianSchema.optional(),
    /** Needed for the built-in chat only. */
    signIn: SignInSchema.optional(),
    claudeAccess: ClaudeAccessSchema.optional(),
  })
  .refine(
    (tenant) => tenant.chatApp !== 'built-in' || Boolean(tenant.signIn && tenant.claudeAccess),
    {
      message: 'the built-in chat needs signIn and claudeAccess',
    },
  )
  .refine((tenant) => !tenant.actions.includes('apps') || tenant.portal !== undefined, {
    message: 'the apps action needs portal',
  })
  .refine((tenant) => !tenant.actions.includes('servicedesk') || tenant.serviceDesk !== undefined, {
    message: 'the servicedesk action needs serviceDesk',
  })
  .refine((tenant) => !tenant.actions.includes('jira') || tenant.atlassian !== undefined, {
    message: 'the jira action needs atlassian',
  })
  .refine((tenant) => tenant.startPage !== 'apps' || tenant.portal !== undefined, {
    message: 'starting on the apps list needs portal',
  })
  .refine((tenant) => tenant.startPage !== 'apps' || tenant.actions.includes('ask'), {
    message: 'starting on the apps list needs the ask action, to reach the text box',
  })

export type Tenant = z.infer<typeof TenantSchema>
```

The whole `tenant.json`.

- `id: z.string().regex(/^[a-z0-9-]+$/)`: lowercase letters, digits and hyphens only. It must also match the tenant's folder name; the tenants test checks that.
- `accentColor`: a `#RRGGBB` colour. The second argument of `regex()` is the error message.
- `z.array(z.enum(ACTION_IDS)).min(1)`: which icons appear above the bubble, in order. Each must be a known action, and there must be at least one.
- `.refine((ids) => new Set(ids).size === ids.length, ...)`: a `Set` drops duplicates, so if it's smaller than the list, an action is listed twice.
- `chatApp: z.enum(CHAT_APPS).default('built-in')`: where questions go. A `tenant.json` without it gets the built-in chat. Because of `.default()`, `chatApp` is always there in the `Tenant` type, so the code never has to handle it missing.
- `systemPrompt`: optional extra instructions for Claude, added after the built-in ones by `buildSystemPrompt()`. Only the built-in chat uses it; Claude Desktop has its own (claude.ai's organization instructions).
- `startPage: z.enum(START_PAGES).default('ask')`: what the panel opens on; the text box unless the tenant says otherwise.
- `portal: PortalSchema.optional()`: only needed for the Apps list.
- `serviceDesk: ServiceDeskSchema.optional()`: only needed for the service desk icon.
- `atlassian: AtlassianSchema.optional()`: only needed for the Jira icon (and for My requests on the service desk page).
- `signIn: SignInSchema.optional()`, `claudeAccess: ClaudeAccessSchema.optional()`: a Claude Desktop tenant can leave both out.

- `.refine((tenant) => tenant.chatApp !== 'built-in' || ...)`: a check on the whole object, so it can compare fields: the built-in chat must have both `signIn` and `claudeAccess`. `||` reads "either it isn't the built-in chat, or both are there". `Boolean(...)` turns the `&&` of the two objects into `true` or `false`. If the check fails, parsing throws with `message`.
- `.refine((tenant) => !tenant.actions.includes('apps') || tenant.portal !== undefined, ...)`: a second check of the same kind: a tenant that shows the Apps icon must say which portal it's for.
- `.refine((tenant) => !tenant.actions.includes('servicedesk') || ...)`: likewise, the service desk icon needs the service desk's address, and the Jira icon (`'jira'`) needs `atlassian`.
- `.refine((tenant) => tenant.startPage !== 'apps' || tenant.portal !== undefined, ...)`: so must a tenant that starts on the Apps list.
- `.refine((tenant) => tenant.startPage !== 'apps' || tenant.actions.includes('ask'), ...)`: and it must show the Ask Claude icon, or there would be no way to reach the text box.

`brandingOf()`

`src/main/tenant.ts`, lines 163–178

```
export function brandingOf(tenant: Tenant): Branding {
  const { companyName, appName, accentColor, actions, chatApp } = tenant
  const portalName = tenant.portal?.name ?? null
  const { startPage } = tenant
  return {
    companyName,
    appName,
    accentColor,
    actions,
    chatApp,
    portalName,
    startPage,
    atlassianSite: tenant.atlassian?.site ?? null,
    requestTypes: tenant.serviceDesk?.requests?.requestTypes ?? null,
  }
}
```

The part of the tenant the pages need: names, the accent colour, the action list, `chatApp`, which tells the panel whether to show the built-in chat or hand questions to Claude Desktop, and the portal's name for the Apps list (`null` without a portal; `?.` reads `name` only if there's a `portal`), the start page, the Atlassian site (`null` without one), and the service desk's request types (`null` without them). It goes into the state snapshot from `getState`. The object is rebuilt field by field rather than passed as is, so the sign-in, Claude, portal and MCP server settings never go to the pages.

`missingSettings()`

src/main/tenant.ts, lines 180–191

```ts
/** The tenant.json settings still to be filled in before anyone can sign in, by name. */
export function missingSettings(signIn: SignInConfig, access: ClaudeAccessConfig): string[] {
  const required: Record<string, string> = {
    'signIn.clientId': signIn.clientId,
    'claudeAccess.organizationId': access.organizationId,
    'claudeAccess.federationRuleId': access.federationRuleId,
    'claudeAccess.serviceAccountId': access.serviceAccountId,
  }
  return Object.entries(required)
    .filter(([, value]) => !value.trim())
    .map(([name]) => name)
}
```

Lists the required IDs that are still blank, by their name in `tenant.json` (for example `signIn.clientId`). `startBuiltInChat()` passes the list to `AuthManager`; if it isn't empty, the status is `unconfigured` and the chat box shows the list instead of a sign-in button.

- `required`: only the IDs a tenant can leave blank. `issuer` and `redirectPort` aren't here because the schema already requires them, and `workspaceId` is optional.
- `!value.trim()`: a value of only spaces counts as blank.
- `.filter(([, value]) => ...)`, `.map(([name]) => name)`: `Object.entries` gives `[name, value]` pairs; the filter keeps the blank ones and the map keeps just their names.

`DevOverrideSchema`

src/main/tenant.ts, lines 193–206

```ts
/**
 * Dev runs only: a JSON file (path in DESKTOP_ASSIST_DEV_CONFIG) can override `chatApp`,
 * `startPage`, `portal`, `atlassian`, `signIn` and `claudeAccess`, so the app can be pointed at
 * test servers without editing tenant.json. Installed builds never read it.
 */
export const DevOverrideSchema = z.object({
  chatApp: z.enum(CHAT_APPS).optional(),
  startPage: z.enum(START_PAGES).optional(),
  portal: PortalSchema.partial().optional(),
  // Without the defaults, so a partial override can't reset what tenant.json sets.
  atlassian: AtlassianFields.partial().optional(),
  signIn: SignInSchema.partial().optional(),
  claudeAccess: ClaudeAccessSchema.partial().optional(),
})
```

The shape of the dev-only override file read by `withDevOverrides()`. `.partial()` makes every field of a schema optional, so the file only needs the settings it changes.

- `chatApp: z.enum(CHAT_APPS).optional()`: lets a dev run try the other kind of chat, for example the built-in chat against a test identity provider while `tenant.json` says `claude-desktop`.
- `startPage: z.enum(START_PAGES).optional()`: lets a dev run try the other start page.
- `portal: PortalSchema.partial().optional()`: lets a dev run point the Apps list at a test server (`appsServer`), without editing `tenant.json`.
- `atlassian: AtlassianFields.partial().optional()`: the same for Atlassian, such as a test MCP server. As the comment says, it's built from the fields without the defaults: otherwise an override that only gives

`mcpServer` would also get `knowledgeSpaces: []`, and the merge in `withDevOverrides()` would wipe out the tenant's spaces.

`src/main/settings.ts`: the user's preferences

`SettingsService` keeps the user's preferences in `preferences.json` in the userData folder: the reply effort (Fast, Balanced, Thorough) for the built-in chat, whether questions are sent in Claude Desktop automatically, the apps starred in the Apps list, light or dark mode, where the bubble was dragged to, and whether "Start with Windows" has been set up. It also reports the settings the Settings menu shows. Created in `start()`; the Settings menu reaches it through the `settings...` IPC channels.

`DEFAULT_EFFORT`, `PreferencesSchema` and `Preferences`

`src/main/settings.ts`, lines 7–27

```ts
/** Fast answers by default; Balanced and Thorough think longer. */
export const DEFAULT_EFFORT: Effort = 'low'

const PreferencesSchema = z.object({
  autoStartInitialized: z.boolean().optional(),
  effort: z.enum(['low', 'medium', 'high']).optional(),
  /** Claude Desktop: send the question in Claude, not just fill it in. On unless turned off. */
  autoSend: z.boolean().optional(),
  /** Light or dark mode; dark until the user switches. */
  theme: z.enum(['dark', 'light']).optional(),
  /** Apps starred in the Apps list (their IDs), shown first. */
  favoriteApps: z.array(z.string().min(1).max(200)).max(500).optional(),
  /** Where the bubble was dragged to: a corner of a particular display. */
  bubbleAnchor: z
    .object({
      displayId: z.number(),
      corner: z.enum(['top-left', 'top-right', 'bottom-left', 'bottom-right']),
    })
    .optional(),
})

type Preferences = z.infer<typeof PreferencesSchema>
```

- `DEFAULT_EFFORT`: the effort until the user picks one. `low` is shown as Fast.
- `PreferencesSchema`: what the file may contain. Every field is optional, so a missing field just means "not set yet", and an older file still loads after a new preference is added.
- `theme`: light or dark mode; as the comment says, dark until the user switches.
- `favoriteApps`: the IDs of the starred apps. `.max(500)` keeps a damaged file from holding an endless list.
- `autoSend`: "Send in Claude automatically", for Claude Desktop. As the comment says, it's on unless the user has turned it off: a missing value counts as on (see `get()`), so it's on for everyone who installed before the setting existed too.
- `bubbleAnchor`: a display ID and one of the four corners (`BubbleAnchor` in `BubbleController.ts`).
- `type Preferences = z.infer<typeof PreferencesSchema>`: the type derived from the schema.

`SettingsService`

`src/main/settings.ts`, lines 30–36

```
private prefs: Preferences = {}

private saving: Promise<unknown> = Promise.resolve()

constructor(
  private readonly preferencesPath: string,
  private readonly screenshotsDir: string,
) {}
```

- `private prefs: Preferences = {}`: the preferences in memory. Reads come from here; every change is also written to the file.
- `private saving: Promise<unknown> = Promise.resolve()`: the most recent write to the file, so the next one can wait for it (see `save()`). It starts as an already-finished promise, so the first write doesn't wait for anything.
- `private readonly preferencesPath: string`: a TypeScript shorthand. A parameter marked `private` (or `public`, `readonly`) in a constructor is also stored as a field of the same name, which is why the constructor body is empty.
- `screenshotsDir`: not a preference; it's included in the settings so the menu can show where screenshots go.

`SettingsService.init()`

`src/main/settings.ts`, lines 38–48

```
/** Loads preferences. The first time an installed build runs, turns on "Start with Windows". */
async init(): Promise<void> {
  const result = await readJsonFile(this.preferencesPath, PreferencesSchema)
  this.prefs = result.status === 'ok' ? result.value : {}
  // Before any window shows, so it never flashes the other mode.
  nativeTheme.themeSource = this.get().theme
  if (app.isPackaged && !this.prefs.autoStartInitialized) {
    this.setAutoStart(true)
    await this.save({ autoStartInitialized: true })
  }
}
```

Loads the file once at startup (called from `start()`).

- `readJsonFile(this.preferencesPath, PreferencesSchema)`: reads and checks the file (see `storage/jsonFile.ts` in The draft, screenshots and safe files). If it's missing or damaged, the result isn't `ok` and the defaults (`{}`) are used; the damaged file is replaced at the next save. A file that exists but can't be read at all (a permissions error) throws, and startup ends with the error box from `fail()`.
- `if (app.isPackaged && !this.prefs.autoStartInitialized)`: the first time an installed build runs, "Start with Windows" is switched on and `autoStartInitialized` is saved. Because of that flag it happens only once: if the user turns it off later, it stays off.
- `nativeTheme.themeSource = this.get().theme`: applies the saved light or dark mode (see `setTheme()`). `init()` runs before the windows are created, so, as the comment says, no window ever shows in the other mode first.

`SettingsService.get()`

`src/main/settings.ts`, lines 50–63

```
get(): Settings {
  const available = app.isPackaged
  return {
    // Read from Windows each time: the user can also change it in Settings > Apps > Startup.
    autoStart: available && app.getLoginItemSettings().openAtLogin,
    autoStartAvailable: available,
    screenshotsDir: this.screenshotsDir,
    effort: this.prefs.effort ?? DEFAULT_EFFORT,
    autoSend: this.prefs.autoSend ?? true,
    favoriteApps: this.prefs.favoriteApps ?? [],
    theme: this.prefs.theme ?? 'dark',
    canUninstall: available,
  }
}
```

The current settings, as the Settings menu shows them (`Settings` in `src/shared/types.ts`).

- `const available = app.isPackaged`: starting with Windows only makes sense for an installed build. A dev run would register the development copy of Electron.
- `app.getLoginItemSettings().openAtLogin`: whether Windows will start the app at sign-in. It's read from Windows every time rather than remembered, because the user can also change it in Windows' Settings > Apps > Startup.
- `this.prefs.effort ?? DEFAULT_EFFORT`: `??` uses the right-hand value when the left is `null` or `undefined`.
- `this.prefs.autoSend ?? true`: "Send in Claude automatically" is on until the user turns it off.
- `canUninstall: available`: only an installed build has an uninstaller.
- `favoriteApps: this.prefs.favoriteApps ?? []`: no stars until the user adds some.
- `theme: this.prefs.theme ?? 'dark'`: dark until the user switches.

`SettingsService.setAutoStart()`

`src/main/settings.ts`, lines 65–68

```
setAutoStart(enabled: boolean): Settings {
  if (app.isPackaged) app.setLoginItemSettings({ openAtLogin: enabled })
  return this.get()
}
```

Turns "Start with Windows" on or off. `app.setLoginItemSettings()` adds or removes the app's entry in the user's startup list (a "Run" entry in the user's part of the Windows registry). Nothing is saved in `preferences.json`, since Windows itself is the record. Returns the new settings, so the menu shows what actually happened; in a dev run that's still "off".

`SettingsService.setEffort()` and `SettingsService.setAutoSend()`

src/main/settings.ts, lines 70–78

```
async setEffort(effort: Effort): Promise<Settings> {
  await this.save({ effort })
  return this.get()
}

async setAutoSend(autoSend: boolean): Promise<Settings> {
  await this.save({ autoSend })
  return this.get()
}
```

Save the reply effort and "Send in Claude automatically", and return the settings as they now are. Both take effect from the next question: `ChatSession` reads the effort through `getEffort()` each time it sends, and `ClaudeDesktop` reads the other through its `autoSend()` dependency each time it hands a question over.

`SettingsService.setTheme()`

src/main/settings.ts, lines 80–88

```
/**
 * Light or dark mode for every Desktop Assist window, whatever Windows uses: Electron's
 * `themeSource` sets what the pages see as prefers-color-scheme, which the styles follow.
 */
async setTheme(theme: Theme): Promise<Settings> {
  nativeTheme.themeSource = theme
  await this.save({ theme })
  return this.get()
}
```

Switches every Desktop Assist window between light and dark, and saves the choice.

- `nativeTheme.themeSource = theme`: Electron's `nativeTheme.themeSource` decides which mode the app's pages are told the system is in: `'dark'` or `'light'` overrides Windows' own setting (`'system'`, the default, would follow it). The pages see it as the CSS media feature `prefers-color-scheme`, which both the hand-written dark styles in `styles.css` and Tailwind's `dark:` classes follow. So both windows switch at once, without the pages doing anything.
- `await this.save({ theme })`, `return this.get()`: saved for next time, and the new settings go back to the panel, which updates the theme icon.

`SettingsService.setFavoriteApp()`

src/main/settings.ts, lines 90–95

```
/** Stars an app in the Apps list, or takes its star away. */
async setFavoriteApp(id: string, favorite: boolean): Promise<Settings> {
  const others = (this.prefs.favoriteApps ?? []).filter((favoriteId) => favoriteId !== id)
  await this.save({ favoriteApps: favorite ? [...others, id] : others })
  return this.get()
}
```

Stars an app in the Apps list, or takes the star away, and returns the settings with the new list, which the panel adopts.

- `const others = (...).filter((favoriteId) => favoriteId !== id)`: the list without this app, whatever it was before. So starring twice doesn't add it twice.
- `favorite ? [...others, id] : others`: starring adds it at the end; unstarring leaves it out. The order here doesn't matter: the list shows starred apps by name (`orderApps()`).

`SettingsService.bubbleAnchor` and `SettingsService.setBubbleAnchor()`

src/main/settings.ts, lines 97–103

```
get bubbleAnchor(): BubbleAnchor | null {
  return this.prefs.bubbleAnchor ?? null
}

async setBubbleAnchor(anchor: BubbleAnchor): Promise<void> {
  await this.save({ bubbleAnchor: anchor })
}
```

Where the bubble rests. `get bubbleAnchor()` is a *getter*: it's read like a property (`settings.bubbleAnchor`, in `start()`), but runs this code. It returns `null` if the bubble has never been moved, and `BubbleController.start()` then uses the bottom-right corner of the main display. `setBubbleAnchor()` is called by `onAnchorChange` in `start()` each time the bubble settles in a new corner.

`SettingsService.save()`

src/main/settings.ts, lines 105–112

```
private async save(changes: Preferences): Promise<void> {
  this.prefs = { ...this.prefs, ...changes }
  // One write at a time, in order, so an older save can't finish last and overwrite a newer one.
  const snapshot = this.prefs
  const write = this.saving.then(() => writeJsonFile(this.preferencesPath, snapshot))
  this.saving = write.catch(() => {})
  await write
}
```

Merges the changes into the preferences in memory, then writes the whole object to the file. Writes are queued: each one starts only after the previous one has finished.

- `{ ...this.prefs, ...changes }`: later properties win, so only the changed fields are replaced.
- `const snapshot = this.prefs`: the preferences as they are now, including this change. A queued write may start a little later, but it writes exactly this.
- `this.saving.then(() => writeJsonFile(this.preferencesPath, snapshot))`: waits for the previous write, then writes this one. Without the queue, two saves close together could finish in the wrong order and leave the older preferences on disk.
- `writeJsonFile(...)`: writes to a temporary file and then renames it over the old one, so a crash mid-write can't leave a half-written file.
- `this.saving = write.catch(() => {})`: the next save waits for this one, but a failed write mustn't stop all later saves, so the queue's copy ignores the error.
- `await write`: the caller still gets this write's own result, so a failure is reported to whoever asked for the save.

`src/main/tray.ts`: the tray icon

The icon in the Windows notification area (by the clock). Clicking it opens the panel, and its right-click menu has Open and Quit. It matters because the app has no taskbar button or window frame: this is the one place that always shows the app is running and offers a way to quit. The icon is the tenant's logo, the same image as the bubble, or a plain circle in the accent colour if the tenant has no `logo.png`.

`logos`, `logoPath`, `TRAY_SIZE` and `SCALE_FACTORS`

src/main/tray.ts, lines 4–15

```ts
// The tenant's logo, the same image as the bubble. Only a PNG can be used: Windows can't draw a
// tray icon from SVG. `?asset` makes the build copy the file and gives its path at runtime.
const logos = import.meta.glob<string>('@tenant/logo.png', {
  query: '?asset',
  import: 'default',
  eager: true,
})

const logoPath = Object.values(logos)[0]

/** The tray icon is 16px at 100% display scaling; Windows picks the size for each scaling. */
const TRAY_SIZE = 16

const SCALE_FACTORS = [1, 1.25, 1.5, 2]
```

- `import.meta.glob<string>('@tenant/logo.png', {`: a Vite build feature. At build time it finds every file matching the pattern and puts them in an object, keyed by path. Unlike a plain `import`, it doesn't break the build when nothing matches: a tenant with only a `logo.svg` just gets an empty object.
- `query: '?asset'`: an electron-vite feature for the main process. The build copies the file into its output and the import gives the file's path at run time, rather than its contents.
- `import: 'default', eager: true`: take each match's default export (the path) straight away, rather than a function that loads it later.
- `Object.values(logos)[0]`: the logo's path, or `undefined` if the tenant has no `logo.png`.
- `TRAY_SIZE = 16`: tray icons are 16×16 at 100% display scaling. `SCALE_FACTORS` are the common Windows scalings (100%, 125%, 150%, 200%) that get an image of their own.

`createTray()`

src/main/tray.ts, lines 17–34

```ts
export function createTray(options: {
  appName: string
  accentColor: string
  onOpen: () => void
  onQuit: () => void
}): Tray {
  const tray = new Tray(trayImage(options.accentColor))
  tray.setToolTip(options.appName)
  tray.setContextMenu(
    Menu.buildFromTemplate([
      { label: `Open ${options.appName}`, click: options.onOpen },
      { type: 'separator' },
      { label: 'Quit', click: options.onQuit },
    ]),
  )
  tray.on('click', options.onOpen)
  return tray
}
```

Called once from `start()`, which keeps the returned `Tray` in the module-level `tray` variable.

- `new Tray(trayImage(options.accentColor))`: adds the icon, using the image from `trayImage()`.
- `tray.setToolTip(options.appName)`: the text shown when the mouse rests on the icon.
- `Menu.buildFromTemplate([...])`: builds the right-click menu from a list of items: a label and a `click` function, or a separator.
- `tray.on('click', options.onOpen)`: a left-click opens the panel too.

`trayImage()`

src/main/tray.ts, lines 36–61

```
/**
 * The logo, resized smoothly for each display scaling (letting Windows shrink the 512px image
 * itself looks jagged). A tenant without logo.png gets a plain circle in its accent colour.
 */
function trayImage(accentColor: string): NativeImage {
  const logo = logoPath ? nativeImage.createFromPath(logoPath) : null
  if (!logo || logo.isEmpty()) {
    const size = TRAY_SIZE * 2
    return nativeImage.createFromBitmap(circleBitmap(size, accentColor), {
      width: size,
      height: size,
      scaleFactor: 2,
    })
  }
  const image = nativeImage.createEmpty()
  for (const scaleFactor of SCALE_FACTORS) {
    const size = Math.round(TRAY_SIZE * scaleFactor)
    image.addRepresentation({
      scaleFactor,
      width: size,
      height: size,
      buffer: logo.resize({ width: size, height: size, quality: 'best' }).toPNG(),
    })
  }
  return image
}
```

Builds the icon image. A *NativeImage* is Electron's image object, and one NativeImage can hold several versions of the same picture (*representations*), one per display scaling.

- `nativeImage.createFromPath(logoPath)`: loads the PNG. If the file can't be read or decoded, the image is empty (`isEmpty()`), and the fallback is used as if there were no logo.
- `nativeImage.createFromBitmap(circleBitmap(size, accentColor), {`: the fallback, made from raw pixels drawn by `circleBitmap()` at 32×32. `scaleFactor: 2` says that bitmap is meant for 200% scaling, so the image counts as 16×16 and is scaled down on a 100% display.
- `nativeImage.createEmpty()`, `image.addRepresentation({`: starts with an empty image and adds the logo at 16, 20, 24 and 32 pixels, one per scaling. The icon on each display then uses the version made for its scaling.
- `logo.resize({ width: size, height: size, quality: 'best' }).toPNG()`: the logo file is 512×512. Shrinking it here with the best-quality filter looks smooth, while leaving Windows to shrink it that far looks jagged (as the comment says).

`src/main/trayIcon.ts`: drawing the fallback tray icon

Draws a tray icon in code: a filled circle in the tenant's accent colour. `trayImage()` uses it only for a tenant that has no `logo.png`. It has no Electron imports, so `tests/trayIcon.test.ts` can check the pixels directly.

`circleBitmap()`

src/main/trayIcon.ts, lines 1–22

```
/**
 * A filled, anti-aliased circle as raw premultiplied BGRA pixels: the format
 * nativeImage.createFromBitmap expects on Windows. The tray icon for a tenant without logo.png.
 */
export function circleBitmap(size: number, hexColor: string): Buffer {
  const [r, g, b] = parseHexColor(hexColor)
  const pixels = Buffer.alloc(size * size * 4)
  const centre = size / 2
  const radius = size / 2 - 0.5
  for (let y = 0; y < size; y++) {
    for (let x = 0; x < size; x++) {
      const distance = Math.hypot(x + 0.5 - centre, y + 0.5 - centre)
      const alpha = Math.min(1, Math.max(0, radius - distance + 0.5))
      const i = (y * size + x) * 4
      pixels[i] = Math.round(b * alpha)
      pixels[i + 1] = Math.round(g * alpha)
      pixels[i + 2] = Math.round(r * alpha)
      pixels[i + 3] = Math.round(255 * alpha)
    }
  }
  return pixels
}
```

Returns the raw pixels of a `size` × `size` image: a circle filling the square, with a smooth edge.

- `Buffer.alloc(size * size * 4)`: four bytes per pixel, all zero to begin with (fully transparent). Pixels go row by row, so pixel (x, y) starts at byte `(y * size + x) * 4`.
- `Math.hypot(x + 0.5 - centre, y + 0.5 - centre)`: the distance from the middle of this pixel to the centre of the circle.
- `Math.min(1, Math.max(0, radius - distance + 0.5))`: how much of the pixel the circle covers, roughly. 1 when the pixel's middle is at least half a pixel inside the edge, 0 when it's at least half a pixel outside, and in between on the edge. This smooth edge is called *anti-aliasing*; without it the circle would look jagged.
- `radius = size / 2 - 0.5`: leaves half a pixel of room so the soft edge fits inside the square.
- `pixels[i] = Math.round(b * alpha)`: the bytes go blue, green, red, alpha (*BGRA*), the order Windows uses. Each colour is multiplied by the coverage (*premultiplied* alpha), which is the form `nativeImage.createFromBitmap` expects.

`parseHexColor()`

src/main/trayIcon.ts, lines 24–28

```
export function parseHexColor(hex: string): [number, number, number] {
  const match = /^#([0-9a-f]{2})([0-9a-f]{2})([0-9a-f]{2})$/i.exec(hex)
  if (!match) throw new Error(`Not a #RRGGBB colour: ${hex}`)
  return [parseInt(match[1]!, 16), parseInt(match[2]!, 16), parseInt(match[3]!, 16)]
}
```

Turns `#RRGGBB` into three numbers from 0 to 255.

- `/^#([0-9a-f]{2})([0-9a-f]{2})([0-9a-f]{2})$/i`: exactly a `#` and three pairs of hex digits; the `i` flag allows upper or lower case. The short form `#fff` is rejected.

- `parseInt(match[1]!, 16)`: reads one pair as a base-16 number. The `!` tells TypeScript the group is definitely there (the regex matched), because this project's settings make it treat any array element as possibly missing.

`src/main/env.d.ts`: types for Vite's build features

A `.d.ts` file holds only type information and produces no code. This one lets TypeScript understand the build-time features `tray.ts` uses, which plain TypeScript doesn't know about.

`src/main/env.d.ts`, lines 1–3

```
// Vite's additions to TypeScript for the main process, e.g. import.meta.glob and `?asset` imports.
/// <reference types="vite/client" />
/// <reference types="electron-vite/node" />
```

- `/// <reference types="vite/client" />`: a *triple-slash directive*, which pulls in another package's type declarations. Vite's declarations add `import.meta.glob` and Vite's other `import.meta` extras.
- `/// <reference types="electron-vite/node" />`: electron-vite's declarations describe its special imports for the main process; for example, a `?asset` import is a file path (a `string`).

JumpCloud sign-in

This part covers how Desktop Assist knows who you are, and how that turns into access to Claude without an API key. You sign in once with JumpCloud in your normal browser. The app keeps a long-lived *refresh token*, encrypted on disk, and uses it to get short-lived *ID tokens*, which Anthropic swaps for Claude tokens. All of it runs in the main process: the pages only ever learn the sign-in status and your name and email (`AuthStatus` in `src/shared/types.ts`), never a token.

This is the built-in chat's sign-in. When a tenant sends questions to Claude Desktop instead (`chatApp` in `tenant.json`; see Claude Desktop), none of this page's code runs: Claude Desktop has its own sign-in.

Signing in

1. The user clicks **Sign in with JumpCloud**. IPC `assist:auth-sign-in` calls `AuthManager.signIn()`, which sets the status to `signing-in`.
2. `signIn()` starts a tiny web server on `127.0.0.1` with `listenForRedirect()` (`loopback.ts`).
3. `Oidc.begin()` (made by `createOidc()` in `oidc.ts`) builds the JumpCloud sign-in address with fresh PKCE, `state` and `nonce` values, and the app opens it in the default browser.
4. The user signs in at JumpCloud (often it finishes by itself). JumpCloud sends the browser back to `http://127.0.0.1:47621/callback?code=…&state=…`. The little server answers "you can close this tab", stops, and hands the address to `signIn()`.
5. `PendingSignIn.finish()` checks `state`, swaps the code (plus the PKCE secret) for tokens at JumpCloud, and has openid-client check the ID token.
6. `signIn()` insists on a refresh token, saves it with your name and email (`startSession()` → `SecretStore.save()`), keeps the unused ID token as a *spare*, and sets `signed-in`. `index.ts` then reopens the panel.

Starting up

1. `startBuiltInChat()` in `index.ts` calls `AuthManager.init()`, which reads the saved sign-in with `loadSession()` → `SecretStore.load()`.
2. If there is one, `renew()` → `refresh()` → `Oidc.refresh()` swaps the refresh token for new tokens, saves JumpCloud's replacement refresh token (if it sent one), and keeps the new ID token as the spare. If JumpCloud can't be reached the status becomes `offline`; if it refuses, `endSession('expired')` signs the user out.

Getting Claude access

1. On the first chat request, the Anthropic SDK inside `AnthropicBackend` needs a Claude token. Its `oidcFederationProvider` calls the `identityToken()` option. `index.ts` set that to a small function that calls `AuthManager.freshIdToken()` and notes a summary of the token for the log (`summarizeIdToken()` in `idToken.ts`).
2. `freshIdToken()` returns the spare ID token if it has more than a minute left, otherwise gets a new one with `refresh()`.
3. The SDK posts the ID token to Anthropic's `/v1/oauth/token`. Anthropic checks it against the federation rule and returns a short-lived Claude token. The SDK keeps that token and only repeats steps 1 to 3 when it's about to expire, or once if Anthropic rejects it.

Signing out

`AuthManager.signOut()` stops any sign-in in progress, `endSession('logout')` deletes the saved file, and `Oidc.revoke()` asks JumpCloud to cancel the refresh token. Through `onSignedOut`, `index.ts` drops the SDK's cached Claude token (`backend.reset()`) and clears the chat.

`src/main/auth/AuthManager.ts`: owning the sign-in

`AuthManager` decides whether someone is signed in, keeps the refresh token, and hands out ID tokens for Claude. `startBuiltInChat()` in `src/main/index.ts` creates the one instance. The IPC handlers in `src/main/ipc.ts` call `signIn()`, `cancelSignIn()`, `signOut()` and `retry()`; `ChatSession` asks `canChat` before sending; `AnthropicBackend` calls `freshIdToken()` (through a small wrapper in `index.ts`). It never touches JumpCloud, the disk or the browser itself: those arrive through `AuthManagerDeps`, so `tests/authManager.test.ts` can drive it with fakes.

The ideas it is built around:

- **OpenID Connect (OIDC)** is the standard way for an app to ask an identity provider (here JumpCloud) "who is this user?". The user signs in on the provider's own web page, so the app never sees a password, and the app receives tokens.
- **ID token**: a short-lived JWT (a signed, base64-encoded JSON document) from JumpCloud saying who the user is (`sub`, `email`, `name`), which app it was made for (`aud`, the app's client ID) and when it expires (`exp`). It's what Anthropic accepts as proof of identity.
- **Refresh token**: a long-lived, opaque string that lets the app get new tokens from JumpCloud without the user signing in again. It's the only thing saved to disk. Its lifetime is set in JumpCloud (up to 90 days; see `docs/JUMPCLOUD_SETUP.md`).
- **Refresh token rotation**: JumpCloud may answer each refresh with a new refresh token and stop accepting the old one. So the newest must always be saved, and two refreshes must never run at once with the same token: the second would present a token that was just replaced, JumpCloud would refuse it, and a refusal ends the sign-in. That's why refreshes go through a queue.
- **Workload Identity Federation (WIF)** is how Anthropic accepts another identity provider's tokens instead of an API key. IT sets up a *federation rule* in the Claude Console: trust tokens from this issuer (JumpCloud), made for this audience (the app's client ID), whose claims match a condition (an `@morsemicro.com` email). The SDK sends an ID token; Anthropic checks its signature against JumpCloud's published keys and checks the rule, then returns a short-lived Claude token for the company's *service account*.
- **Each ID token works once.** Every ID token has a unique ID in its `jti` claim, and Anthropic refuses a token whose `jti` it has already accepted (replay protection: a copied token can't be swapped again). So every swap needs a new ID token, and `freshIdToken()` never hands out the same one twice.

`SignInRequiredError`, `SignInUnavailableError`

src/main/auth/AuthManager.ts, lines 13–25

```
/** Claude was asked for while nobody is signed in (or the sign-in just expired). */
export class SignInRequiredError extends Error {
  constructor() {
    super('Sign in with JumpCloud to use Claude')
  }
}

/** A saved sign-in couldn't be renewed because JumpCloud couldn't be reached. */
export class SignInUnavailableError extends Error {
  constructor(cause: unknown) {
    super("Couldn't reach JumpCloud", { cause })
  }
}
```

The two ways getting an ID token can fail. `refresh()` throws them, and they travel through the Anthropic SDK (which wraps them in its own error, with the original as `cause`) to `classifyError()` in `src/main/claude/errors.ts`. That turns them into the `signed-out` message ("Sign in with JumpCloud to keep chatting.") and the `sign-in-unreachable` message, both of which offer Retry.

- `extends Error`: a custom error class, so other code can tell the cases apart with `instanceof`.
- `{ cause }`: the standard way to attach the original error (for example a network failure) to a new one, so the log still shows what really happened.

`SessionSchema`, `Session`

src/main/auth/AuthManager.ts, lines 27–32

```
/** What's kept on disk (encrypted) between runs. */
const SessionSchema = z.object({
  refreshToken: z.string().min(1),
  user: z.object({ name: z.string().optional(), email: z.string().optional() }),
})

type Session = z.infer<typeof SessionSchema>
```

The sign-in as it's kept in memory and saved (as JSON, encrypted) in `jumpcloud-session.bin`: the refresh token, and the user's name and email so the app can show "Signed in as …" while offline.

- `z.object({`: a zod schema, a description of the shape that can check data at runtime. `loadSession()` uses it to reject a saved file that doesn't fit.
- `z.infer<typeof SessionSchema>`: derives the TypeScript type from the schema, so the type and the runtime check can't drift apart.

AuthManagerDeps

src/main/auth/AuthManager.ts, lines 34–51

```
export interface AuthManagerDeps {
  /** Null when tenant.json isn't filled in yet (`missing` says what's needed). */
  oidc: Oidc | null
  missing: string[]
  store: Pick<SecretStore, 'load' | 'save' | 'clear'>
  redirectPort: number
  listen(port: number): Promise<RedirectListener>
  openBrowser(url: string): Promise<void>
  onStatus(status: AuthStatus): void
  /**
   * Nobody is signed in any more: the user logged out, or JumpCloud ended their sign-in (they'll
   * usually sign straight back in, so the conversation can be kept for them).
   */
  onSignedOut(reason: 'logout' | 'expired'): void
  /** Signing in or renewing failed; gets the full error, for the log. */
  onError?(error: unknown): void
  now?: () => number
}
```

Everything `AuthManager` needs from outside, passed to its constructor. `startBuiltInChat()` in `index.ts` supplies the real ones: `createOidc()`, a `SecretStore` using Windows DPAPI, `listenForRedirect()` and Electron's `shell.openExternal()`.

- `oidc: Oidc | null`: `null` when `missingSettings()` in `src/main/tenant.ts` says tenant.json isn't filled in; `missing` then lists what's needed, for the panel to show.
- `Pick<SecretStore, 'load' | 'save' | 'clear'>`: a TypeScript utility type meaning "any object with these three methods of `SecretStore`", so tests can pass a simple in-memory store.
- `onStatus(status: AuthStatus): void`: called on every status change. `index.ts` broadcasts it to both pages on IPC `assist:auth-status`, and reopens the panel when `signing-in` turns into `signed-in`.
- `onSignedOut(reason: 'logout' | 'expired'): void`: `index.ts` calls `backend.reset()` (forgetting the cached Claude token) for both reasons, and clears the chat only for `'logout'`. After `'expired'` the conversation stays, so Retry can resend the failed message once the user signs back in.
- `now?: () => number`: an optional clock; tests use it to make ID tokens look old.

MIN_ID_TOKEN_LIFE_S

src/main/auth/AuthManager.ts, lines 53–54

```
/** An ID token is only handed out if it's valid for at least this much longer. */
const MIN_ID_TOKEN_LIFE_S = 60
```

`takeIdToken()` only hands out the spare ID token if it has more than 60 seconds left. A token that expires on its way to Anthropic, or that a slightly fast clock at Anthropic already sees as expired, would make the swap fail.

class AuthManager

The class, with the comment that describes it:

src/main/auth/AuthManager.ts, lines 56–67

```ts
/**
 * Owns the JumpCloud sign-in.
 *
 * Signing in opens JumpCloud in the browser (where the user is usually already signed in, so it
 * completes on its own). What's kept is a refresh token, saved encrypted, so the user stays
 * signed in across restarts; `init()` renews it at every start.
 *
 * Claude access is granted per ID token, and each ID token can only be exchanged once, so
 * `freshIdToken()` hands out a new one every time: the unused one from signing in first, then one
 * from a refresh. JumpCloud may replace the refresh token on every refresh; the newest is saved.
 */
export class AuthManager {
  // …
}
```

This block shows only the class's own description (its doc comment) and its first line; the body is left out (`// …`), and its fields and methods follow one by one below. The description sums up the design: sign in through the browser, keep the refresh token so the user stays signed in, renew it at every start, and hand out a new ID token for every swap with Anthropic.

`AuthManager` fields

src/main/auth/AuthManager.ts, lines 68–78

```ts
private state: AuthStatus = { state: 'checking' }

private session: Session | null = null

/** An ID token not yet exchanged for Claude access. */
private spare: { token: string; exp: number } | null = null

private listener: RedirectListener | null = null

/** Cancel was pressed during the current sign-in (possibly before the browser opened). */
private signInCancelled = false

/** Refreshes run one at a time, since each may replace the refresh token. */
private queue: Promise<unknown> = Promise.resolve()

/** Bumped by sign-in and sign-out, so a slow refresh can't bring back a session that ended. */
private generation = 0
```

The class keeps all its state in these private fields.

- `state`: the current `AuthStatus`. It starts as `checking` because `init()` hasn't run yet. It's only ever changed through `setStatus()`, so the pages always hear about it.
- `session`: who is signed in, and their refresh token. `null` means nobody is.
- `spare`: an ID token that JumpCloud handed over (when signing in, or when renewing at startup) and that hasn't been swapped yet. `exp` is its expiry in Unix seconds, straight from the token.
- `listener`: the loopback listener while a browser sign-in is waiting, so `cancelSignIn()` can close it.
- `signInCancelled`: set by `cancelSignIn()` during a sign-in. Closing the listener only stops the wait for the browser; this flag also covers a Cancel pressed before the browser has opened.
- `queue`: a chain of promises. Each refresh is attached to the end, so refreshes run strictly one after another (see rotation above).

- `generation`: a counter that `signIn()` and `endSession()` increase. Slow work notes the value before it starts waiting and compares afterwards; if it changed, the user signed in or out meanwhile and the result is thrown away. This stops, for example, a refresh that finishes after Log out from saving the refresh token again.

`AuthManager.constructor`, `status`, `canChat`

src/main/auth/AuthManager.ts, lines 80–89

```
constructor(private readonly deps: AuthManagerDeps) {}

get status(): AuthStatus {
  return this.state
}

/** Whether chat requests can be made (they may still need JumpCloud to renew the sign-in). */
get canChat(): boolean {
  return this.session !== null && this.state.state !== 'signing-in'
}
```

- `private readonly deps: AuthManagerDeps`: TypeScript shorthand that declares a private, read-only field `deps` and stores the argument in it, so the constructor body is empty.
- `get status()`: a getter, read like a field (`auth.status`). `getState()` in `index.ts` uses it to give a freshly loaded page the current status.
- `get canChat()`: whether `ChatSession` may start a request. It's true whenever a sign-in exists, including `checking` and `offline`: the request itself calls `freshIdToken()`, which tries JumpCloud again. It's false while a browser sign-in is in progress.

`AuthManager.init()`

src/main/auth/AuthManager.ts, lines 91–101

```
/** Loads the saved sign-in and renews it. Called once at startup. */
async init(): Promise<void> {
  if (!this.deps.oidc)
    return this.setStatus({ state: 'unconfigured', missing: this.deps.missing })
  const generation = this.generation
  const session = await this.loadSession()
  if (generation !== this.generation) return // signed in or out meanwhile
  this.session = session
  if (!session) return this.setStatus({ state: 'signed-out' })
  await this.renew()
}
```

Runs once at startup (`startBuiltInChat()` in `index.ts` calls it without waiting). It loads the saved sign-in and renews it straight away, before any chat. That shows an expired sign-in at once rather than on the first message, saves JumpCloud's newest refresh token, and leaves a spare ID token ready for the first message.

- `if (!this.deps.oidc)`: tenant.json isn't filled in, so the status becomes `unconfigured` with the list of missing settings, and JumpCloud is never contacted. `return this.setStatus(...)` is just a short way to set the status and stop.
- `if (generation !== this.generation) return`: reading the file takes a moment. If the user signed in or out in the meantime, what was read is out of date and must not replace the newer state.
- `if (!session) return this.setStatus({ state: 'signed-out' })`: nothing saved (or the file couldn't be read or decrypted), so the panel offers Sign in.

`AuthManager.retry()`

src/main/auth/AuthManager.ts, lines 103–107

```
/** Tries again after `init()` couldn't reach JumpCloud. */
async retry(): Promise<void> {
  if (this.state.state !== 'offline' || !this.session) return
  await this.renew()
}
```

The **Retry** button on the offline banner (IPC `assist:auth-retry`). It renews again, but only if the app really is offline with a saved sign-in, so a stray click in any other state does nothing.

`AuthManager.signIn()`

src/main/auth/AuthManager.ts, lines 109–143

```
/** Signs in through the browser. Resolves when it finishes, fails or is cancelled. */
async signIn(): Promise<void> {
  const oidc = this.deps.oidc
  if (!oidc || this.state.state === 'signing-in') return
  const generation = ++this.generation
  this.signInCancelled = false
  this.setStatus({ state: 'signing-in' })

  let listener: RedirectListener | null = null
  try {
    listener = await this.deps.listen(this.deps.redirectPort)
    this.listener = listener
    const pending = await oidc.begin(listener.redirectUri)
    // Cancel may have been pressed while JumpCloud's settings were being fetched.
    if (this.signInCancelled) throw new SignInCancelledError()
    await this.deps.openBrowser(pending.url)
    const tokens = await pending.finish(await listener.result)
    if (!tokens.refreshToken) {
      throw new Error(
        "JumpCloud didn't allow staying signed in. Ask IT to turn on the Refresh Token grant for this
app.",
      )
    }
    if (generation !== this.generation) return
    await this.startSession(tokens.refreshToken, userOf(tokens.claims))
    this.spare = { token: tokens.idToken, exp: tokens.claims.exp }
    this.setStatus({ state: 'signed-in', user: userOf(tokens.claims) })
  } catch (error) {
    if (generation !== this.generation) return
    if (!(error instanceof SignInCancelledError)) this.deps.onError?.(error)
    this.setStatus({ state: 'signed-out', message: signInFailure(error) })
  } finally {
    listener?.close()
    if (this.listener === listener) this.listener = null
  }
}
```

The whole browser sign-in, from opening JumpCloud to saving the result. The IPC handler starts it and returns at once, because it can take minutes; progress reaches the panel through the status.

- `if (!oidc || this.state.state === 'signing-in') return`: not set up, or already waiting for the browser (a second click does nothing).

- `const generation = ++this.generation`: increases the counter first, so a refresh still running from before (for example the startup renewal) can't overwrite this sign-in when it finishes.
- `this.signInCancelled = false`: a fresh sign-in starts uncancelled.
- `listener = await this.deps.listen(this.deps.redirectPort)`: the loopback server is started before the browser opens, so it's ready when JumpCloud sends the browser back. This is also where "port in use" fails, before the browser opens.
- `const pending = await oidc.begin(listener.redirectUri)`: makes the sign-in address. `pending` keeps the secret PKCE, state and nonce values needed by `finish()`.
- `if (this.signInCancelled) throw new SignInCancelledError()`: `begin()` may have spent a few seconds fetching JumpCloud's settings (the first time only), and the user may have pressed Cancel meanwhile. Without this check the browser would still open JumpCloud, with nothing listening for its return. Throwing lands in the `catch` below, just like a cancel while waiting for the browser.
- `await this.deps.openBrowser(pending.url)`: opens the default browser.
- `const tokens = await pending.finish(await listener.result)`: waits (up to 5 minutes) for the browser to come back, then swaps the code for tokens.
- `if (!tokens.refreshToken)`: without a refresh token the app couldn't keep the user signed in, or get a second ID token, so it stops with a message for IT.
- `if (generation !== this.generation) return`: the user signed out while this was running, so the tokens are thrown away.
- `this.spare = { token: tokens.idToken, exp: tokens.claims.exp }`: the ID token from signing in hasn't been swapped yet, so it's kept for the first chat message, saving a refresh.
- `if (!(error instanceof SignInCancelledError)) this.deps.onError?.(error)`: a cancel is the user's choice, so only real failures are logged. Either way the status goes back to `signed-out`, with the message from `signInFailure()` (none for a cancel).
- `finally`: always closes the listener (closing an already finished listener does nothing), and clears the `listener` field only if it still belongs to this sign-in.

`AuthManager.cancelSignIn()`

`src/main/auth/AuthManager.ts`, lines 145–149

```ts
/** Stops waiting for a sign-in started in the browser. */
cancelSignIn(): void {
  if (this.state.state === 'signing-in') this.signInCancelled = true
  this.listener?.close()
}
```

The **Cancel** button while waiting for the browser (IPC `assist:auth-cancel`), and the first step of `signOut()`. Either way `signIn()` ends with `signed-out` and no message.

- `if (this.state.state === 'signing-in') this.signInCancelled = true`: records the cancel, for the case where Cancel is pressed before the browser has opened (while `begin()` is still running). `signIn()` checks the flag after `begin()` and stops there. Outside a sign-in there's nothing to cancel, so the flag is left alone.
- `this.listener?.close()`: once the listener exists, closing it makes its `result` fail with `SignInCancelledError`, which ends a sign-in that is waiting for the browser.

`AuthManager.signOut()`

src/main/auth/AuthManager.ts, lines 151–160

```
/** Forgets the sign-in here, and tells JumpCloud to revoke it. */
async signOut(): Promise<void> {
  const refreshToken = this.session?.refreshToken
  this.cancelSignIn()
  await this.endSession('logout')
  if (refreshToken) {
    // Best effort: the saved copy is already gone, so failing to reach JumpCloud is harmless.
    await this.deps.oidc?.revoke(refreshToken).catch(() => {})
  }
}
```

Log out in Settings (IPC `assist:auth-sign-out`). It forgets the sign-in on this PC first, then asks JumpCloud to revoke the refresh token.

- `const refreshToken = this.session?.refreshToken`: taken first, because `endSession()` clears `session`.
- `await this.endSession('logout')`: deletes the saved file and sets `signed-out`; `onSignedOut('logout')` makes `index.ts` reset the Claude client and clear the chat.
- `revoke(refreshToken).catch(() => {})`: revoking means JumpCloud stops accepting that refresh token, so a leftover copy (in a backup, say) is useless. If JumpCloud can't be reached the error is ignored: the user is signed out on this PC either way.

`AuthManager.freshIdToken()`

src/main/auth/AuthManager.ts, lines 162–171

```
/**
 * A new, unused ID token for one exchange with Claude. Throws SignInRequiredError when nobody is
 * signed in or the sign-in has expired, and SignInUnavailableError when JumpCloud can't be
 * reached.
 */
freshIdToken(): Promise<string> {
  const next = this.queue.then(() => this.takeIdToken())
  this.queue = next.catch(() => {})
  return next
}
```

Gives the Anthropic SDK one new, never-swapped ID token. `index.ts` hands `AnthropicBackend` an `identityToken` function that calls it (and summarizes the token for the log, see `idToken.ts`), and the SDK calls that whenever it needs a new Claude token.

- `this.queue.then(() => this.takeIdToken())`: runs after every refresh already queued, so two requests at once never refresh with the same refresh token in parallel.
- `this.queue = next.catch(() => {})`: the queue continues from this step even if it fails. Without the `catch`, one failed refresh would leave a rejected promise at the end of the queue, and every later `.then` would be skipped and fail too.
- `return next`: the caller still gets the real outcome, error included (`SignInRequiredError` or `SignInUnavailableError`).

`AuthManager.takeIdToken()`

src/main/auth/AuthManager.ts, lines 173–178

```
private async takeIdToken(): Promise<string> {
  const spare = this.spare
  this.spare = null
  if (spare && spare.exp - this.nowSeconds() > MIN_ID_TOKEN_LIFE_S) return spare.token
  return (await this.refresh()).idToken
}
```

Runs inside the queue: uses the spare ID token if there's a good one, otherwise refreshes for a new one.

- `this.spare = null`: the spare is removed before it's checked, so it can be used at most once. A spare that's too old is simply dropped.
- `spare.exp - this.nowSeconds() > MIN_ID_TOKEN_LIFE_S`: more than a minute left, so it's safe to send.

`AuthManager.renew()`

src/main/auth/AuthManager.ts, lines 180–196

```
/** Renews the saved sign-in at startup (or on Retry), keeping its ID token for the first chat. */
private async renew(): Promise<void> {
  this.setStatus({ state: 'checking' })
  try {
    const tokens = await this.queueRefresh()
    this.spare = { token: tokens.idToken, exp: tokens.claims.exp }
  } catch (error) {
    if (error instanceof SignInUnavailableError && this.session) {
      this.setStatus({
        state: 'offline',
        user: this.session.user,
        message: "Couldn't reach JumpCloud to renew your sign-in. Check your connection.",
      })
    }
    // SignInRequiredError: refresh() has already signed the user out.
  }
}
```

The startup renewal (from `init()`) and the offline Retry. Unlike a refresh made for a chat request, it sets the status along the way and never throws.

- `this.setStatus({ state: 'checking' })`: the panel shows "Checking your JumpCloud sign-in…". `refresh()` sets `signed-in` when it succeeds.
- `this.spare = { token: tokens.idToken, exp: tokens.claims.exp }`: the new ID token is kept for the first chat message.
- `error instanceof SignInUnavailableError && this.session`: JumpCloud couldn't be reached, so the sign-in is kept and the status becomes `offline`, with a banner offering Retry. Chat stays usable (`canChat` is still true) and each send tries JumpCloud again. The `this.session` check also tells TypeScript `this.session.user` on the next lines is safe.
- `// SignInRequiredError: refresh() has already signed the user out.`: so there's nothing left to do here. (The same error also comes back when the user signed in or out while this was running, and then the newer state stands.)

`AuthManager.queueRefresh()`

src/main/auth/AuthManager.ts, lines 198–202

```
private queueRefresh(): Promise<OidcTokens> {
  const next = this.queue.then(() => this.refresh())
  this.queue = next.catch(() => {})
  return next
}
```

The same queueing as `freshIdToken()`, but for `renew()`, which needs the whole `OidcTokens` (including the ID token's expiry) rather than just the token.

`AuthManager.refresh()`

src/main/auth/AuthManager.ts, lines 204–237

```
/** Swaps the refresh token for new tokens and saves the replacement refresh token. */
private async refresh(): Promise<OidcTokens> {
  const session = this.session
  if (!session || !this.deps.oidc) throw new SignInRequiredError()
  const generation = this.generation

  let tokens: OidcTokens
  try {
    tokens = await this.deps.oidc.refresh(session.refreshToken)
  } catch (error) {
    if (generation !== this.generation) throw new SignInRequiredError()
    this.deps.onError?.(error)
    if (error instanceof SessionExpiredError) {
      await this.endSession(
        'expired',
        'Your JumpCloud sign-in has expired. Sign in again to keep chatting.',
      )
      throw new SignInRequiredError()
    }
    throw new SignInUnavailableError(error)
  }
  if (generation !== this.generation) throw new SignInRequiredError()

  const user = userOf(tokens.claims, session.user)
  if (tokens.refreshToken && tokens.refreshToken !== session.refreshToken) {
    await this.startSession(tokens.refreshToken, user)
  } else {
    this.session = { ...session, user }
  }
  if (this.state.state !== 'signed-in' || !sameUser(this.state.user, user)) {
    this.setStatus({ state: 'signed-in', user })
  }
  return tokens
}
```

Swaps the refresh token with JumpCloud for a new ID token (and possibly a new refresh token), and decides what a failure means. It's always called through the queue.

- `if (!session || !this.deps.oidc) throw new SignInRequiredError()`: nobody is signed in, so there is nothing to refresh.
- `tokens = await this.deps.oidc.refresh(session.refreshToken)`: the refresh-token request to JumpCloud (`Oidc.refresh()` in `oidc.ts`).

- `if (generation !== this.generation) throw new SignInRequiredError()` (in the `catch`): the user signed in or out while waiting, so this failure no longer matters and isn't logged or acted on.
- `if (error instanceof SessionExpiredError)`: JumpCloud answered and said no (expired, revoked, or the user lost access). The sign-in is over, so `endSession('expired', …)` signs out with an explanation.
- `throw new SignInUnavailableError(error)`: anything else (offline, a timeout, a JumpCloud server error, an unexpected response) counts as "couldn't reach JumpCloud". The sign-in is kept, because it may well work next time.
- `if (generation !== this.generation) throw new SignInRequiredError()` (after success): the user logged out while the request was on its way. Saving the new refresh token now would bring back a sign-in they just ended.
- `userOf(tokens.claims, session.user)`: the name and email from the new ID token, falling back to the saved ones for anything it leaves out.
- `if (tokens.refreshToken && tokens.refreshToken !== session.refreshToken)`: JumpCloud rotated the refresh token, so the new one is saved at once (the old one may already be dead). Otherwise only the in-memory user is updated, and the saved file is left alone.
- `if (this.state.state !== 'signed-in' || !sameUser(this.state.user, user))`: tells the pages only when something changed, not on every refresh. This is also how `checking` (after `renew()`) and `offline` turn into `signed-in`.

`AuthManager.startSession()`

`src/main/auth/AuthManager.ts`, lines 239–247

```ts
private async startSession(refreshToken: string, user: SignedInUser): Promise<void> {
  this.session = { refreshToken, user }
  try {
    await this.deps.store.save(JSON.stringify(this.session))
  } catch (error) {
    // Still signed in for this run; the user will just be asked again next start.
    console.error('Saving the sign-in failed', error)
  }
}
```

Records a new sign-in, or a rotated refresh token, in memory and on disk (`SecretStore.save()`, as JSON).

- `console.error('Saving the sign-in failed', error)`: saving can fail, for example if Windows encryption isn't available. The user stays signed in for this run; at the next start the old file (or none) is found and they may have to sign in again. That's better than failing a sign-in that worked.

`AuthManager.endSession()`

`src/main/auth/AuthManager.ts`, lines 249–258

```ts
private async endSession(reason: 'logout' | 'expired', message?: string): Promise<void> {
  this.generation++
  this.session = null
  this.spare = null
  await this.deps.store.clear().catch((error: unknown) => {
    console.error('Removing the saved sign-in failed', error)
  })
  this.setStatus(message ? { state: 'signed-out', message } : { state: 'signed-out' })
  this.deps.onSignedOut(reason)
}
```

Ends the sign-in, for Log out (`'logout'`) or because JumpCloud refused it (`'expired'`).

- `this.generation++`: happens first, before anything that waits, so every refresh or sign-in still in flight sees that its result is out of date.
- `this.spare = null`: an unused ID token belongs to the old sign-in and mustn't be swapped any more.
- `this.deps.store.clear()`: if deleting the file fails, the user is still signed out for this run; the error is only logged.
- `this.deps.onSignedOut(reason)`: `index.ts` forgets the cached Claude token, and for Log out clears the chat.

`AuthManager.loadSession()`

`src/main/auth/AuthManager.ts`, lines 260–269

```
private async loadSession(): Promise<Session | null> {
  try {
    const saved = await this.deps.store.load()
    if (!saved) return null
    const parsed = SessionSchema.safeParse(JSON.parse(saved))
    return parsed.success ? parsed.data : null
  } catch {
    return null
  }
}
```

Reads the saved sign-in for `init()`. Any problem means "nothing saved", so the worst case is that the user signs in again.

- `SessionSchema.safeParse(JSON.parse(saved))`: `safeParse` returns success or failure instead of throwing, so a file in an old or unexpected shape is ignored.
- `catch {`: covers a file that can't be read and text that isn't JSON. (`SecretStore.load()` already returns `null` for a missing file or one that can't be decrypted.)

`AuthManager.setStatus()`, `AuthManager.nowSeconds()`

`src/main/auth/AuthManager.ts`, lines 271–278

```
private setStatus(status: AuthStatus): void {
  this.state = status
  this.deps.onStatus(status)
}

private nowSeconds(): number {
  return (this.deps.now?.() ?? Date.now()) / 1000
}
```

- `setStatus`: the one place the status changes, so the stored value and what the pages are told can't differ.
- `nowSeconds`: the current time in seconds, because a token's `exp` is in Unix seconds while `Date.now()` is in milliseconds. `this.deps.now?.()` lets tests supply the clock.

`userOf()`, `sameUser()`, `signInFailure()`

src/main/auth/AuthManager.ts, lines 281–294

```ts
function userOf(claims: IdTokenClaims, fallback: SignedInUser = {}): SignedInUser {
  return { name: claims.name ?? fallback.name, email: claims.email ?? fallback.email }
}

function sameUser(a: SignedInUser, b: SignedInUser): boolean {
  return a.name === b.name && a.email === b.email
}

/** What to tell the user when signing in didn't work (nothing if they cancelled). */
function signInFailure(error: unknown): string | undefined {
  if (error instanceof SignInCancelledError) return undefined
  if (error instanceof SignInTimeoutError) return 'Sign-in timed out. Try again.'
  return `Sign-in didn't work. ${describeSignInError(error)}`
}
```

Small helpers outside the class.

- `claims.name ?? fallback.name`: an ID token may leave out `name` or `email` (they're optional claims), so the previously known value is kept. `??` uses the right side only when the left is `null` or `undefined`.
- `sameUser`: lets `refresh()` skip a status update when nothing changed.
- `signInFailure`: the message under the Sign in button. A cancel shows nothing, the 5-minute timeout has its own message, and anything else is explained by `describeSignInError()` in `oidc.ts`.

`src/main/auth/oidc.ts`: OpenID Connect with JumpCloud

The only file that speaks OIDC. It wraps the `openid-client` library (version 6) behind a small `Oidc` interface with three calls (`begin`, `refresh`, `revoke`), so `AuthManager` and its tests don't depend on the library. `startBuiltInChat()` in `index.ts` creates it with `createOidc(signIn, { allowInsecure: localIssuer })` only when tenant.json is complete.

The ideas behind it:

- **Authorization Code flow**: the app sends the browser to JumpCloud's *authorization endpoint*. After the user signs in, JumpCloud sends the browser back to the app's *redirect URI* with a one-time `code`. The app then posts that code straight to JumpCloud's *token endpoint* and gets the tokens in the reply, so tokens never appear in the browser.
- **Public client with PKCE** (Proof Key for Code Exchange): a desktop app can't keep a secret, since anyone can unpack the installer, so it has no client secret (JumpCloud's "Public (None PKCE)" setting). Instead, each sign-in makes a random *code verifier* and puts only its SHA-256 hash (the *code challenge*) in the browser address. When swapping the code, the app sends the verifier itself, and JumpCloud checks that it matches. A code intercepted on its way back is useless without the verifier, which never left the app.
- `state`: a random value sent out with the browser and expected back unchanged. It proves the callback belongs to the sign-in this app started, so a forged link can't slip in someone else's code (cross-site request forgery).
- `nonce`: a random value JumpCloud copies into the ID token. Checking it proves the ID token was made for this sign-in and isn't an old one replayed.
- **Discovery**: an OIDC provider publishes its endpoints, keys and features at `<issuer>/.well-known/openid-configuration`. openid-client reads that, so only the issuer address and the client ID need to be configured.

What openid-client checks on an ID token: the issuer, the audience (must be this app's client ID), the expiry and issue time, that `sub` is present, the signing algorithm, and (when signing in) the nonce. As the comment above `createOidc` says, it doesn't verify the signature (openid-client 6 only does that when `enableNonRepudiationChecks` is turned on, and this code doesn't turn it on). OIDC allows that: the token came straight from JumpCloud's token endpoint over HTTPS, so TLS already proves who sent it. Anthropic does verify the signature when the token is swapped.

SCOPES

src/main/auth/oidc.ts, lines 4–5

```ts
/** `offline_access` asks for a refresh token, so the user stays signed in across restarts. */
export const SCOPES = 'openid email profile offline_access'
```

What the app asks JumpCloud for: `openid` (this is an OIDC sign-in, so send an ID token), `email` and `profile` (put the email and name in it), and `offline_access` (also send a refresh token).

IdTokenClaims, OidcTokens

src/main/auth/oidc.ts, lines 7–21

```ts
export interface IdTokenClaims {
  sub: string
  email?: string
  name?: string
  /** Expiry, Unix seconds. */
  exp: number
}

export interface OidcTokens {
  /** A signed JWT identifying the user. Exchanged for Claude access (one exchange per token). */
  idToken: string
  /** JumpCloud may rotate this on every refresh; always keep the newest. */
  refreshToken?: string
  claims: IdTokenClaims
}
```

The parts of JumpCloud's answer the app uses. `toTokens()` builds them.

- `sub`: the user's permanent, unique ID at JumpCloud ("subject").
- `exp: number`: when the ID token expires, in Unix seconds. `AuthManager` uses it to decide whether a spare is still fresh enough.
- `idToken: string`: the raw JWT, exactly as JumpCloud signed it, because that's what Anthropic needs.
- `refreshToken?: string`: optional, because a refresh answer may leave it out when JumpCloud doesn't rotate it.

`PendingSignIn`, `Oidc`

src/main/auth/oidc.ts, lines 23–35

```ts
/** A sign-in started in the browser. */
export interface PendingSignIn {
  url: string
  /** Completes the sign-in from the URL the browser was sent back to. */
  finish(callbackUrl: string): Promise<OidcTokens>
}

/** The identity-provider calls the app needs. The real one wraps openid-client; tests use fakes. */
export interface Oidc {
  begin(redirectUri: string): Promise<PendingSignIn>
  refresh(refreshToken: string): Promise<OidcTokens>
  revoke(refreshToken: string): Promise<void>
}
```

The interface `AuthManager` sees.

- `PendingSignIn`: a sign-in in progress. `url` is the address to open in the browser. `finish(callbackUrl)` completes it with the address the browser came back to; it remembers the secret values from `begin()`, so they never pass through `AuthManager`.
- `begin(redirectUri: string)`: starts a sign-in that will come back to `redirectUri` (the loopback listener's address).
- `refresh(refreshToken: string)`: new tokens without the browser. Throws `SessionExpiredError` if JumpCloud refuses.
- `revoke(refreshToken: string)`: asks JumpCloud to stop accepting a refresh token (Log out).

`SessionExpiredError`

src/main/auth/oidc.ts, lines 37–42

```ts
/** JumpCloud refused a saved sign-in (expired, revoked, or the user lost access). */
export class SessionExpiredError extends Error {
  constructor(cause?: unknown) {
    super('Your JumpCloud sign-in has expired', { cause })
  }
}
```

Thrown by `refresh()` when JumpCloud answered "no" (`isRefusal()`), as opposed to not answering. `AuthManager.refresh()` treats it as the end of the sign-in; any other error keeps the sign-in.

`createOidc()`

src/main/auth/oidc.ts, lines 44–113

```ts
/**
 * OpenID Connect with JumpCloud, using openid-client: Authorization Code flow with PKCE as a
 * public client (no client secret; the code verifier proves the app that started the sign-in is
 * the one finishing it), plus state and nonce checks. openid-client checks the ID token's issuer,
 * audience, expiry and nonce. It doesn't check the signature: the token comes straight from
 * JumpCloud over HTTPS, which OpenID Connect allows, and Anthropic verifies the signature when
 * the token is swapped for Claude access.
 */
export function createOidc(config: SignInConfig, options: { allowInsecure?: boolean } = {}): Oidc {
  // Discovery reads the issuer's endpoints once; a failed attempt is retried next time.
```

```
  let discovered: Promise<client.Configuration> | null = null
  const configuration = (): Promise<client.Configuration> => {
    discovered ??= client
      .discovery(new URL(config.issuer), config.clientId, undefined, client.None(), {
        // Plain http is only ever allowed for a local test issuer in dev runs.
        execute: options.allowInsecure ? [client.allowInsecureRequests] : [],
        timeout: 15,
      })
      .catch((error: unknown) => {
        discovered = null
        throw error
      })
    return discovered
  }

  return {
    async begin(redirectUri) {
      const configured = await configuration()
      const codeVerifier = client.randomPKCECodeVerifier()
      const state = client.randomState()
      const nonce = client.randomNonce()
      const url = client.buildAuthorizationUrl(configured, {
        redirect_uri: redirectUri,
        scope: SCOPES,
        code_challenge: await client.calculatePKCECodeChallenge(codeVerifier),
        code_challenge_method: 'S256',
        state,
        nonce,
      })
      return {
        url: url.href,
        finish: async (callbackUrl) =>
          toTokens(
            await client.authorizationCodeGrant(configured, new URL(callbackUrl), {
              pkceCodeVerifier: codeVerifier,
              expectedState: state,
              expectedNonce: nonce,
              idTokenExpected: true,
            }),
          ),
      }
    },

    async refresh(refreshToken) {
      const configured = await configuration()
      try {
        return toTokens(await client.refreshTokenGrant(configured, refreshToken))
      } catch (error) {
        if (isRefusal(error)) throw new SessionExpiredError(error)
        throw error
      }
    },

    async revoke(refreshToken) {
      const configured = await configuration()
      if (!configured.serverMetadata().revocation_endpoint) return
      await client.tokenRevocation(configured, refreshToken, { token_type_hint: 'refresh_token' })
    },
  }
}
```

The real `Oidc`, built on openid-client. It returns an object with the three methods; they share the discovery result through the closure (the local variables they can see).

Discovery (`discovered` and `configuration`):

- `let discovered: Promise<client.Configuration> | null = null`: the cached discovery. It caches the promise, not the result, so callers that arrive while discovery is still running wait for the same request instead of starting another.
- `discovered ??= client`: `??=` assigns only when the variable is `null`, so discovery runs on first use (not at startup) and only once.
- `config.clientId, undefined, client.None()`: the client ID, no extra client settings, and no client authentication. `None()` is how a public client is set up: token requests send just the client ID, with no secret.
- `execute: options.allowInsecure ? [client.allowInsecureRequests] : []`: openid-client refuses plain `http` unless told otherwise. `index.ts` only sets `allowInsecure` in dev runs whose issuer is on this PC (a test sign-in server).
- `timeout: 15`: seconds. openid-client also uses it for every later request made with this configuration, so a hung JumpCloud fails after 15 seconds instead of the default 30.
- `discovered = null`: a failed discovery (say, offline at startup) isn't kept, so the next call tries again rather than failing forever.

`begin(redirectUri)`:

- `client.randomPKCECodeVerifier()`, `client.randomState()`, `client.randomNonce()`: new random values for every sign-in. They live only in this function's variables.
- `client.buildAuthorizationUrl(configured, {`: JumpCloud's authorization endpoint (from discovery) with these settings added as query parameters. openid-client also adds `client_id` and `response_type=code`.
- `code_challenge: await client.calculatePKCECodeChallenge(codeVerifier)`: the SHA-256 hash of the verifier, which is all the browser sees. `code_challenge_method: 'S256'` tells JumpCloud which hash was used.
- `finish: async (callbackUrl) =>`: an arrow function that still has the verifier, state and nonce from this call.
- `client.authorizationCodeGrant(configured, new URL(callbackUrl), {`: does the second half. It throws `AuthorizationResponseError` if the callback carries an `error` instead of a code, checks `state`, posts the code with the verifier to the token endpoint, and checks the ID token, including the nonce. The `redirect_uri` it sends is the callback address without its query, which must match the registered one exactly; see `listenForRedirect()`.
- `idTokenExpected: true`: an answer without an ID token is an error.

`refresh(refreshToken)`:

- `client.refreshTokenGrant(configured, refreshToken)`: the refresh-token request. The new ID token gets the same checks as at sign-in, except the nonce.
- `if (isRefusal(error)) throw new SessionExpiredError(error)`: JumpCloud said no, so the sign-in is over. Every other error is passed on unchanged, and `AuthManager.refresh()` treats it as "couldn't reach JumpCloud".

`revoke(refreshToken)`:

- `if (!configured.serverMetadata().revocation_endpoint) return`: not every provider offers revocation (discovery says whether it does), so without one there is nothing to do.
- `token_type_hint: 'refresh_token'`: tells JumpCloud what kind of token it's being sent, which helps it find it.

`TokenResponse`, `toTokens()`

src/main/auth/oidc.ts, lines 115–130

```ts
type TokenResponse = Awaited<ReturnType<typeof client.refreshTokenGrant>>

function toTokens(response: TokenResponse): OidcTokens {
  const claims = response.claims()
  if (!response.id_token || !claims) throw new Error('JumpCloud did not return an ID token')
  return {
    idToken: response.id_token,
    refreshToken: response.refresh_token,
    claims: {
      sub: claims.sub,
      email: typeof claims.email === 'string' ? claims.email : undefined,
      name: typeof claims.name === 'string' ? claims.name : undefined,
      exp: claims.exp,
    },
  }
}
```

Turns openid-client's token answer into the app's `OidcTokens`, keeping only what's used. JumpCloud's access token is ignored, because the app never calls a JumpCloud API.

- `Awaited<ReturnType<typeof client.refreshTokenGrant>>`: the type of what openid-client's token functions resolve to, taken from the library rather than written out by hand. `ReturnType` gives the function's return type (a promise) and `Awaited` unwraps the promise.
- `response.claims()`: the ID token's contents, already checked by openid-client; `undefined` when there's no ID token.
- `if (!response.id_token || !claims)`: without an ID token there's nothing to give Anthropic. During a refresh this error is treated as "couldn't reach JumpCloud".
- `typeof claims.email === 'string' ? claims.email : undefined`: a JWT's claims can hold any JSON, so only real strings are accepted.

`describeSignInError()`

src/main/auth/oidc.ts, lines 132–143

```ts
/** Why a browser sign-in failed, in words the user can act on. */
export function describeSignInError(error: unknown): string {
  if (error instanceof client.AuthorizationResponseError) {
    return error.error_description ?? `JumpCloud said "${error.error}".`
  }
  // fetch failures are TypeErrors; openid-client reports its timeouts with code OAUTH_TIMEOUT.
  const timedOut = error instanceof client.ClientError && error.code === 'OAUTH_TIMEOUT'
  if (error instanceof TypeError || timedOut) {
    return "Couldn't reach JumpCloud. Check your connection."
  }
  return error instanceof Error ? error.message : String(error)
}
```

Explains a failed browser sign-in in words the user can act on. Used only by `signInFailure()` in `AuthManager.ts`.

- `error instanceof client.AuthorizationResponseError`: JumpCloud sent the browser back with an error instead of a code, most often because the user isn't in a group assigned to the app. `error_description` is JumpCloud's own explanation; if there isn't one, the short error code is shown.

- `error instanceof client.ClientError && error.code === 'OAUTH_TIMEOUT'`: a request to JumpCloud that took longer than the 15-second `timeout` set in `createOidc()`. openid-client doesn't pass on the `TimeoutError` that `fetch` raises: it wraps it in its own `ClientError` with the code `OAUTH_TIMEOUT`, so that's what is checked.
- `error instanceof TypeError || timedOut`: `TypeError` is what Node's `fetch` throws ("fetch failed") when it can't connect at all, for example offline or a DNS failure. Both cases get "Couldn't reach JumpCloud. Check your connection."
- `error instanceof Error ? error.message : String(error)`: anything else shows its own message, for example the "Port … is in use" error from `loopback.ts`.

`isRefusal()`

`src/main/auth/oidc.ts`, lines 145–153

```
/** JumpCloud answered, and said no (as opposed to not being reachable). */
function isRefusal(error: unknown): boolean {
  return (
    error instanceof client.ResponseBodyError &&
    ['invalid_grant', 'invalid_client', 'unauthorized_client', 'access_denied'].includes(
      error.error,
    )
  )
}
```

Tells "JumpCloud said no" apart from "JumpCloud didn't answer".

- `error instanceof client.ResponseBodyError`: JumpCloud's token endpoint answered with a standard OAuth error (`{"error": "..."}`), so it was reachable.
- `'invalid_grant'`: the refresh token is expired, revoked or already replaced. `'invalid_client'` and `'unauthorized_client'`: the app's JumpCloud registration was removed or changed. `'access_denied'`: the user lost access. In every case only signing in again can help.

Other error codes, such as `server_error` or `temporarily_unavailable`, aren't refusals: the sign-in is kept and tried again later.

`src/main/auth/loopback.ts`: catching the browser's return

After the user signs in, JumpCloud has to send the browser somewhere the app can see the `code`. A desktop app has no website, so it uses a **loopback redirect** (RFC 8252, the standard for native apps): for each sign-in, the app runs a tiny web server on `127.0.0.1`, the PC's address for itself, which other computers can't reach. JumpCloud is registered with `http://127.0.0.1:47621/callback` as the redirect URI. Plain `http` is fine because the traffic never leaves the PC, and PKCE protects the code even if another program on the PC sees it. `index.ts` passes `listenForRedirect` to `AuthManager` as `listen`, and `signIn()` starts one listener per sign-in. It uses Node's built-in `http` module, which works in Electron's main process. The Apps list's connection to JumpCloud uses the same listener, on port 47622 (see Your apps), and so does the connection to Atlassian, on the tenant's `atlassian.redirectPort` (47623 for Morse Micro; see Jira and the service desk).

`CALLBACK_PATH`, `RedirectListener`

src/main/auth/loopback.ts, lines 3–13

```ts
export const CALLBACK_PATH = '/callback'

/** Waiting for the browser to come back from the sign-in page. */
export interface RedirectListener {
  /** What JumpCloud must send the browser back to (registered in the JumpCloud app). */
  redirectUri: string
  /** The full URL the browser came back to, including `code` and `state`. */
  result: Promise<string>
  /** Stops waiting (cancel); `result` then rejects with SignInCancelledError. */
  close(): void
}
```

- `CALLBACK_PATH`: the only path the server answers.
- `redirectUri`: the full address to give JumpCloud. `oidc.ts` puts it in the sign-in address, and JumpCloud only accepts it if it exactly matches the one registered in the JumpCloud app.
- `result: Promise<string>`: settles once: with the address the browser came back to, or with an error for cancel or timeout.
- `close()`: cancels the wait. `AuthManager` calls it from `cancelSignIn()` and always when a sign-in ends.

`SignInCancelledError`, `SignInTimeoutError`

src/main/auth/loopback.ts, lines 15–25

```ts
export class SignInCancelledError extends Error {
  constructor() {
    super('Sign-in was cancelled')
  }
}

export class SignInTimeoutError extends Error {
  constructor() {
    super('Sign-in took too long')
  }
}
```

The two ways `result` can fail. `signInFailure()` in `AuthManager.ts` shows nothing for a cancel and "Sign-in timed out. Try again." for a timeout.

`listenForRedirect()`

src/main/auth/loopback.ts, lines 27–81

```ts
/**
 * Starts a one-shot web server on 127.0.0.1 that catches the browser's redirect after sign-in
 * (the "loopback" method for desktop apps, RFC 8252). It only listens on the local machine,
 * answers the browser with a short page saying it can close the tab, and shuts down after the
 * first callback, a cancel, or the timeout.
 */
export function listenForRedirect(port: number, timeoutMs = 5 * 60_000): Promise<RedirectListener> {
  return new Promise((resolveListener, rejectListener) => {
    let settle: { resolve: (url: string) => void; reject: (error: Error) => void }
    const result = new Promise<string>((resolve, reject) => (settle = { resolve, reject }))
    result.catch(() => {}) // a rejection nobody awaited yet (cancel before the caller waits)
```

```
    let timer: ReturnType<typeof setTimeout> | undefined
    const server: Server = createServer((req, res) => {
      const url = new URL(req.url ?? '/', `http://127.0.0.1:${port}`)
      if (req.method !== 'GET' || url.pathname !== CALLBACK_PATH) {
        res.writeHead(404).end()
        return
      }
      const error = url.searchParams.get('error_description') ?? url.searchParams.get('error')
      res.writeHead(200, {
        'content-type': 'text/html; charset=utf-8',
        'cache-control': 'no-store',
      })
      // Shut down only once the page has been sent.
      res.end(callbackPage(error), () => finish(() => settle.resolve(url.href)))
    })

    let done = false
    const finish = (then: () => void) => {
      if (done) return
      done = true
      clearTimeout(timer)
      server.close()
      server.closeAllConnections()
      then()
    }

    server.once('error', (error: NodeJS.ErrnoException) => {
      rejectListener(
        error.code === 'EADDRINUSE'
          ? new Error(`Port ${port} is in use by another program, so sign-in can't finish.`)
          : error,
      )
    })
    server.listen(port, '127.0.0.1', () => {
      timer = setTimeout(() => finish(() => settle.reject(new SignInTimeoutError())), timeoutMs)
      resolveListener({
        redirectUri: `http://127.0.0.1:${port}${CALLBACK_PATH}`,
        result,
        close: () => finish(() => settle.reject(new SignInCancelledError())),
      })
    })
  })
}
```

Starts the one-shot server and returns once it's actually listening. That lets `signIn()` be sure the port is open before it opens the browser.

- `return new Promise((resolveListener, rejectListener) =>`: the outer promise resolves with the `RedirectListener` once the server is listening, or rejects if it can't start.
- `const result = new Promise<string>((resolve, reject) => (settle = { resolve, reject }))`: creates `result` and keeps its `resolve` and `reject` functions in `settle`, so the request handler, the timer and `close()` can settle it later. (The function given to `new Promise` runs immediately, so `settle` is set straight away.)
- `result.catch(() => {})`: if `close()` is called before anyone waits on `result` (a cancel during `begin()`, say), Node would report an unhandled rejection. This empty handler prevents that; anyone who awaits `result` later still gets the error.
- `new URL(req.url ?? '/', `http://127.0.0.1:${port}`)`: `req.url` is only the path and query (`/callback?code=…`), so the base is added to get the full address. It's built with exactly the registered host and port because openid-client takes the `redirect_uri` it sends to JumpCloud from this address.

- `if (req.method !== 'GET' || url.pathname !== CALLBACK_PATH)`: anything else, such as the browser asking for `/favicon.ico`, gets a 404 and doesn't end the wait.
- `url.searchParams.get('error_description') ?? url.searchParams.get('error')`: if JumpCloud sent back an error, the page says so. The address is still handed on, and `finish()` in `oidc.ts` turns it into the proper error.
- `'cache-control': 'no-store'`: the browser shouldn't keep a copy of a page whose address holds a one-time code.
- `res.end(callbackPage(error), () => finish(() => settle.resolve(url.href)))`: the callback runs once the page has been sent, so the server isn't shut down halfway through the reply.
- `if (done) return`: only the first of the callback, the timeout and `close()` counts; later ones do nothing. That's why `AuthManager` can call `close()` after a sign-in has already finished.
- `server.close()`: stops accepting new connections. `server.closeAllConnections()` also drops connections the browser is keeping open for reuse, which would otherwise keep the server alive.
- `error.code === 'EADDRINUSE'`: another program already uses the port, so starting fails with an explanation (IT can pick another port; see `docs/JUMPCLOUD_SETUP.md`).
- `server.listen(port, '127.0.0.1', () => {`: listening on `127.0.0.1` only (not all network interfaces) keeps the server invisible to other computers. The callback runs once it's listening: it starts the timeout (5 minutes by default) and hands back the listener.

`callbackPage()`, `escapeHtml()`

src/main/auth/loopback.ts, lines 83–94

```
function callbackPage(error: string | null): string {
  const message = error
    ? `Sign-in didn't complete: ${escapeHtml(error)}`
    : 'You're signed in to Desktop Assist. You can close this tab.'
  return `<!doctype html><html><head><meta charset="utf-8"><title>Desktop Assist</title>
<style>body{font-family:'Segoe UI',system-ui,sans-serif;display:grid;place-
items:center;height:100vh;margin:0;color:#18181b}
p{font-size:16px}</style></head><body><p>${message}</p><script>setTimeout(()=>window.close(),1500)
</script></body></html>`
}

function escapeHtml(text: string): string {
  return text.replace(/[&<>"']/g, (c) => `&#${c.charCodeAt(0)};`)
}
```

The small page the browser shows after the redirect.

- `escapeHtml(error)`: the error text comes from the address, which anyone could make up, so it's escaped before going into the HTML. Otherwise a crafted link could run its own script on a page served from this PC.
- `setTimeout(()=>window.close(),1500)`: tries to close the tab after 1.5 seconds. Browsers usually only let a script close a tab that a script opened, so the tab may well stay open; the text tells the user they can close it.
- `text.replace(/[&<>"']/g, (c) => `&#${c.charCodeAt(0)};`)`: replaces each character that means something in HTML with its numeric code, for example `<` with `<`.

`src/main/auth/idToken.ts`: describing an ID token for the log

When Anthropic refuses to swap an ID token, the app only gets a short error back, and the token itself must never go in a log file: until it expires, it proves who the user is. This file reads what the token says (the same things a federation rule checks) so the log can show those instead. It only decodes the token and never verifies it, which is fine because nothing trusts the result: it's only for a person reading the log.

It's used only by `startBuiltInChat()` in `index.ts`. The `identityToken` function given to `AnthropicBackend` summarizes every ID token before handing it over, and keeps the latest summary in `lastIdToken`. When a chat request fails and `classifyError()` (in `src/main/claude/errors.ts`) says `not-allowed`, meaning Anthropic's token exchange refused the swap (a status such as 400, 401 or 403), `ChatSession`'s `onError` writes that summary to the app's log (`logs\desktop-assist.log` in the app's data folder).

The case it was written for, and what to look for:

- **Audience.** JumpCloud's ID token has `aud` as a list that contains the JumpCloud client ID. The federation rule's *Expected audience* in the Claude Console must be set to that client ID. When it's left blank, Anthropic instead requires the token's `aud` to be `https://api.anthropic.com`, which a JumpCloud token never is, and refuses with the deny reason `jwt_audience_mismatch` (shown in the Console's authentication history).
- **Organization ID.** If the organization ID in tenant.json's `claudeAccess` is wrong (for example the claude.ai organization ID rather than the Claude Console's, from **Settings** → **Organization**), Anthropic can't find the rule at all and records no authentication event. So when the summary looks right but the Console shows nothing, check the IDs in tenant.json rather than the token.

A JWT is three parts joined by dots: a *header* (how it was signed), a *payload* (the claims) and the *signature*. The first two are JSON encoded with base64url, so anyone can read them; only the signature needs JumpCloud's key to check.

`IdTokenSummary`

`src/main/auth/idToken.ts`, lines 1–19

```ts
/**
 * What an ID token says, for the log, without the token itself (which could be swapped for
 * Claude access) or its signature. These are the things an Anthropic federation rule checks.
 */
export interface IdTokenSummary {
  alg?: string
  kid?: string
  iss?: string
  aud?: string | string[]
  sub?: string
  email?: string
  issuedAt?: string
  expiresAt?: string
  /** `exp` minus `iat`: Anthropic refuses tokens that live longer than the issuer allows. */
  lifetimeSeconds?: number
  hasJti: boolean
  /** Every claim name in the token, to see what a rule could match on. */
  claims: string[]
}
```

What gets logged. Every field is optional except `hasJti` and `claims`, because a token may lack any claim.

- `alg?: string`: the signing algorithm from the header (for example RS256), which Anthropic uses to check the signature.
- `kid?: string`: the key ID from the header: which of JumpCloud's published keys signed the token.
- `iss?: string`: the issuer. It must match the issuer URL in the Claude Console exactly, including the trailing slash.
- `aud?: string | string[]`: the audience. JWTs allow either one string or a list, and JumpCloud sends a list.
- `issuedAt?: string`, `expiresAt?: string`: the `iat` and `exp` claims as readable dates rather than Unix seconds.
- `hasJti: boolean`: only whether the token has a `jti` (the ID Anthropic uses to refuse a second swap of the same token), not its value.
- `claims: string[]`: the names of every claim, not their values, so it's clear what a rule's condition could test (for example whether there's an `email` claim).

`summarizeIdToken()`

src/main/auth/idToken.ts, lines 21–46

```
/** Decodes (without verifying) a JWT's header and claims into a summary. */
export function summarizeIdToken(jwt: string): IdTokenSummary | { error: string } {
  const parts = jwt.split('.')
  if (parts.length !== 3) return { error: 'not a JWT' }
  try {
    const [header, payload] = parts.slice(0, 2).map(decodePart)
    if (!header || !payload) return { error: 'not a JWT' }
    const iat = typeof payload.iat === 'number' ? payload.iat : undefined
    const exp = typeof payload.exp === 'number' ? payload.exp : undefined
    return {
      alg: text(header.alg),
      kid: text(header.kid),
      iss: text(payload.iss),
      aud: Array.isArray(payload.aud) ? payload.aud.map(String) : text(payload.aud),
      sub: text(payload.sub),
      email: text(payload.email),
      issuedAt: iat === undefined ? undefined : new Date(iat * 1000).toISOString(),
      expiresAt: exp === undefined ? undefined : new Date(exp * 1000).toISOString(),
      lifetimeSeconds: iat === undefined || exp === undefined ? undefined : exp - iat,
      hasJti: typeof payload.jti === 'string',
      claims: Object.keys(payload).sort(),
    }
  } catch (error) {
    return { error: `couldn't decode: ${error instanceof Error ? error.message : String(error)}` }
  }
}
```

Turns a raw ID token into an `IdTokenSummary`, or `{ error }` with a short reason if it can't be read.

- `if (parts.length !== 3) return { error: 'not a JWT' }`: a signed JWT always has exactly three parts.
- `parts.slice(0, 2).map(decodePart)`: decodes only the header and the payload. The signature (the third part) is never decoded or logged.
- `if (!header || !payload)`: a part decoded to something other than a JSON object.
- `typeof payload.iat === 'number' ? payload.iat : undefined`: the claims are untyped JSON, so each one is checked before use.
- `Array.isArray(payload.aud) ? payload.aud.map(String) : text(payload.aud)`: keeps a list of audiences as a list. This is what showed that JumpCloud sends `aud` as a list.

- `new Date(iat * 1000).toISOString()`: JWT times are Unix seconds and `Date` wants milliseconds; the result looks like `2026-10-08T05:12:00.000Z` (UTC).
- `lifetimeSeconds: iat === undefined || exp === undefined ? undefined : exp - iat`: how long the token is valid for in total. Anthropic refuses a token that lives longer than the issuer's maximum (1 hour by default).
- `Object.keys(payload).sort()`: the claim names in alphabetical order, easy to scan.
- `catch (error)`: a part that isn't valid base64url JSON gives `{ error: "couldn't decode: …" }`. The function never throws, which matters because it runs on every token before it goes to Anthropic: an exception here would break chat.

`decodePart()`, `text()`

src/main/auth/idToken.ts, lines 48–56

```
function decodePart(part: string | undefined): Record<string, unknown> | null {
  if (!part) return null
  const value: unknown = JSON.parse(Buffer.from(part, 'base64url').toString('utf8'))
  return value && typeof value === 'object' ? (value as Record<string, unknown>) : null
}

function text(value: unknown): string | undefined {
  return typeof value === 'string' ? value : undefined
}
```

- `JSON.parse(Buffer.from(part, 'base64url').toString('utf8'))`: base64url is base64 with `-` and `_` in place of `+` and `/` and no `=` padding, so it's safe in URLs. Node's `Buffer` decodes it directly, `toString('utf8')` turns the bytes into text, and `JSON.parse` reads that. Text that isn't JSON makes `JSON.parse` throw, which `summarizeIdToken()` catches.
- `value && typeof value === 'object'`: valid JSON could also be a number, a string or `null`, none of which can hold claims.
- `text`: returns the value only if it's a string, so a claim of an unexpected type is left out of the summary.

`src/main/storage/SecretStore.ts`: keeping the sign-in safe

Keeps one secret in one file, encrypted so only the same Windows user on the same PC can read it. The app has three such secrets, each in a store of its own: the built-in chat's JumpCloud sign-in (`AuthManager` saves the refresh token and user as JSON in `%APPDATA%\Desktop Assist\jumpcloud-session.bin`), the Apps list's connection to JumpCloud (`jumpcloud-apps.bin`, see Your apps), and the connection to Atlassian (`atlassian.bin`, see Jira and the service desk). The encryption itself is passed in as an `Encryptor`: `index.ts` gives it Electron's `safeStorage`, which on Windows uses **DPAPI** (the Windows Data Protection API, which encrypts with a key tied to the user's Windows login). Tests use a fake.

`Encryptor`

src/main/storage/SecretStore.ts, lines 4–9

```ts
/** Encrypts secrets at rest. In the app this is Electron's safeStorage (Windows DPAPI). */
export interface Encryptor {
  isAvailable(): boolean
  encrypt(text: string): Buffer
  decrypt(data: Buffer): string
}
```

What `SecretStore` needs to encrypt. `safeStorageEncryptor` in `index.ts` maps the three methods onto `safeStorage.isEncryptionAvailable()`, `encryptString()` and `decryptString()`.

- `encrypt(text: string): Buffer`: the result is raw bytes (a Node `Buffer`), not text.
- `decrypt(data: Buffer): string`: throws if the bytes can't be decrypted, for example a file encrypted by another Windows user.

`SecretStore`

The class, with the comment that describes it:

src/main/storage/SecretStore.ts, lines 11–15

```ts
/**
 * Keeps one secret in a file, encrypted so only the signed-in Windows user can read it. Used for
 * the JumpCloud sign-in (its refresh token), which never leaves the main process.
 */
export class SecretStore {
  // …
}
```

As with `AuthManager`, this block shows only the class's description and first line; the members follow below, starting with the constructor, which declares the class's fields:

src/main/storage/SecretStore.ts, lines 16–19

```ts
constructor(
  private readonly filePath: string,
  private readonly encryptor: Encryptor,
) {}
```

The class's two fields, declared with the same constructor shorthand as `AuthManager`: the file to use (`jumpcloud-session.bin` in the app's data folder) and the `Encryptor`.

`SecretStore.load()`

src/main/storage/SecretStore.ts, lines 21–35

```
/** The saved secret, or null if there is none or it can't be decrypted (e.g. another user's). */
async load(): Promise<string | null> {
  let data: Buffer
  try {
    data = await fs.readFile(this.filePath)
  } catch (err) {
    if (isErrno(err, 'ENOENT')) return null
    throw err
  }
  try {
    return this.encryptor.decrypt(data) || null
  } catch {
    return null
  }
}
```

Reads and decrypts the file. `AuthManager.loadSession()` calls it at startup.

- `fs.readFile(this.filePath)`: with no text encoding given, it returns the raw bytes.
- `isErrno(err, 'ENOENT')`: "no such file", meaning nothing is saved. Any other read error (for example access denied) is thrown, and `loadSession()` treats that as nothing saved too.
- `this.encryptor.decrypt(data) || null`: an empty secret counts as none.
- `catch {`: a file that can't be decrypted (copied from another user or PC, or damaged) is treated as no sign-in. The user signs in again, and the file is overwritten.

`SecretStore.save()`, `SecretStore.clear()`

src/main/storage/SecretStore.ts, lines 37–46

```
async save(secret: string): Promise<void> {
  if (!this.encryptor.isAvailable()) {
    throw new Error("Windows can't encrypt data on this PC, so the sign-in wasn't saved.")
  }
  await writeFileAtomic(this.filePath, this.encryptor.encrypt(secret))
}

async clear(): Promise<void> {
  await fs.rm(this.filePath, { force: true })
}
```

- `if (!this.encryptor.isAvailable())`: refuses to save rather than write the refresh token in plain text. `AuthManager.startSession()` logs the error, and the user stays signed in for this run only.
- `writeFileAtomic(this.filePath, this.encryptor.encrypt(secret))`: from `src/main/storage/jsonFile.ts`. It writes a temporary file and then renames it over the real one (retrying briefly if antivirus is holding the file), so a crash halfway through can't leave a half-written file that won't decrypt. It also creates the folder if needed.
- `fs.rm(this.filePath, { force: true })`: deletes the file; `force` means a file that's already gone isn't an error. Called by `AuthManager.endSession()`.

Talking to Claude

Everything that talks to the Claude API lives in four files in `src/main/claude`, all in the main process. `model.ts` holds the settings every request uses, `AnthropicBackend.ts` sends one request through Anthropic's SDK, `errors.ts` turns failures into plain English, and `ChatSession.ts` keeps the conversation and runs each reply. There is no API key: the SDK swaps the user's JumpCloud sign-in for a short-lived Claude token (see `AnthropicBackend.getClient()`).

This is the built-in chat, created by `startBuiltInChat()` in `index.ts`. When a tenant sends questions to Claude Desktop instead (see Claude Desktop), Desktop Assist doesn't call the Claude API at all, and none of this page's code runs.

What happens when you send a message

1. The panel sends IPC `assist:chat-send`. Its handler in `ipc.ts` turns the draft's screenshots into base64 JPEGs (`ScreenshotService.forClaude()`) and calls `ChatSession.send()`.
2. `send()` appends your turn to the conversation `history`, adds your message and an empty "streaming" reply to the chat, and calls `startTurn()`, which runs `runTurn()` in the background. `send()` returns `'sent'` straight away, and the IPC handler clears the draft.
3. `runTurn()` calls `AnthropicBackend.reply()` with the whole history, the system prompt from `buildSystemPrompt()` and the current effort setting.
4. `reply()` gets the SDK client from `getClient()` and calls `client.beta.messages.stream()`. Before the request goes out, the SDK needs a Claude token. The first time, and again shortly before each token expires, it calls `AuthManager.freshIdToken()` and swaps that JumpCloud ID token for a Claude token.
5. As the reply streams in, `onText` adds each piece of text to the reply, and `scheduleEmit()` sends it to the page (IPC `assist:chat-message`) at most once every 50 ms.
6. When the stream ends, `runTurn()` appends the complete reply to the history: every content block, thinking included, unchanged, unless another model took over partway through, in which case `replayContent()` first drops what the API says must not be sent back. If Claude asked for tools, `runTools()` runs them and the loop sends another request. Otherwise `finish()` marks the reply done, with a notice from `stopNotice()` if it was refused or cut short.
7. If anything fails, `classifyError()`, `errorMessage()` and `isRetryable()` decide what the chat shows and whether it offers Retry, and `errorReference()` adds Anthropic's request ID when there is one.

`src/main/claude/model.ts`: what every request uses

The settings that shape every request, kept in one file so the model or the request can be changed in one place. `AnthropicBackend.ts` reads the model, the fallback beta and the token limit; `index.ts` uses `API_BASE_URL` and `buildSystemPrompt()`; `ChatSession.ts` uses `MAX_TOOL_ROUNDS`; and `errors.ts` puts `CLAUDE_MODEL_NAME` in one of its messages.

`CLAUDE_MODEL`, `CLAUDE_MODEL_NAME`

src/main/claude/model.ts, lines 1–4

```
// How Desktop Assist calls Claude. Kept in one place so the model and request shape are easy to change.

export const CLAUDE_MODEL = 'claude-opus-5-5'

export const CLAUDE_MODEL_NAME = 'Claude Opus 5.5'
```

The model's ID, which goes in every request, and its human name, which only appears in the "can't use this model" error message (`errors.ts`).

- `'claude-opus-5-5'`: Claude Opus 5.5. On this model thinking is always on and can't be switched off; effort (see `AnthropicBackend.reply()`) is the only control over how much it thinks.

`FALLBACK_BETA`

src/main/claude/model.ts, lines 6–10

```
/**
 * Server-side refusal fallback, "default" mode: if Claude's safety classifiers decline a request,
 * the API re-runs it on Anthropic's recommended fallback model instead of returning a refusal.
 */
export const FALLBACK_BETA = 'server-side-fallback-2026-07-01'
```

Turns on the **server-side refusal fallback**. Claude Opus 5.5 runs safety classifiers alongside each request. If they decline a request, the reply normally just ends with `stop_reason: 'refusal'` and no proper answer. With the fallback on, the API instead re-runs the request on another model inside the same call, so the user still gets an answer. A `fallback` content block in the reply marks where the other model took over: first in the reply if the decline came before any output, or partway through if it came mid-reply (see `replayContent()` in `ChatSession.ts` for what that means for the history). After a fallback, later requests in the same conversation may go straight to the fallback model for about an hour.

`AnthropicBackend.reply()` turns it on with two request fields: `betas: [FALLBACK_BETA]`, which the SDK sends as an `anthropic-beta` header to unlock the feature (the date is the feature's version), and `fallbacks: 'default'`, which lets Anthropic pick the fallback model by the kind of refusal instead of the app naming one. For kinds of refusal with no recommended fallback model, or if the fallback model declines too, the reply still ends with `refusal`, and `stopNotice()` in `ChatSession.ts` shows "Claude declined to answer this."

`MAX_TOKENS`

src/main/claude/model.ts, lines 12–13

```
/** Replies stream, so a high ceiling costs nothing unless Claude actually writes that much. */
export const MAX_TOKENS = 64_000
```

The most Claude may write in one reply, its thinking included. A token is a short piece of text, a few characters long.

Replies are **streamed**: instead of one response at the end, the API sends the reply in small pieces as it is written. The chat can show text as it arrives, and a long reply never runs into an HTTP timeout. Only the tokens actually written are billed, so a high ceiling costs nothing by itself. A reply that does reach it ends with `stop_reason: 'max_tokens'`, which `stopNotice()` reports as cut short.

`API_BASE_URL`, `MAX_TOOL_ROUNDS`

src/main/claude/model.ts, lines 15–18

```
export const API_BASE_URL = 'https://api.anthropic.com'

/** Never resend a turn more than this many times in one tool-use loop. */
export const MAX_TOOL_ROUNDS = 10
```

- `API_BASE_URL`: Where requests go. `index.ts` passes it to `AnthropicBackend`, except that a dev run may replace it with the `ANTHROPIC_BASE_URL` environment variable to use a local test server. The token exchange uses the same address.
- `MAX_TOOL_ROUNDS`: A safety limit for `ChatSession.runTurn()`. When Claude uses tools, one reply takes several requests (see `ChatTool`); this stops a reply that never settles from looping forever.

`buildSystemPrompt()`

src/main/claude/model.ts, lines 20–36

```
/**
 * The system prompt, fixed for the whole conversation (changing it mid-conversation would throw
 * away the prompt cache). Tenants can add their own instructions in tenant.json.
 */
export function buildSystemPrompt(tenant: {
  appName: string
  companyName: string
  systemPrompt?: string
}): string {
  const base = [
    `You are ${tenant.appName}, a desktop assistant for people at ${tenant.companyName}.`,
    "You appear as a small chat panel in the corner of the user's Windows screen, so keep answers",
    'concise and easy to scan: short paragraphs, lists and code blocks where they help.',
    'The user can attach screenshots of their screen; when they do, use what you can see in them.',
  ].join(' ')
  return tenant.systemPrompt ? `${base}\n\n${tenant.systemPrompt}` : base
}
```

Builds the **system prompt**: standing instructions sent at the top of every request, separate from the conversation. It tells Claude who it is, that it lives in a small panel (so answers should be short and easy to scan), and that screenshots may be attached. `index.ts` calls it once at startup and gives the result to `ChatSession`, which sends the same text with every request.

- `systemPrompt?: string`: Optional extra instructions from `tenant.json` (up to 8,000 characters, checked by `TenantSchema`), added after the built-in text with a blank line between them. The parameter's type lists only the three fields used; `index.ts` passes the whole tenant object.
- `.join(' ')`: The four lines are one paragraph; they are split in the source only to keep lines short.

Why the prompt must never change during a conversation: **prompt caching**. The API remembers nothing between requests, so every request sends the whole conversation again. Prompt caching lets Anthropic keep the start of a recent request for a few minutes. When the next request begins with exactly the same content, that part is read from the cache, which is faster and much cheaper. The match is on the exact beginning of the request (tools, then the system prompt, then the messages), so changing even one character of the system prompt would make everything after it miss. That's why the prompt is built once and contains nothing that changes, such as the date or time.

`src/main/claude/AnthropicBackend.ts`: sending one request

The only file that calls the Anthropic SDK (`@anthropic-ai/sdk`). Its job is narrow: send one request and stream back the reply. It doesn't keep the conversation (that's `ChatSession`) and knows nothing about the UI. `index.ts` creates one `AnthropicBackend` at startup, gives it to `ChatSession`, and calls `reset()` when the user is signed out. `ChatSession` only sees the `ChatBackend` interface, so the tests replace the backend with a scripted fake. There is no API key: requests are made as the signed-in user (see the class comment below and `getClient()`).

`ReplyRequest`, `ChatBackend`

src/main/claude/AnthropicBackend.ts, lines 7–23

```ts
export interface ReplyRequest {
  system: string
  messages: Anthropic.Beta.BetaMessageParam[]
  tools: Anthropic.Beta.BetaTool[]
  effort: Effort
  signal: AbortSignal
  /** Called with each piece of reply text as it streams in. */
  onText: (delta: string) => void
}

/** Where chat requests go. Tests swap in a fake. */
export interface ChatBackend {
  /** Streams one reply and resolves with the complete message (all content blocks). */
  reply(request: ReplyRequest): Promise<Anthropic.Beta.BetaMessage>
  /** Forgets the current Claude access, e.g. after signing out. */
  reset(): void
}
```

`ReplyRequest` is everything one request needs. `ChatBackend` is the interface `ChatSession` talks to, which `AnthropicBackend` implements.

Content blocks. In the Claude API a message's `content` is a list of blocks, each with a `type`. A user turn here holds `image` blocks and a `text` block. A reply can hold `thinking` blocks (Claude's reasoning before it answers), `text` blocks (the answer), `tool_use` blocks (requests to run a tool) and, after a refusal fallback, a `fallback` block. The types come from the SDK's `Beta` namespace because the request uses a beta feature (`FALLBACK_BETA`).

- `messages: Anthropic.Beta.BetaMessageParam[]`: The whole conversation so far, one entry per turn. The API keeps nothing between requests, so every request sends all of it again.
- `tools: Anthropic.Beta.BetaTool[]`: The definitions of the tools Claude may call. Always empty for now.
- `signal: AbortSignal`: Lets `ChatSession` cancel the request (Stop, New conversation, quitting).
- `onText: (delta: string) => void`: Called with each new piece of answer text. Thinking is never passed here.
- `reply(request: ReplyRequest): Promise<Anthropic.Beta.BetaMessage>`: Resolves only once the reply is complete, with every content block, because `ChatSession` sends them back next time. Rejects if the request fails or is cancelled.

`AnthropicBackendOptions`

src/main/claude/AnthropicBackend.ts, lines 25–30

```ts
export interface AnthropicBackendOptions {
  baseURL: string
  access: ClaudeAccessConfig
  /** A new, unused JumpCloud ID token (each can only be exchanged once). */
  identityToken(): Promise<string>
}
```

What `index.ts` passes when it creates the backend.

- `baseURL: string`: `API_BASE_URL`, or a local test server in dev runs.
- `access: ClaudeAccessConfig`: The `claudeAccess` section of `tenant.json`: the Claude Console organization, federation rule and service account IDs, and an optional workspace ID. None of these is secret; they say which federation rule should check the user's ID token.
- `identityToken(): Promise<string>`: In the app this is `() => auth.freshIdToken()`. In `index.ts`, `auth` is declared a few lines after the backend; that works because the function only reads `auth` when the SDK calls it, long after startup has finished.

`AnthropicBackend`

The class, with the comment that describes it:

src/main/claude/AnthropicBackend.ts, lines 32–38

```ts
/**
 * Calls Claude as the signed-in user, with no API key. Anthropic's Workload Identity Federation
 * exchanges the user's JumpCloud ID token for a short-lived access token for the company's
 * service account. The SDK does the exchange, keeps the token, gets a new one shortly before it
 * expires, and retries once with a new one if Anthropic rejects it.
 */
export class AnthropicBackend implements ChatBackend {
  // …
}
```

In plain terms: a JumpCloud ID token is a short-lived, signed statement of who the user is. Anthropic accepts it in place of an API key because IT has set up a federation rule and a service account in the Claude Console (see `docs/JUMPCLOUD_SETUP.md`). `getClient()` below shows how the SDK is told to do this, and the steps it follows.

- `implements ChatBackend`: TypeScript checks that the class has everything the interface lists (`reply()` and `reset()`), so it can be passed wherever a `ChatBackend` is expected.

The fields:

src/main/claude/AnthropicBackend.ts, lines 39–41

```ts
private client: Anthropic | null = null

constructor(private readonly options: AnthropicBackendOptions) {}
```

- `private client: Anthropic | null = null`: The SDK client, created on first use by `getClient()`. The client also holds the current Claude token, so dropping it drops the token.
- `private readonly options`: TypeScript shorthand. `private readonly` on a constructor parameter declares a field and stores the argument in it, so the constructor body can stay empty.

`AnthropicBackend.reply()`

src/main/claude/AnthropicBackend.ts, lines 43–61

```
reply(request: ReplyRequest): Promise<Anthropic.Beta.BetaMessage> {
  const stream = this.getClient().beta.messages.stream(
    {
      model: CLAUDE_MODEL,
      max_tokens: MAX_TOKENS,
      betas: [FALLBACK_BETA],
      fallbacks: 'default',
      // Thinking is always on for this model; effort sets how much it does.
      output_config: { effort: request.effort },
      system: request.system,
      cache_control: { type: 'ephemeral' }, // cache the conversation so far
      messages: request.messages,
      ...(request.tools.length > 0 ? { tools: request.tools } : {}),
    },
    { signal: request.signal },
  )
  stream.on('text', (delta) => request.onText(delta))
  return stream.finalMessage()
}
```

Sends one request and streams back the reply. `ChatSession.runTurn()` calls it once per round.

- `this.getClient().beta.messages.stream(`: `beta.messages` is the Messages API with beta features allowed. `stream()` starts a streaming request and returns a stream object straight away; the reply arrives as events on it.
- `betas: [FALLBACK_BETA]`: Sent as the `anthropic-beta` header; together with `fallbacks: 'default'` it turns on the refusal fallback (see `FALLBACK_BETA`).
- `output_config: { effort: request.effort }`: **Effort** tells the model how much work to put in: how long it thinks and how thorough its answer is. The user picks it in the settings menu: Fast (`low`, the default in `settings.ts`), Balanced (`medium`) or Thorough (`high`). There is no `thinking` field: on this model thinking is always on, and Claude decides how much it needs. Changing effort mid-conversation is allowed, but the next request can't reuse the cached conversation.
- `cache_control: { type: 'ephemeral' }`: Prompt caching (see `buildSystemPrompt()`) for the whole request. At the top level of a request, it makes the API put a cache point on the last block, so everything up to there is kept. The next request repeats that content and adds one new turn, so everything before the new turn is read from the cache. An `'ephemeral'` entry lasts 5 minutes, and each use restarts the clock. Very short requests aren't cached at all (no error, just no saving).
- `...(request.tools.length > 0 ? { tools: request.tools } : {})`: Adds the `tools` field only when there are tools. None are registered yet, so today's requests have no `tools` field.
- `{ signal: request.signal }`: The second argument holds options for the HTTP request rather than the API. Aborting the signal cancels the request, and the stream then fails with the SDK's `APIUserAbortError`.
- `stream.on('text', (delta) => request.onText(delta))`: The `'text'` event fires for each new piece of answer text. Thinking arrives as separate `'thinking'` events, which nothing listens to.
- `return stream.finalMessage()`: A promise for the complete reply once the stream ends: all its content blocks, the `stop_reason` and the token usage. It rejects if the request failed, including when getting the Claude token failed.

Thinking blocks. Because thinking is on, a reply usually starts with one or more `thinking` blocks before its `text`. On Claude Opus 5.5 their text is empty by default (the reasoning isn't shown), but each carries a `signature` that ties it to this model and to the exact conversation it was produced in. They must be sent back unchanged in later

requests, which is why `ChatSession` keeps whole replies and never edits its history. (After a refusal fallback, the declining model's thinking blocks are the exception; see `replayContent()`.)

`AnthropicBackend.reset()`

src/main/claude/AnthropicBackend.ts, lines 63–65

```
reset(): void {
  this.client = null
}
```

Forgets the SDK client, and with it the Claude token it holds. `index.ts` calls it from `AuthManager`'s `onSignedOut`, so it runs on Log out and when JumpCloud ends the sign-in. Nothing keeps using access that belonged to a sign-in that has ended; the next `reply()` builds a new client, which asks for a new ID token.

`AnthropicBackend.getClient()`

src/main/claude/AnthropicBackend.ts, lines 67–85

```
private getClient(): Anthropic {
  const { baseURL, access, identityToken } = this.options
  this.client ??= new Anthropic({
    // null stops an API key or token in the environment from being used instead.
    apiKey: null,
    authToken: null,
    baseURL,
    credentials: oidcFederationProvider({
      identityTokenProvider: identityToken,
      organizationId: access.organizationId,
      federationRuleId: access.federationRuleId,
      serviceAccountId: access.serviceAccountId,
      workspaceId: access.workspaceId || undefined,
      baseURL,
      fetch,
    }),
  })
  return this.client
}
```

Creates the SDK client the first time it's needed and returns the same one after that.

- `this.client ??= new Anthropic({`: `??=` assigns only when `this.client` is `null` (or `undefined`). So the client is built on the first message, not at startup, when nobody may be signed in yet.
- `apiKey: null,`: Left out, the SDK would read `ANTHROPIC_API_KEY` (and `authToken` would read `ANTHROPIC_AUTH_TOKEN`) from the environment, and either would win over `credentials`. `null` means "none", so the app always calls Claude as the signed-in user, even on a PC where a developer has a key set.
- `credentials: oidcFederationProvider({`: The SDK's Workload Identity Federation helper. It returns a function that, each time it is called, gets an ID token from `identityTokenProvider` and swaps it for a Claude access token. The SDK client wraps that function in its token cache.
- `identityTokenProvider: identityToken,`: `AuthManager.freshIdToken()` in the app. A swap uses up the ID token and doesn't return a refresh token, so every new Claude token needs a new ID token.
- `workspaceId: access.workspaceId || undefined,`: An empty `workspaceId` in `tenant.json` becomes `undefined`, so it isn't sent at all, and Anthropic uses the federation rule's only workspace. It is needed only when the rule covers several workspaces.

- `fetch,`: The provider makes its own HTTP call to the token endpoint, so it needs the address (`baseURL` again) and a `fetch` function; this is the one built into Node.js.

How a JumpCloud sign-in becomes a Claude token (all inside the SDK, during the request):

1. Before each request, the client asks its token cache for a token.
2. If it has none (the first message, or after `reset()`), the cache calls the provider and the request waits.
3. The provider calls `AuthManager.freshIdToken()`. That hands out the unused ID token kept from signing in or from the startup renewal, or gets a new one from JumpCloud with the saved refresh token. If nobody is signed in, or JumpCloud ends the sign-in, it throws `SignInRequiredError`; if JumpCloud can't be reached, `SignInUnavailableError`.
4. The provider posts the ID token to `<baseURL>/v1/oauth/token` with the organization, federation rule, service account and (if set) workspace IDs. It refuses to send it over plain `http` unless the address is this PC (`localhost`, `127.0.0.1`, `::1`). Anthropic checks the ID token against the federation rule set up in the Claude Console and returns a short-lived access token and how long it lasts. A refusal or failure throws the SDK's `WorkloadIdentityError`.
5. The client sends the access token as `Authorization: Bearer …` with the chat request.
6. Later requests reuse it. With 30 seconds to 2 minutes left, the cache hands out the old token and fetches a new one in the background; with under 30 seconds left, the request waits for a new one. Requests that need a token at the same moment share one swap.
7. If the API still answers 401 (token not accepted), the client throws the token away, gets a new one, and retries the request once.

Any error from steps 3 or 4 fails the chat request before it is sent and comes out of `finalMessage()`, where `classifyError()` sorts it.

`src/main/claude/errors.ts`: what went wrong

Turns any error from a chat request into one of a fixed set of kinds, and each kind into a message for the chat, plus Anthropic's request ID when the error has one. Only `ChatSession.runTurn()` uses it. The full error still goes to the log through `onError` (printed in the terminal in dev runs, see `index.ts`); the user sees only the plain English and the reference.

By the time an error gets here, the SDK has already retried temporary failures that happened before the reply started arriving (no connection, and statuses 408, 409, 429 and 5xx), twice by default.

`ErrorKind`

`src/main/claude/errors.ts`, lines 6–20

```ts
export type ErrorKind =
  | 'aborted'
  | 'signed-out'
  | 'sign-in-unreachable'
  | 'not-allowed'
  | 'auth'
  | 'permission'
  | 'billing'
  | 'not-found'
  | 'rate-limit'
  | 'overloaded'
  | 'network'
  | 'bad-request'
  | 'server'
  | 'unknown'
```

The kinds of failure the app tells apart. It's a union of string literals: a value of this type must be one of these strings, and TypeScript checks every use.

- `'aborted'`: Stop was pressed, or the conversation was cleared.
- `'signed-out'`, `'sign-in-unreachable'`, `'not-allowed'`: Getting a Claude token failed: nobody is signed in (or the sign-in just expired), JumpCloud couldn't be reached, or Anthropic refused the swap.
- `'auth'`, `'permission'`, `'billing'`, `'not-found'`: The Claude API turned the request down: token not accepted (401), no access (403), out of credit (402), model not available (404).
- `'rate-limit'`, `'overloaded'`, `'network'`, `'server'`: Temporary trouble: too many requests (429), Anthropic busy (529), no connection or a timeout, or another server error (5xx).
- `'bad-request'`: The API rejected the request itself (400).

`classifyError()`

src/main/claude/errors.ts, lines 22–46

```
/** Sorts an error from a chat request into what it means for the user. Most specific first. */
export function classifyError(error: unknown): ErrorKind {
  if (error instanceof Anthropic.APIUserAbortError) return 'aborted'
  // The SDK wraps errors it doesn't know (like the sign-in ones) and keeps them as the cause.
  const cause = error instanceof Anthropic.AnthropicError ? error.cause : undefined
  if (error instanceof SignInRequiredError || cause instanceof SignInRequiredError) {
    return 'signed-out'
  }
  if (error instanceof SignInUnavailableError || cause instanceof SignInUnavailableError) {
    return 'sign-in-unreachable'
  }
  if (error instanceof WorkloadIdentityError) return exchangeErrorKind(error.statusCode)
  if (error instanceof Anthropic.APIConnectionError) return 'network' // includes timeouts
  if (error instanceof Anthropic.AuthenticationError) return 'auth'
  if (error instanceof Anthropic.PermissionDeniedError) return 'permission'
  if (error instanceof Anthropic.NotFoundError) return 'not-found'
  if (error instanceof Anthropic.RateLimitError) return 'rate-limit'
  if (error instanceof Anthropic.BadRequestError) return 'bad-request'
  if (error instanceof Anthropic.APIError) {
    if (error.status === 402) return 'billing'
    if (error.status === 529) return 'overloaded'
    if (error.status !== undefined && error.status >= 500) return 'server'
  }
  return 'unknown'
}
```

Sorts an error into an `ErrorKind`. Order matters: SDK error classes inherit from each other (a cancelled request and a connection failure are both `APIError`s too), so the specific checks come first and the general `APIError` check comes last.

- `error instanceof Anthropic.APIUserAbortError`: The request was cancelled through its signal by Stop or New conversation.
- `const cause = error instanceof Anthropic.AnthropicError ? error.cause : undefined`: An error that isn't one of the SDK's own, such as the sign-in errors from `AuthManager.freshIdToken()`, comes out of the stream wrapped in a plain `AnthropicError`, with the original kept as `cause`. So the sign-in checks look at both the error and its cause.
- `error instanceof SignInRequiredError || cause instanceof SignInRequiredError`: Nobody is signed in, or JumpCloud just ended the sign-in. `AuthManager` has already switched the chat box to Sign in with JumpCloud.
- `error instanceof WorkloadIdentityError`: The swap at `/v1/oauth/token` failed. This class is an `AnthropicError` itself, so it arrives unwrapped; `exchangeErrorKind()` sorts it by status code.
- `error instanceof Anthropic.APIConnectionError`: No answer at all: offline, the address couldn't be found, or a timeout (the SDK's timeout error is a subclass of this one).
- `error instanceof Anthropic.AuthenticationError`: A 401 even after the SDK's retry with a new token.
- `if (error instanceof Anthropic.APIError) {`: Any other API error, sorted by HTTP status. 402 and 529 have no class of their own in the SDK; 529 is Anthropic's "overloaded" status.

`exchangeErrorKind()`

`src/main/claude/errors.ts`, lines 48–54

```
/** Anthropic refused (or couldn't do) the swap of the JumpCloud sign-in for Claude access. */
function exchangeErrorKind(status: number | null): ErrorKind {
  if (status === null) return 'network'
  if (status === 429) return 'rate-limit'
  if (status >= 500) return 'server'
  return 'not-allowed' // 400/401/403: the federation rule doesn't accept this user
}
```

Sorts a failed token swap by the HTTP status Anthropic returned.

- `if (status === null) return 'network'`: No status means the swap never got an answer (it's also how the SDK reports a problem it finds before sending, such as an address that isn't `https`).
- `return 'not-allowed'`: A 400, 401 or 403: Anthropic got the ID token and said no, usually because the federation rule doesn't accept it. Only IT can fix that, so the message says to contact them, and it carries the request ID (see `errorReference()`).

In practice, `'not-allowed'` has appeared because the Console's federation rule had no **Expected audience**. An ID token's `aud` (audience) claim names who the token is for, and a JumpCloud ID token's audience is the app's JumpCloud client ID. With no Expected audience set, Anthropic requires `aud` to be `https://api.anthropic.com`, so every swap is refused. The fix is to set the rule's Expected audience to the JumpCloud client ID.

`isRetryable()`

`src/main/claude/errors.ts`, lines 56–67

```
/** Problems worth simply trying again. */
export function isRetryable(kind: ErrorKind): boolean {
  return (
    kind === 'signed-out' || // after signing back in
    kind === 'sign-in-unreachable' ||
    kind === 'rate-limit' ||
    kind === 'overloaded' ||
    kind === 'network' ||
    kind === 'server' ||
    kind === 'unknown'
  )
}
```

Whether the chat offers Retry for a failed reply: yes for problems that may go away on their own, and for `'unknown'` to be safe. No for anything that needs IT or a different message (not-allowed, auth, permission, billing, not-found, bad-request). `ChatSession` also withholds Retry when part of the reply was kept.

- `kind === 'signed-out' || // after signing back in`: The Retry button stays on the failed reply. `ChatSession.retry()` does nothing until the user has signed in again, then resends.

`MESSAGES`, `errorMessage()`

src/main/claude/errors.ts, lines 69–88

```ts
const MESSAGES: Record<ErrorKind, string> = {
  aborted: 'Stopped.',
  'signed-out': 'Sign in with JumpCloud to keep chatting.',
  'sign-in-unreachable': "Couldn't reach JumpCloud to confirm your sign-in. Check your connection.",
  'not-allowed': "Your JumpCloud account isn't set up to use Claude yet. Contact IT.",
  auth: "Claude didn't accept your sign-in. Try again, or log out and sign in again.",
  permission: "Your account doesn't have access to Claude. Contact IT.",
  billing: "Your organisation's Claude account has run out of credit. Contact IT.",
  'not-found': `Your organisation can't use ${CLAUDE_MODEL_NAME}. Contact IT.`,
  'rate-limit': 'Claude is getting too many requests right now. Try again in a moment.',
  overloaded: 'Claude is busy right now. Try again in a moment.',
  network: "Couldn't reach Claude. Check your internet connection.",
  'bad-request': "Claude couldn't process that message.",
  server: 'Claude had a problem answering. Try again.',
  unknown: 'Something went wrong talking to Claude.',
}

export function errorMessage(kind: ErrorKind): string {
  return MESSAGES[kind]
}
```

The text shown under a failed reply, one per kind.

- `Record<ErrorKind, string>`: An object with exactly one string for every kind. If a kind is added to `ErrorKind` without a message, TypeScript refuses to compile.
- `aborted: 'Stopped.'`: Never actually shown: `ChatSession.runTurn()` handles Stop itself, with a notice that depends on whether any text arrived. It's here because the `Record` needs every kind.
- `'not-found'`: The model is the only thing a chat request names that could be missing, so the message names it (`CLAUDE_MODEL_NAME`).
- `'not-allowed'`: Shown when Anthropic refuses the swap. `runTurn()` adds the request ID after it, so IT can look up the refused attempt.

`errorReference()`

src/main/claude/errors.ts, lines 90–98

```ts
/**
 * Anthropic's ID for the failed request, if it has one. Shown with the error so IT can find the
 * attempt (for a refused sign-in swap, in the Claude Console's Authentication history).
 */
export function errorReference(error: unknown): string | undefined {
  if (error instanceof WorkloadIdentityError) return error.requestId ?? undefined
  if (error instanceof Anthropic.APIError) return error.requestID ?? undefined
  return undefined
}
```

Anthropic's ID for the failed request, if the error carries one. `ChatSession.runTurn()` shows it after the message, as "(Reference: …)". The user can pass it to IT, who can find that exact attempt on Anthropic's side: for a refused token swap, in the Claude Console's Authentication history, which has more detail on why it was refused.

- `error.requestId ?? undefined`: A failed token swap (`WorkloadIdentityError`) keeps the `Request-Id` header from Anthropic's answer. It is `null` when there was no answer, and `??` turns that into `undefined` so the function has one "none" value.

- `error.requestID ?? undefined`: Any error the Claude API answered (`APIError`) keeps its `request-id` header. The two SDK classes spell the field differently (`requestId` and `requestID`).
- `return undefined`: Everything else never reached Anthropic, for example the sign-in errors (which arrive wrapped in a plain `AnthropicError`). A connection failure is an `APIError` with no ID, so the line above returns `undefined` for it.

`src/main/claude/ChatSession.ts`: one conversation

The conversation, and everything that happens during a reply: sending, streaming text to the page, the tool loop, Stop, errors and Retry. It knows nothing about Electron. `index.ts` creates it with the backend and callbacks (`onMessage` broadcasts IPC `assist:chat-message`, `onReset` broadcasts `assist:chat-reset`); the IPC handlers in `ipc.ts` call `send()`, `stop()`, `retry()` and `newConversation()`; and `getState()` in `index.ts` uses `list()` when a page loads. One `ChatSession` lasts for the whole run: New conversation empties it rather than replacing it.

It keeps two lists, because the API and the page need different things. `history` is exactly what is sent to the API: base64 image data and Claude's thinking blocks included. `messages` is what the chat shows: ids, statuses, notices and the attached screenshots' file names.

`MessageParam`, `ContentBlock`, `OutgoingImage`

src/main/claude/ChatSession.ts, lines 8–15

```ts
type MessageParam = Anthropic.Beta.BetaMessageParam

type ContentBlock = Anthropic.Beta.BetaContentBlock

/** A screenshot ready to send: base64 image data. */
export interface OutgoingImage {
  mediaType: 'image/jpeg' | 'image/png'
  data: string
}
```

- `type MessageParam = Anthropic.Beta.BetaMessageParam`: Short local names for two SDK types: one turn of the conversation as it is sent, and one content block of a reply (used by `replayContent()` and `runTools()`). The file imports only types from the SDK (`import type`); it never calls the SDK itself.
- `mediaType: 'image/jpeg' | 'image/png'`: The image format, which the API needs to know. `ScreenshotService.forClaude()` always produces JPEG, scaled so its longest side is at most 1,568 pixels.
- `data: string`: The image as base64 text, since the request body is JSON.

`ChatTool`

src/main/claude/ChatSession.ts, lines 17–21

```ts
/** A tool Claude may call. None are registered yet; the loop is ready for them. */
export interface ChatTool {
  definition: Anthropic.Beta.BetaTool
  run(input: unknown): Promise<string>
}
```

A tool Claude may call. **Tool use** lets Claude ask the app to do something, such as read a file, and then use the result. Each request lists the tools' definitions (name, description and input schema). If Claude wants one, its reply contains a `tool_use` block with the tool name and input and ends with `stop_reason: 'tool_use'`. The app runs the tool and sends the result back as a `tool_result` block in a new user turn, and Claude carries on from there. One answer can take several of these rounds. No tools are registered yet (`index.ts` passes none), so today every reply is a single round; `runTurn()` and `runTools()` are ready for them.

- `definition: Anthropic.Beta.BetaTool`: What's sent to Claude: name, description and input schema.
- `run(input: unknown): Promise<string>`: The tool's code. `input` is whatever Claude sent, so it is `unknown` and each tool must check it. The returned string becomes the `tool_result`.

ChatSessionDeps

src/main/claude/ChatSession.ts, lines 23–38

```ts
export interface ChatSessionDeps {
  backend: ChatBackend
  /** Whether someone is signed in, so a request can be made. */
  canChat(): boolean
  getEffort(): Effort
  /** Fixed for the session, so the cached prompt prefix stays valid. */
  system: string
  tools?: ChatTool[]
  /** A message was added or changed. */
  onMessage(message: ChatMessage): void
  onReset(): void
  /** A reply failed; gets the full error, for the log (the chat shows a plain-English message). */
  onError?(error: unknown): void
  /** Streamed text is sent to the UI at most this often. */
  emitIntervalMs?: number
}
```

What `index.ts` gives the session. Everything comes in through here rather than being imported, so the tests can drive the class with fakes.

- `canChat(): boolean`: `AuthManager.canChat` in the app: whether a sign-in exists.
- `getEffort(): Effort`: A function rather than a value, so each request uses the setting as it is at that moment.
- `system: string`: The prompt from `buildSystemPrompt()`. It never changes; see prompt caching there.
- `onMessage(message: ChatMessage): void`: Called with a copy whenever a message is added or changes, including every streaming update.
- `onError?(error: unknown): void`: `index.ts` logs it. The `?` makes it optional.
- `emitIntervalMs?: number`: How often streamed text may reach the page, 50 ms when not set. The tests set 0 to see every update.

SendStatus

src/main/claude/ChatSession.ts, line 40

```ts
export type SendStatus = 'sent' | 'busy' | 'empty' | 'signed-out'
```

What `send()` returns. The `assist:chat-send` handler in `ipc.ts` passes anything other than `'sent'` back to the page as the reason (`SendResult`), and only clears the draft on `'sent'`.

`ChatSession`

The class, with the comment that describes it:

src/main/claude/ChatSession.ts, lines 42–52

```
/**
 * One conversation with Claude.
 *
 * It keeps two lists: `history`, exactly what is sent to the API, and `messages`, what the chat
 * shows. History is append-only: earlier turns are never edited or removed, because Claude's
 * thinking is tied to the exact conversation it was produced in. Every reply is appended with all
 * of its content blocks (thinking included) unchanged, except what the API says to drop after a
 * server-side fallback (`replayContent()`); a reply cut short by Stop or an error is appended as
 * plain text.
 */
export class ChatSession {
  // …
}
```

Why thinking forces an append-only history: each thinking block's `signature` records the conversation it was produced in, meaning the system prompt, the tools and every earlier message. When the history comes back, the API checks that this conversation is unchanged, and if an earlier turn has been edited or removed it can reject the request. Append-only also keeps prompt caching working, because each request starts with exactly what the previous one sent.

Append-only means that once a turn is in the history, it is never changed or removed (New conversation starts again with an empty history instead). What goes in is decided once, as the turn is added. A complete reply normally goes in with all its content blocks unchanged; the one exception, as the comment says, is after a server-side fallback, when `replayContent()` first drops the blocks the API says must not be sent back. A reply cut short by Stop or an error goes in as plain text: just the text that had arrived.

The fields:

src/main/claude/ChatSession.ts, lines 53–61

```
private history: MessageParam[] = []

private messages: ChatMessage[] = []

private controller: AbortController | null = null

private turn: Promise<void> = Promise.resolve()

/** Bumped by New conversation, so a reply still finishing can't touch the new one. */
private generation = 0

private readonly emitTimers = new Map<string, ReturnType<typeof setTimeout>>()

constructor(private readonly deps: ChatSessionDeps) {}
```

- `private history: MessageParam[] = []`: Exactly what is sent to the API.
- `private messages: ChatMessage[] = []`: What the chat shows. A reply's object is changed in place while it streams; the page always gets copies.
- `private controller: AbortController | null = null`: The running reply's cancel switch. It is set exactly while a reply is in progress, which is what `busy` checks.

- `private turn: Promise<void> = Promise.resolve()`: The running reply, for `idle()`. It starts as an already-finished promise.
- `private generation = 0`: A counter bumped by `newConversation()`. A reply remembers the value it started with, and stops touching anything once it no longer matches.
- `private readonly emitTimers = new Map<string, ReturnType<typeof setTimeout>>()`: At most one waiting page update per message id, for `scheduleEmit()`. `ReturnType<typeof setTimeout>` means "whatever `setTimeout` returns", which differs between Node.js and browsers.

`ChatSession.busy` and `ChatSession.list()`

src/main/claude/ChatSession.ts, lines 63–69

```
get busy(): boolean {
  return this.controller !== null
}

list(): ChatMessage[] {
  return this.messages.map((m) => ({ ...m }))
}
```

- `get busy(): boolean`: A getter, read as `chat.busy` without brackets. True while a reply is in progress. The `assist:chat-send` handler checks it before collecting screenshots.
- `this.messages.map((m) => ({ ...m }))`: Copies of the messages, for `getState()` when a page loads. Copies, so nothing outside can change the session's own objects.

`ChatSession.send()`

src/main/claude/ChatSession.ts, lines 71–87

```
/** Starts a reply and returns straight away; the reply streams in through `onMessage`. */
send(text: string, images: OutgoingImage[], attachments: Attachment[]): SendStatus {
  if (this.busy) return 'busy'
  if (!text.trim() && images.length === 0) return 'empty'
  if (!this.deps.canChat()) return 'signed-out'

  const content: Anthropic.Beta.BetaContentBlockParam[] = images.map((image) => ({
    type: 'image',
    source: { type: 'base64', media_type: image.mediaType, data: image.data },
  }))
  if (text.trim()) content.push({ type: 'text', text })
  this.history.push({ role: 'user', content })
  this.add({ role: 'user', text, attachments, status: 'done' })

  this.startTurn(this.add({ role: 'assistant', text: '', attachments: [], status: 'streaming' }))
  return 'sent'
}
```

Adds the user's message and starts a reply, then returns at once. The reply arrives through `onMessage`.

- `if (this.busy) return 'busy'`: One reply at a time.
- `if (!text.trim() && images.length === 0) return 'empty'`: A message needs some text or at least one screenshot.
- `if (!this.deps.canChat()) return 'signed-out'`: The panel shows the sign-in instead of the chat when nobody is signed in, so this is a safety net.

- `images.map((image) => ({`: Each screenshot becomes an `image` content block holding its base64 data. Images go before the text, the order Anthropic recommends.
- `if (text.trim()) content.push({ type: 'text', text })`: A blank text box with screenshots sends only the images. Text is sent as typed, not trimmed.
- `this.history.push({ role: 'user', content })`: If the previous reply failed before any text, the history already ends with a user turn. That's fine: the API joins consecutive user turns into one.
- `this.add({ role: 'user', text, attachments, status: 'done' })`: The chat's copy keeps the screenshot file details (`attachments`) for the chips under the message, not the image data.
- `this.startTurn(this.add({ role: 'assistant', text: '', attachments: [], status: 'streaming' }))`: An empty reply appears at once (the page shows "Thinking…" until text arrives), and `runTurn()` fills in that same object.

`ChatSession.retry()`

src/main/claude/ChatSession.ts, lines 89–96

```
/** Asks again after a failed reply (only if nothing from it was kept). */
retry(): void {
  const last = this.messages.at(-1)
  if (this.busy || !last?.retryable || !this.deps.canChat()) return
  Object.assign(last, { text: '', status: 'streaming', notice: undefined, retryable: undefined })
  this.emit(last)
  this.startTurn(last)
}
```

Asks again after a failed reply, from the Retry button (IPC `assist:chat-retry`).

- `const last = this.messages.at(-1)`: `at(-1)` is the last item. Only the latest reply can be retried; once the user sends something new, the old failure is no longer last.
- `!last?.retryable`: `runTurn()` sets `retryable` only when nothing from the failed request went into the history, so the history still ends with a user turn. `?.` copes with an empty chat.
- `!this.deps.canChat()`: After an expired sign-in, Retry does nothing until the user has signed back in.
- `Object.assign(last, {`: Resets the failed reply in place, keeping its id, so the page replaces it rather than adding another.
- `this.startTurn(last)`: Nothing is added to the history: it still ends with a user turn, so this simply sends the same conversation again.

`ChatSession.stop()` and `ChatSession.newConversation()`

src/main/claude/ChatSession.ts, lines 98–111

```
stop(): void {
  this.controller?.abort()
}

newConversation(): void {
  this.generation++
  this.stop()
  this.controller = null
  for (const timer of this.emitTimers.values()) clearTimeout(timer)
  this.emitTimers.clear()
  this.history = []
  this.messages = []
  this.deps.onReset()
}
```

`stop()` is the Stop button (IPC `assist:chat-stop`), and `index.ts` also calls it when the app quits. `newConversation()` is New conversation (IPC `assist:chat-new`), and `index.ts` also calls it on Log out. When the sign-in merely expires, the conversation is kept so the user can carry on.

- `this.controller?.abort()`: Cancels the request, if a reply is running. The stream then fails with `APIUserAbortError`, and `runTurn()` marks the reply stopped.
- `this.generation++`: Marks any reply still running as belonging to the old conversation; `runTurn()` checks this before touching anything.
- `this.controller = null`: Not busy any more, straight away, so a new message can be sent while the cancelled request is still winding down.
- `for (const timer of this.emitTimers.values()) clearTimeout(timer)`: Cancels page updates still waiting, so nothing from the old reply reaches the page after the chat has been cleared.
- `this.deps.onReset()`: `index.ts` broadcasts `assist:chat-reset`, and the page empties its list.

`ChatSession.idle()` and `ChatSession.startTurn()`

src/main/claude/ChatSession.ts, lines 113–124

```
/** Resolves when the current reply (if any) has finished. */
idle(): Promise<void> {
  return this.turn
}

private startTurn(reply: ChatMessage): void {
  const controller = new AbortController()
  this.controller = controller
  this.turn = this.runTurn(reply, controller).finally(() => {
    if (this.controller === controller) this.controller = null
  })
}
```

- `return this.turn`: For the tests: `await session.idle()` waits for the reply to finish. It never rejects, because `runTurn()` catches its own errors.
- `const controller = new AbortController()`: A new cancel switch for each reply. `send()` and `retry()` have already checked that no reply is running.

- `this.turn = this.runTurn(reply, controller).finally(() => {`: `runTurn()` is async, so calling it starts the reply in the background. The promise is stored, not awaited, which is why `send()` can return straight away.
- `if (this.controller === controller) this.controller = null`: Clears the controller only if it is still this reply's. After New conversation a newer reply may already be running with its own controller, and the old reply finishing must not make the session look idle.

`ChatSession.runTurn()`

src/main/claude/ChatSession.ts, lines 126–185

```
private async runTurn(reply: ChatMessage, controller: AbortController): Promise<void> {
  const generation = this.generation
  const current = () => generation === this.generation
  const tools = this.deps.tools ?? []
  let roundText = ''
  let keptPartial = false

  try {
    for (let round = 0; round < MAX_TOOL_ROUNDS; round++) {
      roundText = ''
      const message = await this.deps.backend.reply({
        system: this.deps.system,
        messages: this.history,
        tools: tools.map((t) => t.definition),
        effort: this.deps.getEffort(),
        signal: controller.signal,
        onText: (delta) => {
          if (!current()) return
          roundText += delta
          reply.text += delta
          this.scheduleEmit(reply)
        },
      })
      if (!current()) return
      const content = replayContent(message.content)
      this.history.push({ role: 'assistant', content })

      if (message.stop_reason === 'tool_use') {
        this.history.push({ role: 'user', content: await runTools(content, tools) })
        continue
      }
      if (message.stop_reason === 'pause_turn') continue // the API asks us to send it back to resume

      return this.finish(reply, 'done', stopNotice(message.stop_reason))
    }
    this.finish(reply, 'done', 'Claude stopped after too many steps.')
  } catch (error) {
    if (!current()) return
    // Keep what streamed before the failure, as plain text (the history stays append-only).
    if (roundText) {
      this.history.push({ role: 'assistant', content: [{ type: 'text', text: roundText }] })
      keptPartial = true
    }
    const kind = classifyError(error)
    if (kind === 'aborted') {
      return this.finish(
        reply,
        'stopped',
        reply.text ? 'Stopped.' : 'Stopped before Claude replied.',
      )
    }
    this.deps.onError?.(error)
```

```
    const retryable = !keptPartial && isRetryable(kind)
    const reference = errorReference(error)
    const notice = reference
      ? `${errorMessage(kind)} (Reference: ${reference})`
      : errorMessage(kind)
    this.finish(reply, 'error', notice, retryable)
  }
}
```

One whole reply: one or more requests to Claude, the streamed text, and the outcome. It never throws; every failure becomes a status and a notice on the reply.

- `const current = () => generation === this.generation`: True while this reply's conversation is still the current one.
- `if (!current()) return`: (three places) New conversation was pressed while this reply was running, so it leaves everything alone: no text, no history, no error.
- `let roundText = ''`: The text streamed in the current round only, so a round that's cut off can keep its text (below). `reply.text` collects the text of all rounds.
- `for (let round = 0; round < MAX_TOOL_ROUNDS; round++) {`: The tool-use loop: each round is one request. Without tools, the first round is also the last.
- `messages: this.history,`: The whole conversation each time, ending with the user's turn or with tool results.
- `effort: this.deps.getEffort(),`: Read again each round, so a changed setting applies to the next request.
- `this.scheduleEmit(reply)`: The new text goes to the page in batches (see `scheduleEmit()`).
- `const content = replayContent(message.content)`: The reply as it must be sent back. Normally that is every block exactly as it came back: thinking blocks with their signatures, text, and any tool calls. Only after a server-side fallback partway through the reply does `replayContent()` drop some of the declining model's blocks.
- `this.history.push({ role: 'assistant', content })`: The reply goes into the history once and is never changed afterwards. This is what keeps the next request valid.
- `if (message.stop_reason === 'tool_use') {`: Claude asked for tools. `runTools()` runs them, the results are appended as a user turn, and `continue` starts the next round so Claude can carry on.
- `await runTools(content, tools)`: Tools are run from the trimmed `content`, not the raw reply. A tool call the declining model made before a fallback isn't in the history, so it isn't run: a result for it would answer a call the API never sees.
- `if (message.stop_reason === 'pause_turn') continue`: The API can pause a long turn, which happens with tools that run on Anthropic's servers (this app doesn't use any yet). Sending the conversation back as it is lets Claude resume.
- `return this.finish(reply, 'done', stopNotice(message.stop_reason))`: Any other stop reason ends the reply: normally `end_turn` (Claude finished), or `refusal` or `max_tokens`, which get a notice.
- `this.finish(reply, 'done', 'Claude stopped after too many steps.')`: Reached only when every round asked for more tools or paused.
- `if (roundText) {`: The round was cut off after some text had arrived. That text stays on screen, so it also goes into the history, as a plain text block, and Claude will see next time what the user saw. Nothing else from the unfinished round is kept.
- `keptPartial = true`: Then Retry isn't offered. The history now ends with Claude's partial reply instead of a user turn, so the same request can't simply be sent again (this model doesn't accept a request whose last turn is Claude's). The user can send a new message instead.
- `if (kind === 'aborted') {`: Stop isn't an error: the reply is marked `stopped`, without Retry, and the notice says whether anything had arrived.

- `this.deps.onError?.(error)`: The full error goes to the log. `?.` calls it only if it was given.
- `const retryable = !keptPartial && isRetryable(kind)`: Retry is offered only when nothing from this reply went into the history and the problem may be temporary.
- `const reference = errorReference(error)`: Anthropic's request ID for the failed request, if there is one (see `errorReference()`).
- `const notice = reference`: The notice is the plain-English message from `errorMessage()`, with "(Reference: …)" and the ID added after it when there is one, so the user can pass it to IT.

`ChatSession.add()` and `ChatSession.finish()`

src/main/claude/ChatSession.ts, lines 187–202

```
private add(fields: Omit<ChatMessage, 'id'>): ChatMessage {
  const message: ChatMessage = { id: randomUUID(), ...fields }
  this.messages.push(message)
  this.emit(message)
  return message
}

private finish(
  message: ChatMessage,
  status: ChatMessage['status'],
  notice?: string,
  retryable?: boolean,
): void {
  Object.assign(message, { status, notice, retryable: retryable || undefined })
  this.emit(message)
}
```

- `Omit<ChatMessage, 'id'>`: A `ChatMessage` without its `id`; `add()` makes the id itself with `randomUUID()`. The page uses the id to replace a message when an update arrives.
- `return message`: The same object that is in `messages`, so `send()` can hand it to `runTurn()` to fill in.
- `retryable: retryable || undefined`: Stores nothing rather than `false`, so a message that can't be retried simply has no `retryable` field.
- `this.emit(message)`: New messages and final states go to the page at once, skipping the batching.

`ChatSession.scheduleEmit()` and `ChatSession.emit()`

src/main/claude/ChatSession.ts, lines 204–223

```
/** Streamed text arrives in many small pieces; batch them into fewer UI updates. */
private scheduleEmit(message: ChatMessage): void {
  const interval = this.deps.emitIntervalMs ?? 50
  if (interval <= 0) return this.emit(message)
  if (this.emitTimers.has(message.id)) return
  this.emitTimers.set(
    message.id,
    setTimeout(() => {
      this.emitTimers.delete(message.id)
      this.deps.onMessage({ ...message })
    }, interval),
  )
}

private emit(message: ChatMessage): void {
  const timer = this.emitTimers.get(message.id)
  if (timer) clearTimeout(timer)
  this.emitTimers.delete(message.id)
  this.deps.onMessage({ ...message })
}
```

A reply streams in as many small pieces. Sending each one over IPC, and re-rendering the Markdown each time, would be wasteful, so streaming updates are batched: at most one per message every 50 ms.

- `if (interval <= 0) return this.emit(message)`: With an interval of 0 (the tests) every piece is sent at once.
- `if (this.emitTimers.has(message.id)) return`: An update is already waiting. The message object is changed in place, so when that update fires it carries all the text received by then.
- `this.deps.onMessage({ ...message })`: Sends a copy, a snapshot of the message at that moment.
- `if (timer) clearTimeout(timer)`: `emit()` sends straight away and cancels any waiting update, which would only repeat what this one already contains.

`stopNotice()`

src/main/claude/ChatSession.ts, lines 226–230

```
function stopNotice(stopReason: string | null): string | undefined {
  if (stopReason === 'refusal') return 'Claude declined to answer this.'
  if (stopReason === 'max_tokens') return 'This reply hit the length limit and was cut short.'
  return undefined
}
```

The notice for a reply that finished but not normally. Any other stop reason (usually `end_turn`) gets no notice.

- `if (stopReason === 'refusal')`: The safety classifiers declined, and either there was no fallback model for that kind of refusal or it declined too (see `FALLBACK_BETA`). Any text that had already streamed stays.
- `if (stopReason === 'max_tokens')`: The reply reached `MAX_TOKENS`.

`DECLINED_REASONING`, `replayContent()`

src/main/claude/ChatSession.ts, lines 232–254

```ts
/** Reasoning that belongs to a model that declined, so it isn't sent back after a fallback. */
const DECLINED_REASONING = new Set(['thinking', 'redacted_thinking', 'connector_text'])

/**
 * A reply's content as it must be sent back on the next turn: normally every block, unchanged.
 * After a server-side fallback (another model took over partway through), the API requires
 * dropping what the declining model produced before the last `fallback` block: its reasoning,
 * its client-side tool calls, and server tool calls that got no result. The `fallback` block
 * itself stays exactly where it was; the API checks the blocks around it.
 */
export function replayContent(content: ContentBlock[]): ContentBlock[] {
  const lastFallback = content.findLastIndex((block) => block.type === 'fallback')
  if (lastFallback === -1) return content
  const answered = new Set(
    content.flatMap((block) => ('tool_use_id' in block ? [block.tool_use_id] : [])),
  )
  return content.filter((block, index) => {
    if (index >= lastFallback) return true
    if (DECLINED_REASONING.has(block.type) || block.type === 'tool_use') return false
    if (block.type === 'server_tool_use') return answered.has(block.id)
    return true
  })
}
```

Prepares a reply for the history, following the rules on Anthropic's "Refusals and fallback" page. `runTurn()` passes every reply through it. Almost always it changes nothing. The exception is a **mid-reply fallback**: the safety classifiers declined after Claude Opus 5.5 had already started answering, and the fallback model carried on in the same stream. The reply then holds both models' output, separated by a `fallback` block. Some of what came before that block must not be sent back, because it belongs to a model that declined. A decline before any output puts the `fallback` block first, so nothing comes before it and nothing is dropped.

- `new Set(['thinking', 'redacted_thinking', 'connector_text'])`: The declining model's reasoning: thinking blocks, `redacted_thinking` (thinking that comes back encrypted instead of readable), and `connector_text` (narration some tool-using replies include between tool calls).
- `content.findLastIndex((block) => block.type === 'fallback')`: The position of the last `fallback` block. A reply can have several if more than one model declined; only the last one matters. `-1` means there is none.
- `if (lastFallback === -1) return content`: The normal case: the very same array comes back, untouched.
- `const answered = new Set(`: The ids of every tool call that got a result in this reply. A server tool's result block has a `tool_use_id` naming the call it answers. `'tool_use_id' in block` checks the property exists, and `flatMap` with `[]` or `[id]` collects only the blocks that have one.
- `if (index >= lastFallback) return true`: The `fallback` block itself and everything after it stay. The block must stay exactly where it was, because the API uses its position to check the thinking blocks around it.
- `if (DECLINED_REASONING.has(block.type) || block.type === 'tool_use') return false`: Before the boundary, the declining model's reasoning and its calls to the app's own tools are dropped.
- `if (block.type === 'server_tool_use') return answered.has(block.id)`: A call to a tool that Anthropic runs (such as web search) stays only if its result is in the reply too. The app uses no server tools yet.
- `return true`: Everything else before the boundary stays, notably the declining model's text. The user has already seen it, and it's what the fallback model continued from.
- `export function replayContent`: Exported so `tests/chatSession.test.ts` can check it directly.

`runTools()`

src/main/claude/ChatSession.ts, lines 256–278

```ts
/** Runs every tool call in a reply and returns the results to send back. */
async function runTools(
  content: ContentBlock[],
  tools: ChatTool[],
): Promise<Anthropic.Beta.BetaToolResultBlockParam[]> {
  const results: Anthropic.Beta.BetaToolResultBlockParam[] = []
  for (const block of content) {
    if (block.type !== 'tool_use') continue
    const tool = tools.find((t) => t.definition.name === block.name)
    try {
      if (!tool) throw new Error(`Unknown tool: ${block.name}`)
      results.push({
        type: 'tool_result',
        tool_use_id: block.id,
        content: await tool.run(block.input),
      })
    } catch (error) {
      const message = error instanceof Error ? error.message : String(error)
      results.push({ type: 'tool_result', tool_use_id: block.id, content: message, is_error: true })
    }
  }
  return results
}
```

Runs every tool call in one reply and builds the results to send back, as described under `ChatTool`. It gets the reply's content after `replayContent()`, so only calls that are in the history are run. The tools run one after another, in the order Claude asked for them.

- `if (block.type !== 'tool_use') continue`: The reply also holds thinking and text blocks; only tool calls are run.
- `tools.find((t) => t.definition.name === block.name)`: Finds the tool by the name Claude used.
- `if (!tool) throw new Error`: An unknown name is reported to Claude as a failed result, through the `catch` below, instead of breaking the reply.
- `tool_use_id: block.id`: Each result names the call it answers. Claude can ask for several tools in one reply, and all the results go back together in one user turn.
- `is_error: true`: A tool that throws is reported to Claude as a failed result with the error message, so Claude can try something else or explain the problem.

The bubble: windows, layout, bounce and the state machine

The bubble is the always-on-top logo in a corner of the screen; clicking it opens the panel (the chat card and the action icons). Each lives in its own transparent, frameless *overlay window*, created in `windows.ts`. `BubbleController` decides what the bubble is doing and moves, shows and hides the two windows to match, using the pure maths in `layout.ts` (where things go) and `bounce.ts` (how they move).

The controller is always in exactly one *mode* (the `Mode` type in `src/shared/types.ts`). Every change is broadcast to both pages, which draw themselves to match:

- `collapsed`: only the bubble shows, resting in its corner. The app starts here.
- `expanded`: the panel is open beside the bubble.
- `dragging`: the bubble follows the mouse pointer.
- `bouncing`: the bubble flies around its display (the Bounce action).
- `returning`: the bubble glides back into its corner after a drag or a bounce.
- `capturing`: both windows are hidden while a screenshot is taken.

What moves the controller from one mode to another:

- `collapsed` → `expanded`: a click on the bubble (`clickBubble()`), or the tray icon, launching the app a second time, or finishing sign-in in the browser (all `expand()`).
- `expanded` → `collapsed`: a click on the bubble, Esc in the panel (`collapse()`), handing a question to Claude Desktop, opening an app from the Apps list, the IT service desk or an IT request's form on the portal (also `collapse()`), or clicking somewhere else so the panel loses focus (`panelBlurred()`, except in the panel's first 400 ms).
- `collapsed` or `expanded` → `dragging` → `returning`: pressing and moving the bubble (`startDrag()`), then letting go (`endDrag()`).
- `collapsed` or `expanded` → `bouncing` → `returning`: the Bounce action (`startBounce()`), then a click on the bubble or anything that calls `expand()`.
- `returning` → `collapsed`: the glide finishes (inside `glideHome()`). If something asked to open the panel during the glide, it carries straight on to `expanded`.
- `expanded` → `capturing` → `expanded`: the screenshot action (`whileHidden()`).

`src/main/bubble/BubbleController.ts`: the state machine

This file holds the `BubbleController` class and the small interfaces it is built from. The controller owns three pieces of state: the mode, the bubble's position, and its *anchor* (which display, and which corner of it, the bubble rests in). Each public method is a request such as "the bubble was clicked" or "a drag started", and the controller decides what that means in the current mode.

Who calls it:

- `src/main/index.ts` creates the one controller in `start()` and wires Electron events to it: the panel window's `blur` event to `panelBlurred()`, screen changes to `displayChanged()`, the tray icon, a second launch, a finished sign-in and a click on a notification to `expand()`, and quitting to `dispose()`.

- `src/main/ipc.ts` forwards the pages' requests: the bubble's click and drag (`clickBubble()`, `startDrag()`, `endDrag()`) and the panel's Esc (`collapse()`). It also calls `collapse()` after handing a question to Claude Desktop (from the text box, or about a Jira issue), opening an app from the Apps list, and opening an IT request's form or the user's requests on the service desk portal.
- `src/main/actions.ts` calls `startBounce()` for the Bounce icon, `whileHidden()` for the screenshot icon, and `collapse()` once the IT service desk has opened in the browser.

The controller never imports Electron. It drives windows through the `Surface` interface and asks about screens through the `Displays` interface. `index.ts` passes real ones: the adapters from `windows.ts`, and `electronDisplays`, a thin wrapper around Electron's `screen` module. `tests/bubbleController.test.ts` passes a `FakeSurface` that just records its bounds and whether it's visible, a pretend pair of monitors, and Vitest's fake timers. That way a whole drag or bounce can be checked in milliseconds without opening a window.

`Surface`, `PanelSurface`

`src/main/bubble/BubbleController.ts`, lines 12–21

```ts
/** The parts of a window the controller drives. Electron windows are adapted in windows.ts. */
export interface Surface {
  setBounds(bounds: Rect): void
  show(): void
  hide(): void
}

export interface PanelSurface extends Surface {
  focus(): void
}
```

What the controller needs from a window, and nothing more.

- `setBounds(bounds: Rect)`: moves and sizes the window in one call. `Rect` (from `src/shared/geometry.ts`) is `x`, `y`, `width` and `height`, in DIPs (see `DisplayArea` below).
- `show(): void`: what "show" does differs per window. In `windows.ts`, the bubble appears without taking focus, and the panel resets its click-through first.
- `interface PanelSurface extends Surface`: a `PanelSurface` has everything a `Surface` has, plus `focus()`, so you can type as soon as the panel opens.

`DisplayArea`, `Displays`

src/main/bubble/BubbleController.ts, lines 23–40

```ts
/** One display: its id, its whole screen, and its work area (the screen minus the taskbar). */
export interface DisplayArea {
  id: number
  bounds: Rect
  workArea: Rect
}

/** What the controller needs to know about the screens. Electron's `screen` is adapted in index.ts. */
export interface Displays {
  primary(): DisplayArea
  /** Every display (Bounce roams across them all). */
  all(): DisplayArea[]
  byId(id: number): DisplayArea | undefined
  /** The display containing `point`, or the closest one. */
  nearest(point: Point): DisplayArea
  /** Where the mouse pointer is. */
  cursor(): Point
}
```

The controller's view of the monitors. Each display has an id (a number Electron assigns), its `bounds` (the whole screen) and a *work area*: the part of the screen that windows may use, which is the screen minus the taskbar. The bubble rests inside the work area, so it never sits on top of the taskbar. The bounds are only needed for Bounce, to find taskbars that lie between two monitors.

All coordinates are DIPs (device-independent pixels). Windows scales the desktop up on high-DPI screens: at 150 % scaling, a 1920×1080 monitor is 1280×720 DIPs. Electron's window and screen functions all use DIPs, so the bubble is the same visual size on every monitor and none of this code needs to know the scale factor. With several monitors, all displays share one coordinate space: the main display's top-left is (0, 0), and a monitor to its left or above it has negative coordinates. That is why a work area has its own `x` and `y`, and why the layout code always measures from them.

- `primary()`: the main display. It's the fallback when the bubble's display has gone.
- `all()`: every connected display. Bounce roams across all of them.
- `byId(id: number): DisplayArea | undefined`: `undefined` when no connected display has that id, for example after a monitor was unplugged.
- `nearest(point: Point)`: the display containing a point, or the closest one if the point is in a gap between monitors. Used to decide which display the bubble was dropped on.
- `cursor()`: where the mouse pointer is, in the same DIP coordinates. Dragging reads it every frame.

In `index.ts`, `electronDisplays` implements these five with Electron's `screen` module (`getPrimaryDisplay()`, `getAllDisplays()`, `getDisplayNearestPoint()` and `getCursorScreenPoint()`).

`BubbleAnchor`

src/main/bubble/BubbleController.ts, lines 42–46

```ts
/** Where the bubble rests: a corner of a particular display. */
export interface BubbleAnchor {
  displayId: number
  corner: Corner
}
```

Where the bubble rests between moves. `Corner` (from `geometry.ts`) is one of `'top-left'`, `'top-right'`, `'bottom-left'` or `'bottom-right'`. Storing a display and a corner, rather than a pixel position, means the bubble still lands in the right place if that display's resolution or taskbar has changed since. Whenever the anchor changes, `index.ts` saves it to `preferences.json` (`SettingsService.setBubbleAnchor()`), and it passes the saved one back in at the next start.

`BubbleControllerDeps`

`src/main/bubble/BubbleController.ts`, lines 48–62

```ts
export interface BubbleControllerDeps {
  bubble: Surface
  panel: PanelSurface
  displays: Displays
  actionCount: number
  /** Where the bubble was last time, if saved. */
  initialAnchor?: BubbleAnchor | null
  onModeChange(mode: Mode): void
  /** The bubble settled in a new corner or on a new display. */
  onAnchorChange(anchor: BubbleAnchor): void
  now?: () => number
  random?: () => number
  /** Things worth a line in the log, such as a click on the bubble that didn't open it. */
  onDiagnostic?: (message: string) => void
}
```

Everything the constructor needs, passed as one object (its *dependencies*).

- `actionCount: number`: how many action icons the tenant has. The panel window's size depends on it (`panelWindowSize()` in `geometry.ts`), and so does its position next to the bubble.
- `initialAnchor?: BubbleAnchor | null`: the `?` means the field may be left out; `null` means "nothing saved yet". The constructor treats both the same.
- `onModeChange(mode: Mode): void`: `index.ts` broadcasts the new mode to both pages (`IPC.modeChanged`). That's how the panel knows to animate open or closed, and the bubble knows to show its ring.
- `onAnchorChange(anchor: BubbleAnchor): void`: `index.ts` broadcasts the new corner (`IPC.cornerChanged`) so the panel can lay itself out to open toward the middle of the screen, and saves the anchor.
- `now?: () => number`: a clock, and `random?: () => number` a random-number source. The app leaves them out and gets the real ones; tests pass their own so timings and bounce angles are predictable.
- `onDiagnostic?: (message: string) => void`: a line for the problem log when the bubble does something a user could take for a fault: a click that didn't open the panel, or a blur that didn't close it. `index.ts` passes the log. Such reports ("I clicked the bubble and nothing happened") are otherwise impossible to look into on someone else's PC.

Constants

src/main/bubble/BubbleController.ts, lines 64–84

```ts
const FRAME_MS = 16

const DEFAULT_CORNER: Corner = 'bottom-right'

/** Matches the panel's fade-out in the renderer, so the window hides once it's invisible. */
export const PANEL_FADE_MS = 150

/**
 * Clicking the bubble while the panel is open blurs the panel as the button goes down (the bubble
 * takes the focus), so the panel has already closed when the click arrives. When the click says
 * when it was pressed, a panel that closed after that press (give or take this much) closed
 * because of it, and stays closed. Without that time, a click this soon after a blur-collapse is
 * taken to be the same gesture.
 */
export const BLUR_CLICK_GRACE_MS = 300

export const PRESS_SLACK_MS = 50

/**
 * A blur this soon after the panel opened doesn't close it. Another app can take the focus back
 * just as the panel opens (Claude Desktop, say, right after a question was handed to it); closing
 * then would make the click look like it did nothing. The panel stays open, unfocused, instead.
 */
export const OPEN_BLUR_GRACE_MS = 400

/** Lets Windows repaint the area under our windows before a screenshot. */
export const HIDE_SETTLE_MS = 150
```

- `FRAME_MS = 16`: how often the animation timer fires. 1000 / 16 is about 62 times a second, which this guide calls the 60 fps ticker (see `runTicker()`).
- `DEFAULT_CORNER`: the corner used when nothing was saved.
- `PANEL_FADE_MS = 150`: when the panel closes, its page fades its contents out. The window is hidden only after this delay, otherwise the fade would be cut off (see `collapse()`). It matches the page's fade exactly: the renderer's transitions are also 150 ms (`duration-150`).
- `BLUR_CLICK_GRACE_MS = 300`, `PRESS_SLACK_MS = 50`: see `clickBubble()`. The comment above them covers both.
- `OPEN_BLUR_GRACE_MS = 400`: see `panelBlurred()`. As its comment says, another app can take the focus back just as the panel opens.
- `HIDE_SETTLE_MS = 150`: after the windows are hidden, Windows needs a moment to redraw what was behind them. A screenshot taken sooner could still show them.
- `export const`: the last five are exported so the tests can wait exactly the right time.

BubbleController

The class, with the comment that describes it:

src/main/bubble/BubbleController.ts, lines 86–95

```
/**
 * The bubble's state machine. Owns the mode, the bubble's position and its anchor (the display
 * and corner it rests in), and moves/shows/hides the two windows to match:
 *
 *   collapsed ⇄ expanded                    click, Esc, blur
 *   collapsed/expanded → dragging → returning → collapsed     drag, release (snaps to a corner)
 *   * → bouncing → returning → collapsed     Bounce action, then a click
 *   expanded → capturing → expanded          screenshot
 */
export class BubbleController {
  // …
}
```

The comment sums up the state machine in four lines; the lists at the top of this page are the long version. These are the class's fields:

src/main/bubble/BubbleController.ts, lines 96–106

```
private mode: Mode = 'collapsed'

private position: Point = { x: 0, y: 0 }

private anchor: BubbleAnchor

private motion: Motion | null = null

private ticker: ReturnType<typeof setInterval> | null = null

private hideTimer: ReturnType<typeof setTimeout> | null = null

private lastBlurCollapse = Number.NEGATIVE_INFINITY

private expandedAt = Number.NEGATIVE_INFINITY

private expandAfterReturn = false

private readonly now: () => number

private readonly random: () => number
```

- `private mode: Mode = 'collapsed'`: `private` means only this class's own methods can read or change the field (TypeScript checks this when compiling). Everything starts at rest.
- `private position: Point = { x: 0, y: 0 }`: the top-left corner of the bubble *circle* (not of its window), in DIPs. A placeholder until `start()` sets it.
- `private anchor: BubbleAnchor`: no starting value here; the constructor sets it. With the project's `strict` settings, TypeScript refuses to compile if the constructor could finish without assigning it.
- `private motion: Motion | null = null`: the bounce's current position and velocity (see `bounce.ts`), or `null` when not bouncing. `| null` makes TypeScript insist on a check before it's used.
- `ReturnType<typeof setInterval>`: "whatever type `setInterval` returns". It saves caring whether that's Node's `Timeout` object or a browser's number. `ticker` is the running animation timer, if any.
- `private hideTimer`: the pending "hide the panel window after the fade" timer from `collapse()`.
- `private lastBlurCollapse = Number.NEGATIVE_INFINITY`: when the panel last closed because it lost focus. Starting at minus infinity means "never", so the first click can't fall inside the grace period.

- `private expandedAt = Number.NEGATIVE_INFINITY`: when the panel last opened, for the grace period in `panelBlurred()`.
- `private expandAfterReturn = false`: set when something asks to open the panel while the bubble is still on its way home; `glideHome()` opens it on arrival.
- `private readonly now: () => number`: `readonly` means it's set once, in the constructor, and never reassigned. The same goes for `random`.

`BubbleController.constructor()`

src/main/bubble/BubbleController.ts, lines 108–119

```
constructor(private readonly deps: BubbleControllerDeps) {
  // Wall-clock time, so it compares with the time a click was pressed in the bubble's page.
  this.now = deps.now ?? (() => Date.now())
  this.random = deps.random ?? Math.random
  // Set now, not in start(): the pages ask for the corner as soon as they load, which can be
  // before start() runs.
  const saved = deps.initialAnchor
  this.anchor =
    saved && deps.displays.byId(saved.displayId)
      ? { ...saved }
      : { displayId: deps.displays.primary().id, corner: saved?.corner ?? DEFAULT_CORNER }
}
```

Stores the dependencies, picks the clock and random source, and decides the anchor. Nothing appears on screen until `start()`.

- `constructor(private readonly deps: BubbleControllerDeps)`: TypeScript shorthand. Marking a constructor parameter `private readonly` declares a field called `deps` and stores the argument in it.
- `deps.now ?? (() => Date.now())`: `??` uses the right-hand side only when the left is `null` or `undefined`. As the comment says, the clock is the wall-clock time (milliseconds since 1970), because `clickBubble()` compares it with the time the bubble's page saw the mouse button go down, and the page's `Date.now()` is the same clock. (`performance.now()`, which never jumps when the PC's clock is adjusted, counts from a different starting point in every process, so the two couldn't be compared.)
- `// Set now, not in start()`: `index.ts` registers the IPC handlers (including `getState()`, which reads `corner`) before it waits for the pages to load, and only calls `start()` after both have drawn. A page can ask for the corner in between. Choosing the anchor here means that answer is already right, so the panel lays itself out for the correct corner from the start.
- `saved && deps.displays.byId(saved.displayId)`: the saved anchor is used only if its display is still connected. Otherwise the bubble goes to the main display, keeping the saved corner if there was one.
- `saved?.corner ?? DEFAULT_CORNER`: `?.` gives `undefined` instead of an error when `saved` is `null`, and `??` then falls back to bottom-right.
- `{ ...saved }`: a copy, so the controller's anchor isn't the same object the settings hold.
- `this.anchor =`: sets the field directly rather than through `setAnchor()`, so `onAnchorChange` doesn't fire and a fallback anchor isn't saved. The pages get the corner from `getState()`.

`BubbleController.currentMode`, `bubblePosition`, `corner`

src/main/bubble/BubbleController.ts, lines 121–131

```
get currentMode(): Mode {
  return this.mode
}

get bubblePosition(): Point {
  return { ...this.position }
}

get corner(): Corner {
  return this.anchor.corner
}
```

`get` defines a read-only property: other code writes `controller.currentMode` (no brackets) but can't assign to it. `index.ts` reads `currentMode` and `corner` in its `getState()`, which is how a page that has just loaded learns the current state.

- `return { ...this.position }`: returns a copy, so a caller can't move the bubble by editing the object it got back. Nothing outside the class reads `bubblePosition` at the moment.

`BubbleController.start()`

src/main/bubble/BubbleController.ts, lines 133–138

```
/** Puts the bubble in its saved corner (or bottom-right of the main display) and shows it. */
start(): void {
  this.position = this.home()
  this.placeWindows()
  this.deps.bubble.show()
}
```

Called once by `index.ts`, after both pages have drawn for the first time (their windows' `ready-to-show` event), so the bubble doesn't appear before its page has anything to show. The anchor was already chosen in the constructor; this just puts the windows there.

- `this.position = this.home()`: the resting position in the anchor's corner (see `home()`).
- `this.placeWindows()`: positions both windows, although only the bubble is shown. The panel is then already in the right place when it first opens.
- `this.deps.bubble.show()`: for the real window this is `showInactive()` (see `bubbleSurface()`), so the bubble appears without stealing focus from whatever you're doing.

`BubbleController.clickBubble()`

src/main/bubble/BubbleController.ts, lines 140–164

```
/** A click on the bubble. `pressedAt` is when its mouse button went down (wall-clock ms). */
clickBubble(pressedAt?: number): void {
  switch (this.mode) {
    case 'collapsed': {
      const closedByThisPress =
        pressedAt === undefined
          ? this.now() - this.lastBlurCollapse < BLUR_CLICK_GRACE_MS
          : this.lastBlurCollapse >= pressedAt - PRESS_SLACK_MS
      if (closedByThisPress) return // the press closed the panel: that was the click's job
      this.expand()
      return
    }
    case 'expanded':
      this.collapse()
      return
    case 'bouncing':
      this.glideHome()
      return
    case 'dragging':
    case 'returning':
    case 'capturing':
      this.deps.onDiagnostic?.(`Bubble click ignored while ${this.mode}`)
      return
  }
}
```

What a click on the bubble means depends on the mode. The bubble page sends `IPC.bubbleClick` with the time the mouse button went down (`pressedAt`), and leaves the decision to the main process, which knows the mode and when the panel last closed.

- `clickBubble(pressedAt?: number)`: `?` makes the press time optional; the tests, and anything older, can leave it out.
- `case 'collapsed': {`: the braces give this `case` a block of its own, so `closedByThisPress` can be declared inside it.
- `closedByThisPress`: with the panel open, pressing on the bubble takes the keyboard focus from the panel (the bubble window can take the focus, see `windows.ts` below), and the blur closes the panel as the button goes down. By the time the click arrives the mode is already `collapsed`, and without this check the click would open the panel straight back up: it would seem to ignore you.
- `this.lastBlurCollapse >= pressedAt - PRESS_SLACK_MS`: with a press time, the question is simple: did the panel close at or after this press? Then this press closed it, and the click has done its job. The 50 ms of slack covers the two processes noticing things in a slightly different order. However long the button is held, the answer is right (a slow click is still one click), and a click soon after closing the panel some other way (clicking another app) opens it at once.
- `this.now() - this.lastBlurCollapse < BLUR_CLICK_GRACE_MS`: without a press time, the older rule: a click within 300 ms of a blur-close counts as part of the same gesture.
- `if (closedByThisPress) return`: as the comment says, closing the panel was the click's job. Otherwise `this.expand()` opens it.
- `case 'bouncing':`: clicking a bouncing bubble catches it and sends it home, without opening the panel.
- `case 'capturing':`: the dragging, returning and capturing modes ignore clicks. Listing them explicitly makes it clear none was forgotten. (A drag ends with a click event in the page too, but `BubbleView` swallows that one

before it's sent.) Each ignored click is logged with the mode it arrived in (`Bubble click ignored while returning`, say).

`BubbleController.expand()`

src/main/bubble/BubbleController.ts, lines 166–190

```
/** Opens the panel. While bouncing, glides home first and then opens. */
expand(): void {
  switch (this.mode) {
    case 'expanded':
      this.deps.panel.focus()
      return
    case 'bouncing':
      this.expandAfterReturn = true
      this.glideHome()
      return
    case 'returning':
      this.expandAfterReturn = true
      return
    case 'dragging':
    case 'capturing':
      return
    case 'collapsed':
      this.cancelHide()
      this.setMode('expanded')
      this.expandedAt = this.now()
      this.deps.panel.show()
      this.deps.panel.focus()
      return
  }
}
```

Opens the panel. Unlike `clickBubble()` it never closes it, so it's what "open" requests from outside use: the tray icon, launching the app again while it's running, finishing a sign-in in the browser, and clicking one of the app's Windows notifications (all in `index.ts`).

- `this.deps.panel.focus()`: already open, so just give it the keyboard again.
- `this.expandAfterReturn = true`: the panel opens beside the bubble's corner, so it can't open while the bubble is away from it. The request is remembered, and `glideHome()` opens the panel once the bubble has arrived. A bouncing bubble is sent home first; a returning one is already on its way.
- `case 'dragging':`: ignored while you're holding the bubble, and while capturing (the panel reopens by itself when the screenshot is done).
- `this.cancelHide()`: if the panel is still fading out from a moment ago, its pending "hide" timer is cancelled. Otherwise the window would vanish 150 ms after reopening.
- `this.setMode('expanded')`: the pages hear the new mode (through `onModeChange`) and animate open; the window is then shown and focused so you can type at once.
- `this.expandedAt = this.now()`: when it opened, for `panelBlurred()`.

`BubbleController.collapse()`

src/main/bubble/BubbleController.ts, lines 192–200

```
collapse(): void {
  if (this.mode !== 'expanded') return
  this.setMode('collapsed')
  this.cancelHide()
  this.hideTimer = setTimeout(() => {
    this.hideTimer = null
    this.deps.panel.hide()
  }, PANEL_FADE_MS)
}
```

Closes the panel in two steps: the mode changes now, so the page starts its fade-out, and the window itself is hidden `PANEL_FADE_MS` later. Hiding the window at once would cut the animation off. Called by `clickBubble()`, `panelBlurred()` and, for Esc in the panel or after a question has been handed to Claude Desktop, by `ipc.ts`.

- `if (this.mode !== 'expanded') return`: closing only makes sense when open. It also makes a repeated request harmless.
- `this.cancelHide()`: never leaves two hide timers running.
- `this.hideTimer = null`: inside the timer, marks that nothing is pending any more.

`BubbleController.panelBlurred()`

src/main/bubble/BubbleController.ts, lines 202–210

```
panelBlurred(): void {
  if (this.mode !== 'expanded') return
  if (this.now() - this.expandedAt < OPEN_BLUR_GRACE_MS) {
    this.deps.onDiagnostic?.('The panel lost the focus as it opened; kept it open')
    return
  }
  this.lastBlurCollapse = this.now()
  this.collapse()
}
```

`index.ts` calls this on the panel window's `blur` event, which Electron fires when the window loses keyboard focus, for example because you clicked another app or the desktop. The panel behaves like a popup and closes.

- `if (this.mode !== 'expanded') return`: blurs in any other mode are ignored, such as the one caused when `whileHidden()` hides the panel for a screenshot.
- `if (this.now() - this.expandedAt < OPEN_BLUR_GRACE_MS)`: a blur within 400 ms of opening doesn't close the panel. Windows can give the focus straight back to the app that had it, for example Claude Desktop just after a question was handed to it, and closing then would make the click on the bubble look like it did nothing. The panel stays open without the focus (clicking in it gives the focus back), and the case is logged. A real click away, later, still closes it.
- `this.lastBlurCollapse = this.now()`: noted for the grace period in `clickBubble()`.

`BubbleController.startDrag()`

src/main/bubble/BubbleController.ts, lines 212–224

```
/** The mouse pressed on the bubble and moved: the bubble follows the pointer until released. */
startDrag(): void {
  if (this.mode !== 'collapsed' && this.mode !== 'expanded') return
  this.cancelHide()
  this.deps.panel.hide()
  const cursor = this.deps.displays.cursor()
  const grab = { x: cursor.x - this.position.x, y: cursor.y - this.position.y }
  this.setMode('dragging')
  this.runTicker(() => {
    const now = this.deps.displays.cursor()
    this.moveBubble({ x: now.x - grab.x, y: now.y - grab.y })
  })
}
```

The bubble page decides a press has become a drag once the pointer moves more than 5 pixels (`DRAG_THRESHOLD` in `BubbleView.tsx`), and sends `IPC.bubbleDragStart`. From then on the main process moves the window and the page just waits for the release.

- `if (this.mode !== 'collapsed' && this.mode !== 'expanded') return`: a drag can only start from rest, not mid-bounce, mid-glide or during a screenshot.
- `this.deps.panel.hide()`: the panel can't follow the bubble, so it's hidden at once, without the fade. `cancelHide()` drops any fade timer that was still pending.
- `const grab`: the offset from the bubble's top-left to the pointer. Keeping that offset fixed means the spot you grabbed stays under the pointer, instead of the bubble jumping so its corner is there.
- `this.runTicker(() => {`: every frame, read where the pointer is now and move the bubble to match. The callback ignores the frame time, since following the pointer only needs its current position.

Reading the real pointer position in the main process keeps the drag smooth, even across monitors with different scaling. The page's own coordinates would keep shifting as its window moves under it.

`BubbleController.endDrag()`

src/main/bubble/BubbleController.ts, lines 226–240

```
/** Released: snap to the nearest corner of the display the bubble was dropped on. */
endDrag(): void {
  if (this.mode !== 'dragging') return
  this.stopTicker()
  const centre = {
    x: Math.round(this.position.x + UI.bubbleSize / 2),
    y: Math.round(this.position.y + UI.bubbleSize / 2),
  }
  const display = this.deps.displays.nearest(centre)
  this.setAnchor({
    displayId: display.id,
    corner: nearestCorner(display.workArea, this.position),
  })
  this.glideHome()
}
```

On release, `BubbleView` sends `IPC.bubbleDragEnd`. The bubble snaps to the nearest corner of whichever display it was dropped on.

- `if (this.mode !== 'dragging') return`: a release that doesn't end a drag (say, a drag that was refused because the bubble was bouncing) does nothing.
- `const centre`: the display is picked by the bubble's centre, not its top-left, so a bubble straddling two monitors goes to the one its centre is on. It's rounded because Electron's display lookup works in whole DIPs.
- `nearestCorner(display.workArea, this.position)`: the corner of the quarter of that display the bubble is in (`layout.ts`).
- `this.setAnchor({`: saves and broadcasts the new anchor, but only if it actually changed.
- `this.glideHome()`: the bubble glides into its new corner; the mode goes to `returning`, then `collapsed`.

`BubbleController.startBounce()`

`src/main/bubble/BubbleController.ts`, lines 242–254

```
startBounce(): void {
  if (this.mode !== 'collapsed' && this.mode !== 'expanded') return
  this.cancelHide()
  this.deps.panel.hide()
  this.motion = launch(this.position, this.random, this.anchor.corner)
  this.setMode('bouncing')
  this.runTicker((dtMs) => {
    if (!this.motion) return
    // Across every display. The anchor doesn't change, so a click brings it home.
    this.motion = step(this.motion, dtMs, travelAreas(this.deps.displays.all()))
    this.moveBubble({ x: this.motion.x, y: this.motion.y })
  })
}
```

The Bounce action: the bubble flies around the screen, across every monitor, bouncing off the outer edges, until clicked. `actions.ts` calls this for the Bounce icon. A click glides it back to its corner of its own display (`glideHome()`), wherever it has wandered, because bouncing never changes the anchor.

- `if (this.mode !== 'collapsed' && this.mode !== 'expanded') return`: the same rule as for dragging. In practice it's `expanded`, because the Bounce icon is in the panel.
- `launch(this.position, this.random, this.anchor.corner)`: the starting position and velocity, heading away from the bubble's corner into the screen (`bounce.ts`).
- `if (!this.motion) return`: mostly for TypeScript. `motion` is typed `Motion | null`, and inside the callback TypeScript can't assume it's still set.
- `travelAreas(this.deps.displays.all())`: where the bubble may go: every display's work area, plus any taskbar lying between two displays (`layout.ts`), read again every frame. If a monitor is unplugged or the resolution or taskbar changes mid-bounce, the bubble carries on within the displays as they are now (`displayChanged()` deliberately leaves a bounce alone).
- `step(this.motion, dtMs,`: moves the bubble by the real time since the last frame.
- `this.moveBubble(`: moves only the bubble window. The panel is hidden, and is put back next to the bubble when it settles.

`BubbleController.whileHidden()`

src/main/bubble/BubbleController.ts, lines 256–272

```
/** Hides both windows, runs `task` (a screenshot), then reopens the panel. */
async whileHidden<T>(task: () => Promise<T>): Promise<T> {
  if (this.mode !== 'expanded') throw new Error(`Can't hide the windows while ${this.mode}`)
  this.setMode('capturing')
  this.deps.panel.hide()
  this.deps.bubble.hide()
  try {
    await new Promise((resolve) => setTimeout(resolve, HIDE_SETTLE_MS))
    return await task()
  } finally {
    this.deps.bubble.show()
    this.setMode('expanded')
    this.expandedAt = this.now()
    this.deps.panel.show()
    this.deps.panel.focus()
  }
}
```

Used by the screenshot action in `actions.ts`, which passes a task that calls `ScreenshotService.capture()`. The overlay windows mustn't appear in your screenshot, so they're hidden first and brought back afterwards.

- `async whileHidden<T>(task: () => Promise<T>): Promise<T>`: `<T>` makes it *generic*: it returns whatever `task` returns, with the same type.
- `throw new Error(`: only allowed from `expanded`, since the camera icon is in the panel. `actions.ts` catches the error and shows "Couldn't take a screenshot".
- `this.setMode('capturing')`: comes before hiding the windows. Hiding the focused panel makes it lose focus, and `panelBlurred()` ignores that blur only because the mode is no longer `expanded`.
- `await new Promise((resolve) => setTimeout(resolve, HIDE_SETTLE_MS))`: a pause written as a promise that resolves after 150 ms.
- `return await task()`: the `await` matters. Without it, the `finally` block would run as soon as the task started, bringing the windows back before the screenshot was taken.
- `finally {`: runs whether the task succeeded or threw, so a failed capture never leaves the app invisible. The panel comes back open and focused, as it was before, and `expandedAt` is set as if it had just opened.

`BubbleController.displayChanged()`

src/main/bubble/BubbleController.ts, lines 274–285

```
/**
 * The resolution, taskbar or monitors changed. If the bubble's display was unplugged, it moves
 * to the same corner of the main display. Moving bubbles re-read the display every frame.
 */
displayChanged(): void {
  if (!this.deps.displays.byId(this.anchor.displayId)) {
    this.setAnchor({ displayId: this.deps.displays.primary().id, corner: this.anchor.corner })
  }
  if (this.mode === 'bouncing' || this.mode === 'returning' || this.mode === 'dragging') return
  this.position = this.home()
  this.placeWindows()
}
```

`index.ts` calls this on Electron's `screen` events `display-added`, `display-removed` and `display-metrics-changed`. The last covers resolution, scaling and rotation changes, and the taskbar moving or resizing (which changes the work area).

- `if (!this.deps.displays.byId(this.anchor.displayId))`: the bubble's display is gone, for example unplugged or a laptop undocked. The anchor moves to the same corner of the main display, through `setAnchor()`, so the change is saved and broadcast.
- `this.mode === 'bouncing' || this.mode === 'returning'`: moving bubbles adapt by themselves. The bounce and the glide re-read the display every frame, and a drag follows the pointer. Snapping the windows here would make the bubble jump.
- `this.position = this.home()`: in every other mode, both windows go back to the corner of the (possibly resized) work area. That includes `expanded`, so an open panel moves with the bubble, and `capturing`, where the windows are hidden but will come back in the right place.

If a bouncing bubble's display is unplugged, the next frame's `step()` pulls it into the main display's travel box, so it reappears there.

`BubbleController.dispose()`

`src/main/bubble/BubbleController.ts`, lines 287–290

```
dispose(): void {
  this.stopTicker()
  this.cancelHide()
}
```

Stops both timers. `index.ts` calls it while quitting (in `before-quit`), so no timer fires against a window that is being destroyed. The tests call it after each test.

`BubbleController.display()`, `home()`

`src/main/bubble/BubbleController.ts`, lines 292–299

```
/** The anchor's display, or the main display if it's gone. */
private display(): DisplayArea {
  return this.deps.displays.byId(this.anchor.displayId) ?? this.deps.displays.primary()
}

private home(): Point {
  return homePosition(this.display().workArea, this.anchor.corner)
}
```

- `?? this.deps.displays.primary()`: if the anchor's display has gone and `displayChanged()` hasn't run yet, use the main display rather than fail.
- `homePosition(this.display().workArea, this.anchor.corner)`: the resting position for the current anchor (`layout.ts`). It's worked out fresh on every call, from the display's current work area, so it's never stale.

`BubbleController.glideHome()`

src/main/bubble/BubbleController.ts, lines 301–325

```
/** Glides the bubble into its anchor corner, then switches to collapsed. */
private glideHome(): void {
  this.motion = null
  const from = { ...this.position }
  const duration = glideDuration(from, this.home())
  const startedAt = this.now()
  this.setMode('returning')
  this.runTicker(() => {
    // Re-read home every frame in case the display changes mid-glide.
    const home = this.home()
    const t = (this.now() - startedAt) / duration
    if (t < 1) {
      this.moveBubble(glidePosition(from, home, t))
      return
    }
    this.stopTicker()
    this.position = home
    this.placeWindows()
    this.setMode('collapsed')
    if (this.expandAfterReturn) {
      this.expandAfterReturn = false
      this.expand()
    }
  })
}
```

Animates the bubble from wherever it is into its anchor corner, then switches to `collapsed`. Used after a drag (`endDrag()`) and to end a bounce (`clickBubble()`, `expand()`).

- `this.motion = null`: any bounce is over.
- `glideDuration(from, this.home())`: longer trips take longer, between 0.3 and 0.9 s (`bounce.ts`). The duration is fixed when the glide starts.
- `const home = this.home()`: inside the frame callback, so it's re-read every frame. If the display changes mid-glide, the bubble still ends up in the right place.
- `const t = (this.now() - startedAt) / duration`: progress from 0 to 1, based on real elapsed time, so a late frame doesn't slow the glide down.
- `this.moveBubble(glidePosition(from, home, t))`: only the bubble window moves during the glide.
- `this.position = home`: when `t` reaches 1, the timer stops and the bubble is put exactly on its home (the last frame was probably just short). `placeWindows()` moves the hidden panel next to the bubble's corner, ready to open.
- `if (this.expandAfterReturn)`: something asked to open the panel during the trip (see `expand()`). The flag is cleared, then `expand()` runs from `collapsed` as normal.

`BubbleController.runTicker()`, `stopTicker()`

src/main/bubble/BubbleController.ts, lines 327–340

```
private runTicker(onFrame: (dtMs: number) => void): void {
  this.stopTicker()
  let last = this.now()
  this.ticker = setInterval(() => {
    const now = this.now()
    onFrame(now - last)
    last = now
  }, FRAME_MS)
}

private stopTicker(): void {
  if (this.ticker) clearInterval(this.ticker)
  this.ticker = null
}
```

The 60 fps ticker. It drives all three animations (drag, bounce and glide), but only one runs at a time.

- `this.stopTicker()`: starting a new animation always replaces the old one. That's how a click on a bouncing bubble switches from the bounce to the glide.
- `this.ticker = setInterval(`: calls the frame function about every 16 ms (`FRAME_MS`).
- `onFrame(now - last)`: Node's timers aren't exact, and can run late when the main process is busy, so each frame is told how many milliseconds really passed. The bounce uses this to move the right distance.
- `if (this.ticker) clearInterval(this.ticker)`: safe to call when nothing is running.

`BubbleController.moveBubble()`, `placeWindows()`

src/main/bubble/BubbleController.ts, lines 342–352

```
private moveBubble(position: Point): void {
  this.position = position
  this.deps.bubble.setBounds(bubbleWindowBounds(position))
}

private placeWindows(): void {
  this.deps.bubble.setBounds(bubbleWindowBounds(this.position))
  this.deps.panel.setBounds(
    panelWindowBounds(this.position, this.deps.actionCount, this.anchor.corner),
  )
}
```

- `moveBubble`: changes the position and moves only the bubble window. Animations call it every frame; the panel is hidden during them, so moving it too would be wasted work.
- `placeWindows`: positions both windows for the current position and anchor. The panel's place depends on the corner, because it always opens toward the middle of the screen.
- `bubbleWindowBounds(this.position)`: turns the bubble's position (the circle's top-left) into its window's rectangle, which includes the transparent padding around the circle (`layout.ts`).
- `panelWindowBounds(this.position, this.deps.actionCount, this.anchor.corner)`: the panel's rectangle (`layout.ts`).

`BubbleController.setAnchor()`, `cancelHide()`, `setMode()`

src/main/bubble/BubbleController.ts, lines 354–369

```
private setAnchor(anchor: BubbleAnchor): void {
  if (anchor.displayId === this.anchor.displayId && anchor.corner === this.anchor.corner) return
  this.anchor = anchor
  this.deps.onAnchorChange({ ...anchor })
}

private cancelHide(): void {
  if (this.hideTimer) clearTimeout(this.hideTimer)
  this.hideTimer = null
}

private setMode(mode: Mode): void {
  if (mode === this.mode) return
  this.mode = mode
  this.deps.onModeChange(mode)
}
```

Small helpers.

- `if (anchor.displayId === this.anchor.displayId)`: dropping the bubble back in its own corner changes nothing, so nothing is saved or broadcast.
- `this.deps.onAnchorChange({ ...anchor })`: passes a copy, so the receiver can't change the controller's own anchor.
- `cancelHide`: cancels the panel hide that `collapse()` scheduled, if there is one.
- `if (mode === this.mode) return`: `onModeChange` (a broadcast to both pages) fires only for real changes.

`src/main/bubble/layout.ts`: where the windows go

Pure maths, no Electron. Given a work area and a corner, it works out where the bubble rests, which corner a dropped bubble belongs to, where it may bounce, and the exact rectangles of the two windows. `BubbleController` is its only user in the app; `tests/layout.test.ts` checks each function on its own.

Key ideas:

- A position always means the top-left corner of the bubble *circle* (56×56 DIPs, `UI.bubbleSize`), not of its window. The bubble window is bigger, `BUBBLE_BOX` = 72×72, because it has 8 DIPs of transparent padding (`UI.bubblePad`) on every side for the shadow and the ring. Only `bubbleWindowBounds()` deals with the difference.
- Everything is measured from the work area's own `x` and `y`, never from (0, 0), so the same code works on a second monitor or with the taskbar at the top or left.
- All sizes come from `UI` in `src/shared/geometry.ts`, which the pages use too, so the window sizes and the page layout can't drift apart.

`TravelBox`

src/main/bubble/layout.ts, lines 12–19

```ts
// Positions are the top-left corner of the bubble circle itself, not of its window.

export interface TravelBox {
  minX: number
  minY: number
  maxX: number
  maxY: number
}
```

The range of positions where the bubble is wholly on one display: lowest and highest `x`, lowest and highest `y`. Made by `travelBox()` below; `bounce.ts` uses it to put a bubble that's ended up off screen back on the nearest display. The comment above it applies to the whole file.

`homePosition()`

src/main/bubble/layout.ts, lines 21–31

```ts
/** The bubble's resting place in `corner` of the work area (the screen minus the taskbar). */
export function homePosition(workArea: Rect, corner: Corner = 'bottom-right'): Point {
  return {
    x: isLeftCorner(corner)
      ? workArea.x + UI.edgeMargin
      : workArea.x + workArea.width - UI.edgeMargin - UI.bubbleSize,
    y: isTopCorner(corner)
      ? workArea.y + UI.edgeMargin
      : workArea.y + workArea.height - UI.edgeMargin - UI.bubbleSize,
  }
}
```

The bubble's resting position in a corner: `UI.edgeMargin` (16 DIPs) in from both edges of the work area.

- `isLeftCorner(corner)`: from `geometry.ts`; true for `top-left` and `bottom-left`. `isTopCorner()` works the same way for the top.
- `workArea.x + workArea.width - UI.edgeMargin - UI.bubbleSize`: on the right, the bubble's own width is subtracted too, because the position is its left edge.
- `corner: Corner = 'bottom-right'`: a default value, used if the caller leaves the argument out.

`nearestCorner()`

src/main/bubble/layout.ts, lines 33–40

```ts
/** The corner nearest the bubble: whichever quarter of the work area its centre is in. */
export function nearestCorner(workArea: Rect, bubble: Point): Corner {
  const centreX = bubble.x + UI.bubbleSize / 2
  const centreY = bubble.y + UI.bubbleSize / 2
  const top = centreY < workArea.y + workArea.height / 2
  const left = centreX < workArea.x + workArea.width / 2
  return `${top ? 'top' : 'bottom'}-${left ? 'left' : 'right'}`
}
```

Used by `endDrag()`. It splits the work area into four quarters and returns the corner of the quarter the bubble's centre is in.

- `bubble.x + UI.bubbleSize / 2`: the centre, not the top-left, so a bubble across the middle line goes to the side most of it is on.
- `` `${top ? 'top' : 'bottom'}-${left ? 'left' : 'right'}` ``: builds the corner's name. TypeScript works out that this can only produce the four `Corner` values, so it needs no cast.

travelAreas()

src/main/bubble/layout.ts, lines 42–83

```
/**
 * Where a bouncing bubble may go: every display's work area, plus the parts of a taskbar that lie
 * between two displays (where a monitor stacked above another has its taskbar along the edge they
 * share, say), so the bubble can pass over that taskbar from one display to the other. A taskbar
 * on an outer edge stays a wall.
 */
export function travelAreas(displays: { bounds: Rect; workArea: Rect }[]): Rect[] {
  const areas = displays.map((display) => display.workArea)
  for (const { bounds: b, workArea: w } of displays) {
    // The strips of the screen outside its work area (normally just the taskbar's).
    const strips = [
      { side: 'top', rect: { x: b.x, y: b.y, width: b.width, height: w.y - b.y } },
      {
        side: 'bottom',
        rect: {
          x: b.x,
          y: w.y + w.height,
          width: b.width,
          height: b.y + b.height - (w.y + w.height),
        },
      },
      { side: 'left', rect: { x: b.x, y: b.y, width: w.x - b.x, height: b.height } },
      {
        side: 'right',
        rect: {
          x: w.x + w.width,
          y: b.y,
          width: b.x + b.width - (w.x + w.width),
          height: b.height,
        },
      },
    ].filter((strip) => strip.rect.width > 0 && strip.rect.height > 0)
    for (const { side, rect } of strips) {
      for (const { bounds: n } of displays) {
        if (n === b) continue
        const bridge = bridgeTo(side, rect, b, n)
        if (bridge) areas.push(bridge)
      }
    }
  }
  return areas
}
```

Where a bouncing bubble may go, as a list of rectangles. Mostly that's each display's work area, but a taskbar can sit right between two monitors: on a laptop with a monitor stacked above it, for example, the upper monitor's taskbar runs along the edge they share. Work areas alone would make that taskbar a wall the bubble could never get past, so the stretch of it that another display lies right beyond is added too. A taskbar on an outer edge, with no display beyond, stays a wall.

- `strips`: the four bands of the screen outside its work area (top, bottom, left and right). Normally only one has any size, the taskbar's; `.filter(...)` drops the empty ones.
- `if (n === b) continue`: a display isn't its own neighbour.
- `bridgeTo(side, rect, b, n)`: the part of that band with display `n` right beyond it, if any.

`bridgeTo()`, `TOUCHING`

`src/main/bubble/layout.ts`, lines 85–113

```
/** Edges this close count as touching (scaled displays can be a pixel out). */
const TOUCHING = 1

/**
 * The part of `strip` (on `side` of the display with `bounds`) that another display (`other`)
 * lies right beyond, or null if `other` isn't there.
 */
function bridgeTo(side: string, strip: Rect, bounds: Rect, other: Rect): Rect | null {
  const near = (a: number, b: number) => Math.abs(a - b) <= TOUCHING
  const vertical = side === 'top' || side === 'bottom'
  const touching =
    side === 'top'
      ? near(other.y + other.height, bounds.y)
      : side === 'bottom'
        ? near(other.y, bounds.y + bounds.height)
        : side === 'left'
          ? near(other.x + other.width, bounds.x)
          : near(other.x, bounds.x + bounds.width)
  if (!touching) return null
  // Only as far along the edge as the other display reaches.
  if (vertical) {
    const from = Math.max(strip.x, other.x)
    const to = Math.min(strip.x + strip.width, other.x + other.width)
    return to > from ? { ...strip, x: from, width: to - from } : null
  }
  const from = Math.max(strip.y, other.y)
  const to = Math.min(strip.y + strip.height, other.y + other.height)
  return to > from ? { ...strip, y: from, height: to - from } : null
}
```

The part of a taskbar band that another display lies right beyond. Not exported; `travelAreas()` uses it.

- `TOUCHING = 1`: two edges within a pixel of each other count as touching. Windows lays the monitors out in real pixels and Electron converts that to DIPs, so a scaled monitor's edge can come out a fraction of a pixel away from its neighbour's.
- `touching`: whether `other` starts where this display ends, on the band's side: for a band along the bottom, `other`'s top edge must be at this display's bottom edge, and so on.
- `Math.max(strip.x, other.x)` and `Math.min(...)`: only as far along the edge as the other display reaches. Past the end of the laptop under a wide monitor, there's nothing below its taskbar, so that part stays a wall.
- `to > from ? ... : null`: the two don't overlap along the edge at all (they only meet at a corner, say).

`travelBox()`

`src/main/bubble/layout.ts`, lines 115–123

```ts
/** Every position where the bubble is fully inside the work area. */
export function travelBox(workArea: Rect): TravelBox {
  return {
    minX: workArea.x,
    minY: workArea.y,
    maxX: workArea.x + workArea.width - UI.bubbleSize,
    maxY: workArea.y + workArea.height - UI.bubbleSize,
  }
}
```

Every position where the whole circle is inside the work area. Unlike the resting position there's no 16 DIP margin, so a bouncing bubble touches the edges, but it never goes under the taskbar or off the screen.

- `workArea.x + workArea.width - UI.bubbleSize`: the highest `x` is a bubble's width short of the right edge, because the position is the circle's left edge. If the work area were narrower than the bubble, the maximum would be less than the minimum; `onScreen()` in `bounce.ts` allows for that.

`bubbleWindowBounds()`

`src/main/bubble/layout.ts`, lines 125–132

```ts
export function bubbleWindowBounds(bubble: Point): Rect {
  return {
    x: Math.round(bubble.x) - UI.bubblePad,
    y: Math.round(bubble.y) - UI.bubblePad,
    width: BUBBLE_BOX,
    height: BUBBLE_BOX,
  }
}
```

The bubble window's rectangle: 8 DIPs up and to the left of the circle, 72×72.

- `Math.round(bubble.x)`: the bounce and the glide produce fractional positions, but a window can only be placed on whole pixels.

`panelWindowBounds()`

`src/main/bubble/layout.ts`, lines 134–150

```ts
/**
 * The panel window shares the bubble window's corner that points into the screen's `corner`, so
 * it extends toward the middle of the screen (left and up from bottom-right, and so on).
 */
export function panelWindowBounds(
  bubble: Point,
  actionCount: number,
  corner: Corner = 'bottom-right',
): Rect {
  const box = bubbleWindowBounds(bubble)
  const size = panelWindowSize(actionCount)
  return {
    x: isLeftCorner(corner) ? box.x : box.x + box.width - size.width,
    y: isTopCorner(corner) ? box.y : box.y + box.height - size.height,
    ...size,
  }
}
```

The panel window's rectangle. Its size comes from `panelWindowSize()` in `geometry.ts` (big enough for the chat card at full height, a toast, and the action icons), and it's placed so it shares one corner with the bubble window: the corner that points into the screen's corner. With the bubble bottom-right, the two windows' bottom-right corners coincide and the panel extends left and up, toward the middle of the screen.

- `isLeftCorner(corner) ? box.x : box.x + box.width - size.width`: in a left corner the left edges line up; in a right corner the right edges do. The `y` line does the same for top and bottom.
- `...size`: copies `width` and `height` into the result.

The two windows overlap. The panel's page lays out its content from the shared corner (`PANEL_LAYOUT` in `geometry.ts`) and leaves the bubble's square empty. Empty parts of the panel are click-through, so the bubble still gets its clicks.

`src/main/bubble/bounce.ts`: bounce physics

Pure maths for the two animations: the bounce (the bubble moves at a constant speed across all the monitors and reflects off their outer edges, like an old DVD screensaver) and the glide home (a smooth slow-down into the corner). Each function takes a state and returns a new one; none keeps anything between calls or touches a window. `BubbleController` calls them from its frame timer (`startBounce()` and `glideHome()`), and `tests/bounce.test.ts` tests them directly.

`Motion`

`src/main/bubble/bounce.ts`, lines 4–10

```ts
/** Position (DIP) and velocity (DIP per second) of the bouncing bubble. */
export interface Motion {
  x: number
  y: number
  vx: number
  vy: number
}
```

A position plus a velocity: how many DIPs per second the bubble moves in each direction. A positive `vx` means moving right; a positive `vy` means moving down, because screen coordinates grow downward.

`BOUNCE_SPEED`, `MAX_STEP_MS`

src/main/bubble/bounce.ts, lines 12–14

```
export const BOUNCE_SPEED = 280

/** Longer gaps (e.g. the machine was asleep) are treated as one short step. */
const MAX_STEP_MS = 100
```

- `BOUNCE_SPEED = 280`: DIPs per second, about 4.5 DIPs per frame.
- `MAX_STEP_MS = 100`: if the timer stalls (the PC slept, or the main process was busy), the next frame reports a long gap. Moving the full distance would make the bubble teleport; capping the step means it just carries on from where it was.

`launch()`

src/main/bubble/bounce.ts, lines 16–32

```
/** Sets off away from the bubble's corner, into the screen, at a random 25°–65° angle. */
export function launch(
  from: Point,
  random: () => number = Math.random,
  corner: Corner = 'bottom-right',
  speed = BOUNCE_SPEED,
): Motion {
  const angle = ((25 + random() * 40) * Math.PI) / 180
  const dx = isLeftCorner(corner) ? 1 : -1
  const dy = isTopCorner(corner) ? 1 : -1
  return {
    x: from.x,
    y: from.y,
    vx: dx * speed * Math.cos(angle),
    vy: dy * speed * Math.sin(angle),
  }
}
```

Sets the bubble off: away from the corner it rests in, into the screen, at a random angle between 25° and 65° from horizontal. The range keeps it from running flat along an edge or straight up.

- `((25 + random() * 40) * Math.PI) / 180`: `random()` gives a number from 0 to 1, so the angle is 25° to 65°. It's converted to radians, which `Math.cos` and `Math.sin` expect.
- `const dx = isLeftCorner(corner) ? 1 : -1`: from a left corner go right (+1); from a right corner go left (-1). `dy` does the same vertically: from a bottom corner, -1 means up.
- `vx: dx * speed * Math.cos(angle)`: splits the speed into its horizontal and vertical parts, so the overall speed is always `speed`, whatever the angle.
- `random: () => number = Math.random`: a default, like `speed = BOUNCE_SPEED`. The controller passes its own `random` so the tests can fix the angle.

`step()`

`src/main/bubble/bounce.ts`, lines 34–48

```ts
/**
 * Advances the motion by `dtMs` across `areas` (where the bubble may go: the displays' work areas
 * and the taskbars between them, from travelAreas()), reflecting off their outer edges. The bubble
 * can be anywhere it's wholly inside them, so it crosses from one display to the next where they
 * meet, and bounces off an edge with no display beyond it (such as the part of a taller monitor's
 * side that a shorter one doesn't reach).
 */
export function step(motion: Motion, dtMs: number, areas: Rect[]): Motion {
  const dt = Math.min(Math.max(dtMs, 0), MAX_STEP_MS) / 1000
  const start = onScreen(motion, areas)
  // One direction at a time, so hitting a side wall only turns the bubble sideways.
  const [x, vx] = reflect(start.x, motion.vx * dt, motion.vx, (x) => fits(x, start.y, areas))
  const [y, vy] = reflect(start.y, motion.vy * dt, motion.vy, (y) => fits(x, y, areas))
  return { x, y, vx, vy }
}
```

Advances one frame: moves by velocity × time on each axis, bouncing off any edge it would have crossed. `startBounce()` calls it every frame with the real milliseconds since the last one and the areas from `travelAreas()`. Where two monitors meet, the bubble passes from one to the other (over a taskbar between them, if there is one); an edge with no monitor beyond it is a wall.

- `Math.min(Math.max(dtMs, 0), MAX_STEP_MS) / 1000`: clamps the elapsed time to 0–100 ms (it should never be negative, but that costs nothing to rule out), then converts it to seconds to match the velocity.
- `onScreen(motion, areas)`: where the bubble starts from, put back on screen first if a display went away under it.
- `const [x, vx] = reflect(`: `reflect()` returns a pair, and this *destructures* it into two variables. The two axes are handled separately, which is how a bounce off a wall works: hitting a side wall flips only `vx`.
- `(x) => fits(x, start.y, areas)`: how `reflect()` asks whether a position is allowed. The sideways move is tried at the old height, then the up-or-down move at the new `x`.
- `return { x, y, vx, vy }`: a new `Motion`; the old one is left unchanged.

`reflect()`

`src/main/bubble/bounce.ts`, lines 65–87

```ts
/**
 * The position after moving `delta` along one direction from `from` (where the bubble fits),
 * with the velocity: reflected off the wall it would have gone through.
 */
function reflect(
  from: number,
  delta: number,
  velocity: number,
  fitsAt: (position: number) => boolean,
): [number, number] {
  const to = from + delta
  if (fitsAt(to)) return [to, velocity]
  // Find the wall: the furthest point towards `to` where the bubble still fits.
  let inside = from
  let outside = to
  while (Math.abs(outside - inside) > 0.001) {
    const middle = (inside + outside) / 2
    if (fitsAt(middle)) inside = middle
    else outside = middle
  }
  const back = inside - (to - inside)
  return [fitsAt(back) ? back : inside, -velocity]
}
```

One axis of a bounce. It isn't exported: only `step()` uses it. With several monitors of different sizes the walls aren't one simple box, so instead of comparing with limits it asks `fitsAt` (which `step()` builds from `fits()`).

- `if (fitsAt(to)) return [to, velocity]`: the usual case: the bubble is still wholly on screen, so it just moves.
- `while (Math.abs(outside - inside) > 0.001)`: otherwise the wall is somewhere between `from` (where it fits) and `to` (where it doesn't). Halving the gap each time (a *binary search*) finds it to a thousandth of a DIP in about 15 steps.
- `const back = inside - (to - inside)`: the bubble went past the wall by `to - inside`, so it's mirrored back by the same amount, as if it had bounced exactly at the wall.
- `fitsAt(back) ? back : inside`: if even that doesn't fit (a space barely bigger than the bubble), it stays at the wall rather than jittering.
- `-velocity`: the bubble now heads the other way along this axis.

`fits()`

src/main/bubble/bounce.ts, lines 50–63

```ts
/** Whether the bubble at (x, y) is wholly inside the displays' work areas. */
function fits(x: number, y: number, areas: Rect[]): boolean {
  const size = UI.bubbleSize
  // Its corners, the middle of each side and its centre: enough to catch a gap between displays.
  for (const px of [x, x + size / 2, x + size]) {
    for (const py of [y, y + size / 2, y + size]) {
      const inside = areas.some(
        (a) => px >= a.x && px <= a.x + a.width && py >= a.y && py <= a.y + a.height,
      )
      if (!inside) return false
    }
  }
  return true
}
```

Whether the bubble at a position is wholly on screen. Not exported; `step()` and `onScreen()` use it.

- `[x, x + size / 2, x + size]` and the same for `y`: nine points around the bubble's square: its corners, the middle of each side and its centre. Checking the middles too catches a narrow gap between two monitors that the corners alone could straddle.
- `areas.some(...)`: each point has to be inside at least one display's work area. The edges count as inside (`<=`), so where two monitors touch, a bubble half on each still fits.

`onScreen()`

src/main/bubble/bounce.ts, lines 89–107

```ts
/**
 * The bubble's position if it fits on screen; otherwise (a display was unplugged or shrank
 * mid-bounce) the nearest place on the closest display.
 */
function onScreen(position: Point, areas: Rect[]): Point {
  if (fits(position.x, position.y, areas)) return position
  // Only areas the bubble fits in on its own (a strip over a taskbar is too thin).
  const placed = areas
    .map(travelBox)
    .filter((box) => box.maxX >= box.minX && box.maxY >= box.minY)
    .map((box) => {
      const x = Math.min(Math.max(position.x, box.minX), box.maxX)
      const y = Math.min(Math.max(position.y, box.minY), box.maxY)
      return { x, y, distance: Math.hypot(x - position.x, y - position.y) }
    })
  if (placed.length === 0) return position
  const nearest = placed.reduce((best, p) => (p.distance < best.distance ? p : best))
  return { x: nearest.x, y: nearest.y }
}
```

Where to carry on from. Normally that's simply where the bubble is; but if a monitor was unplugged or shrank mid-bounce, the bubble may be somewhere no display covers any more.

- `.map(travelBox)`: for each area, the positions where the bubble is wholly inside it (`layout.ts`).
- `.filter((box) => box.maxX >= box.minX && box.maxY >= box.minY)`: leaves out areas too small to hold the bubble on their own, such as the strip over a taskbar between two monitors, which the bubble only passes over.

- `Math.min(Math.max(position.x, box.minX), box.maxX)`: the closest such position to where the bubble is (clamping each coordinate into the box).
- `placed.reduce(...)`: picks the area whose closest position is the least distance away.

`glideDuration()`

`src/main/bubble/bounce.ts`, lines 109–113

```ts
/** Glide length scales with distance, kept between 0.3 s and 0.9 s. */
export function glideDuration(from: Point, to: Point): number {
  const distance = Math.hypot(to.x - from.x, to.y - from.y)
  return Math.min(900, Math.max(300, distance / 1.5))
}
```

How long the glide home takes, in milliseconds: the straight-line distance (`Math.hypot`) divided by 1.5, so roughly 1.5 DIPs per millisecond, but never less than 300 ms or more than 900 ms. A short hop still looks deliberate, and a long trip across the screen doesn't drag on.

`glidePosition()`

`src/main/bubble/bounce.ts`, lines 115–120

```ts
/** Position at progress `t` (0–1) along the glide home, easing out as it settles. */
export function glidePosition(from: Point, to: Point, t: number): Point {
  const progress = Math.min(Math.max(t, 0), 1)
  const eased = 1 - (1 - progress) ** 3
  return { x: from.x + (to.x - from.x) * eased, y: from.y + (to.y - from.y) * eased }
}
```

Where the bubble is at progress `t` along the glide (0 at the start, 1 at home).

- `Math.min(Math.max(t, 0), 1)`: clamps `t`, so the bubble can never overshoot home.
- `1 - (1 - progress) ** 3`: a cubic *ease-out* curve (`**` means "to the power of"). It climbs quickly at first and flattens out toward 1, so the bubble moves fast at the start and slows as it settles into the corner.
- `from.x + (to.x - from.x) * eased`: moves that fraction of the way from the start to home, on each axis.

`src/main/bubble/windows.ts`: creating the Electron windows

The only file in this folder that uses Electron. It creates the two overlay windows, locks them down, and adapts them to the controller's `Surface` interface. `index.ts` calls `createOverlayWindow()` twice at startup, once per view, and wraps the results with `bubbleSurface()` and `panelSurface()` before handing them to `BubbleController`.

Key ideas:

- **Two windows, one page.** Both windows load the same renderer page; `?view=bubble` or `?view=panel` in its address tells `main.tsx` which UI to draw. The bubble has its own tiny window so it never resizes when the panel opens, and only a 72×72 window moves during a drag or a bounce.
- **Click-through.** The panel window is a large rectangle (sized for the chat card at its tallest), but most of it is transparent. `setIgnoreMouseEvents(true, { forward: true })` makes Windows pass clicks on it to whatever is underneath, while still telling the page where the pointer is. While the pointer is over a piece of real UI (an

element marked `data-hit`), the page's `useClickThrough` hook asks the main process to take mouse input again (`IPC.setInteractive`, handled in `ipc.ts`). The bubble window is never click-through: it's barely larger than the bubble, and relying on Windows' mouse forwarding there made clicks occasionally fall through the bubble to the window behind it.

`View`

`src/main/bubble/windows.ts`, line 7

```ts
export type View = 'bubble' | 'panel'
```

A *union type*: a `View` can only be the string `'bubble'` or `'panel'`, and TypeScript rejects anything else.

`createOverlayWindow()`

`src/main/bubble/windows.ts`, lines 9–62

```ts
/**
 * Creates one of the two overlay windows. Both are frameless, transparent, always on top and
 * hidden from the taskbar and Alt+Tab.
 *
 * The panel window is mostly see-through, so it's click-through: clicks on its empty areas go to
 * whatever is underneath, and the renderer turns mouse input back on while the pointer is over
 * real UI (see useClickThrough). The bubble window is never click-through: it's barely larger
 * than the bubble, and relying on Windows' mouse forwarding there made clicks occasionally fall
 * through the bubble to the window behind it.
 */
export function createOverlayWindow(view: View, size: Size): BrowserWindow {
  const win = new BrowserWindow({
    ...size,
    show: false,
    frame: false,
    transparent: true,
    backgroundColor: '#00000000',
    hasShadow: false,
    thickFrame: false,
    resizable: false,
    minimizable: false,
    maximizable: false,
    fullscreenable: false,
    skipTaskbar: true,
    alwaysOnTop: true,
    type: 'toolbar',
    // Both can take the focus. A bubble that couldn't had its mouse presses dropped by Windows /
    // Chromium after clicking elsewhere (only the release arrived, so clicks did nothing).
    // Clicking the bubble now takes the focus from an open panel; BubbleController.clickBubble
    // knows that press closed it.
    focusable: true,
    webPreferences: {
      preload: join(__dirname, '../preload/index.js'),
      sandbox: true,
      contextIsolation: true,
      nodeIntegration: false,
      backgroundThrottling: false,
      spellcheck: view === 'panel',
    },
  })
  lockDown(win, view)
  if (view === 'panel') {
    win.setIgnoreMouseEvents(true, { forward: true })
```

```
    addEditContextMenu(win)
  }

  const devServer = process.env['ELECTRON_RENDERER_URL']
  if (!app.isPackaged && devServer) {
    void win.loadURL(`${devServer}?view=${view}`)
  } else {
    void win.loadFile(join(__dirname, '../renderer/index.html'), { query: { view } })
  }
  return win
}
```

Creates one overlay window. `new BrowserWindow({...})` makes a native window with a Chromium page inside; the options say what kind of window.

- `...size`: the window's `width` and `height`: `BUBBLE_BOX` square for the bubble, `panelWindowSize(actionCount)` for the panel. Neither window changes size later, only position.
- `show: false`: created hidden. The controller decides when each one appears.
- `transparent: true`: together with `frame: false` (no title bar or border) and a fully transparent `backgroundColor` (in `'#00000000'` the last two hex digits are the alpha), only what the page draws is visible.
- `thickFrame: false`: Windows-only. The standard resizable frame would bring the system shadow and the minimise and maximise animations. With `hasShadow: false`, Windows adds no shadow; the page draws its own.
- `type: 'toolbar'`: on Windows, a toolbar window is left out of Alt+Tab. `skipTaskbar: true` keeps it off the taskbar too.
- `focusable: true`: both windows can take the keyboard focus. The panel must, since you type in it. The bubble used to be unable to (`focusable: false`), so that clicking it wouldn't take the focus from the panel or from the app you were working in. But, as the comment says, in some states (after clicking somewhere else) Windows and Chromium dropped the mouse *press* on that never-focusable window and delivered only the release. The page then saw no `pointerdown` and no `click`, so clicks on the bubble did nothing at all, neither opening nor closing the panel. This was found by attaching to the running app on a PC where it happened. With a focusable bubble every press arrives; the cost is that pressing the bubble now blurs an open panel, which `BubbleController.clickBubble()` allows for with the press time.
- `preload: join(__dirname, '../preload/index.js')`: the preload script, which gives the page `window.assist`. `__dirname` is the folder of the built main-process file (`out/main`), so this points at the built preload in `out/preload`.
- `sandbox: true`: with `contextIsolation: true` and `nodeIntegration: false`, the page runs like an ordinary web page: no Node.js, no access to the preload's internals, only what the preload hands it. Even hostile code in the page couldn't reach files.
- `backgroundThrottling: false`: Chromium normally slows timers and animation frames in windows it thinks are hidden or in the background. These windows are hidden often and must respond at once when shown.
- `spellcheck: view === 'panel'`: spell checking only where there's a text box.
- `win.setIgnoreMouseEvents(true, { forward: true })`: the panel starts click-through. The `forward` option (Windows and macOS only) keeps sending mouse moves to the page, which is what lets `useClickThrough` notice the pointer arriving over real UI.
- `process.env['ELECTRON_RENDERER_URL']`: set by `electron-vite` during `npm run dev` to the address of its dev server, which serves the page with live reloading. `!app.isPackaged` makes sure an installed app always loads the page bundled with it.
- `{ query: { view } }`: `loadFile()` adds `?view=bubble` or `?view=panel` to the built page's address.

- `void win.loadURL(`: loading returns a promise that isn't awaited; `void` marks that as deliberate.

`bubbleSurface()`, `panelSurface()`

src/main/bubble/windows.ts, lines 64–82

```
export function bubbleSurface(win: BrowserWindow): Surface {
  return {
    setBounds: (bounds) => win.setBounds(bounds),
    show: () => win.showInactive(),
    hide: () => win.hide(),
  }
}

export function panelSurface(win: BrowserWindow): PanelSurface {
  return {
    setBounds: (bounds) => win.setBounds(bounds),
    show: () => {
      resetClickThrough(win)
      win.show()
    },
    hide: () => win.hide(),
    focus: () => win.focus(),
  }
}
```

Adapters: each wraps a real `BrowserWindow` in the small object the controller expects, and forwards each call to the matching Electron method.

- `show: () => win.showInactive()`: shows the bubble without activating it, so it never steals focus.
- `resetClickThrough(win)`: the panel's click-through is reset each time, just before it's shown.
- `win.show()`: unlike `showInactive()`, this shows and activates the panel.
- `focus: () => win.focus()`: gives the panel keyboard focus, so you can type straight away.
- `hide: () => win.hide()`: hides the window; its page keeps running.

`resetClickThrough()`

src/main/bubble/windows.ts, lines 84–96

```
/**
 * Starts the panel's click-through afresh each time it's shown. While hidden, the page can't
 * follow the pointer, so its idea of whether the pointer is over real UI is out of date (and
 * Windows may have dropped the mouse forwarding). Switching forwarding off and on re-installs it,
 * and the page is told exactly where the pointer is now, so it can decide straight away.
 */
function resetClickThrough(win: BrowserWindow): void {
  win.setIgnoreMouseEvents(false)
  win.setIgnoreMouseEvents(true, { forward: true })
  const cursor = screen.getCursorScreenPoint()
  const bounds = win.getBounds()
  win.webContents.send(IPC.clickThroughReset, { x: cursor.x - bounds.x, y: cursor.y - bounds.y })
}
```

Starts the panel's click-through afresh each time it's shown. While the panel is hidden, its page can't follow the pointer, so its idea of whether the pointer is over real UI is out of date, and Windows may have dropped the mouse forwarding. This fixes clicks going astray after the panel has been hidden, for example for a screenshot.

- `win.setIgnoreMouseEvents(false)`: switching off and then on again installs the forwarding again, and leaves the window click-through.
- `screen.getCursorScreenPoint()`: with `win.getBounds()`, both in DIPs, so subtracting gives the pointer's position inside the window. That's what the page calls `clientX` and `clientY`, since a frameless page fills its window exactly.
- `win.webContents.send(IPC.clickThroughReset,`: tells the page. `useClickThrough` checks what's under that point on the next animation frame and takes mouse input straight away if it's over real UI.

`lockDown()`

`src/main/bubble/windows.ts`, lines 98–107

```
function lockDown(win: BrowserWindow, view: View): void {
  const contents = win.webContents
  contents.setWindowOpenHandler(() => ({ action: 'deny' }))
  contents.on('will-navigate', (event) => event.preventDefault())
  contents.on('render-process-gone', () => {
    // A crashed panel can't report the pointer, so let clicks through until it has reloaded.
    if (view === 'panel') win.setIgnoreMouseEvents(true, { forward: true })
    contents.reload()
  })
}
```

Basic safety and crash recovery for both pages.

- `setWindowOpenHandler(() => ({ action: 'deny' }))`: the page may never open a new Electron window (`window.open`, or a link with `target="_blank"`). Links open in your browser through `IPC.openExternal` instead.
- `contents.on('will-navigate', (event) => event.preventDefault())`: the page can never be replaced by another page, for example by following a link or by a file dropped onto the window.
- `render-process-gone`: Electron fires this when the page's process crashes or is killed. The page is reloaded.
- `if (view === 'panel') win.setIgnoreMouseEvents(true, { forward: true })`: a crashed panel can't report where the pointer is, and might be left taking the mouse over its whole invisible rectangle. Making it click-through until the page has reloaded avoids a dead patch of screen. The reloaded `useClickThrough` then starts from "not interactive" again.

`addEditContextMenu()`

src/main/bubble/windows.ts, lines 109–130

```ts
/** Right-click menu for the text box: spelling suggestions plus the usual edit commands. */
function addEditContextMenu(win: BrowserWindow): void {
  win.webContents.on('context-menu', (_event, params) => {
    if (!params.isEditable) return
    const suggestions: MenuItemConstructorOptions[] = params.dictionarySuggestions
      .slice(0, 5)
      .map((word) => ({ label: word, click: () => win.webContents.replaceMisspelling(word) }))
    const template: MenuItemConstructorOptions[] = [
      ...suggestions,
      ...(suggestions.length > 0 ? [{ type: 'separator' as const }] : []),
      { role: 'undo' },
      { role: 'redo' },
      { type: 'separator' },
      { role: 'cut' },
      { role: 'copy' },
      { role: 'paste' },
      { type: 'separator' },
      { role: 'selectAll' },
    ]
    Menu.buildFromTemplate(template).popup({ window: win })
  })
}
```

The right-click menu for the panel's text box. Electron shows no context menu by default, so without this you couldn't cut, paste or fix spelling with the mouse.

- `if (!params.isEditable) return`: only in editable fields, not on the rest of the panel.
- `params.dictionarySuggestions`: Chromium's spelling suggestions for the word under the pointer (none if it's spelt correctly). At most five are shown; choosing one calls `replaceMisspelling(word)`.
- `{ type: 'separator' as const }`: a separator line, added only when there are suggestions. `as const` tells TypeScript the type is exactly `'separator'`, not just any string, which the menu item type requires.
- `{ role: 'undo' }`: a `role` gives Electron's built-in item, with its standard label, shortcut and behaviour. The same goes for redo, cut, copy, paste and select all.
- `Menu.buildFromTemplate(template).popup({ window: win })`: builds the menu and shows it at the pointer.

The draft, screenshots and safe files

This part covers the main-process code that keeps things on disk. `NotesStore` saves the unsent draft so it survives a restart; `ScreenshotService` and its helpers take, preview, shrink and combine screenshots, and make sure the pages can only touch files in the screenshots folder. Underneath both, `jsonFile.ts` reads saved files with a validity check and writes them so that a crash can't leave a half-written file. All of this runs in the main process (Electron's Node.js side, which can use the disk); the pages reach it only through the IPC handlers in `src/main/ipc.ts`.

`src/main/notes/NotesStore.ts`: saving the draft

The draft is the text typed into the chat box but not yet sent, plus any screenshots attached to it. `NotesStore` holds it in memory and saves it to `notes.json` in the app's data folder (`app.getPath('userData')`, under `%APPDATA%`), so a half-written message is still there after a restart or a crash. `src/main/index.ts` creates one store at startup, loads it, and flushes it when the app quits. `src/main/ipc.ts` calls it whenever the page changes the draft, and clears it when a message is sent or handed to Claude Desktop, or when the user picks "Clear text box".

Key ideas:

- **Debounced autosave.** The page sends the whole text on every keystroke. Writing a file each time would be wasteful, so every change restarts a 500 ms timer, and the file is written only once typing pauses. To "debounce" means to wait until calls stop coming for a moment, then act once.
- **Serialised writes.** Saves wait in a queue (a chain of promises), so two writes to the file never run at the same time.
- **Overtaken writes are dropped.** Each save is numbered. A save that would land after a newer one has already reached the disk throws itself away instead of putting older text back.
- **Validated loading.** The file is checked against a zod schema when it's read. A damaged file is renamed out of the way instead of crashing the app or being silently overwritten.

`MAX_NOTE_LENGTH` and `NotesFileSchema`

src/main/notes/NotesStore.ts, lines 8–17

```
export const MAX_NOTE_LENGTH = 200_000

const NotesFileSchema = z.object({
  version: z.literal(1),
  text: z.string(),
  attachments: z.array(
    z.object({ id: z.string(), path: z.string(), fileName: z.string(), addedAt: z.string() }),
  ),
  updatedAt: z.string().nullable(),
})
```

`MAX_NOTE_LENGTH` is a safety limit on the draft's size. `NotesFileSchema` describes exactly what a valid `notes.json` looks like.

- `200_000`: the underscore is only a digit separator, for readability (200,000 characters). `ipc.ts` uses the same constant to reject over-long text in its `notesSetText` and `chatSend` handlers.
- `z.object({`: this is **zod**, a validation library. A schema describes the expected shape; `schema.parse(data)` returns the data if it matches and throws if it doesn't. TypeScript's types disappear when the code runs, so a file

read from disk (which a crash could damage, or anyone could edit) must be checked at run time. `readJsonFile()` in `jsonFile.ts` does the checking. Unknown extra fields are dropped by `parse`.

- `version: z.literal(1)`: the file records its format version, and only `1` is accepted, so a future format can be told apart. Today a file with any other version counts as invalid and is set aside like a damaged one (see `load()`).
- `updatedAt: z.string().nullable()`: a timestamp string, or `null` if nothing has been typed yet.

`emptyNotes` and `NotesStoreOptions`

src/main/notes/NotesStore.ts, lines 19–27

```ts
export const emptyNotes = (): Notes => ({ text: '', attachments: [], updatedAt: null })

export interface NotesStoreOptions {
  debounceMs?: number
  /** Called when everything changed so far has reached the disk. */
  onSaved?: () => void
  onError?: (error: unknown) => void
  now?: () => Date
}
```

`emptyNotes()` makes a blank draft. `NotesStoreOptions` are the optional settings the constructor accepts.

- `emptyNotes = (): Notes =>`: a function rather than one shared constant, so every caller gets its own fresh `attachments` array. `Notes` is the draft type from `src/shared/types.ts`.
- `debounceMs?`: the `?` makes the option optional. It's the pause before saving, 500 ms when left out; the tests set it very short or very long.
- `onSaved`: only the tests use it, to know when a save has finished.
- `onError`: `index.ts` passes a function that logs "Saving the text box failed".
- `now`: a way to supply the current time, for testing. When it's left out, as in the app, the real clock is used.

`class NotesStore`

The class, with the comment that describes it:

src/main/notes/NotesStore.ts, lines 29–33

```ts
/**
 * The text box's contents. Changes are kept in memory and written to disk shortly after the
 * last edit; writes are serialised, and a write overtaken by a newer one is discarded.
 */
export class NotesStore {
  // …
}
```

`NotesStore` fields and constructor

src/main/notes/NotesStore.ts, lines 34–44

```
private notes: Notes = emptyNotes()

private dirty = false

private timer: ReturnType<typeof setTimeout> | null = null

private writes: Promise<void> = Promise.resolve()

private seq = 0

private committedSeq = 0

constructor(
  private readonly filePath: string,
  private readonly options: NotesStoreOptions = {},
) {}
```

The in-memory `notes` is the real draft while the app runs; the file is only a copy. The other fields are bookkeeping for saving.

- `dirty`: true when there are changes that haven't been handed to a write yet.
- `ReturnType<typeof setTimeout>`: "whatever `setTimeout` returns". That's an object in Node and a number in browsers, and this type fits both. `timer` is the pending autosave, or `null`.
- `writes`: the end of the write queue. It starts as an already-finished promise, and each new write is chained on with `.then`, so writes happen strictly one after another.
- `seq`: counts the snapshots handed to a write. `committedSeq` is the number of the newest snapshot actually on disk. Comparing the two is how an overtaken write knows to give up (see `write()`).
- `private readonly filePath: string`: in a constructor, putting `private` or `readonly` before a parameter is TypeScript shorthand that declares a field and stores the argument in it. That's why the constructor body is empty yet the class has `this.filePath` and `this.options`.
- `options: NotesStoreOptions = {}`: `{}` is the default when no options are passed.

`NotesStore.load()`

src/main/notes/NotesStore.ts, lines 46–56

```
async load(): Promise<Notes> {
  const result = await readJsonFile(this.filePath, NotesFileSchema)
  if (result.status === 'ok') {
    const { text, attachments, updatedAt } = result.value
    this.notes = { text, attachments, updatedAt }
  } else {
    if (result.status === 'invalid') await this.setAsideCorruptFile()
    this.notes = emptyNotes()
  }
  return this.get()
}
```

Reads `notes.json` and makes it the current draft. `index.ts` calls it once at startup, before it registers the IPC handlers.

- `readJsonFile(this.filePath, NotesFileSchema)`: returns one of three results: `ok` (with the checked value), `missing` (no file yet, e.g. on the first run) or `invalid` (not JSON, or the wrong shape). Any other read error, such as permission denied, is thrown to the caller.
- `const { text, attachments, updatedAt } = result.value`: keeps just the draft's fields, leaving the file's `version` behind.
- `this.setAsideCorruptFile()`: a damaged file is renamed, not deleted, so its contents can still be recovered by hand and the next save doesn't overwrite it. A missing file needs nothing; both cases start from an empty draft.

`NotesStore.get()`

src/main/notes/NotesStore.ts, lines 58–60

```
get(): Notes {
  return { ...this.notes, attachments: this.notes.attachments.map((a) => ({ ...a })) }
}
```

Returns a copy of the draft. `ipc.ts` uses it when sending a message, and `index.ts` includes it in the app state the pages ask for when they load.

- `attachments: this.notes.attachments.map((a) => ({ ...a }))`: `{ ...x }` (the spread syntax) copies an object's fields into a new object. Spreading `this.notes` alone would copy only the top level and share the attachments array, so each attachment is copied too. Callers can then change what they got without changing the store (a test checks this).

`NotesStore.setText()`

src/main/notes/NotesStore.ts, lines 62–65

```
setText(text: string): void {
  const next = text.slice(0, MAX_NOTE_LENGTH)
  if (next !== this.notes.text) this.update({ text: next })
}
```

Replaces the draft's text. `ipc.ts` calls it for the `notesSetText` message, which the page sends on every change in the text box.

- `text.slice(0, MAX_NOTE_LENGTH)`: cuts over-long text. `ipc.ts` already ignores a message longer than this, so this is a second line of defence for any other caller.
- `if (next !== this.notes.text)`: unchanged text does nothing, so it doesn't move `updatedAt` or trigger a pointless save.

`NotesStore.isAttached()`

src/main/notes/NotesStore.ts, lines 67–71

```
/** Adds a screenshot to the text box. Returns null if that file is already attached. */
/** Whether the screenshot is already attached to the draft. */
isAttached(path: string): boolean {
  return this.notes.attachments.some((a) => samePath(a.path, path))
}
```

Whether a screenshot is already on the draft. The **Attach screenshot** handler in `ipc.ts` passes it to `ScreenshotService.latest()` so the screenshots already attached are passed over, and `addAttachment()` uses it to refuse a second copy.

- `samePath(a.path, path)`: compares the paths the way Windows does (see `samePath` below).

`NotesStore.addAttachment()`

src/main/notes/NotesStore.ts, lines 73–83

```
addAttachment(path: string): Notes | null {
  if (this.isAttached(path)) return null
  const attachment = {
    id: randomUUID(),
    path,
    fileName: basename(path),
    addedAt: this.now().toISOString(),
  }
  this.update({ attachments: [...this.notes.attachments, attachment] })
  return this.get()
}
```

Attaches a screenshot file to the draft. `ipc.ts` calls it from the **Attach screenshot** handler (`notesAttachLatest`) with the path from `ScreenshotService.latest()`, so the page never supplies the path.

- `this.isAttached(path)`: refuses a file that's already attached, answering `null`. The handler already skips attached screenshots, so this is a second line of defence.
- `randomUUID()`: a random unique ID from Node's `crypto` module, used later to remove this attachment.
- `basename(path)`: the file name without its folder, e.g. `Screenshot 2026-10-02 140311.png`. The page turns it into the chip's label.
- `this.now().toISOString()`: a timestamp such as `2026-10-02T03:03:11.000Z` (always in UTC).

`NotesStore.removeAttachment()` and `NotesStore.clear()`

src/main/notes/NotesStore.ts, lines 85–93

```
removeAttachment(id: string): Notes {
  this.update({ attachments: this.notes.attachments.filter((a) => a.id !== id) })
  return this.get()
}

clear(): Notes {
  this.update({ text: '', attachments: [] })
  return this.get()
}
```

`removeAttachment()` drops one screenshot by its ID (the × on a chip). `clear()` empties the draft: `ipc.ts` calls it after a message has been handed to `ChatSession.send()` or a question to Claude Desktop, and for "Clear text box" in the settings menu, and returns the empty draft to the page. Only the draft's list of attachments is emptied; the screenshot files stay in the folder. Both go through the same debounced autosave as any other change, so the saved file is emptied too shortly after a message is sent.

- `filter((a) => a.id !== id)`: an unknown ID removes nothing, but it still counts as a change and schedules a save.

`NotesStore.flush()`

src/main/notes/NotesStore.ts, lines 95–105

```ts
/** Writes any pending change now and waits for all writes to finish. */
flush(): Promise<void> {
  this.clearTimer()
  if (this.dirty) {
    this.dirty = false
    const seq = ++this.seq
    const snapshot = this.toFile()
    this.writes = this.writes.then(() => this.write(snapshot, seq))
  }
  return this.writes
}
```

Saves any pending change now instead of waiting for the timer, and returns a promise that settles when every queued write has finished. The autosave timer calls it, and so does `index.ts` when the app is quitting (giving it at most 2 seconds before quitting anyway).

- `this.clearTimer()`: a pending autosave is no longer needed.
- `const seq = ++this.seq`: numbers this snapshot. `++x` adds one first, then gives the new value.
- `const snapshot = this.toFile()`: copies the draft as it is now. Edits made while this write waits in the queue don't change what it saves; they mark the store dirty again and get a write of their own.
- `this.writes = this.writes.then(() => this.write(snapshot, seq))`: adds the write to the end of the queue. It starts only after every earlier write has finished, so writes never overlap.
- `return this.writes`: even when nothing was dirty, the caller waits for any write already in progress. That's what makes awaiting `flush()` on quit mean "everything is on disk".

`NotesStore.flushSync()`

src/main/notes/NotesStore.ts, lines 107–115

```ts
/** Blocking write for when the OS is ending the session and async work may never finish. */
flushSync(): void {
  this.clearTimer()
  if (!this.dirty) return
  this.dirty = false
  const seq = ++this.seq
  writeJsonFileSync(this.filePath, this.toFile())
  this.committedSeq = seq
}
```

The emergency version of `flush()`. When Windows logs off or shuts down, Electron fires `session-end` on the window, and `index.ts` calls this. Windows may end the process before any async work completes, so this can't use the queue.

- `writeJsonFileSync(this.filePath, this.toFile())`: a blocking write. The event handler doesn't return until the file is saved.
- `this.committedSeq = seq`: records that this newer snapshot is on disk. An async write still in the queue has a smaller number, so when it reaches its commit check it discards itself instead of putting older text back (the "overtaken write" test checks exactly this).

Unlike `write()`, it doesn't catch errors: a failure is thrown from the event handler.

`NotesStore.update()`

src/main/notes/NotesStore.ts, lines 117–122

```
private update(patch: Partial<Notes>): void {
  this.notes = { ...this.notes, ...patch, updatedAt: this.now().toISOString() }
  this.dirty = true
  this.clearTimer()
  this.timer = setTimeout(() => void this.flush(), this.options.debounceMs ?? 500)
}
```

The private helper every change goes through. It applies the change, stamps the time and restarts the autosave timer; this is where the debounce happens.

- `Partial<Notes>`: TypeScript for "a `Notes` with every field optional", so callers pass only what changed.
- `{ ...this.notes, ...patch, updatedAt: this.now().toISOString() }`: builds a new draft object. Later fields win, so the values in `patch` replace the old ones.
- `this.clearTimer()`: each change cancels the previous timer before starting a new one. While you keep typing the save keeps being pushed back; 500 ms after the last keystroke the timer finally fires and saves once.
- `() => void this.flush()`: `void` marks the promise as deliberately not awaited. Nothing here could act on the result, and errors are handled inside `write()`.
- `this.options.debounceMs ?? 500`: `??` uses the right-hand value only when the left is `null` or `undefined`, so a caller could still pass `0`.

`NotesStore.write()`

src/main/notes/NotesStore.ts, lines 124–133

```
private async write(snapshot: object, seq: number): Promise<void> {
  try {
    const committed = await writeJsonFile(this.filePath, snapshot, () => seq > this.committedSeq)
    if (committed) this.committedSeq = seq
    if (!this.dirty) this.options.onSaved?.()
  } catch (error) {
    this.dirty = true // try again on the next change or flush
    this.options.onError?.(error)
  }
}
```

Performs one queued save, through `writeJsonFile()` in `jsonFile.ts`.

That call is an **atomic write**: it doesn't write `notes.json` directly. It writes the whole new content to a temporary file next to it, then renames the temporary file over the real one. A rename is all-or-nothing, so if the app crashes mid-save, `notes.json` holds either the old draft or the new one, never a mix. See `writeFileAtomic()` below.

- `() => seq > this.committedSeq`: the commit check, which `writeJsonFile` calls just before the rename. If a newer snapshot has already reached the disk (from `flushSync()`), this one is out of date and is thrown away.
- `if (committed) this.committedSeq = seq`: records the newest snapshot on disk.
- `if (!this.dirty) this.options.onSaved?.()`: reports "saved" only when no newer change is waiting. `?.()` calls the function only if one was provided.
- `this.dirty = true`: if the save failed (say, the disk is full), the draft is marked unsaved again, so the next change or flush retries with the latest text. No new timer is started.

- `catch (error)`: catching also protects the queue. Because `write()` never rejects, one failed save can't break the promise chain in `writes`; a rejected promise in the chain would make every later `.then` step skip its write.

`NotesStore.toFile()`

src/main/notes/NotesStore.ts, lines 135–137

```
private toFile() {
  return { version: 1 as const, ...this.notes }
}
```

The object that gets written to disk: the draft plus `version: 1`.

- `1 as const`: tells TypeScript the type is exactly `1`, not any number, matching `z.literal(1)` in `NotesFileSchema`.
- `...this.notes`: only the top level is copied. That's safe because every change replaces `notes` and its attachments array with new values rather than editing them in place, so a snapshot can't change after it's taken.

`NotesStore.setAsideCorruptFile()`

src/main/notes/NotesStore.ts, lines 139–146

```
private async setAsideCorruptFile(): Promise<void> {
  const backup = `${this.filePath}.corrupt-${this.now().getTime()}`
  try {
    await fs.rename(this.filePath, backup)
  } catch (error) {
    this.options.onError?.(error)
  }
}
```

Renames a damaged `notes.json` to `notes.json.corrupt-<time>`, so it's kept for inspection while the app starts with an empty draft.

- `this.now().getTime()`: the time in milliseconds since 1970, which makes each backup name unique.
- `catch (error)`: if even the rename fails, the error is reported through `onError` and startup carries on. The next save will then overwrite the damaged file.

`NotesStore.clearTimer()` and `NotesStore.now()`

src/main/notes/NotesStore.ts, lines 148–155

```
private clearTimer(): void {
  if (this.timer) clearTimeout(this.timer)
  this.timer = null
}

private now(): Date {
  return this.options.now?.() ?? new Date()
}
```

Two small helpers. `clearTimer()` cancels any pending autosave. `now()` takes the time from the `now` option when there is one, and otherwise from the real clock.

`samePath`

src/main/notes/NotesStore.ts, lines 158–161

```
function samePath(a: string, b: string): boolean {
  const norm = (p: string) => resolve(p).toLowerCase()
  return norm(a) === norm(b)
}
```

Compares two file paths the way Windows does, for `addAttachment()`.

- `resolve(p)`: turns a path into a full, normalised absolute path (working out `.` and `..` parts and using one kind of slash).
- `.toLowerCase()`: Windows file names ignore case, so `C:\Pics\Shot.png` and `c:\pics\shot.png` are the same file.

`src/main/screenshots/ScreenshotService.ts`: capturing and serving screenshots

Everything the app does with screenshot images goes through this class: taking one, finding the newest, making small previews, preparing one for Claude, combining several into one image for the clipboard, and opening one in another app. `index.ts` creates a single instance with the screenshots folder (`Pictures\<appName>`, for example `Pictures\Desktop Assist`) and a function that returns the display the bubble is on. `actions.ts` calls `capture()` for the camera button, `index.ts` uses `forClipboard()` when a question goes to Claude Desktop or an IT request to the service desk portal, and `ipc.ts` exposes the rest to the pages.

Key ideas:

- **Electron's image tools.** `desktopCapturer` grabs the screen, and `nativeImage` loads, resizes and encodes images, so no extra image library is needed. Both are explained where they're first used below.
- **The inside-the-folder check.** Several methods take a file path from a page. The main process doesn't trust what pages send (`ipc.ts` validates every argument for the same reason): if a page were ever compromised, it mustn't be able to make the main process preview, read or open any file on the PC. Opening an `.exe` this way would even run it. So every method that takes a path first checks, with `owns()`, that the file is inside the screenshots folder. How the check works is in `isInsideDir()` in `files.ts`.

Constants

src/main/screenshots/ScreenshotService.ts, lines 8–10

```
const THUMBNAIL_HEIGHT = 112

const MAX_CACHED_THUMBNAILS = 50

const JPEG_QUALITY = 85
```

- `THUMBNAIL_HEIGHT = 112`: a chip's preview is drawn 56 pixels tall. Making the image twice that keeps it sharp on high-DPI screens, where Windows scales the display up.
- `MAX_CACHED_THUMBNAILS = 50`: how many previews to keep in memory.
- `JPEG_QUALITY = 85`: the JPEG quality (0 to 100) for images sent to Claude, a common balance between file size and sharpness.

`ScreenshotService` fields and constructor

src/main/screenshots/ScreenshotService.ts, lines 13–19

```
private readonly thumbnails = new Map<string, string>()

constructor(
  readonly dir: string,
  /** The display to capture: the one the bubble is on. */
  private readonly targetDisplay: () => Display,
) {}
```

- `new Map<string, string>()`: the preview cache. Each key is a path plus the file's modified time; each value is a preview as a data URL.
- `readonly dir: string`: the screenshots folder. Without `private` it's a public, read-only field (the same constructor shorthand as in `NotesStore`).
- `targetDisplay: () => Display`: a function rather than a fixed value, because the bubble can be dragged to another monitor. `index.ts` passes `screen.getDisplayMatching(bubbleWindow.getBounds())`, the display that most of the bubble window is on. A `Display` is Electron's description of a monitor: its ID, size, position and scale factor.

`ScreenshotService.capture()`

src/main/screenshots/ScreenshotService.ts, lines 21–34

```
/** Captures the target display at full resolution and saves it. Returns the file path. */
async capture(): Promise<string> {
  const display = this.targetDisplay()
  const thumbnailSize = {
    width: Math.round(display.size.width * display.scaleFactor),
    height: Math.round(display.size.height * display.scaleFactor),
  }
  const sources = await desktopCapturer.getSources({ types: ['screen'], thumbnailSize })
  const source =
    sources.find((s) => s.display_id === String(display.id)) ??
    (sources.length === 1 ? sources[0] : undefined)
  if (!source || source.thumbnail.isEmpty()) throw new Error('Could not capture the screen')
  return saveScreenshot(this.dir, source.thumbnail.toPNG(), new Date())
}
```

Takes a picture of the whole display the bubble is on, saves it as a PNG, and returns the new file's path. The camera button's handler in `actions.ts` runs it inside `BubbleController.whileHidden()`, which hides the bubble and the panel first so they aren't in the picture.

`desktopCapturer` is Electron's way to list the screens and windows that can be captured. It's mostly used to start screen sharing, but every source it returns comes with a still `thumbnail` image. Asking for a thumbnail as big as the screen turns that "thumbnail" into a full-resolution screenshot.

- `display.size.width * display.scaleFactor`: Electron measures displays in device-independent pixels (DIPs), which ignore Windows' display scaling. Multiplying by the scale factor gives the real pixel size: a 3840×2160 screen at 150% scaling is 2560×1440 DIPs with a scale factor of 1.5. `Math.round` keeps the result a whole number.
- `types: ['screen']`: whole screens only, not individual windows.
- `s.display_id === String(display.id)`: picks the source for the bubble's display. A capture source names its display with a string, while `Display.id` is a number.

- `(sources.length === 1 ? sources[0] : undefined)`: Electron documents that `display_id` can be empty when it isn't available. With only one screen there's no doubt which one it is, so it's used anyway; with several, a guess could capture the wrong screen, so it fails instead.
- `source.thumbnail.isEmpty()`: an empty image means the capture itself failed.
- `throw new Error('Could not capture the screen')`: `actions.ts` catches this, logs it, and the page shows "Couldn't take a screenshot".
- `source.thumbnail.toPNG()`: encodes the image as PNG bytes (a Node `Buffer`). PNG is lossless, so the file is exactly what was on screen.
- `saveScreenshot(this.dir, source.thumbnail.toPNG(), new Date())`: from `files.ts`. It names the file after the current local time and never overwrites an existing one.

`ScreenshotService.latest()`

src/main/screenshots/ScreenshotService.ts, lines 36–39

```
/** The newest screenshot, passing over any that `skip` says to (e.g. already attached). */
latest(skip?: (path: string) => boolean): Promise<string | null> {
  return findLatestScreenshot(this.dir, skip)
}
```

The newest screenshot in the folder, or `null` if there isn't one. The **Attach screenshot** handler in `ipc.ts` calls it with `skip` set to "already attached", so each click attaches the next one back, and passes the result to `NotesStore.addAttachment()`. The work is done by `findLatestScreenshot()` in `files.ts`.

`ScreenshotService.thumbnail()`

src/main/screenshots/ScreenshotService.ts, lines 41–61

```
/** A small preview as a data URL, or null if the file is missing or not one of ours. */
async thumbnail(path: string): Promise<string | null> {
  if (!this.owns(path)) return null
  let mtimeMs: number
  try {
    mtimeMs = (await fs.stat(path)).mtimeMs
  } catch {
    return null
  }
  const key = `${path}|${mtimeMs}`
  const cached = this.thumbnails.get(key)
  if (cached) return cached
  const image = nativeImage.createFromPath(path)
  if (image.isEmpty()) return null
  const url = image.resize({ height: THUMBNAIL_HEIGHT, quality: 'good' }).toDataURL()
  this.thumbnails.set(key, url)
  if (this.thumbnails.size > MAX_CACHED_THUMBNAILS) {
    this.thumbnails.delete(this.thumbnails.keys().next().value!)
  }
  return url
}
```

Makes a small preview of a screenshot for the page to show, on the chips in the text box and on sent messages. The page asks through `ipc.ts` (`screenshotThumbnail`), from its `useThumbnail` hook.

`nativeImage` is Electron's image class. It can load a PNG or JPEG file, report its size, resize it, and encode it as PNG, JPEG or a data URL.

- `if (!this.owns(path)) return null`: the inside-the-folder check from the overview.
- `(await fs.stat(path)).mtimeMs`: the file's last-modified time. If the file is gone, the method returns `null`, and a draft chip says "File missing" instead of showing a preview.
- `this.thumbnails.get(key)`: the cache key is the path plus that modified time. If the screenshot is edited (cropped in Paint, say), the key changes and a fresh preview is made.
- `nativeImage.createFromPath(path)`: loads and decodes the file. It doesn't throw for a file it can't read; it returns an empty image, hence the `isEmpty()` check.
- `image.resize({ height: THUMBNAIL_HEIGHT, quality: 'good' })`: when only the height is given, the width is scaled to keep the image's shape. `'good'` is the fastest of Electron's three quality settings (`good`, `better`, `best`), plenty for a tiny preview.
- `.toDataURL()`: encodes the preview as a PNG inside a `...` string. The page can put that straight into an image's `src`, so it never needs access to the file itself.
- `this.thumbnails.keys().next().value!`: once the cache holds more than 50 previews, the oldest is removed. A `Map` remembers the order entries were added, so its first key is the oldest (first in, first out: reading an entry doesn't move it to the back). The `!` tells TypeScript the value can't be `undefined` here, because the map isn't empty.

`ScreenshotService.forClaude()`

src/main/screenshots/ScreenshotService.ts, lines 63–76

```
/**
 * A screenshot prepared for Claude: scaled down to a size the model reads well and re-encoded
 * as JPEG, which keeps requests small. Null if the file is missing or not one of ours.
 */
forClaude(path: string): OutgoingImage | null {
  if (!this.owns(path)) return null
  const image = nativeImage.createFromPath(path)
  if (image.isEmpty()) return null
  const original = image.getSize()
  const size = fitWithin(original)
  const scaled =
    size.width === original.width ? image : image.resize({ ...size, quality: 'best' })
  return { mediaType: 'image/jpeg', data: scaled.toJPEG(JPEG_QUALITY).toString('base64') }
}
```

Prepares a screenshot to send to Claude. The `chatSend` handler in `ipc.ts` calls it for each attached screenshot; if any returns `null`, nothing is sent and the page is told `missing-screenshot`. Unlike most methods here it isn't async, because `nativeImage` does its work straight away; the main process waits while each image is decoded and re-encoded.

The image is **resized and re-encoded as JPEG**. A full-screen PNG can be several megabytes, and pixels beyond what Claude uses only make the request bigger and slower (see `imageSize.ts`).

- `image.getSize()`: the image's width and height in pixels.
- `fitWithin(original)`: from `imageSize.ts`. The size to scale to, so the longest side is at most 1,568 pixels.
- `size.width === original.width ? image :`: skips the resize when the image is already small enough. Comparing the width is enough for screen-shaped images, where shrinking always changes both sides.
- `quality: 'best'`: the slowest, sharpest resize, because Claude needs to read small text in the screenshot.
- `scaled.toJPEG(JPEG_QUALITY)`: encodes as JPEG at quality 85. JPEG throws away fine detail the eye barely notices, which keeps the request small.

- `.toString('base64')`: turns the bytes into text. The Claude API takes images as base64 text inside the JSON request; `ChatSession.send()` puts this into an `image` block with the `mediaType` given here.

`ScreenshotService.forClipboard()`

src/main/screenshots/ScreenshotService.ts, lines 78–103

```
/**
 * The screenshots as one image for the clipboard (a link to Claude Desktop can't carry images).
 * One screenshot is left as it is; several are scaled to the same width and stacked top to
 * bottom. Null if a file is missing or not one of ours.
 */
forClipboard(paths: string[]): NativeImage | null {
  const images: NativeImage[] = []
  for (const path of paths) {
    const image = this.owns(path) ? nativeImage.createFromPath(path) : null
    if (!image || image.isEmpty()) return null
    images.push(image)
  }
  if (images.length <= 1) return images[0] ?? null
  const width = stackWidth(images.map((image) => image.getSize()))
  const stacked = stackBitmaps(
    images.map((image) => {
      const scaled =
        image.getSize().width === width ? image : image.resize({ width, quality: 'best' })
      return { ...scaled.getSize(), data: scaled.toBitmap() }
    }),
  )
  return nativeImage.createFromBitmap(stacked.data, {
    width: stacked.width,
    height: stacked.height,
  })
}
```

Prepares the attached screenshots for Claude Desktop. A question goes to Claude Desktop as a link, and a link can carry text but not images, so the screenshots go on the Windows clipboard to be pasted into Claude with Ctrl+V, by Desktop Assist or by the user (see Claude Desktop). An IT request raised from the panel goes to the service desk portal as a link too, so its screenshots go the same way, to be pasted into the portal's form (see Jira and the service desk). The clipboard holds only one image at a time, so several screenshots are combined into one, stacked top to bottom. `copyScreenshots` in `index.ts` calls this and puts the result on the clipboard as a PNG.

Unlike `forClaude()`, nothing is shrunk to Claude's preferred size or turned into a JPEG: the screenshots keep their full size (apart from the scaling needed to stack them), and `index.ts` copies them as a lossless PNG.

- `images: NativeImage[]`: each file is loaded first. If any one is missing, not inside the screenshots folder, or can't be read, the whole method returns `null`, and the question isn't sent with some of its screenshots silently left out.
- `this.owns(path) ? nativeImage.createFromPath(path) : null`: the inside-the-folder check, then loading the file, as in `thumbnail()`.
- `if (images.length <= 1) return images[0] ?? null`: one screenshot is used exactly as it is. With an empty list `images[0]` is `undefined`, so `?? null` gives `null` (the caller only asks when there's at least one).
- `stackWidth(images.map((image) => image.getSize()))`: the width to stack at, from `stack.ts` below: the narrowest screenshot's width, at most 2,560 pixels.
- `image.getSize().width === width ? image : image.resize({ width, quality: 'best' })`: scales each screenshot to that width (the height follows, keeping its shape). One that is already that wide is left alone.

`'best'` keeps small text readable.

- `{ ...scaled.getSize(), data: scaled.toBitmap() }`: the image as raw pixels (a `Bitmap`), which is what `stackBitmaps()` works on. `toBitmap()` gives 4 bytes per pixel, in Electron's BGRA order (blue, green, red, alpha).
- `nativeImage.createFromBitmap(stacked.data, { width, height })`: turns the stacked pixels back into an image, which `index.ts` encodes as PNG.

`ScreenshotService.open()` and `ScreenshotService.openFolder()`

src/main/screenshots/ScreenshotService.ts, lines 105–119

```
/** Opens a screenshot in the default image viewer. Returns false if it no longer exists. */
async open(path: string): Promise<boolean> {
  if (!this.owns(path)) return false
  try {
    await fs.access(path)
  } catch {
    return false
  }
  return (await shell.openPath(path)) === ''
}

async openFolder(): Promise<void> {
  await fs.mkdir(this.dir, { recursive: true })
  await shell.openPath(this.dir)
}
```

`open()` shows a screenshot in Windows' default image viewer, when a chip or a sent screenshot is clicked. `openFolder()` opens the screenshots folder in File Explorer, from the settings menu.

- `fs.access(path)`: checks the file exists, and throws if it doesn't. Returning `false` lets the page show "That screenshot can't be found".
- `shell.openPath(path)`: Electron's "open this file with its default app", like double-clicking it. It resolves to an empty string on success and to an error message otherwise, hence the `=== ''`.
- `fs.mkdir(this.dir, { recursive: true })`: creates the folder if it doesn't exist yet (before the first screenshot), so there's always something to open. `recursive: true` also creates missing parent folders and doesn't fail if the folder already exists.

`openFolder()` takes no path from the page, so it needs no `owns()` check.

`ScreenshotService.owns()`

src/main/screenshots/ScreenshotService.ts, lines 121–124

```
/** Renderers may only ask about files inside the screenshots folder. */
private owns(path: string): boolean {
  return isInsideDir(this.dir, path)
}
```

The inside-the-folder check used by `thumbnail()`, `forClaude()`, `forClipboard()` and `open()`. It's a one-line wrapper around `isInsideDir()` in `files.ts`, which explains how the check works.

`src/main/screenshots/files.ts`: naming, saving and finding screenshot files

Plain file helpers with no Electron in them, so they can be tested with ordinary files (`tests/screenshotFiles.test.ts`). `ScreenshotService` uses them to name and save captures, to find the newest one, and to check that a path is inside the screenshots folder.

`screenshotFileName` and `isScreenshotFileName`

src/main/screenshots/files.ts, lines 5–19

```ts
const SCREENSHOT_NAME = /^Screenshot \d{4}-\d{2}-\d{2} \d{6}(?: \(\d+\))?\.png$/

const pad = (n: number) => String(n).padStart(2, '0')

/** "Screenshot 2026-10-02 140311.png", or "… (2).png" for a second capture in the same second. */
export function screenshotFileName(date: Date, copy = 1): string {
  const stamp =
    `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())} ` +
    `${pad(date.getHours())}${pad(date.getMinutes())}${pad(date.getSeconds())}`
  return copy > 1 ? `Screenshot ${stamp} (${copy}).png` : `Screenshot ${stamp}.png`
}

export function isScreenshotFileName(name: string): boolean {
  return SCREENSHOT_NAME.test(name)
}
```

`screenshotFileName()` builds a name such as `Screenshot 2026-10-02 140311.png` from a date, in local time. The time is written `140311` rather than `14:03:11` because Windows doesn't allow `:` in file names. `isScreenshotFileName()` checks whether a name has exactly this pattern.

- `SCREENSHOT_NAME`: a regular expression for our names. `\d{4}` means four digits, `(?: \(\d+\))?` an optional copy number such as `(2)` after a space, and `^` and `$` pin it to the whole name.
- `pad`: turns `7` into `07`, so every date part has a fixed width and the names sort in time order.
- `date.getMonth() + 1`: JavaScript numbers months from 0.
- `copy > 1 ?`: the second capture in the same second becomes `... (2).png`, and so on.

`saveScreenshot`

src/main/screenshots/files.ts, lines 21–34

```ts
/** Saves a PNG into `dir` without ever overwriting an existing file. Returns the new path. */
export async function saveScreenshot(dir: string, png: Buffer, date: Date): Promise<string> {
  await fs.mkdir(dir, { recursive: true })
  for (let copy = 1; copy <= 100; copy++) {
    const path = join(dir, screenshotFileName(date, copy))
    try {
      await fs.writeFile(path, png, { flag: 'wx' })
      return path
    } catch (err) {
      if (!isErrno(err, 'EEXIST')) throw err
    }
  }
  throw new Error('Too many screenshots with the same timestamp')
}
```

Writes the PNG into the folder under a new name, never replacing an existing file. This isn't an atomic write (see `jsonFile.ts`): it never replaces a file, so at worst a crash leaves one incomplete new image.

- `fs.mkdir(dir, { recursive: true })`: creates the folder on the first capture.
- `{ flag: 'wx' }`: "write, but fail if the file already exists". The check and the creation happen in one step in the operating system, so two captures can't both claim the same name.
- `isErrno(err, 'EEXIST')`: the name is taken, so the loop tries `(2)`, `(3)` and so on. Any other error (disk full, no permission) is thrown at once.
- `copy <= 100`: a limit, so the loop can't run forever.

`findLatestScreenshot`

src/main/screenshots/files.ts, lines 36–63

```ts
/**
 * The most recently written screenshot in `dir`, ignoring files we didn't name and any that
 * `skip` says to pass over (those already attached, so each click goes one further back).
 */
export async function findLatestScreenshot(
  dir: string,
  skip: (path: string) => boolean = () => false,
): Promise<string | null> {
  let names: string[]
  try {
    names = await fs.readdir(dir)
  } catch (err) {
    if (isErrno(err, 'ENOENT')) return null
    throw err
  }
  let latest: { path: string; mtimeMs: number } | null = null
  for (const name of names.filter(isScreenshotFileName)) {
    const path = join(dir, name)
    if (skip(path)) continue
    try {
      const { mtimeMs } = await fs.stat(path)
      if (!latest || mtimeMs > latest.mtimeMs) latest = { path, mtimeMs }
    } catch {
      // Deleted between readdir and stat.
    }
  }
  return latest?.path ?? null
}
```

Returns the path of the most recently written screenshot in the folder, or `null` if there are none (or none that aren't skipped).

- `isErrno(err, 'ENOENT')`: `ENOENT` means "no such file or directory". Before the first screenshot the folder doesn't exist, which just means there are no screenshots.
- `names.filter(isScreenshotFileName)`: only files with our names count, so other pictures dropped into the folder are never attached by accident.
- `skip = () => false`: by default nothing is passed over. `if (skip(path)) continue` leaves out the ones the caller names, so with the attached ones skipped, clicking **Attach screenshot** again and again goes back through the screenshots, newest first (a test checks this).
- `mtimeMs > latest.mtimeMs`: picks by the time each file was last written, not by its name, so the file written last wins even if its name says otherwise (a test checks this).

- `// Deleted between readdir and stat.`: a file can disappear between listing the folder and looking at it; it's simply skipped.
- `latest?.path ?? null`: `?.` reads `path` only if `latest` isn't `null`.

`isInsideDir`

src/main/screenshots/files.ts, lines 65–69

```ts
/** True if `file` is inside `dir` (not `dir` itself). Case-insensitive on Windows. */
export function isInsideDir(dir: string, file: string): boolean {
  const rel = relative(resolve(dir), resolve(file))
  return rel !== '' && rel.split(/[\\/]/)[0] !== '..' && !isAbsolute(rel)
}
```

The heart of the inside-the-folder check: is `file` really inside `dir` (or one of its subfolders), and not `dir` itself?

A simple test such as "does the file's path start with the folder's path?" isn't safe. `C:\Users\me\Pictures\Desktop Assist\..\..\Documents\taxes.pdf` starts with the folder's path but points outside it, and `C:\Users\me\Pictures\Desktop Assist Copy\x.png` starts with the same text but is a different folder. This function compares the real path structure instead. It works on the path text alone and doesn't look at the disk.

- `relative(resolve(dir), resolve(file))`: `resolve` turns both into full, normalised paths, so any `..` parts are worked out first. `relative` then gives the route from the folder to the file. On Windows it ignores case, so `C:\USERS\...` still matches.
- `rel !== ''`: an empty route means `file` is the folder itself.
- `rel.split(/[\\/]/)[0] !== '..'`: a route that starts by going up a level leads outside the folder. Only the first part of the route is compared, so a file genuinely named `..notes.png` inside the folder is still allowed.
- `!isAbsolute(rel)`: on Windows a file on another drive has no relative route, so `relative` returns its full path (such as `D:\elsewhere\shot.png`). An absolute result means "not inside".

`src/main/screenshots/imageSize.ts`: sizing images for Claude

One small calculation, kept apart from `ScreenshotService` so it can be tested without Electron (in `tests/claudeHelpers.test.ts`). `ScreenshotService.forClaude()` uses it to choose the size to scale a screenshot to.

src/main/screenshots/imageSize.ts, lines 1–15

```ts
import type { Size } from '@shared/geometry'

/** Screenshots sent to Claude are scaled so their longest side is at most this many pixels. */
export const MAX_IMAGE_EDGE = 1568

/** Scales `size` down (never up) so its longest side fits within `maxEdge`, keeping the shape. */
export function fitWithin(size: Size, maxEdge: number = MAX_IMAGE_EDGE): Size {
  const longest = Math.max(size.width, size.height)
  if (longest <= maxEdge) return { ...size }
  const scale = maxEdge / longest
  return {
    width: Math.max(1, Math.round(size.width * scale)),
    height: Math.max(1, Math.round(size.height * scale)),
  }
}
```

- `MAX_IMAGE_EDGE = 1568`: the long-edge limit in Anthropic's guidance for images. Bigger images are scaled down by the API before Claude looks at them, so sending more pixels only makes requests bigger and slower.
- `size: Size`: `Size` is `{ width, height }`, from `src/shared/geometry.ts`.
- `maxEdge: number = MAX_IMAGE_EDGE`: a default parameter. The app always uses the default; the tests pass a value explicitly.
- `return { ...size }`: already small enough, so the size stays the same (returned as a copy). It never scales up.
- `const scale = maxEdge / longest`: one factor for both sides keeps the shape. For example, 1920×1080 becomes 1568×882.
- `Math.max(1, Math.round(size.width * scale))`: rounding could make a very thin image's short side 0 pixels; this keeps every side at least 1.

`src/main/screenshots/stack.ts`: stacking screenshots into one image

The pixel arithmetic behind `ScreenshotService.forClipboard()`: putting several screenshots one above the other in a single image, with a grey band between them. It works on raw pixels and has no Electron in it, so `tests/screenshotStack.test.ts` can check it with tiny made-up images.

`Bitmap`, `MAX_STACK_WIDTH`, `STACK_GAP` and `GAP_PIXEL`

src/main/screenshots/stack.ts, lines 3–14

```ts
/** Raw pixels, 4 bytes each (Electron's BGRA bitmaps), row after row with no padding. */
export interface Bitmap extends Size {
  data: Buffer
}

/** Stacked screenshots are never wider than this, to keep the combined image a sensible size. */
export const MAX_STACK_WIDTH = 2560

/** Height of the grey band between stacked screenshots, in pixels. */
export const STACK_GAP = 12

const GAP_PIXEL = Buffer.from([0x99, 0x99, 0x99, 0xff])
```

- `interface Bitmap extends Size`: a `Size` (`width` and `height`, from `src/shared/geometry.ts`) plus `data`, the pixels. `extends` adds fields to an existing type.
- `data: Buffer`: as the comment says, 4 bytes per pixel, row after row: the top row's pixels first, left to right, then the next row, and so on. So the pixel at column x, row y starts at byte `(y * width + x) * 4`. Some image formats add padding at the end of each row; `stackBitmaps()` checks there is none.
- `MAX_STACK_WIDTH = 2560`: the widest a stacked image gets (see `stackWidth()`).
- `STACK_GAP = 12`: the grey band, so it's clear where one screenshot ends and the next begins.
- `Buffer.from([0x99, 0x99, 0x99, 0xff])`: one pixel. Blue, green and red are all `0x99` (153 out of 255, a mid-grey), and alpha is `0xff`, fully opaque. With the three colours equal, the byte order doesn't matter.

`stackWidth()`

src/main/screenshots/stack.ts, lines 16–19

```ts
/** The width screenshots are scaled to before stacking: the narrowest one's, so none is enlarged. */
export function stackWidth(sizes: Size[]): number {
  return Math.min(MAX_STACK_WIDTH, ...sizes.map((size) => size.width))
}
```

Every screenshot in a stack must be the same width, so they're all scaled to this one first.

- `...sizes.map((size) => size.width)`: `...` spreads the list of widths into separate arguments for `Math.min()`.
- `Math.min(MAX_STACK_WIDTH, ...)`: the narrowest screenshot's width, so screenshots are only ever shrunk to match, never enlarged (which would blur them). The cap stops two screenshots of a 4K screen (3,840 pixels wide) from making a needlessly huge image.

`stackBitmaps()`

src/main/screenshots/stack.ts, lines 21–35

```ts
/** One image of `bitmaps` from top to bottom, with a grey band between each. All must be as wide. */
export function stackBitmaps(bitmaps: Bitmap[]): Bitmap {
  const width = bitmaps[0]?.width
  if (width === undefined) throw new Error('Nothing to stack')
  for (const bitmap of bitmaps) {
    if (bitmap.width !== width) throw new Error('Stacked images must be the same width')
    if (bitmap.data.length !== bitmap.width * bitmap.height * 4) {
      throw new Error('Unexpected bitmap layout')
    }
  }
  const gap = Buffer.alloc(width * STACK_GAP * 4, GAP_PIXEL)
  const parts = bitmaps.flatMap((bitmap, i) => (i === 0 ? [bitmap.data] : [gap, bitmap.data]))
  const height = bitmaps.reduce((sum, b) => sum + b.height, 0) + STACK_GAP * (bitmaps.length - 1)
  return { width, height, data: Buffer.concat(parts) }
}
```

Joins the bitmaps into one, top to bottom. Because each bitmap is stored row after row, and they are all the same width, stacking is just putting one buffer after another: the first image's rows, then the gap's rows, then the second image's rows. No pixel has to be moved one at a time.

- `bitmaps[0]?.width`, `throw new Error('Nothing to stack')`: an empty list is a mistake in the calling code. (`forClipboard()` only stacks two or more.)
- `if (bitmap.width !== width) throw`, `bitmap.data.length !== bitmap.width * bitmap.height * 4`: checks the two things the joining relies on. A wrong width or padded rows would give a garbled image, so it fails loudly instead.
- `Buffer.alloc(width * STACK_GAP * 4, GAP_PIXEL)`: the grey band, `STACK_GAP` rows of `width` pixels. `Buffer.alloc(size, fill)` repeats the 4-byte `GAP_PIXEL` to fill it.
- `bitmaps.flatMap((bitmap, i) => (i === 0 ? [bitmap.data] : [gap, bitmap.data]))`: the pieces in order: the first image, then a gap before each of the others. `flatMap` turns each item into a list and joins the lists into one.
- `bitmaps.reduce((sum, b) => sum + b.height, 0) + STACK_GAP * (bitmaps.length - 1)`: the total height, the images' heights added up plus one gap fewer than there are images.
- `Buffer.concat(parts)`: copies the pieces into one buffer, the stacked image.

`src/main/storage/jsonFile.ts`: safe files

The low-level helpers for files the app saves: reading a JSON file and checking it with a schema, and writing files atomically. `NotesStore` (the draft), `SettingsService` in `settings.ts` (`preferences.json`) and `SecretStore` (the encrypted sign-in, through `writeFileAtomic`) all use them, and `isErrno` is used across the main process.

Key ideas:

- **Reading never trusts the file.** The text must parse as JSON and match a zod schema, and the caller gets a clear result: `ok`, `missing` or `invalid`.
- **Atomic writes.** Never write over the real file directly. Write the whole content to a temporary file in the same folder, then rename it over the real one. A rename within one drive replaces the file in a single step, so anyone reading the file, or the app after a crash, sees either the complete old version or the complete new one.

`ReadResult` and `readJsonFile`

src/main/storage/jsonFile.ts, lines 5–21

```ts
export type ReadResult<T> =
  { status: 'ok'; value: T } | { status: 'missing' } | { status: 'invalid'; error: unknown }

export async function readJsonFile<T>(path: string, schema: ZodType<T>): Promise<ReadResult<T>> {
  let raw: string
  try {
    raw = await fs.readFile(path, 'utf8')
  } catch (err) {
    if (isErrno(err, 'ENOENT')) return { status: 'missing' }
    throw err
  }
  try {
    return { status: 'ok', value: schema.parse(JSON.parse(raw)) }
  } catch (error) {
    return { status: 'invalid', error }
  }
}
```

`readJsonFile()` reads a JSON file and checks it against a zod schema. `ReadResult` is what it returns.

- `{ status: 'ok'; value: T }`: one of three possible shapes, told apart by `status` (a "discriminated union"). Once the caller has checked `result.status === 'ok'`, TypeScript knows `result.value` exists.
- `ZodType<T>`: `T` is a generic type parameter: whatever type the schema describes. For `NotesFileSchema`, the value comes back typed as the notes file's shape, with no casts needed.
- `fs.readFile(path, 'utf8')`: reads the whole file as text.
- `isErrno(err, 'ENOENT')`: no file is normal (the first run), so it's `missing`, not an error. Any other error, such as no permission, is thrown.
- `schema.parse(JSON.parse(raw))`: `JSON.parse` throws on text that isn't JSON, and `schema.parse` throws if the JSON has the wrong shape. Both end up as `invalid`, with the error attached. Callers decide what that means: `NotesStore.load()` sets the file aside, and `SettingsService.init()` falls back to default preferences.

tmpPath

src/main/storage/jsonFile.ts, lines 23–24

```ts
let tmpCounter = 0

const tmpPath = (path: string) => `${path}.${process.pid}.${++tmpCounter}.tmp`
```

Makes a unique temporary file name next to the target, such as `notes.json.12345.3.tmp`.

- `process.pid`: the process ID, so two running copies of the app can't pick the same name.
- `++tmpCounter`: a counter, so two writes in progress at once in this process can't either. That can happen: `SettingsService` doesn't queue its saves the way `NotesStore` does.

The temporary file is in the same folder as the target on purpose: a rename only swaps the file in one step when both names are on the same drive.

writeJsonFile

src/main/storage/jsonFile.ts, lines 26–37

```ts
/**
 * Writes via a temp file and a rename, so a crash mid-write can't leave a half-written file.
 * `shouldCommit` runs just before the rename; returning false discards the write (used to drop
 * a write that a newer one has already overtaken). Resolves to whether the file was replaced.
 */
export function writeJsonFile(
  path: string,
  value: unknown,
  shouldCommit: () => boolean = () => true,
): Promise<boolean> {
  return writeFileAtomic(path, JSON.stringify(value, null, 2), shouldCommit)
}
```

Saves a value as JSON, atomically. It's a thin wrapper around `writeFileAtomic()`, and resolves to `true` if the file was replaced or `false` if `shouldCommit` said no.

- `JSON.stringify(value, null, 2)`: turns the value into JSON text, indented by 2 spaces so a person can read the file.
- `shouldCommit: () => boolean = () => true`: an optional last-moment check, passed on to `writeFileAtomic()`. By default every write is kept; `NotesStore.write()` passes one that drops overtaken writes.

writeFileAtomic

src/main/storage/jsonFile.ts, lines 39–54

```ts
/** The same temp-file-and-rename write for any content (text or binary). */
export async function writeFileAtomic(
  path: string,
  data: string | Buffer,
  shouldCommit: () => boolean = () => true,
): Promise<boolean> {
  await fs.mkdir(dirname(path), { recursive: true })
  const tmp = tmpPath(path)
  await fs.writeFile(tmp, data)
  if (!shouldCommit()) {
    await fs.rm(tmp, { force: true })
    return false
  }
  await renameWithRetry(tmp, path)
  return true
}
```

The atomic write itself, for any content: JSON text or binary data (`SecretStore.save()` writes encrypted bytes with it).

- `fs.mkdir(dirname(path), { recursive: true })`: makes sure the folder exists first.
- `await fs.writeFile(tmp, data)`: the slow part, writing all the bytes, happens on the temporary file. The real file is untouched meanwhile.
- `if (!shouldCommit())`: asked after the temporary file is written and just before the rename, so it sees the latest state. If the answer is no, the temporary file is deleted and the real file stays as it was.
- `fs.rm(tmp, { force: true })`: `force` means "don't fail if it's already gone".
- `renameWithRetry(tmp, path)`: moves the new file into place. Node's `fs.rename` replaces the target if it exists.

If writing or renaming fails, the error goes to the caller and the real file is unchanged, though the temporary file may be left behind.

writeJsonFileSync

src/main/storage/jsonFile.ts, lines 56–61

```ts
export function writeJsonFileSync(path: string, value: unknown): void {
  mkdirSync(dirname(path), { recursive: true })
  const tmp = tmpPath(path)
  writeFileSync(tmp, JSON.stringify(value, null, 2), 'utf8')
  renameSync(tmp, path)
}
```

The same temp-file-and-rename write, but blocking: each step finishes before the next line runs, and the function returns only when the file is in place. `NotesStore.flushSync()` uses it when Windows is ending the session. It has no `shouldCommit` check and doesn't retry a failed rename.

- `mkdirSync`, `writeFileSync` and `renameSync`: Node's blocking versions of `fs.mkdir`, `fs.writeFile` and `fs.rename`.

renameWithRetry

src/main/storage/jsonFile.ts, lines 63–75

```
// On Windows, antivirus and the search indexer can hold the target open for a moment.
async function renameWithRetry(from: string, to: string, attempts = 5): Promise<void> {
  for (let attempt = 1; ; attempt++) {
    try {
      await fs.rename(from, to)
      return
    } catch (err) {
      const transient = ['EPERM', 'EBUSY', 'EACCES'].some((code) => isErrno(err, code))
      if (!transient || attempt >= attempts) throw err
      await new Promise((resolve) => setTimeout(resolve, 25 * attempt))
    }
  }
}
```

Renames a file, trying again a few times if Windows briefly refuses. As the comment above it says, antivirus software and the search indexer can hold a file open for a moment, and a rename onto it then fails.

- `['EPERM', 'EBUSY', 'EACCES']`: the error codes Windows gives in that case (operation not permitted, busy, access denied). Any other error is thrown at once.
- `for (let attempt = 1; ; attempt++)`: a loop with no end condition. It leaves by `return` (success) or `throw` (giving up after the fifth attempt).
- `new Promise((resolve) => setTimeout(resolve, 25 * attempt))`: the usual way to `await` a pause in JavaScript. The waits grow: 25, 50, 75 and then 100 ms between the five attempts, at most a quarter of a second in all.

isErrno

src/main/storage/jsonFile.ts, lines 77–79

```
export function isErrno(err: unknown, code: string): boolean {
  return typeof err === 'object' && err !== null && (err as NodeJS.ErrnoException).code === code
}
```

Checks whether an error is a Node file-system error with a given code, such as `ENOENT` (not found) or `EEXIST` (already exists). It's used by `readJsonFile()` and `renameWithRetry()` here, by `saveScreenshot()` and `findLatestScreenshot()` in `files.ts`, and by `SecretStore.load()`.

- `err: unknown`: with the project's `strict` TypeScript settings, a caught error has the type `unknown`, because any value can be thrown. It has to be checked before reading `.code`.
- `typeof err === 'object' && err !== null`: `typeof null` is also `'object'`, hence the second test.
- `(err as NodeJS.ErrnoException).code`: once it's known to be an object, it's treated as Node's error type so its `code` can be read. If there's no `code`, the comparison is simply false.

The pages: entry points, styles, hooks and views

This part covers the renderer's own files: the HTML page both windows load, the startup script, the stylesheet, two small helpers, the hooks that connect React to the main process, and the two views (the bubble and the panel). The building blocks the views use are covered in the components part. If React is new to you: a React *component* is a function that returns a description of what to show, written in JSX (HTML-like tags inside TypeScript, in `.tsx` files), and whenever the data it uses changes, React calls it again and updates only the parts of the page that changed.

`src/renderer/index.html`: the page both windows load

The bubble window and the panel window both load this one page. It's almost empty: a `<div>` for React to fill and a script tag that starts the app. The interesting part is the security policy.

`src/renderer/index.html`, lines 1–15

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      http-equiv="Content-Security-Policy"
      content="default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
data:; connect-src 'self'; object-src 'none'; base-uri 'none'; form-action 'none'"
    />
    <title>Desktop Assist</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="./src/main.tsx"></script>
  </body>
</html>
```

- `http-equiv="Content-Security-Policy"`: a Content-Security-Policy (CSP) is a set of rules the browser enforces about what the page may load and run. Even if something managed to slip markup into the page (say, a crafted reply from Claude), the browser would refuse to run scripts from anywhere but the app itself. It's a second lock behind the sandbox and context isolation set in `createOverlayWindow` (src/main/bubble/windows.ts).
- `default-src 'self'`: anything without its own rule below may only come from the app's own origin: the built files, or the Vite dev server during `npm run dev`.
- `script-src 'self'`: scripts only from the app's own files. Inline `<script>` blocks and `eval` are blocked.
- `style-src 'self' 'unsafe-inline'`: the app's stylesheets plus inline styles. During `npm run dev`, Vite injects the CSS as `<style>` tags, which needs `'unsafe-inline'`.
- `img-src 'self' data:`: images from the app (the tenant's logo) or `data:` URLs, which is how screenshot thumbnails arrive (see `useThumbnail` below).
- `connect-src 'self'`: the page can't `fetch` anything from the internet. Only the main process talks to Claude and JumpCloud.
- `object-src 'none'; base-uri 'none'; form-action 'none'`: no plugins (`<object>`, `<embed>`), no `<base>` tag that could redirect relative links, and no form submissions anywhere.
- `<div id="root"></div>`: the empty element React takes over (see `main.tsx`).

- `<script type="module" src="./src/main.tsx"></script>`: the entry point. Vite compiles `main.tsx` and everything it imports into plain JavaScript and, in the build, rewrites this tag to point at the bundled file.

`src/renderer/src/main.tsx`: choosing bubble or panel

The first code that runs in each window. It works out which window it's in and renders the matching view. `BubbleView` and `PanelView` are React components: functions whose names start with a capital letter and return JSX. In JSX, `<BubbleView />` means "render the `BubbleView` component here".

src/renderer/src/main.tsx, lines 1–12

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles.css'
import { BubbleView } from './views/BubbleView'
import { PanelView } from './views/PanelView'

// Both overlay windows load this page; the query string says which one this is.
const view = new URLSearchParams(window.location.search).get('view')

createRoot(document.getElementById('root')!).render(
  <StrictMode>{view === 'bubble' ? <BubbleView /> : <PanelView />}</StrictMode>,
)
```

- `import './styles.css'`: importing a CSS file from code is a Vite feature; it adds the stylesheet (after Tailwind has processed it) to the page.
- `new URLSearchParams(window.location.search).get('view')`: reads the `?view=bubble` or `?view=panel` that `createOverlayWindow` (src/main/bubble/windows.ts) adds when it loads the page. Anything other than `bubble` gets the panel.
- `document.getElementById('root')!`: the `!` tells TypeScript "this is not null"; the div is in `index.html`.
- `createRoot(`: hands that div to React, which from now on owns everything inside it. `render` draws the first screen.
- `<StrictMode>`: a development-only checker. In `npm run dev` it calls each component twice and runs each effect, cleans it up, then runs it again, to expose code that doesn't clean up after itself. That's why every hook below has a careful cleanup. It does nothing in the installed app.
- `{view === 'bubble' ? <BubbleView /> : <PanelView />}`: curly braces switch from JSX back to ordinary TypeScript, here a choice between the two views.

`src/renderer/src/env.d.ts`: telling TypeScript about `window.assist`

The preload script puts an `assist` object on `window`, but TypeScript only knows the browser's own `Window` type. This file adds the field, so `window.assist.getState()` and the rest are type-checked in every renderer file. It contains types only and produces no JavaScript.

src/renderer/src/env.d.ts, lines 1–10

```ts
import type { AssistApi } from '@shared/ipc'

declare global {
  interface Window {
    /** Exposed by the preload script (src/preload/index.ts). */
    assist: AssistApi
  }
}

export {}
```

- `declare global`: adds to the global types instead of declaring something new in this file. `interface Window` merges with the browser's own `Window` interface.
- `assist: AssistApi`: the type is defined in src/shared/ipc.ts. The object itself is created by src/preload/index.ts, which calls `contextBridge.exposeInMainWorld('assist', api)`.
- `export {}`: makes sure the file counts as a module, which `declare global` requires. The `import type` line already does that, so this is a safety net.

`src/renderer/src/styles.css`: Tailwind setup and the few hand-written styles

Tailwind in brief. Tailwind CSS provides small, single-purpose class names: `flex`, `rounded-full`, `px-3` (horizontal padding), `bg-white`, `text-xs`. Components style themselves by listing these in `className` instead of writing a separate stylesheet. A prefix applies a class only in some situation: `hover:` while the pointer is over it, `dark:` when the page is in dark mode (Desktop Assist's own setting, dark by default), `motion-reduce:` when Windows animations are turned off, and `group-data-open:` when a parent marked `group` has a `data-open` attribute (used by the panel). At build time Tailwind scans the source for the class names actually used and generates CSS for just those. This file sets Tailwind up and holds the styles that classes can't express.

src/renderer/src/styles.css, lines 1–231

```css
@import 'tailwindcss';

@theme {
  /* Overridden at runtime with the tenant's colour (see useAccentColor). Every highlight uses it. */
  --color-accent: #9333ea;
  /* A lighter accent, for accent-coloured text on dark backgrounds. */
  --color-accent-soft: color-mix(in srgb, var(--color-accent) 60%, white);
  --font-sans: 'Segoe UI Variable Text', 'Segoe UI', system-ui, sans-serif;
}

@layer base {
  html,
  body,
  #root {
    width: 100%;
    height: 100%;
    margin: 0;
    overflow: hidden;
    background: transparent;
  }

  body {
    font-family: var(--font-sans);
    user-select: none;
    -webkit-font-smoothing: antialiased;
```

```
  }

  textarea {
    user-select: text;
  }

  /* Selected text uses the accent instead of the system blue. */
  ::selection {
    background-color: color-mix(in srgb, var(--color-accent) 25%, transparent);
  }

  /* Keyboard focus too; Chromium's default outline follows the Windows accent colour. */
  :focus-visible {
    outline: 2px solid var(--color-accent);
    outline-offset: 2px;
  }

  /* Slim scrollbars suit the small panel. */
  * {
    scrollbar-width: thin;
    scrollbar-color: rgb(0 0 0 / 0.2) transparent;
  }
  @media (prefers-color-scheme: dark) {
    * {
      scrollbar-color: rgb(255 255 255 / 0.2) transparent;
    }
  }
}

/* Claude's replies (rendered by components/Markdown.tsx). Sized for a narrow chat panel. */
@layer components {
  .markdown {
    user-select: text;
    line-height: 1.55;
    overflow-wrap: anywhere;
  }
  .markdown > :first-child {
    margin-top: 0;
  }
  .markdown > :last-child {
    margin-bottom: 0;
  }
  .markdown :where(p, ul, ol, pre, table, blockquote) {
    margin: 0.5em 0;
  }
  .markdown :where(h1, h2, h3, h4) {
    margin: 0.8em 0 0.3em;
    font-weight: 600;
    line-height: 1.3;
  }
  .markdown h1 {
    font-size: 1.15em;
  }
  .markdown h2 {
    font-size: 1.08em;
  }
  .markdown :where(h3, h4) {
    font-size: 1em;
  }
  .markdown ul {
    list-style: disc;
    padding-left: 1.25em;
  }
  .markdown ol {
    list-style: decimal;
```

```
  padding-left: 1.4em;
}
.markdown li + li {
  margin-top: 0.15em;
}
.markdown a {
  color: var(--color-accent);
  text-decoration: underline;
  cursor: pointer;
}
.markdown blockquote {
  border-left: 3px solid var(--color-zinc-300);
  padding-left: 0.75em;
  color: var(--color-zinc-500);
}
.markdown :not(pre) > code {
  border-radius: 4px;
  background: var(--color-zinc-100);
  padding: 0.1em 0.3em;
  font-size: 0.9em;
}
.markdown pre {
  overflow-x: auto;
  border-radius: 8px;
  background: var(--color-zinc-100);
  padding: 0.6em 0.75em;
  font-size: 0.85em;
  line-height: 1.45;
}
.markdown code {
  font-family: 'Cascadia Code', Consolas, monospace;
}
.markdown table {
  display: block;
  overflow-x: auto;
  border-collapse: collapse;
  font-size: 0.9em;
}
.markdown :where(th, td) {
  border: 1px solid var(--color-zinc-200);
  padding: 0.25em 0.5em;
  text-align: left;
}
.markdown hr {
  margin: 0.8em 0;
  border-color: var(--color-zinc-200);
}

@media (prefers-color-scheme: dark) {
  .markdown a {
    color: var(--color-accent-soft);
  }
  .markdown blockquote {
    border-color: var(--color-zinc-600);
    color: var(--color-zinc-400);
  }
  .markdown :not(pre) > code,
  .markdown pre {
    background: var(--color-zinc-800);
  }
  .markdown :where(th, td),
  .markdown hr {
    border-color: var(--color-zinc-700);
  }
}
```

```
/* Code highlighting (highlight.js classes), light then dark. Purple in place of the usual
   blues, with names in orange so every kind of token still stands apart. */
.hljs-comment,
.hljs-quote {
  color: #6e7781;
  font-style: italic;
}
.hljs-keyword,
.hljs-selector-tag,
.hljs-built_in,
.hljs-literal {
  color: #cf222e;
}
.hljs-string,
.hljs-regexp,
.hljs-addition {
  color: #6b21a8;
}
.hljs-number,
.hljs-attr,
.hljs-attribute,
.hljs-variable {
  color: #9333ea;
}
.hljs-title,
.hljs-section,
.hljs-function {
  color: #953800;
}
.hljs-type,
.hljs-name,
.hljs-tag {
  color: #116329;
}
.hljs-deletion {
  color: #82071e;
}

@media (prefers-color-scheme: dark) {
  .hljs-comment,
  .hljs-quote {
    color: #8b949e;
  }
  .hljs-keyword,
  .hljs-selector-tag,
  .hljs-built_in,
  .hljs-literal {
    color: #ff7b72;
  }
  .hljs-string,
  .hljs-regexp,
  .hljs-addition {
    color: #e9d5ff;
  }
  .hljs-number,
  .hljs-attr,
  .hljs-attribute,
  .hljs-variable {
    color: #c084fc;
  }
  .hljs-title,
  .hljs-section,
  .hljs-function {
    color: #ffa657;
```

```
    }
    .hljs-type,
    .hljs-name,
    .hljs-tag {
      color: #7ee787;
    }
    .hljs-deletion {
      color: #ffa198;
    }
  }
}
```

- `@import 'tailwindcss'`: brings in Tailwind: its reset of browser default styles ("Preflight", which among other things removes list bullets, margins and heading sizes), its default theme (colours such as `zinc`, spacing, fonts) and the generated utility classes.
- `@theme`: adds to Tailwind's theme. Each `--color-*` variable becomes a colour every utility can use, so `--color-accent` gives `bg-accent`, `text-accent`, `ring-accent` and so on. Those classes read the CSS variable when the page is drawn, not at build time.
- `--color-accent: #9333ea`: the default purple. `useAccentColor` (below) overwrites this variable with the tenant's colour once the app state arrives, which recolours every accent use at once.
- `--color-accent-soft: color-mix(in srgb, var(--color-accent) 60%, white)`: 60% accent mixed with white, for accent-coloured text on dark backgrounds (`dark:text-accent-soft`). Because it refers to `--color-accent`, it follows the tenant's colour too.
- `--font-sans`: the Windows 11 system font, falling back to older Segoe UI and then any system font.
- `@layer base`: page-wide defaults. Tailwind orders its CSS in layers (theme, base, components, utilities) and a later layer wins, so a utility class on an element always beats these rules.
- `background: transparent`: both windows are transparent (`transparent: true` in `createOverlayWindow`), so the page must not paint a background, or the see-through areas would be white. `overflow: hidden` stops the page itself ever showing scrollbars.
- `user-select: none`: the UI shouldn't highlight like a web page when you click or drag around. `textarea` and `.markdown` turn selection back on, so the text box and Claude's replies can still be selected and copied.
- `::selection`: selected text gets a 25% tint of the accent instead of the Windows blue.
- `:focus-visible`: the outline drawn around whatever has keyboard focus, in the accent colour. `:focus-visible` only matches focus that came from the keyboard, so mouse clicks don't draw it.
- `scrollbar-width: thin`: slim scrollbars (the conversation scrolls), with a lighter thumb in dark mode.
- `@layer components`: the styles for Claude's replies, which `Markdown` wraps in `<div className="markdown">` (see the components part). The Markdown library produces plain `<p>`, `<ul>`, `<pre>` and similar elements that can't carry Tailwind classes, so they're styled here by element.
- `overflow-wrap: anywhere`: long words and URLs break instead of overflowing the 400-pixel card.
- `.markdown > :first-child`: with the matching `:last-child` rule, removes the gap above the first block and below the last, so a reply has no extra space at its top and bottom.
- `.markdown :where(p, ul, ol, pre, table, blockquote)`: `:where()` groups selectors without adding specificity, so these rules stay easy to override.
- `list-style: disc`: puts back the bullets (and, for `ol`, the numbers) that Preflight removes.
- `.markdown :not(pre) > code`: inline code gets a small grey pill. Code blocks (`.markdown pre`) get a grey box that scrolls sideways instead of wrapping.
- `.markdown table`: `display: block` with `overflow-x: auto` lets a wide table scroll inside the card.

- `@media (prefers-color-scheme: dark)`: the dark-mode colours for the same elements. This media query follows Desktop Assist's own light/dark setting: the main process sets Electron's `nativeTheme.themeSource`, which decides what the pages see (see `SettingsService.setTheme()` in Main process: startup, IPC and app plumbing).
- `.hljs-comment`: the start of the code-highlighting colours. `rehype-highlight` (used by `Markdown`) wraps each token of a code block in a `<span>` with a highlight.js class such as `hljs-keyword` or `hljs-string`; these rules colour them. The palette is GitHub's, light then dark, with purple in place of the usual blues.

`src/renderer/src/lib/cn.ts`: joining class names

`cn`

src/renderer/src/lib/cn.ts, lines 1–4

```ts
/** Joins class names, skipping falsy ones. */
export function cn(...classes: Array<string | false | null | undefined>): string {
  return classes.filter(Boolean).join(' ')
}
```

Builds a `className` string from several pieces, some of them conditional: `cn('rounded-full', open && 'ring-accent')` gives `'rounded-full ring-accent'` when `open` is true and `'rounded-full'` otherwise. Many React projects use the `clsx` library for this; four lines do the job here.

- `...classes: Array<string | false | null | undefined>`: a rest parameter collects any number of arguments into an array. `false` is allowed so that `condition && 'class'` can be passed directly.
- `classes.filter(Boolean)`: keeps only truthy values, dropping `false`, `null`, `undefined` and empty strings.

`src/renderer/src/lib/anchor.ts`: positioning from the bubble's corner

The panel window is placed so that one of its corners sits on the bubble's corner of the screen (`panelWindowBounds` in src/main/bubble/layout.ts), and everything inside it is positioned from that corner. These helpers turn "this far from the bubble's corner" into CSS for whichever corner the bubble is in. That's what makes the whole panel mirror itself when the bubble is dragged to another corner.

`anchored`

src/renderer/src/lib/anchor.ts, lines 4–14

```ts
/**
 * Positions an element inside the panel window measured from the bubble's corner: `x` across
 * and `y` away from the screen edge. In the bottom-right corner that's `right`/`bottom`; in the
 * top-left it's `left`/`top`, and so on, so the panel mirrors itself to open into the screen.
 */
export function anchored(corner: Corner, x: number, y: number): CSSProperties {
  return {
    [isLeftCorner(corner) ? 'left' : 'right']: x,
    [isTopCorner(corner) ? 'top' : 'bottom']: y,
  }
}
```

Returns a `style` object that places an element `x` pixels across and `y` pixels away from the screen edge, measured from the bubble's corner. `ChatBox`, `ActionStack` and `SettingsMenu` call it with the offsets in `PANEL_LAYOUT` (src/shared/geometry.ts), the same numbers the main process uses to size the window.

- `CSSProperties`: React's type for the `style` prop. Plain numbers in it mean pixels.
- `[isLeftCorner(corner) ? 'left' : 'right']: x`: a computed property name, where the key is chosen at runtime. In a left corner the element is placed `x` pixels from the left edge, otherwise from the right.
- `[isTopCorner(corner) ? 'top' : 'bottom']: y`: the same, vertically. In a bottom corner, `y` is measured up from the bottom.

`originClass`

src/renderer/src/lib/anchor.ts, lines 16–24

```ts
/** Tailwind transform-origin class for animations that should grow out of the bubble's corner. */
export function originClass(corner: Corner): string {
  return {
    'top-left': 'origin-top-left',
    'top-right': 'origin-top-right',
    'bottom-left': 'origin-bottom-left',
    'bottom-right': 'origin-bottom-right',
  }[corner]
}
```

The Tailwind `origin-*` class that sets `transform-origin` to the bubble's corner. `ChatBox` opens by scaling from 95% to 100% size; with the origin at the bubble's corner, it appears to grow out of the bubble.

- `'top-left': 'origin-top-left'`: a lookup table instead of building the name as `'origin-' + corner`. Tailwind finds classes by scanning the source text, so every class name must appear in full somewhere or its CSS is never generated.
- `}[corner]`: picks the entry for this corner. TypeScript checks the table has all four corners.

`src/renderer/src/hooks/useAssistState.ts`: the app state, kept live

Hooks in brief. Functions whose names start with `use` are *hooks*. A component calls them at the top of its body, in the same order on every render.

- **State:** `useState(initial)` gives a value that survives between renders, and a setter. Calling the setter stores a new value and makes React render the component again.
- **Effects:** `useEffect(fn, deps)` runs `fn` after React has updated the page. It's the place for work outside React, such as subscribing to IPC events or adding a `window` listener. If `fn` returns a function, React calls it to clean up before running the effect again and when the component goes away. `deps` lists the values the effect uses: it re-runs when one of them changes, and `[]` means "only when the component first appears".
- **Custom hooks:** an ordinary function that calls other hooks, so several components can share the same stateful logic. Each component that calls it gets its own separate copy of the state.

This file's hooks are used by both views. Each window is a separate page, so each keeps its own copy of the state.

useAssistState

src/renderer/src/hooks/useAssistState.ts, lines 4–38

```ts
/**
 * The app state, kept live for the parts the main process changes on its own: the mode, the
 * bubble's corner, the sign-in status and the chat. (Notes and settings only change through this
 * window's own requests, so the panel tracks their results itself.)
 *
 * Subscriptions start before the state is fetched. Events that arrive before the reply are
 * already reflected in it; events after it are applied on top.
 */
export function useAssistState(): AppState | null {
  const [state, setState] = useState<AppState | null>(null)

  useEffect(() => {
    let alive = true
    const update = (change: (s: AppState) => AppState) =>
      setState((current) => (current ? change(current) : current))
    const unsubscribe = [
      window.assist.onModeChanged((mode) => update((s) => ({ ...s, mode }))),
      window.assist.onCornerChanged((corner) => update((s) => ({ ...s, corner }))),
      window.assist.onAuthStatus((auth) => update((s) => ({ ...s, auth }))),
      window.assist.onChatMessage((message) =>
        update((s) => ({ ...s, chat: upsert(s.chat, message) })),
      ),
      window.assist.onChatReset(() => update((s) => ({ ...s, chat: [] }))),
    ]
    void window.assist.getState().then((initial) => {
      if (alive) setState(initial)
    })
    return () => {
      alive = false
      for (const off of unsubscribe) off()
    }
  }, [])

  return state
}
```

Fetches the whole app state (`AppState` in src/shared/types.ts: mode, corner, draft, settings, branding, version, sign-in status and chat) when the page loads, then applies the changes the main process broadcasts. It returns `null` until the first reply arrives.

The order matters. The listeners are attached before `getState` is called, and the main process builds the reply in one go. Messages from the main process reach a window in the order they were sent, so a change broadcast before the reply was built is already included in the reply, and any later one arrives after it. Nothing is missed and nothing is applied twice.

- `useState<AppState | null>(null)`: the `<AppState | null>` tells TypeScript what the state can hold.
- `let alive = true`: set to `false` by the cleanup, so a reply that arrives after the component has gone is ignored. (In development, `StrictMode` unmounts and remounts once, so this really happens.)
- `setState((current) => (current ? change(current) : current))`: passing a function to the setter means "compute the new state from the latest one". The effect runs only once, so the `state` variable it could see would be the `null` from the first render forever. A change that arrives before the first state is dropped (`current` is still `null`), which is safe for the reason above.
- `({ ...s, mode })`: a copy of the state with one field replaced. React notices a change by getting a new object, so state is always replaced, never edited in place.

- `window.assist.onModeChanged(`: each `on...` function listens for one main-process broadcast (sent to both windows by `broadcast` in src/main/index.ts) and returns a function that stops listening (`subscribe` in src/preload/index.ts).
- `upsert(s.chat, message)`: a streaming reply arrives many times with the same id and growing text (from `ChatSession`, at most every 50 ms), so it replaces the old copy rather than being added again.
- `window.assist.onChatReset(() => update((s) => ({ ...s, chat: [] })))`: New conversation, or Log out, cleared the chat.
- `void window.assist.getState().then(`: asks for the full state, which `getState` in src/main/index.ts assembles. `void` marks a promise the code deliberately doesn't wait for.
- `return () => {`: the cleanup: ignore a late reply and unsubscribe everything.

The draft (`notes`) and `settings` aren't kept live here: only this window's own requests change them, so `Panel` tracks their results itself.

upsert

src/renderer/src/hooks/useAssistState.ts, lines 40–46

```
function upsert(messages: ChatMessage[], message: ChatMessage): ChatMessage[] {
  const index = messages.findIndex((m) => m.id === message.id)
  if (index === -1) return [...messages, message]
  const next = messages.slice()
  next[index] = message
  return next
}
```

Replaces the message with the same `id`, or adds it at the end if it's new. `messages.slice()` copies the array first, so the old array React is holding is left unchanged.

useAccentColor

src/renderer/src/hooks/useAssistState.ts, lines 48–53

```
/** Applies the tenant's accent colour to the `accent` Tailwind colour. */
export function useAccentColor(color: string | undefined): void {
  useEffect(() => {
    if (color) document.documentElement.style.setProperty('--color-accent', color)
  }, [color])
}
```

Applies the tenant's accent colour (`accentColor` in tenant.json) to the page by overwriting the `--color-accent` variable from `styles.css`.

- `document.documentElement.style.setProperty('--color-accent', color)`: sets the variable as an inline style on `<html>`, the element Tailwind declares its theme variables on, and an inline style beats the stylesheet. `--color-accent-soft` is declared on the same element and recomputes from the new value.
- `if (color)`: while the state is loading, `color` is `undefined` and the default purple stays.
- `[color]`: the effect re-runs only if the colour changes.

`src/renderer/src/hooks/useClickThrough.ts`: the panel's half of click-through

The panel window is a large rectangle that's mostly transparent. So that clicks on its empty areas reach whatever is underneath, the main process makes the window ignore the mouse, while still passing pointer movements to the page. This hook watches those movements and asks the main process to switch mouse input on only while the pointer is over real UI, which is any element marked `data-hit` (the chat card, each action icon and the settings menu). See Code Guide section 1 for the background. Only `PanelView` uses it; the bubble window always takes the mouse.

`useClickThrough`

src/renderer/src/hooks/useClickThrough.ts, lines 3–61

```ts
/**
 * Makes the panel window's see-through areas pass clicks to whatever is underneath. The main
 * process keeps the window ignoring the mouse (while still forwarding pointer moves to it), and
 * this hook switches mouse input on only while the pointer is over an element marked `data-hit`.
 *
 * There's deliberately no "pointer left the window" handling: switching mouse input on makes
 * Windows send a stale leave message, and reacting to it left the window click-through while the
 * pointer sat on real UI. A window with input on gets a real move as soon as the pointer is over
 * an empty area again, which switches it back off.
 */
export function useClickThrough(): void {
  useEffect(() => {
    // null = unknown (after a reset): the next decision is always sent.
    let interactive: boolean | null = false
    let pointer: { x: number; y: number } | null = null

    const set = (next: boolean) => {
      if (next === interactive) return
      interactive = next
      window.assist.setInteractive(next)
    }
    const hitAt = (x: number, y: number) =>
      document.elementFromPoint(x, y)?.closest('[data-hit]') != null

    const onMove = (event: MouseEvent) => {
      pointer = { x: event.clientX, y: event.clientY }
      // While a button is held (selecting text, say) keep mouse input on, so the window doesn't
      // lose the release when the pointer briefly runs outside it.
      if (event.buttons !== 0) return
      set(hitAt(pointer.x, pointer.y))
    }
    // Content can change under a still pointer (a menu closes, the chat grows).
    const recheck = () =>
      requestAnimationFrame(() => {
        if (pointer) set(hitAt(pointer.x, pointer.y))
      })

    // The panel was just shown: the main process has made it click-through and says where the
    // pointer is. Decide once the page has rendered its open state.
    const offReset = window.assist.onClickThroughReset((point) => {
      interactive = null
      pointer = point
      requestAnimationFrame(() => {
        if (pointer) set(hitAt(pointer.x, pointer.y))
      })
    })

    // After a reload the main process may still have input switched on; resync.
```

```
    window.assist.setInteractive(false)
    window.addEventListener('mousemove', onMove)
    window.addEventListener('mouseup', recheck)
    return () => {
      offReset()
      window.removeEventListener('mousemove', onMove)
      window.removeEventListener('mouseup', recheck)
      set(false)
    }
  }, [])
}
```

- `let interactive: boolean | null = false`: what the page last told the main process. This and `pointer` are plain variables inside the effect, not React state: they change on every mouse move and must not cause re-renders. They live as long as the effect does.
- `if (next === interactive) return`: sends a message only when the answer changes, not on every mouse move.
- `window.assist.setInteractive(next)`: the main process's handler (src/main/ipc.ts) calls `setIgnoreMouseEvents(!interactive, { forward: true })` on this window. `forward: true` keeps mouse moves coming to the page even while clicks pass through, which is what lets this hook keep watching.
- `document.elementFromPoint(x, y)?.closest('[data-hit]') != null`: finds the topmost element under the pointer, then checks it and its ancestors for `data-hit`. `elementFromPoint` skips elements with `pointer-events: none`, so while the panel is closed (`Panel` adds `pointer-events-none`) nothing counts as a hit.
- `if (event.buttons !== 0) return`: while a mouse button is held (selecting text, say), input stays on, so the release isn't lost if the pointer briefly runs outside the UI.
- `const recheck = () =>`: runs after each mouse button release. The click may have changed what's under a pointer that hasn't moved (a menu closed, the chat grew), so the answer is worked out again.
- `requestAnimationFrame(`: runs the callback just before the browser draws the next frame, by which time React has applied the change the click caused.
- `window.assist.onClickThroughReset((point) => {`: each time the panel is shown, `resetClickThrough` (src/main/bubble/windows.ts) makes the window ignore the mouse again and sends where the pointer is inside the window. The new mode is broadcast just before the window is shown (`BubbleController.expand`), so waiting a frame lets the page draw its open state first.
- `interactive = null`: the main process has just switched input off, so what the page last sent may no longer be true. `null` equals neither `true` nor `false`, so the next `set` always sends.
- `window.assist.setInteractive(false)`: after a reload (the page crashed and `lockDown` in src/main/bubble/windows.ts reloaded it, or a development reload) the main process may still have input switched on for the old page.
- `set(false)`: in the cleanup, leaves the window click-through when the hook goes away.

The doc comment explains the one thing deliberately missing: a "pointer left the window" handler.

`src/renderer/src/hooks/useToast.ts`: short messages

A toast is a small message that appears briefly and goes away by itself, such as "Copied" or "Screenshot saved". `Panel` owns the toast; `ChatBox` draws it just outside the chat card.

`ToastMessage`, `TOAST_MS`

src/renderer/src/hooks/useToast.ts, lines 3–9

```ts
export interface ToastMessage {
  id: number
  text: string
  tone: 'info' | 'error'
}

const TOAST_MS = 2600
```

One toast, and how long it stays (2.6 seconds). `tone` picks its colours in `ChatBox`: a dark pill for `info`, red for `error`.

`useToast`

src/renderer/src/hooks/useToast.ts, lines 11–29

```ts
/** One short-lived message at a time; a new one replaces the old. */
export function useToast(): [
  ToastMessage | null,
  (text: string, tone?: ToastMessage['tone']) => void,
] {
  const [toast, setToast] = useState<ToastMessage | null>(null)

  useEffect(() => {
    if (!toast) return
    const timer = setTimeout(() => setToast(null), TOAST_MS)
    return () => clearTimeout(timer)
  }, [toast])

  const show = useCallback((text: string, tone: ToastMessage['tone'] = 'info') => {
    setToast({ id: Date.now(), text, tone })
  }, [])

  return [toast, show]
}
```

Returns the current toast (or `null`) and a function to show a new one, as a pair, the same shape `useState` returns. A new toast replaces the old one.

- `ToastMessage['tone']`: "the type of `ToastMessage`'s `tone` field", so `'info' | 'error'`.
- `if (!toast) return`: nothing to time when no toast is showing.
- `return () => clearTimeout(timer)`: when a new toast replaces the old one, React runs this cleanup before starting the new timer. The old timer can't clear the new toast early, and the new one gets its full 2.6 seconds.
- `useCallback(`: returns the same function on every render, since its dependency list `[]` never changes. Without it, `show` would be a new function each time, which matters when it's passed to other components or listed in an effect's dependencies.
- `id: Date.now()`: every call creates a new object, even for the same text, so the effect restarts the timer. `ChatBox` uses the `id` as the toast's React `key`, so a repeated message is drawn as a fresh element.

`src/renderer/src/hooks/useThumbnail.ts`: loading a screenshot preview

`useThumbnail`

src/renderer/src/hooks/useThumbnail.ts, lines 3–18

```ts
/** A screenshot preview: undefined while loading, null if the file is missing. */
export function useThumbnail(path: string): string | null | undefined {
  const [result, setResult] = useState<{ path: string; url: string | null } | null>(null)

  useEffect(() => {
    let alive = true
    void window.assist.screenshots.thumbnail(path).then((url) => {
      if (alive) setResult({ path, url })
    })
    return () => {
      alive = false
    }
  }, [path])

  return result?.path === path ? result.url : undefined
}
```

Asks the main process for a small preview of a screenshot. It returns three kinds of value: `undefined` while loading, `null` if the file is missing, or the image as a `data:` URL. `AttachmentChip` uses the difference to show "File missing" only when the file really is gone; the sent-screenshot thumbnails in `MessageList` use it too (see the components part).

- `window.assist.screenshots.thumbnail(path)`: `ScreenshotService.thumbnail` in the main process returns a cached, shrunken copy of the image, or `null` if the file is gone or isn't inside the screenshots folder. `data:` images are allowed by the `img-src` rule in `index.html`.
- `useState<{ path: string; url: string | null } | null>(null)`: the result remembers which path it belongs to.
- `[path]`: fetch again whenever the path changes.
- `if (alive) setResult({ path, url })`: ignore a reply for a path that's no longer wanted (the component moved on to another screenshot, or went away).
- `return result?.path === path ? result.url : undefined`: while a new path loads, the previous picture isn't shown for it. Comparing paths here means the effect doesn't have to clear the old result itself.

`src/renderer/src/views/BubbleView.tsx`: the logo bubble

The page shown in the small bubble window: one round button with the tenant's logo. A click asks the main process what to do; pressing and moving drags the bubble.

`logos`, `logoUrl`

src/renderer/src/views/BubbleView.tsx, lines 7–14

```
// The tenant's logo: logo.png, or logo.svg (if a tenant has both, the PNG wins).
const logos = import.meta.glob<string>('@tenant/logo.{png,svg}', {
  eager: true,
  query: '?url',
  import: 'default',
})

const logoUrl =
  Object.entries(logos).find(([path]) => path.endsWith('.png'))?.[1] ?? Object.values(logos)[0]
```

Finds the tenant's logo at build time.

- `import.meta.glob<string>('@tenant/logo.{png,svg}', {`: a Vite feature that imports every file matching a pattern. `@tenant` is the build's tenant folder (set up in electron.vite.config.ts), so this matches `logo.png` and/or `logo.svg`. The result is an object from file path to import.
- `eager: true`: import the files straight away, rather than as functions that load them later.
- `query: '?url'`: import each file as its URL (Vite copies the image into the build).
- `import: 'default'`: take each module's default export, which is that URL.
- `Object.entries(logos).find(([path]) => path.endsWith('.png'))?.[1]`: the PNG's URL if there is one.
- `?? Object.values(logos)[0]`: otherwise the only other match, the SVG. Every tenant must have a logo; tests/tenants.test.ts checks.

`ACTION_LABEL`, `DRAG_THRESHOLD`

src/renderer/src/views/BubbleView.tsx, lines 16–26

```
const ACTION_LABEL: Record<Mode, string> = {
  collapsed: 'Open',
  expanded: 'Close',
  dragging: 'Moving',
  bouncing: 'Stop bouncing',
  returning: 'Returning',
  capturing: 'Taking screenshot',
}

/** Moving the pointer further than this while pressed turns a click into a drag. */
const DRAG_THRESHOLD = 5
```

- `Record<Mode, string>`: a label for every `Mode` (src/shared/types.ts). TypeScript reports an error if a mode is ever added without one. The label becomes the button's accessible name for screen readers, such as "Open Desktop Assist".
- `DRAG_THRESHOLD = 5`: a press that moves no more than 5 pixels is still a click, so a slightly shaky click doesn't start a drag.

`<BubbleView>`

src/renderer/src/views/BubbleView.tsx, lines 28–109

```
/**
 * The always-on-top logo bubble. A click goes to the main process, which decides what it means.
 * Press and move to drag it: the main process makes it follow the mouse, and on release snaps it
```

```tsx
 * to the nearest corner of whichever display it's on.
 */
export function BubbleView() {
  // No useClickThrough here: the bubble window always takes the mouse (see bubble/windows.ts).
  const state = useAssistState()
  useAccentColor(state?.branding.accentColor)
  const mode = state?.mode ?? 'collapsed'
  const appName = state?.branding.appName ?? 'Desktop Assist'

  const press = useRef<{ x: number; y: number; dragging: boolean } | null>(null)
  // When the button last went down: the main process uses it to tell whether this press is what
  // closed the panel (see BubbleController.clickBubble).
  const pressedAt = useRef<number | undefined>(undefined)
  // A drag ends with a click event on the bubble; this stops it counting as a click.
  const swallowClick = useRef(false)

  function onPointerDown(event: PointerEvent<HTMLButtonElement>) {
    if (event.button !== 0) return
    swallowClick.current = false
    pressedAt.current = Date.now()
    event.currentTarget.setPointerCapture(event.pointerId)
    press.current = { x: event.screenX, y: event.screenY, dragging: false }
  }

  function onPointerMove(event: PointerEvent<HTMLButtonElement>) {
    const current = press.current
    if (!current || current.dragging) return
    // Screen coordinates, because the window itself moves once the drag starts.
    if (Math.hypot(event.screenX - current.x, event.screenY - current.y) > DRAG_THRESHOLD) {
      current.dragging = true
      window.assist.bubbleDragStart()
    }
  }

  function endPress() {
    const current = press.current
    press.current = null
    if (current?.dragging) {
      swallowClick.current = true
      window.assist.bubbleDragEnd()
    }
  }

  return (
    <div className="flex h-full w-full items-center justify-center">
      <button
        data-hit
        type="button"
        aria-label={`${ACTION_LABEL[mode]} ${appName}`}
        onPointerDown={onPointerDown}
        onPointerMove={onPointerMove}
        onPointerUp={endPress}
        onLostPointerCapture={endPress}
        onClick={() => {
          if (swallowClick.current) swallowClick.current = false
          // A click from the keyboard has no press: it's "now".
          else window.assist.bubbleClick(pressedAt.current ?? Date.now())
          pressedAt.current = undefined
        }}
        style={{ width: UI.bubbleSize, height: UI.bubbleSize }}
        className={cn(
          'overflow-hidden rounded-full shadow-[0_2px_6px_rgba(0,0,0,0.35)] outline-none',
          'transition-[transform,box-shadow] duration-150 ease-out motion-reduce:transition-none',
          mode === 'dragging' ? 'scale-105' : 'hover:scale-105 active:scale-95',
          // Always a 2px white border; while the panel is open, a purple one outside it.
```

```
          'ring-2',
          mode === 'expanded' ? 'ring-accent ring-offset-2 ring-offset-white' : 'ring-white',
        )}
      >
        <img
          src={logoUrl}
          alt=""
          draggable={false}
          className="pointer-events-none size-full object-contain"
        />
      </button>
    </div>
  )
}
```

The bubble itself. It decides nothing on its own: clicks and drags are sent to `BubbleController` in the main process, which owns the bubble's mode and position. In JSX, attributes are *props*, the inputs a component receives. On built-in elements like `<button>` they become HTML attributes and event handlers (`onClick`, `onPointerDown`); on our own components they arrive as the function's argument (see `Panel` below).

- `const state = useAssistState()`: the bubble needs only the mode (for its look and label) and the branding. Until the state arrives, it falls back to `'collapsed'` and `'Desktop Assist'`.
- `useAccentColor(state?.branding.accentColor)`: `?.` because `state` is `null` at first.
- `const press = useRef<{ x: number; y: number; dragging: boolean } | null>(null)`: `useRef` gives a box (`.current`) that keeps its contents between renders, but unlike state, changing it doesn't re-render. It suits values that only event handlers need: here, where the press started and whether it has become a drag.
- `const pressedAt = useRef<number | undefined>(undefined)`: when the mouse button last went down on the bubble. As the comment says, the main process uses it to tell whether this press is what closed the panel (see `BubbleController.clickBubble` in The bubble).
- `const swallowClick = useRef(false)`: releasing after a drag still fires a `click` on the button. This flag makes that one click be ignored.
- `if (event.button !== 0) return`: only the main (left) mouse button.
- `pressedAt.current = Date.now()`: the press time, on the wall clock the main process uses too.
- `event.currentTarget.setPointerCapture(event.pointerId)`: sends all further events from this pointer to the button, even when the pointer is outside it, until it's released. The window is only 72 pixels square and gets moved during a drag, so without this the release could be lost.
- `if (!current || current.dragging) return`: only needed until the drag starts. From then on the main process moves the window with the mouse itself (`BubbleController.startDrag`), so the page just waits for the release.
- `Math.hypot(event.screenX - current.x, event.screenY - current.y) > DRAG_THRESHOLD`: the distance moved since the press, in screen coordinates, because the window itself moves once the drag starts.
- `window.assist.bubbleDragStart()`: tells the main process to start following the mouse.
- `function endPress() {`: runs on release. If the press had become a drag, it arms `swallowClick` and calls `bubbleDragEnd()`, and `BubbleController.endDrag` snaps the bubble to the nearest corner. If it was a plain click, it does nothing and the `click` event that follows handles it.
- `onLostPointerCapture={endPress}`: capture can also end without a release (the system can take it away), so this ends the press too. After a normal release both events fire; the second call finds `press.current` already `null` and does nothing.
- `onClick={() => {`: a real click calls `bubbleClick()`, and `BubbleController.clickBubble` decides what it means: open, close, or glide home from a bounce.

- `window.assist.bubbleClick(pressedAt.current ?? Date.now())`: the click goes with its press time. As the comment says, a click from the keyboard (Enter or Space on the focused button) has no press, so it's "now". `pressedAt.current = undefined` then forgets it, so it can't be reused by a later keyboard click.
- There's no `title`, so nothing pops up when the pointer rests on the bubble. The `aria-label` still names it for screen readers.
- `data-hit`: the marker `useClickThrough` looks for. The bubble window doesn't use click-through, so here it has no effect.
- `ACTION_LABEL[mode]`: the `aria-label` reads, for example, "Close Desktop Assist" while the panel is open.
- `style={{ width: UI.bubbleSize, height: UI.bubbleSize }}`: 56 pixels, the same number the main process uses to size the window (`BUBBLE_BOX`, 56 plus 8 on each side).
- `<div className="flex h-full w-full items-center justify-center">`: centres the button in the window. The 8-pixel margin around it leaves room for the shadow and ring.
- `shadow-[0_2px_6px_rgba(0,0,0,0.35)]`: square brackets give a Tailwind class a custom value (the underscores stand for spaces).
- `motion-reduce:transition-none`: no animation if animations are turned off in Windows.
- `mode === 'dragging' ? 'scale-105' : 'hover:scale-105 active:scale-95'`: slightly larger while dragged or hovered, slightly smaller while pressed.
- `ring-accent ring-offset-2 ring-offset-white`: while the panel is open, a 2-pixel white gap and then an accent-coloured ring. Otherwise the ring itself is white. Tailwind rings are drawn with `box-shadow`, which is why `box-shadow` is in the transition list.
- `draggable={false}`: stops the browser's own drag-the-image behaviour, which would take over the pointer.
- `pointer-events-none size-full object-contain`: the image ignores the mouse, so every event goes to the button; it fills the button without distortion.

`src/renderer/src/views/PanelView.tsx`: everything around the bubble

The page shown in the panel window: the chat card, the action icons and the settings menu. All the panel's shared state and handlers live in `Panel`; the components it renders (`ChatBox`, `ActionStack`, `SettingsMenu`, `SignInPanel`, `Banner`, `AppsList`, `JiraPage`, `ServiceDeskPage`, `IssueView`) get their data and callbacks from it as props. Those are covered in the components part, except `AppsList`, which is in Your apps, and the Jira and service desk pages, which are in The Jira and service desk pages.

The panel works in one of two ways, set by the tenant's `chatApp` (`branding.chatApp`). With the built-in chat it's a chat with Claude, behind a JumpCloud sign-in. With Claude Desktop it's just the text box: asking opens the question in the Claude Desktop app (and, with "Send in Claude automatically" on, sends it there), and there's no sign-in and no conversation in the panel (`state.auth` is `null` and `state.chat` stays empty). Either way, if the tenant has a portal, the card can also show the Apps list instead of the text box. Which of the two the panel opens on is the tenant's `startPage` (`branding.startPage`); the other is behind its icon in the stack: **Ask Claude** for the text box, **Your apps** for the list. Morse Micro opens on the Apps list. With an Atlassian site, the bell icon swaps in the Jira page, and with the service desk's request types, the service desk icon swaps in the service desk page; a Jira issue opens over either (see Jira and the service desk).

`ASK_ERRORS`

src/renderer/src/views/PanelView.tsx, lines 31–37

```
/** What to say when a question couldn't be handed to Claude Desktop. */
const ASK_ERRORS: Record<Exclude<Extract<AskResult, { ok: false }>['reason'], 'empty'>, string> = {
  'too-long': "That's too long to open in Claude. Shorten it, or paste it into Claude yourself.",
  'not-installed': "Claude Desktop isn't installed on this PC. Ask IT to install it.",
  'missing-screenshot': 'An attached screenshot is missing. Remove it and try again.',
  failed: "Couldn't open Claude Desktop. Try again.",
}
```

The toast for each way handing a question to Claude Desktop can fail (`AskResult` in `src/shared/types.ts`).

- `Extract<AskResult, { ok: false }>['reason']`: `Extract` picks the failure shape out of `AskResult`, and `['reason']` takes the type of its `reason` field: the five reasons.
- `Exclude<..., 'empty'>`: leaves out `empty`, which needs no message (Ask is disabled while the box is empty anyway).
- `Record<..., string>`: an object with a message for every remaining reason. If a reason is ever added to `AskResult`, this won't compile until it has a message.
- `'not-installed'`: the panel already shows a banner when it opens on a PC without Claude Desktop; this toast is for asking anyway.

`UNINSTALL_ERRORS`

src/renderer/src/views/PanelView.tsx, lines 39–44

```
/** What to say when Settings → Uninstall couldn't start the uninstaller. */
const UNINSTALL_ERRORS: Record<Extract<UninstallResult, { ok: false }>['reason'], string> = {
  'not-installed': 'Only the installed app can be uninstalled from here.',
  missing: 'The uninstaller is missing. Uninstall from Windows Settings → Apps.',
  failed: "Couldn't start the uninstaller. Uninstall from Windows Settings → Apps.",
}
```

The toast for each way Settings → Uninstall can fail (`UninstallResult`), built the same way as `ASK_ERRORS`. The two that can happen in an installed copy point to Windows Settings, where the app can always be uninstalled.

`View` and `EMPTY_FEED`

src/renderer/src/views/PanelView.tsx, lines 46–54

```
/**
 * What the card shows: the text box (`ask`), or a page in its place while that page's icon is on.
 */
type View = StartPage | 'jira' | 'servicedesk'

const EMPTY_FEED: JiraFeed = {
  notifications: { status: 'loading', items: [] },
  requests: { status: 'loading', items: [] },
}
```

- `View`: as the comment says, what the card shows: one of the two start pages (`StartPage`: the text box or the Apps list), or the Jira page or the service desk page while its icon is on.
- `EMPTY_FEED`: the notifications and requests before the main process has said anything: two empty lists, both still loading.

`<PanelView>`

src/renderer/src/views/PanelView.tsx, lines 56–62

```tsx
/** Everything that appears around the bubble when it's clicked. */
export function PanelView() {
  useClickThrough()
  const state = useAssistState()
  useAccentColor(state?.branding.accentColor)
  return state ? <Panel state={state} /> : null
}
```

Sets up click-through and the live state, then renders `Panel` once the state has arrived.

- `useClickThrough()`: the panel window is the click-through one (see above).
- `return state ? <Panel state={state} /> : null`: returning `null` renders nothing. `Panel` is only created once the state exists because its `useState` calls take their starting values from it, and hooks can't be called conditionally. Splitting into two components is the usual way to wait for data before those hooks run.
- `state={state}`: passes the state to `Panel` as a prop. Each time `useAssistState` produces a new state, `PanelView` re-renders and `Panel` gets the new one.

`<Panel>`

`Panel` is long, so it's shown whole first, with notes on its state, effects and the JSX it returns. The sections after it take its values and functions one at a time.

src/renderer/src/views/PanelView.tsx, lines 64–532

```tsx
function Panel({ state }: { state: AppState }) {
  const open = state.mode === 'expanded'
  const { branding, auth } = state
  // Questions go to the Claude Desktop app instead of the chat in the panel (no sign-in here).
  const inClaudeDesktop = branding.chatApp === 'claude-desktop'

  // The draft is edited here and mirrored to the main process, which saves it.
  const [draft, setDraft] = useState(state.notes.text)
  const [attachments, setAttachments] = useState(state.notes.attachments)
  const [settings, setSettings] = useState(state.settings)
  const [settingsOpen, setSettingsOpen] = useState(false)
  const [asking, setAsking] = useState(false)
  // The card shows the tenant's start page (the text box or the Apps list) when the panel opens,
  // and another page while its icon is on (Ask Claude, Your apps, Jira, the service desk).
  const { startPage, atlassianSite, requestTypes } = branding
  const [view, setView] = useState<View>(startPage)
  const [apps, setApps] = useState<AppsState>({ status: 'loading' })
  const [openingApp, setOpeningApp] = useState<string | null>(null)
  const [atlassian, setAtlassian] = useState<AtlassianState>({ status: 'loading' })
  const [feed, setFeed] = useState<JiraFeed>(EMPTY_FEED)
  /** The Jira issue shown in the card (over whichever page it was opened from), or null. */
  const [issueKey, setIssueKey] = useState<string | null>(null)
  const [serviceDeskTab, setServiceDeskTab] = useState<ServiceDeskTab>('new')
  /** The Jira projects' keys, to only offer real issue keys from the text box. */
  const [projectKeys, setProjectKeys] = useState<string[] | null>(null)
  /** Claude Desktop isn't installed, so questions can't be opened in it (checked on opening). */
  const [claudeMissing, setClaudeMissing] = useState(false)
  const [toast, showToast] = useToast()

  useEffect(
    () =>
      window.assist.onModeChanged((mode) => {
```

```
      if (mode !== 'expanded') setSettingsOpen(false)
      // The panel always reopens on the start page (but taking a screenshot isn't closing it).
      if (mode === 'collapsed') {
        setView(startPage)
        setIssueKey(null)
      }
    }),
  [startPage],
)
useEffect(() => window.assist.onAppsState(setApps), [])
useEffect(() => {
  if (!atlassianSite) return
  const unsubscribe = [
    window.assist.onAtlassianState(setAtlassian),
    window.assist.onJiraFeed(setFeed),
  ]
  void window.assist.atlassian.get().then(setAtlassian)
  void window.assist.atlassian.feed().then(setFeed)
  return () => unsubscribe.forEach((stop) => stop())
}, [atlassianSite])
const atlassianConnected = atlassian.status === 'connected'
useEffect(() => {
  if (!atlassianConnected) return
  let alive = true
  void window.assist.atlassian.projectKeys().then((keys) => alive && setProjectKeys(keys))
  return () => {
    alive = false
  }
}, [atlassianConnected])
// The Apps list is loaded (or remembered) by the main process; ask for it whenever it shows.
useEffect(() => {
  if (!open || view !== 'apps' || !branding.portalName) return
  let alive = true
  void window.assist.apps.get().then((current) => {
    if (alive) setApps(current)
  })
  return () => {
    alive = false
  }
}, [open, view, branding.portalName])
useEffect(() => {
  if (!open || !inClaudeDesktop) return
  let alive = true
  void window.assist.claudeDesktop.isInstalled().then((installed) => {
    if (alive) setClaudeMissing(!installed)
  })
  return () => {
    alive = false
  }
}, [open, inClaudeDesktop])
useEffect(() => {
  const onKeyDown = (event: KeyboardEvent) => {
    if (event.key !== 'Escape') return
    if (settingsOpen) setSettingsOpen(false)
    else if (issueKey) setIssueKey(null)
    else if (view !== startPage) setView(startPage)
    else window.assist.collapse()
  }
  window.addEventListener('keydown', onKeyDown)
  return () => window.removeEventListener('keydown', onKeyDown)
}, [settingsOpen, issueKey, view, startPage])

// Not signed in: the chat box shows the sign-in instead of the chat. While a saved sign-in is
// being renewed, or JumpCloud can't be reached, the chat stays (sending tries JumpCloud again).
// With Claude Desktop there's no sign-in here at all.
```

```
const signedIn = auth === null || auth.state === 'signed-in' || auth.state === 'offline'
// `busy` is the built-in chat's reply streaming (the text box then offers Stop). Handing a
// question to Claude Desktop only takes a moment, so it just holds the button back.
const busy = state.chat.some((message) => message.status === 'streaming')
const canSend = signedIn && !busy && !asking && (draft.trim() !== '' || attachments.length > 0)

async function runAction(id: ActionId) {
  // With request types, the service desk icon opens a page for them instead of the portal.
  if (id === 'apps' || id === 'ask' || id === 'jira' || (id === 'servicedesk' && requestTypes)) {
    // Swaps the card to that page, or back again: the text box and the Apps list swap with
    // each other, other pages go back to the start page.
    setSettingsOpen(false)
    setIssueKey(null)
    if (id === 'servicedesk') setServiceDeskTab('new')
    const back: View = id === 'apps' ? 'ask' : id === 'ask' ? 'apps' : startPage
    setView(view === id && !issueKey ? back : id)
    return
  }
  if (id === 'theme') {
    setSettings(
      await window.assist.settings.setTheme(settings.theme === 'dark' ? 'light' : 'dark'),
    )
    return
  }
  if (id === 'settings') {
    setSettingsOpen(!settingsOpen)
    // Refresh: "Start with Windows" can also be changed in Windows Settings.
    if (!settingsOpen) setSettings(await window.assist.settings.get())
    return
  }
  setSettingsOpen(false)
  const result = await window.assist.invokeAction(id)
  if (result.message) showToast(result.message, result.ok ? 'info' : 'error')
}

function changeDraft(next: string) {
  setDraft(next)
  window.assist.notes.setText(next)
}

async function send() {
  if (!canSend) return
  if (inClaudeDesktop) return askInClaudeDesktop()
  const result = await window.assist.chat.send(draft)
  if (result.ok) {
    setDraft(result.notes.text)
    setAttachments(result.notes.attachments)
  } else if (result.reason === 'missing-screenshot') {
    showToast('An attached screenshot is missing. Remove it and try again.', 'error')
  } else if (result.reason === 'signed-out') {
    showToast('Sign in with JumpCloud first.', 'error')
  }
}

async function askInClaudeDesktop() {
  setAsking(true)
  const result = await window.assist.claudeDesktop.ask(draft).finally(() => setAsking(false))
  if (result.ok) setClaudeMissing(false)
  else if (result.reason === 'not-installed') setClaudeMissing(true)
  if (result.ok) {
    // The panel has closed and Claude Desktop has opened with the question filled in.
    setDraft(result.notes.text)
    setAttachments(result.notes.attachments)
  } else if (result.reason !== 'empty') {
    showToast(ASK_ERRORS[result.reason], 'error')
```

```
  }
}

async function toggleFavorite(app: PortalApp, favorite: boolean) {
  setSettings(await window.assist.settings.setFavoriteApp(app.id, favorite))
}

async function openApp(app: PortalApp) {
  // The tile spins until the browser is opening (JumpCloud can take a few seconds to give the
  // sign-in link); on success the main process closes the panel.
  setOpeningApp(app.id)
  try {
    if (!(await window.assist.apps.open(app.id))) {
      showToast(`Couldn't open ${app.name}. Try it from the portal.`, 'error')
    }
  } finally {
    setOpeningApp((current) => (current === app.id ? null : current))
  }
}

/** Opens a Jira issue in the card if connected to Atlassian, otherwise in the browser. */
function openIssue(key: string) {
  setSettingsOpen(false)
  if (atlassianConnected) setIssueKey(key)
  else if (atlassianSite) void window.assist.openExternal(browseUrl(atlassianSite, key))
}

/** Starts an IT request from what's in the text box. */
function report() {
  setSettingsOpen(false)
  setIssueKey(null)
  setServiceDeskTab('new')
  setView('servicedesk')
}

async function disconnectAtlassian() {
  setSettingsOpen(false)
  setIssueKey(null)
  await window.assist.atlassian.signOut()
  showToast('Disconnected from Atlassian')
}

async function attachLatest() {
  const result = await window.assist.notes.attachLatestScreenshot()
  if (result.ok) setAttachments(result.notes.attachments)
  else if (result.reason === 'no-screenshots') {
    showToast('No screenshots yet. Take one with the camera button.', 'error')
  } else showToast('No more screenshots to attach', 'error')
}

async function removeAttachment(attachment: Attachment) {
  const notes = await window.assist.notes.removeAttachment(attachment.id)
  setAttachments(notes.attachments)
}

async function openAttachment(attachment: Attachment) {
  const opened = await window.assist.screenshots.open(attachment.path)
  if (!opened) showToast("That screenshot can't be found", 'error')
}

async function copy(text: string) {
  await window.assist.copyText(text)
  showToast('Copied')
}
```

```
  async function signOut() {
    setSettingsOpen(false)
    await window.assist.auth.signOut()
    showToast('Logged out')
  }

  async function clearText() {
    const notes = await window.assist.notes.clear()
    setDraft(notes.text)
    setAttachments(notes.attachments)
    setSettingsOpen(false)
    showToast('Text box cleared')
  }

  async function uninstall() {
    setSettingsOpen(false)
    const result = await window.assist.uninstall()
    // On success the uninstaller has started and the app is quitting.
    if (!result.ok) showToast(UNINSTALL_ERRORS[result.reason], 'error')
  }

  async function newConversation() {
    await window.assist.chat.newConversation()
    setSettingsOpen(false)
    showToast('Started a new conversation')
  }

  async function setEffort(effort: Effort) {
    setSettings(await window.assist.settings.setEffort(effort))
  }

  async function toggleAutoSend() {
    setSettings(await window.assist.settings.setAutoSend(!settings.autoSend))
  }

  // Clicking anywhere else in the panel closes the settings menu.
  function closeSettingsOnOutsideClick(event: PointerEvent) {
    if (!settingsOpen || !(event.target instanceof Element)) return
    if (!event.target.closest('[data-settings-menu], [data-action="settings"]')) {
      setSettingsOpen(false)
    }
  }

  const signInPrompt =
    auth?.state === 'unconfigured' ||
    auth?.state === 'signed-out' ||
    auth?.state === 'signing-in' ? (
      <SignInPanel
        open={open}
        status={auth}
        appName={branding.appName}
        onSignIn={() => void window.assist.auth.signIn()}
        onCancel={() => void window.assist.auth.cancel()}
      />
    ) : null

  const banner =
    inClaudeDesktop && claudeMissing ? (
      <Banner
        action={
          <button
            type="button"
            onClick={() => void window.assist.openExternal(CLAUDE_DOWNLOAD_PAGE)}
            className="shrink-0 rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100
dark:text-accent-soft dark:hover:bg-zinc-800"
```

```
          >
            Download
          </button>
        }
      >
        Claude Desktop isn't installed on this PC, and {branding.appName} opens your questions in
        it. Ask IT to install it, or download it.
      </Banner>
    ) : auth?.state === 'checking' ? (
      <Banner spinner>Checking your JumpCloud sign-in…</Banner>
    ) : auth?.state === 'offline' ? (
      <Banner
        action={
          <button
            type="button"
            onClick={() => void window.assist.auth.retry()}
            className="rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100 dark:text-
accent-soft dark:hover:bg-zinc-800"
          >
            Retry
          </button>
        }
      >
        {auth.message}
      </Banner>
    ) : null

  const settingsIndex = branding.actions.indexOf('settings')
  // Only keys of real projects, once the projects are known.
  const issueKeys = atlassianSite
    ? findIssueKeys(draft).filter(
        (key) =>
          !atlassianConnected ||
          !projectKeys ||
          projectKeys.includes(key.slice(0, key.lastIndexOf('-'))),
      )
    : []
  const unreadNotifications = feed.notifications.items.filter((n) => n.unread).length
  const requestNews = feed.requests.items.filter((r) => r.news).length

  const pageContent =
    view === 'apps' && branding.portalName ? (
      <AppsList
        open={open}
        state={apps}
        appName={branding.appName}
        portalName={branding.portalName}
        favorites={settings.favoriteApps}
        onToggleFavorite={(app, favorite) => void toggleFavorite(app, favorite)}
        onOpenApp={(app) => void openApp(app)}
        openingId={openingApp}
        onOpenPortal={() => void window.assist.apps.openPortal()}
        onSignIn={() => void window.assist.apps.signIn()}
        onCancelSignIn={() => void window.assist.apps.cancelSignIn()}
        onRetry={() => void window.assist.apps.get(true).then(setApps)}
      />
    ) : view === 'jira' && atlassianSite ? (
      <JiraPage
        open={open}
        state={atlassian}
        feed={feed}
        appName={branding.appName}
        site={atlassianSite}
        onOpenIssue={openIssue}
      />
```

```
    ) : view === 'servicedesk' && requestTypes ? (
      <ServiceDeskPage
        open={open}
        initialTab={serviceDeskTab}
        requestTypes={requestTypes}
        atlassian={atlassianSite ? atlassian : null}
        feed={feed}
        appName={branding.appName}
        draft={draft}
        attachments={attachments}
        onAttachLatest={() => void attachLatest()}
        onOpenAttachment={(attachment) => void openAttachment(attachment)}
        onRemoveAttachment={(attachment) => void removeAttachment(attachment)}
        onOpenIssue={openIssue}
        showToast={showToast}
      />
    ) : null
  // An issue opens over the page it was opened from, which stays (hidden) so Back finds it as it
  // was: the same tab, search or half-filled form.
  const page =
    issueKey || pageContent ? (
      <>
        {issueKey && (
          <IssueView
            key={issueKey}
            issueKey={issueKey}
            inClaudeDesktop={inClaudeDesktop}
            onBack={() => setIssueKey(null)}
            onAskedInChat={() => {
              setIssueKey(null)
              setView('ask')
            }}
            showToast={showToast}
          />
        )}
        <div hidden={issueKey !== null} className="contents">
          {pageContent}
        </div>
      </>
    ) : null

  return (
    <div
      className={cn('group relative h-full w-full', !open && 'pointer-events-none')}
      data-open={open ? '' : undefined}
      onPointerDown={closeSettingsOnOutsideClick}
    >
      <ChatBox
        corner={state.corner}
        open={open}
        toast={toast}
        inClaudeDesktop={inClaudeDesktop}
        autoSend={settings.autoSend}
        page={page}
        signInPrompt={signInPrompt}
        banner={banner}
        messages={state.chat}
        busy={busy}
        canSend={canSend}
        draft={draft}
        attachments={attachments}
        onDraftChange={changeDraft}
        onSend={() => void send()}
        onStop={() => void window.assist.chat.stop()}
        onRetry={() => void window.assist.chat.retry()}
```

```
        onCopy={(text) => void copy(text)}
        onAttachLatest={() => void attachLatest()}
        onOpenAttachment={(attachment) => void openAttachment(attachment)}
        onRemoveAttachment={(attachment) => void removeAttachment(attachment)}
        issueKeys={issueKeys}
        onOpenIssue={openIssue}
        onReport={requestTypes ? report : null}
      />
      <ActionStack
        corner={state.corner}
        actions={branding.actions}
        open={open}
        activeId={settingsOpen ? 'settings' : view !== startPage ? view : null}
        theme={settings.theme}
        pages={requestTypes ? ['servicedesk'] : []}
        dotted={[
          ...(view !== 'ask' && (draft.trim() !== '' || attachments.length > 0)
            ? (['ask'] as const)
            : []),
          ...(view !== 'jira' && unreadNotifications > 0 ? (['jira'] as const) : []),
          ...(view !== 'servicedesk' && requestNews > 0 ? (['servicedesk'] as const) : []),
        ]}
        onAction={(id) => void runAction(id)}
      />
      {open && settingsOpen && settingsIndex >= 0 && (
        <SettingsMenu
          corner={state.corner}
          offset={actionOffset(settingsIndex)}
          settings={settings}
          branding={branding}
          version={state.version}
          user={auth?.state === 'signed-in' || auth?.state === 'offline' ? auth.user : null}
          atlassianUser={atlassian.status === 'connected' ? atlassian.user : null}
          onDisconnectAtlassian={() => void disconnectAtlassian()}
          onToggleAutoStart={async () =>
            setSettings(await window.assist.settings.setAutoStart(!settings.autoStart))
          }
          onSetEffort={(effort) => void setEffort(effort)}
          onToggleAutoSend={() => void toggleAutoSend()}
          onSignOut={() => void signOut()}
          onOpenScreenshotsFolder={() => void window.assist.screenshots.openFolder()}
          onNewConversation={() => void newConversation()}
          onClearText={() => void clearText()}
          onUninstall={() => void uninstall()}
        />
      )}
    </div>
  )
}
```

Props and state:

- `function Panel({ state }: { state: AppState })`: a component receives its props as one object. This takes `state` out of it, and `{ state: AppState }` is the type of the props.
- `const open = state.mode === 'expanded'`: the panel is open only in `expanded`. In every other mode (collapsed, dragging, bouncing and so on) it's closed or hidden.
- `const inClaudeDesktop = branding.chatApp === 'claude-desktop'`: whether questions go to Claude Desktop. Much of what follows checks it.
- `const [draft, setDraft] = useState(state.notes.text)`: the text box's contents. Like `attachments` and `settings`, it starts from the app state but is then owned here: the panel updates it as you type and from the

main process's replies.

- `const [settingsOpen, setSettingsOpen] = useState(false)`: whether the settings menu is showing.
- `const [asking, setAsking] = useState(false)`: true while a question is being handed to Claude Desktop, so it can't be sent twice. (Only for that moment: sending it in Claude carries on in the main process after the panel has closed.)
- `const { startPage, atlassianSite, requestTypes } = branding`: the tenant's start page, its Atlassian site (`null` without one) and its service desk's request types (`null` without them).
- `const [view, setView] = useState<View>(startPage)`: what the card shows: the text box (`ask`), the Apps list (`apps`), the Jira page or the service desk page. It starts on the tenant's start page, as the comment says, and another page shows while its icon is on. `<View>` gives the state its type, since TypeScript would otherwise take it to be just the starting value's type.
- `const [apps, setApps] = useState<AppsState>({ status: 'loading' })`: the Apps list as the main process last reported it. It isn't part of the app state the page loads at the start; it's asked for whenever the list shows (the effect below).
- `const [openingApp, setOpeningApp] = useState<string | null>(null)`: the ID of the app being opened, so its tile can show a spinner, or `null`.
- `atlassian`, `feed`: the Atlassian connection and the notifications and requests, as the main process last reported them.
- `issueKey`: as the comment says, the Jira issue shown in the card over the page it was opened from, or `null`.
- `serviceDeskTab`: which tab the service desk page opens on.
- `projectKeys`: as the comment says, the keys of the Jira projects the user can see, or `null` until they're known.
- `const [claudeMissing, setClaudeMissing] = useState(false)`: whether Claude Desktop was found to be missing, which shows a banner with a Download button.
- `const [toast, showToast] = useToast()`: the current toast and the function to show one.

Effects:

- `window.assist.onModeChanged((mode) => {`: closes the settings menu whenever the panel closes, so it isn't still open the next time. The arrow returns what `onModeChanged` returns, the unsubscribe function, which React uses as the cleanup.
- `if (mode === 'collapsed') {`: once the panel has really closed, the next opening is on the start page again (`setView(startPage)`), with no issue open (`setIssueKey(null)`). Only `collapsed` counts: taking a screenshot hides the panel too (`capturing`), and the user expects to come back to the same page afterwards.
- `useEffect(() => window.assist.onAppsState(setApps), [])`: keeps the Apps list up to date as the main process reports changes (connecting, loading, ready). `setApps` itself is the callback.
- `if (!atlassianSite) return`: with an Atlassian site, the connection and the feed are kept up to date from their two broadcasts (`onAtlassianState`, `onJiraFeed`), and asked for once at the start (`atlassian.get()`, `atlassian.feed()`), so the bell's dot can show before the Jira page is ever opened. The cleanup stops both subscriptions: `unsubscribe` is the list of the two stop functions, and `forEach` calls each.
- `const atlassianConnected = atlassian.status === 'connected'`: used by several things below.
- `if (!atlassianConnected) return`: once connected, the Jira project keys are asked for (`atlassian.projectKeys()`), with the same `alive` guard as below. The effect runs again if the user disconnects and connects again.
- `if (!open || view !== 'apps' || !branding.portalName) return`: whenever the panel is open on the Apps list, it asks the main process for the list (`window.assist.apps.get()`). The main process already started

loading it at startup and keeps it, so this is usually instant; the first time, it may still be on its way, and arrives through `onAppsState` too. `alive` guards against a late answer, as below.

- `if (!open || !inClaudeDesktop) return`: with Claude Desktop, each time the panel opens it asks whether Claude Desktop is installed (`window.assist.claudeDesktop.isInstalled()`), so a missing app is pointed out before anything is typed. Asking every time, rather than once, picks up Claude Desktop being installed while Desktop Assist is running.
- `let alive = true`, `if (alive) setClaudeMissing(!installed)`: the same guard as in `useAssistState`. If the panel closes before the answer arrives, the cleanup sets `alive` to `false` and the late answer is ignored.
- `}, [open, inClaudeDesktop])`: the effect runs again when the panel opens or closes.
- `if (event.key !== 'Escape') return`: Esc closes the settings menu if it's open, otherwise closes an open issue (back to the page under it), otherwise goes back to the start page if another page is showing, otherwise asks the main process to close the panel (`BubbleController.collapse`). Keys reach the panel because the main process focuses it when it opens.
- `}, [settingsOpen, issueKey, view, startPage])`: the listener reads these, and a function only sees the values from the render it was created in. Listing them here makes React remove the old listener and add a fresh one whenever one changes.

The JSX it returns:

- `className={cn('group relative h-full w-full', !open && 'pointer-events-none')}`: the root fills the window. `relative` makes it the reference box for the children's `absolute` positions (from `anchored`). `group` lets children style themselves from this element's attributes.
- `pointer-events-none`: while closed, nothing in the panel can be clicked, and `useClickThrough` finds no `data-hit` element, so the window stays click-through. This covers the moment the panel is fading out before the main process hides the window.
- `data-open={open ? '' : undefined}`: the attribute is present while open and left out while closed (React omits attributes set to `undefined`). Children use `group-data-open:` classes, such as `group-data-open:opacity-100`, so the open and close animations are plain CSS transitions.
- `onPointerDown={closeSettingsOnOutsideClick}`: see below.
- `<ChatBox`: the card. It gets what to draw (corner, toast, sign-in prompt, banner, messages, draft and attachments) and a callback for each thing you can do there.
- `inClaudeDesktop={inClaudeDesktop}`: the card shows an "Ask in Claude" button instead of the round Send arrow, and a note about what happens to attached screenshots.
- `autoSend={settings.autoSend}`: whether "Send in Claude automatically" is on, which changes that note and the button's tooltip. It comes from the panel's own `settings`, so flipping the switch in the menu shows at once.
- `onSend={() => void send()}`: the handlers are `async`, so they return promises. Each is wrapped in an arrow with `void`, which marks the promise as deliberately not awaited and makes the callback return nothing.
- `page={page}`: the page the card shows in place of everything else (the Apps list, the Jira page, the service desk page, or an issue), built further down (`pageContent` and `page`), or `null` for the text box.
- `issueKeys={issueKeys}`, `onOpenIssue={openIssue}`: the Jira issue keys typed in the text box, which get an **Open** button each, and what that button does.
- `onReport={requestTypes ? report : null}`: **Report to IT** under the text box, only when the tenant lists request types.
- `activeId={settingsOpen ? 'settings' : view !== startPage ? view : null}`: the gear stays highlighted while its menu is open, and otherwise the icon of the page showing, when it isn't the start page (Ask Claude

while the text box is on, for a panel that starts on the Apps list; the bell while the Jira page is on). The start page itself has no icon to highlight.

- `theme={settings.theme}`: the current mode, so the theme icon shows the mode it switches to.
- `pages={requestTypes ? ['servicedesk'] : []}`: the service desk icon opens a page when there are request types, so `ActionStack` marks it as expandable like the popover icons.
- `dotted={[...]}`: small dots on icons with something behind them: Ask Claude while there's an unsent question or screenshot, so it isn't forgotten while another page is showing; the bell while there are unread notifications; and the service desk while one of the user's requests has news. No icon gets a dot while its own page is showing. `...(condition ? (['ask'] as const) : [])` adds an entry only when its condition holds; `as const` keeps it the exact id `'ask'` rather than any string.
- `{open && settingsOpen && settingsIndex >= 0 && (`: JSX's way of saying "only if": when any part is false, nothing is rendered. Removing the menu also resets its own state (its "click again to confirm" on New conversation or Clear text box).
- `offset={actionOffset(settingsIndex)}`: how far the gear is from the bubble, so the menu lines up with it.
- `user={auth?.state === 'signed-in' || auth?.state === 'offline' ? auth.user : null}`: who's signed in, for the menu's "Signed in as" and Log out. `auth?.state` is `undefined` when `auth` is `null` (Claude Desktop), so then there's no user. After the check, TypeScript knows `auth` is one of the two states that have a `user`.
- `atlassianUser={atlassian.status === 'connected' ? atlassian.user : null}`, `onDisconnectAtlassian`: who's connected to Atlassian, for the menu's "Connected to Atlassian as" and **Disconnect Atlassian** (`disconnectAtlassian`, below).
- `onToggleAutoStart={async () =>`: flips Start with Windows and shows the settings the main process returns.
- `onToggleAutoSend={() => void toggleAutoSend()}`: the menu's "Send in Claude automatically" switch, offered with Claude Desktop instead of Response style.
- `onClearText={() => void clearText()}`: the menu's Clear text box, offered instead of New conversation with Claude Desktop.
- `onUninstall={() => void uninstall()}`: the menu's Uninstall.

`signedIn`, `busy`, `canSend`

src/renderer/src/views/PanelView.tsx, lines 158–165

```
// Not signed in: the chat box shows the sign-in instead of the chat. While a saved sign-in is
// being renewed, or JumpCloud can't be reached, the chat stays (sending tries JumpCloud again).
// With Claude Desktop there's no sign-in here at all.
const signedIn = auth === null || auth.state === 'signed-in' || auth.state === 'offline'

// `busy` is the built-in chat's reply streaming (the text box then offers Stop). Handing a
// question to Claude Desktop only takes a moment, so it just holds the button back.
const busy = state.chat.some((message) => message.status === 'streaming')

const canSend = signedIn && !busy && !asking && (draft.trim() !== '' || attachments.length > 0)
```

Values worked out on every render from the current state.

- `auth === null`: with Claude Desktop there's no sign-in in the panel (Claude Desktop has its own), so the panel always counts as signed in. Checking `null` first also lets TypeScript allow `auth.state` in the rest of the line.
- `auth.state === 'signed-in' || auth.state === 'offline'`: `offline` counts as signed in: the sign-in is kept while JumpCloud can't be reached, and sending a message tries JumpCloud again.

- `state.chat.some((message) => message.status === 'streaming')`: the built-in chat's Claude is still replying. As the comment says, that's all `busy` means: while it's true the text box offers Stop instead of Send. With Claude Desktop the chat is always empty, so `busy` is never true there.
- `!asking`: a question is being handed to Claude Desktop. That only takes a moment, so it just holds the button back rather than counting as `busy` (which would show a Stop button with nothing to stop).
- `draft.trim() !== '' || attachments.length > 0`: there must be some text or at least one screenshot. `ChatBox` disables Send, and Enter does nothing, unless `canSend` is true.

runAction

src/renderer/src/views/PanelView.tsx, lines 167–194

```
async function runAction(id: ActionId) {
  // With request types, the service desk icon opens a page for them instead of the portal.
  if (id === 'apps' || id === 'ask' || id === 'jira' || (id === 'servicedesk' && requestTypes)) {
    // Swaps the card to that page, or back again: the text box and the Apps list swap with
    // each other, other pages go back to the start page.
    setSettingsOpen(false)
    setIssueKey(null)
    if (id === 'servicedesk') setServiceDeskTab('new')
    const back: View = id === 'apps' ? 'ask' : id === 'ask' ? 'apps' : startPage
    setView(view === id && !issueKey ? back : id)
    return
  }
  if (id === 'theme') {
    setSettings(
      await window.assist.settings.setTheme(settings.theme === 'dark' ? 'light' : 'dark'),
    )
    return
  }
  if (id === 'settings') {
    setSettingsOpen(!settingsOpen)
    // Refresh: "Start with Windows" can also be changed in Windows Settings.
    if (!settingsOpen) setSettings(await window.assist.settings.get())
    return
  }
  setSettingsOpen(false)
  const result = await window.assist.invokeAction(id)
  if (result.message) showToast(result.message, result.ok ? 'info' : 'error')
}
```

Runs when an action icon is clicked (`ActionStack`'s `onAction`).

- `if (id === 'apps' || id === 'ask' || id === 'jira' || (id === 'servicedesk' && requestTypes))`: the icons that swap the card to a page. Your apps, Ask Claude and Jira are "popover" actions (src/shared/actions.ts); the service desk is a command, but, as the comment says, with request types it opens its page instead of the portal. They close the settings menu and any open issue, and the service desk page always opens on New request. A tenant shows only one of Ask Claude and Your apps: the icon of the page it doesn't start on.
- `const back: View = id === 'apps' ? 'ask' : id === 'ask' ? 'apps' : startPage`: where clicking the icon of the page that's showing goes back to. As the comment says, the text box and the Apps list swap with each other, and the other pages go back to the start page.
- `setView(view === id && !issueKey ? back : id)`: clicking the icon of a page that isn't showing shows it; clicking it again goes back. With an issue open over its page, the click shows the page itself instead. The Apps list is fetched by the effect above as soon as it shows.

- `if (id === 'theme') {`: the light or dark mode icon, a "switch" action. It asks the main process for the other mode (`settings.setTheme()`), which switches every window at once, and keeps the settings it returns, so the icon shows the new mode.
- `if (id === 'settings') {`: Settings is a "popover" action too: it opens UI inside the panel and never reaches the main process.
- `setSettingsOpen(!settingsOpen)`: setting state doesn't change the variable in this run of the function. `settingsOpen` still holds the value from before the toggle, so the next line's `if (!settingsOpen)` means "the menu is being opened now".
- `setSettings(await window.assist.settings.get())`: refreshes the settings as the menu opens, because Start with Windows can also be changed in Windows Settings.
- `window.assist.invokeAction(id)`: after the `apps`, `ask`, `jira`, `theme` and `settings` branches return, TypeScript knows `id` is `servicedesk` (without request types), `screenshot`, `bounce` or `close`, which is exactly the `CommandActionId` type `invokeAction` accepts. The main process runs the matching handler from src/main/actions.ts.
- `if (result.message) showToast(result.message, result.ok ? 'info' : 'error')`: for example "Screenshot saved", or "Couldn't take a screenshot" in red. Bounce, close and the service desk return no message when they work (the service desk closes the panel as the browser opens).
- There's no change of page after a screenshot: the panel comes back showing whatever it showed before (the Apps list, say). The view isn't reset either, because taking a screenshot moves the mode through `capturing`, not `collapsed` (see the mode effect above).

changeDraft

src/renderer/src/views/PanelView.tsx, lines 196–199

```
function changeDraft(next: string) {
  setDraft(next)
  window.assist.notes.setText(next)
}
```

Runs on every keystroke in the text box. It updates the box, then sends the text to the main process (`notes.setText`, which doesn't wait for a reply). There, `NotesStore.setText` keeps it and writes notes.json about half a second after typing stops, so an unsent message survives a restart.

send

src/renderer/src/views/PanelView.tsx, lines 201–213

```
async function send() {
  if (!canSend) return
  if (inClaudeDesktop) return askInClaudeDesktop()
  const result = await window.assist.chat.send(draft)
  if (result.ok) {
    setDraft(result.notes.text)
    setAttachments(result.notes.attachments)
  } else if (result.reason === 'missing-screenshot') {
    showToast('An attached screenshot is missing. Remove it and try again.', 'error')
  } else if (result.reason === 'signed-out') {
    showToast('Sign in with JumpCloud first.', 'error')
  }
}
```

Sends the draft to Claude: to the built-in chat, or to Claude Desktop.

- `if (!canSend) return`: a second guard; `ChatBox` checks too.
- `if (inClaudeDesktop) return askInClaudeDesktop()`: with Claude Desktop, the rest of this function doesn't apply; see `askInClaudeDesktop` below.
- `window.assist.chat.send(draft)`: only the text is sent. The main process takes the attachments from its own copy of the draft, prepares the screenshots and starts the reply (the `chatSend` handler in src/main/ipc.ts). Claude's reply doesn't come back here: it streams in through `onChatMessage` in `useAssistState`.
- `setDraft(result.notes.text)`: the main process cleared the draft after sending; the box adopts the cleared version, as do the attachments.
- `result.reason === 'missing-screenshot'`: an attached file was deleted after it was attached.
- `result.reason === 'signed-out'`: the sign-in ran out just before sending.

The other two reasons, `busy` and `empty`, are ignored: `canSend` already rules them out.

askInClaudeDesktop

src/renderer/src/views/PanelView.tsx, lines 215–227

```
async function askInClaudeDesktop() {
  setAsking(true)
  const result = await window.assist.claudeDesktop.ask(draft).finally(() => setAsking(false))
  if (result.ok) setClaudeMissing(false)
  else if (result.reason === 'not-installed') setClaudeMissing(true)
  if (result.ok) {
    // The panel has closed and Claude Desktop has opened with the question filled in.
    setDraft(result.notes.text)
    setAttachments(result.notes.attachments)
  } else if (result.reason !== 'empty') {
    showToast(ASK_ERRORS[result.reason], 'error')
  }
}
```

Hands the draft to Claude Desktop (see Claude Desktop). When it works, the main process has opened Claude Desktop with the question filled in, copied any screenshots for pasting, closed the panel and emptied the draft. Sending the question in Claude (with "Send in Claude automatically" on) carries on in the main process; the panel isn't told how it went.

- `setAsking(true)`: makes `canSend` false, so Enter and the Ask button do nothing until the answer comes back.
- `window.assist.claudeDesktop.ask(draft)`: as with `chat.send()`, only the text is passed; the main process takes the screenshots from its own copy of the draft.
- `.finally(() => setAsking(false))`: runs whether the request succeeded or failed, so `asking` can't get stuck on.
- `if (result.ok) setClaudeMissing(false)`, `else if (result.reason === 'not-installed')`: a question that got through hides the Download banner, and `not-installed` shows it. Other failures (too long, say) say nothing about whether Claude Desktop is installed, so they leave the banner as it was.
- `setDraft(result.notes.text)`: the draft comes back empty, and the box and the chips adopt it, ready for the next question.
- `else if (result.reason !== 'empty')`: any other failure shows its message from `ASK_ERRORS` as a red toast. Once `empty` is ruled out, TypeScript knows `result.reason` is one of `ASK_ERRORS`'s keys.

`toggleFavorite`

src/renderer/src/views/PanelView.tsx, lines 229–231

```
async function toggleFavorite(app: PortalApp, favorite: boolean) {
  setSettings(await window.assist.settings.setFavoriteApp(app.id, favorite))
}
```

A star click in the Apps list. The main process saves the change with the other preferences and returns the settings; adopting them updates `settings.favoriteApps`, and the list re-orders itself with the starred apps first.

`openApp`

src/renderer/src/views/PanelView.tsx, lines 233–244

```
async function openApp(app: PortalApp) {
  // The tile spins until the browser is opening (JumpCloud can take a few seconds to give the
  // sign-in link); on success the main process closes the panel.
  setOpeningApp(app.id)
  try {
    if (!(await window.assist.apps.open(app.id))) {
      showToast(`Couldn't open ${app.name}. Try it from the portal.`, 'error')
    }
  } finally {
    setOpeningApp((current) => (current === app.id ? null : current))
  }
}
```

A click on an app in the Apps list. Only the app's ID goes to the main process, which asks JumpCloud for the app's sign-in link and opens it in the browser (see Your apps). As the comment says, if it works the main process closes the panel; if there was no link, the panel stays open with a toast, and the portal itself is one click away at the bottom of the list.

- `setOpeningApp(app.id)`: JumpCloud can take a few seconds to give the link, so the app's tile shows a spinner meanwhile, and the user can see the click worked.
- `finally {`: runs whether opening worked, failed or threw, so the spinner never stays on.
- `setOpeningApp((current) => (current === app.id ? null : current))`: passing a function gives the latest value. The spinner is cleared only if it's still this app's: if the user clicked a second app meanwhile, its spinner stays.

`openIssue`, `report`, `disconnectAtlassian`

src/renderer/src/views/PanelView.tsx, lines 246–266

```
/** Opens a Jira issue in the card if connected to Atlassian, otherwise in the browser. */
function openIssue(key: string) {
  setSettingsOpen(false)
  if (atlassianConnected) setIssueKey(key)
  else if (atlassianSite) void window.assist.openExternal(browseUrl(atlassianSite, key))
}

/** Starts an IT request from what's in the text box. */
function report() {
  setSettingsOpen(false)
  setIssueKey(null)
  setServiceDeskTab('new')
  setView('servicedesk')
}

async function disconnectAtlassian() {
  setSettingsOpen(false)
  setIssueKey(null)
  await window.assist.atlassian.signOut()
  showToast('Disconnected from Atlassian')
}
```

- `openIssue(key)`: as its comment says, opens a Jira issue in the card if connected to Atlassian (`setIssueKey(key)`, and `page` below shows an `IssueView` for it), otherwise in the browser. `browseUrl()` (from `src/shared/jira.ts`) makes the issue's address on the company's site. The pages, the notifications and the text box's Open buttons all come here.
- `report()`: **Report to IT** under the text box: as its comment says, starts an IT request from what's in the text box, by showing the service desk page on New request. The form there picks the text up itself.
- `disconnectAtlassian()`: **Disconnect Atlassian** in the settings menu. The main process forgets the connection, and the new `sign-in` state arrives through `onAtlassianState`, so the Jira page shows its Connect button again.

`attachLatest`, `removeAttachment`, `openAttachment`

src/renderer/src/views/PanelView.tsx, lines 268–284

```
async function attachLatest() {
  const result = await window.assist.notes.attachLatestScreenshot()
  if (result.ok) setAttachments(result.notes.attachments)
  else if (result.reason === 'no-screenshots') {
    showToast('No screenshots yet. Take one with the camera button.', 'error')
  } else showToast('No more screenshots to attach', 'error')
}

async function removeAttachment(attachment: Attachment) {
  const notes = await window.assist.notes.removeAttachment(attachment.id)
  setAttachments(notes.attachments)
}

async function openAttachment(attachment: Attachment) {
  const opened = await window.assist.screenshots.open(attachment.path)
  if (!opened) showToast("That screenshot can't be found", 'error')
}
```

The screenshot chips on the draft. Each asks the main process, which owns the draft, and shows the result.

- `window.assist.notes.attachLatestScreenshot()`: the main process adds the newest screenshot that isn't attached yet, so each click goes one further back.
- `'no-screenshots'`: there are none at all, so the toast says how to take one.
- `else showToast('No more screenshots to attach', 'error')`: the only other failure, `no-more`: every screenshot is already attached. Both are shown in red, as errors.
- `window.assist.notes.removeAttachment(attachment.id)`: returns the updated draft, whose attachment list the panel adopts.
- `window.assist.screenshots.open(attachment.path)`: opens the file in the default image viewer; `false` means the file is gone.

copy

src/renderer/src/views/PanelView.tsx, lines 286–289

```
async function copy(text: string) {
  await window.assist.copyText(text)
  showToast('Copied')
}
```

The Copy button on Claude's replies. The main process writes the text to the clipboard (`clipboard.writeText` in src/main/ipc.ts).

signOut, **clearText**, **uninstall**, **newConversation**, **setEffort**, **toggleAutoSend**

src/renderer/src/views/PanelView.tsx, lines 291–324

```
async function signOut() {
  setSettingsOpen(false)
  await window.assist.auth.signOut()
  showToast('Logged out')
}

async function clearText() {
  const notes = await window.assist.notes.clear()
  setDraft(notes.text)
  setAttachments(notes.attachments)
  setSettingsOpen(false)
  showToast('Text box cleared')
}

async function uninstall() {
  setSettingsOpen(false)
  const result = await window.assist.uninstall()
  // On success the uninstaller has started and the app is quitting.
  if (!result.ok) showToast(UNINSTALL_ERRORS[result.reason], 'error')
}

async function newConversation() {
  await window.assist.chat.newConversation()
  setSettingsOpen(false)
  showToast('Started a new conversation')
}

async function setEffort(effort: Effort) {
  setSettings(await window.assist.settings.setEffort(effort))
}

async function toggleAutoSend() {
  setSettings(await window.assist.settings.setAutoSend(!settings.autoSend))
}
```

Settings menu actions.

- `await window.assist.auth.signOut()`: `AuthManager.signOut` forgets the saved sign-in. Because this is a log-out, the main process also starts a new conversation (`onSignedOut` in src/main/index.ts). The new `signed-out` status and the chat reset arrive through `useAssistState`, and the chat card switches to the sign-in.
- `const notes = await window.assist.notes.clear()`: Clear text box, which the menu offers with Claude Desktop instead of New conversation (there's no conversation in the panel to clear). The main process empties the draft and its screenshots and returns it; the box and the chips adopt it.
- `const result = await window.assist.uninstall()`: starts the uninstaller (see `src/main/uninstall.ts` in Main process: startup, IPC and app plumbing). If it worked, the app is already quitting; otherwise the toast from `UNINSTALL_ERRORS` says what to do instead.
- `await window.assist.chat.newConversation()`: `ChatSession` forgets the conversation and broadcasts a reset, which empties the chat in `useAssistState`.
- `setSettings(await window.assist.settings.setEffort(effort))`: saves the response style (Fast, Balanced or Thorough) and shows the settings the main process returns.
- `setSettings(await window.assist.settings.setAutoSend(!settings.autoSend))`: flips "Send in Claude automatically" (Claude Desktop only) and shows the settings the main process returns. The menu stays open, as for the other switch.

`closeSettingsOnOutsideClick`

src/renderer/src/views/PanelView.tsx, lines 326–332

```
// Clicking anywhere else in the panel closes the settings menu.
function closeSettingsOnOutsideClick(event: PointerEvent) {
  if (!settingsOpen || !(event.target instanceof Element)) return
  if (!event.target.closest('[data-settings-menu], [data-action="settings"]')) {
    setSettingsOpen(false)
  }
}
```

Closes the settings menu when you press anywhere else in the panel. The page only receives the press where the window takes the mouse, so "elsewhere" means the chat card or another icon. Clicking outside the panel altogether takes focus away from the window, the main process closes the panel (`BubbleController.panelBlurred`), and the mode effect above closes the menu.

- `event: PointerEvent`: React's own event type, imported from `react`, not the browser's.
- `!(event.target instanceof Element)`: `target` is typed as a general `EventTarget`; this check lets TypeScript allow `closest`.
- `closest('[data-settings-menu], [data-action="settings"]')`: presses inside the menu or on the gear don't count. Without the gear exception, pressing the gear would close the menu on the way down and its click would open it again straight away.

`signInPrompt`, `banner`: sign-in or chat

src/renderer/src/views/PanelView.tsx, lines 334–379

```
const signInPrompt =
  auth?.state === 'unconfigured' ||
  auth?.state === 'signed-out' ||
  auth?.state === 'signing-in' ? (
    <SignInPanel
      open={open}
      status={auth}
      appName={branding.appName}
      onSignIn={() => void window.assist.auth.signIn()}
      onCancel={() => void window.assist.auth.cancel()}
    />
  ) : null

const banner =
  inClaudeDesktop && claudeMissing ? (
    <Banner
      action={
        <button
          type="button"
          onClick={() => void window.assist.openExternal(CLAUDE_DOWNLOAD_PAGE)}
          className="shrink-0 rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100
dark:text-accent-soft dark:hover:bg-zinc-800"
        >
          Download
        </button>
      }
    >
      Claude Desktop isn't installed on this PC, and {branding.appName} opens your questions in
      it. Ask IT to install it, or download it.
    </Banner>
  ) : auth?.state === 'checking' ? (
    <Banner spinner>Checking your JumpCloud sign-in…</Banner>
```

```
  ) : auth?.state === 'offline' ? (
    <Banner
      action={
        <button
          type="button"
          onClick={() => void window.assist.auth.retry()}
          className="rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100 dark:text-accent-
soft dark:hover:bg-zinc-800"
        >
          Retry
        </button>
      }
    >
      {auth.message}
    </Banner>
  ) : null
```

This is how the panel decides what the chat card shows. `signInPrompt` is a `SignInPanel` element or `null`, and `banner` is a `Banner` element or `null`. `ChatBox` shows `signInPrompt` in place of the conversation when it isn't `null`; otherwise it shows the messages, then the banner, then the text box. The sign-in status comes from `AuthManager` in the main process, through `onAuthStatus` in `useAssistState`.

With Claude Desktop, `auth` is `null`: there's never a sign-in prompt, and the only banner is the one saying Claude Desktop isn't installed. With the built-in chat, it depends on `auth.state`:

`auth.state`	The chat card shows
`unconfigured`	Sign-in: "Sign-in isn't set up yet"
`signed-out`	Sign-in: "Sign in with JumpCloud"
`signing-in`	Sign-in: "Finish signing in in your browser", Cancel
`checking`	The chat, "Checking..." banner; sending is off
`offline`	The chat, the reason and Retry; sending works
`signed-in`	The chat

- `auth?.state === 'unconfigured' || auth?.state === 'signed-out' || ...`: `?.` gives `undefined` when `auth` is `null`, so every comparison is false and there's no sign-in prompt. Inside the `? (` branch, TypeScript has narrowed `auth` to those three states, which is exactly the type `SignInPanel`'s `status` prop accepts.
- `onSignIn={() => void window.assist.auth.signIn()}`: `AuthManager.signIn` opens JumpCloud in the browser and returns at once. Progress arrives as status changes (`signing-in`, then `signed-in`), and the main process reopens the panel when it's done.
- `onCancel={() => void window.assist.auth.cancel()}`: stops waiting for the browser.
- `inClaudeDesktop && claudeMissing ? (`: the first banner checked. Claude Desktop isn't installed (found when the panel opened, or when asking), so the banner says so and offers a **Download** button. It goes away the next time the panel opens and finds Claude Desktop, or when a question is handed over successfully.
- `window.assist.openExternal(CLAUDE_DOWNLOAD_PAGE)`: opens Anthropic's download page in the browser (see `src/shared/claudeDesktop.ts` in Shared code and the preload bridge). `shrink-0` stops the long message squeezing the button.
- `{branding.appName}`: the tenant's app name, so the message reads "...and Desktop Assist opens your questions in it."

- `<Banner spinner>`: shown while a saved sign-in is being renewed, at startup or after Retry. The chat is visible, but `signedIn` is false, so nothing can be sent yet.
- `onClick={() => void window.assist.auth.retry()}`: `AuthManager.retry` tries again to renew the saved sign-in.
- `{auth.message}`: the offline reason, written by `AuthManager`.

`settingsIndex`

src/renderer/src/views/PanelView.tsx, line 381

```
const settingsIndex = branding.actions.indexOf('settings')
```

Where the gear is in the tenant's list of icons (`actions` in tenant.json), counting from the bubble. It's `-1` if the tenant doesn't show Settings, and then the menu is never drawn.

`issueKeys`, `unreadNotifications`, `requestNews`, `pageContent`, `page`

src/renderer/src/views/PanelView.tsx, lines 382–458

```
// Only keys of real projects, once the projects are known.
const issueKeys = atlassianSite
  ? findIssueKeys(draft).filter(
      (key) =>
        !atlassianConnected ||
        !projectKeys ||
        projectKeys.includes(key.slice(0, key.lastIndexOf('-'))),
    )
  : []

const unreadNotifications = feed.notifications.items.filter((n) => n.unread).length

const requestNews = feed.requests.items.filter((r) => r.news).length

const pageContent =
  view === 'apps' && branding.portalName ? (
    <AppsList
      open={open}
      state={apps}
      appName={branding.appName}
      portalName={branding.portalName}
      favorites={settings.favoriteApps}
      onToggleFavorite={(app, favorite) => void toggleFavorite(app, favorite)}
      onOpenApp={(app) => void openApp(app)}
      openingId={openingApp}
      onOpenPortal={() => void window.assist.apps.openPortal()}
      onSignIn={() => void window.assist.apps.signIn()}
      onCancelSignIn={() => void window.assist.apps.cancelSignIn()}
      onRetry={() => void window.assist.apps.get(true).then(setApps)}
    />
  ) : view === 'jira' && atlassianSite ? (
    <JiraPage
      open={open}
      state={atlassian}
      feed={feed}
      appName={branding.appName}
      site={atlassianSite}
      onOpenIssue={openIssue}
    />
  ) : view === 'servicedesk' && requestTypes ? (
    <ServiceDeskPage
```

```
      open={open}
      initialTab={serviceDeskTab}
      requestTypes={requestTypes}
      atlassian={atlassianSite ? atlassian : null}
      feed={feed}
      appName={branding.appName}
      draft={draft}
      attachments={attachments}
      onAttachLatest={() => void attachLatest()}
      onOpenAttachment={(attachment) => void openAttachment(attachment)}
      onRemoveAttachment={(attachment) => void removeAttachment(attachment)}
      onOpenIssue={openIssue}
      showToast={showToast}
    />
  ) : null

// An issue opens over the page it was opened from, which stays (hidden) so Back finds it as it
// was: the same tab, search or half-filled form.
const page =
  issueKey || pageContent ? (
    <>
      {issueKey && (
        <IssueView
          key={issueKey}
          issueKey={issueKey}
          inClaudeDesktop={inClaudeDesktop}
          onBack={() => setIssueKey(null)}
          onAskedInChat={() => {
            setIssueKey(null)
            setView('ask')
          }}
          showToast={showToast}
        />
      )}
      <div hidden={issueKey !== null} className="contents">
        {pageContent}
      </div>
    </>
  ) : null
```

What the card shows besides the text box, worked out on every render.

- `issueKeys`: the issue keys in the text box (`findIssueKeys()`, at most three), for its Open buttons. As the comment says, once connected and the project keys are known, only keys of real projects count: `key.slice(0, key.lastIndexOf('-'))` is the project part (`ISD` of `ISD-12`). Until then, every key that looks right is offered. Without an Atlassian site, none.
- `unreadNotifications`, `requestNews`: how many notifications have a dot, and how many requests have news, for the dots on the bell and the service desk icon.
- `pageContent`: the page for `view`: an `AppsList` (see Your apps), a `JiraPage` or a `ServiceDeskPage` (see The Jira and service desk pages); `null` for the text box. Each is only shown when the tenant has what it needs (`portalName`, `atlassianSite`, `requestTypes`), which also gives TypeScript definite values for their props.
- `openingId={openingApp}`: which tile of the Apps list shows the "opening" spinner.
- `favorites={settings.favoriteApps}`, `onToggleFavorite={...}`: the starred apps, from the panel's own copy of the settings, and what a star click does (`toggleFavorite`, above).
- `onRetry={() => void window.assist.apps.get(true).then(setApps)}`: the list's Retry and Refresh load it again (`true` means "even if already loaded").

- `atlassian={atlassianSite ? atlassian : null}`: the service desk page gets the Atlassian connection only when there's a site; without one it shows just the form.
- `draft={draft}`, `attachments={attachments}`: the service desk form starts from the text box, and shares its screenshots, with the same attach, open and remove handlers.
- `page`: `pageContent`, with an issue over it when one is open. As the comment says, the page the issue was opened from stays, hidden, so Back finds it as it was: the same tab, search or half-filled form. `<div hidden={issueKey !== null} className="contents">` hides it without removing it; `contents` makes the wrapper itself take no space, so the page lays out as if it weren't there.
- `<IssueView key={issueKey} ... />`: `key` makes React create a new `IssueView` for each issue, rather than reusing one with the old issue's state. `onAskedInChat` closes the issue and shows the text box, where the built-in chat's answer arrives.
- `<>` and `</>`: a *fragment*, which groups the two without adding an element of its own.

The UI components

These seven files draw almost everything that appears around the bubble when you click it. `Panel` in `PanelView.tsx` (see The pages) renders three of them directly: the `ActionStack` of round icons in a column beside the bubble, the `ChatBox` card next to the bubble on the side facing the middle of the screen, and the `SettingsMenu`, which opens beside the gear icon. Everything else lives inside the chat card: `SignInPanel` replaces the chat while nobody is signed in, `MessageList` shows the conversation (using `Markdown` for Claude's replies), and an `AttachmentChip` above the text box stands for each attached screenshot. When questions go to Claude Desktop (the tenant's `chatApp`), there's no sign-in and no conversation in the panel, so the card is just the text box, with an "Ask in Claude" button. The card can show the Apps list instead, either from the start or behind an icon, depending on the tenant; that component, `AppsList`, is covered in Your apps. It can also show the Jira page, the service desk page or a Jira issue, which are in The Jira and service desk pages.

`src/renderer/src/components/ChatBox.tsx`: the chat card

The card beside the bubble, with the text box at its bottom. `Panel` renders one `ChatBox` and passes it everything it shows: the bubble's `corner`, whether the panel is `open`, the current `toast`, the conversation (`messages`), the `draft` and its `attachments`, whether Claude is replying (`busy`), whether Send is allowed (`canSend`), whether questions go to Claude Desktop (`inClaudeDesktop`) and, if so, whether they're sent there automatically (`autoSend`). `Panel` also builds the optional pieces that go inside it: a page to show instead (`page`), the sign-in (`signInPrompt`) and a status line (`banner`). The `on…` props are what the buttons and the text box call; `issueKeys`, `onOpenIssue` and `onReport` are for Jira and the service desk. The file has three components: `ChatBox` (the card), `Composer` (the text box, private to this file) and `Banner` (exported, used by `Panel`).

`ChatBox`

src/renderer/src/components/ChatBox.tsx, lines 19–111

```
/**
 * The card beside the bubble, on the side facing the middle of the screen. It's anchored at the
 * bubble's edge (bottom in a bottom corner, top in a top corner), so it starts as just the text
 * box and grows away from that edge as the conversation gets longer, then scrolls. When nobody is
 * signed in, `signInPrompt` is shown instead of the chat. With `inClaudeDesktop`, it's only the
 * text box, and sending opens the question in Claude Desktop. While a page's icon is on (Apps,
 * Jira, the service desk), the page shows instead.
 */
export function ChatBox(props: {
  corner: Corner
  open: boolean
  toast: ToastMessage | null
  /** Sending hands the question to Claude Desktop instead of the chat in the panel. */
  inClaudeDesktop: boolean
  /** With Claude Desktop: the question (and screenshot) is sent there for the user. */
  autoSend: boolean
  /** Replaces the text box with a page while its icon is on (the Apps list, Jira), or null. */
  page: ReactNode
  /** Replaces the chat (the JumpCloud sign-in), or null to show the chat. */
  signInPrompt: ReactNode
  /** A line above the text box, e.g. "Checking your sign-in…". */
  banner: ReactNode
  messages: ChatMessage[]
  busy: boolean
  canSend: boolean
```

```
  draft: string
  attachments: Attachment[]
  onDraftChange: (text: string) => void
  onSend: () => void
  onStop: () => void
  onRetry: () => void
  onCopy: (text: string) => void
  onAttachLatest: () => void
  onOpenAttachment: (attachment: Attachment) => void
  onRemoveAttachment: (attachment: Attachment) => void
  /** Jira issue keys in the text, each offered as a chip that opens the issue. */
  issueKeys: string[]
  onOpenIssue: (key: string) => void
  /** Turns the text (and screenshots) into an IT request; null without a service desk form. */
  onReport: (() => void) | null
}) {
  return (
    <div
      data-hit
      className={cn(
        'absolute flex flex-col rounded-2xl border shadow-xl',
        'border-black/10 bg-white text-zinc-900 dark:border-white/10 dark:bg-zinc-900 dark:text-zinc-
100',
        originClass(props.corner),
        'scale-95 opacity-0 transition duration-150 ease-out',
        'group-data-open:scale-100 group-data-open:opacity-100',
        'motion-reduce:transition-none',
      )}
      style={{
        ...anchored(props.corner, PANEL_LAYOUT.chatX, PANEL_LAYOUT.chatY),
        width: UI.panelWidth,
        maxHeight: UI.panelMaxHeight,
      }}
    >
      {props.toast && (
        <div
          key={props.toast.id}
          role="status"
          className={cn(
            'absolute rounded-full px-3 py-1.5 text-xs font-medium shadow-lg',
            // Just outside the card, on the side away from the screen edge.
            isTopCorner(props.corner) ? 'top-full mt-2' : 'bottom-full mb-2',
            isLeftCorner(props.corner) ? 'left-0' : 'right-0',
            props.toast.tone === 'error'
              ? 'bg-red-600 text-white'
              : 'bg-zinc-900 text-white dark:bg-white dark:text-zinc-900',
          )}
        >
          {props.toast.text}
        </div>
      )}

      {props.page ?? props.signInPrompt ?? (
        <>
          {props.messages.length > 0 && (
            <MessageList
              messages={props.messages}
              onRetry={props.onRetry}
              onCopy={props.onCopy}
              onOpenScreenshot={props.onOpenAttachment}
            />
          )}
          {props.banner}
          <Composer {...props} />
        </>
```

```
      )}
    </div>
  )
}
```

`ChatBox` draws the card and places it in the panel window. Inside it shows a toast (if there is one), then either the sign-in or the chat: the conversation, the banner and the text box. It needs a few ideas that the rest of this page relies on.

Components, props and JSX. A React component is a function that returns a description of the UI. React calls `ChatBox(props)` again whenever its inputs change. The HTML-like `<div …>` it returns is JSX: the build turns each tag into a function call that creates a lightweight description, and React then updates the real page (the DOM) to match. Lowercase tags (`div`, `button`) are HTML elements; capitalised ones (`MessageList`, `Composer`) are other components. The attributes written on a component's tag become its *props*, the single object it receives; the type after `props:` lists every prop it accepts. Inside JSX, curly braces `{…}` switch back to ordinary TypeScript. `className` is JSX's name for HTML's `class`.

Callbacks. The `on…` props are functions. The card never talks to the main process itself: when you press Send it calls `props.onSend()`, and `Panel` does the work (see `send()` in The pages). That keeps every decision in one place.

Tailwind classes and `cn()`. These components have almost no CSS of their own. Each element lists small *utility classes* that each do one thing: `absolute` is `position: absolute`, `rounded-2xl` rounds the corners, `px-3` sets the left and right padding, `text-xs` is small text. Tailwind generates CSS for just the classes the code uses. A prefix makes a class conditional: `hover:` applies while the pointer is over the element, `dark:` while the app is in dark mode, `disabled:` while a button is disabled. `bg-accent` and `text-accent` use the tenant's colour, which `styles.css` defines and `useAccentColor` sets at runtime (see The pages). `cn()` (from `lib/cn.ts`) joins class strings and drops `false`, `null` and `undefined`, which is what lets you write `condition && 'some-class'` inside it.

`data-hit`. The panel window is a large transparent rectangle that normally lets clicks fall through to whatever is underneath. `useClickThrough` (see The pages) switches mouse input on only while the pointer is over an element that is, or is inside, one marked `data-hit`. So every piece of real UI must carry it. Here it's the whole card, which covers everything inside it; in `SettingsMenu` it's the menu, and in `ActionStack` each icon.

How the layout mirrors for each corner. The bubble can rest in any corner of the screen, and the panel always opens toward the middle. The panel window shares its outer corner with the bubble's window (`panelWindowBounds` in The bubble), so the components measure their positions from that corner. `PANEL_LAYOUT` in `src/shared/geometry.ts` gives each piece's distance from it, `X` across and `Y` away from the screen edge (see Shared code and the preload bridge). `anchored(corner, x, y)` from `lib/anchor.ts` turns those two numbers into CSS: `right` and `bottom` in the bottom-right corner, `left` and `top` in the top-left, and so on. So the same numbers work in all four corners, and a component only has to check the corner where something must change direction (the toast here, the order of the icons in `ActionStack`). In the bottom-right corner it looks roughly like this; the other corners are mirror images:

```
                                  [close]
                                  [bounce]
+-----------------------------+   [settings]    <- SettingsMenu opens leftward, over the card
| MessageList                 |   [screenshot]
| Banner (only sometimes)     |
| Composer                    |  ( bubble )
+-----------------------------+
```

The parts of `ChatBox` itself:

- `inClaudeDesktop: boolean`: `ChatBox` only passes it on to `Composer`, which changes the send button. The rest of the card needs no change: with Claude Desktop, `Panel` never passes a sign-in prompt or any messages, so the card is just the text box (and sometimes a banner).
- `autoSend: boolean`: the "Send in Claude automatically" setting, also only for `Composer`'s wording.
- `page: ReactNode`: as the comment says, a page that replaces the text box while its icon is on (the Apps list, the Jira page, the service desk page, or an issue over one of them), built by `Panel` like `signInPrompt`; otherwise `null`.
- `originClass(props.corner)`: sets the CSS `transform-origin` to the bubble's corner (for example `origin-bottom-right`), so the zoom-in animation grows out of the bubble.
- `'scale-95 opacity-0 transition duration-150 ease-out'`: the card is always rendered, even while the panel is closed. It's just invisible and slightly shrunk.
- `'group-data-open:scale-100 group-data-open:opacity-100'`: `Panel`'s outer `<div>` has the class `group`, and a `data-open` attribute while the panel is open. `group-data-open:` means "when an ancestor marked `group` has `data-open`". So opening animates the card in over 150 ms, and closing animates it out; the main process waits 140 ms (`PANEL_FADE_MS` in `BubbleController.ts`) before hiding the window, so the fade-out can be seen.
- `'motion-reduce:transition-none'`: when Windows is set to show fewer animations (the system's reduced-motion setting), the card appears and disappears instantly.
- `...anchored(props.corner, PANEL_LAYOUT.chatX, PANEL_LAYOUT.chatY)`: `style` takes an object of CSS properties, and plain numbers mean pixels. `chatX` is 76 (the 8 px padding around the bubble, the 56 px bubble and a 12 px gap), so the card starts just past the bubble. `chatY` is 8, so the card's near edge is level with the bubble's.
- `maxHeight: UI.panelMaxHeight`: nothing sets a height. The card is pinned at the bubble's edge and grows away from it as content is added, up to 520 px. After that the message list scrolls.
- `{props.toast && (`: the part after `&&` is rendered only when the left side is truthy, so with no toast nothing is drawn. Toasts come from `useToast` in `Panel` and vanish after 2.6 seconds.
- `key={props.toast.id}`: a `key` tells React which element is which. Each toast has a new id, so React puts in a fresh element for each new message instead of editing the old one's text.
- `role="status"`: screen readers read the toast out without moving the keyboard focus.
- `isTopCorner(props.corner) ? 'top-full mt-2' : 'bottom-full mb-2'`: the toast sits just outside the card, on the side facing the middle of the screen: above the card in a bottom corner, below it in a top corner. The panel window has 40 px spare beyond the card's full height (`UI.toastRoom`), so the toast always fits.
- `isLeftCorner(props.corner) ? 'left-0' : 'right-0'`: lines the toast up with the card's edge nearest the bubble.
- `{props.page ?? props.signInPrompt ?? (`: `??` uses the left side unless it's `null` or `undefined`, so the first of the three that isn't `null` is shown: a page if one is on, otherwise the sign-in if nobody is signed in, otherwise the chat. Never more than one. The toast above stays outside this choice, so it shows over the pages too.
- `<>`: a *fragment*. It groups several elements without adding an extra `<div>` to the page.
- `props.messages.length > 0 &&`: before the first message there's no list, so the card is just the text box.
- `{props.banner}`: whatever `Banner` `Panel` built (or `null`), between the conversation and the text box.
- `<Composer {...props} />`: `{...props}` passes every one of `ChatBox`'s props on to `Composer` unchanged.

Composer

src/renderer/src/components/ChatBox.tsx, lines 113–254

```
/** The text box: attached screenshots, the text, and the attach / send / stop buttons. */
function Composer(props: Parameters<typeof ChatBox>[0]) {
  const textarea = useRef<HTMLTextAreaElement>(null)
  const placedCaret = useRef(false)

  // Focus the text box whenever the panel opens; the first time, put the caret at the end.
  useEffect(() => {
    const el = textarea.current
    if (!props.open || !el) return
    el.focus()
    if (!placedCaret.current) {
      placedCaret.current = true
      el.setSelectionRange(el.value.length, el.value.length)
    }
  }, [props.open])

  // Enter sends; Shift+Enter starts a new line.
  function onKeyDown(event: KeyboardEvent<HTMLTextAreaElement>) {
    if (event.key === 'Enter' && !event.shiftKey && !event.nativeEvent.isComposing) {
      event.preventDefault()
      if (props.canSend) props.onSend()
    }
  }

  return (
    <div
      className={cn(
        'shrink-0',
        props.messages.length > 0 && 'border-t border-black/5 dark:border-white/10',
      )}
    >
      {props.attachments.length > 0 && (
        <div className="flex gap-2 overflow-x-auto px-3 pt-3">
          {props.attachments.map((attachment) => (
            <AttachmentChip
              key={attachment.id}
              attachment={attachment}
              onOpen={() => props.onOpenAttachment(attachment)}
              onRemove={() => props.onRemoveAttachment(attachment)}
            />
          ))}
        </div>
      )}
      {props.inClaudeDesktop && props.attachments.length > 0 && (
        <p className="px-3.5 pt-2 text-[11px] text-zinc-500">
          {props.attachments.length === 1 ? 'The screenshot' : 'The screenshots'}
          {props.autoSend && props.draft.trim()
            ? ' will be pasted into Claude for you.'
            : ' will be copied for you to paste into Claude with Ctrl+V.'}
        </p>
      )}

      {props.issueKeys.length > 0 && (
        <div className="flex flex-wrap gap-1.5 px-3 pt-2.5">
          {props.issueKeys.map((key) => (
            <button
              key={key}
              type="button"
              onClick={() => props.onOpenIssue(key)}
              title={`Open ${key}`}
```

```
              className="inline-flex items-center gap-1 rounded-full border border-black/10 px-2 py-0.5
text-[11px] font-medium text-zinc-700 hover:bg-zinc-100 dark:border-white/10 dark:text-zinc-200
dark:hover:bg-zinc-800"
            >
              <Ticket size={12} className="text-accent dark:text-accent-soft" aria-hidden />
              Open {key}
            </button>
          ))}
        </div>
      )}

      <textarea
        ref={textarea}
        value={props.draft}
        onChange={(event) => props.onDraftChange(event.target.value)}
        onKeyDown={onKeyDown}
        placeholder="Ask anything…"
        aria-label="Message Claude"
        spellCheck
        className={cn(
          'block max-h-40 min-h-11 w-full resize-none overflow-y-auto bg-transparent field-sizing-
content',
          'px-3.5 pt-3 pb-1 text-sm leading-relaxed outline-none placeholder:text-zinc-400',
        )}
      />

      <div className="flex items-center gap-1 px-2 pb-2">
        <button
          type="button"
          onClick={props.onAttachLatest}
          title="Attach the latest screenshot (click again for the one before)"
          className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-zinc-
600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
        >
          <Paperclip size={14} aria-hidden />
          Attach screenshot
        </button>
        {props.onReport && (
          <button
            type="button"
            onClick={props.onReport}
            title="Report a problem to IT, with this text and screenshot"
            className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-
zinc-600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
          >
            <LifeBuoy size={14} aria-hidden />
            Report to IT
          </button>
        )}
        <span className="flex-1" />
        {props.busy ? (
          <button
            type="button"
            onClick={props.onStop}
            aria-label="Stop"
            title="Stop"
            className="grid size-8 place-items-center rounded-full bg-zinc-900 text-white hover:bg-zinc-
700 dark:bg-white dark:text-zinc-900"
          >
            <Square size={12} fill="currentColor" aria-hidden />
          </button>
        ) : props.inClaudeDesktop ? (
          <button
```

```
            type="button"
            onClick={props.onSend}
            disabled={!props.canSend}
            title={props.autoSend ? 'Send in Claude (Enter)' : 'Open in Claude (Enter)'}
            className="inline-flex h-8 items-center gap-1 rounded-full bg-accent pr-2.5 pl-3 text-xs
font-medium text-white disabled:opacity-40"
          >
            Ask in Claude
            <ArrowUpRight size={14} aria-hidden />
          </button>
        ) : (
          <button
            type="button"
            onClick={props.onSend}
            disabled={!props.canSend}
            aria-label="Send"
            title="Send (Enter)"
            className="grid size-8 place-items-center rounded-full bg-accent text-white disabled:opacity-
40"
          >
            <ArrowUp size={16} aria-hidden />
          </button>
        )}
      </div>
    </div>
  )
}
```

The text box at the bottom of the card: the attached screenshots, the text itself, and the buttons under it. It's a separate component so it can have its own refs and effect; only `ChatBox` uses it.

Refs. React runs a component's function again on every render, so its ordinary local variables start afresh each time. `useRef(initial)` gives a small box, `.current`, that keeps its value between renders. Changing it doesn't redraw anything. `Composer` has two: `textarea` is attached to the `<textarea>` element with `ref={textarea}`, so React puts the real DOM element in `textarea.current` (needed to call `.focus()`); `placedCaret` is just a remembered flag.

Effects. `useEffect(fn, [deps])` runs `fn` after React has updated the page, and again only when a value in the dependency list `[deps]` has changed. It also runs once when the component first appears. Effects are for work outside drawing the UI, such as moving the keyboard focus.

- `function Composer(props: Parameters<typeof ChatBox>[0])`: `Parameters<typeof ChatBox>` is TypeScript for "the types of `ChatBox`'s arguments", and `[0]` is its props object. So `Composer` takes exactly the same props without repeating the long list.
- `}, [props.open])`: the effect runs when the panel opens or closes; the early `return` skips the close. On open it focuses the text box so you can type straight away. Because it also runs when `Composer` first appears, the box is focused after you sign in with the panel open.
- `el.setSelectionRange(el.value.length, el.value.length)`: only the first time, puts the caret at the end, since the draft may have been restored from the last session. After that the caret stays wherever you left it.
- `!event.nativeEvent.isComposing`: while typing with an input method (for example for Japanese or Chinese), Enter confirms the chosen characters. This check stops that Enter from sending.
- `event.preventDefault()`: stops Enter adding a new line. Enter never adds a line, even when the message can't be sent yet (for example while Claude is replying); Shift+Enter skips this whole `if` and adds one.
- `{props.attachments.map((attachment) => (`: `.map` turns each attachment into an `AttachmentChip`. The arrow functions passed as `onOpen` and `onRemove` remember which attachment the chip is for.

- `key={attachment.id}`: every item in a list needs a `key` that stays the same between renders, so React can tell the items apart when one is removed.
- `overflow-x-auto`: with many attachments, the row of chips scrolls sideways.
- `{props.inClaudeDesktop && props.attachments.length > 0 && (`: with Claude Desktop, a small note under the chips says what will happen to the screenshots. A link to Claude Desktop can carry text but not images, so they go on the clipboard to be pasted (see Claude Desktop). It says "The screenshot" or "The screenshots" to match how many there are.
- `props.autoSend && props.draft.trim() ? ... : ...`: with "Send in Claude automatically" on and a question typed, Desktop Assist pastes them itself ("will be pasted into Claude for you"). Otherwise the user pastes them with Ctrl+V. A screenshot without a question is never pasted for the user, so the second wording also covers that.
- `{props.issueKeys.length > 0 && (`: a button for each Jira issue key in the text, which `Panel` finds, such as **Open ISD-1234** with a ticket icon (`Ticket`). Clicking one opens the issue in the card (`Panel.openIssue`), or in the browser when not connected to Atlassian. `flex-wrap` lets several wrap onto more lines.
- `value={props.draft}`: a *controlled* text box. What it shows always comes from the `draft` prop, not from the element's own memory.
- `onChange={(event) => props.onDraftChange(event.target.value)}`: React's `onChange` fires on every keystroke. It calls `Panel`'s `changeDraft`, which updates `draft` and sends the text to the main process to be saved (`NotesStore.setText`; see The draft, screenshots and safe files).
- `field-sizing-content`: CSS `field-sizing: content` makes the text box grow with its text, from `min-h-11` (44 px) to `max-h-40` (160 px), after which it scrolls.
- `spellCheck`: turns on Chromium's spell checker. Right-clicking a misspelt word shows suggestions in the menu built by `addEditContextMenu` in `src/main/bubble/windows.ts`.
- `<Paperclip size={14} aria-hidden />`: the icons come from lucide-react, where every icon is a small React component that draws an SVG. `size` sets its width and height in pixels. `aria-hidden` hides it from screen readers, since the button's text or `aria-label` already says what it does.
- `onClick={props.onAttachLatest}`: **Attach screenshot**. `Panel.attachLatest` adds the newest screenshot to the draft, or shows a toast saying why it couldn't; the tooltip says it's the latest one.
- `{props.onReport && (`: **Report to IT**, with a lifebuoy icon (`LifeBuoy`), only when the tenant lists request types (`Panel` passes `null` otherwise). It opens the service desk page on New request, which starts from the text box and its screenshots (`Panel.report`).
- `<span className="flex-1" />`: an empty element that takes up the spare room, pushing the send button to the right-hand end of the row.
- `props.busy ? (`: while Claude is replying, Send is replaced by Stop. `Panel` sets `busy` when any message is still `streaming`, which only happens in the built-in chat.
- `fill="currentColor"`: lucide icons are outlines; this fills the square with the text colour so it reads as a solid "stop" symbol.
- `) : props.inClaudeDesktop ? (`: a second condition, chained on the first. Not busy and handing questions to Claude Desktop: the button is a pill that says **Ask in Claude**, with an arrow pointing up and out (`ArrowUpRight`), since the question opens in another app. Otherwise it's the round Send arrow (`ArrowUp`). Enter does the same as either button.
- `title={props.autoSend ? 'Send in Claude (Enter)' : 'Open in Claude (Enter)'}`: the tooltip says whether the question will be sent in Claude or only opened there. The Ask button has no `aria-label`, because its visible text already names it; the round Send button has only an icon, so it needs one.

- `disabled={!props.canSend}`: `Panel` allows sending only when you're signed in (always, with Claude Desktop), Claude isn't replying, no question is already being handed to Claude Desktop, and there's text or a screenshot. `disabled:opacity-40` fades the button out.

Banner

src/renderer/src/components/ChatBox.tsx, lines 256–265

```
/** A one-line status above the text box, with an optional action. */
export function Banner(props: { spinner?: boolean; children: ReactNode; action?: ReactNode }) {
  return (
    <div className="flex shrink-0 items-center gap-2 border-t border-black/5 px-3.5 py-2 text-xs text-
zinc-500 dark:border-white/10">
      {props.spinner && <LoaderCircle size={12} className="animate-spin" aria-hidden />}
      <span className="flex-1">{props.children}</span>
      {props.action}
    </div>
  )
}
```

A one-line status strip between the conversation and the text box. `Panel` uses it in three cases: "Checking your JumpCloud sign-in..." with a spinner while a saved sign-in is renewed at startup, the "couldn't reach JumpCloud" message with a Retry button (see JumpCloud sign-in), and, with Claude Desktop, "Claude Desktop isn't installed on this PC..." with a Download button.

- `spinner?: boolean`: the `?` makes the prop optional. Writing `<Banner spinner>` with no value means `spinner={true}`.
- `children: ReactNode`: `children` is a special prop holding whatever is written between `<Banner>` and `</Banner>`. `ReactNode` is the type for anything React can display: text, elements, or `null`.
- `action?: ReactNode`: an optional element at the right-hand end, such as `Panel`'s Retry or Download button.
- `animate-spin`: Tailwind's spinning animation, applied to lucide's `LoaderCircle` icon.
- `flex-1`: the text takes all the spare width, which pushes the action to the right.

`src/renderer/src/components/MessageList.tsx`: the conversation

The scrolling list of messages at the top of the chat card. `ChatBox` renders it once there's at least one message. Its props are `messages` (the chat from the app state, kept up to date by `useAssistState`; see The pages), and `onRetry` and `onCopy` from `Panel`. Your messages and Claude's are drawn by different components.

MessageList

src/renderer/src/components/MessageList.tsx, lines 8–66

```
/** Within this many pixels of the bottom counts as "at the bottom". */
const STICK_THRESHOLD = 40

/**
 * The conversation. It follows new text as it streams in, unless you've scrolled up to read
 * something earlier. Sending a message always brings it back to the bottom.
 */
export function MessageList(props: {
  messages: ChatMessage[]
  onRetry: () => void
  onCopy: (text: string) => void
```

```
  onOpenScreenshot: (attachment: Attachment) => void
}) {
  const scroller = useRef<HTMLDivElement>(null)
  const stickToBottom = useRef(true)
  const lastSentId = useRef<string | undefined>(undefined)

  useLayoutEffect(() => {
    const el = scroller.current
    if (!el) return
    // A message you've just sent: follow it and the reply, even if you'd scrolled up.
    const sentId = props.messages.findLast((m) => m.role === 'user')?.id
    if (sentId !== lastSentId.current) {
      lastSentId.current = sentId
      stickToBottom.current = true
    }
    if (stickToBottom.current) el.scrollTop = el.scrollHeight
  }, [props.messages])

  return (
    <div
      ref={scroller}
      onScroll={(event) => {
        const el = event.currentTarget
        stickToBottom.current = el.scrollHeight - el.scrollTop - el.clientHeight < STICK_THRESHOLD
      }}
      className="min-h-0 flex-1 space-y-3 overflow-y-auto px-3.5 pt-3.5 pb-1"
      aria-live="polite"
    >
      {props.messages.map((message, index) =>
        message.role === 'user' ? (
          <UserMessage
            key={message.id}
            message={message}
            onOpenScreenshot={props.onOpenScreenshot}
          />
        ) : (
          <AssistantMessage
            key={message.id}
            message={message}
            canRetry={index === props.messages.length - 1}
            onRetry={props.onRetry}
            onCopy={props.onCopy}
          />
        ),
      )}
    </div>
  )
}
```

Draws each message, and keeps the view scrolled to the bottom as a reply streams in. If you scroll up to read something earlier it stops following; scrolling back to the bottom, or sending a message, turns following on again.

- `const stickToBottom = useRef(true)`: whether to follow new text. It's a ref rather than state because changing it doesn't need anything redrawn.
- `const lastSentId = useRef<string | undefined>(undefined)`: the id of the newest message you sent, as of the last update, so the effect can tell when you've just sent another.
- `useLayoutEffect(() => {`: like `useEffect`, but it runs after React has changed the page and before the browser paints it. Scrolling there means you never see a frame with the new text hidden below the visible area.
- `}, [props.messages])`: runs whenever the list changes. `useAssistState` makes a new array for every update, including each batch of streamed text (at most one every 50 ms), so this runs as the reply grows.

- `const sentId = props.messages.findLast((m) => m.role === 'user')?.id`: the newest message you sent. When it differs from last time you've just sent one, so following is switched back on: your message and Claude's reply come into view even if you'd scrolled up to read something.
- `el.scrollTop = el.scrollHeight`: scrolls to the bottom. The browser clamps the value to the furthest it can actually scroll.
- `onScroll={(event) => {`: on every scroll, works out how far the view is from the bottom (`scrollHeight - scrollTop - clientHeight`) and follows only if that's under `STICK_THRESHOLD` (40 px). The effect's own scrolling lands at 0, so following stays on.
- `min-h-0 flex-1 overflow-y-auto`: the list takes whatever height the card has left after the text box, and scrolls. `min-h-0` matters because an item in a flex column normally refuses to shrink below its content's height, which would push the text box out of the card.
- `aria-live="polite"`: screen readers announce new text in the list when they're not busy.
- `message.role === 'user' ? (`: chooses the component by who sent the message.
- `key={message.id}`: each message's id from `ChatSession` is its list key.
- `canRetry={index === props.messages.length - 1}`: only the last message may offer Retry, since `ChatSession.retry()` asks again for the latest reply (see Talking to Claude).

`UserMessage`

`src/renderer/src/components/MessageList.tsx`, lines 68–95

```
function UserMessage({
  message,
  onOpenScreenshot,
}: {
  message: ChatMessage
  onOpenScreenshot: (attachment: Attachment) => void
}) {
  return (
    <div className="ml-auto flex max-w-[85%] flex-col items-end gap-1.5">
      {message.attachments.length > 0 && (
        <div className="flex flex-wrap justify-end gap-1.5">
          {message.attachments.map((attachment) => (
            <SentScreenshot
              key={attachment.id}
              attachment={attachment}
              onOpen={() => onOpenScreenshot(attachment)}
            />
          ))}
        </div>
      )}
      {message.text && (
        <div className="rounded-2xl rounded-br-md bg-accent px-3 py-2 text-sm whitespace-pre-wrap text-
white select-text">
          {message.text}
        </div>
      )}
    </div>
  )
}
```

Your own message, on the right: any screenshots, then the text in an accent-coloured bubble.

- `function UserMessage({`: the braces in the parameter list pull `message` and `onOpenScreenshot` straight out of the props object (*destructuring*).

- `ml-auto`: with `items-end`, pushes the message to the right. `max-w-[85%]` is Tailwind's syntax for a one-off value; it keeps the message from spanning the whole card.
- `{message.text && (`: a message can be screenshots only, with no text.
- `whitespace-pre-wrap`: keeps the line breaks you typed with Shift+Enter. Your text is shown exactly as typed, not as Markdown.
- `select-text`: `styles.css` turns text selection off for the whole page (so the UI doesn't feel like a web page); this turns it back on so you can select and copy your own message.
- `rounded-br-md`: a smaller bottom-right corner, the usual chat-bubble "tail".

SentScreenshot

`src/renderer/src/components/MessageList.tsx`, lines 97–110

```
/** A sent screenshot's thumbnail. Opening it goes through the panel, which says if it's gone. */
function SentScreenshot({ attachment, onOpen }: { attachment: Attachment; onOpen: () => void }) {
  const thumbnail = useThumbnail(attachment.path)
  return (
    <button
      type="button"
      title={attachment.fileName}
      onClick={onOpen}
      className="h-12 w-20 overflow-hidden rounded-lg border border-black/10 bg-zinc-200 dark:border-
white/10 dark:bg-zinc-700"
    >
      {thumbnail && <img src={thumbnail} alt="" className="size-full object-cover" />}
    </button>
  )
}
```

A small thumbnail of a screenshot sent with a message. Clicking it opens the file in your image viewer.

- `useThumbnail(attachment.path)`: a hook (see The pages) that asks the main process for a small preview. It returns `undefined` while loading, `null` if the file is gone, or a `data:` URL to use as the image. Until there's an image the button is an empty grey box.
- `onClick={onOpen}`: opening goes through the panel. `ChatBox` passes the same handler the draft chips use, `Panel.openAttachment` (see The pages), which asks the main process to open the file and shows "That screenshot can't be found" if it has been deleted.
- `alt=""`: the image is decorative; the button's `title` tooltip shows the file name.
- `object-cover`: fills the 80 × 48 px box, cropping the screenshot rather than squashing it.

AssistantMessage

`src/renderer/src/components/MessageList.tsx`, lines 112–160

```
function AssistantMessage(props: {
  message: ChatMessage
  canRetry: boolean
  onRetry: () => void
  onCopy: (text: string) => void
}) {
  const { message } = props
  const waiting = message.status === 'streaming' && !message.text

  return (
    <div className="group/message max-w-full text-sm text-zinc-800 dark:text-zinc-100">
```

```
      {waiting ? <Thinking /> : message.text && <Markdown text={message.text} />}

      {(message.notice || message.status === 'done') && (
        <div className="mt-1 flex items-center gap-2 text-xs">
          {message.notice && (
            <span
              className={cn(
                message.status === 'error' ? 'text-red-600 dark:text-red-400' : 'text-zinc-500',
              )}
            >
              {message.notice}
            </span>
          )}
          {message.retryable && props.canRetry && (
            <button
              type="button"
              onClick={props.onRetry}
              className="inline-flex items-center gap-1 rounded-md px-1.5 py-0.5 font-medium text-accent
hover:bg-zinc-100 dark:text-accent-soft dark:hover:bg-zinc-800"
            >
              <RotateCcw size={12} aria-hidden /> Retry
            </button>
          )}
          {message.text && message.status !== 'streaming' && (
            <button
              type="button"
              aria-label="Copy reply"
              title="Copy reply"
              onClick={() => props.onCopy(message.text)}
              className="rounded-md p-1 text-zinc-400 opacity-0 group-hover/message:opacity-100 hover:bg-
zinc-100 hover:text-zinc-700 focus-visible:opacity-100 dark:hover:bg-zinc-800 dark:hover:text-zinc-200"
            >
              <Copy size={12} aria-hidden />
            </button>
          )}
        </div>
      )}
    </div>
  )
}
```

Claude's reply: full width, with no bubble, rendered as Markdown. Below it is a small row with any notice, a Retry button for a failed reply, and Copy.

- `const { message } = props`: destructuring again, so the code can say `message` rather than `props.message`.
- `const waiting = message.status === 'streaming' && !message.text`: the reply has started but no text has arrived yet (Claude may think for a while first).
- `{waiting ? <Thinking /> : message.text && <Markdown text={message.text} />}`: the `? :` picks one of the two. As each chunk of text arrives the whole reply is rendered as Markdown again.
- `(message.notice || message.status === 'done') &&`: the footer row appears once the reply has finished (`done`), or when there's a notice. `ChatSession` gives every stopped or failed reply a notice ("Stopped.", or the error message), so in practice the row shows for every finished reply and never while text is streaming.
- `message.status === 'error' ? 'text-red-600 dark:text-red-400' : 'text-zinc-500'`: errors in red; other notices (stopped, refused, cut off at the length limit) in grey.
- `message.retryable && props.canRetry`: Retry only for a failed reply that `ChatSession` marked retryable (a temporary error, with no text kept from it), and only on the last message. It calls `window.assist.chat.retry()` through `Panel`.
- `message.text && message.status !== 'streaming'`: Copy appears once there's a finished text to copy.

- `group/message`: a *named group*. `group-hover/message:opacity-100` shows the Copy button (normally `opacity-0`) only while the pointer is over this message. The name keeps it separate from `Panel`'s `group`, which wraps the whole panel. `focus-visible:opacity-100` shows it when you reach it with Tab.
- `onClick={() => props.onCopy(message.text)}`: copies the reply's Markdown source, not the formatted text. `Panel.copy` puts it on the clipboard (through the main process) and shows a "Copied" toast.

`Thinking`

`src/renderer/src/components/MessageList.tsx`, lines 162–177

```
function Thinking() {
  return (
    <div className="flex items-center gap-1.5 py-1 text-xs text-zinc-500" role="status">
      <span className="flex gap-0.5" aria-hidden>
        {[0, 150, 300].map((delay) => (
          <span
            key={delay}
            className="size-1.5 animate-bounce rounded-full bg-zinc-400"
            style={{ animationDelay: `${delay}ms` }}
          />
        ))}
      </span>
      Thinking…
    </div>
  )
}
```

Three bouncing dots and the word "Thinking…", shown until the first text of a reply arrives.

- `[0, 150, 300].map((delay) => (`: three dots, each starting its `animate-bounce` 150 ms after the one before, so they ripple.
- `animationDelay`: set as an inline style because Tailwind only generates classes it finds written out in full in the source, so it can't build one from a variable.
- `role="status"`: screen readers announce "Thinking…"; the dots are `aria-hidden`, so they're skipped.

`src/renderer/src/components/Markdown.tsx`: rendering Claude's replies

Turns a reply's Markdown into formatted text: headings, lists, tables, links and highlighted code. Only `AssistantMessage` uses it, with one prop, `text`. The look (spacing, code backgrounds, the highlight colours) comes from the `.markdown` and `.hljs-…` rules in `styles.css`.

components

src/renderer/src/components/Markdown.tsx, lines 5–18

```
// Links open in the default browser instead of inside the panel.
const components: Components = {
  a: ({ href, children }) => (
    <a
      href={href}
      onClick={(event) => {
        event.preventDefault()
        if (href) void window.assist.openExternal(href)
      }}
    >
      {children}
    </a>
  ),
}
```

Tells react-markdown how to draw particular HTML elements. Only links are changed.

- `const components: Components = {`: declared outside the component, so the object is made once rather than on every render.
- `a: ({ href, children }) => (`: react-markdown calls this for every link instead of making a plain `<a>`, passing the props it would have used. `children` is the link text.
- `event.preventDefault()`: stops the click navigating the panel's own page to the link. The main process would refuse that navigation anyway (`lockDown` in `src/main/bubble/windows.ts`).
- `if (href) void window.assist.openExternal(href)`: asks the main process to open the link in your default browser, or an email link (`mailto:`) in your email app. Anything else, such as `file:` or `javascript:`, is refused (`ExternalUrl` in `src/main/ipc.ts`).

Markdown

src/renderer/src/components/Markdown.tsx, lines 20–36

```
/**
 * Renders Claude's reply: GitHub-flavoured Markdown (tables, task lists) with highlighted code.
 * Raw HTML in the text is never rendered, so a reply can't inject markup.
 */
export function Markdown({ text }: { text: string }) {
  return (
    <div className="markdown">
      <ReactMarkdown
        remarkPlugins={[remarkGfm]}
        rehypePlugins={[[rehypeHighlight, { detect: false }]]}
        components={components}
      >
        {text}
      </ReactMarkdown>
    </div>
  )
}
```

react-markdown parses the text and builds React elements from it directly, so no HTML string is ever inserted into the page. Raw HTML written in a reply is shown as plain text rather than rendered (react-markdown does this unless a plugin such as `rehype-raw` is added, and none is), and its default link check blanks out unsafe addresses such as `javascript:`. So a reply can't inject markup or scripts into the panel.

- `className="markdown"`: hooks into the `.markdown` styles in `styles.css`, sized for the narrow panel. They also turn text selection back on, so replies can be selected and copied.
- `remarkPlugins={[remarkGfm]}`: *remark* plugins work on the Markdown before it becomes HTML. remark-gfm adds GitHub's extensions: tables, task lists (`- [ ]`), `~~strikethrough~~`, footnotes, and bare web addresses turned into links.
- `rehypePlugins={[[rehypeHighlight, { detect: false }]]}`: *rehype* plugins work on the HTML tree afterwards. rehype-highlight colours code blocks with highlight.js (through the lowlight library), adding the `hljs-…` classes that `styles.css` colours. The inner array is the plugin followed by its options. `detect: false` means only a code block that names its language (a fence like ```` ```ts ````) is highlighted; unlabelled blocks stay plain rather than being guessed at. It knows highlight.js's common languages; a block in any other language is left plain.
- `{text}`: the Markdown goes in as the component's children.

`src/renderer/src/components/SignInPanel.tsx`: the sign-in prompts

Shown in the chat card instead of the conversation while nobody is signed in. `Panel` builds it and passes it to `ChatBox` as `signInPrompt` when the sign-in status (`AuthStatus`, kept by `AuthManager`; see JumpCloud sign-in) is `unconfigured`, `signed-out` or `signing-in`. Its props are `open`, that `status`, the app's name, and `onSignIn` and `onCancel`, which `Panel` wires to `window.assist.auth.signIn()` and `window.assist.auth.cancel()`.

`SignInPanel`

src/renderer/src/components/SignInPanel.tsx, lines 5–88

```tsx
/**
 * Shown in place of the chat while nobody is signed in. Signing in happens in the browser, on
 * JumpCloud's own page; if the user is already signed in to JumpCloud there, it finishes by itself
 * and the chat appears.
 */
export function SignInPanel(props: {
  open: boolean
  status: Extract<AuthStatus, { state: 'unconfigured' | 'signed-out' | 'signing-in' }>
  appName: string
  onSignIn: () => void
  onCancel: () => void
}) {
  const { status } = props
  const button = useRef<HTMLButtonElement>(null)

  useEffect(() => {
    if (props.open) button.current?.focus()
  }, [props.open, status.state])

  if (status.state === 'unconfigured') {
    return (
      <Card icon={<Settings2 size={16} aria-hidden />} title="Sign-in isn't set up yet">
        <p className="text-xs text-zinc-500">
          {props.appName} still needs to be connected to JumpCloud and Claude. Your IT team can do
          this by following docs/JUMPCLOUD_SETUP.md. Missing settings:
        </p>
        <ul className="list-inside list-disc text-[11px] text-zinc-500">
          {status.missing.map((name) => (
            <li key={name}>
              <code>{name}</code>
            </li>
```

```
          ))}
        </ul>
      </Card>
    )
  }

  if (status.state === 'signing-in') {
    return (
      <Card
        icon={<LoaderCircle size={16} className="animate-spin" aria-hidden />}
        title="Finish signing in in your browser"
      >
        <p className="text-xs text-zinc-500">
          JumpCloud opened in your browser. Once you're signed in there, come back here.
        </p>
        <div className="flex justify-end">
          <button
            ref={button}
            type="button"
            onClick={props.onCancel}
            className="rounded-md px-2 py-1 text-xs text-zinc-600 hover:bg-zinc-100 dark:text-zinc-300
dark:hover:bg-zinc-800"
          >
            Cancel
          </button>
        </div>
      </Card>
    )
  }

  return (
    <Card icon={<LogIn size={16} aria-hidden />} title="Sign in to start chatting">
      {status.message ? (
        <p role="alert" className="text-xs text-amber-700 dark:text-amber-400">
          {status.message}
        </p>
      ) : (
        <p className="text-xs text-zinc-500">Use your work JumpCloud account.</p>
      )}
      <button
        ref={button}
        type="button"
        onClick={props.onSignIn}
        className="w-full rounded-lg bg-accent px-3 py-2 text-sm font-medium text-white hover:bg-
accent/90"
      >
        Sign in with JumpCloud
      </button>
      <p className="text-[11px] leading-relaxed text-zinc-500">
        This opens JumpCloud in your browser. If you're already signed in there, it finishes on its
        own, and you'll stay signed in here.
      </p>
    </Card>
  )
}
```

Three versions, one for each state: "Sign-in isn't set up yet", "Finish signing in in your browser" with Cancel, and "Sign in to start chatting" with the Sign in button.

- `status: Extract<AuthStatus, { state: 'unconfigured' | 'signed-out' | 'signing-in' }>`: `AuthStatus` is a *union* of six shapes, one per `state`. `Extract` keeps just the three that match, so TypeScript won't let anyone pass a signed-in status here.

- `const button = useRef<HTMLButtonElement>(null)`: the same ref is attached (`ref={button}`) to whichever button is showing, Sign in or Cancel.
- `if (props.open) button.current?.focus()`: moves the keyboard focus to that button, so Enter or Space presses it. With no text box on screen, nothing else would have focus. In the `unconfigured` version there's no button; `?.` skips the call when `button.current` is `null`.
- `}, [props.open, status.state]`: focuses again when the panel opens and when the state changes, because a different button appears.
- `if (status.state === 'unconfigured') {`: checking `state` *narrows* the type. Inside this branch TypeScript knows `status` has `missing`; after the two early `return`s it knows only `signed-out` is left, which has the optional `message`.
- `status.missing.map((name) => (`: lists the tenant.json settings that are still empty (from `missingSettings()` in `src/main/tenant.ts`), each name also serving as its `key`.
- `if (status.state === 'signing-in') {`: while `AuthManager.signIn()` waits for the browser to come back from JumpCloud.
- `onClick={props.onCancel}`: `AuthManager.cancelSignIn()` stops waiting, and the status goes back to `signed-out` with no message.
- `{status.message ? (`: after a sign-in expires or fails, `AuthManager` sets a message saying why. It's shown in amber with `role="alert"`, which screen readers read out at once. Otherwise there's a short hint.
- `onClick={props.onSignIn}`: starts the browser sign-in (`AuthManager.signIn()`).

Card

`src/renderer/src/components/SignInPanel.tsx`, lines 90–102

```
function Card(props: { icon: ReactNode; title: string; children: ReactNode }) {
  return (
    <div className="space-y-3 p-4">
      <div className="flex items-center gap-2.5">
        <span className="grid size-8 shrink-0 place-items-center rounded-full bg-accent/10 text-accent
dark:text-accent-soft">
          {props.icon}
        </span>
        <p className="text-sm font-semibold">{props.title}</p>
      </div>
      {props.children}
    </div>
  )
}
```

The layout the three versions share: a round icon badge and a bold title, then whatever content is passed as `children`.

- `bg-accent/10`: the `/10` makes the accent colour 10% opaque, a pale tint behind the icon.
- `dark:text-accent-soft`: a lighter accent (`--color-accent-soft` in `styles.css`) that's easier to read on a dark background.

`src/renderer/src/components/SettingsMenu.tsx`: the settings menu

The small menu that opens beside the gear icon. `Panel` renders it only while the panel is open, the gear has toggled it on, and the tenant's list of actions includes `settings`. Its props are the `corner` and `offset` for positioning; the current `settings`, `branding` (app and company name, and which kind of chat), `version`, signed-in `user` and the `atlassianUser` connected to Atlassian to display; and one callback per menu item, `onUninstall` and `onDisconnectAtlassian` included. `Panel` closes it on Esc, on a click anywhere else in the panel, and when the panel closes.

`CONFIRM_MS`, `EFFORTS`

src/renderer/src/components/SettingsMenu.tsx, lines 16–22

```
const CONFIRM_MS = 3000

const EFFORTS: { value: Effort; label: string; hint: string }[] = [
  { value: 'low', label: 'Fast', hint: 'Quick answers' },
  { value: 'medium', label: 'Balanced', hint: 'Thinks a little longer' },
  { value: 'high', label: 'Thorough', hint: 'Thinks hardest; slower' },
]
```

- `const CONFIRM_MS = 3000`: how long "New conversation" (or "Clear text box") waits for its second click.
- `EFFORTS: { value: Effort; label: string; hint: string }[]`: the three response styles. Each maps an `Effort` value (what's sent to the Claude API) to the label on its button and a tooltip. Typing `value` as `Effort` means TypeScript rejects a misspelt value. The API side is `AnthropicBackend.reply` in Talking to Claude.

`SettingsMenu`

src/renderer/src/components/SettingsMenu.tsx, lines 24–221

```
/** The menu that opens beside the gear icon, on the side facing the middle of the screen. */
export function SettingsMenu(props: {
  corner: Corner
  /** Distance from the bubble's edge of the window to the gear, so the menu lines up with it. */
  offset: number
  settings: Settings
  branding: Branding
  version: string
  /** Who's signed in, or null when nobody is. */
  user: SignedInUser | null
  /** Who's connected to Atlassian, or null when nobody is. */
  atlassianUser: { name: string; email?: string } | null
  onDisconnectAtlassian: () => void
  onToggleAutoStart: () => void
  onSetEffort: (effort: Effort) => void
  onToggleAutoSend: () => void
  onSignOut: () => void
  onOpenScreenshotsFolder: () => void
  onNewConversation: () => void
  onClearText: () => void
  onUninstall: () => void
}) {
  // "New conversation" (or "Clear text box") needs a second click within a few seconds.
  const [confirmingNew, setConfirmingNew] = useState(false)
  useEffect(() => {
    if (!confirmingNew) return
    const timer = setTimeout(() => setConfirmingNew(false), CONFIRM_MS)
```

```
    return () => clearTimeout(timer)
  }, [confirmingNew])
  // So does "Uninstall".
  const [confirmingUninstall, setConfirmingUninstall] = useState(false)
  useEffect(() => {
    if (!confirmingUninstall) return
    const timer = setTimeout(() => setConfirmingUninstall(false), CONFIRM_MS)
    return () => clearTimeout(timer)
  }, [confirmingUninstall])

  const { settings, branding } = props
  // With Claude Desktop, the chat and its settings live there; only the text box is here.
  const builtInChat = branding.chatApp === 'built-in'

  return (
    <div
      data-hit
      data-settings-menu
      role="menu"
      aria-label="Settings"
      className={cn(
        'absolute rounded-xl border p-1.5 shadow-xl',
        'border-black/10 bg-white text-zinc-800 dark:border-white/10 dark:bg-zinc-900 dark:text-zinc-
100',
      )}
      style={{
        ...anchored(props.corner, PANEL_LAYOUT.settingsX, props.offset),
        width: UI.settingsMenuWidth,
      }}
    >
      <button
        type="button"
        role="menuitemcheckbox"
        aria-checked={settings.autoStart}
        disabled={!settings.autoStartAvailable}
        onClick={props.onToggleAutoStart}
        className={cn(
          'flex w-full items-center justify-between gap-3 rounded-lg px-2.5 py-2 text-left text-sm',
          settings.autoStartAvailable
            ? 'hover:bg-zinc-100 dark:hover:bg-zinc-800'
            : 'cursor-default opacity-60',
        )}
      >
        <span>
          Start with Windows
          {!settings.autoStartAvailable && (
            <span className="block text-[11px] text-zinc-500">Only in the installed app</span>
          )}
        </span>
        <Switch on={settings.autoStart} />
      </button>

      {!builtInChat && (
        <button
          type="button"
          role="menuitemcheckbox"
          aria-checked={settings.autoSend}
          onClick={props.onToggleAutoSend}
          className="flex w-full items-center justify-between gap-3 rounded-lg px-2.5 py-2 text-left
text-sm hover:bg-zinc-100 dark:hover:bg-zinc-800"
        >
          <span>
            Send in Claude automatically
            <span className="block text-[11px] text-zinc-500">
              {settings.autoSend ? 'Questions are sent for you' : 'You press Enter in Claude'}
```

```
            </span>
          </span>
          <Switch on={settings.autoSend} />
        </button>
      )}

      {builtInChat && (
        <div className="px-2.5 pt-1.5 pb-2">
          <p className="mb-1.5 text-sm">Response style</p>
          <div
            role="radiogroup"
            aria-label="Response style"
            className="flex rounded-lg bg-zinc-100 p-0.5 dark:bg-zinc-800"
          >
            {EFFORTS.map((option) => (
              <button
                key={option.value}
                type="button"
                role="radio"
                aria-checked={settings.effort === option.value}
                title={option.hint}
                onClick={() => props.onSetEffort(option.value)}
                className={cn(
                  'flex-1 rounded-md py-1 text-xs font-medium',
                  settings.effort === option.value
                    ? 'bg-white text-zinc-900 shadow-sm dark:bg-zinc-600 dark:text-white'
                    : 'text-zinc-500 hover:text-zinc-800 dark:hover:text-zinc-200',
                )}
              >
                {option.label}
              </button>
            ))}
          </div>
        </div>
      )}

      <MenuItem icon={FolderOpen} onClick={props.onOpenScreenshotsFolder}>
        Open screenshots folder
      </MenuItem>

      <MenuItem
        icon={builtInChat ? MessageSquarePlus : Eraser}
        danger={confirmingNew}
        onClick={() => {
          if (!confirmingNew) return setConfirmingNew(true)
          setConfirmingNew(false)
          if (builtInChat) props.onNewConversation()
          else props.onClearText()
        }}
      >
        {builtInChat
          ? confirmingNew
            ? 'Click again to clear the chat'
            : 'New conversation'
          : confirmingNew
            ? 'Click again to clear the text'
            : 'Clear text box'}
      </MenuItem>

      {props.user && (
        <div className="mt-1 border-t border-black/10 pt-1 dark:border-white/10">
          <p className="truncate px-2.5 pt-1 text-[11px] text-zinc-500" title={props.user.email}>
            Signed in as {props.user.name ?? props.user.email ?? 'you'}
          </p>
          <MenuItem icon={LogOut} onClick={props.onSignOut}>
```

```
            Log out
          </MenuItem>
        </div>
      )}

      {props.atlassianUser && (
        <div className="mt-1 border-t border-black/10 pt-1 dark:border-white/10">
          <p
            className="truncate px-2.5 pt-1 text-[11px] text-zinc-500"
            title={props.atlassianUser.email}
          >
            Connected to Atlassian as {props.atlassianUser.name}
          </p>
          <MenuItem icon={Unplug} onClick={props.onDisconnectAtlassian}>
            Disconnect Atlassian
          </MenuItem>
        </div>
      )}

      <div className="mt-1 border-t border-black/10 pt-1 dark:border-white/10">
        <MenuItem
          icon={Trash2}
          danger={confirmingUninstall}
          disabled={!settings.canUninstall}
          onClick={() => {
            if (!confirmingUninstall) return setConfirmingUninstall(true)
            setConfirmingUninstall(false)
            props.onUninstall()
          }}
        >
          {confirmingUninstall ? 'Click again to uninstall' : `Uninstall ${branding.appName}`}
          {!settings.canUninstall && (
            <span className="block text-[11px] text-zinc-500">Only in the installed app</span>
          )}
        </MenuItem>
      </div>

      <div className="mt-1 border-t border-black/10 px-2.5 pt-1.5 pb-0.5 text-[11px] text-zinc-500
dark:border-white/10">
        {branding.appName} {props.version} · {branding.companyName}
      </div>
    </div>
  )
}
```

Draws the menu's rows. With the built-in chat: Start with Windows, Response style, Open screenshots folder, New conversation, who's signed in with Log out, and the version. With Claude Desktop the chat and its settings live in Claude Desktop, so there's no Response style and no sign-in; instead there's a "Send in Claude automatically" switch, and New conversation becomes Clear text box. When connected to Atlassian, either kind also says who as, with Disconnect Atlassian. Both kinds end with Uninstall, then the version. It keeps two pieces of its own state.

State. `useState(initial)` returns the current value and a function that changes it. Calling that function makes React run the component again with the new value, which redraws it. Use a ref for values that don't affect what's shown, and state for values that do. Here `confirmingNew` switches the "New conversation" (or "Clear text box") row into its "Click again" form, and `confirmingUninstall` does the same for Uninstall.

- `offset: number`: `Panel` passes `actionOffset()` of the gear's place in the icon stack.
- `...anchored(props.corner, PANEL_LAYOUT.settingsX, props.offset)`: `settingsX` (64 px) puts the menu 8 px past the action icons, on the side facing the middle of the screen. `offset` makes its near edge level with the

gear's: in a bottom corner its bottom lines up with the gear's bottom, so it opens upward; in a top corner it opens downward. It overlaps the chat card and is drawn on top because `Panel` renders it after `ChatBox`.

- `data-settings-menu`: lets `Panel`'s `closeSettingsOnOutsideClick` recognise a click inside the menu, so it doesn't close it.
- `role="menu"`: this and `role="menuitemcheckbox"`, `role="radiogroup"` and the rest tell screen readers what each control is (and `aria-checked` whether it's on), since they're all styled buttons rather than real checkboxes and radio buttons.
- `const [confirmingNew, setConfirmingNew] = useState(false)`: the square brackets unpack the two things `useState` returns: the value and its setter.
- `return () => clearTimeout(timer)`: when `confirmingNew` turns on, the effect starts a 3-second timer that turns it off again. The function an effect returns is its *cleanup*: React calls it before running the effect again and when the component disappears. So the timer is cancelled after a confirming second click, or when the menu closes.
- `// So does "Uninstall".`: the same pair of state and timer for the Uninstall row, kept separate so arming one row never arms the other.
- `const { settings, branding } = props`: shorthand for two props used often below.
- `const builtInChat = branding.chatApp === 'built-in'`: which rows to show. The rest of the menu checks this one value.
- `disabled={!settings.autoStartAvailable}`: Start with Windows only works in the installed app, not in `npm run dev`; there the row is greyed out with a note. `onToggleAutoStart` asks the main process to flip it, and `Panel` keeps the settings it returns (see `settings.ts` in Main process: startup, IPC and app plumbing).
- `<Switch on={settings.autoStart} />`: the on/off graphic at the end of the row.
- `{!builtInChat && (`: with Claude Desktop, the "Send in Claude automatically" row, built the same way as Start with Windows: a `menuitemcheckbox` button with the setting's name, a grey line under it, and a `Switch`. Clicking it calls `onToggleAutoSend`, which flips the setting in the main process.
- `{settings.autoSend ? 'Questions are sent for you' : 'You press Enter in Claude'}`: the grey line says what the switch means in practice.
- `{builtInChat && (`: Response style only with the built-in chat. With Claude Desktop the chat, and how Claude answers, are Claude Desktop's business, so the setting would do nothing.
- `aria-checked={settings.effort === option.value}`: the response styles work like radio buttons. The current one is white with a shadow; the others are plain text on the grey track.
- `onClick={() => props.onSetEffort(option.value)}`: `Panel.setEffort` saves the choice in the main process. `ChatSession` reads it at the start of each request, so it applies from the next message.
- `icon={builtInChat ? MessageSquarePlus : Eraser}`: one row that does either job, with a speech bubble for New conversation or an eraser for Clear text box. Both clear something and need the same second click, so they share the row and its confirm state.
- `if (!confirmingNew) return setConfirmingNew(true)`: the first click only arms the row (the `return` just ends the function early). A second click within 3 seconds turns it off and calls `onNewConversation`, which clears the chat (`ChatSession.newConversation()`), or with Claude Desktop `onClearText`, which empties the text box and its screenshots.
- `danger={confirmingNew}`: the row turns red while it's armed.
- `{builtInChat ? confirmingNew ? ... : ... : ...}`: the label, from two questions in turn: which kind of chat, then whether the row is armed. So it reads "New conversation" or "Click again to clear the chat", or "Clear text box" or "Click again to clear the text".

- `{props.user && (`: only when someone is signed in; `Panel` passes `null` otherwise, which is always the case with Claude Desktop.
- `props.user.name ?? props.user.email ?? 'you'`: the first of the name or the email that JumpCloud provided. `truncate` cuts a long one short with "…", and `title={props.user.email}` shows the email as a tooltip.
- `{props.atlassianUser && (`: when connected to Atlassian, who as ("Connected to Atlassian as Casey Jones", with the email as a tooltip), and **Disconnect Atlassian** with a plug icon (`Unplug`), which calls `onDisconnectAtlassian`. It needs no second click: nothing is lost, and connecting again is a moment in the browser.
- `icon={Trash2}`, `danger={confirmingUninstall}`: Uninstall, in a section of its own below a line, needs the same second click within 3 seconds, and turns red while armed. The second click calls `onUninstall`; `Panel.uninstall` starts the uninstaller and the app quits.
- `disabled={!settings.canUninstall}`: only the installed app has an uninstaller. In `npm run dev` the row is greyed out, with "Only in the installed app" under its label (the same note as Start with Windows).
- `` `Uninstall ${branding.appName}` ``: the label names the app, "Uninstall Desktop Assist".
- `{branding.appName} {props.version} · {branding.companyName}`: the footer, for example "Desktop Assist 0.1.0 · Morse Micro".

`MenuItem`

`src/renderer/src/components/SettingsMenu.tsx`, lines 223–250

```tsx
function MenuItem(props: {
  icon: LucideIcon
  danger?: boolean
  disabled?: boolean
  onClick: () => void
  children: ReactNode
}) {
  const Icon = props.icon
  return (
    <button
      type="button"
      role="menuitem"
      disabled={props.disabled}
      onClick={props.onClick}
      className={cn(
        'flex w-full items-center gap-2.5 rounded-lg px-2.5 py-2 text-left text-sm',
        props.danger
          ? 'bg-red-600 text-white'
          : props.disabled
            ? 'cursor-default opacity-60'
            : 'hover:bg-zinc-100 dark:hover:bg-zinc-800',
      )}
    >
      <Icon size={16} aria-hidden />
      {props.children}
    </button>
  )
}
```

One row of the menu: an icon and a label, as a button.

- `icon: LucideIcon`: the prop is the icon component itself (`FolderOpen`), not a drawn icon (`<FolderOpen />`), so `MenuItem` decides the size.

- `const Icon = props.icon`: JSX treats a lowercase tag as an HTML element, so the component has to be in a capitalised variable before it can be written as `<Icon size={16} aria-hidden />`.
- `disabled?: boolean`: a greyed-out row that can't be clicked (`disabled={props.disabled}` on the button), used for Uninstall in a dev run.
- `props.danger ? 'bg-red-600 text-white'`: the red version, used for the armed "New conversation", "Clear text box" or Uninstall. Otherwise a disabled row is faded (`opacity-60`) with no hover, and an enabled one gets the grey hover.

`Switch`

`src/renderer/src/components/SettingsMenu.tsx`, lines 252–269

```
function Switch({ on }: { on: boolean }) {
  return (
    <span
      aria-hidden
      className={cn(
        'relative inline-flex h-5 w-9 shrink-0 rounded-full transition-colors',
        on ? 'bg-accent' : 'bg-zinc-300 dark:bg-zinc-600',
      )}
    >
      <span
        className={cn(
          'absolute top-0.5 size-4 rounded-full bg-white shadow transition-transform',
          on ? 'translate-x-4.5' : 'translate-x-0.5',
        )}
      />
    </span>
  )
}
```

A pill-shaped track with a white knob. It's purely a picture: the `menuitemcheckbox` button around it tells screen readers whether it's on, so the switch itself is `aria-hidden`.

- `on ? 'bg-accent' : 'bg-zinc-300 dark:bg-zinc-600'`: the track is the accent colour when on.
- `on ? 'translate-x-4.5' : 'translate-x-0.5'`: the 16 px knob sits 2 px from the left of the 36 px track when off and slides to 18 px when on; `transition-transform` animates the slide.

`src/renderer/src/components/ActionStack.tsx`: the action icons

The column of round icon buttons that pops out of the bubble when the panel opens (for Morse Micro: Ask Claude, Jira, IT service desk, screenshot, light or dark mode, settings, bounce and close), in the order the tenant's `tenant.json` lists them, nearest the bubble first. `Panel` always renders it; while closed it's simply invisible. Its props are the `corner`, the tenant's `actions`, `open`, `activeId` (`settings` while the settings menu is open, otherwise the icon of the page showing when it isn't the start page, or `null`), `theme` (the current light or dark mode), `pages` (command icons that open a page in the panel instead: the service desk, when the tenant lists request types), `dotted` (icons that get a small dot) and `onAction`, which is `Panel.runAction`.

ICONS

src/renderer/src/components/ActionStack.tsx, lines 20–30

```
const ICONS: Record<ActionId, LucideIcon> = {
  ask: MessageCircle,
  apps: LayoutGrid,
  jira: Bell,
  servicedesk: Headset,
  screenshot: Camera,
  theme: Sun,
  settings: Settings,
  bounce: Volleyball,
  close: Power,
}
```

Which lucide icon each action shows. Ask Claude is `MessageCircle`, a speech bubble; Apps is `LayoutGrid`, a grid of four squares, the usual sign for "apps"; Jira is `Bell`, for its notifications; and the IT service desk is `Headset`, as for a help desk. The theme icon is `Sun`, but see below: it changes with the mode.

- `Record<ActionId, LucideIcon>`: an object with exactly one entry for every action id. If a new id is added to `ACTION_IDS` in `src/shared/actions.ts` without an icon here, TypeScript reports an error.

ActionStack

src/renderer/src/components/ActionStack.tsx, lines 32–112

```
/**
 * The icons that pop out of the bubble, nearest-first in the tenant's order: stacked above it in
 * a bottom corner, below it in a top corner.
 */
export function ActionStack(props: {
  corner: Corner
  actions: ActionId[]
  open: boolean
  activeId: ActionId | null
  /** The current mode: the theme icon shows the one it switches to. */
  theme: Theme
  /** Icons with a small dot, e.g. Ask Claude while an unsent question is waiting behind it. */
  dotted?: ActionId[]
  /** Command icons that open a page in the panel instead (the service desk, with a form). */
  pages?: ActionId[]
  onAction: (id: ActionId) => void
}) {
  const top = isTopCorner(props.corner)
  return (
    <div
      className={cn('absolute flex', top ? 'flex-col' : 'flex-col-reverse')}
      style={{
        ...anchored(props.corner, PANEL_LAYOUT.actionsX, PANEL_LAYOUT.actionsY),
        gap: UI.actionGap,
      }}
    >
      {props.actions.map((id, index) => {
        const Icon = id === 'theme' && props.theme === 'light' ? Moon : ICONS[id]
        const label =
          id === 'theme'
            ? props.theme === 'dark'
              ? 'Switch to light mode'
              : 'Switch to dark mode'
            : ACTIONS[id].label
```

```
        const active = id === props.activeId
        return (
          <button
            key={id}
            data-hit
            data-action={id}
            type="button"
            title={label}
            aria-label={label}
            aria-expanded={
              ACTIONS[id].kind === 'popover' || props.pages?.includes(id) ? active : undefined
            }
            onClick={() => props.onAction(id)}
            style={{
              width: UI.actionSize,
              height: UI.actionSize,
              // Pop out one after another, nearest the bubble first.
              transitionDelay: props.open ? `${index * 30}ms` : '0ms',
            }}
            className={cn(
              'relative grid place-items-center rounded-full border shadow-md outline-none',
              'transition duration-150 ease-out motion-reduce:transition-none',
              'focus-visible:ring-2 focus-visible:ring-accent',
              // Hidden icons sit tucked toward the bubble, then slide out.
              top ? '-translate-y-3' : 'translate-y-3',
              'scale-75 opacity-0',
              'group-data-open:translate-y-0 group-data-open:scale-100 group-data-open:opacity-100',
              active
                ? 'border-transparent bg-accent text-white'
                : id === 'close'
                  ? 'border-black/10 bg-white text-zinc-700 hover:border-transparent hover:bg-red-600
hover:text-white dark:border-white/10 dark:bg-zinc-800 dark:text-zinc-200'
                  : 'border-black/10 bg-white text-zinc-700 hover:bg-zinc-100 dark:border-white/10
dark:bg-zinc-800 dark:text-zinc-200 dark:hover:bg-zinc-700',
            )}
          >
            <Icon size={18} strokeWidth={2} aria-hidden />
            {props.dotted?.includes(id) && !active && (
              <span
                aria-hidden
                className="absolute top-1 right-1 size-2.5 rounded-full border-2 border-white bg-accent
dark:border-zinc-800"
              />
            )}
          </button>
        )
      })}
    </div>
  )
}
```

Lays the icons out in a column beside the bubble and animates them in and out.

- `top ? 'flex-col' : 'flex-col-reverse'`: the first action must be nearest the bubble. In a top corner the bubble is above the stack, so it's a normal top-to-bottom column. In a bottom corner the bubble is below, so the column is reversed and the first action sits at the bottom.
- `...anchored(props.corner, PANEL_LAYOUT.actionsX, PANEL_LAYOUT.actionsY)`: `actionsX` (16 px) centres the 40 px icons on the 56 px bubble. `actionsY` (76 px) starts the stack 12 px past the bubble.
- `gap: UI.actionGap`: 8 px between icons. The main process sizes the panel window from the same numbers (`panelWindowSize` in `src/shared/geometry.ts`), so the stack always fits.

- `const Icon = id === 'theme' && props.theme === 'light' ? Moon : ICONS[id]`: the theme icon shows what clicking it switches to: a sun while dark (switch to light), a moon while light.
- `const label = id === 'theme' ? ... : ACTIONS[id].label`: the tooltip and screen-reader name, from the action registry, except for the theme icon, which says what it will do ("Switch to light mode" or "Switch to dark mode").
- `data-hit`: on each button, not on the column, so the gaps between icons stay click-through.
- `data-action={id}`: `Panel.closeSettingsOnOutsideClick` looks for `[data-action="settings"]`, so clicking the gear toggles the menu instead of counting as a click outside it.
- `aria-expanded={ACTIONS[id].kind === 'popover' || props.pages?.includes(id) ? active : undefined}`: for a popover action (Settings, Ask Claude, Apps and Jira), or an icon that opens a page (the service desk, with request types), it tells screen readers whether what it opens is showing. `undefined` leaves the attribute off the others, the theme icon included, which opens nothing.
- `onClick={() => props.onAction(id)}`: `Panel.runAction` toggles the menu for Settings, switches the card's page for Ask Claude, Apps, Jira and (with request types) the service desk, flips light or dark mode for the theme icon, and asks the main process to run any other action (see `src/main/actions.ts` in Main process: startup, IPC and app plumbing).
- `transitionDelay: props.open ? \`${index * 30}ms\` : '0ms'`: each icon starts its animation 30 ms after the one before, so they pop out one after another. Closing has no delay, so they all go together.
- `top ? '-translate-y-3' : 'translate-y-3'`: while closed, each icon sits 12 px toward the bubble, and `'scale-75 opacity-0'` shrinks it to 75% and hides it. The `group-data-open:` classes (explained under `ChatBox`) move it to its place at full size when the panel opens.
- `outline-none`: together with `'focus-visible:ring-2 focus-visible:ring-accent'`, swaps the usual square focus outline for an accent ring that follows the round button.
- `active`: the active action (the gear while its menu is open, or a page's icon while its page is showing) is filled with the accent colour.
- `'relative grid ...'`: `relative` makes each button the reference box for the dot below.
- `{props.dotted?.includes(id) && !active && (`: a small accent-coloured dot in the icon's corner (`absolute top-1 right-1`), with a ring in the button's own colour so it stands out. `Panel` puts one on Ask Claude while an unsent question or screenshot is waiting behind it, on the bell while there are unread notifications, and on the service desk while one of the user's requests has news. Not on the active icon, whose page is already showing. Otherwise `id === 'close'` turns red on hover, as a warning that it quits the app, and the rest get a grey hover.

`src/renderer/src/components/AttachmentChip.tsx`: a screenshot in the draft

A screenshot attached to the draft, shown in the row above the text box. `Composer` renders one per attachment. Its props are the `attachment`, `onOpen` (`Panel.openAttachment`, which shows a toast if the file can't be found) and `onRemove` (`Panel.removeAttachment`).

AttachmentChip

src/renderer/src/components/AttachmentChip.tsx, lines 6–43

```
/** A screenshot attached to the text box: click to open it, × to remove it. */
export function AttachmentChip(props: {
  attachment: Attachment
  onOpen: () => void
  onRemove: () => void
}) {
  const thumbnail = useThumbnail(props.attachment.path)
  const missing = thumbnail === null
  const label = missing ? 'File missing' : screenshotLabel(props.attachment.fileName)

  return (
    <div className="relative flex shrink-0 items-center rounded-lg border border-black/10 bg-zinc-50
dark:border-white/10 dark:bg-zinc-800">
      <button
        type="button"
        title={props.attachment.fileName}
        onClick={props.onOpen}
        className="flex items-center gap-2 rounded-lg py-1 pr-8 pl-1 text-left hover:bg-zinc-100
dark:hover:bg-zinc-700"
      >
        <span className="grid h-10 w-14 place-items-center overflow-hidden rounded-md bg-zinc-200 text-
zinc-500 dark:bg-zinc-700">
          {thumbnail ? (
            <img src={thumbnail} alt="" draggable={false} className="size-full object-cover" />
          ) : (
            missing && <ImageOff size={16} aria-hidden />
          )}
        </span>
        <span className="text-xs whitespace-nowrap">{label}</span>
      </button>
      <button
        type="button"
        aria-label={`Remove ${label}`}
        onClick={props.onRemove}
        className="absolute top-1/2 right-1 -translate-y-1/2 rounded-full p-1 text-zinc-500 hover:bg-
black/10 hover:text-zinc-900 dark:hover:bg-white/10 dark:hover:text-white"
      >
        <X size={14} aria-hidden />
      </button>
    </div>
  )
}
```

A thumbnail and a short label. Clicking the chip opens the screenshot; the × removes it from the draft (the file itself stays in the screenshots folder).

- `const thumbnail = useThumbnail(props.attachment.path)`: the preview, as for `SentScreenshot`.
- `const missing = thumbnail === null`: `useThumbnail` returns `null` only when the main process can't make a preview (for example the file was deleted or moved out of the screenshots folder), and `undefined` while it's still loading. `===` keeps the two apart, so a chip that's still loading doesn't say "File missing".
- `screenshotLabel(props.attachment.fileName)`: from `src/shared/format.ts`, gives "Screenshot 14:03" for one taken today and "Screenshot 2 Oct 14:03" for an older one.
- `draggable={false}`: stops the browser letting you drag the image out of the chip.
- `missing && <ImageOff size={16} aria-hidden />`: a "no image" icon for a missing file; while loading, the box is simply empty.

- `whitespace-nowrap`: keeps the label on one line.
- `pr-8`: the chip is two separate buttons, because HTML doesn't allow a button inside another button. The open button leaves 32 px of padding on its right for the remove button.
- `absolute top-1/2 right-1 -translate-y-1/2`: places the remove button over the right end of the chip, centred vertically (its top at half the height, then moved up by half its own height).
- `aria-label={`Remove ${label}`}`: the button shows only an ×, so this gives it a name for screen readers, such as "Remove Screenshot 14:03".
- `onClick={props.onRemove}`: `Panel.removeAttachment` asks the main process to drop it from the saved draft and shows the updated list.

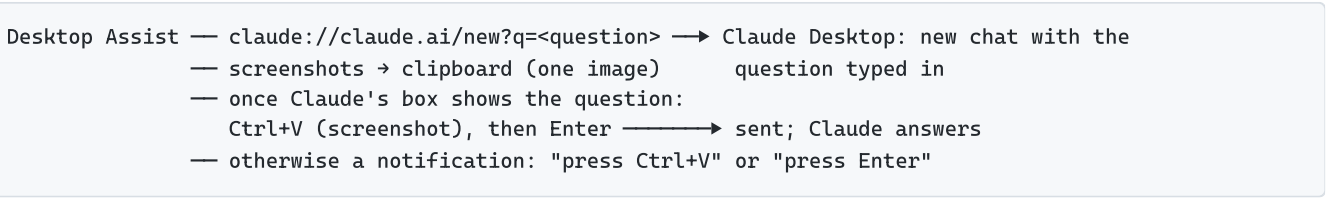
Claude Desktop: handing questions over

Since feature 3.1, a tenant can send questions to the **Claude Desktop** app instead of the chat built into the panel (`"chatApp": "claude-desktop"` in `tenant.json`; Morse Micro does). The point is whose Claude it is. Each person asks in their own Claude account, on the company's Claude plan, so every question counts against their own usage limit, which the company already manages in claude.ai, and the conversation carries on in Claude Desktop. Desktop Assist then has no sign-in of its own and sends nothing to Anthropic itself: there's no JumpCloud app, no Claude Console service account, and no usage billed to the company's Console account.

What Desktop Assist does is small. It opens a new chat in Claude Desktop with the question already typed in, and then, unless the user has turned **Send in Claude automatically** off, presses the keys that send it. That's in two main-process files, covered below: `src/main/claudeDesktop.ts` (the hand-over) and `src/main/sendInClaude.ts` (the key presses). The other pieces are on other pages: `start()` in `index.ts` creates the `ClaudeDesktop` and wires in the Electron parts, and `registerIpc()` in `ipc.ts` answers the panel (both in Main process: startup, IPC and app plumbing); the setting is in `settings.ts` (same page); the panel's side is `askInClaudeDesktop` in `PanelView.tsx` (The pages); and several screenshots are combined into one by `ScreenshotService.forClipboard()` (The draft, screenshots and safe files).

Asking a question

1. The user types a question, perhaps attaches screenshots, and presses Enter or **Ask in Claude**. `Panel.askInClaudeDesktop()` calls `window.assist.claudeDesktop.ask(text)`, which sends IPC `assist:claude-desktop-ask`.
2. The handler in `ipc.ts` calls `ClaudeDesktop.ask()` with the text and the draft's attached screenshots.
3. `ask()` checks that there's something to ask, that the text fits in a link (12,000 characters at most), and that Claude Desktop is installed.
4. If screenshots are attached, it copies them to the clipboard first, as one image.
5. It opens the link `claude://claude.ai/new?q=<the question>`. Windows hands it to Claude Desktop, which comes to the front with a new chat and the question typed in, but not sent.
6. `ask()` returns straight away, and `ipc.ts` closes the panel and empties the draft. The rest carries on in the background (`ClaudeDesktop.finish()`).
7. With **Send in Claude automatically** on and a question typed, `sendInClaude()` runs a hidden PowerShell helper. It waits until Claude's focused text box holds exactly the question, presses Ctrl+V if there's a screenshot, then Enter, and Claude answers. If it can't, a Windows notification says what's left to do (paste, press Enter, or check the question). Clicking a notification opens the panel, as the bubble would.
8. With the setting off, or for a screenshot without a question, nothing is pressed. If screenshots were copied, a notification says to press Ctrl+V in Claude; the user then sends the question there.



Why press keys at all? Claude Desktop's link only fills the question in; Anthropic documents no way for a link to send it. The user wanted it sent for them, so the helper presses the keys a person would. Pressing keys into whatever window is in front would be dangerous, though: a different app could have come to the front, or the same Claude Desktop window could be showing another Claude text box (it can also hold a Claude Code session). So

before every key press the helper asks Windows which element has the keyboard focus, and only goes ahead when that element belongs to Claude Desktop **and** holds exactly the question, nothing more. A screenshot on its own is never sent, because there would be no question to recognise the right box by.

Why exactly the question? If the new chat's box already had something typed in it (a draft the user left unsent), Claude Desktop adds the linked question to that text rather than replacing it. Pressing Enter would then send the old draft too. So when the box holds the question *and* other text, the helper presses nothing (`extra-text`), and a notification asks the user to check the question in Claude and send it themselves.

Is Claude Desktop installed? When it's installed, Claude Desktop registers itself with Windows as the app that opens `claude://` links. Electron's `app.getApplicationNameForProtocol()` asks Windows for that app's name, and an empty answer means nothing is registered. The panel asks each time it opens (IPC `assist:claude-desktop-installed`) and, if the answer is no, shows a banner with a Download button. `ask()` checks again before doing anything.

`src/main/claudeDesktop.ts`: opening a question in Claude Desktop

There's no Electron in this file. Everything that touches Windows (the protocol lookup, opening the link, the clipboard, the setting, sending in Claude, the notification) comes in through `ClaudeDesktopDeps`, which `start()` fills in. That's what lets `tests/claudeDesktop.test.ts` check the whole flow with fakes.

`NEW_CHAT_LINK` and `MAX_PROMPT_LENGTH`

`src/main/claudeDesktop.ts`, lines 4–8

```
/** Claude Desktop's documented link for a new chat. A `q` fills in the prompt; it isn't sent. */
export const NEW_CHAT_LINK = 'claude://claude.ai/new'

/** Claude Desktop cuts a `q` prompt off at about 14,000 characters, so stay under that. */
export const MAX_PROMPT_LENGTH = 12_000
```

- `'claude://claude.ai/new'`: Claude Desktop's documented link for a new chat (see Anthropic's Open Claude Desktop with a link). `claude://` is a *URL scheme* of its own: just as Windows opens `mailto:` links in the email app, it opens `claude://` links in the app registered for them, Claude Desktop.
- `q`: adding `?q=...` puts text in the new chat's message box, but doesn't send it. A link can't send a message on anyone's behalf; that's what `sendInClaude.ts` is for.
- `MAX_PROMPT_LENGTH = 12_000`: Claude Desktop cuts a `q` off at about 14,000 characters, so a longer question would arrive with its end missing and no warning. 12,000 leaves a margin. Anything longer is refused (`too-long`), and the panel suggests shortening it or pasting it into Claude directly. The text box itself takes far more (`MAX_NOTE_LENGTH`, 200,000 characters).

`newChatLink()`

src/main/claudeDesktop.ts, lines 10–15

```
/** The link that opens a new Claude Desktop chat with `text` typed in, ready to send. */
export function newChatLink(text: string): string {
  // encodeURIComponent rather than URLSearchParams, which writes spaces as "+": a reader that
  // decodes with decodeURIComponent would show those as plus signs.
  return text.trim() ? `${NEW_CHAT_LINK}?q=${encodeURIComponent(text)}` : NEW_CHAT_LINK
}
```

Builds the link for a question.

- `encodeURIComponent(text)`: turns every character that means something in a link into `%` followed by its code. A space becomes `%20`, `&` becomes `%26`, `#` becomes `%23`, a line break becomes `%0A`, and anything beyond plain ASCII (accents, emoji, other scripts) becomes its UTF-8 bytes. So the whole question arrives as the value of `q`: a `&` in it can't start another parameter, and a `#` can't cut off the rest.
- `// encodeURIComponent rather than URLSearchParams`: `URLSearchParams` is the usual way to build a query string, but it follows the rules for web forms, which write a space as `+`. A reader that decodes with `decodeURIComponent` (which leaves `+` alone) would then show every space as a plus sign. `encodeURIComponent` writes spaces as `%20`, which every reader turns back into a space, and a real `+` as `%2B`.
- `text.trim() ? ... : NEW_CHAT_LINK`: a question that's only screenshots (or only spaces) opens an empty new chat, ready for the paste. Otherwise the text goes in exactly as typed, untrimmed.

`pasteHint()`, `sendHint()` and `extraTextHint()`

src/main/claudeDesktop.ts, lines 17–40

```
/** What to tell the user when their screenshots are on the clipboard, still to be pasted. */
export function pasteHint(count: number): { title: string; body: string } {
  return {
    title: count === 1 ? 'Screenshot copied' : `${count} screenshots copied as one image`,
    body: 'In Claude, press Ctrl+V to add it to your message, then send.',
  }
}

/** What to tell the user when their question is in Claude but wasn't sent for them. */
export function sendHint(): { title: string; body: string } {
  return { title: 'Your question is in Claude', body: 'Press Enter in Claude to send it.' }
}

/** What to tell the user when Claude's text box already had something else in it. */
export function extraTextHint(screenshots: number): { title: string; body: string } {
  return {
    title: 'Check your question in Claude',
    body:
      "Claude's message box already had text in it, so it wasn't sent." +
      (screenshots > 0
        ? ' Press Ctrl+V to add the screenshot, then send.'
        : ' Send it when ready.'),
  }
}
```

The three Windows notifications, for when there's something left for the user to do in Claude. The panel can't say it, because it closes as Claude Desktop opens. `finish()` (below) picks one.

- `pasteHint`: the screenshots are on the clipboard but haven't been pasted, so the user pastes them and sends. The title says how many, and that several became one image, so pasting a single image isn't a surprise.
- `sendHint`: the question (and the screenshot, if any) is already in Claude's message box, and only Enter is left.
- `extraTextHint`: Claude's box already had other text in it, so nothing was sent. The user should check the question and send it when ready, pasting the screenshot first if there is one (nothing was pressed, so it's still on the clipboard).

`AskOutcome` and `ClaudeDesktopDeps`

`src/main/claudeDesktop.ts`, lines 42–60

```ts
export type AskOutcome = { ok: true; screenshots: number } | Extract<AskResult, { ok: false }>

export interface ClaudeDesktopDeps {
  /** Whether anything on this PC opens claude:// links (Claude Desktop registers them). */
  isInstalled(): boolean
  openLink(url: string): Promise<void>
  /** Puts the screenshots on the clipboard as one image. False if one of them is missing. */
  copyScreenshots(paths: string[]): Promise<boolean>
  /** Whether to send the question in Claude for the user (the "Send in Claude" setting). */
  autoSend(): boolean
  /**
   * Once Claude's text box shows the question: presses Ctrl+V when `paste` is set, then Enter.
   * Presses nothing anywhere else (see sendInClaude.ts).
   */
  sendInClaude(question: string, paste: boolean): Promise<SendOutcome>
  /** A Windows notification, e.g. to say the screenshot is ready to paste. */
  notify(message: { title: string; body: string }): void
  onError?(error: unknown): void
}
```

`AskOutcome` is what `ask()` returns. A success carries the number of screenshots copied. A failure is exactly the failure half of `AskResult` (from `src/shared/types.ts`; `Extract` picks it out), so `ipc.ts` can pass it straight to the panel. The success has no `notes`: emptying the draft is `ipc.ts`'s job, once it knows Claude Desktop has opened.

`ClaudeDesktopDeps` is everything that touches Windows. `start()` in `index.ts` passes:

- `isInstalled()`: `app.getApplicationNameForProtocol(NEW_CHAT_LINK) !== ''`, whether any app is registered for `claude://` links.
- `openLink(url)`: `shell.openExternal(url)`. Its promise rejects if Windows can't open the link.
- `copyScreenshots(paths)`: `ScreenshotService.forClipboard()`, then Electron's clipboard. It returns `false` if a screenshot is missing, and throws if the clipboard can't be written.
- `autoSend()`: the user's **Send in Claude automatically** setting, read for each question, so a change in the settings menu applies to the next one.
- `sendInClaude(question, paste)`: `sendInClaude()` below, wrapped so that any outcome but `sent` is written to the log.
- `notify(message)`: a Windows notification, if Windows can show one. Clicking it opens the panel (the notification appears over the bubble's usual corner).
- `onError?(error)`: writes a failure to the problem log. The `?` makes it optional.

`ClaudeDesktop`

src/main/claudeDesktop.ts, lines 62–68

```
/**
 * Hands questions over to the Claude Desktop app, so they're answered with the person's own Claude
 * account and count against their own usage limit. A link can carry text but not images, so the
 * screenshots go on the clipboard. A link only fills the question in; with "Send in Claude" on,
 * Desktop Assist then presses Ctrl+V and Enter in Claude for the user.
 */
export class ClaudeDesktop {
  // …
}
```

The class the rest of the app uses. `start()` creates the one instance, only when the tenant's `chatApp` is `claude-desktop`, and `registerClaudeDesktop()` in `ipc.ts` calls `isInstalled()` and `ask()`.

`ClaudeDesktop` fields, constructor and `isInstalled()`

src/main/claudeDesktop.ts, lines 69–76

```
private finishing: Promise<void> = Promise.resolve()

constructor(private readonly deps: ClaudeDesktopDeps) {}

/** Checked each time the panel opens, so the panel can say so before anything is typed. */
isInstalled(): boolean {
  return this.deps.isInstalled()
}
```

- `private finishing: Promise<void> = Promise.resolve()`: the background work for the last question (see `finish()` below). It starts out as a promise that has already resolved, so `idle()` has something to return before any question is asked.
- `constructor(private readonly deps: ClaudeDesktopDeps) {}`: the constructor shorthand also used by `NotesStore`: `private readonly` makes the parameter a field, `this.deps`, so the body is empty.
- `isInstalled()`: just asks the dependency. The panel calls it through IPC each time it opens, so it can warn before anything is typed.

`ClaudeDesktop.ask()`

`src/main/claudeDesktop.ts`, lines 78–100

```
/**
 * Opens a new Claude Desktop chat with `text` filled in, with the screenshots ready to paste.
 * Returns once Claude is opening; sending it there (or saying what's left to do) carries on in
 * the background, see idle().
 */
async ask(text: string, attachments: Attachment[]): Promise<AskOutcome> {
  if (!text.trim() && attachments.length === 0) return { ok: false, reason: 'empty' }
  if (text.length > MAX_PROMPT_LENGTH) return { ok: false, reason: 'too-long' }
  if (!this.deps.isInstalled()) return { ok: false, reason: 'not-installed' }
  try {
    // Copied first, so the screenshot is ready by the time Claude opens.
    const paths = attachments.map((attachment) => attachment.path)
    if (paths.length > 0 && !(await this.deps.copyScreenshots(paths))) {
      return { ok: false, reason: 'missing-screenshot' }
    }
    await this.deps.openLink(newChatLink(text))
  } catch (error) {
    this.deps.onError?.(error)
    return { ok: false, reason: 'failed' }
  }
  this.finishing = this.finish(text, attachments.length)
  return { ok: true, screenshots: attachments.length }
}
```

Hands one question over. The checks come first, and neither the clipboard nor Claude Desktop is touched until they've all passed.

- `if (!text.trim() && attachments.length === 0)`: nothing to ask. The panel's Ask button is disabled then, so this is a second guard.
- `text.length > MAX_PROMPT_LENGTH`: too long for a link, so it's refused rather than cut short.
- `!this.deps.isInstalled()`: checked here as well as when the panel opened, because Claude Desktop may have been uninstalled since. A missing app gets a clear message instead of whatever Windows does with a link nothing can open.
- `try {`: copying and opening are the two steps that can fail for reasons outside the app (the clipboard held by another program, Windows refusing the link). Either one becomes `failed`, and the error goes to `onError` for the log. `onError?.(error)` calls it only if it was given.
- `// Copied first, so the screenshot is ready by the time Claude opens.`: Claude Desktop comes to the front as soon as the link opens, and the screenshot may be pasted straight away (by the helper or by the user). Copying first also means a missing screenshot stops everything before Claude Desktop opens: the user removes it and asks again.
- `paths.length > 0 && !(await this.deps.copyScreenshots(paths))`: only when there are screenshots. `false` means one has gone missing (`missing-screenshot`). Copying replaces whatever was on the clipboard before.
- `await this.deps.openLink(newChatLink(text))`: opens the new chat with the question in it.
- `this.finishing = this.finish(text, attachments.length)`: starts the rest (sending, or telling the user what to do) without waiting for it. Sending can take several seconds while Claude opens, and the panel shouldn't hang on that. Keeping the promise lets `idle()` wait for it.
- `return { ok: true, screenshots: attachments.length }`: `ipc.ts` then closes the panel and empties the draft, while `finish()` carries on.

`ClaudeDesktop.idle()`

src/main/claudeDesktop.ts, lines 102–105

```
/** Resolves when the last question has been sent in Claude, or the user told what to do. */
idle(): Promise<void> {
  return this.finishing
}
```

A promise that resolves once the last question's background work is done: sent in Claude, or the user told what's left. The app itself never waits for it. Like `ChatSession.idle()`, it's there for the tests, which `await claude.idle()` before checking what happened.

`ClaudeDesktop.finish()`

src/main/claudeDesktop.ts, lines 107–130

```
/**
 * Sends the question in Claude if the setting is on, and otherwise (or if it couldn't) tells
 * the user what's left: pasting the screenshot, or pressing Enter. A screenshot on its own is
 * never sent for them: with no question to look for in Claude's text box, there's no telling it
 * apart from another Claude box.
 */
private async finish(text: string, screenshots: number): Promise<void> {
  if (!this.deps.autoSend() || !text.trim()) {
    if (screenshots > 0) this.deps.notify(pasteHint(screenshots))
    return
  }
  const outcome = await this.deps
    .sendInClaude(text, screenshots > 0)
    .catch((error: unknown): SendOutcome => {
      this.deps.onError?.(error)
      return 'failed'
    })
  if (outcome === 'sent') return
  if (outcome === 'extra-text') return this.deps.notify(extraTextHint(screenshots))
  // Nothing was pressed: the screenshot (if any) still needs pasting. Otherwise it was pasted
  // and only Enter is left.
  const pasted = outcome === 'focus-lost' || outcome === 'not-sent'
  this.deps.notify(screenshots > 0 && !pasted ? pasteHint(screenshots) : sendHint())
}
```

What happens after Claude Desktop has opened with the question. A problem with sending never reaches the panel (which has closed by now); it turns into a notification instead.

- `if (!this.deps.autoSend() || !text.trim())`: nothing is pressed when the user has turned **Send in Claude automatically** off, or when there's no question, only screenshots. As the comment explains, a screenshot alone can't be sent safely: the helper recognises Claude's box by the question in it, and an empty box looks like any other.
- `if (screenshots > 0) this.deps.notify(pasteHint(screenshots))`: in that case, a notification only if there's something to paste. A text-only question is already in Claude's box, where the user can see it and press Enter.
- `this.deps.sendInClaude(text, screenshots > 0)`: sends it, pasting first when there are screenshots. The promise resolves with how it went.
- `.catch((error: unknown): SendOutcome => {`: if the helper couldn't run at all (PowerShell missing or blocked, the script failing), the error is logged and treated as `failed`, so the user still gets told what to do.

- `if (outcome === 'sent') return`: the question went and Claude is answering; nothing to say.
- `if (outcome === 'extra-text') return this.deps.notify(extraTextHint(screenshots))`: Claude's box held other text as well, so nothing was pressed; the user checks the question and sends it.
- `const pasted = outcome === 'focus-lost' || outcome === 'not-sent'`: these two come after the screenshot, if there was one, has been pasted, so only Enter is left. `not-ready` means nothing was pressed, and `failed` is treated the same.
- `screenshots > 0 && !pasted ? pasteHint(screenshots) : sendHint()`: the notification that matches what's left:
  - nothing pressed and there are screenshots: paste them, then send (`pasteHint`);
  - nothing pressed and no screenshots: press Enter (`sendHint`);
  - pasted but not sent: press Enter (`sendHint`).

`src/main/sendInClaude.ts`: pressing Ctrl+V and Enter in Claude

Sends the question once Claude Desktop shows it, by pressing the keys a person would. Node.js can't press keys in another app or see inside its window, but Windows PowerShell, which comes with Windows, can load the .NET parts of Windows that do both. So this file builds a short PowerShell script, runs it hidden, and reads back one word saying how it went.

Two parts of Windows do the work:

- **UI Automation** is Windows' accessibility interface, the one screen readers use to find out what's on screen. Through it, any program can ask which element (button, text box) has the keyboard focus, which process it belongs to, and what text it holds. That's how the script makes sure it's looking at Claude Desktop's message box with the question in it.
- **SendKeys** (from Windows Forms) types keys into whichever window has the focus, as if they came from the keyboard.

`SendOutcome` and `SendOptions`

src/main/sendInClaude.ts, lines 5–28

```ts
/**
 * How sending the question in Claude Desktop went.
 * - `sent`: the question left Claude's text box after Enter.
 * - `not-ready`: Claude's focused text box never showed the question (Claude didn't come to the
 *   front, or the focus is somewhere else), so nothing was pressed.
 * - `extra-text`: Claude's text box holds the question and something else (Claude adds a linked
 *   question to an unsent draft), so nothing was pressed: the user should check it first.
 * - `focus-lost`: the screenshot was pasted, but then the focus moved away, so Enter wasn't pressed.
 * - `not-sent`: Enter was pressed but the question stayed in the text box.
 * - `failed`: the helper couldn't run.
 */
export type SendOutcome = 'sent' | 'not-ready' | 'extra-text' | 'focus-lost' | 'not-sent' | 'failed'

export interface SendOptions {
  question: string
  /** Press Ctrl+V first, to add the screenshot that was copied to the clipboard. */
  paste: boolean
  /** The process whose text box must hold the question. Tests use another app. */
  processName?: string
  /** How long to wait for Claude to show the question, in ms. */
  readyTimeoutMs?: number
  /** After pasting, how long Claude gets to take the screenshot in before Enter, in ms. */
  pasteSettleMs?: number
}
```

`SendOutcome` is the word the script ends with (the comment explains each). `failed` is never printed by the script itself: it's what anything else (an error, no output) comes to. `extra-text` is the "why exactly the question?" case from the top of this page.

- `question: string`: the question as typed. The script gets only its start (`questionKey()`) and a fingerprint of the whole of it (`questionHash()`), never the question itself.
- `paste: boolean`: whether to press Ctrl+V before Enter. `ClaudeDesktop.finish()` sets it when screenshots were copied.
- `processName?: string`: the program whose text box must hold the question: `claude`, Claude Desktop's process, unless a test names another.
- `readyTimeoutMs?`, `pasteSettleMs?`: the two waits below, which tests can shorten.

`READY_TIMEOUT_MS` and `PASTE_SETTLE_MS`

src/main/sendInClaude.ts, lines 30–31

```ts
export const READY_TIMEOUT_MS = 20_000

export const PASTE_SETTLE_MS = 2_500
```

- `READY_TIMEOUT_MS = 20_000`: how long to wait for Claude to show the question: 20 seconds, to allow for Claude Desktop starting up if it wasn't already running. If it hasn't by then, nothing is pressed (`not-ready`).
- `PASTE_SETTLE_MS = 2_500`: after Ctrl+V, Claude needs a moment to take the image in. Pressing Enter too soon could send the question without it.

`WHITESPACE` and `normalizeQuestion()`

src/main/sendInClaude.ts, lines 33–46

```
/**
 * Whitespace as the PowerShell side sees it too (spelled out, because JavaScript's `\s` and
 * .NET's differ on a couple of rare characters).
 */
const WHITESPACE =
  '[\\t\\n\\v\\f\\r \\u0085\\u00a0\\u1680\\u2000-\\u200a\\u2028\\u2029\\u202f\\u205f\\u3000\\ufeff]+'

/**
 * The question as it's compared with Claude's text box: every run of whitespace made one space
 * and the ends trimmed (Claude's editor may show line breaks differently from the text box here).
 */
export function normalizeQuestion(question: string): string {
  return question.replace(new RegExp(WHITESPACE, 'g'), ' ').trim()
}
```

The question in the form it's compared with Claude's text box. The script reads the box's text, evens it out the same way, and compares the two, so both sides must agree exactly on what counts as whitespace.

- `WHITESPACE`: a *character class* (`[...]`, "any one of these") listing every whitespace character by its code: tab, line breaks, the ordinary space, non-breaking spaces and other Unicode spaces. `+` means "one or more in a row". As the comment says, it's spelled out because JavaScript's shorthand `\s` and .NET's (which PowerShell uses) disagree on a few rare characters; written out, the same text means the same thing to both.
- `'[\\t\\n...'`: the backslashes are doubled because this is a JavaScript string: the string itself holds `\t`, `\u00a0` and so on, which is what both regular-expression engines expect. The same text is pasted into the PowerShell script.
- `question.replace(new RegExp(WHITESPACE, 'g'), ' ').trim()`: every run of whitespace becomes one space (`'g'` means every match, not just the first), and the ends are trimmed. Claude's editor may show line breaks differently from the text box here, and this makes that not matter.

`questionKey()`

src/main/sendInClaude.ts, lines 48–51

```
/** How to recognise the question in Claude's text box: its start, whitespace evened out. */
export function questionKey(question: string): string {
  return normalizeQuestion(question).slice(0, 60)
}
```

How the script spots the question in Claude's text box: the first 60 characters of the evened-out question, plenty to tell it from anything else in a text box. The script only looks closer (with the hash below) at a box that contains this.

`questionHash()`

src/main/sendInClaude.ts, lines 53–59

```
/**
 * A fingerprint of the whole question, so the helper can tell the text box holds exactly the
 * question (and nothing else) without the question itself going on its command line.
 */
export function questionHash(question: string): string {
  return createHash('sha256').update(normalizeQuestion(question), 'utf8').digest('hex')
}
```

A *fingerprint* of the whole evened-out question: SHA-256, a hash function that turns any text into 64 hexadecimal characters. The same text always gives the same hash, and any change, even one extra character, gives a completely different one. The script hashes the text it finds in Claude's box and compares: equal means the box holds exactly the question.

- Why not pass the whole question and compare the text? As the comment says, so the question itself never goes on PowerShell's command line, where other programs on the PC can see the command lines of running processes. The hash can't be turned back into the question.
- `createHash('sha256').update(..., 'utf8').digest('hex')`: Node's built-in hashing: hash the text as UTF-8 bytes and write the result in hexadecimal. The script does the same with .NET's `SHA256` class.

`sendScript()`

src/main/sendInClaude.ts, lines 61–132

```
/**
 * The PowerShell that presses the keys. It uses Windows UI Automation to find the focused text
 * box, and only presses anything while that box belongs to Claude Desktop and already contains the
 * question; so a key never lands in another app, or in another Claude box (such as a Claude Code
 * session in the same window). The question goes in base64, never as PowerShell code.
 */
export function sendScript(options: SendOptions): string {
  const key = Buffer.from(questionKey(options.question), 'utf8').toString('base64')
  const hash = questionHash(options.question)
  const processName = options.processName ?? 'claude'
  if (!/^[\w.-]+$/.test(processName)) throw new Error('Unexpected process name')
  const timeout = Math.round(options.readyTimeoutMs ?? READY_TIMEOUT_MS)
  const settle = Math.round(options.pasteSettleMs ?? PASTE_SETTLE_MS)
  return `
$ErrorActionPreference = 'Stop'
Add-Type -AssemblyName UIAutomationClient, UIAutomationTypes, System.Windows.Forms
$key = [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('${key}'))
$hash = '${hash}'
$processName = '${processName}'
$sha = [Security.Cryptography.SHA256]::Create()

# What Claude Desktop's focused text box holds: 'exact' (just the question), 'extra' (the question
# and other text), or $null (not Claude's box, or no question in it).
function BoxState {
  try {
    $el = [System.Windows.Automation.AutomationElement]::FocusedElement
    if (-not $el) { return $null }
    if ((Get-Process -Id $el.Current.ProcessId).ProcessName -ne $processName) { return $null }
    $pattern = $null
    $text = ''
    if ($el.TryGetCurrentPattern([System.Windows.Automation.ValuePattern]::Pattern, [ref]$pattern)) {
      $text = $pattern.Current.Value
    }
```

```
    if (-not $text -and $el.TryGetCurrentPattern([System.Windows.Automation.TextPattern]::Pattern,
[ref]$pattern)) {
      $text = $pattern.DocumentRange.GetText(20000)
    }
    $text = ([string]$text -replace '${WHITESPACE}', ' ').Trim()
    if (-not $text.Contains($key)) { return $null }
    $digest = -join ($sha.ComputeHash([Text.Encoding]::UTF8.GetBytes($text)) | ForEach-Object {
$_.ToString('x2') })
    if ($digest -eq $hash) { return 'exact' }
    return 'extra'
  } catch {}
  return $null
}
function QuestionBox { (BoxState) -eq 'exact' }

function WaitFor([int]$ms, [scriptblock]$test) {
  $end = (Get-Date).AddMilliseconds($ms)
  do {
    if (& $test) { return $true }
    Start-Sleep -Milliseconds 200
  } while ((Get-Date) -lt $end)
  return $false
}

if (-not (WaitFor ${timeout} { BoxState })) { 'not-ready'; exit }
# Claude adds a linked question to whatever was already in its text box: send only the question.
if ((BoxState) -eq 'extra') { 'extra-text'; exit }
if (${options.paste ? '$true' : '$false'}) {
  [System.Windows.Forms.SendKeys]::SendWait('^v')
  Start-Sleep -Milliseconds ${settle}
  if (-not (QuestionBox)) { 'focus-lost'; exit }
}
# Enter sends. If the question is still there (Claude may still be taking in a screenshot), try
# again, but only while the box still holds it.
for ($try = 0; $try -lt 3; $try++) {
  [System.Windows.Forms.SendKeys]::SendWait('{ENTER}')
  if (WaitFor 2000 { -not (QuestionBox) }) { 'sent'; exit }
}
'not-sent'
`
}
```

Builds the PowerShell script, as one string. The TypeScript at the top fills in a few values; the rest is the script itself, in a template literal (the backquoted string, where `${...}` inserts a value).

Filling in the values, safely

- `Buffer.from(questionKey(options.question), 'utf8').toString('base64')`: the question's start goes into the script as base64, and the script decodes it (`[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('...'))`). Base64 uses only letters, digits, `+`, `/` and `=`, none of which can end the quoted string around it. So whatever the question says (quotes, `$(...)`, backticks, `; exit`), it can only ever be data, never PowerShell code.
- `if (!/^[\w.-]+$/.test(processName)) throw`: the process name is written into the script as it is, so it may only be letters, digits, `_`, `.` and `-`. A name with a quote in it is refused.
- `const hash = questionHash(options.question)`: the whole question's fingerprint, written in as `$hash`. It's only hexadecimal digits, so it can't break out of its quotes either.
- `Math.round(...)`: the two waits are written in as whole numbers.
- `'${WHITESPACE}'`: the same whitespace class as `normalizeQuestion()`, so the script evens out the box's text exactly as the question was.

- `${options.paste ? '$true' : '$false'}`: writes PowerShell's `$true` or `$false` into the `if` that decides whether to paste.

The script: setting up

- `$ErrorActionPreference = 'Stop'`: any error that isn't caught (see `QuestionBox`) stops the script. It then prints no outcome, so `parseOutcome()` reports `failed` rather than a half-run script carrying on.
- `Add-Type -AssemblyName UIAutomationClient, UIAutomationTypes, System.Windows.Forms`: loads UI Automation and Windows Forms (for SendKeys) into PowerShell.
- `$key = ...`, `$hash = ...`, `$processName = ...`: the question's start, its fingerprint and the process name, from above.
- `$sha = [Security.Cryptography.SHA256]::Create()`: .NET's SHA-256, created once and used for every check.

The script: `BoxState` and `QuestionBox`

`BoxState` is the check made before every key press. As its comment says, it answers `'exact'` (Claude's focused box holds just the question), `'extra'` (the question and other text) or `$null` (not Claude's box, or no question in it). `QuestionBox` is the yes/no form: is it `'exact'`?

- `[System.Windows.Automation.AutomationElement]::FocusedElement`: the element that has the keyboard focus right now, in whichever app is in front.
- `(Get-Process -Id $el.Current.ProcessId).ProcessName -ne $processName`: the element belongs to some other program, so it isn't Claude's box.
- `ValuePattern`, `TextPattern`: UI Automation's two ways a text box can give out its text: a plain box has a `Value`; a rich editor has a document, read here up to 20,000 characters. `TryGetCurrentPattern(..., [ref]$pattern)` asks whether the element supports one, and fills in `$pattern` if it does. The second is only tried if the first gave no text.
- `([string]$text -replace '${WHITESPACE}', ' ').Trim()`: the box's text evened out exactly as `normalizeQuestion()` evens out the question. `[string]` turns a missing value into empty text.
- `if (-not $text.Contains($key)) { return $null }`: the question's start isn't in the box, so this isn't the new chat's box. This tells it apart from any other Claude box, such as a Claude Code session in the same window.
- `$sha.ComputeHash([Text.Encoding]::UTF8.GetBytes($text))`: the box's text hashed as UTF-8. `ForEach-Object { $_.ToString('x2') }` writes each byte as two hexadecimal digits, and `-join` strings them together, the same form as `questionHash()` produces.
- `if ($digest -eq $hash) { return 'exact' }`, `return 'extra'`: the same fingerprint means the box holds exactly the question; otherwise it holds the question plus something else.
- `try { ... } catch {}`: if anything goes wrong while looking (the element vanishes, the process has just ended), the answer is simply "not Claude's box".
- `function QuestionBox { (BoxState) -eq 'exact' }`: true only for exactly the question. The paste and Enter steps use this, so any extra text that appears stops them too.

The script: waiting and pressing

- `function WaitFor([int]$ms, [scriptblock]$test)`: runs `$test` every 200 ms until it's true (then returns `$true`) or the time is up (`$false`). A `scriptblock`, `{ ... }`, is PowerShell's way to pass code to run later.
- `if (-not (WaitFor ${timeout} { BoxState })) { 'not-ready'; exit }`: waits up to 20 seconds for Claude to come to the front with the question in its focused box (exactly, or with other text). If it never does, nothing is

pressed. A bare string, such as `'not-ready'`, is PowerShell's way of printing it.

- `if ((BoxState) -eq 'extra') { 'extra-text'; exit }`: as the comment says, Claude added the question to text that was already in its box. Nothing is pressed, so an unsent draft is never sent along with the question.
- `[System.Windows.Forms.SendKeys]::SendWait('^v')`: Ctrl+V (`^` means Ctrl in SendKeys) pastes the screenshot from the clipboard. `SendWait` waits until the app has processed the keys.
- `Start-Sleep -Milliseconds ${settle}`, then `if (-not (QuestionBox)) { 'focus-lost'; exit }`: gives Claude time to take the image in, then checks again that the focus is still in Claude's box before pressing Enter. If the user clicked elsewhere meanwhile, it stops.
- `for ($try = 0; $try -lt 3; $try++)`: presses Enter up to three times. After each press it waits up to 2 seconds for the question to leave the box (`WaitFor 2000 { -not (QuestionBox) }`), and only presses again if the focused Claude box still holds the question, for example because Claude was still taking in the screenshot and didn't send yet.
- `'sent'`: printed as soon as the focused element no longer passes `QuestionBox`, normally because Claude sent the question and emptied its box. (Should the focus move away at that instant, it looks the same, but then no further key is pressed either.)
- `'not-sent'`: three Enters and the question is still there.

`OUTCOMES` and `parseOutcome()`

`src/main/sendInClaude.ts`, lines 134–140

```ts
const OUTCOMES: SendOutcome[] = ['sent', 'not-ready', 'extra-text', 'focus-lost', 'not-sent']

/** The outcome the script printed last, or `failed`. */
export function parseOutcome(output: string): SendOutcome {
  const last = output.trim().split(/\r?\n/).at(-1)?.trim()
  return OUTCOMES.find((outcome) => outcome === last) ?? 'failed'
}
```

Reads the script's answer from everything it printed.

- `output.trim().split(/\r?\n/).at(-1)?.trim()`: the last line. PowerShell ends lines with `\r\n`, and may print warnings before the outcome; only the last line counts.
- `OUTCOMES.find((outcome) => outcome === last) ?? 'failed'`: one of the four words the script prints, or `failed` for anything else, such as an error message or nothing at all.

`sendInClaude()`

src/main/sendInClaude.ts, lines 142–177

```ts
/**
 * Sends the question in Claude Desktop: waits until Claude's text box shows it, then presses
 * Ctrl+V (when there's a screenshot) and Enter. Runs the script in Windows PowerShell, hidden.
 */
export function sendInClaude(options: SendOptions): Promise<SendOutcome> {
  const script = sendScript(options)
  const powershell = join(
    process.env['SystemRoot'] ?? 'C:\\Windows',
    'System32',
    'WindowsPowerShell',
    'v1.0',
    'powershell.exe',
  )
  const encoded = Buffer.from(script, 'utf16le').toString('base64')
  const timeoutMs = (options.readyTimeoutMs ?? READY_TIMEOUT_MS) + 30_000
  return new Promise((resolve, reject) => {
    const child = spawn(
      powershell,
      ['-NoProfile', '-NonInteractive', '-ExecutionPolicy', 'Bypass', '-EncodedCommand', encoded],
      { windowsHide: true, timeout: timeoutMs },
    )
    let output = ''
    let errors = ''
    child.stdout.on('data', (chunk: Buffer) => (output += chunk.toString()))
    child.stderr.on('data', (chunk: Buffer) => (errors += chunk.toString()))
    child.on('error', reject)
    child.on('close', (code) => {
      const outcome = parseOutcome(output)
      if (outcome === 'failed') {
        reject(
          new Error(`Sending in Claude failed (exit ${code}): ${errors.trim() || output.trim()}`),
        )
      } else resolve(outcome)
    })
  })
}
```

Runs the script and resolves with its outcome. `index.ts` wraps it to log anything but `sent`, and gives it to `ClaudeDesktop` as its `sendInClaude` dependency.

- `join(process.env['SystemRoot'] ?? 'C:\\Windows', 'System32', 'WindowsPowerShell', ...)`: the full path of Windows PowerShell, which every Windows PC has. Giving the full path, rather than just `powershell.exe`, means a different program of that name earlier on the PATH can't be run instead.
- `Buffer.from(script, 'utf16le').toString('base64')`: the script for `-EncodedCommand`, which takes base64 of UTF-16 text. Passing it this way means nothing in the script needs quoting on the command line.
- `-NoProfile`: doesn't run the user's PowerShell profile script, so the helper starts faster and behaves the same on every PC. `-NonInteractive`: never waits for input. `-ExecutionPolicy Bypass`: sets PowerShell's script policy for this one run only, so the PC's own setting doesn't get in the way.
- `windowsHide: true`: no console window flashes up.
- `timeout: timeoutMs`: Node stops PowerShell if it's still running 30 seconds after the ready wait should have ended, so a stuck helper can't linger.
- `child.stdout.on('data', ...)`, `child.stderr.on('data', ...)`: collects what the script prints, and its error messages.

- `child.on('error', reject)`: PowerShell couldn't be started at all.
- `child.on('close', (code) => {`: once it has finished, the last line printed is the outcome. A `failed` outcome rejects the promise with the exit code and the error output, which `ClaudeDesktop.finish()` logs; anything else resolves.

What's tested

`tests/claudeDesktop.test.ts` runs `ClaudeDesktop` with fake dependencies that record what happened, in order (`fakeDeps()`). So a test can check not only that the screenshots were copied and the link opened, but which happened first. "Send in Claude" is off in the fakes unless a test says what sending comes to.

- `newChatLink`: the documented link with `q`; spaces written as `%20` (never `+`) and every character that means something in a link encoded; any text (emoji, other scripts, tabs, line breaks) decodes back unchanged; no `q` when there's no text.
- `pasteHint`, `sendHint`: the title for one and for several screenshots, and that the bodies mention Ctrl+V and Enter.
- `ClaudeDesktop.ask`, with "Send in Claude" off: a question without screenshots only opens the link (no clipboard, no notification); with screenshots, copy, then open, then the paste notification; a screenshot-only question opens an empty chat; an empty question does nothing; exactly 12,000 characters is accepted and one more is refused; when Claude Desktop isn't installed nothing is touched; a missing screenshot stops Claude opening; and a link Windows can't open, or a clipboard error, is `failed`, logged, with no notification.
- `ClaudeDesktop.ask`, with "Send in Claude" on: the question is sent and nothing more is said; screenshots are copied before opening and pasted when sending; `ask()` returns while sending carries on (`idle()` waits for it); when Claude never showed the question, the right hint (paste, or Enter); when the screenshot was pasted but not sent, press Enter; a helper that couldn't run is logged and falls back to the hint; a screenshot alone is never sent; and when Claude's box already had other text, nothing is sent and the "check your question" notification says so.

`tests/sendInClaude.test.ts` checks the helper without running it:

- `questionKey`: whitespace evened out, and at most 60 characters.
- `normalizeQuestion`, `questionHash`: non-breaking and other Unicode spaces are evened out too; the hash covers the whole evened-out question, so any extra text changes it.
- `sendScript`: a question full of PowerShell (`$(Remove-Item ...)`, quotes, backticks) appears in the script only as base64, which decodes back to its start; the script looks for process `claude` and waits 20 seconds; no key is pressed before the `not-ready` check, nor before the `extra-text` one; Ctrl+V only when there's a screenshot; another process name can be given, but not one that could break out of its quotes.
- `parseOutcome`: the last line is the outcome, and anything else is `failed`.

Two other test files cover the rest of 3.1. `tests/screenshotStack.test.ts` checks the stacking (see The draft, screenshots and safe files): the width rule, the order of the images and the grey band between them (pixel by pixel), a single image left as it is, and refusing different widths, padded rows or an empty list. `tests/tenants.test.ts` checks that `chatApp` defaults to the built-in chat, which needs `signIn` and `claudeAccess`, while Claude Desktop needs neither.

The unit tests can't reach what needs a real Windows desktop with Claude Desktop: asking Windows about `claude://`, the real clipboard, the notification, and the PowerShell helper actually finding Claude's box and pressing keys. "Checking it works" in `docs/CLAUDE_DESKTOP_SETUP.md` goes through them by hand.

Your apps: the JumpCloud apps list

Feature 3.2 adds **Your apps** to the card beside the bubble: the apps the user can open from their company's JumpCloud User Portal (Slack, Atlassian and so on), each with its logo. Clicking one opens it in the default browser, signed in through JumpCloud just as it would be from the portal itself. A tenant gives a `portal` (see `PortalSchema` in Main process: startup, IPC and app plumbing) and chooses where the list goes. With `"startPage": "apps"` (Morse Micro), the panel opens on the list, and the text box for asking Claude is behind the **Ask Claude** icon (a speech bubble). Otherwise the panel opens on the text box, and an **Apps** icon (a grid of squares) in `actions` swaps it for the list.

The list comes from **JumpCloud's "MCP Server for Users"** (`https://usermcp.jumpcloud.com/v1`), which an admin turns on in the JumpCloud Admin Portal. *MCP*, the Model Context Protocol, is an open standard for giving AI assistants access to tools and data: a server offers named *tools*, and a client calls them with JSON arguments. Desktop Assist is simply a client here, with no AI involved: it calls two of the server's tools itself, `list_applications` and `launch_application`. It talks to the server through the official MCP SDK (`@modelcontextprotocol/sdk`).

Connecting, once per person. The server only answers a signed-in user, so each person connects Desktop Assist to their portal once, in the browser. This is OAuth, the same family of standards as the built-in chat's JumpCloud sign-in (JumpCloud sign-in), and the MCP SDK carries out its steps:

- **Discovery**: the SDK asks the MCP server where its sign-in (authorization) server is, and reads that server's addresses.
- **Dynamic client registration**: the first time, Desktop Assist registers itself with that server and gets a client ID back. Nothing has to be set up for the app in JumpCloud beforehand.
- **A public client with PKCE**: an app installed on people's PCs can't keep a secret, so it has none. Instead each sign-in uses PKCE, a one-time secret the app makes up, sends only a hash of at the start, and reveals when it swaps the code for tokens; a stolen code is useless without it.
- **A loopback redirect**: the browser comes back to `http://127.0.0.1:47622/callback`, a tiny web server the app runs on this PC just for the sign-in (the same `listenForRedirect()` as the built-in chat's sign-in, on a port of its own).
- **Refresh tokens**: the tokens are saved encrypted (Windows DPAPI) in `jumpcloud-apps.bin` in the app's data folder. When the access token stops working, the SDK gets a new one with the refresh token, so the user stays connected; only if that fails does the list ask them to sign in again.

No token ever reaches the page: it only sees the list's state and the apps' names and logos.

Shared with Jira. None of this connecting is particular to JumpCloud: Atlassian's Remote MCP server, which Milestone 5's Jira page and service desk use, signs people in the same way. So it lives in `src/main/mcp/`: `McpAccount` (connecting once, then calling tools), `mcpConnector()` (the MCP SDK) and `payloadOf()` (reading a tool's answer). `PortalApps` and `Atlassian` (see Jira and the service desk) each have their own `McpAccount`, with a file and a port of their own.

How it fits together

1. **Showing the list**: at startup `index.ts` already asks `PortalApps.get()` to load the list in the background. It connects with the saved sign-in (`mcpConnector().connect()`), calls `list_applications`, reads the answer (`parseAppList()`), fetches the logos, and broadcasts the `ready` list on `assist:apps-state`. Whenever the panel shows the list (on opening, or through the Apps icon), `Panel` asks again with `window.assist.apps.get()` → IPC `assist:apps-get`, and gets the list already loaded, until Refresh.

2. **Connecting**: with no saved sign-in, the list shows **Sign in with JumpCloud** → `apps.signIn()` → `PortalApps.signIn()` → `McpAccount.signIn()`. It starts the local listener, lets the SDK register and build the sign-in address (`authorize()`), opens it in the browser, waits for the browser to come back with a code, checks it's the sign-in it started (`state`), swaps the code for tokens (`authorize()` again, with the code) and loads the list. `index.ts` reopens the panel when it's done.
3. **Opening an app**: a click → `Panel.openApp()` → IPC `assist:apps-open` with the app's ID → `PortalApps.open()` calls `launch_application` for that app, takes the sign-in link from the answer, and opens it in the browser; the panel closes. JumpCloud can take a few seconds, so meanwhile a spinner turns over the app's logo. If it takes more than 15 seconds, the link the list itself gave is used instead.
4. **Favourites**: the star on a tile → `Panel.toggleFavorite()` → `window.assist.settings.setFavoriteApp()` saves the app's ID with the user's preferences (`favoriteApps` in `preferences.json`). The list shows starred apps first (`orderApps()` in `src/shared/apps.ts`), and they stay starred after a restart.

Reading answers defensively. JumpCloud documents the server's tools but not every field of their answers. So `portalData.ts` reads them tolerantly: it looks for the usual names for a list, an ID, a name, a logo and a link, takes only `https` links, and ignores what it doesn't know. If no list can be found at all, the log gets the field names it did find (never the values), so the reader can be fixed.

`src/main/apps/portalData.ts`: reading JumpCloud's answers

Plain functions with no Electron or network in them, so `tests/portalApps.test.ts` checks them with made-up answers. `PortalApps` uses them on what the tools return.

`PortalAppInfo`

src/main/apps/portalData.ts, lines 6–13

```ts
/** An app from the portal, before its logo is fetched. */
export interface PortalAppInfo {
  id: string
  name: string
  logoUrl: string | null
  /** The sign-in link the list itself gave, if any (launch_application is asked first). */
  launchUrl: string | null
}
```

- The comment at the top of the file sums up the approach: look for the usual names, ignore the rest. A tool's answer is first turned into data by `payloadOf()`, from `src/main/mcp/toolResult.ts` (below).
- `PortalAppInfo`: one app as the main process keeps it. It has two things the page never sees: the logo's web address (the page gets the logo itself, as a `data:` URL) and `launchUrl`, a sign-in link the list may have included, kept as a fallback for `launch_application`.

`Item`, `LIST_KEYS`, `URL_KEYS`, `isObject` and `text`

src/main/apps/portalData.ts, lines 15–24

```
type Item = Record<string, unknown>

const LIST_KEYS = ['applications', 'apps', 'items', 'results', 'data', 'records']

const URL_KEYS = ['launchUrl', 'launch_url', 'url', 'ssoUrl', 'sso_url', 'redirectUrl', 'location']

const isObject = (value: unknown): value is Item =>
  typeof value === 'object' && value !== null && !Array.isArray(value)

const text = (value: unknown): string | null =>
  typeof value === 'string' && value.trim() ? value.trim() : null
```

Small helpers for poking around in data of unknown shape.

- `type Item = Record<string, unknown>`: a plain object whose fields could be anything.
- `LIST_KEYS`: the field names an answer might keep its list of apps under. `URL_KEYS`: the names a sign-in link might have.
- `isObject`: a *type guard*, like `COMMAND_ACTION_IDS`'s: when it returns true, TypeScript treats the value as an `Item`, so its fields can be read. Arrays and `null` don't count (`typeof` says `'object'` for both).
- `text`: a string with something in it, trimmed, or `null`.

`httpsUrl()`

src/main/apps/portalData.ts, lines 26–35

```
/** Only web addresses are ever opened from the list (never file:, javascript: and so on). */
export function httpsUrl(value: unknown): string | null {
  const candidate = text(value)
  if (!candidate) return null
  try {
    return new URL(candidate).protocol === 'https:' ? candidate : null
  } catch {
    return null
  }
}
```

The one gate every link from JumpCloud goes through. Only `https:` web addresses pass, so an answer can never make the app open a local file (`file:`), run script (`javascript:`) or start another program through its own scheme. `new URL()` throws for text that isn't a web address, which counts as "no".

`findList()` and `appFrom()`

src/main/apps/portalData.ts, lines 37–77

```
/** The list inside the answer: the answer itself, or an array under a usual key. */
function findList(value: unknown): unknown[] | null {
  if (Array.isArray(value)) return value
  if (!isObject(value)) return null
  for (const key of LIST_KEYS) {
    const found = findList(value[key])
    if (found) return found
  }
  return Object.values(value).find((v) => Array.isArray(v) && v.some(isObject)) as unknown[] | null
}

function appFrom(item: unknown): PortalAppInfo | null {
  if (!isObject(item)) return null
  const sso = isObject(item['sso']) ? item['sso'] : {}
  if (sso['hidden'] === true || item['hidden'] === true) return null // "Show in User Portal" off
  const logo = item['logo']
  const id = text(item['id']) ?? text(item['_id']) ?? text(item['applicationId'])
  const name =
    text(item['displayLabel']) ??
    text(item['displayName']) ??
    text(item['display_name']) ??
    text(item['name']) ??
    text(item['label'])
  if (!id || !name) return null
  const bookmark = isObject(sso['bookmark']) ? sso['bookmark'] : {}
  return {
    id,
    name,
    logoUrl:
      (isObject(logo) ? httpsUrl(logo['url']) : httpsUrl(logo)) ??
      httpsUrl(item['logoUrl']) ??
      httpsUrl(item['logo_url']) ??
      httpsUrl(item['icon']),
    launchUrl:
      httpsUrl(item['ssoUrl']) ??
      httpsUrl(item['launchUrl']) ??
      httpsUrl(sso['url']) ??
      httpsUrl(bookmark['url']) ??
      httpsUrl(item['url']),
  }
}
```

`findList()` finds the list of apps in an answer, and `appFrom()` reads one app from it.

- `if (Array.isArray(value)) return value`: the answer may simply be the list.
- `for (const key of LIST_KEYS)`, `findList(value[key])`: otherwise, a list under one of the usual names, looked for at any depth (`findList` calls itself).
- `Object.values(value).find((v) => Array.isArray(v) && v.some(isObject))`: as a last resort, any field that holds a list of objects.
- `const sso = isObject(item['sso']) ? item['sso'] : {}`: JumpCloud keeps an app's single sign-on settings in `sso`. Using `{}` when there isn't one means the lines below can read `sso[...]` without checking again.
- `if (sso['hidden'] === true || item['hidden'] === true) return null`: an app whose "Show in User Portal" setting is off is left out of the list, as it is in the portal.

- `text(item['id']) ?? text(item['_id']) ?? ...`: the first of the usual names that has a value. `displayLabel` comes first for the name, because that's the label the portal shows.
- `if (!id || !name) return null`: an item without both can't be shown or opened.
- `logoUrl`: the logo is either `{ url }` or the address itself, or under another usual name; `https` only.
- `launchUrl`: the sign-in link, if the list includes one, including a bookmark app's address (`sso.bookmark.url`); `https` only.

parseAppList()

src/main/apps/portalData.ts, lines 79–86

```
/** The apps in a list_applications answer, by name; null if it holds no list at all. */
export function parseAppList(result: ToolResult): PortalAppInfo[] | null {
  const list = findList(payloadOf(result))
  if (!list) return null
  const apps = list.map(appFrom).filter((app): app is PortalAppInfo => app !== null)
  const unique = [...new Map(apps.map((app) => [app.id, app])).values()]
  return unique.sort((a, b) => a.name.localeCompare(b.name, undefined, { sensitivity: 'base' }))
}
```

The apps in a `list_applications` answer, ready to show.

- `if (!list) return null`: `null` means "no list at all", which is a problem worth logging. An empty list (`[]`) is a perfectly good answer: the user has no apps assigned yet.
- `.filter((app): app is PortalAppInfo => app !== null)`: drops the items `appFrom()` couldn't read, and tells TypeScript the result has no `null`s.
- `[...new Map(apps.map((app) => [app.id, app])).values()]`: removes duplicates. A `Map` keeps one entry per key, so an app listed twice (under two groups, say) appears once.
- `a.name.localeCompare(b.name, undefined, { sensitivity: 'base' })`: sorts by name the way a person would, ignoring upper and lower case (`sensitivity: 'base'`).

describeShape()

src/main/apps/portalData.ts, lines 88–96

```
/** The field names in the answer's first item, for the log when no apps could be read. */
export function describeShape(result: ToolResult): string {
  const payload = payloadOf(result)
  const list = findList(payload)
  const first = list?.find(isObject)
  if (first) return `items with fields ${Object.keys(first).join(', ')}`
  if (isObject(payload)) return `an object with fields ${Object.keys(payload).join(', ')}`
  return typeof payload
}
```

For the log, when `parseAppList()` found no list: the field names in the answer's first item, or in the answer itself. Only names, never values, so nothing about the user's apps or account ends up in the log. With them, the readers above can be taught the server's actual field names.

`idArgumentName()`

src/main/apps/portalData.ts, lines 98–114

```ts
/** Which argument of a tool takes the app's ID (from its input schema). */
export function idArgumentName(inputSchema: unknown): string {
  const properties = isObject(inputSchema) && isObject(inputSchema['properties'])
  const names = properties ? Object.keys(inputSchema['properties'] as Item) : []
  const required =
    isObject(inputSchema) && Array.isArray(inputSchema['required'])
      ? (inputSchema['required'] as unknown[]).filter((n): n is string => typeof n === 'string')
      : []
  const looksLikeId = (n: string) => /^(application_?id|app_?id|id)$/i.test(n)
  return (
    required.find(looksLikeId) ??
    names.find(looksLikeId) ??
    required.find((n) => /id$/i.test(n)) ??
    names.find((n) => /id$/i.test(n)) ??
    'id'
  )
}
```

Which argument of `launch_application` takes the app's ID. Every MCP tool describes its arguments with a *JSON Schema* (`inputSchema`): an object whose `properties` lists the argument names and whose `required` says which must be given. Rather than assume a name, this reads it.

- `looksLikeId`: `application_id`, `applicationId`, `app_id`, `appId` or `id`, in any case.
- `required.find(looksLikeId) ?? names.find(looksLikeId) ?? ...`: a required argument with one of those names first, then any argument with one, then anything ending in "id", and `'id'` if all else fails.

`findLaunchUrl()`

src/main/apps/portalData.ts, lines 116–150

```
/** The sign-in link in a launch_application answer: a URL field, or the first web address in it. */
export function findLaunchUrl(result: ToolResult): string | null {
  const payload = payloadOf(result)
  const search = (value: unknown, depth: number): string | null => {
    if (depth > 4) return null
    if (typeof value === 'string') return httpsUrl(value)
    if (Array.isArray(value)) {
      for (const v of value) {
        const found = search(v, depth + 1)
        if (found) return found
      }
      return null
    }
    if (!isObject(value)) return null
    for (const key of URL_KEYS) {
      const found = httpsUrl(value[key])
      if (found) return found
    }
    for (const v of Object.values(value)) {
      const found = search(v, depth + 1)
      if (found) return found
    }
    return null
  }
  const found = search(payload, 0)
  if (found) return found
  const blocks = Array.isArray(result.content) ? result.content : []
  for (const block of blocks) {
    if (isObject(block) && typeof block['text'] === 'string') {
      const match = /https:\/\/[^\s"'<>)]+/.exec(block['text'])
      if (match) return httpsUrl(match[0])
    }
  }
  return null
}
```

The sign-in link in a `launch_application` answer.

- `const search = (value: unknown, depth: number) =>`: looks through the answer: a string that is an `https` address, the items of a list, or an object's usual link fields (`URL_KEYS`) before its other fields.
- `if (depth > 4) return null`: doesn't go deeper than a few levels, so an odd answer can't make it search for ever.
- `/https:\/\/[^\s"'<>)]+/.exec(block['text'])`: if the data held no link, the first `https` address anywhere in the text blocks (up to the first space, quote, angle bracket or closing bracket), in case the tool answers in words. It still goes through `httpsUrl()`.

`src/main/apps/PortalApps.ts`: the list, the sign-in and opening apps

The Apps list's logic: loading the list, connecting the user, and opening apps. The connecting and the tool calls are done by an `McpAccount` (below). It has no Electron in it: the connection to the server, the encrypted store, the local listener, the browser and logo fetching all come in through `PortalAppsDeps`, so `tests/portalApps.test.ts` runs it against a fake portal. `start()` in `index.ts` creates the one instance when the tenant has a `portal`, and `registerApps()` in `ipc.ts` calls it.

`LIST_TOOL`, `LAUNCH_TOOL`, `LAUNCH_TIMEOUT_MS` and `SCOPE`

src/main/apps/PortalApps.ts, lines 18–29

```
/** The portal's tools, from JumpCloud's MCP Server for Users. */
export const LIST_TOOL = 'list_applications'

export const LAUNCH_TOOL = 'launch_application'

/**
 * How long opening an app waits for JumpCloud's sign-in link before using the link the list gave
 * (so the spinner on the tile can't go on for ever).
 */
export const LAUNCH_TIMEOUT_MS = 15_000

/** Everything JumpCloud's apps server offers; offline_access keeps the user connected. */
const SCOPE = 'openid profile email offline_access organization userconsole'
```

- `LIST_TOOL`, `LAUNCH_TOOL`: the names of the two tools used, from JumpCloud's documentation of the server.
- `LAUNCH_TIMEOUT_MS = 15_000`: how long opening an app waits for `launch_application` before falling back on the link from the list, so, as the comment says, the spinner on the tile can't turn for ever.
- `SCOPE`: what the app asks to be allowed to do when the user connects (handed to `McpAccount`, which puts it in `SavedAuth.clientMetadata`). As the comment says, it's everything the apps server offers: the user's identity (`openid profile email`), `organization` and `userconsole`, and `offline_access`, which asks for a refresh token, so the user doesn't have to sign in again every time the short-lived access token runs out.

`PortalAppsDeps`

src/main/apps/PortalApps.ts, lines 31–45

```
export interface PortalAppsDeps {
  appName: string
  redirectPort: number
  connector: McpConnector
  /** The connection to the portal, saved encrypted (Windows DPAPI). */
  store: SavedSecret
  listen(port: number): Promise<RedirectListener>
  openBrowser(url: string): Promise<void>
  /** An app's logo as a data: URL, or null. */
  fetchLogo(url: string): Promise<string | null>
  onState(state: AppsState): void
  onError?(message: string, error?: unknown): void
  /** Tests make it short. */
  launchTimeoutMs?: number
}
```

`PortalAppsDeps` is everything `PortalApps` needs from outside; `start()` in `index.ts` passes the real ones (see Main process: startup, IPC and app plumbing).

- `appName`, `redirectPort`, `connector`, `store`, `listen` and `openBrowser` are handed on to the `McpAccount` (see `McpAccountDeps` below): how the app names itself when it registers with JumpCloud, the port the browser comes back to, the connection to the server, the encrypted file (`SecretStore` over `jumpcloud-apps.bin`), the local web server and the browser.
- `fetchLogo(url)`: an app's logo as a `data:` URL (`fetchLogo()` in `logos.ts`, below).
- `onState(state)`: called on every change of the list's state; `index.ts` broadcasts it.
- `onError?(message, error?)`: a line for the problem log.

- `launchTimeoutMs?`: replaces `LAUNCH_TIMEOUT_MS`, so the tests don't have to wait 15 seconds.

`PortalApps`

src/main/apps/PortalApps.ts, lines 47–52

```
/**
 * The Apps list: the apps in the user's JumpCloud User Portal, from JumpCloud's MCP Server for
 * Users, and opening one in the browser through its sign-in link. The user connects once, in the
 * browser; the connection is saved and renewed by itself.
 */
export class PortalApps {
  // …
}
```

The class `index.ts` and `ipc.ts` use, with the comment that sums it up.

`PortalApps` fields, constructor and `current`

src/main/apps/PortalApps.ts, lines 53–80

```
private state: AppsState = { status: 'loading' }

private apps: PortalAppInfo[] = []

private readonly account: McpAccount

private loading: Promise<AppsState> | null = null

private loadedOnce = false

/** Logos already fetched, by address, so a refresh doesn't download them all again. */
private readonly logos = new Map<string, string>()

/**
 * The launch tool's argument for the app ID, once known (null: the server has no launch tool).
 * Remembered so opening an app is one request to JumpCloud, not two.
 */
private launchArgument: string | null | undefined

constructor(private readonly deps: PortalAppsDeps) {
  this.account = new McpAccount({
    appName: deps.appName,
    redirectPort: deps.redirectPort,
    scope: SCOPE,
    connector: deps.connector,
    store: deps.store,
    listen: (port) => this.deps.listen(port),
    openBrowser: (url) => this.deps.openBrowser(url),
  })
}

get current(): AppsState {
  return this.state
}
```

- `state`: what the list is doing (`AppsState`). Every change goes through `setState()`, which also reports it.
- `apps`: the last list read, with the details the page doesn't get (logo addresses, links), for `open()`.

- `account`: the user's connection to the apps server (`McpAccount`, below), made in the constructor from the deps and JumpCloud's `SCOPE`. `listen` and `openBrowser` are passed as arrow functions that call `this.deps`, so each is still called as a method of `deps`.
- `loading`: the list load under way, if any, so two requests at once share one load.
- `loadedOnce`: whether the list has been loaded (or found to need a sign-in) at least once.
- `logos`: the logos already fetched, by address, as the comment says (see `logo()` below).
- `launchArgument: string | null | undefined`: the name of the launch tool's ID argument, found the first time an app is opened (see `launchUrl()` below). Three states: `undefined` means "not looked up yet", `null` "the server has no launch tool", and a string the name itself.
- `get current()`: the state as it is now, for the tests.

`PortalApps.get()`

src/main/apps/PortalApps.ts, lines 82–87

```
/** The list as it stands, loading it the first time (or again with `refresh`). */
async get(refresh = false): Promise<AppsState> {
  if (this.state.status === 'signing-in') return this.state
  if (this.loadedOnce && !refresh && this.state.status !== 'error') return this.state
  return this.load()
}
```

What the page gets whenever the list shows, or on Refresh or Retry; `index.ts` also calls it once at startup, to load the list in the background.

- `if (this.state.status === 'signing-in') return this.state`: while the user is signing in in the browser, nothing else happens; the list loads when that finishes.
- `if (this.loadedOnce && !refresh && this.state.status !== 'error') return this.state`: once loaded, the list is answered from memory, so opening the Apps list again is instant. An error is always tried again.

`PortalApps.signIn()`

src/main/apps/PortalApps.ts, lines 89–109

```
/** Connects to the portal: the user signs in in the browser, then the list loads. */
async signIn(): Promise<void> {
  if (this.state.status === 'signing-in') return
  this.setState({ status: 'signing-in' })
  try {
    await this.account.signIn()
    this.state = { status: 'loading' }
    await this.load()
  } catch (error) {
    if (error instanceof SignInCancelledError) {
      this.setState({ status: 'sign-in' })
      return
    }
    const message =
      error instanceof SignInTimeoutError
        ? 'Signing in took too long. Try again.'
        : `Signing in didn't work: ${error instanceof Error ? error.message : String(error)}`
    this.deps.onError?.('Connecting to the apps list failed:', error)
    this.setState({ status: 'sign-in', message })
  }
}
```

Connects the user's portal: **Sign in with JumpCloud**. It runs for as long as the user takes in the browser; the page isn't waiting on it, and follows along through the state.

- `if (this.state.status === 'signing-in') return`: a second click while the browser is open changes nothing.
- `await this.account.signIn()`: the browser part, in `McpAccount.signIn()` (below). It resolves once the user has signed in and the tokens are saved, and fails if they cancel, take too long or the sign-in goes wrong.
- `this.state = { status: 'loading' }`, `await this.load()`: the list loads with the new sign-in. Setting `state` directly (not through `setState()`) avoids reporting a `loading` that `load()` reports itself.
- `catch (error) {`: `SignInCancelledError` (Cancel) just goes back to the Sign in button; a timeout (the listener gives up after 5 minutes) or any other failure goes back to it with a message, and is logged.

`PortalApps.cancelSignIn()`, `open()` and `dispose()`

src/main/apps/PortalApps.ts, lines 111–134

```
cancelSignIn(): void {
  this.account.cancelSignIn()
}

/**
 * Opens the app in the browser through its sign-in link. False if there's no link for it. If
 * JumpCloud is slow to give the link, the one from the list is used instead.
 */
async open(id: string): Promise<boolean> {
  const app = this.apps.find((a) => a.id === id)
  if (!app) return false
  const launched = await withTimeout(
    this.launchUrl(app).catch(() => null),
    this.deps.launchTimeoutMs ?? LAUNCH_TIMEOUT_MS,
  )
  const url = launched ?? app.launchUrl
  if (!url) return false
  await this.deps.openBrowser(url)
  return true
}

async dispose(): Promise<void> {
  await this.account.dispose()
}
```

- `cancelSignIn()`: `McpAccount.cancelSignIn()` closes the listener, which makes `signIn()`'s wait end with `SignInCancelledError`.
- `open(id)`: only an ID from the last list is accepted. The link from `launch_application` is asked for first, through `withTimeout()` (below): if that fails (`.catch(() => null)`) or takes longer than `LAUNCH_TIMEOUT_MS`, the link the list itself gave (`app.launchUrl`) is used, if any. With no link at all, `false` lets the panel say so. Either way the call ends within about 15 seconds, so the tile's spinner always stops.
- `dispose()`: when the app quits, `McpAccount.dispose()` stops a waiting sign-in and closes the connection.

`PortalApps.load()` and `PortalApps.fetchList()`

src/main/apps/PortalApps.ts, lines 136–163

```
private load(): Promise<AppsState> {
  this.loading ??= this.fetchList().finally(() => (this.loading = null))
  return this.loading
}

private async fetchList(): Promise<AppsState> {
  if (this.state.status !== 'ready') this.setState({ status: 'loading' })
  try {
    const result = await this.call(LIST_TOOL, {})
    const apps = parseAppList(result)
    if (!apps) {
      this.deps.onError?.(
        `The apps list came back in an unexpected form: ${describeShape(result)}`,
      )
      return this.setState({ status: 'error', message: "Couldn't read your apps list." })
    }
    this.apps = apps
    this.loadedOnce = true
    return this.setState({ status: 'ready', apps: await this.withLogos(apps) })
  } catch (error) {
    if (error instanceof NeedsSignInError) {
      this.loadedOnce = true
      return this.setState({ status: 'sign-in' })
    }
    this.deps.onError?.('Loading the apps list failed:', error)
    return this.setState({ status: 'error', message: friendlyError(error) })
  }
}
```

- `this.loading ??= ...`: `??=` assigns only if `loading` is `null`. So a second request while a load is under way gets the same promise rather than starting another; `.finally()` clears it when the load ends.
- `if (this.state.status !== 'ready') this.setState({ status: 'loading' })`: on a refresh, the old list stays on screen while the new one loads, rather than flashing "Getting your apps…".
- `if (!apps) {`: the answer held no list at all. The log gets its shape (`describeShape()`), and the panel says the list couldn't be read.
- `apps: await this.withLogos(apps)`: the list only appears once the logos are in, so icons don't pop in one by one.
- `if (error instanceof NeedsSignInError)`: nobody has connected yet, or the connection has ended: the Sign in button, without a message.
- `friendlyError(error)`: any other failure, in plain English; the log gets the details.

`PortalApps.call()`

src/main/apps/PortalApps.ts, lines 165–167

```
private call(name: string, args: Record<string, unknown>): Promise<ToolResult> {
  return this.account.call(name, args)
}
```

Every tool call goes through the account: `McpAccount.call()` (below) connects first if there's no open connection, and tries once more if a kept connection has gone stale.

`PortalApps.launchUrl()`, `PortalApps.withLogos()` and `PortalApps.logo()`

src/main/apps/PortalApps.ts, lines 169–192

```
private async launchUrl(app: PortalAppInfo): Promise<string | null> {
  if (this.launchArgument === undefined) {
    const tool = (await this.account.listTools()).find((t) => t.name === LAUNCH_TOOL)
    this.launchArgument = tool ? idArgumentName(tool.inputSchema) : null
  }
  if (this.launchArgument === null) return null
  const result = await this.call(LAUNCH_TOOL, { [this.launchArgument]: app.id })
  return findLaunchUrl(result)
}

private async withLogos(apps: PortalAppInfo[]): Promise<PortalApp[]> {
  return Promise.all(
    apps.map(async ({ id, name, logoUrl }) => ({ id, name, logo: await this.logo(logoUrl) })),
  )
}

private async logo(url: string | null): Promise<string | null> {
  if (!url) return null
  const known = this.logos.get(url)
  if (known) return known
  const logo = await this.deps.fetchLogo(url).catch(() => null)
  if (logo) this.logos.set(url, logo) // a failure is tried again next time
  return logo
}
```

- `if (this.launchArgument === undefined) {`: only the first time. The server's list of tools (`this.account.listTools()`) gives the launch tool's description, and its schema the name of the ID argument. It's remembered, so every later open is a single request to JumpCloud instead of two.
- `this.launchArgument = tool ? idArgumentName(tool.inputSchema) : null`: `null` when the server has no launch tool; then `return null`, and `open()` uses the list's own link.
- `{ [this.launchArgument]: app.id }`: the arguments object, with the ID under the name the tool's schema uses. `[...]` in an object literal computes the field's name.
- `withLogos(apps)`: gets every logo at the same time (`Promise.all`); one that fails is `null`, and the list shows the app's first letter instead. Only `id`, `name` and `logo` go into the result, which is what the page receives.
- `logo(url)`: one logo. `const known = this.logos.get(url)`: one fetched before (on an earlier load, or for another app with the same logo) is used again, so Refresh doesn't download every logo again. `if (logo) this.logos.set(url, logo)`: only a logo that arrived is kept; as the comment says, a failure is tried again on the next load.

`PortalApps.setState()`

src/main/apps/PortalApps.ts, lines 194–198

```
private setState(state: AppsState): AppsState {
  this.state = state
  this.deps.onState(state)
  return state
}
```

Records the new state, reports it (`onState`), and returns it, so `fetchList()` can `return this.setState(...)`.

`withTimeout()`

src/main/apps/PortalApps.ts, lines 201–210

```
/** The promise's value, or null if it takes longer than `ms`. */
function withTimeout<T>(promise: Promise<T | null>, ms: number): Promise<T | null> {
  return new Promise((resolve) => {
    const timer = setTimeout(() => resolve(null), ms)
    void promise.then((value) => {
      clearTimeout(timer)
      resolve(value)
    })
  })
}
```

The promise's value if it arrives within `ms`, otherwise `null`. `open()` uses it so a slow JumpCloud can't keep the user waiting.

- `new Promise((resolve) => {`: a promise this function resolves itself, whichever comes first.
- `setTimeout(() => resolve(null), ms)`: when the time is up, it resolves with `null`.
- `promise.then((value) => { clearTimeout(timer); resolve(value) })`: if the value comes first, the timer is cancelled and the value passed on. A promise resolves only once, so whichever happens second changes nothing. The slow request isn't cancelled; its late answer is just ignored.
- `promise: Promise<T | null>`: the promise must not fail, since nothing here catches an error; `open()` passes one with `.catch(() => null)` already on it.

`friendlyError()`

src/main/apps/PortalApps.ts, lines 212–223

```
/** A plain-English reason for the panel. */
function friendlyError(error: unknown): string {
  const status = (error as { code?: unknown; status?: unknown } | null)?.status
  const message = error instanceof Error ? error.message : String(error)
  if (status === 403 || /\b403\b|forbidden/i.test(message)) {
    return "The apps list isn't turned on for your company yet. Ask IT to turn on JumpCloud's MCP Server
for users."
  }
  if (/fetch failed|ENOTFOUND|ECONNREFUSED|ETIMEDOUT|network/i.test(message)) {
    return "Couldn't reach JumpCloud. Check you're online and try again."
  }
  return "Couldn't get your apps from JumpCloud. Try again."
}
```

What the list says when loading fails.

- `status === 403 || /\b403\b|forbidden/i.test(message)`: a refusal (403, "forbidden") is taken to mean the usual cause, that the MCP Server for Users hasn't been turned on for the company. The message says so, and who can fix it.
- `/fetch failed|ENOTFOUND|ECONNREFUSED|ETIMEDOUT|network/i`: the usual signs of no connection.
- Anything else gets a general "try again".

`src/main/mcp/McpAccount.ts`: connecting once, then calling tools

What signing in to an MCP server in the browser and calling its tools needs, whichever server it is: `PortalApps` uses it for JumpCloud's apps server, and `Atlassian` for Atlassian's Remote MCP server (Jira and the service desk). It has no Electron in it either, so `tests/portalApps.test.ts` and `tests/atlassian.test.ts` run it against fake servers.

`McpSession`, `NeedsSignInError`, `ToolError` and `McpConnector`

`src/main/mcp/McpAccount.ts`, lines 11–38

```ts
/** A connection to an MCP server, signed in as the user. */
export interface McpSession {
  listTools(): Promise<{ name: string; inputSchema?: unknown }[]>
  callTool(name: string, args: Record<string, unknown>): Promise<ToolResult>
  close(): Promise<void>
}

/** Thrown when there's no usable sign-in: the user has to connect, in the browser. */
export class NeedsSignInError extends Error {
  constructor() {
    super('Sign-in needed')
  }
}

/** Thrown when a tool answers with an error (the server's message, if it gave one). */
export class ToolError extends Error {}

/** Talks to an MCP server (src/main/mcp/connector.ts; tests use a fake). */
export interface McpConnector {
  /** Connects with the saved sign-in (renewing it if needed); throws NeedsSignInError if none. */
  connect(provider: OAuthClientProvider): Promise<McpSession>
  /**
   * The OAuth steps. Without a code: registers Desktop Assist with the server if it isn't yet,
   * and asks the provider to send the user to the sign-in page ('REDIRECT'). With the code from
   * the browser: swaps it for tokens ('AUTHORIZED').
   */
  authorize(provider: OAuthClientProvider, code?: string): Promise<'AUTHORIZED' | 'REDIRECT'>
}
```

The small interfaces `McpAccount` talks to the server through. The real connector is `mcpConnector()` in `connector.ts`; the tests use fakes.

- `McpSession`: one open connection, signed in as the user: list the server's tools, call one, close.
- `NeedsSignInError`: the connector throws this when there's no sign-in it can use, so `PortalApps` and `Atlassian` can tell "the user has to connect" apart from every other failure (`instanceof`).
- `ToolError`: a tool answered, but with an error (`isError`). Its message is the server's own words when it gave any (`errorText()` below), so Atlassian's "issue does not exist", say, can reach the user. Being a class of its own lets `call()` tell it apart from a broken connection.
- `connect(provider)`: opens a session with the saved sign-in, renewing it if needed. The `provider` is the `SavedAuth` below, which the SDK reads the tokens from.
- `authorize(provider, code?)`: the OAuth steps, as the comment explains. It answers `'REDIRECT'` when the user needs to go to the sign-in page, and `'AUTHORIZED'` once there are tokens.

`SavedSecret` and `Saved`

src/main/mcp/McpAccount.ts, lines 40–50

```ts
/** Where a connection is saved, encrypted (Windows DPAPI). */
export interface SavedSecret {
  load(): Promise<string | null>
  save(secret: string): Promise<void>
  clear(): Promise<void>
}

interface Saved {
  client?: OAuthClientInformationMixed
  tokens?: OAuthTokens
}
```

- `SavedSecret`: where the connection is kept, seen only as load, save and clear. `index.ts` passes a `SecretStore` (see JumpCloud sign-in): over `jumpcloud-apps.bin` for the apps list, and over `atlassian.bin` for Atlassian.
- `Saved`: what goes in the file: the app's registration with the server (`client`) and the user's tokens. Both are optional, since either may not exist yet.

`SavedAuth`

src/main/mcp/McpAccount.ts, lines 52–57

```ts
/**
 * The OAuth client the MCP SDK drives: Desktop Assist as a public client (PKCE, no secret) with a
 * loopback redirect. It remembers the registration and the tokens; the sign-in page is only opened
 * by McpAccount.signIn(), never by just using the connection.
 */
export class SavedAuth implements OAuthClientProvider {
  // …
}
```

The MCP SDK does the OAuth steps, but asks its *provider* (an `OAuthClientProvider`) for everything particular to the app: what to register as, where the browser should come back to, the saved tokens, and what to do when the user has to sign in. `SavedAuth` is that provider. `implements OAuthClientProvider` makes TypeScript check it has every method the SDK expects.

As the comment says, it never opens the browser itself. When the SDK decides the user has to sign in, it calls `redirectToAuthorization()`, which here only notes the address. So merely using the connection (loading the apps list, checking Jira) can't pop up a browser window; only a Sign in or Connect button does, through `McpAccount.signIn()`.

`SavedAuth` fields, constructor and `load()`

src/main/mcp/McpAccount.ts, lines 58–79

```ts
private saved: Saved = {}

private verifier = ''

private expectedState = ''

/** Where the user has to go to sign in, once the SDK has asked for it. */
authorizationUrl: URL | null = null

constructor(
  private readonly appName: string,
  private readonly redirectPort: number,
  private readonly store: SavedSecret,
  /** The scopes to ask for; left out when the server doesn't define any. */
  private readonly scope?: string,
) {}

async load(): Promise<void> {
  try {
    const text = await this.store.load()
    this.saved = text ? (JSON.parse(text) as Saved) : {}
  } catch {
    this.saved = {}
  }
}
```

- `saved`: the registration and tokens, as in the file.
- `verifier`: the current sign-in's PKCE secret. It's needed only until the code is swapped, so it's kept in memory, not saved.
- `expectedState`: the random `state` sent with the current sign-in, to check the browser's return against.
- `authorizationUrl`: the sign-in page's address, once the SDK has asked for one.
- `scope?`: the scopes to ask for, as the comment says, left out when the server doesn't define any. `PortalApps` gives JumpCloud's `SCOPE`; `Atlassian` gives none. `private readonly` before a constructor parameter makes it a field too.
- `load()`: reads the file. A missing, undecryptable or damaged file just means "not connected" (`{}`), never an error.

`SavedAuth.redirectUrl` and `SavedAuth.clientMetadata`

src/main/mcp/McpAccount.ts, lines 81–94

```
get redirectUrl(): string {
  return `http://127.0.0.1:${this.redirectPort}/callback`
}

get clientMetadata(): OAuthClientMetadata {
  return {
    client_name: this.appName,
    redirect_uris: [this.redirectUrl],
    grant_types: ['authorization_code', 'refresh_token'],
    response_types: ['code'],
    token_endpoint_auth_method: 'none',
    ...(this.scope ? { scope: this.scope } : {}),
  }
}
```

What the app registers as. Both are *getters*, read like fields.

- `redirectUrl`: the local listener's address, which the browser returns to.
- `client_name`: the app's name, shown on the server's consent page if it shows one.
- `grant_types: ['authorization_code', 'refresh_token']`: signing in with a code from the browser, and renewing with a refresh token.
- `token_endpoint_auth_method: 'none'`: a *public client*: the app proves nothing with a secret when it fetches tokens, because it has none. PKCE protects the code instead.
- `...(this.scope ? { scope: this.scope } : {})`: the scopes, only when there are some. `...` copies an object's fields into this one, and the empty object `{}` adds nothing, so without a scope there's no `scope` field at all.

`SavedAuth.state()` and `SavedAuth.isExpectedState()`

src/main/mcp/McpAccount.ts, lines 96–104

```
state(): string {
  this.expectedState = randomBytes(16).toString('base64url')
  return this.expectedState
}

/** Whether the browser came back from the sign-in this app started. */
isExpectedState(state: string | null): boolean {
  return state !== null && this.expectedState !== '' && state === this.expectedState
}
```

OAuth's `state` check. The SDK calls `state()` when it builds the sign-in address, and the value travels to the sign-in server and back on the browser's return. `isExpectedState()` then makes sure the browser came back from the sign-in this app started, not a link someone else sent the browser to.

- `randomBytes(16).toString('base64url')`: 16 random bytes, written with letters, digits, `-` and `_`, so it's safe in a web address.
- `this.expectedState !== ''`: before any sign-in has started, nothing is accepted.

Saving the registration and the tokens

src/main/mcp/McpAccount.ts, lines 106–122

```
clientInformation(): OAuthClientInformationMixed | undefined {
  return this.saved.client
}

async saveClientInformation(client: OAuthClientInformationMixed): Promise<void> {
  this.saved = { ...this.saved, client }
  await this.persist()
}

tokens(): OAuthTokens | undefined {
  return this.saved.tokens
}

async saveTokens(tokens: OAuthTokens): Promise<void> {
  this.saved = { ...this.saved, tokens }
  await this.persist()
}
```

The SDK reads the registration and tokens through these, and hands over new ones (after registering, signing in or renewing) to be saved. Each save writes the file at once, so a renewed token isn't lost if the app quits.

`SavedAuth.redirectToAuthorization()`, `saveCodeVerifier()` and `codeVerifier()`

src/main/mcp/McpAccount.ts, lines 124–134

```
redirectToAuthorization(authorizationUrl: URL): void {
  this.authorizationUrl = authorizationUrl
}

saveCodeVerifier(verifier: string): void {
  this.verifier = verifier
}

codeVerifier(): string {
  return this.verifier
}
```

- `redirectToAuthorization(authorizationUrl)`: only notes the address; `PortalApps.signIn()` opens it.
- `saveCodeVerifier()`, `codeVerifier()`: the SDK makes the PKCE secret when it builds the sign-in address, and asks for it back when it swaps the code for tokens.

`SavedAuth.invalidateCredentials()` and `SavedAuth.persist()`

src/main/mcp/McpAccount.ts, lines 136–146

```
async invalidateCredentials(scope: 'all' | 'client' | 'tokens' | 'verifier' | 'discovery') {
  if (scope === 'all' || scope === 'client') this.saved = {}
  else if (scope === 'tokens') this.saved = { client: this.saved.client }
  if (scope === 'all' || scope === 'verifier') this.verifier = ''
  await this.persist()
}

private async persist(): Promise<void> {
  if (this.saved.client || this.saved.tokens) await this.store.save(JSON.stringify(this.saved))
  else await this.store.clear()
}
```

- `invalidateCredentials(scope)`: the SDK calls this when the server rejects something saved: the registration (`client`, or `all`), so the app registers again, or the tokens, so the user signs in again. It then tries once more. `McpAccount.signOut()` calls it too, with `'all'`.
- `{ client: this.saved.client }`: forgetting the tokens keeps the registration.
- `persist()`: writes the file, or deletes it when there's nothing left to keep.

`McpAccountDeps`

src/main/mcp/McpAccount.ts, lines 149–158

```
export interface McpAccountDeps {
  appName: string
  redirectPort: number
  /** The scopes to ask for, if the server defines any. */
  scope?: string
  connector: McpConnector
  store: SavedSecret
  listen(port: number): Promise<RedirectListener>
  openBrowser(url: string): Promise<void>
}
```

Everything `McpAccount` needs from outside, which `PortalApps` and `Atlassian` pass on from their own deps.

- `appName`: how the app names itself when it registers with the server.
- `redirectPort`: the local port the browser comes back to: 47622 for the apps list, and the tenant's `atlassian.redirectPort` for Atlassian, so two sign-ins can never collide.
- `scope?`: the scopes to ask for, if the server defines any (see `SavedAuth`).
- `connector`, `store`: the connection to the server and the encrypted file, as above.
- `listen(port)`: starts the local web server for the browser to come back to (`listenForRedirect()`).
- `openBrowser(url)`: opens the sign-in page in the user's browser.

`McpAccount`

`src/main/mcp/McpAccount.ts`, lines 160–165

```ts
/**
 * The user's connection to an MCP server that signs people in with OAuth in the browser (the
 * JumpCloud apps server, Atlassian's): connecting once, then calling tools with the saved
 * connection, which the SDK renews by itself.
 */
export class McpAccount {
  // …
}
```

The connection as its users see it, with the comment that sums it up: connect once in the browser, then call tools; renewing the tokens is the SDK's business.

`McpAccount` fields and constructor

`src/main/mcp/McpAccount.ts`, lines 166–174

```ts
private readonly auth: SavedAuth

private readonly loaded: Promise<void>

private session: McpSession | null = null

private listener: RedirectListener | null = null

constructor(private readonly deps: McpAccountDeps) {
  this.auth = new SavedAuth(deps.appName, deps.redirectPort, deps.store, deps.scope)
  this.loaded = this.auth.load()
}
```

- `auth`: the `SavedAuth` above, made in the constructor with the deps' name, port, store and scope.
- `loaded`: the promise of reading the saved connection, started in the constructor. Everything that needs it waits for it first (`await this.loaded`).
- `session`: the open connection to the server, reused between calls, or `null`.
- `listener`: the local web server while a sign-in is waiting for the browser.

`McpAccount.signIn()`

src/main/mcp/McpAccount.ts, lines 176–206

```
/**
 * Connects in the browser: resolves once signed in. Rejects with SignInCancelledError if the
 * user cancels, SignInTimeoutError if they take too long, or an Error saying what went wrong.
 */
async signIn(): Promise<void> {
  try {
    await this.loaded
    this.listener = await this.deps.listen(this.deps.redirectPort)
    this.auth.authorizationUrl = null
    if ((await this.deps.connector.authorize(this.auth)) === 'REDIRECT') {
      const url = this.auth.authorizationUrl as URL | null
      if (!url) throw new Error('The sign-in page address is missing')
      await this.deps.openBrowser(url.href)
      const back = new URL(await this.listener.result)
      const problem = back.searchParams.get('error_description') ?? back.searchParams.get('error')
      if (problem) throw new Error(problem)
      if (!this.auth.isExpectedState(back.searchParams.get('state'))) {
        throw new Error("The sign-in didn't come from this app")
      }
      const code = back.searchParams.get('code')
      if (!code) throw new Error('The sign-in came back without a code')
      await this.deps.connector.authorize(this.auth, code)
    }
    await this.closeSession()
  } catch (error) {
    this.listener?.close()
    throw error
  } finally {
    this.listener = null
  }
}
```

Connects in the browser. It runs for as long as the user takes there, and, as the comment says, resolves once they're signed in or fails with the reason; the caller (`PortalApps.signIn()`, `Atlassian.signIn()`) turns that into the state its page shows.

- `this.listener = await this.deps.listen(this.deps.redirectPort)`: the local web server starts first, so it's ready before the browser could come back.
- `this.auth.authorizationUrl = null`: forgets the address from any earlier try.
- `this.deps.connector.authorize(this.auth)`: no code yet. The SDK registers the app if it isn't registered, then either renews a saved sign-in (`'AUTHORIZED'`, and there's nothing for the user to do) or builds the sign-in address and hands it to `redirectToAuthorization()` (`'REDIRECT'`).
- `await this.deps.openBrowser(url.href)`: the user signs in, in their own browser (at JumpCloud, or at Atlassian, which sends them through JumpCloud too).
- `const back = new URL(await this.listener.result)`: waits for the browser to come back to the local server, and reads the address it came back to.
- `back.searchParams.get('error_description') ?? back.searchParams.get('error')`: the server reports a refusal (the user said no, say) in the address; it becomes the message.
- `if (!this.auth.isExpectedState(back.searchParams.get('state')))`: the `state` check above.
- `await this.deps.connector.authorize(this.auth, code)`: swaps the code for tokens, which `SavedAuth` saves.
- `await this.closeSession()`: any old connection is dropped, so the next call connects with the new sign-in.

- `catch (error) { this.listener?.close(); throw error }`: the listener is closed in every failure (cancelling has closed it already, and closing twice does no harm), and the error is passed on. `finally` forgets the listener either way.

`McpAccount.cancelSignIn()`, `listTools()`, `signOut()`, `dispose()` and `closeSession()`

src/main/mcp/McpAccount.ts, lines 208–252

```
cancelSignIn(): void {
  this.listener?.close()
}

async listTools(): Promise<{ name: string; inputSchema?: unknown }[]> {
  await this.loaded
  this.session ??= await this.deps.connector.connect(this.auth)
  return this.session.listTools()
}

/** Forgets the saved connection; the next call needs a new sign-in. */
async signOut(): Promise<void> {
  await this.loaded
  await this.closeSession()
  await this.auth.invalidateCredentials('all')
}

async dispose(): Promise<void> {
  this.listener?.close()
  await this.closeSession()
}

private async closeSession(): Promise<void> {
  const session = this.session
  this.session = null
  await session?.close().catch(() => {})
}
```

- `cancelSignIn()`: closing the listener makes `signIn()`'s wait end with `SignInCancelledError`.
- `listTools()`: the server's tools, connecting first if needed. `PortalApps.launchUrl()` reads the launch tool's arguments from it.
- `signOut()`: as the comment says, forgets the saved connection: the session is closed and `invalidateCredentials('all')` deletes the file, so the next call throws `NeedsSignInError`. **Disconnect Atlassian** in the settings menu ends here.
- `dispose()`: when the app quits, a waiting sign-in is stopped and the connection closed.
- `closeSession()`: forgets the session first, then closes it, ignoring a failure to close (it may be broken already).

`McpAccount.call()`

src/main/mcp/McpAccount.ts, lines 212–228

```
/** Calls a tool, connecting first if needed and once more if the connection has gone stale. */
async call(name: string, args: Record<string, unknown>): Promise<ToolResult> {
  await this.loaded
  for (let attempt = 0; ; attempt++) {
    try {
      this.session ??= await this.deps.connector.connect(this.auth)
      const result = await this.session.callTool(name, args)
      if (result.isError) throw new ToolError(errorText(result) ?? `The ${name} tool failed`)
      return result
    } catch (error) {
      // The server answered: trying again won't change its mind.
      if (error instanceof ToolError) throw error
      await this.closeSession()
      if (error instanceof NeedsSignInError || attempt > 0) throw error
    }
  }
}
```

Calls a tool, connecting first if there's no open session.

- `for (let attempt = 0; ; attempt++)`: a loop with no end condition, left by `return` or `throw`.
- `this.session ??= await this.deps.connector.connect(this.auth)`: connects only when there's no session.
- `if (result.isError) throw new ToolError(...)`: a tool that reports its own failure counts as an error, with the server's words if it gave any, or else "The … tool failed".
- `if (error instanceof ToolError) throw error`: as the comment says, the server did answer, so asking again wouldn't change its mind. The session is kept, since it works.
- `await this.closeSession()`, `if (error instanceof NeedsSignInError || attempt > 0) throw error`: after any other failure the session is dropped. A connection kept open between uses can go stale (the server forgets it, say), so the first failure gets one more try with a fresh connection. A missing sign-in, or a second failure, is passed on.

`errorText()`

src/main/mcp/McpAccount.ts, lines 255–263

```
/** The text of an error answer, if it has any. */
function errorText(result: ToolResult): string | null {
  const blocks = Array.isArray(result.content) ? (result.content as unknown[]) : []
  for (const block of blocks) {
    const text = (block as { text?: unknown } | null)?.text
    if (typeof text === 'string' && text.trim()) return text.trim().slice(0, 500)
  }
  return null
}
```

The words of an error answer: the first text block with something in it, trimmed and cut to 500 characters, so a long error page can't flood a message. `null` when there are none, and `call()` uses its own words instead.

`src/main/mcp/connector.ts`: the connection, through the MCP SDK

The only file that runs the MCP SDK's client (`McpAccount.ts` only borrows its types). It turns the SDK into the `McpConnector` that `McpAccount` expects, for any server: `index.ts` makes one for JumpCloud's apps server and one for Atlassian's. It isn't unit-tested against a real server.

`Fetch` and `mcpConnector()`

src/main/mcp/connector.ts, lines 6–51

```ts
export type Fetch = (url: string | URL, init?: RequestInit) => Promise<Response>

/**
 * The real connection to an MCP server (JumpCloud's MCP Server for Users, Atlassian's Remote MCP
 * server), through the official MCP SDK: its Streamable HTTP transport, and its OAuth (discovery,
 * client registration, PKCE, token renewal). `fetch` is Electron's, so company proxies set in
 * Windows are used.
 */
export function mcpConnector(options: {
  serverUrl: string
  client: { name: string; version: string }
  fetch: Fetch
}): McpConnector {
  const { serverUrl, fetch } = options
  return {
    async connect(provider) {
      const client = new Client(options.client)
      const transport = new StreamableHTTPClientTransport(new URL(serverUrl), {
        authProvider: provider,
        fetch,
      })
      try {
        await client.connect(transport)
      } catch (error) {
        await client.close().catch(() => {})
        if (error instanceof UnauthorizedError) throw new NeedsSignInError()
        throw error
      }
      return {
        listTools: async () => (await client.listTools()).tools,
        callTool: async (name, args) => {
          const result: Record<string, unknown> = await client.callTool({ name, arguments: args })
          return {
            content: result['content'],
            // Servers on an older protocol answer with `toolResult` instead.
            structuredContent: result['structuredContent'] ?? result['toolResult'],
            isError: result['isError'] === true,
          }
        },
        close: () => client.close(),
      }
    },
    authorize: (provider, code) =>
      auth(provider, { serverUrl, authorizationCode: code, fetchFn: fetch }),
  }
}
```

- `type Fetch`: the shape of the standard `fetch()`. `index.ts` passes Electron's `net.fetch()` in this shape, so requests use a proxy set in Windows, as the comment says.
- `new Client(options.client)`: the SDK's MCP client, introducing itself with the app's name and version.

- `new StreamableHTTPClientTransport(new URL(serverUrl), { authProvider: provider, fetch })`: how the client reaches the server: *Streamable HTTP*, MCP's transport over ordinary web requests. With `authProvider`, the transport adds the saved access token to each request, and when the server answers "unauthorized" it runs the SDK's OAuth steps (renewing with the refresh token if it can).
- `await client.connect(transport)`: the MCP handshake, in which client and server agree on the protocol version.
- `if (error instanceof UnauthorizedError) throw new NeedsSignInError()`: the SDK's way of saying the user must sign in (there were no tokens, or they couldn't be renewed). `client.close()` first, so a failed connection doesn't linger.
- `callTool: async (name, args) => {`: calls a tool and returns just the parts `McpAccount` uses. As the comment says, a server on an older version of the protocol puts its data in `toolResult` instead.
- `authorize: (provider, code) => auth(provider, { ... })`: the SDK's `auth()` function, which does the OAuth steps: discovery, registration if needed, then either a code swap (with a code), a renewal, or building the sign-in address (`'REDIRECT'`).

`src/main/mcp/toolResult.ts`: a tool's answer

What a tool call returns, and reading it as data. `portalData.ts` (the apps list) and `jiraData.ts` (Jira) both start from `payloadOf()`.

`ToolResult`, `isObject` and `payloadOf()`

src/main/mcp/toolResult.ts, lines 1–25

```ts
/** What an MCP tool call returns (the parts used here). */
export interface ToolResult {
  content?: unknown
  structuredContent?: unknown
  isError?: boolean
}

const isObject = (value: unknown): value is Record<string, unknown> =>
  typeof value === 'object' && value !== null && !Array.isArray(value)

/** The tool's answer as data: its structured content, or the JSON in its first text block. */
export function payloadOf(result: ToolResult): unknown {
  if (result.structuredContent !== undefined) return result.structuredContent
  const blocks = Array.isArray(result.content) ? result.content : []
  for (const block of blocks) {
    if (isObject(block) && block['type'] === 'text' && typeof block['text'] === 'string') {
      try {
        return JSON.parse(block['text'])
      } catch {
        return block['text']
      }
    }
  }
  return undefined
}
```

- `ToolResult`: what an MCP tool call returns. A tool can answer with `content` (a list of blocks, usually text) and, in newer versions of the protocol, `structuredContent` (data as JSON). `isError` is set when the tool itself reports a failure. Both contents are typed `unknown`, because nothing about their shape is promised.

- `isObject`: the same type guard as in `portalData.ts`: a plain object, not an array or `null`.
- `payloadOf()`: a tool's answer as data. `structuredContent` is taken as it is when there is one. Otherwise the first text block is read as JSON, which is how most servers send data; if it isn't JSON, the text itself is returned (`findLaunchUrl()` can still find a link in it).

`src/main/apps/logos.ts`: the apps' logos

Fetching the logos the Apps list shows. `index.ts` hands `fetchLogo()` to `PortalApps`, with Electron's `net.fetch()` as the `fetch` to use. It's unit-tested with a fake `fetch`.

`MAX_LOGO_BYTES` and `sniffImageType()`

src/main/apps/logos.ts, lines 3–26

```ts
/** Largest logo fetched for the Apps list. */
const MAX_LOGO_BYTES = 300_000

/**
 * What kind of image a file is, from its first bytes; null if it isn't one. Needed because servers
 * don't always say: JumpCloud serves the logos a company uploads itself as
 * application/octet-stream, while its own catalogue logos come as image/png.
 */
export function sniffImageType(data: Buffer): string | null {
  const startsWith = (...bytes: number[]) => bytes.every((byte, i) => data[i] === byte)
  if (startsWith(0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a)) return 'image/png'
  if (startsWith(0xff, 0xd8, 0xff)) return 'image/jpeg'
  if (startsWith(0x47, 0x49, 0x46, 0x38)) return 'image/gif'
  if (startsWith(0x52, 0x49, 0x46, 0x46) && data.toString('latin1', 8, 12) === 'WEBP') {
    return 'image/webp'
  }
  if (startsWith(0x00, 0x00, 0x01, 0x00)) return 'image/x-icon'
  if (startsWith(0x42, 0x4d)) return 'image/bmp'
  const start = data.toString('utf8', 0, 1024).trimStart().toLowerCase()
  if (start.startsWith('<svg') || (start.startsWith('<?xml') && start.includes('<svg'))) {
    return 'image/svg+xml'
  }
  return null
}
```

What kind of image a file is, judged from the file itself. As the comment says, the server's label can't be relied on: JumpCloud serves the logos a company uploads itself (from `assets.jumpcloud.com`) labelled `application/octet-stream`, "some bytes", while its catalogue logos come labelled `image/png`. Trusting the label left the uploaded logos out. Most file formats start with a fixed *signature*, a few bytes that are always the same, and this checks for them.

- `MAX_LOGO_BYTES = 300_000`: nothing over 300 KB is fetched; a logo shown 40 pixels wide never needs more.
- `const startsWith = (...bytes: number[]) => ...`: whether the file begins with these bytes. `...bytes` collects all the arguments into one list (a *rest parameter*).
- `0x89, 0x50, 0x4e, 0x47, ...`: PNG's signature (a non-text byte, then `PNG` and line-ending characters). `0xff, 0xd8, 0xff` starts every JPEG; `GIF8` (`0x47 0x49 0x46 0x38`) every GIF; `0x00 0x00 0x01 0x00` an icon file; `BM` (`0x42 0x4d`) a Windows bitmap.
- `data.toString('latin1', 8, 12) === 'WEBP'`: a WebP file starts with `RIFF`, like several other formats, so bytes 8 to 11 are checked too.

- `data.toString('utf8', 0, 1024).trimStart().toLowerCase()`: SVG is text, not a fixed signature. The start of the file (ignoring spaces and case) must be an `<svg` element, or an XML declaration (`<?xml`) followed by one. An SVG shown as an image can't run scripts, so it's safe for the page.
- `return null`: anything else, such as an HTML error page sent in place of a logo.

`fetchLogo()`

src/main/apps/logos.ts, lines 28–39

```
/** Fetches an app's logo (https only, an image, not too big) as a data: URL for the panel. */
export async function fetchLogo(fetch: Fetch, url: string): Promise<string | null> {
  if (!url.startsWith('https://')) return null
  const response = await fetch(url, { signal: AbortSignal.timeout(10_000) })
  if (!response.ok) return null
  const data = Buffer.from(await response.arrayBuffer())
  if (data.length === 0 || data.length > MAX_LOGO_BYTES) return null
  // Trust the file's own bytes over the server's label: a label of image/* on something that
  // isn't one, or a missing label on a real image, are both common.
  const type = sniffImageType(data)
  return type ? `data:${type}base64,${data.toString('base64')}` : null
}
```

Fetches a logo and turns it into a `data:` URL for the page, which can't load images from the web itself (its Content-Security-Policy only allows the app's own images and `data:` URLs).

- `if (!url.startsWith('https://')) return null`: `https` only.
- `AbortSignal.timeout(10_000)`: gives up after 10 seconds, so one slow logo can't hold up the list for long.
- `if (!response.ok) return null`: an error answer (404, say) is no logo.
- `data.length === 0 || data.length > MAX_LOGO_BYTES`: an empty file, or one over 300 KB, is refused.
- `const type = sniffImageType(data)`: as the comment says, the file's own bytes decide what it is, not the `content-type` label: that label can be wrong both ways.
- `` `data:${type}base64,${data.toString('base64')}` ``: the image itself, written into the address with the type that was found.

`src/renderer/src/components/AppsList.tsx`: the list in the panel

The component `Panel` puts in the card while the Apps list is showing (through `ChatBox`'s `page` prop; see The pages). It shows whichever `AppsState` the main process last reported, and calls back to `Panel` for everything you can do.

`SEARCH_FROM`

src/renderer/src/components/AppsList.tsx, lines 7–8

```
/** Above this many apps, a search box appears. */
const SEARCH_FROM = 8
```

With more than 8 apps, a search box appears above them. Fewer fit on screen at a glance.

AppsList

src/renderer/src/components/AppsList.tsx, lines 10–206

```
/**
 * Shown in the card instead of the text box while the Apps icon is on: the apps in the user's
 * JumpCloud User Portal. Clicking one opens it in the default browser, signed in through
 * JumpCloud like it would be from the portal. The star in a tile's corner puts it first.
 */
export function AppsList(props: {
  open: boolean
  state: AppsState
  appName: string
  portalName: string
  /** IDs of the starred apps, shown first. */
  favorites: string[]
  onToggleFavorite: (app: PortalApp, favorite: boolean) => void
  onOpenApp: (app: PortalApp) => void
  /** The app being opened (its tile shows a spinner until the browser is opening), or null. */
  openingId: string | null
  onOpenPortal: () => void
  onSignIn: () => void
  onCancelSignIn: () => void
  onRetry: () => void
}) {
  const [query, setQuery] = useState('')
  const search = useRef<HTMLInputElement>(null)
  const { state } = props
  const apps = useMemo(
    () => (state.status === 'ready' ? orderApps(state.apps, props.favorites) : []),
    [state, props.favorites],
  )

  const shown = useMemo(() => {
    const q = query.trim().toLowerCase()
    return q ? apps.filter((app) => app.name.toLowerCase().includes(q)) : apps
  }, [apps, query])

  useEffect(() => {
    if (props.open && apps.length > SEARCH_FROM) search.current?.focus()
  }, [props.open, apps.length])

  return (
    <div className="flex min-h-0 flex-col">
      <div className="flex shrink-0 items-center gap-2 px-3.5 pt-3 pb-2">
        <LayoutGrid size={15} className="text-accent dark:text-accent-soft" aria-hidden />
        <h2 className="flex-1 text-sm font-semibold">Your apps</h2>
        {state.status === 'ready' && (
          <button
            type="button"
            onClick={props.onRetry}
            title="Refresh"
            aria-label="Refresh the list"
            className="grid size-6 place-items-center rounded-md text-zinc-500 hover:bg-zinc-100
hover:text-zinc-800 dark:hover:bg-zinc-800 dark:hover:text-zinc-200"
          >
            <RotateCw size={13} aria-hidden />
          </button>
        )}
      </div>

      {state.status === 'loading' && (
        <Message>
          <LoaderCircle size={14} className="animate-spin" aria-hidden /> Getting your apps…
```

```
        </Message>
      )}

      {state.status === 'sign-in' && (
        <div className="space-y-2 px-3.5 pb-3">
          <p className="text-xs text-zinc-500">
            {state.message ??
              `Sign in with ${props.portalName} once so ${props.appName} can show your apps.`}
          </p>
          <button
            type="button"
            onClick={props.onSignIn}
            className="inline-flex items-center gap-1.5 rounded-lg bg-accent px-3 py-1.5 text-xs font-
medium text-white"
          >
            <LogIn size={13} aria-hidden /> Sign in with {props.portalName}
          </button>
        </div>
      )}

      {state.status === 'signing-in' && (
        <div className="flex items-center gap-2 px-3.5 pb-3 text-xs text-zinc-500">
          <LoaderCircle size={14} className="animate-spin" aria-hidden />
          <span className="flex-1">Finish signing in in your browser…</span>
          <button
            type="button"
            onClick={props.onCancelSignIn}
            className="rounded-md px-1.5 py-0.5 text-zinc-600 hover:bg-zinc-100 dark:text-zinc-300
dark:hover:bg-zinc-800"
          >
            Cancel
          </button>
        </div>
      )}

      {state.status === 'error' && (
        <div className="flex items-start gap-2 px-3.5 pb-3 text-xs text-zinc-500">
          <p className="flex-1">{state.message}</p>
          <button
            type="button"
            onClick={props.onRetry}
            className="shrink-0 rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100
dark:text-accent-soft dark:hover:bg-zinc-800"
          >
            Retry
          </button>
        </div>
      )}

      {state.status === 'ready' && (
        <>
          {apps.length > SEARCH_FROM && (
            <label className="mx-3 mb-2 flex shrink-0 items-center gap-2 rounded-lg bg-zinc-100 px-2.5
py-1.5 dark:bg-zinc-800">
              <Search size={13} className="text-zinc-400" aria-hidden />
              <input
                ref={search}
                value={query}
                onChange={(event) => setQuery(event.target.value)}
                placeholder="Search apps"
                aria-label="Search apps"
                className="w-full bg-transparent text-xs outline-none placeholder:text-zinc-400"
              />
            </label>
          )}
```

```
          {apps.length === 0 ? (
            <Message>No apps are assigned to you in {props.portalName} yet.</Message>
          ) : shown.length === 0 ? (
            <Message>No apps match “{query.trim()}”.</Message>
          ) : (
            <ul className="grid min-h-0 grid-cols-3 gap-1 overflow-y-auto px-2 pb-2">
              {shown.map((app) => {
                const favorite = props.favorites.includes(app.id)
                return (
                  <li key={app.id} className="group/tile relative">
                    <button
                      type="button"
                      onClick={() => props.onOpenApp(app)}
                      title={`Open ${app.name}`}
                      aria-busy={props.openingId === app.id}
                      className="flex w-full flex-col items-center gap-1.5 rounded-xl px-1 py-2 text-
center hover:bg-zinc-100 dark:hover:bg-zinc-800"
                    >
                      <span className="relative">
                        <AppLogo app={app} />
                        {props.openingId === app.id && (
                          <span
                            role="status"
                            aria-label={`Opening ${app.name}`}
                            className="absolute inset-0 grid place-items-center rounded-lg bg-white/75
dark:bg-zinc-900/75"
                          >
                            <LoaderCircle
                              size={22}
                              className="animate-spin text-accent dark:text-accent-soft"
                              aria-hidden
                            />
                          </span>
                        )}
                      </span>
                      <span className="line-clamp-2 text-[11px] leading-tight">{app.name}</span>
                    </button>
                    <button
                      type="button"
                      onClick={() => props.onToggleFavorite(app, !favorite)}
                      aria-pressed={favorite}
                      aria-label={
                        favorite
                          ? `Remove ${app.name} from favourites`
                          : `Add ${app.name} to favourites`
                      }
                      title={favorite ? 'Remove from favourites' : 'Add to favourites'}
                      // On the logo's top-right corner (the 40px logo is centred in the tile).
                      style={{ left: 'calc(50% + 9px)', top: 0 }}
                      className={cn(
                        'absolute grid size-6 place-items-center rounded-full',
                        'hover:bg-zinc-200/70 dark:hover:bg-zinc-700',
                        favorite
                          ? 'text-amber-400'
                          : 'text-zinc-300 opacity-60 group-hover/tile:opacity-100 hover:text-amber-400
focus-visible:opacity-100 dark:text-zinc-600',
                      )}
                    >
                      <Star size={13} fill={favorite ? 'currentColor' : 'none'} aria-hidden />
                    </button>
                  </li>
                )
              })}
            </ul>
          )}
```

```
        </>
      )}

      <div className="shrink-0 border-t border-black/5 px-2 py-1.5 dark:border-white/10">
        <button
          type="button"
          onClick={props.onOpenPortal}
          className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-zinc-
600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
        >
          <ExternalLink size={13} aria-hidden /> Open the {props.portalName} portal
        </button>
      </div>
    </div>
  )
}
```

A heading, then one of five views depending on `state.status`, then a link to the portal at the bottom, always there.

- `const [query, setQuery] = useState('')`: what's typed in the search box.
- `favorites`, `onToggleFavorite`: the starred apps' IDs, and what a star click does.
- `useMemo(() => ..., [state, props.favorites])`: `useMemo` keeps a computed value between renders and only works it out again when something in its list changes. `apps` is the list with the starred apps first (`orderApps()`), or `[]` when there isn't one yet, and `shown` the apps whose names contain the search text, ignoring case.
- `if (props.open && apps.length > SEARCH_FROM) search.current?.focus()`: when there's a search box, it gets the keyboard as the panel opens, so you can start typing an app's name at once.
- `{state.status === 'ready' && (`: Refresh (`RotateCw`, a circular arrow) only when there's a list to refresh. `onRetry` loads it again.
- `{state.status === 'loading' && (`: "Getting your apps…" with a spinner.
- `{state.status === 'sign-in' && (`: why the user should connect (or the message from a failed sign-in), and **Sign in with JumpCloud** (the portal's name comes from `tenant.json`).
- `{state.status === 'signing-in' && (`: "Finish signing in in your browser…" with Cancel, like the built-in chat's sign-in.
- `{state.status === 'error' && (`: the reason, and Retry.
- `apps.length === 0 ? (`: no apps assigned yet, or (next) none matching the search, each with its own message.
- `grid min-h-0 grid-cols-3 ... overflow-y-auto`: three apps per row, scrolling when there are more than fit. `min-h-0` lets a flex item shrink below its content's height, which is what makes the scrolling work inside the card's maximum height.
- `<li key={app.id} className="group/tile relative">`: each tile holds two buttons side by side in the page, the app and its star. A button can't contain another, and a click on the star mustn't open the app. `relative` lets the star be placed over the tile; `group/tile` names the tile as a *group*, so the star can react to the pointer being anywhere on the tile.
- `openingId`: the app being opened, or `null`. `Panel` sets it while the main process gets the app's sign-in link, which can take a few seconds.
- `onClick={() => props.onOpenApp(app)}`: `Panel.openApp` opens it in the browser.
- `aria-busy={props.openingId === app.id}`: tells screen readers the tile is busy while its app opens.

- `<span className="relative">`: wraps the logo, so the spinner can be laid over exactly the logo (`absolute inset-0`, the same size).
- `{props.openingId === app.id && (`: only that app's tile shows the spinner: a `LoaderCircle` turning in the accent colour on a translucent white layer (zinc in dark mode, `bg-white/75 dark:bg-zinc-900/75`), so the logo shows faintly through. `role="status"` with `aria-label` "Opening Slack" is read out by screen readers. It disappears when `Panel` clears `openingId`: as the browser opens, or when opening fails.
- `onClick={() => props.onToggleFavorite(app, !favorite)}`: stars the app, or takes the star away.
- `aria-pressed={favorite}`, `aria-label`: for screen readers, a toggle button that's on or off, named "Add Slack to favourites" or "Remove Slack from favourites". `title` gives the shorter tooltip.
- `style={{ left: 'calc(50% + 9px)', top: 0 }}`: as the comment says, the 40 px logo is centred in the tile, so its right edge is 20 px right of the middle. The 24 px star button starting 9 px right of the middle sits over the logo's top-right corner.
- `favorite ? 'text-amber-400' : 'text-zinc-300 opacity-60 group-hover/tile:opacity-100 ...'`: a starred app's star is amber and filled (`fill={favorite ? 'currentColor' : 'none'}`). Otherwise it's a faint outline that becomes clearer while the pointer is over the tile, turns amber under the pointer, and shows fully when reached with the keyboard (`focus-visible:opacity-100`).
- `line-clamp-2`: a long name is cut to two lines with "…".
- `Open the {props.portalName} portal`: opens the User Portal itself, for anything the list doesn't cover.

`AppLogo` and `Message`

`src/renderer/src/components/AppsList.tsx`, lines 208–236

```
/** The app's logo, or its first letter on the accent colour when there isn't one. */
function AppLogo({ app }: { app: PortalApp }) {
  const [broken, setBroken] = useState(false)
  if (app.logo && !broken) {
    return (
      <img
        src={app.logo}
        alt=""
        onError={() => setBroken(true)}
        className="size-10 rounded-lg bg-white object-contain p-1 shadow-sm"
      />
    )
  }
  return (
    <span
      aria-hidden
      className={cn(
        'grid size-10 place-items-center rounded-lg text-base font-semibold text-white shadow-sm',
        'bg-accent',
      )}
    >
      {app.name.trim().charAt(0).toUpperCase() || '?'}
    </span>
  )
}

function Message({ children }: { children: ReactNode }) {
  return <p className="flex items-center gap-2 px-3.5 pb-3 text-xs text-zinc-500">{children}</p>
}
```

- `AppLogo`: the logo on a white tile, or, when there's none, the app's first letter on the accent colour, as the comment says.

- `onError={() => setBroken(true)}`: if the browser can't show the image after all, the letter takes over.
- `alt=""`: the logo is decoration; the app's name is right under it.
- `app.name.trim().charAt(0).toUpperCase() || '?'`: the first letter, in capitals.
- `Message`: one line of grey text, used for the list's simple messages.

What's tested

`tests/portalApps.test.ts` checks the readers (and `payloadOf()`) with made-up answers, and runs `PortalApps`, with its `McpAccount`, against a fake portal: a connector with two apps and a launch tool that only works once "signed in", a fake browser that "signs in" and comes back with a code, and an in-memory store.

- Reading the list: JumpCloud-style applications sorted by name, hidden ones skipped; a bare list or structured content; `null` when there's no list but `[]` for an empty one; never a link that isn't `https`; the launch tool's ID argument found from its schema; and the sign-in link found in a launch answer.
- `sniffImageType`: images known by their first bytes (PNG, JPEG, GIF, WebP, icon, bitmap, SVG), and `null` for anything else.
- `fetchLogo`: an `https` image becomes a `data:` URL, including JumpCloud's uploaded logos that come labelled `application/octet-stream`; `http`, non-images (whatever their label says, such as an HTML page labelled `image/png`), failed requests and anything too big are refused.
- `orderApps`: starred apps first, each group keeping its order.
- `PortalApps`: asking to sign in when nobody has connected, without opening the browser; signing in through the browser, saving the registration and tokens, then listing the apps with logos; remembering the list until Refresh; opening an app through its launch link; asking for the launch tool once, so each later open is one request; using the list's own link when JumpCloud is slow (the fake portal's launch can be made slow, and the timeout is cut to 30 ms); refusing a sign-in that comes back with a `state` it didn't send; Cancel going back to the Sign in button; and keeping the logos it already has, so a refresh doesn't download them again.
- `SavedAuth`: a public client with the loopback redirect; tokens kept across a reload of the file; forgetting everything clears the file; and the `state` check.

`tests/atlassian.test.ts` uses `McpAccount` too, through `Atlassian`: connecting, a connection that has ended, and disconnecting (see Jira and the service desk).

`tests/tenants.test.ts` checks that the Apps icon needs the portal settings, that the portal's own address must be `https`, and that starting on the Apps list needs both a portal and the Ask Claude icon (the text box would be out of reach otherwise). The real server and a real browser sign-in aren't part of the unit tests.

Jira and the service desk

Milestone 5 brings Jira and the IT service desk into the card beside the bubble. It all goes through **Atlassian's Remote MCP server** (`https://mcp.atlassian.com/v1/mcp`), the MCP server Atlassian runs for its cloud sites. Like JumpCloud's apps server (Your apps), it offers named tools (`searchJiraIssuesUsingJql`, `getJiraIssue`, `addCommentToJiraIssue` and so on) and only answers a signed-in user. So each person connects Desktop Assist to Atlassian once, in the browser: Atlassian's sign-in sends them through the company's JumpCloud single sign-on, they agree, and the browser comes back to the app. The connecting is the same `McpAccount` code as the Apps list's, with a file (`atlassian.bin`) and a port of its own. No token ever reaches the page.

What the user gets:

- **The Jira icon** (a bell) opens the Jira page, with two tabs. **Notifications** lists who else assigned the user an issue or mentioned them in the last 14 days, newest first, with a dot on what's new; it's checked in the background every 5 minutes, and the bell gets a dot too. **Search** looks through Jira and Confluence as the user types, and an issue key finds that issue.
- **An issue**, opened from either tab, shows its status, description and latest comments, with what can be done from here: **Ask Claude** to summarise it, draft a reply or suggest next steps (sent to Claude Desktop with the issue in the prompt), **Move** it to another status, **Comment**, and **Log time**.
- **The service desk icon** opens a page instead of the portal when the tenant lists its request types. **New request** starts from what's in the text box, offers the PC's details and IT's how-to articles (from the `INFRA` Confluence space at Morse Micro) while the user types, then opens the portal's own form in the browser, already filled in. A screenshot attached to the text box goes on the clipboard, ready to paste with Ctrl+V. **My requests** lists the user's requests, marked when IT replies or changes their status, with a Windows notification when it happens.
- **Issue keys** typed in the text box ("ISD-1234") get an **Open** button above it.

A tenant turns this on with `atlassian` (the site, and optionally the MCP server, the port and the knowledge spaces) and `serviceDesk.requests` (the portal, the Jira project and the request types); see `AtlassianSchema` and `ServiceDeskSchema` in Main process: startup, IPC and app plumbing.

How it fits together

1. **Connecting**: at startup `index.ts` calls `Atlassian.init()`. With a saved connection it finds out who the user is and starts checking; otherwise the Jira page shows **Connect to Atlassian** → `window.assist.atlassian.signIn()` → IPC `assist:atlassian-sign-in` → `Atlassian.signIn()` → `McpAccount.signIn()`, which opens Atlassian's sign-in in the browser. Every change of state is broadcast on `assist:atlassian-state`, and `index.ts` reopens the panel when connecting is done.
2. **Notifications and requests**: `Atlassian.refresh()` runs right after connecting and then every 5 minutes. It searches Jira with JQL (Jira's query language) for issues recently assigned to the user and issues that may mention them, reads each one's history and comments to see who did it (`jiraData.ts`), and checks the user's service desk requests. The result, a `JiraFeed`, is broadcast on `assist:jira-feed`, and `PanelView` puts dots on the icons from it. What the user has already seen is remembered in `atlassian.json`.
3. **Working on an issue**: clicking an issue → `PanelView.openIssue()` → `IssueView` → `window.assist.atlassian.issue(key)` → `Atlassian.issue()`. Moving, commenting and logging time each make one tool call, then answer with the issue as it is now. **Ask Claude** → `atlassian.ask(key, prompt)` → IPC `assist:atlassian-ask`: `Atlassian.prompt()` builds the question with the issue in it (`issuePrompt()`), and `ipc.ts` hands it to Claude Desktop as if it had been typed in the text box (see Claude Desktop), or sends it in the built-in chat.

4. **A new IT request**: **Continue on the portal** → `window.assist.serviceDesk.raise()` → IPC `assist:service-desk-raise` → `ServiceDesk.raise()`. It copies the text box's screenshots to the clipboard, builds the portal's create-request address with the summary and description in it (`requestFormUrl()`), and opens it in the browser; the panel closes. The user checks the form, pastes the screenshot, and sends it on the portal, signed in as themselves. Desktop Assist never creates the request itself, so nothing is raised that the user hasn't seen.

The pages themselves (`JiraPage`, `IssueView`, `ServiceDeskPage`) are in The Jira and service desk pages, and how `PanelView` swaps them in is in The pages.

Reading answers defensively. Atlassian's MCP server wraps Jira's and Confluence's own web APIs and passes their fields through, sometimes nested differently (`issues.nodes` rather than `issues`, say), and can give comments and descriptions either as Markdown or in Jira's own format, ADF. So, as with the Apps list, `jiraData.ts` reads answers tolerantly: it follows those APIs' field names, accepts the usual variations, and ignores what it doesn't know.

`src/main/atlassian/jiraData.ts`: reading Atlassian's answers

Plain functions with no Electron or network in them, so `tests/atlassian.test.ts` checks them with made-up answers. `Atlassian` uses them on what the tools return, and to write the queries it sends. The comment at the top sums up the approach. The file is divided into parts by comment lines such as `// ---- queries`, which show at the top of the first block of each part.

`Item`, `isObject`, `text`, `items` and `field`

src/main/atlassian/jiraData.ts, lines 20–31

```ts
type Item = Record<string, unknown>

const isObject = (value: unknown): value is Item =>
  typeof value === 'object' && value !== null && !Array.isArray(value)

const text = (value: unknown): string | null =>
  typeof value === 'string' && value.trim() ? value.trim() : null

const items = (value: unknown): Item[] => (Array.isArray(value) ? value.filter(isObject) : [])

const field = (value: unknown, ...path: string[]): unknown =>
  path.reduce<unknown>((at, key) => (isObject(at) ? at[key] : undefined), value)
```

Small helpers for poking around in data of unknown shape. `Item`, `isObject` and `text` are the same as in `portalData.ts` (Your apps).

- `items(value)`: the objects in a list, or an empty list if `value` isn't one, so a loop over it never has to check.
- `field(value, ...path)`: a value deep inside an object, or `undefined` if any step of the way is missing. `field(fields, 'status', 'name')` is `fields.status.name` without the risk of an error. `...path` collects the names into a list, and `reduce` steps through it one name at a time.

`NOTIFICATION_DAYS`, `APP_ACCOUNT` and `isPerson`

src/main/atlassian/jiraData.ts, lines 33–40

```ts
/** How far back the Notifications tab looks. */
export const NOTIFICATION_DAYS = 14

/** Atlassian's own accounts (Automation for Jira and other apps) have IDs starting with this. */
const APP_ACCOUNT = '557058:'

const isPerson = (accountId: string | null): boolean =>
  accountId !== null && !accountId.startsWith(APP_ACCOUNT)
```

- `NOTIFICATION_DAYS = 14`: the Notifications tab shows the last two weeks.
- `APP_ACCOUNT`: as the comment says, Atlassian gives its own accounts (Automation for Jira, other apps) IDs that start with `557058:`.
- `isPerson(accountId)`: whether an account is a person, so an automatic "Thanks, we've got your request" doesn't count as IT replying.

`searchWords()`

src/main/atlassian/jiraData.ts, lines 42–54

```ts
// ---------------------------------------------------------------------------------------- queries

/**
 * The words of a search, safe inside a quoted JQL or CQL string: the characters Jira's and
 * Confluence's text search treat as operators are dropped.
 */
export function searchWords(query: string): string {
  return query
    .replace(/[+\-&|!(){}[\]^~*?\\:"/<>=;]/g, ' ')
    .replace(/\s+/g, ' ')
    .trim()
    .slice(0, 200)
}
```

The words the user typed, made safe to put inside quotes in a JQL or CQL query. Characters such as `"`, `(`, `*` and `~` mean something to Jira's and Confluence's search; a `"` could even end the quoted text early and add to the query. So they're all replaced with spaces, runs of spaces become one, and the result is cut to 200 characters. What's left is only ever searched for.

`ASSIGNED_JQL` and `MENTIONED_JQL`

src/main/atlassian/jiraData.ts, lines 56–67

```
/** Issues assigned to the user lately, by someone else or by themselves (sorted out later). */
export const ASSIGNED_JQL =
  `assignee = currentUser() AND (assignee CHANGED TO currentUser() AFTER -${NOTIFICATION_DAYS}d` +
  ` OR created >= -${NOTIFICATION_DAYS}d) ORDER BY updated DESC`

/**
 * Issues that may mention the user: Jira's text search finds mentions of currentUser() in
 * comments and descriptions. The comments themselves show who mentioned them, and when.
 */
export const MENTIONED_JQL =
  `(comment ~ currentUser() OR description ~ currentUser()) AND updated >= -${NOTIFICATION_DAYS}d` +
  ` ORDER BY updated DESC`
```

The two searches behind the Notifications tab, in JQL. `currentUser()` is whoever is signed in, and `-14d` means 14 days ago.

- `ASSIGNED_JQL`: issues assigned to the user, where the assignee changed to them in the last two weeks, or which were created in that time (an issue created already assigned has no change of assignee). As the comment says, it can't tell who did the assigning; that's sorted out later from each issue's history (`assignedNotification()`).
- `MENTIONED_JQL`: issues updated lately whose comments or description mention the user. Jira's text search finds mentions of `currentUser()`; the comments themselves then say who wrote them, and when (`mentionNotifications()`).
- Both are `ORDER BY updated DESC`, so if there are more than are asked for, the newest come first.

`myRequestsJql()`, `textSearchJql()` and `confluenceCql()`

src/main/atlassian/jiraData.ts, lines 69–85

```
/** The user's requests on the service desk: the open ones, and those closed this month. */
export function myRequestsJql(projectKey: string): string {
  return (
    `project = "${projectKey}" AND reporter = currentUser()` +
    ` AND (statusCategory != Done OR updated >= -30d) ORDER BY updated DESC`
  )
}

export function textSearchJql(query: string): string {
  return `text ~ "${searchWords(query)}" ORDER BY updated DESC`
}

/** Confluence pages and blog posts, ranked like Confluence's own search; only `spaces` if given. */
export function confluenceCql(query: string, spaces: string[] = []): string {
  const inSpaces = spaces.length ? ` AND space in (${spaces.map((s) => `"${s}"`).join(', ')})` : ''
  return `siteSearch ~ "${searchWords(query)}" AND type in (page, blogpost)${inSpaces}`
}
```

- `myRequestsJql(projectKey)`: the user's service desk requests (`reporter = currentUser()` in the service desk's Jira project): every open one, and those closed in the last 30 days (`statusCategory != Done OR updated >= -30d`).
- `textSearchJql(query)`: Jira issues whose text (summary, description, comments) matches the words.
- `confluenceCql(query, spaces)`: the same for Confluence, in CQL, Confluence's query language. `siteSearch` ranks results the way Confluence's own search box does. `type in (page, blogpost)` leaves out attachments

and comments. With `spaces`, only those spaces are searched, as in `space in ("INFRA")`.

`issueNodes()`

src/main/atlassian/jiraData.ts, lines 87–97

```
// ---------------------------------------------------------------------------------------- reading

/** The issues in a search or issue answer (Atlassian's `issues.nodes`, or Jira's `issues`). */
export function issueNodes(result: ToolResult): Item[] {
  const payload = payloadOf(result)
  const issues = field(payload, 'issues')
  if (Array.isArray(issues)) return items(issues)
  const nodes = field(issues, 'nodes') ?? field(payload, 'nodes')
  if (Array.isArray(nodes)) return items(nodes)
  return isObject(payload) && text(payload['key']) ? [payload] : []
}
```

The issues in an answer, wherever they are. As the comment says, a search can come back as Jira has it (`issues` is the list) or as Atlassian's server rearranges it (`issues.nodes`); a single issue comes back as the issue itself, recognised by its `key`.

`CATEGORIES` and `statusOf()`

src/main/atlassian/jiraData.ts, lines 99–108

```
const CATEGORIES: StatusCategory[] = ['new', 'indeterminate', 'done']

export function statusOf(fields: unknown): IssueStatus {
  const status = field(fields, 'status')
  const category = text(field(status, 'statusCategory', 'key'))
  return {
    name: text(field(status, 'name')) ?? 'Unknown',
    category: CATEGORIES.find((c) => c === category) ?? 'indeterminate',
  }
}
```

An issue's status. Jira lets each company name its statuses ("Waiting for support", "In review"...), but every status belongs to one of three *categories*: `new` (to do), `indeterminate` (in progress) and `done`. The page colours a status by its category.

- `CATEGORIES.find((c) => c === category) ?? 'indeterminate'`: a category that isn't one of the three is shown as in progress, rather than trusted.

`nameOf`, `accountOf`, `issueSummaryOf()` and `issueSummaries()`

src/main/atlassian/jiraData.ts, lines 110–130

```
const nameOf = (user: unknown): string | null => text(field(user, 'displayName'))

const accountOf = (user: unknown): string | null => text(field(user, 'accountId'))

export function issueSummaryOf(node: Item, site: string): IssueSummary | null {
  const key = text(node['key'])
  if (!key || !isIssueKey(key)) return null
  const fields = node['fields']
  return {
    key,
    summary: text(field(fields, 'summary')) ?? '(no summary)',
    status: statusOf(fields),
    type: text(field(fields, 'issuetype', 'name')),
    url: browseUrl(site, key),
  }
}

export function issueSummaries(result: ToolResult, site: string): IssueSummary[] {
  return issueNodes(result)
    .map((node) => issueSummaryOf(node, site))
    .filter((issue): issue is IssueSummary => issue !== null)
}
```

- `nameOf(user)`, `accountOf(user)`: a Jira user's display name and account ID.
- `issueSummaryOf(node, site)`: an issue as a search result shows it: key, summary, status, type, and its address in Jira. `isIssueKey(key)` (from `src/shared/jira.ts`) means a strange key can never become part of an address.
- `issueSummaries(result, site)`: every issue in an answer. `.filter((issue): issue is IssueSummary => issue !== null)` drops the ones that couldn't be read; the `issue is IssueSummary` part tells TypeScript that what's left has no `null` in it.

`adfMentions()`

src/main/atlassian/jiraData.ts, lines 132–147

```
// ---------------------------------------------------------------- Atlassian Document Format

/** The account IDs mentioned in an ADF document (a comment or description in Jira's own format). */
export function adfMentions(doc: unknown): string[] {
  const found: string[] = []
  const walk = (node: unknown, depth: number) => {
    if (depth > 30 || !isObject(node)) return
    if (node['type'] === 'mention') {
      const id = text(field(node, 'attrs', 'id'))
      if (id) found.push(id)
    }
    for (const child of items(node['content'])) walk(child, depth + 1)
  }
  walk(doc, 0)
  return found
}
```

The people mentioned in a comment or description in **ADF**, the Atlassian Document Format: Jira's own way of storing rich text, as a tree of JSON nodes (a paragraph holds text nodes, a mention is a node with the person's account ID, and so on).

- `const walk = (node, depth) => {`: visits a node and then, by calling itself, each node in its `content`.
- `if (depth > 30 || !isObject(node)) return`: a strangely deep document can't make it go on for ever.
- `if (node['type'] === 'mention')`: a mention keeps the account ID in `attrs.id`.

`BLOCKS` and `adfText()`

src/main/atlassian/jiraData.ts, lines 149–170

```
const BLOCKS = new Set(['paragraph', 'heading', 'listItem', 'blockquote', 'codeBlock', 'rule'])

/** An ADF document as plain text (mentions as "@Name"). */
export function adfText(doc: unknown): string {
  const parts: string[] = []
  const walk = (node: unknown, depth: number) => {
    if (depth > 30 || !isObject(node)) return
    const type = node['type']
    if (type === 'text') parts.push(typeof node['text'] === 'string' ? node['text'] : '')
    else if (type === 'mention' || type === 'emoji')
      parts.push(text(field(node, 'attrs', 'text')) ?? '')
    else if (type === 'hardBreak') parts.push('\n')
    else if (type === 'inlineCard') parts.push(text(field(node, 'attrs', 'url')) ?? '')
    for (const child of items(node['content'])) walk(child, depth + 1)
    if (typeof type === 'string' && BLOCKS.has(type)) parts.push('\n')
  }
  walk(doc, 0)
  return parts
    .join('')
    .replace(/\n{3,}/g, '\n\n')
    .trim()
}
```

An ADF document as plain text, for the start of a comment shown in a notification.

- Text nodes give their text; a mention or emoji its `attrs.text` (a mention's is "@Name"); a line break a new line; a link card its address.
- `BLOCKS`: the nodes that are blocks of their own (paragraphs, headings, list items and so on), each followed by a new line once its content is done.
- `.replace(/\n{3,}/g, '\n\n')`: no more than one empty line in a row.

`cleanMarkdown()` and `bodyText()`

src/main/atlassian/jiraData.ts, lines 172–189

```ts
/**
 * Markdown from Atlassian, tidied for showing and for Claude: its mention and emoji tags become
 * their text, and pasted images (which only open inside Jira) become "[image]".
 */
export function cleanMarkdown(markdown: string): string {
  return markdown
    .replace(/<custom data-type="[a-z]+"[^>]*>([^<]*)<\/custom>/g, '$1')
    .replace(/!\[[^\]]*\]\(blob:[^)]*\)/g, '[image]')
    .replace(//g, '')
    .replace(/\n{3,}/g, '\n\n')
    .trim()
}

/** A comment or description (ADF or Markdown) as text. */
function bodyText(body: unknown): string {
  if (typeof body === 'string') return cleanMarkdown(body)
  return adfText(body)
}
```

Atlassian's server can turn ADF into Markdown itself, which is easier to show and to give to Claude, but it writes mentions and emoji as tags of its own, such as `<custom data-type="mention" ...>@Sam Bolin</custom>`.

- `cleanMarkdown()`: as the comment says, the tags become their text, and a pasted image (a `blob:` address that only works inside Jira) becomes `[image]`. An invisible zero-width character that Atlassian's Markdown can contain is removed, and empty lines are squeezed as in `adfText()`.
- `bodyText(body)`: a comment or description either way: a string is Markdown, anything else ADF.

`mentionsUser()` and `excerpt()`

src/main/atlassian/jiraData.ts, lines 191–201

```ts
/** Whether a comment or description mentions the user. */
export function mentionsUser(body: unknown, user: { accountId: string; name: string }): boolean {
  if (typeof body === 'string') return body.includes(`>@${user.name}</custom>`)
  return adfMentions(body).includes(user.accountId)
}

/** The start of a text, on one line. */
export function excerpt(value: string, length = 140): string {
  const line = value.replace(/\s+/g, ' ').trim()
  return line.length > length ? `${line.slice(0, length - 1).trimEnd()}…` : line
}
```

- `mentionsUser(body, user)`: whether a comment or description mentions the user. In Markdown the mention tag ends with `>@Their Name</custom>`; in ADF the mention carries their account ID, which is the surer test. The Notifications search asks for ADF for this reason.
- `excerpt(value, length)`: the start of a text on one line, with `…` if it was cut.

`FoundNotification`, `time` and `iso`

src/main/atlassian/jiraData.ts, lines 203–212

```
// ---------------------------------------------------------------------------- notifications

export type FoundNotification = Omit<JiraNotification, 'unread'>

const time = (value: unknown): number => {
  const parsed = Date.parse(typeof value === 'string' ? value : '')
  return Number.isNaN(parsed) ? 0 : parsed
}

const iso = (value: unknown): string => new Date(time(value)).toISOString()
```

- `FoundNotification`: a notification as it's found, before `Atlassian` knows whether the user has seen it. `Omit<JiraNotification, 'unread'>` is `JiraNotification` without its `unread` field.
- `time(value)`: a date from Jira as milliseconds, or 0 if it can't be read, so sorting and comparing never fail.
- `iso(value)`: the same date written the standard way (`2026-10-12T09:30:00.000Z`), which sorts correctly as text.

`assignedNotification()`

src/main/atlassian/jiraData.ts, lines 214–262

```
/**
 * Who assigned the user this issue, from its change history: the latest change of assignee to
 * them. Null if they assigned it to themselves, it was longer ago than `since`, or it isn't
 * theirs. An issue created already assigned has no such change; its creator assigned it.
 */
export function assignedNotification(
  node: Item,
  me: { accountId: string },
  since: number,
): FoundNotification | null {
  const key = text(node['key'])
  if (!key) return null
  const fields = node['fields']
  const summary = text(field(fields, 'summary')) ?? '(no summary)'
  const histories = items(field(node, 'changelog', 'histories')).sort(
    (a, b) => time(b['created']) - time(a['created']),
  )
  const change = histories.find((history) =>
    items(history['items']).some(
      (item) =>
        item['field'] === 'assignee' &&
        (item['to'] === me.accountId || item['tmpToAccountId'] === me.accountId),
    ),
  )
  if (change) {
    const by = change['author']
    if (accountOf(by) === me.accountId || time(change['created']) < since) return null
    return {
      id: `assigned:${key}:${text(change['id']) ?? time(change['created'])}`,
      kind: 'assigned',
      key,
      summary,
      by: nameOf(by) ?? 'Someone',
      at: iso(change['created']),
    }
  }
```

```
  const creator = field(fields, 'creator')
  const created = field(fields, 'created')
  if (accountOf(field(fields, 'assignee')) !== me.accountId) return null
  if (!creator || accountOf(creator) === me.accountId || time(created) < since) return null
  return {
    id: `assigned:${key}:created`,
    kind: 'assigned',
    key,
    summary,
    by: nameOf(creator) ?? 'Someone',
    at: iso(created),
  }
}
```

Who assigned the user this issue, as the comment says, from its change history (the *changelog*, which `Atlassian` asks for issue by issue).

- `items(field(node, 'changelog', 'histories')).sort(...)`: the issue's changes, newest first.
- `histories.find(...)`: the latest change that set the assignee to the user. `tmpToAccountId` is where some answers keep the new assignee's ID.
- `if (accountOf(by) === me.accountId || time(change['created']) < since) return null`: the user assigned it to themselves, or it happened more than two weeks ago: nothing to tell them.
- `id: \`assigned:${key}:${...}\``: an ID that stays the same each time the issue is checked, so the page can keep track of it.
- After the `if (change)`: no change of assignee, so if the issue is the user's, it was created already assigned to them, and whoever created it assigned it (unless that was the user, or it was too long ago).

`mentionNotifications()`

src/main/atlassian/jiraData.ts, lines 264–308

```
/** The comments (and the description) of an issue that mention the user, by someone else. */
export function mentionNotifications(
  node: Item,
  me: { accountId: string; name: string },
  since: number,
): FoundNotification[] {
  const key = text(node['key'])
  if (!key) return []
  const fields = node['fields']
  const summary = text(field(fields, 'summary')) ?? '(no summary)'
  const found: FoundNotification[] = []
  for (const comment of items(field(fields, 'comment', 'comments'))) {
    const author = comment['author']
    if (accountOf(author) === me.accountId || time(comment['created']) < since) continue
    if (!mentionsUser(comment['body'], me)) continue
    found.push({
      id: `mentioned:${key}:${text(comment['id']) ?? time(comment['created'])}`,
      kind: 'mentioned',
      key,
      summary,
      by: nameOf(author) ?? 'Someone',
      at: iso(comment['created']),
      excerpt: excerpt(bodyText(comment['body'])),
    })
  }
  const creator = field(fields, 'creator')
  const description = field(fields, 'description')
  if (
```

```
    description &&
    accountOf(creator) !== me.accountId &&
    time(field(fields, 'created')) >= since &&
    mentionsUser(description, me)
  ) {
    found.push({
      id: `mentioned:${key}:description`,
      kind: 'mentioned',
      key,
      summary,
      by: nameOf(creator) ?? 'Someone',
      at: iso(field(fields, 'created')),
      excerpt: excerpt(bodyText(description)),
    })
  }
  return found
}
```

Every comment on the issue in the last two weeks, by someone else, that mentions the user, as a notification with the comment's start (`excerpt(bodyText(...))`). Then the description, if someone else wrote the issue in that time and mentioned the user in it.

`RequestInfo` and `requestInfoOf()`

src/main/atlassian/jiraData.ts, lines 310–345

```ts
// ---------------------------------------------------------------------- service requests

/** One of the user's requests, with what's needed to tell when IT has done something new. */
export interface RequestInfo {
  key: string
  summary: string
  status: IssueStatus
  updated: string
  /** The latest comment by someone else (not the user, not an app), or null. */
  reply: { id: string; author: string } | null
}

export function requestInfoOf(node: Item, me: { accountId: string }): RequestInfo | null {
  const key = text(node['key'])
  if (!key) return null
  const fields = node['fields']
  const replies = items(field(fields, 'comment', 'comments'))
    .filter((comment) => {
      const author = accountOf(comment['author'])
      return author !== me.accountId && isPerson(author)
    })
    .sort((a, b) => time(a['created']) - time(b['created']))
  const last = replies.at(-1)
  return {
    key,
    summary: text(field(fields, 'summary')) ?? '(no summary)',
    status: statusOf(fields),
    updated: iso(field(fields, 'updated')),
    reply: last
      ? {
          id: text(last['id']) ?? String(time(last['created'])),
          author: nameOf(last['author']) ?? 'IT',
        }
      : null,
  }
}
```

One of the user's service desk requests, with what's needed to tell when IT has done something.

- `reply`: as the comment says, the latest comment by someone else. `.filter(...)` keeps comments by people other than the user (`isPerson()` leaves out automatic ones), and the sort puts them oldest first, so `replies.at(-1)` is the latest. `at(-1)` is a list's last item.
- `id: text(last['id']) ?? String(time(last['created']))`: the reply's ID, which is how `Atlassian` notices a new reply: a different ID.

`portalRequestUrl()`

src/main/atlassian/jiraData.ts, lines 347–350

```ts
/** A request on the help centre portal. */
export function portalRequestUrl(helpCentre: string, portalId: string, key: string): string {
  return `${new URL(helpCentre).origin}/servicedesk/customer/portal/${portalId}/${key}`
}
```

A request's page on the help centre portal, where IT's customers see it, which is what My requests links to. `new URL(helpCentre).origin` is just the `https://...atlassian.net` part.

`MAX_DESCRIPTION`, `MAX_COMMENTS` and `issueDetailOf()`

src/main/atlassian/jiraData.ts, lines 352–395

```
// ---------------------------------------------------------------------------- issue details

/** The longest description shown (and given to Claude). */
const MAX_DESCRIPTION = 4000

/** How many of the latest comments are shown. */
const MAX_COMMENTS = 10

export function issueDetailOf(
  node: Item,
  site: string,
  options: { transitions: IssueTransition[]; portalUrl: string | null },
): IssueDetail | null {
  const summary = issueSummaryOf(node, site)
  if (!summary) return null
  const fields = node['fields']
  const description = bodyText(field(fields, 'description') ?? '')
  // Atlassian adds the comments as Markdown beside Jira's own (ADF) ones; either will do.
  const markdownComments = items(node['comments'])
  const comments = (
    markdownComments.length ? markdownComments : items(field(fields, 'comment', 'comments'))
  )
    .map((comment): IssueComment => ({
      id: text(comment['id']) ?? String(time(comment['created'])),
      author: nameOf(comment['author']) ?? 'Someone',
      created: iso(comment['created']),
      body: bodyText(comment['body']),
    }))
    .sort((a, b) => time(a.created) - time(b.created))
    .slice(-MAX_COMMENTS)
  return {
    ...summary,
    assignee: nameOf(field(fields, 'assignee')),
    reporter: nameOf(field(fields, 'reporter')),
    created: iso(field(fields, 'created')),
    updated: iso(field(fields, 'updated')),
    description:
      description.length > MAX_DESCRIPTION
        ? `${description.slice(0, MAX_DESCRIPTION).trimEnd()}\n\n…`
        : description,
    comments,
    portalUrl: options.portalUrl,
    transitions: options.transitions,
  }
}
```

An issue as `IssueView` shows it.

- `const summary = issueSummaryOf(node, site)`: the key, summary, status, type and address, as in search results; `...summary` copies them into the result.
- `items(node['comments'])`: as the comment says, Atlassian adds the comments as Markdown beside Jira's own ADF ones. The Markdown ones are used when they're there.

- `.sort(...)` then `.slice(-MAX_COMMENTS)`: oldest first, and only the latest 10. `slice(-10)` takes a list's last 10 items.
- `description.length > MAX_DESCRIPTION`: a description over 4,000 characters is cut, with `…` on a line of its own, so a huge one can't fill the card (or the question to Claude).
- `portalUrl`, `transitions`: given by `Atlassian`, which asks for them separately.

`transitionsOf()`

src/main/atlassian/jiraData.ts, lines 397–403

```
export function transitionsOf(result: ToolResult): IssueTransition[] {
  const payload = payloadOf(result)
  const list = Array.isArray(payload) ? payload : field(payload, 'transitions')
  return items(list)
    .map((t) => ({ id: text(t['id']) ?? '', name: text(t['name']) ?? '' }))
    .filter((t) => t.id && t.name)
}
```

The statuses an issue can move to. In Jira a *transition* is a step in the issue's workflow ("Start progress", "Resolve"), with its own ID; moving an issue means making a transition. Only ones with both an ID and a name are kept.

`ENTITIES` and `plainSnippet()`

src/main/atlassian/jiraData.ts, lines 405–419

```
// ---------------------------------------------------------------------------- confluence

const ENTITIES: Record<string, string> = { amp: '&', lt: '<', gt: '>', quot: '"', apos: "'" }

/** Confluence's highlighted, HTML-escaped snippets as plain text. */
export function plainSnippet(value: string): string {
  return value
    .replace(/@@@(end)?hl@@@/g, '')
    .replace(/<[^>]*>/g, '')
    .replace(/&(#\d+|[a-z]+);/g, (whole, name: string) =>
      name.startsWith('#') ? String.fromCharCode(Number(name.slice(1))) : (ENTITIES[name] ?? whole),
    )
    .replace(/\s+/g, ' ')
    .trim()
}
```

Confluence's search gives titles and snippets ready for a web page: the matching words wrapped in `@@@hl@@@...@@@endhl@@@` markers, HTML tags, and characters written as *entities* (`&` for `&`, `'` for `'`). The page shows plain text, so:

- `.replace(/@@@(end)?hl@@@/g, '')`, `.replace(/<[^>]*>/g, '')`: the markers and tags go.
- `.replace(/&(#\d+|[a-z]+);/g, ...)`: a numbered entity becomes its character (`String.fromCharCode`), and a named one is looked up in `ENTITIES`; one it doesn't know is left as it was.

`confluencePagesOf()`

src/main/atlassian/jiraData.ts, lines 421–448

```
export function confluencePagesOf(result: ToolResult, site: string): ConfluencePage[] {
  const payload = payloadOf(result)
  const list =
    field(payload, 'content', 'nodes') ?? field(payload, 'results') ?? field(payload, 'nodes')
  const origin = new URL(site).origin
  const pages: ConfluencePage[] = []
  for (const page of items(Array.isArray(payload) ? payload : list)) {
    const content = isObject(page['content']) ? page['content'] : page
    const id = text(content['id'])
    const title = text(content['title']) ?? text(page['title'])
    const webui = text(field(content, '_links', 'webui')) ?? text(field(page, 'url'))
    const url =
      httpsUrl(page['webUrl']) ??
      httpsUrl(content['webUrl']) ??
      (webui?.startsWith('/') ? `${origin}/wiki${webui}` : null)
    if (!id || !title || !url || !url.startsWith(origin)) continue
    pages.push({
      id,
      title: plainSnippet(title),
      excerpt: excerpt(plainSnippet(text(page['summary']) ?? text(page['excerpt']) ?? ''), 160),
      space:
        text(field(content, 'space', 'name')) ??
        text(field(page, 'resultGlobalContainer', 'title')),
      url,
    })
  }
  return pages
}
```

The pages in a Confluence search answer.

- `const content = isObject(page['content']) ? page['content'] : page`: a search result keeps the page itself under `content`; some answers give the page directly.
- `const url = ...`: the page's full address if the answer has one (`webUrl`), otherwise its path (`_links.webui`) on the site's Confluence (`/wiki`).
- `if (!id || !title || !url || !url.startsWith(origin)) continue`: only pages on the company's own site are kept, so a search answer can never make the app open somewhere else.
- `space`: the space's name, shown under the page's title.

`AtlassianUser`, `userOf()` and `cloudIdOf()`

src/main/atlassian/jiraData.ts, lines 450–483

```
// ---------------------------------------------------------------------------- the user

export interface AtlassianUser {
  accountId: string
  name: string
  email?: string
}

export function userOf(result: ToolResult): AtlassianUser | null {
  const payload = payloadOf(result)
  const accountId = text(field(payload, 'account_id')) ?? text(field(payload, 'accountId'))
  const name =
    text(field(payload, 'name')) ??
    text(field(payload, 'displayName')) ??
    text(field(payload, 'nickname'))
  if (!accountId || !name) return null
  const email = text(field(payload, 'email')) ?? text(field(payload, 'emailAddress'))
  return email ? { accountId, name, email } : { accountId, name }
}

/** The site's cloud ID from the list of sites the user can reach; null if it isn't there. */
export function cloudIdOf(result: ToolResult, site: string): string | null {
  const payload = payloadOf(result)
  const host = new URL(site).host
  for (const resource of items(Array.isArray(payload) ? payload : field(payload, 'resources'))) {
    const url = text(resource['url'])
    try {
      if (url && new URL(url).host === host) return text(resource['id'])
    } catch {
      // not a URL: skip it
    }
  }
  return null
}
```

- `AtlassianUser`: who's connected: their account ID (to know their own comments and changes), name (the settings menu shows "Connected to Atlassian as ...") and email, if given.
- `userOf(result)`: reads the answer of `atlassianUserInfo`, whichever way it names the fields (`account_id` or `accountId`, `name` or `displayName`).
- `cloudIdOf(result, site)`: every tool wants the site's *cloud ID*, Atlassian's ID for one company's site. `getAccessibleAtlassianResources` lists the sites the user can reach, each with its address and ID; this finds the one whose host (`morsemicro.atlassian.net`) matches the tenant's site. An address that can't be read is skipped (`catch`).

`INSTRUCTIONS` and `day`

src/main/atlassian/jiraData.ts, lines 485–495

```
// ---------------------------------------------------------------------------- asking Claude

const INSTRUCTIONS: Record<IssuePrompt, string> = {
  summarise:
    "Summarise this Jira issue for me: what it's about, where it's up to, and anything that's waiting on
me.",
  reply:
    "Draft a reply I could post as a comment on this Jira issue. Keep it short, friendly and
professional, and don't promise anything the issue doesn't support.",
  'next-steps': 'Suggest the next steps to move this Jira issue forward, most important first.',
}

const day = (value: string): string => value.slice(0, 10)
```

- `INSTRUCTIONS`: what each **Ask Claude** button asks, in the user's voice. `Record<IssuePrompt, string>` makes TypeScript check there's one for each of `summarise`, `reply` and `next-steps`.
- `day(value)`: the date part of an ISO time (`2026-10-12`), for the comments in the prompt.

`issuePrompt()`

src/main/atlassian/jiraData.ts, lines 497–538

```ts
/**
 * What to ask Claude about an issue: the request, then the issue itself. Kept under `max`
 * characters (Claude Desktop's limit for a linked prompt) by leaving out the oldest comments
 * first, then shortening the description.
 */
export function issuePrompt(issue: IssueDetail, kind: IssuePrompt, max: number): string {
  const meta = [
    `Status: ${issue.status.name}`,
    issue.type && `Type: ${issue.type}`,
    `Assignee: ${issue.assignee ?? 'Unassigned'}`,
    issue.reporter && `Reporter: ${issue.reporter}`,
  ].filter(Boolean)
  const head = [
    INSTRUCTIONS[kind],
    '',
    `${issue.key}: ${issue.summary}`,
    meta.join(' · '),
    issue.url,
  ].join('\n')
  const comments = issue.comments.map(
    (c) => `${c.author} (${day(c.created)}):\n${c.body.trim() || '(no text)'}`,
  )
  let description = issue.description.trim() || '(no description)'
  const build = () =>
    [
      head,
      '',
      'Description:',
      description,
      ...(comments.length ? ['', 'Comments, oldest first:', comments.join('\n\n')] : []),
    ].join('\n')
  let prompt = build()
  while (prompt.length > max && comments.length > 0) {
    comments.shift()
    prompt = build()
  }
  if (prompt.length > max) {
    description = `${description.slice(0, Math.max(0, description.length - (prompt.length - max) - 2))}…`
    prompt = build()
  }
  return prompt.slice(0, max)
}
```

The question for Claude: the request, then the issue itself, so Claude has everything it needs without access to Jira.

- `head`: the instruction, then the key and summary, the status, type, assignee and reporter on one line (`.filter(Boolean)` drops the ones that are missing), and the issue's address.
- `comments`: each as "Name (date):" and its text.
- `const build = () => [...]`: puts the prompt together from the parts as they are at the time, so it can be rebuilt after shortening them. The comments come oldest first, as a conversation reads.
- `while (prompt.length > max && comments.length > 0) { comments.shift() ... }`: as the comment says, the prompt has to fit `max` characters (Claude Desktop's limit for a question in a link, 12,000). The oldest comments are left out first, one at a time (`shift()` removes a list's first item).
- `description = \`${description.slice(...)}…\`: if it's still too long without any comments, the description is shortened by as much as it's over (2 more characters make room for the …` and

some slack).

- `return prompt.slice(0, max)`: never over the limit, whatever happens.

`src/main/atlassian/Atlassian.ts`: the connection, the feed and working on issues

Everything the Jira page and My requests do, on top of an `McpAccount`. Like `PortalApps`, it has no Electron in it: the connection, the encrypted store, the remembered state, the browser and Windows notifications all come in through `AtlassianDeps`, so `tests/atlassian.test.ts` runs it against a fake Atlassian. `start()` in `index.ts` creates the one instance when the tenant has `atlassian`, and `registerAtlassian()` in `ipc.ts` calls it.

`TOOLS`

src/main/atlassian/Atlassian.ts, lines 50–62

```
/** The tools of Atlassian's Remote MCP server that Desktop Assist uses. */
export const TOOLS = {
  user: 'atlassianUserInfo',
  sites: 'getAccessibleAtlassianResources',
  searchJira: 'searchJiraIssuesUsingJql',
  issue: 'getJiraIssue',
  transitions: 'getTransitionsForJiraIssue',
  transition: 'transitionJiraIssue',
  comment: 'addCommentToJiraIssue',
  worklog: 'addWorklogToJiraIssue',
  searchConfluence: 'searchConfluenceUsingCql',
  projects: 'getVisibleJiraProjects',
} as const
```

The names of the Atlassian MCP server's tools that are used, in one place. `as const` makes TypeScript treat each name as that exact text, and the object as read-only.

`FEED_INTERVAL_MS`, `STALE_MS`, `DAY_MS`, `MAX_NOTIFICATIONS` and `PARALLEL`

src/main/atlassian/Atlassian.ts, lines 64–71

```
/** How often notifications and requests are checked in the background. */
export const FEED_INTERVAL_MS = 5 * 60_000

/** Opening the Jira page checks again if the last check is older than this. */
const STALE_MS = 60_000

const DAY_MS = 24 * 60 * 60_000

const MAX_NOTIFICATIONS = 30

/** How many issues are looked at in parallel (each is one request to Atlassian). */
const PARALLEL = 4
```

- `FEED_INTERVAL_MS`: notifications and requests are checked every 5 minutes in the background.
- `STALE_MS`: opening the Jira page checks again if the last check is over a minute old, so what the user sees is never more than a minute out of date, without a request every time.
- `MAX_NOTIFICATIONS = 30`: the most the tab lists.
- `PARALLEL = 4`: finding out who assigned each issue takes one request per issue; as the comment says, at most four run at once, so a long list doesn't flood Atlassian.

`AtlassianMemorySchema` and `AtlassianMemory`

src/main/atlassian/Atlassian.ts, lines 73–84

```ts
/** What the app remembers between runs (atlassian.json): what the user has already seen. */
export const AtlassianMemorySchema = z.object({
  /** Notifications up to this ISO time have been seen. */
  notificationsReadAt: z.string().nullable().optional(),
  /**
   * For each of the user's requests, as "status|latest reply": what they've seen in the panel,
   * and what they've had a Windows notification about.
   */
  requests: z.record(z.string(), z.object({ seen: z.string(), notified: z.string() })).optional(),
})

export type AtlassianMemory = z.infer<typeof AtlassianMemorySchema>
```

What's remembered between runs, in `atlassian.json` in the app's data folder (not encrypted: it holds issue keys, times and statuses, no secrets). `index.ts` reads it through this zod schema, so a damaged file just means starting afresh.

- `notificationsReadAt`: notifications up to this time have been seen; newer ones get a dot.
- `requests`: for each of the user's requests, by key, its state written as `"status|latest reply"`, twice: what the user has `seen` in the panel, and what they've been `notified` about with a Windows notification. Keeping the two apart means a change gives one notification, but stays marked in My requests until the user looks.
- `AtlassianMemory`: the TypeScript type, worked out from the schema (`z.infer`).

`AtlassianDeps` and `loading()`

src/main/atlassian/Atlassian.ts, lines 86–109

```ts
export interface AtlassianDeps {
  appName: string
  site: string
  redirectPort: number
  /** Confluence spaces searched for how-to articles while raising an IT request. */
  knowledgeSpaces: string[]
  /** The service desk's requests, for My requests; null without. */
  requests: { helpCentre: string; portalId: string; projectKey: string } | null
  connector: McpConnector
  /** The connection to Atlassian, saved encrypted (Windows DPAPI). */
  store: SavedSecret
  memory: { load(): Promise<AtlassianMemory>; save(memory: AtlassianMemory): Promise<void> }
  listen(port: number): Promise<RedirectListener>
  openBrowser(url: string): Promise<void>
  onState(state: AtlassianState): void
  onFeed(feed: JiraFeed): void
  /** A Windows notification: IT replied to one of the user's requests, or changed its status. */
  notify(message: { title: string; body: string }): void
  onError?(message: string, error?: unknown): void
  now?: () => number
  feedIntervalMs?: number
}

const loading = <T>(): FeedList<T> => ({ status: 'loading', items: [] })
```

`AtlassianDeps` is everything `Atlassian` needs from outside; `start()` in `index.ts` passes the real ones.

- `appName`, `redirectPort`, `connector`, `store`, `listen`, `openBrowser`: handed on to the `McpAccount`, as `PortalApps` does. `store` is a `SecretStore` over `atlassian.bin`.
- `site`: the company's Atlassian site, from the tenant (`https://morsemicro.atlassian.net`).
- `knowledgeSpaces`: as the comment says, the Confluence spaces searched for how-to articles.
- `requests`: the service desk's portal and Jira project, for My requests; `null` when the tenant has no request types.
- `memory`: reads and writes `atlassian.json`.
- `onState(state)`, `onFeed(feed)`: called on every change; `index.ts` broadcasts them to the page.
- `notify(message)`: a Windows notification, as the comment says.
- `now?`, `feedIntervalMs?`: replace the clock and the 5 minutes in the tests.
- `loading<T>()`: an empty list that's still loading, the starting point for both lists. `<T>` makes it work for a list of anything.

`Atlassian`

src/main/atlassian/Atlassian.ts, lines 111–116

```
/**
 * Jira and Confluence through Atlassian's Remote MCP server: the user connects once, in the
 * browser, and then sees who assigned them issues or mentioned them, their IT requests, search,
 * and can work on an issue. Notifications and requests are checked every few minutes.
 */
export class Atlassian {
  // …
}
```

The class `index.ts` and `ipc.ts` use, with the comment that sums it up.

`Atlassian` fields, constructor, `current` and `currentFeed`

src/main/atlassian/Atlassian.ts, lines 117–157

```
private state: AtlassianState = { status: 'loading' }

private feedState: JiraFeed = { notifications: loading(), requests: loading() }

private readonly account: McpAccount

private user: AtlassianUser | null = null

private cloudId: string | null = null

private memory: AtlassianMemory = {}

private readonly memoryLoaded: Promise<void>

private refreshing: Promise<JiraFeed> | null = null

private lastRefresh = 0

private timer: ReturnType<typeof setInterval> | null = null

/** Who assigned each issue, by key, until the issue changes. */
private readonly assignments = new Map<
  string,
  { updated: string; found: FoundNotification | null }
```

```
>()

/** Each request's state as last loaded ("status|latest reply"). */
private readonly requestSignatures = new Map<string, string>()

private projectKeyList: Promise<string[] | null> | null = null

constructor(private readonly deps: AtlassianDeps) {
  this.account = new McpAccount({
    appName: deps.appName,
    redirectPort: deps.redirectPort,
    connector: deps.connector,
    store: deps.store,
    listen: (port) => this.deps.listen(port),
    openBrowser: (url) => this.deps.openBrowser(url),
  })
  this.memoryLoaded = deps.memory.load().then(
    (memory) => void (this.memory = memory),
    () => {},
  )
}

get current(): AtlassianState {
  return this.state
}

get currentFeed(): JiraFeed {
  return this.feedState
}
```

- `state`: the connection (`AtlassianState`: loading, sign-in, signing-in, connected or error). Every change goes through `setState()`, which also reports it.
- `feedState`: the two lists, notifications and requests, each with its own status (`JiraFeed`).
- `account`: the connection to Atlassian's server (`McpAccount`). Unlike JumpCloud's, it's given no `scope`: Atlassian's server decides what the app may do.
- `user`, `cloudId`: who's connected, and their site's cloud ID, found by `identify()`.
- `memory`, `memoryLoaded`: what's remembered (above), and the promise of reading it, started in the constructor. A file that can't be read leaves `memory` empty (`() => {}`).
- `refreshing`: the check under way, if any, so two at once share one. `lastRefresh`: when the last one finished.
- `timer`: the 5-minute check.
- `assignments`: as the comment says, who assigned each issue, kept until the issue changes, so each check doesn't read every issue's history again.
- `requestSignatures`: each request's `"status|latest reply"` as last loaded, for marking it seen.
- `projectKeyList`: the Jira project keys the user can see, loaded once (see `projectKeys()`).
- `get current()`, `get currentFeed()`: the state and lists as they are now, for the tests and for `atlassian.get()`.

`Atlassian.init()`

src/main/atlassian/Atlassian.ts, lines 159–173

```
/** Uses the saved connection, if there is one (never opens the browser), and starts checking. */
async init(): Promise<AtlassianState> {
  await this.memoryLoaded
  try {
    await this.identify()
    this.connected()
  } catch (error) {
    if (error instanceof NeedsSignInError) this.setState({ status: 'sign-in' })
    else {
      this.deps.onError?.('Connecting to Atlassian failed:', error)
      this.setState({ status: 'error', message: this.problem(error) })
    }
  }
  return this.state
}
```

What `index.ts` calls at startup. As the comment says, it only uses a saved connection, and never opens the browser.

- `await this.identify()`: who the user is; this is the first tool call, so it's also what finds out whether the saved connection still works.
- `this.connected()`: shows the user as connected and starts the checks.
- `if (error instanceof NeedsSignInError)`: nobody has connected yet, or the connection has ended: the Connect button, without a message.
- Anything else (Atlassian can't be reached, say) is an `error` state with a message, and the page offers Retry.

`Atlassian.signIn()`, `cancelSignIn()`, `signOut()` and `retry()`

src/main/atlassian/Atlassian.ts, lines 175–213

```
/** Connects in the browser (through the company's sign-in), then loads the feed. */
async signIn(): Promise<void> {
  if (this.state.status === 'signing-in') return
  this.setState({ status: 'signing-in' })
  try {
    await this.account.signIn()
    await this.identify()
    this.connected()
  } catch (error) {
    if (error instanceof SignInCancelledError) {
      this.setState({ status: 'sign-in' })
      return
    }
    const message =
      error instanceof SignInTimeoutError
        ? 'Connecting took too long. Try again.'
        : `Connecting didn't work: ${error instanceof Error ? error.message : String(error)}`
    this.deps.onError?.('Connecting to Atlassian failed:', error)
    this.setState({ status: 'sign-in', message })
  }
}

cancelSignIn(): void {
  this.account.cancelSignIn()
}

/** Forgets the connection (Settings → Disconnect Atlassian). */
async signOut(): Promise<void> {
  this.stopChecking()
  await this.account.signOut()
  this.forget()
  this.setState({ status: 'sign-in' })
}

/** Tries the saved connection again after an error. */
retry(): Promise<AtlassianState> {
  this.setState({ status: 'loading' })
  return this.init()
}
```

- `signIn()`: **Connect to Atlassian**. The same pattern as `PortalApps.signIn()`: `signing-in` while the browser is open, `McpAccount.signIn()` for the browser part, then `identify()` and `connected()`. Cancel goes back to the Connect button; a timeout or any other failure goes back to it with a message, and is logged.
- `cancelSignIn()`: closes the listener, which ends `signIn()`'s wait.
- `signOut()`: **Disconnect Atlassian** in the settings menu. The checks stop, `McpAccount.signOut()` deletes the saved connection, `forget()` clears everything known about the user, and the page goes back to Connect.
- `retry()`: the Retry button after an error: `loading` again, then `init()`.

`Atlassian.feed()`

src/main/atlassian/Atlassian.ts, lines 215–220

```
/** Notifications and requests, checked again first if asked or if the last check is old. */
async feed(refresh = false): Promise<JiraFeed> {
  if (this.state.status !== 'connected') return this.feedState
  if (refresh || this.now() - this.lastRefresh > STALE_MS) return this.refresh()
  return this.feedState
}
```

What the Jira page and My requests ask for when they show, and on Refresh (`refresh` true).

- `if (this.state.status !== 'connected') return this.feedState`: nothing to check until connected.
- `this.now() - this.lastRefresh > STALE_MS`: checks again first if the last check is over a minute old; otherwise the lists as they are.

`Atlassian.markNotificationsRead()` and `Atlassian.markRequestsRead()`

src/main/atlassian/Atlassian.ts, lines 222–253

```
/** The user has looked at the Notifications tab. */
async markNotificationsRead(): Promise<JiraFeed> {
  const { items } = this.feedState.notifications
  if (!items.some((n) => n.unread)) return this.feedState
  const newest = items.reduce((latest, n) => (n.at > latest ? n.at : latest), '')
  this.memory.notificationsReadAt = newest || new Date(this.now()).toISOString()
  this.setFeed({
    ...this.feedState,
    notifications: { ...this.feedState.notifications, items: items.map(read) },
  })
  await this.saveMemory()
  return this.feedState
}

/** The user has seen what's new on a request (or on all of them, without a key). */
async markRequestsRead(key?: string): Promise<JiraFeed> {
  const requests = this.memory.requests ?? {}
  let changed = false
  const items = this.feedState.requests.items.map((request) => {
    if (key !== undefined && request.key !== key) return request
    const signature = this.requestSignatures.get(request.key)
    if (signature) requests[request.key] = { seen: signature, notified: signature }
    if (!request.news) return request
    changed = true
    return { ...request, news: null }
  })
  if (!changed) return this.feedState
  this.memory.requests = requests
  this.setFeed({ ...this.feedState, requests: { ...this.feedState.requests, items } })
  await this.saveMemory()
  return this.feedState
}
```

- `markNotificationsRead()`: the Jira page calls this two seconds after the Notifications tab shows. `items.reduce(...)` finds the newest notification's time (ISO times compare correctly as text), which becomes `notificationsReadAt`; every notification loses its dot (`read`, at the end of the file), and the memory is saved.
- `markRequestsRead(key?)`: the user has seen what's new on one request (opening it in the card), or on all of them without a key. Its current signature becomes both `seen` and `notified`, and its `news` is cleared. `changed`

makes sure nothing is reported or saved if there was nothing new.

`Atlassian.search()`

src/main/atlassian/Atlassian.ts, lines 255–273

```ts
/** Jira issues and Confluence pages matching the words (an issue key finds that issue). */
async search(query: string): Promise<SearchResult> {
  const q = query.trim()
  if (!q) return { ok: true, issues: [], pages: [] }
  let failure: unknown = null
  const failed = (error: unknown) => {
    failure ??= error
    return []
  }
  const [issues, pages] = await Promise.all([
    this.searchIssues(q).catch(failed),
    this.searchPages(q, [], 6).catch(failed),
  ])
  if (failure && issues.length === 0 && pages.length === 0) {
    this.deps.onError?.('Searching Atlassian failed:', failure)
    return { ok: false, message: this.problem(failure) }
  }
  return { ok: true, issues, pages }
}
```

The Search tab: Jira issues and Confluence pages, asked for at the same time (`Promise.all`).

- `const failed = (error) => { failure ??= error; return [] }`: a search that fails counts as no results, and the first failure is kept.
- `if (failure && issues.length === 0 && pages.length === 0)`: only when nothing at all came back does the tab show the problem. If Confluence failed but Jira found issues, the issues show.

`Atlassian.issue()`

src/main/atlassian/Atlassian.ts, lines 275–312

```
/** An issue with its latest comments and the statuses it can move to. */
async issue(key: string): Promise<IssueResult> {
  try {
    const node = issueNodes(
      await this.call(TOOLS.issue, {
        issueIdOrKey: key,
        fields: [
          'summary',
          'status',
          'issuetype',
          'assignee',
          'reporter',
          'created',
          'updated',
          'description',
          'comment',
        ],
        responseContentFormat: 'markdown',
      }),
    )[0]
    const transitions = await this.call(TOOLS.transitions, { issueIdOrKey: key })
      .then(transitionsOf)
      .catch(() => [])
    const { requests } = this.deps
    const portalUrl =
      requests && key.startsWith(`${requests.projectKey}-`)
        ? portalRequestUrl(requests.helpCentre, requests.portalId, key)
        : null
    const issue = node && issueDetailOf(node, this.deps.site, { transitions, portalUrl })
    if (!issue) return { ok: false, message: `Couldn't find ${key}.` }
    // Opening one of the user's requests counts as seeing what's new on it.
    if (portalUrl) void this.markRequestsRead(key)
    return { ok: true, issue }
  } catch (error) {
    this.deps.onError?.(`Opening ${key} failed:`, error)
    return { ok: false, message: this.problem(error) }
  }
}
```

An issue for `IssueView`, with its latest comments and the statuses it can move to.

- `fields: [...]`: only the fields that are shown, so the answer stays small.
- `responseContentFormat: 'markdown'`: the description and comments as Markdown, which the page shows with its `Markdown` component.
- `this.call(TOOLS.transitions, ...).then(transitionsOf).catch(() => [])`: the possible moves, asked for separately. If that fails, the issue still opens, just without Move.
- `key.startsWith(\`${requests.projectKey}-`)`: an issue in the service desk's project (`ISD-`) is one of the user's requests, so it gets a link to its portal page too.
- `if (portalUrl) void this.markRequestsRead(key)`: as the comment says, opening a request counts as seeing what's new on it.
- `catch (error)`: any failure is logged and becomes a message for the page.

`Atlassian.transition()`, `comment()` and `logWork()`

src/main/atlassian/Atlassian.ts, lines 314–329

```
/** Moves the issue to another status, then answers with it as it is now. */
transition(key: string, transitionId: string): Promise<IssueResult> {
  return this.change(key, 'Moving', TOOLS.transition, { transition: { id: transitionId } })
}

comment(key: string, body: string): Promise<IssueResult> {
  return this.change(key, 'Commenting on', TOOLS.comment, {
    commentBody: body,
    contentFormat: 'markdown',
  })
}

/** Logs time on the issue, in Jira's format ("30m", "1h 30m", "2d"). */
logWork(key: string, timeSpent: string): Promise<IssueResult> {
  return this.change(key, 'Logging time on', TOOLS.worklog, { timeSpent })
}
```

The three changes **Move**, **Comment** and **Log time** make. Each goes through `change()` (below), which makes the call and then answers with the issue as it is now.

- `transition`: makes the transition with that ID.
- `comment`: `contentFormat: 'markdown'` lets Atlassian turn the comment's Markdown into ADF.
- `logWork`: the time in Jira's own format; `ipc.ts` has already checked it (`TimeSpent`).

`Atlassian.articles()`, `projectKeys()`, `prompt()` and `dispose()`

src/main/atlassian/Atlassian.ts, lines 331–364

```
/** How-to pages from the knowledge spaces about what the user is typing (up to three). */
async articles(value: string): Promise<ConfluencePage[]> {
  if (this.state.status !== 'connected' || searchWords(value).length < 3) return []
  return this.searchPages(value, this.deps.knowledgeSpaces, 3).catch((error: unknown) => {
    this.deps.onError?.('Looking for articles failed:', error)
    return []
  })
}

/** The keys of the Jira projects the user can see (to spot issue keys in the text box). */
projectKeys(): Promise<string[] | null> {
  if (this.state.status !== 'connected') return Promise.resolve(null)
  this.projectKeyList ??= this.loadProjectKeys().catch((error: unknown) => {
    this.deps.onError?.('Listing Jira projects failed:', error)
    this.projectKeyList = null
    return null
  })
  return this.projectKeyList
}

/** What to ask Claude about the issue, with the issue's details in it. */
async prompt(
  key: string,
  kind: IssuePrompt,
  max: number,
): Promise<{ ok: true; prompt: string } | { ok: false; message: string }> {
  const result = await this.issue(key)
  return result.ok ? { ok: true, prompt: issuePrompt(result.issue, kind, max) } : result
}

dispose(): void {
  this.stopChecking()
  void this.account.dispose()
}
```

- `articles(value)`: the "These might help:" links on the New request form: up to three pages from the knowledge spaces about what the user is typing. Nothing is searched until there are three characters of words, and a failure just means no links.
- `projectKeys()`: the keys of the Jira projects the user can see, so the text box only offers to open issue keys that could be real (a "COVID-19" in the text isn't an issue). `??=` loads them once and shares the promise; a failure is logged and forgotten (`this.projectKeyList = null`), so the next call tries again. `null` means "not known", and then every key is offered.
- `prompt(key, kind, max)`: the question for **Ask Claude**: the issue as `issue()` reads it, put into words by `issuePrompt()`. If the issue can't be read, its message is passed on.
- `dispose()`: when the app quits, the checks stop and the connection is closed.

`Atlassian.now()`, `identify()`, `connected()` and `forget()`

src/main/atlassian/Atlassian.ts, lines 366–396

```
// ---------------------------------------------------------------------------- inner

private now(): number {
  return (this.deps.now ?? Date.now)()
}

/** Who the user is, and the site's cloud ID (the tools want one). */
private async identify(): Promise<void> {
  const user = userOf(await this.account.call(TOOLS.user, {}))
  if (!user) throw new Error("Couldn't read your Atlassian account")
  const sites = await this.account.call(TOOLS.sites, {}).catch(() => null)
  this.user = user
  this.cloudId = (sites && cloudIdOf(sites, this.deps.site)) ?? new URL(this.deps.site).host
}

private connected(): void {
  const { name, email } = this.user!
  this.setState({ status: 'connected', user: email ? { name, email } : { name } })
  void this.refresh()
  this.startChecking()
}

private forget(): void {
  this.user = null
  this.cloudId = null
  this.assignments.clear()
  this.requestSignatures.clear()
  this.projectKeyList = null
  this.lastRefresh = 0
  this.setFeed({ notifications: loading(), requests: loading() })
}
```

- `// ---- inner`: the comment line that starts the class's private helpers.
- `now()`: the time, from the tests' clock if they give one. `(this.deps.now ?? Date.now)()` picks the function, then calls it.
- `identify()`: who the user is (`atlassianUserInfo`), and the site's cloud ID from the sites they can reach. These two calls go to `this.account` directly, since `call()` (below) adds the cloud ID, which isn't known yet. If the sites can't be listed, or the site isn't among them, the site's host is used instead (`?? new URL(this.deps.site).host`): the tools accept a site's address in place of its ID too.
- `connected()`: the `connected` state with the user's name (and email, if any) for the settings menu, a first check straight away, and the 5-minute checks. `this.user!`: the `!` tells TypeScript that `user` can't be `null` here, because `identify()` has just set it.
- `forget()`: everything known about the user goes, and both lists go back to loading.

`Atlassian.startChecking()` and `Atlassian.stopChecking()`

src/main/atlassian/Atlassian.ts, lines 398–410

```
private startChecking(): void {
  this.stopChecking()
  this.timer = setInterval(
    () => void this.refresh(),
    this.deps.feedIntervalMs ?? FEED_INTERVAL_MS,
  )
  this.timer.unref?.()
}

private stopChecking(): void {
  if (this.timer) clearInterval(this.timer)
  this.timer = null
}
```

- `startChecking()`: a check every 5 minutes (`setInterval`), stopping any earlier timer first so there's never two. `this.timer.unref?.()`: a waiting timer doesn't keep the app from quitting. The `?.` calls `unref` only if it exists, as it does in Node.
- `stopChecking()`: stops it.

`Atlassian.call()` and `Atlassian.change()`

src/main/atlassian/Atlassian.ts, lines 412–442

```
/** A tool call on the user's site. A connection that has ended takes the page back to Connect. */
private async call(name: string, args: Record<string, unknown>): Promise<ToolResult> {
  try {
    return await this.account.call(name, { cloudId: this.cloudId, ...args })
  } catch (error) {
    if (error instanceof NeedsSignInError && this.state.status === 'connected') {
      this.stopChecking()
      this.forget()
      this.setState({
        status: 'sign-in',
        message: 'Your Atlassian connection has ended. Connect again.',
      })
    }
    throw error
  }
}

private async change(
  key: string,
  doing: string,
  tool: string,
  args: Record<string, unknown>,
): Promise<IssueResult> {
  try {
    await this.call(tool, { issueIdOrKey: key, ...args })
  } catch (error) {
    this.deps.onError?.(`${doing} ${key} failed:`, error)
    return { ok: false, message: this.problem(error) }
  }
  return this.issue(key)
}
```

- `call(name, args)`: a tool call on the user's site: `{ cloudId: this.cloudId, ...args }` adds the cloud ID to the arguments. As the comment says, a connection that has ended (`NeedsSignInError` while `connected`: the refresh token has expired, say) stops the checks, forgets the user and takes the page back to Connect, with a message saying why. The error is passed on either way.
- `change(key, doing, tool, args)`: one change to an issue. A failure is logged as, say, "Moving ISD-12 failed" and becomes a message; on success it answers with `issue(key)`, so the page shows the new status or comment straight away.

`Atlassian.refresh()`, `loadFeed()` and `failedList()`

src/main/atlassian/Atlassian.ts, lines 444–475

```
private refresh(): Promise<JiraFeed> {
  this.refreshing ??= this.loadFeed().finally(() => (this.refreshing = null))
  return this.refreshing
}

private async loadFeed(): Promise<JiraFeed> {
  const user = this.user
  if (!user || this.state.status !== 'connected') return this.feedState
  await this.memoryLoaded
  const [notifications, requests] = await Promise.all([
    this.loadNotifications(user).catch((error: unknown) =>
      this.failedList<JiraNotification>('notifications', error),
    ),
    this.loadRequests(user).catch((error: unknown) =>
      this.failedList<ServiceRequest>('requests', error),
    ),
  ])
  // Disconnected meanwhile: drop what came back.
  if (this.user !== user) return this.feedState
  this.lastRefresh = this.now()
  this.setFeed({ notifications, requests })
  await this.saveMemory()
  return this.feedState
}

private failedList<T>(what: string, error: unknown): FeedList<T> {
  this.deps.onError?.(`Checking ${what} failed:`, error)
  const previous =
    what === 'notifications' ? this.feedState.notifications : this.feedState.requests
  // Keep what was there before; say why it couldn't be updated.
  return { status: 'error', items: previous.items as T[], message: this.problem(error) }
}
```

One check of notifications and requests.

- `refresh()`: `??=` shares a check that's under way, as `PortalApps.load()` does.
- `const user = this.user`: remembers who the check is for.
- `Promise.all([...])`: both lists at once. Each failure is caught separately (`failedList()`), so a problem with requests doesn't stop notifications.
- `if (this.user !== user) return this.feedState`: as the comment says, if the user disconnected while the check was running, what came back is dropped.
- `this.setFeed(...)`, `await this.saveMemory()`: reports the lists, and saves what `loadRequests()` learned.
- `failedList(what, error)`: logs the failure and, as its comment says, keeps the list that was there before, with the reason it couldn't be updated. `previous.items as T[]` tells TypeScript which kind of list it is.

`Atlassian.loadNotifications()`

src/main/atlassian/Atlassian.ts, lines 477–490

```
private async loadNotifications(user: AtlassianUser): Promise<FeedList<JiraNotification>> {
  const since = this.now() - NOTIFICATION_DAYS * DAY_MS
  const [assigned, mentioned] = await Promise.all([
    this.assignedNotifications(user, since),
    this.mentionedNotifications(user, since),
  ])
  const readAt = this.memory.notificationsReadAt ?? null
  const items = [...assigned, ...mentioned]
    .filter((n) => Date.parse(n.at) >= since)
    .sort((a, b) => b.at.localeCompare(a.at))
    .slice(0, MAX_NOTIFICATIONS)
    .map((n) => ({ ...n, unread: readAt === null || n.at > readAt }))
  return { status: 'ready', items }
}
```

- `const since = this.now() - NOTIFICATION_DAYS * DAY_MS`: two weeks ago.
- The assignments and mentions are found at the same time, then put together: `.filter(...)` keeps the last two weeks, `.sort(...)` puts the newest first (ISO times sort as text, and `b.at.localeCompare(a.at)` is the reverse order), and `.slice(0, MAX_NOTIFICATIONS)` keeps 30.
- `unread: readAt === null || n.at > readAt`: newer than the last time the tab was looked at. Before the tab has ever been opened, everything is new.

`Atlassian.assignedNotifications()`

src/main/atlassian/Atlassian.ts, lines 492–520

```
private async assignedNotifications(
  user: AtlassianUser,
  since: number,
): Promise<FoundNotification[]> {
  const result = await this.call(TOOLS.searchJira, {
    jql: ASSIGNED_JQL,
    fields: ['summary', 'updated'],
    maxResults: 20,
  })
  const found = await mapLimit(issueNodes(result), PARALLEL, async (node) => {
    const key = typeof node['key'] === 'string' ? node['key'] : ''
    const updated = String((node['fields'] as { updated?: unknown } | undefined)?.updated ?? '')
    const known = this.assignments.get(key)
    if (known && known.updated === updated) return known.found
    // Only the issue's history says who assigned it.
    const detail = issueNodes(
      await this.call(TOOLS.issue, {
        issueIdOrKey: key,
        fields: ['summary', 'assignee', 'creator', 'created'],
        expand: 'changelog',
        updateHistory: false,
      }),
    )[0]
    const notification = detail ? assignedNotification(detail, user, since) : null
    this.assignments.set(key, { updated, found: notification })
    return notification
  })
  return found.filter((n): n is FoundNotification => n !== null)
}
```

Who assigned the user each issue found by `ASSIGNED_JQL`.

- `fields: ['summary', 'updated']`, `maxResults: 20`: the search itself only needs to say which issues, and when they last changed.
- `mapLimit(issueNodes(result), PARALLEL, async (node) => {`: each issue, four at a time.
- `if (known && known.updated === updated) return known.found`: an issue that hasn't changed since the last check has the same answer, so it isn't asked about again. Checking every 5 minutes costs one search, plus one request per issue that changed.
- `expand: 'changelog'`: asks for the issue's history, which says who changed the assignee. As the comment says, nothing else does. `updateHistory: false`: looking an issue up this way doesn't count as the user viewing it in Jira.
- `this.assignments.set(key, { updated, found: notification })`: remembered, including "nobody else" (`null`).

`Atlassian.mentionedNotifications()`

src/main/atlassian/Atlassian.ts, lines 522–534

```
private async mentionedNotifications(
  user: AtlassianUser,
  since: number,
): Promise<FoundNotification[]> {
  const result = await this.call(TOOLS.searchJira, {
    jql: MENTIONED_JQL,
    fields: ['summary', 'updated', 'created', 'creator', 'description', 'comment'],
    maxResults: 20,
    // Jira's own format: its mentions carry the account ID.
    responseContentFormat: 'adf',
  })
  return issueNodes(result).flatMap((node) => mentionNotifications(node, user, since))
}
```

The issues `MENTIONED_JQL` finds, with their comments and descriptions. `responseContentFormat: 'adf'`: as the comment says, in Jira's own format, where a mention carries the account ID (`mentionsUser()`). `flatMap` puts each issue's notifications into one list.

`Atlassian.loadRequests()`

src/main/atlassian/Atlassian.ts, lines 536–576

```
private async loadRequests(user: AtlassianUser): Promise<FeedList<ServiceRequest>> {
  const { requests } = this.deps
  if (!requests) return { status: 'ready', items: [] }
  const result = await this.call(TOOLS.searchJira, {
    jql: myRequestsJql(requests.projectKey),
    fields: ['summary', 'status', 'updated', 'comment'],
    maxResults: 30,
    responseContentFormat: 'markdown',
  })
  const infos = issueNodes(result)
    .map((node) => requestInfoOf(node, user))
    .filter((info): info is RequestInfo => info !== null)
  // The very first check only learns where things are; nothing is "new" yet.
  const firstCheck = this.memory.requests === undefined
  const known = this.memory.requests ?? {}
  const memory: NonNullable<AtlassianMemory['requests']> = {}
  this.requestSignatures.clear()
  const items = infos.map((info): ServiceRequest => {
    const signature = `${info.status.name}|${info.reply?.id ?? ''}`
    const entry = known[info.key] ?? { seen: signature, notified: signature }
    memory[info.key] = entry
    this.requestSignatures.set(info.key, signature)
    const news = newsOf(entry.seen, info)
    // Each change gets one Windows notification, saying what's changed since the last one.
    const fresh = newsOf(entry.notified, info)
    if (fresh) {
      if (!firstCheck) this.deps.notify({ title: `${info.key}: ${fresh}`, body: info.summary })
      entry.notified = signature
    }
    return {
      key: info.key,
      summary: info.summary,
      status: info.status,
      updated: info.updated,
      url: portalRequestUrl(requests.helpCentre, requests.portalId, info.key),
      news,
    }
  })
  this.memory.requests = memory
  return { status: 'ready', items }
}
```

My requests, and what's new on them.

- `if (!requests) return { status: 'ready', items: [] }`: without request types, an empty list.
- `const signature = \`${info.status.name}|${info.reply?.id ?? ''}\``: the request's state as text: its status and its latest reply's ID. If either changes, so does the signature.
- `const firstCheck = this.memory.requests === undefined`: as the comment says, the very first check (just after connecting for the first time) only learns where things are. Otherwise every open request would be "new".
- `known[info.key] ?? { seen: signature, notified: signature }`: a request seen for the first time starts as already seen, so a request the user has just raised isn't news to them.
- `const news = newsOf(entry.seen, info)`: what's changed since the user last looked, shown in My requests ("Now Resolved", "Mark Duong replied").
- `const fresh = newsOf(entry.notified, info)`: what's changed since the last Windows notification. As the comment says, each change gets one: `this.deps.notify(...)`, then `entry.notified = signature`.

- `memory[info.key] = entry`, `this.memory.requests = memory`: only requests still in the list are remembered, so the file doesn't grow for ever.

`Atlassian.searchIssues()`, `searchPages()` and `loadProjectKeys()`

src/main/atlassian/Atlassian.ts, lines 578–625

```
private async searchIssues(query: string): Promise<IssueSummary[]> {
  const fields = ['summary', 'status', 'issuetype']
  const key = query.toUpperCase()
  if (isIssueKey(key)) {
    const result = await this.call(TOOLS.issue, { issueIdOrKey: key, fields }).catch(() => null)
    return result ? issueSummaries(result, this.deps.site) : []
  }
  if (!searchWords(query)) return []
  const result = await this.call(TOOLS.searchJira, {
    jql: textSearchJql(query),
    fields,
    maxResults: 10,
  })
  return issueSummaries(result, this.deps.site)
}

private async searchPages(
  query: string,
  spaces: string[],
  limit: number,
): Promise<ConfluencePage[]> {
  if (!searchWords(query)) return []
  const result = await this.call(TOOLS.searchConfluence, {
    cql: confluenceCql(query, spaces),
    limit,
  })
  return confluencePagesOf(result, this.deps.site).slice(0, limit)
}

private async loadProjectKeys(): Promise<string[]> {
  const keys: string[] = []
  for (let startAt = 0; startAt < 500; startAt += 50) {
    const payload = payloadOf(
      await this.call(TOOLS.projects, {
        action: 'browse',
        expandIssueTypes: false,
        maxResults: 50,
        startAt,
      }),
    ) as { values?: { key?: unknown }[]; isLast?: unknown } | undefined
    const page = (payload?.values ?? [])
      .map((project) => project.key)
      .filter((k): k is string => typeof k === 'string')
    keys.push(...page)
    if (payload?.isLast !== false || page.length === 0) break
  }
  return keys
}
```

- `searchIssues(query)`: a query that is an issue key (in any case: `isd-12` is `ISD-12`) asks for that issue directly; if it doesn't exist, no results. Anything else is a text search for up to 10 issues. `if (!searchWords(query)) return []`: a query of only symbols isn't sent.

- `searchPages(query, spaces, limit)`: a Confluence search, `limit` pages long, in the given spaces or everywhere.
- `loadProjectKeys()`: the projects come 50 at a time (`startAt` says where each page starts). It stops when Atlassian says it's the last page (`isLast`), when a page is empty, or after 500, whichever comes first. `action: 'browse'` asks for the projects the user can look at.

`Atlassian.problem()`, `saveMemory()`, `setState()` and `setFeed()`

src/main/atlassian/Atlassian.ts, lines 627–652

```
/** A plain-English reason for the panel. */
private problem(error: unknown): string {
  if (error instanceof NeedsSignInError) return 'Connect to Atlassian first.'
  const message = error instanceof Error ? error.message : String(error)
  if (/fetch failed|ENOTFOUND|ECONNREFUSED|ETIMEDOUT|network/i.test(message)) {
    return "Couldn't reach Atlassian. Check you're online and try again."
  }
  if (error instanceof ToolError) return `Atlassian said: ${excerpt(message, 200)}`
  return "Couldn't get that from Atlassian. Try again."
}

private async saveMemory(): Promise<void> {
  await this.deps.memory.save(this.memory).catch((error: unknown) => {
    this.deps.onError?.('Saving what you have seen in Jira failed:', error)
  })
}

private setState(state: AtlassianState): void {
  this.state = state
  this.deps.onState(state)
}

private setFeed(feed: JiraFeed): void {
  this.feedState = feed
  this.deps.onFeed(feed)
}
```

- `problem(error)`: what the page says when something fails: connect first, check you're online, `Atlassian said: ...` with the server's own words for a `ToolError` (an issue that doesn't exist, a transition that isn't allowed), or a general "try again".
- `saveMemory()`: writes `atlassian.json`; a failure is only logged, since the worst that can happen is a dot that comes back.
- `setState()`, `setFeed()`: record the new state or lists, and report them.

`read` and `newsOf()`

src/main/atlassian/Atlassian.ts, lines 655–669

```ts
const read = (notification: JiraNotification): JiraNotification =>
  notification.unread ? { ...notification, unread: false } : notification

/**
 * What's changed on a request since it was `before` ("status|latest reply"): "Now Resolved",
 * "Mark Duong replied", or both.
 */
function newsOf(before: string, info: RequestInfo): string | null {
  const [status, reply] = before.split('|')
  const news = [
    info.status.name !== status && `Now ${info.status.name}`,
    info.reply && info.reply.id !== reply && `${info.reply.author} replied`,
  ].filter(Boolean)
  return news.length ? news.join(' · ') : null
}
```

- `read(notification)`: the notification without its dot. One that's already read is returned as it is.
- `newsOf(before, info)`: as the comment says, what's changed on a request since an earlier signature. `before.split('|')` takes the signature apart again; a different status gives "Now ...", a different latest reply "... replied", and both are joined with `·`. `null` means nothing has changed.

`mapLimit()`

src/main/atlassian/Atlassian.ts, lines 671–687

```ts
/** Runs `run` over the items, at most `limit` at a time, keeping their order. */
async function mapLimit<T, R>(
  list: T[],
  limit: number,
  run: (item: T) => Promise<R>,
): Promise<R[]> {
  const results: R[] = new Array<R>(list.length)
  let next = 0
  const worker = async () => {
    while (next < list.length) {
      const index = next++
      results[index] = await run(list[index] as T)
    }
  }
  await Promise.all(Array.from({ length: Math.min(limit, list.length) }, worker))
  return results
}
```

Like `Promise.all(list.map(run))`, but with no more than `limit` running at once.

- `results`: the answers, in the same order as the list, whichever finishes first.
- `const worker = async () => {`: takes the next item that nobody has started (`next++` takes the current number and moves it on), runs it, and goes again until the list is done. JavaScript runs one piece of code at a time, so two workers can never take the same item.
- `Array.from({ length: Math.min(limit, list.length) }, worker)`: starts `limit` workers (fewer for a short list), and waits until they've all finished.

`src/main/serviceDesk.ts`: raising an IT request

The New request form's work in the main process. As the class's comment says, the request is filled in here and finished on the help centre portal, where the user is signed in as themselves. That avoids the app creating requests on anyone's behalf, and the portal can take the screenshot, which a link can't carry. Like the others, it has no Electron in it, and `tests/serviceDesk.test.ts` checks it. `start()` in `index.ts` creates it when the tenant has `serviceDesk.requests`.

`MAX_LINK_DESCRIPTION` and `PcInfo`

src/main/serviceDesk.ts, lines 3–15

```ts
/**
 * The longest description put in the portal link. Browsers and Atlassian accept much longer
 * addresses, but past a few thousand characters some proxies don't.
 */
export const MAX_LINK_DESCRIPTION = 1800

/** What's sent about the PC, so IT doesn't have to ask. */
export interface PcInfo {
  computerName: string
  windows: string
  appName: string
  appVersion: string
}
```

- `MAX_LINK_DESCRIPTION = 1800`: the description travels in the form's address, and, as the comment says, some company proxies refuse very long addresses.
- `PcInfo`: what **Include PC details** sends about the PC, so IT doesn't have to ask: its name, the Windows version, and the app's own name and version.

`windowsVersion()` and `pcDetails()`

src/main/serviceDesk.ts, lines 17–33

```ts
/**
 * Windows' name and build, e.g. "Windows 11 Pro (10.0.26200)". Windows 11 still calls itself
 * Windows 10 in the name it reports (its build, 22000 or later, gives it away).
 */
export function windowsVersion(name: string, release: string): string {
  const build = Number(release.split('.')[2] ?? 0)
  const fixed = build >= 22000 ? name.replace(/^Windows 10\b/, 'Windows 11') : name
  return `${fixed} (${release})`
}

export function pcDetails(info: PcInfo): string {
  return [
    `PC: ${info.computerName}`,
    `Windows: ${info.windows}`,
    `${info.appName}: ${info.appVersion}`,
  ].join('\n')
}
```

- `windowsVersion(name, release)`: Windows' name and version number, from Node's `os.version()` and `os.release()`. As the comment says, Windows 11 still reports itself as "Windows 10 ..." for old programs' sake;

its build number, the third part of `10.0.26200`, is 22000 or more, so the name is corrected. `\b` makes sure "Windows 10" is a whole word.

- `pcDetails(info)`: the three lines that go at the end of the description, such as `PC: MM-LAPTOP-42`.

`requestDescription()`

src/main/serviceDesk.ts, lines 35–42

```
/** The request's description: the user's words, then (if they want) the PC's details. */
export function requestDescription(details: string, pc: string | null): string {
  const parts = [details.trim(), pc ? `---\n${pc}` : ''].filter(Boolean)
  const description = parts.join('\n\n')
  return description.length > MAX_LINK_DESCRIPTION
    ? `${description.slice(0, MAX_LINK_DESCRIPTION - 30).trimEnd()}\n… (shortened)`
    : description
}
```

The description for the form: the user's words, then (if they want) a `---` line and the PC's details. If it's over `MAX_LINK_DESCRIPTION`, it's cut and ends with "… (shortened)", leaving room (30 characters) for that note; the user can add the rest on the portal.

`requestFormUrl()` and `myRequestsUrl()`

src/main/serviceDesk.ts, lines 44–67

```
/**
 * The portal's form for a request type, filled in: Jira Service Management fills the summary
 * and description from these parameters.
 */
export function requestFormUrl(options: {
  helpCentre: string
  portalId: string
  requestTypeId: string
  summary: string
  description: string
}): string {
  const origin = new URL(options.helpCentre).origin
  const query = [
    options.summary.trim() && `summary=${encodeURIComponent(options.summary.trim())}`,
    options.description && `description=${encodeURIComponent(options.description)}`,
  ].filter(Boolean)
  const base =
`${origin}/servicedesk/customer/portal/${options.portalId}/create/${options.requestTypeId}`
  return query.length ? `${base}?${query.join('&')}` : base
}

/** The user's requests on the help centre, all of them. */
export function myRequestsUrl(helpCentre: string): string {
  return `${new URL(helpCentre).origin}/servicedesk/customer/user/requests`
}
```

- `requestFormUrl(options)`: the portal's create-request form for one request type, as in `https://morsemicro.atlassian.net/servicedesk/customer/portal/1/create/1561`. As the comment says, Jira Service Management fills in the form's summary and description from `?summary=` and `&description=` on the end of the address. `encodeURIComponent` writes the text so that spaces, `&` and new lines survive the trip. Empty parts are left out.

- `myRequestsUrl(helpCentre)`: the portal's page listing all the user's requests, for **All your requests on the portal**.

`ServiceDeskDeps`

src/main/serviceDesk.ts, lines 69–81

```ts
export interface ServiceDeskDeps {
  helpCentre: string
  portalId: string
  /** The request types offered (the IPC handler only accepts these). */
  requestTypeIds: string[]
  pcInfo(): PcInfo
  openLink(url: string): Promise<void>
  /** Puts the screenshots on the clipboard as one image. False if one of them is missing. */
  copyScreenshots(paths: string[]): Promise<boolean>
  /** A Windows notification, e.g. to say the screenshot is ready to paste. */
  notify(message: { title: string; body: string }): void
  onError?(error: unknown): void
}
```

What `ServiceDesk` needs from outside; `index.ts` passes the real ones.

- `helpCentre`, `portalId`: the tenant's `serviceDesk.url` and `serviceDesk.requests.portalId`.
- `requestTypeIds`: the request types the tenant offers; as the comment says, nothing else is accepted.
- `pcInfo()`: the PC's details, read when they're needed (`hostname()` and the Windows version).
- `openLink(url)`: opens the form in the browser.
- `copyScreenshots(paths)`: as the comment says, puts the screenshots on the clipboard as one image (the same helper Claude Desktop uses).
- `notify(message)`: a Windows notification.

`ServiceDesk`, its constructor and `ServiceDesk.pcDetails()`

src/main/serviceDesk.ts, lines 83–88

```ts
/**
 * Raising IT requests from the panel: the request is filled in here and finished on the help
 * centre portal, where the user is signed in as themselves (and can attach the screenshot, which
 * a link can't carry: it goes on the clipboard).
 */
export class ServiceDesk {
  // …
}
```

src/main/serviceDesk.ts, lines 89–94

```ts
constructor(private readonly deps: ServiceDeskDeps) {}

/** What "Include PC details" adds, for the form to show. */
pcDetails(): string {
  return pcDetails(this.deps.pcInfo())
}
```

- `constructor(private readonly deps: ServiceDeskDeps) {}`: keeps the deps as a field, with nothing else to do.

- `pcDetails()`: what **Include PC details** will add, which the form shows under the checkbox before anything is sent.

`ServiceDesk.raise()`

src/main/serviceDesk.ts, lines 96–130

```
async raise(request: NewRequest, attachments: Attachment[]): Promise<RaiseResult> {
  if (!request.summary.trim() && !request.details.trim()) return { ok: false, reason: 'empty' }
  if (!this.deps.requestTypeIds.includes(request.requestTypeId)) {
    return { ok: false, reason: 'failed' }
  }
  try {
    const paths = attachments.map((attachment) => attachment.path)
    if (paths.length > 0 && !(await this.deps.copyScreenshots(paths))) {
      return { ok: false, reason: 'missing-screenshot' }
    }
    const description = requestDescription(
      request.details,
      request.includePcDetails ? this.pcDetails() : null,
    )
    await this.deps.openLink(
      requestFormUrl({
        helpCentre: this.deps.helpCentre,
        portalId: this.deps.portalId,
        requestTypeId: request.requestTypeId,
        summary: request.summary,
        description,
      }),
    )
  } catch (error) {
    this.deps.onError?.(error)
    return { ok: false, reason: 'failed' }
  }
  if (attachments.length > 0) {
    this.deps.notify({
      title: attachments.length === 1 ? 'Screenshot copied' : 'Screenshots copied as one image',
      body: 'On the request form, click in the description and press Ctrl+V to add it.',
    })
  }
  return { ok: true, screenshots: attachments.length }
}
```

Continue on the portal. `ipc.ts` passes the form's fields and the screenshots attached to the text box.

- `if (!request.summary.trim() && !request.details.trim())`: a request needs a summary or details.
- `if (!this.deps.requestTypeIds.includes(request.requestTypeId))`: only the tenant's own request types; anything else is refused.
- `await this.deps.copyScreenshots(paths)`: the screenshots go on the clipboard first. If one is missing (deleted from the screenshots folder, say), nothing is opened and the form says so.
- `await this.deps.openLink(requestFormUrl({ ... }))`: the filled-in form opens in the browser.
- `catch (error)`: anything failing is logged and becomes `failed`.
- `this.deps.notify({ ... })`: with screenshots, a Windows notification says one is on the clipboard and how to add it: click in the description and press Ctrl+V. Several screenshots are joined into one image, and the title says so.
- `return { ok: true, screenshots: attachments.length }`: `ipc.ts` then closes the panel, as the browser comes to the front.

What's tested

`tests/atlassian.test.ts` checks the readers with made-up answers, and runs `Atlassian` against a fake Atlassian: a server with a user (Casey), an issue Sam assigned them, a mention by Mark, and one IT request whose comments and status a test can change; a fake browser that "connects"; an in-memory store and memory; and a fixed clock.

- Reading: issue keys in text, each once; only the words of a search, so a query can't break JQL or CQL; mentions and text from ADF; who assigned an issue, unless it was the user themselves; the comments that mention the user, by someone else, with the comment's start; a request's latest reply from a person, not the user or an automatic one; Confluence's results as plain text, only on the company's site; the user and the cloud ID of the configured site; and the prompt to Claude kept under the limit, dropping the oldest comments first.
- `Atlassian`: asking to connect when nobody has, without opening the browser; connecting in the browser, then showing who assigned or mentioned the user; remembering which notifications have been seen, and not asking about unchanged issues again; telling the user once when IT replies to their request, and marking it until they look; opening an issue with its comments, the statuses it can move to and its portal page; moving, commenting and logging time, each answering with the issue; searching Jira and Confluence, and finding an issue by its key; asking Claude with the issue's details in the prompt; and going back to Connect when the connection ends, and forgetting it on Disconnect.

`tests/serviceDesk.test.ts` checks the portal's form address for a request type, the PC details under the description with the link kept a sensible length, and Windows 11 called by its name. It runs `ServiceDesk` with fakes: opening the filled-in form with the screenshot copied to paste; leaving the PC details out when asked, with no screenshot needed; and refusing an empty request, an unknown request type and a missing screenshot.

`tests/tenants.test.ts` checks the request types (at least one, with numeric IDs, passed to the panel through `Branding`); that the Jira icon needs `atlassian` with an `https` site, and that the MCP server and knowledge spaces get their defaults; and that a developer's override can change one Atlassian setting without resetting the others. The real Atlassian server and a real browser sign-in aren't part of the unit tests.

The Jira and service desk pages

The renderer's side of Jira and the service desk: the pages that take the text box's place in the card while the Jira icon or the service desk icon is on, and the issue view that opens over them. `Panel` in `PanelView.tsx` decides which page shows and passes it what the main process has reported: the Atlassian connection (`AtlassianState`) and the notifications and requests (`JiraFeed`), both kept up to date by broadcasts (see The pages). Everything else the pages need, they ask for themselves through `window.assist.atlassian` and `window.assist.serviceDesk` (see Shared code and the preload bridge).

Four files:

- `PageParts.tsx`: small pieces the pages share: a title row, tabs, icon buttons, a status pill, a grey note, and the Connect to Atlassian prompt.
- `JiraPage.tsx`: the Jira page, with its Notifications and Search tabs.
- `IssueView.tsx`: one issue, with Ask Claude, Move, Comment and Log time.
- `ServiceDeskPage.tsx`: the service desk page, with New request and My requests.

The pages are styled with Tailwind classes, like the rest of the panel (see The UI components); only the ones that matter are explained here.

`src/renderer/src/components/PageParts.tsx`: pieces the pages share

As the comment at the top says, these are shared by the pages that replace the text box, so the Jira and service desk pages look alike.

`PageHeader`

src/renderer/src/components/PageParts.tsx, lines 6–18

```
// Pieces shared by the pages that replace the text box (Jira, the service desk).

/** A page's title row: its icon and name, then tabs or buttons. */
export function PageHeader(props: { icon: LucideIcon; title: string; children?: ReactNode }) {
  const Icon = props.icon
  return (
    <div className="flex shrink-0 items-center gap-2 px-3.5 pt-3 pb-2">
      <Icon size={15} className="text-accent dark:text-accent-soft" aria-hidden />
      <h2 className="text-sm font-semibold">{props.title}</h2>
      <div className="ml-auto flex items-center gap-1">{props.children}</div>
    </div>
  )
}
```

A page's title row: its icon and name on the left, and whatever the page puts in it (tabs, a Refresh button) on the right (`ml-auto` pushes them there).

- `icon: LucideIcon`: the icon component itself (such as `Bell`), not a drawn icon. `const Icon = props.icon` gives it a capitalised name, because JSX treats `<Icon />` as a component but `<icon />` as an HTML tag.
- `children?: ReactNode`: what's written between `<PageHeader>` and `</PageHeader>`.
- `shrink-0`: the row keeps its height when the page's list below fills the card.

`Tabs`

src/renderer/src/components/PageParts.tsx, lines 20–56

```tsx
/** A row of tabs; `badges` puts a count beside a tab's name. */
export function Tabs<T extends string>(props: {
  tabs: { id: T; label: string }[]
  current: T
  badges?: Partial<Record<T, number>>
  onChange: (tab: T) => void
}) {
  return (
    <div role="tablist" className="flex gap-1 rounded-lg bg-zinc-100 p-0.5 dark:bg-zinc-800">
      {props.tabs.map((tab) => {
        const badge = props.badges?.[tab.id] ?? 0
        return (
          <button
            key={tab.id}
            type="button"
            role="tab"
            aria-selected={tab.id === props.current}
            onClick={() => props.onChange(tab.id)}
            className={cn(
              'inline-flex items-center gap-1 rounded-md px-2 py-0.5 text-xs font-medium',
              tab.id === props.current
                ? 'bg-white text-zinc-900 shadow-sm dark:bg-zinc-700 dark:text-white'
                : 'text-zinc-500 hover:text-zinc-800 dark:text-zinc-400 dark:hover:text-zinc-200',
            )}
          >
            {tab.label}
            {badge > 0 && (
              <span className="rounded-full bg-accent px-1.5 text-[10px] leading-4 text-white">
                {badge}
              </span>
            )}
          </button>
        )
      })}
    </div>
  )
}
```

A row of tabs, as the comment says, with an optional count beside a tab's name.

- `Tabs<T extends string>`: a *generic* component: `T` is whichever tab names the page uses (such as `'notifications' | 'search'`), so TypeScript checks that `current`, `badges` and `onChange` use only those.
- `badges?: Partial<Record<T, number>>`: a count for some of the tabs. `Partial` makes every tab's entry optional, so the Jira page can give just `{ notifications: unread }`.
- `role="tablist"`, `role="tab"`, `aria-selected`: tell screen readers these buttons are tabs, and which one is chosen.
- `{badge > 0 && (...)}`: the count, in a small accent-coloured circle, only when there's something to count.

`IconButton`

src/renderer/src/components/PageParts.tsx, lines 58–79

```
/** A small round button with just an icon (Refresh, Back, Open in Jira). */
export function IconButton(props: {
  icon: LucideIcon
  label: string
  onClick: () => void
  disabled?: boolean
  spinning?: boolean
}) {
  const Icon = props.icon
  return (
    <button
      type="button"
      onClick={props.onClick}
      title={props.label}
      aria-label={props.label}
      disabled={props.disabled}
      className="grid size-6 place-items-center rounded-md text-zinc-500 hover:bg-zinc-100 hover:text-
zinc-800 disabled:opacity-40 dark:hover:bg-zinc-800 dark:hover:text-zinc-200"
    >
      <Icon size={13} className={cn(props.spinning && 'animate-spin')} aria-hidden />
    </button>
  )
}
```

A small button with just an icon, as the comment says.

- `title={props.label}`, `aria-label={props.label}`: the label shows as a tooltip, and is the button's name for screen readers.
- `spinning`: turns the icon (`animate-spin`) while the button's work is going on, such as a Refresh.
- `disabled:opacity-40`: a disabled button is faded.

`STATUS_COLOURS` and `StatusPill`

src/renderer/src/components/PageParts.tsx, lines 81–100

```
const STATUS_COLOURS: Record<IssueStatus['category'], string> = {
  new: 'bg-zinc-200 text-zinc-700 dark:bg-zinc-700 dark:text-zinc-200',
  indeterminate: 'bg-blue-100 text-blue-800 dark:bg-blue-950 dark:text-blue-200',
  done: 'bg-green-100 text-green-800 dark:bg-green-950 dark:text-green-200',
}

/** An issue's status, coloured like Jira colours it. */
export function StatusPill({ status }: { status: IssueStatus }) {
  return (
    <span
      className={cn(
        'inline-block max-w-32 shrink-0 truncate rounded px-1.5 py-px text-[10px] font-semibold
uppercase',
        STATUS_COLOURS[status.category],
      )}
      title={status.name}
    >
      {status.name}
    </span>
  )
}
```

An issue's status as a small coloured label, like Jira's: grey for to do (`new`), blue for in progress (`indeterminate`), green for done. Each colour has a darker version for dark mode (`dark:`).

- `Record<IssueStatus['category'], string>`: one set of classes for each of the three categories; TypeScript complains if one is missing.
- `max-w-32 truncate`, `title={status.name}`: a long status name is cut short with `…`, and the whole name shows on hovering.

`Note` and `TextButton`

src/renderer/src/components/PageParts.tsx, lines 102–124

```
/** A line of grey text, with a spinner if it's waiting on something. */
export function Note(props: { spinner?: boolean; children: ReactNode; action?: ReactNode }) {
  return (
    <div className="flex items-start gap-2 px-3.5 pb-3 text-xs text-zinc-500">
      {props.spinner && <LoaderCircle size={14} className="shrink-0 animate-spin" aria-hidden />}
      <p className="flex-1">{props.children}</p>
      {props.action}
    </div>
  )
}

/** A small text button in the accent colour (Retry, Cancel). */
export function TextButton(props: { onClick: () => void; children: ReactNode }) {
  return (
    <button
      type="button"
      onClick={props.onClick}
      className="shrink-0 rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100 dark:text-
accent-soft dark:hover:bg-zinc-800"
    >
      {props.children}
    </button>
  )
}
```

- `Note`: a line of grey text, the pages' way of saying "loading", "nothing here" or what went wrong. `spinner` adds a turning circle in front; `action` puts a button (such as Retry) at the end.
- `TextButton`: a small text-only button in the accent colour, for the `action` of a note.

`AtlassianConnect`

src/renderer/src/components/PageParts.tsx, lines 126–177

```
/**
 * Connecting to Atlassian, in place of a page's content until it's connected: the button that
 * starts it in the browser, waiting for the browser, or what went wrong.
 */
export function AtlassianConnect(props: {
  state: AtlassianState
  appName: string
  /** What connecting shows, e.g. "who assigned you issues or mentioned you". */
  purpose: string
}) {
  const { state } = props
  if (state.status === 'loading') return <Note spinner>Connecting to Atlassian…</Note>
  if (state.status === 'signing-in') {
    return (
```

```
      <Note
        spinner
        action={
          <TextButton onClick={() => void window.assist.atlassian.cancelSignIn()}>
            Cancel
          </TextButton>
        }
      >
        Finish connecting in your browser…
      </Note>
    )
  }
  if (state.status === 'error') {
    return (
      <Note
        action={<TextButton onClick={() => void window.assist.atlassian.retry()}>Retry</TextButton>}
      >
        {state.message}
      </Note>
    )
  }
  if (state.status !== 'sign-in') return null
  return (
    <div className="space-y-2 px-3.5 pb-3">
      <p className="text-xs text-zinc-500">
        {state.message ??
          `Connect ${props.appName} to Atlassian once to see ${props.purpose}. You'll sign in with your
work account in your browser.`}
      </p>
      <button
        type="button"
        onClick={() => void window.assist.atlassian.signIn()}
        className="inline-flex items-center gap-1.5 rounded-lg bg-accent px-3 py-1.5 text-xs font-medium
text-white"
      >
        <LogIn size={13} aria-hidden /> Connect to Atlassian
      </button>
    </div>
  )
}
```

What a page shows instead of its content until the user is connected to Atlassian, for each state the main process can report:

- `loading`: "Connecting to Atlassian…" with a spinner, while `Atlassian.init()` tries a saved connection.
- `signing-in`: "Finish connecting in your browser…", with Cancel (`window.assist.atlassian.cancelSignIn()`).
- `error`: the message, with Retry (`atlassian.retry()`).
- `if (state.status !== 'sign-in') return null`: `connected` shows nothing; the page shows its content instead.
- `sign-in`: a sentence saying what connecting is for, then **Connect to Atlassian** (`atlassian.signIn()`). `state.message ?? ...`: when the connection has ended, or the last try failed, the main process's message replaces the sentence. `purpose` fills in the sentence, as the comment says, so each page explains itself ("...to see your IT requests, and when IT replies").

`src/renderer/src/components/JiraPage.tsx`: the Jira page

The page behind the Jira icon (a bell). `Panel` shows it while `view` is `'jira'`, and opens an issue over it when one is clicked.

`Tab`, `MARK_READ_AFTER_MS` and `SEARCH_DELAY_MS`

src/renderer/src/components/JiraPage.tsx, lines 16–21

```
type Tab = 'notifications' | 'search'

/** How long the unread dots stay after the Notifications tab opens, before they're marked seen. */
const MARK_READ_AFTER_MS = 2000

/** Typing pauses this long before searching. */
const SEARCH_DELAY_MS = 350
```

- `Tab`: the page's two tabs.
- `MARK_READ_AFTER_MS`: as the comment says, the dots on new notifications stay for 2 seconds after the tab opens, so the user sees what's new before it counts as seen.
- `SEARCH_DELAY_MS`: searching waits until typing has paused for 350 ms, so each key press doesn't send a search.

`JiraPage`

src/renderer/src/components/JiraPage.tsx, lines 23–117

```
/**
 * The Jira page, in the card while the Jira icon is on: who else assigned the user an issue or
 * mentioned them lately, and a search of Jira and Confluence. Opening an issue shows it here.
 */
export function JiraPage(props: {
  open: boolean
  state: AtlassianState
  feed: JiraFeed
  appName: string
  site: string
  onOpenIssue: (key: string) => void
}) {
  const [tab, setTab] = useState<Tab>('notifications')
  const [refreshing, setRefreshing] = useState(false)
  const connected = props.state.status === 'connected'
  const { notifications } = props.feed
  const unread = notifications.items.filter((n) => n.unread).length

  // Ask for the latest whenever the page shows (the main process skips it if it's recent).
  useEffect(() => {
    if (props.open && connected) void window.assist.atlassian.feed()
  }, [props.open, connected])

  // The dots show what's new for a moment, then it all counts as seen.
  useEffect(() => {
    if (!props.open || tab !== 'notifications' || unread === 0) return
    const timer = setTimeout(
      () => void window.assist.atlassian.markNotificationsRead(),
      MARK_READ_AFTER_MS,
    )
    return () => clearTimeout(timer)
  }, [props.open, tab, unread])
```

```
  async function refresh() {
    setRefreshing(true)
    await window.assist.atlassian.feed(true).finally(() => setRefreshing(false))
  }

  return (
    <div className="flex min-h-0 flex-col">
      <PageHeader icon={Bell} title="Jira">
        <Tabs
          tabs={[
            { id: 'notifications', label: 'Notifications' },
            { id: 'search', label: 'Search' },
          ]}
          current={tab}
          badges={{ notifications: unread }}
          onChange={setTab}
        />
        {connected && tab === 'notifications' && (
          <IconButton
            icon={RotateCw}
            label="Refresh"
            onClick={() => void refresh()}
            disabled={refreshing}
            spinning={refreshing}
          />
        )}
      </PageHeader>

      {!connected ? (
        <AtlassianConnect
          state={props.state}
          appName={props.appName}
          purpose={
            tab === 'notifications'
              ? 'who assigned you issues or mentioned you'
              : 'search results from Jira and Confluence'
          }
        />
      ) : tab === 'notifications' ? (
        <NotificationList
          list={notifications}
          onOpenIssue={props.onOpenIssue}
          onRetry={() => void refresh()}
        />
      ) : (
        <SearchTab open={props.open} onOpenIssue={props.onOpenIssue} />
      )}

      <div className="shrink-0 border-t border-black/5 px-2 py-1.5 dark:border-white/10">
        <button
          type="button"
          onClick={() =>
            void window.assist.openExternal(`${new URL(props.site).origin}/jira/your-work`)
          }
          className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-zinc-
600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
        >
          <ExternalLink size={13} aria-hidden /> Open Jira
        </button>
      </div>
    </div>
  )
}
```

The page itself, with the comment that sums it up.

- `open`: whether the panel is open. The page can be in the card while the panel is hidden (while a screenshot is taken, say), so this says when it's actually on screen.
- `site`: the company's Atlassian site, for **Open Jira**.
- `const unread = notifications.items.filter((n) => n.unread).length`: how many notifications have a dot, for the tab's badge.
- The first `useEffect`: as its comment says, whenever the page shows (and is connected) it asks for the latest (`atlassian.feed()`). The main process answers from memory if it checked less than a minute ago; either way the answer also arrives through the `assist:jira-feed` broadcast, which `Panel` passes back in as `feed`.
- The second `useEffect`: while the Notifications tab is on screen with something unread, a timer marks everything seen after 2 seconds (`atlassian.markNotificationsRead()`). The function it returns clears the timer, so switching tab or closing the panel sooner leaves the dots for next time.
- `refresh()`: the Refresh button (and Retry): `feed(true)` checks again even if the last check was recent. `refreshing` turns the button's icon meanwhile; `.finally()` stops it whether the check worked or not.
- `{connected && tab === 'notifications' && (<IconButton ... />)}`: Refresh only makes sense for the notifications, and only when connected.
- `{!connected ? (<AtlassianConnect ... />) : tab === 'notifications' ? (...) : (...)}`: until connected, the Connect prompt, with a `purpose` for whichever tab is chosen; then the tab's content.
- `` `${new URL(props.site).origin}/jira/your-work` ``: **Open Jira** opens Jira's "Your work" page in the browser.

`NotificationList`

src/renderer/src/components/JiraPage.tsx, lines 119–149

```tsx
function NotificationList(props: {
  list: JiraFeed['notifications']
  onOpenIssue: (key: string) => void
  onRetry: () => void
}) {
  const { list } = props
  if (list.status === 'loading' && list.items.length === 0) {
    return <Note spinner>Looking for anything new…</Note>
  }
  return (
    <>
      {list.status === 'error' && (
        <Note action={<TextButton onClick={props.onRetry}>Retry</TextButton>}>{list.message}</Note>
      )}
      {list.items.length === 0 && list.status === 'ready' && (
        <Note>
          Nothing new in the last two weeks: nobody else has assigned you an issue or mentioned you.
        </Note>
      )}
      {list.items.length > 0 && (
        <ul className="min-h-0 overflow-y-auto px-1.5 pb-1.5" aria-label="Notifications">
          {list.items.map((n) => (
            <li key={n.id}>
              <NotificationRow notification={n} onOpen={() => props.onOpenIssue(n.key)} />
            </li>
          ))}
        </ul>
      )}
    </>
  )
}
```

The Notifications tab.

- `list.status === 'loading' && list.items.length === 0`: the first check: "Looking for anything new…".
- `list.status === 'error'`: the reason, with Retry. As `failedList()` in the main process keeps the previous notifications, they still show below it.
- `list.items.length === 0 && list.status === 'ready'`: nothing in the last two weeks, said in full so the user knows what the tab is for.
- `overflow-y-auto`: a long list scrolls inside the card.

NotificationRow

src/renderer/src/components/JiraPage.tsx, lines 151–192

```
function NotificationRow({
  notification: n,
  onOpen,
}: {
  notification: JiraNotification
  onOpen: () => void
}) {
  const Icon = n.kind === 'assigned' ? UserPlus : AtSign
  return (
    <button
      type="button"
      onClick={onOpen}
      className="flex w-full items-start gap-2.5 rounded-xl px-2 py-2 text-left hover:bg-zinc-100
dark:hover:bg-zinc-800"
    >
      <span className="relative mt-0.5 grid size-7 shrink-0 place-items-center rounded-full bg-accent/10
text-accent dark:text-accent-soft">
        <Icon size={14} aria-hidden />
        {n.unread && (
          <span
            aria-label="New"
            className="absolute -top-0.5 -right-0.5 size-2.5 rounded-full border-2 border-white bg-accent
dark:border-zinc-900"
          />
        )}
      </span>
      <span className="min-w-0 flex-1 text-xs">
        <span className="flex items-baseline gap-2">
          <span className={cn('min-w-0 flex-1 truncate', n.unread && 'font-semibold')}>
            {n.by} {n.kind === 'assigned' ? 'assigned you' : 'mentioned you'}
          </span>
          <span className="shrink-0 text-[11px] text-zinc-500">{timeAgo(n.at)}</span>
        </span>
        <span className="block truncate text-zinc-500">
          <span className="font-medium text-zinc-600 dark:text-zinc-300">{n.key}</span> {n.summary}
        </span>
        {n.excerpt && (
          <span className="mt-0.5 line-clamp-2 text-[11px] text-zinc-500 italic">
            “{n.excerpt}”
          </span>
        )}
      </span>
    </button>
  )
}
```

One notification, as a button that opens its issue.

- `notification: n`: renames the prop to `n` for this function, to keep the lines short.
- `const Icon = n.kind === 'assigned' ? UserPlus : AtSign`: a person with a plus for an assignment, `@` for a mention.
- `{n.unread && (<span aria-label="New" ... />)}`: the dot, on the icon's top-right corner (`absolute -top-0.5 -right-0.5`). Its white border (`border-2 border-white`) separates it from the icon.
- `n.unread && 'font-semibold'`: a new notification's first line is bold too.
- `{n.by} {n.kind === 'assigned' ? 'assigned you' : 'mentioned you'}`: "Sam Bolin assigned you".
- `timeAgo(n.at)`: from `src/shared/format.ts`: "5m ago", "2d ago" and so on.

- `{n.key}` then `{n.summary}`: the issue, on one line, cut short if it's long (`truncate`).
- `n.excerpt`: for a mention, the start of the comment, in quotes, on up to two lines (`line-clamp-2`).

SearchTab

src/renderer/src/components/JiraPage.tsx, lines 194–304

```
function SearchTab(props: { open: boolean; onOpenIssue: (key: string) => void }) {
  const [query, setQuery] = useState('')
  const [result, setResult] = useState<SearchResult | null>(null)
  const [searching, setSearching] = useState(false)
  const input = useRef<HTMLInputElement>(null)

  useEffect(() => {
    if (props.open) input.current?.focus()
  }, [props.open])

  const q = query.trim()
  // Results only show for a query long enough to search.
  const shown = q.length >= 2 ? result : null
  useEffect(() => {
    if (q.length < 2) return
    let alive = true
    const timer = setTimeout(() => {
      setSearching(true)
      void window.assist.atlassian
        .search(q)
        .then((found) => alive && setResult(found))
        .finally(() => alive && setSearching(false))
    }, SEARCH_DELAY_MS)
    return () => {
      alive = false
      clearTimeout(timer)
    }
  }, [q])

  return (
    <div className="flex min-h-0 flex-col">
      <label className="mx-3 mb-2 flex shrink-0 items-center gap-2 rounded-lg bg-zinc-100 px-2.5 py-1.5
dark:bg-zinc-800">
        <Search
          size={13}
          className={cn('text-zinc-400', searching && 'animate-pulse')}
          aria-hidden
        />
        <input
          ref={input}
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Search Jira and Confluence, or type an issue key"
          aria-label="Search Jira and Confluence"
          className="w-full bg-transparent text-xs outline-none placeholder:text-zinc-400"
        />
      </label>
      {shown && !shown.ok && <Note>{shown.message}</Note>}
      {shown?.ok && shown.issues.length === 0 && shown.pages.length === 0 && !searching && (
        <Note>Nothing matches “{query.trim()}”.</Note>
      )}
      {shown?.ok && (shown.issues.length > 0 || shown.pages.length > 0) && (
        <div className="min-h-0 overflow-y-auto px-1.5 pb-1.5">
          {shown.issues.length > 0 && (
            <section aria-label="Jira issues">
              <h3 className="px-2 pt-1 pb-0.5 text-[10px] font-semibold tracking-wide text-zinc-500
```

```
uppercase">
                Jira
              </h3>
              <ul>
                {shown.issues.map((issue) => (
                  <li key={issue.key}>
                    <button
                      type="button"
                      onClick={() => props.onOpenIssue(issue.key)}
                      className="flex w-full items-center gap-2 rounded-lg px-2 py-1.5 text-left text-xs
hover:bg-zinc-100 dark:hover:bg-zinc-800"
                    >
                      <span className="shrink-0 font-medium text-zinc-500">{issue.key}</span>
                      <span className="min-w-0 flex-1 truncate">{issue.summary}</span>
                      <StatusPill status={issue.status} />
                    </button>
                  </li>
                ))}
              </ul>
            </section>
          )}
          {shown.pages.length > 0 && (
            <section aria-label="Confluence pages">
              <h3 className="px-2 pt-2 pb-0.5 text-[10px] font-semibold tracking-wide text-zinc-500
uppercase">
                Confluence
              </h3>
              <ul>
                {shown.pages.map((page) => (
                  <li key={page.id}>
                    <button
                      type="button"
                      onClick={() => void window.assist.openExternal(page.url)}
                      title={`Open ${page.title} in Confluence`}
                      className="flex w-full items-start gap-2 rounded-lg px-2 py-1.5 text-left text-xs
hover:bg-zinc-100 dark:hover:bg-zinc-800"
                    >
                      <BookOpen size={13} className="mt-0.5 shrink-0 text-zinc-400" aria-hidden />
                      <span className="min-w-0 flex-1">
                        <span className="block truncate font-medium">{page.title}</span>
                        {page.excerpt && (
                          <span className="line-clamp-2 text-[11px] text-zinc-500">
                            {page.excerpt}
                          </span>
                        )}
                        {page.space && (
                          <span className="block text-[10px] text-zinc-400">{page.space}</span>
                        )}
                      </span>
                    </button>
                  </li>
                ))}
              </ul>
            </section>
          )}
        </div>
      )}
    </div>
  )
}
```

The Search tab: a search box, then the Jira issues and Confluence pages that match.

- `useEffect(() => { if (props.open) input.current?.focus() }, [props.open])`: the cursor is in the search box when the tab shows, so the user can just type.
- `const shown = q.length >= 2 ? result : null`: as the comment says, results only show for a query of two characters or more. Shortening the query hides the old results without throwing them away.
- The `useEffect` on `[q]`: each change of the query starts a 350 ms timer; the function it returns cancels the timer if the query changes again first, so only a pause sends a search (`atlassian.search(q)`). `alive` goes `false` when the query changes, so an answer that arrives late, for an old query, is ignored.
- `searching && 'animate-pulse'`: the magnifying glass pulses while a search is out.
- `shown && !shown.ok`: the search failed: its message.
- `shown?.ok && shown.issues.length === 0 && shown.pages.length === 0 && !searching`: nothing matches. `?.` reads `ok` only if there's a result at all.
- Under **Jira**, each issue is a button that opens it in the card (`onOpenIssue`), with its key, summary and `StatusPill`.
- Under **Confluence**, each page is a button that opens it in the browser (`window.assist.openExternal(page.url)`), with its title, a snippet and its space. The main process only lets through pages on the company's site.

`src/renderer/src/components/IssueView.tsx`: one issue

One Jira issue, shown in the card over the page it was opened from (Notifications, Search or My requests), or from the Open button above the text box. `Panel` gives it `key={issueKey}`, so opening another issue makes a new `IssueView` rather than reusing this one.

`PROMPTS` and `TIME_SPENT`

src/renderer/src/components/IssueView.tsx, lines 18–25

```
const PROMPTS: { id: IssuePrompt; label: string }[] = [
  { id: 'summarise', label: 'Summarise' },
  { id: 'reply', label: 'Draft a reply' },
  { id: 'next-steps', label: 'Next steps' },
]

/** Jira's time format, as the main process accepts it. */
const TIME_SPENT = /^(?=.*\d)(\d{1,3}w ?)?(\d{1,3}d ?)?(\d{1,3}h ?)?(\d{1,4}m)?$/
```

- `PROMPTS`: the three **Ask Claude** buttons: the request each one sends (an `IssuePrompt`) and its label. The words sent to Claude are in the main process (`INSTRUCTIONS` in `jiraData.ts`).
- `TIME_SPENT`: as the comment says, Jira's way of writing time, the same pattern `ipc.ts` checks (`TimeSpent`): weeks, days, hours and minutes, each optional and in that order, such as `30m`, `1h 30m` or `2d`. `(?=.*\d)` at the start is a *lookahead*: it checks there's a digit somewhere without using it up, so an empty box doesn't match. Checking here too means a mistake is explained straight away, without a trip to Atlassian.

`IssueView`

src/renderer/src/components/IssueView.tsx, lines 27–270

```
/**
 * One Jira issue: its status, description and latest comments, with what can be done from here:
 * asking Claude about it, moving it to another status, commenting, and logging time.
```

```
 */
export function IssueView(props: {
  issueKey: string
  /** Asking Claude goes to Claude Desktop (the panel closes) rather than the chat here. */
  inClaudeDesktop: boolean
  onBack: () => void
  /** The question went to the chat in the panel: show it. */
  onAskedInChat: () => void
  showToast: (text: string, tone?: 'info' | 'error') => void
}) {
  const { issueKey, showToast } = props
  const [result, setResult] = useState<IssueResult | null>(null)
  const [busy, setBusy] = useState<string | null>(null)
  const [editor, setEditor] = useState<'comment' | 'time' | 'move' | null>(null)
  const [comment, setComment] = useState('')
  const [timeSpent, setTimeSpent] = useState('')

  const load = useCallback(async () => {
    setResult(null)
    setResult(await window.assist.atlassian.issue(issueKey))
  }, [issueKey])

  // A new key gets a new IssueView (PanelView keys it), so this only runs once per issue.
  useEffect(() => {
    let alive = true
    void window.assist.atlassian.issue(issueKey).then((loaded) => {
      if (alive) setResult(loaded)
    })
    return () => {
      alive = false
    }
  }, [issueKey])

  /** Runs a change; on success shows the issue as it is now, and says what happened. */
  async function change(name: string, run: () => Promise<IssueResult>, done: string) {
    setBusy(name)
    const changed = await run().finally(() => setBusy(null))
    if (changed.ok) {
      setResult(changed)
      setEditor(null)
      showToast(done)
    } else {
      showToast(changed.message, 'error')
    }
    return changed.ok
  }

  async function ask(prompt: IssuePrompt) {
    setBusy(prompt)
    const asked = await window.assist.atlassian.ask(issueKey, prompt).finally(() => setBusy(null))
    if (!asked.ok) showToast(asked.message, 'error')
    else if (asked.inChat) props.onAskedInChat()
  }

  const issue = result?.ok ? result.issue : null

  return (
    <div className="flex min-h-0 flex-col">
      <div className="flex shrink-0 items-center gap-1.5 px-2.5 pt-2.5 pb-1.5">
        <IconButton icon={ArrowLeft} label="Back" onClick={props.onBack} />
        <span className="text-xs font-semibold text-zinc-500">{issueKey}</span>
        <div className="ml-auto flex items-center gap-1">
          <IconButton icon={RotateCw} label="Refresh" onClick={() => void load()} />
          {issue?.portalUrl && (
            <IconButton
```

```
            icon={Headset}
            label="Open on the service desk portal"
            onClick={() => void window.assist.openExternal(issue.portalUrl!)}
          />
        )}
        {issue && (
          <IconButton
            icon={ExternalLink}
            label="Open in Jira"
            onClick={() => void window.assist.openExternal(issue.url)}
          />
        )}
      </div>
    </div>

    {!result && <Note spinner>Opening {issueKey}…</Note>}
    {result && !result.ok && (
      <Note action={<TextButton onClick={() => void load()}>Retry</TextButton>}>
        {result.message}
      </Note>
    )}

    {issue && (
      <>
        <div className="min-h-0 flex-1 overflow-y-auto px-3.5 pb-2">
          <h2 className="text-sm leading-snug font-semibold">{issue.summary}</h2>
          <Meta issue={issue} />
          {issue.description && (
            <div className="mt-2 rounded-lg bg-zinc-50 px-2.5 py-1.5 text-xs dark:bg-zinc-800/60">
              <Markdown text={issue.description} />
            </div>
          )}
          {issue.comments.length > 0 && (
            <ol className="mt-3 space-y-2" aria-label="Latest comments">
              {issue.comments.map((c) => (
                <li key={c.id} className="text-xs">
                  <p className="text-[11px] text-zinc-500">
                    <span className="font-medium text-zinc-700 dark:text-zinc-300">
                      {c.author}
                    </span>{' '}
                    · {timeAgo(c.created)}
                  </p>
                  <div className="mt-0.5">
                    <Markdown text={c.body || '(no text)'} />
                  </div>
                </li>
              ))}
            </ol>
          )}
        </div>

        <div className="shrink-0 space-y-2 border-t border-black/5 px-2.5 py-2 dark:border-white/10">
          <div className="flex flex-wrap items-center gap-1">
            <Sparkles size={13} className="mx-1 text-accent dark:text-accent-soft" aria-hidden />
            <span className="sr-only">Ask Claude:</span>
            {PROMPTS.map((p) => (
              <Chip
                key={p.id}
                onClick={() => void ask(p.id)}
                busy={busy === p.id}
                disabled={busy !== null}
                title={
                  props.inClaudeDesktop
                    ? `Ask Claude (opens in Claude Desktop): ${p.label}`
                    : `Ask Claude: ${p.label}`
```

```
            }
          >
            {p.label}
          </Chip>
        ))}
      </div>
      <div className="flex flex-wrap items-center gap-1">
        {issue.transitions.length > 0 && (
          <Chip
            onClick={() => setEditor(editor === 'move' ? null : 'move')}
            pressed={editor === 'move'}
          >
            <ArrowRightLeft size={12} aria-hidden /> Move
          </Chip>
        )}
        <Chip
          onClick={() => setEditor(editor === 'comment' ? null : 'comment')}
          pressed={editor === 'comment'}
        >
          <MessageSquarePlus size={12} aria-hidden /> Comment
        </Chip>
        <Chip
          onClick={() => setEditor(editor === 'time' ? null : 'time')}
          pressed={editor === 'time'}
        >
          <Clock size={12} aria-hidden /> Log time
        </Chip>
      </div>

      {editor === 'move' && (
        <div className="flex flex-wrap gap-1" role="group" aria-label="Move to">
          {issue.transitions.map((t) => (
            <Chip
              key={t.id}
              busy={busy === `move-${t.id}`}
              disabled={busy !== null}
              onClick={() =>
                void change(
                  `move-${t.id}`,
                  () => window.assist.atlassian.transition(issueKey, t.id),
                  `${issueKey}: ${t.name}`,
                )
              }
            >
              {t.name}
            </Chip>
          ))}
        </div>
      )}
      {editor === 'comment' && (
        <form
          className="space-y-1.5"
          onSubmit={(event) => {
            event.preventDefault()
            if (!comment.trim()) return
            void change(
              'comment',
              () => window.assist.atlassian.comment(issueKey, comment.trim()),
              'Comment added',
            ).then((ok) => ok && setComment(''))
          }}
        >
          <textarea
            value={comment}
            onChange={(event) => setComment(event.target.value)}
```

```
                placeholder={`Comment on ${issueKey}…`}
                aria-label="Comment"
                autoFocus
                className="block max-h-28 min-h-14 w-full resize-none rounded-lg bg-zinc-100 px-2.5 py-
1.5 text-xs outline-none field-sizing-content placeholder:text-zinc-400 dark:bg-zinc-800"
              />
              <SubmitRow busy={busy === 'comment'} disabled={!comment.trim() || busy !== null}>
                Post comment
              </SubmitRow>
            </form>
          )}
          {editor === 'time' && (
            <form
              className="flex items-center gap-1.5"
              onSubmit={(event) => {
                event.preventDefault()
                const spent = timeSpent.trim()
                if (!TIME_SPENT.test(spent)) {
                  showToast('Write the time like 30m, 1h 30m or 2d.', 'error')
                  return
                }
                void change(
                  'time',
                  () => window.assist.atlassian.logWork(issueKey, spent),
                  `Logged ${spent} on ${issueKey}`,
                ).then((ok) => ok && setTimeSpent(''))
              }}
            >
              <input
                value={timeSpent}
                onChange={(event) => setTimeSpent(event.target.value)}
                placeholder="e.g. 30m or 1h 30m"
                aria-label="Time spent"
                autoFocus
                className="min-w-0 flex-1 rounded-lg bg-zinc-100 px-2.5 py-1.5 text-xs outline-none
placeholder:text-zinc-400 dark:bg-zinc-800"
              />
              <SubmitRow busy={busy === 'time'} disabled={!timeSpent.trim() || busy !== null}>
                Log
              </SubmitRow>
            </form>
          )}
        </div>
      </>
    )}
  </div>
 )
}
```

The issue, with the comment that sums it up.

- `inClaudeDesktop`, `onAskedInChat`: as their comments say, where Ask Claude's question goes. With Claude Desktop the panel closes; with the built-in chat, `Panel` closes the issue and shows the chat, where Claude's answer arrives.
- `result`: the issue as last loaded (`IssueResult`), or `null` while it's loading.
- `busy`: which button's work is under way: an Ask Claude prompt, `'comment'`, `'time'` or `move-` and a transition's ID. That button pulses, and the others wait.
- `editor`: which of Move, Comment and Log time is open below the buttons, if any.
- `comment`, `timeSpent`: what's typed in those editors.

- `load`: Refresh and Retry: clears the issue (so the spinner shows) and asks again. `useCallback` keeps the same function between renders while `issueKey` is the same.
- The `useEffect`: loads the issue when the view appears. As its comment says, a new key means a new `IssueView`, so this runs once per issue. `alive` ignores an answer that arrives after the view has gone.
- `change(name, run, done)`: as its comment says, runs one change (move, comment or log time). On success the main process answers with the issue as it is now, which replaces the one shown; the editor closes, and a toast says what happened ("ISD-12: Done", "Comment added"). A failure shows the main process's message. It returns whether it worked, so the comment and time boxes are only emptied on success.
- `ask(prompt)`: an Ask Claude button: `atlassian.ask(issueKey, prompt)`. With Claude Desktop, the main process closes the panel on success, so there's nothing more to do here; `asked.inChat` means it went to the built-in chat instead.
- `const issue = result?.ok ? result.issue : null`: the issue, once it has loaded successfully.
- The header: Back, the key, then Refresh, **Open on the service desk portal** (only for the user's requests, which have a `portalUrl`) and **Open in Jira**. `issue.portalUrl!`: the `!` tells TypeScript it isn't `null`, which the `issue?.portalUrl &&` around it has checked, but TypeScript doesn't carry that check into the click function.
- `{!result && <Note spinner>Opening {issueKey}…</Note>}`, then a failure's message with Retry.
- The body scrolls (`min-h-0 flex-1 overflow-y-auto`): the summary as a heading, the `Meta` line, the description, and the latest comments, oldest first, each with its author and how long ago. The description and comments come as Markdown and are drawn by `Markdown`, the same component as Claude's replies (see The UI components).
- The actions stay at the bottom (`shrink-0 ... border-t`). First the Ask Claude chips, after a sparkle icon, with "Ask Claude:" for screen readers only (`sr-only`). Their tooltip says when the question opens in Claude Desktop.
- `{issue.transitions.length > 0 && (...)}`: **Move** only when the issue can move somewhere. Each of Move, Comment and Log time opens its editor, or closes it if it's already open (`editor === 'move' ? null : 'move'`), and shows as pressed while open.
- `editor === 'move'`: a chip for each transition; clicking one makes it (`atlassian.transition()`).
- `editor === 'comment'`: a form with a text area (`autoFocus` puts the cursor in it; `field-sizing-content` makes it grow with the text, up to `max-h-28`). Submitting posts the trimmed comment. `event.preventDefault()` stops the browser's own form submission, which would reload the page.
- `editor === 'time'`: a form with one box. `TIME_SPENT.test(spent)` checks the time first, and a toast shows how to write it if it's wrong; otherwise `atlassian.logWork()`.

`Meta`

src/renderer/src/components/IssueView.tsx, lines 272–287

```
function Meta({ issue }: { issue: IssueDetail }) {
  const parts = [
    issue.type,
    issue.assignee ? `Assigned to ${issue.assignee}` : 'Unassigned',
    issue.reporter && `Reported by ${issue.reporter}`,
    `updated ${timeAgo(issue.updated)}`,
  ].filter(Boolean)
  return (
    <div className="mt-1 flex flex-wrap items-center gap-x-1.5 gap-y-0.5 text-[11px] text-zinc-500">
      <StatusPill status={issue.status} />
      {parts.map((part) => (
        <span key={part}>· {part}</span>
      ))}
    </div>
  )
}
```

The line under the summary: the status pill, then the type, who it's assigned to (or "Unassigned"), who reported it, and when it was last updated, each after a `·`. `.filter(Boolean)` drops the parts that are missing (an issue with no type, say).

`Chip` and `SubmitRow`

src/renderer/src/components/IssueView.tsx, lines 289–334

```
function Chip(props: {
  onClick: () => void
  children: ReactNode
  busy?: boolean
  disabled?: boolean
  pressed?: boolean
  title?: string
}) {
  return (
    <button
      type="button"
      onClick={props.onClick}
      disabled={props.disabled}
      aria-pressed={props.pressed}
      aria-busy={props.busy}
      title={props.title}
      className={cn(
        'inline-flex items-center gap-1 rounded-full border px-2.5 py-1 text-[11px] font-medium
disabled:opacity-50',
        props.pressed
          ? 'border-transparent bg-accent text-white'
          : 'border-black/10 text-zinc-700 hover:bg-zinc-100 dark:border-white/10 dark:text-zinc-200
dark:hover:bg-zinc-800',
        props.busy && 'animate-pulse',
      )}
    >
      {props.children}
    </button>
  )
}

function SubmitRow(props: { busy: boolean; disabled: boolean; children: ReactNode }) {
  return (
```

```
    <div className="flex justify-end">
      <button
        type="submit"
        disabled={props.disabled}
        aria-busy={props.busy}
        className={cn(
          'rounded-full bg-accent px-3 py-1 text-xs font-medium text-white disabled:opacity-40',
          props.busy && 'animate-pulse',
        )}
      >
        {props.children}
      </button>
    </div>
  )
}
```

- `Chip`: the round-ended buttons. `pressed` fills it with the accent colour (and sets `aria-pressed`, so screen readers say it's on); `busy` makes it pulse (and sets `aria-busy`).
- `SubmitRow`: a form's submit button, on the right. `type="submit"` means clicking it (or pressing Enter in the time box) submits the form.

`src/renderer/src/components/ServiceDeskPage.tsx`: the service desk page

The page behind the service desk icon, when the tenant lists request types (otherwise the icon just opens the portal). `Panel` shows it while `view` is `'servicedesk'`; **Report to IT** under the text box opens it too.

`ServiceDeskTab`, `RAISE_ERRORS` and `ARTICLES_DELAY_MS`

src/renderer/src/components/ServiceDeskPage.tsx, lines 23–33

```
export type ServiceDeskTab = 'new' | 'mine'

/** What to say when the portal's form couldn't be opened. */
const RAISE_ERRORS: Record<Extract<RaiseResult, { ok: false }>['reason'], string> = {
  empty: 'Say what the problem is first.',
  'missing-screenshot': 'An attached screenshot is missing. Remove it and try again.',
  failed: "Couldn't open the service desk. Try again.",
}

/** Typing pauses this long before looking for articles. */
const ARTICLES_DELAY_MS = 600
```

- `ServiceDeskTab`: the page's two tabs, exported because `Panel` chooses which one it opens on.
- `RAISE_ERRORS`: as the comment says, what to say for each way raising can fail. The type picks the `reason`s of `RaiseResult`'s failures, so TypeScript makes sure each has a message.
- `ARTICLES_DELAY_MS`: looking for articles waits until typing has paused for 600 ms.

`ServiceDeskPage`

src/renderer/src/components/ServiceDeskPage.tsx, lines 35–104

```
/**
 * The service desk page, in the card while the service desk icon is on: a new request, filled
 * in from what's in the text box and finished on the help centre portal (with helpful articles
 * while typing), and the user's own requests, marked when IT has replied or changed them.
```

```
 */
export function ServiceDeskPage(props: {
  open: boolean
  initialTab: ServiceDeskTab
  requestTypes: RequestType[]
  /** The Atlassian connection for My requests and articles, or null when there's none. */
  atlassian: AtlassianState | null
  feed: JiraFeed
  appName: string
  /** The text box's words, to start the request from. */
  draft: string
  attachments: Attachment[]
  onAttachLatest: () => void
  onOpenAttachment: (attachment: Attachment) => void
  onRemoveAttachment: (attachment: Attachment) => void
  onOpenIssue: (key: string) => void
  showToast: (text: string, tone?: 'info' | 'error') => void
}) {
  const [tab, setTab] = useState<ServiceDeskTab>(props.atlassian ? props.initialTab : 'new')
  const news = props.feed.requests.items.filter((r) => r.news).length

  return (
    <div className="flex min-h-0 flex-col">
      <PageHeader icon={Headset} title="IT service desk">
        {props.atlassian && (
          <Tabs
            tabs={[
              { id: 'new', label: 'New request' },
              { id: 'mine', label: 'My requests' },
            ]}
            current={tab}
            badges={{ mine: news }}
            onChange={setTab}
          />
        )}
      </PageHeader>

      {tab === 'new' || !props.atlassian ? (
        <NewRequestForm {...props} connected={props.atlassian?.status === 'connected'} />
      ) : (
        <MyRequests
          open={props.open}
          state={props.atlassian}
          list={props.feed.requests}
          appName={props.appName}
          onOpenIssue={props.onOpenIssue}
        />
      )}

      <div className="shrink-0 border-t border-black/5 px-2 py-1.5 dark:border-white/10">
        <button
          type="button"
          onClick={() =>
            void (tab === 'mine'
              ? window.assist.serviceDesk.openMyRequests()
              : window.assist.invokeAction('servicedesk'))
          }
          className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-zinc-
600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
        >
          <ExternalLink size={13} aria-hidden />
          {tab === 'mine' ? 'All your requests on the portal' : 'Open the service desk portal'}
        </button>
      </div>
```

```
    </div>
  )
}
```

The page, with the comment that sums it up.

- `atlassian`: as its comment says, the Atlassian connection, or `null` when the tenant has no Atlassian site. Without one there's no My requests and no articles: just the form.
- `draft`, `attachments`, `onAttachLatest`, `onOpenAttachment`, `onRemoveAttachment`: the text box's words and screenshots, which the request starts from. They're the same as the text box's, so a screenshot attached here is attached there too.
- `useState<ServiceDeskTab>(props.atlassian ? props.initialTab : 'new')`: starts on the tab `Panel` asks for (always New request so far), or New request when there's no Atlassian.
- `const news = props.feed.requests.items.filter((r) => r.news).length`: how many requests have news, for My requests' badge.
- `{tab === 'new' || !props.atlassian ? (...) : (...)}`: the form, or My requests. `<NewRequestForm {...props} ... />` passes all of the page's props on, plus `connected`.
- The link at the bottom: on My requests, **All your requests on the portal** (`serviceDesk.openMyRequests()`); otherwise **Open the service desk portal**, which runs the service desk icon's ordinary action in the main process (`invokeAction('servicedesk')`): open the portal and close the panel.

NewRequestForm

src/renderer/src/components/ServiceDeskPage.tsx, lines 106–270

```
function NewRequestForm(props: Parameters<typeof ServiceDeskPage>[0] & { connected: boolean }) {
  const { requestTypes, showToast } = props
  const [requestTypeId, setRequestTypeId] = useState(requestTypes[0]?.id ?? '')
  // Starts from the text box: its first line as the summary, all of it as the details.
  const [summary, setSummary] = useState(() => firstLine(props.draft))
  const [details, setDetails] = useState(props.draft.trim())
  const [includePc, setIncludePc] = useState(true)
  const [pcDetails, setPcDetails] = useState('')
  const [articles, setArticles] = useState<ConfluencePage[]>([])
  const [sending, setSending] = useState(false)

  useEffect(() => {
    let alive = true
    void window.assist.serviceDesk.pcDetails().then((text) => alive && setPcDetails(text))
    return () => {
      alive = false
    }
  }, [])

  // How-to articles about what they're typing, from the company's knowledge spaces.
  const lookFor = `${summary} ${details}`.trim().slice(0, 300)
  const lookingFor = props.connected && lookFor.length >= 4
  const shownArticles = lookingFor ? articles : []
  useEffect(() => {
    if (!lookingFor) return
    let alive = true
    const timer = setTimeout(() => {
      void window.assist.atlassian.articles(lookFor).then((found) => alive && setArticles(found))
    }, ARTICLES_DELAY_MS)
    return () => {
      alive = false
      clearTimeout(timer)
```

```
    }
  }, [lookFor, lookingFor])

  async function raise() {
    setSending(true)
    const result = await window.assist.serviceDesk
      .raise({ requestTypeId, summary, details, includePcDetails: includePc })
      .finally(() => setSending(false))
    // On success the panel has closed and the portal's form is opening in the browser.
    if (!result.ok) showToast(RAISE_ERRORS[result.reason], 'error')
  }

  const canSend = !sending && (summary.trim() !== '' || details.trim() !== '')
  const screenshots = props.attachments.length

  return (
    <form
      className="flex min-h-0 flex-col"
      onSubmit={(event) => {
        event.preventDefault()
        if (canSend) void raise()
      }}
    >
      <div className="min-h-0 space-y-2 overflow-y-auto px-3.5 pb-2">
        <Field label="What kind of request?">
          <select
            value={requestTypeId}
            onChange={(event) => setRequestTypeId(event.target.value)}
            className="w-full rounded-lg bg-zinc-100 px-2 py-1.5 text-xs outline-none dark:bg-zinc-800"
          >
            {requestTypes.map((type) => (
              <option key={type.id} value={type.id}>
                {type.name}
              </option>
            ))}
          </select>
        </Field>
        <Field label="Summary">
          <input
            value={summary}
            onChange={(event) => setSummary(event.target.value)}
            maxLength={255}
            placeholder="e.g. Outlook keeps crashing"
            autoFocus
            className="w-full rounded-lg bg-zinc-100 px-2.5 py-1.5 text-xs outline-none placeholder:text-
zinc-400 dark:bg-zinc-800"
          />
        </Field>
        <Field label="Details">
          <textarea
            value={details}
            onChange={(event) => setDetails(event.target.value)}
            placeholder="What happened, and what you've tried"
            className="block max-h-32 min-h-16 w-full resize-none rounded-lg bg-zinc-100 px-2.5 py-1.5
text-xs outline-none field-sizing-content placeholder:text-zinc-400 dark:bg-zinc-800"
          />
        </Field>

        {shownArticles.length > 0 && (
          <div className="rounded-lg border border-black/5 px-2.5 py-1.5 dark:border-white/10">
            <p className="text-[11px] font-medium text-zinc-500">These might help:</p>
            <ul>
              {shownArticles.map((page) => (
                <li key={page.id}>
                  <button
```

```
                    type="button"
                    onClick={() => void window.assist.openExternal(page.url)}
                    title={page.excerpt}
                    className="flex w-full items-center gap-1.5 rounded-md py-0.5 text-left text-xs text-
accent hover:underline dark:text-accent-soft"
                  >
                    <BookOpen size={12} className="shrink-0" aria-hidden />
                    <span className="truncate">{page.title}</span>
                  </button>
                </li>
              ))}
            </ul>
          </div>
        )}

        <div className="flex flex-wrap items-center gap-2">
          {props.attachments.map((attachment) => (
            <AttachmentChip
              key={attachment.id}
              attachment={attachment}
              onOpen={() => props.onOpenAttachment(attachment)}
              onRemove={() => props.onRemoveAttachment(attachment)}
            />
          ))}
          <button
            type="button"
            onClick={props.onAttachLatest}
            title="Attach the latest screenshot (click again for the one before)"
            className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-
zinc-600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
          >
            <Paperclip size={14} aria-hidden />
            Attach screenshot
          </button>
        </div>

        <label className="flex items-start gap-2 text-xs" title={pcDetails}>
          <input
            type="checkbox"
            checked={includePc}
            onChange={(event) => setIncludePc(event.target.checked)}
            className="mt-0.5 accent-accent"
          />
          <span>
            Include PC details
            {pcDetails && (
              <span className="block text-[11px] text-zinc-500">
                {pcDetails.split('\n').join(' · ')}
              </span>
            )}
          </span>
        </label>
      </div>

      <div className="flex shrink-0 items-center gap-2 px-3.5 pb-2.5">
        <p className="flex-1 text-[11px] text-zinc-500">
          You&apos;ll check it and send it on the portal
          {screenshots > 0 ? ', pasting the screenshot with Ctrl+V.' : '.'}
        </p>
        <button
          type="submit"
          disabled={!canSend}
          aria-busy={sending}
          className="inline-flex h-8 shrink-0 items-center gap-1 rounded-full bg-accent pr-2.5 pl-3 text-
```

```
xs font-medium text-white disabled:opacity-40"
        >
          Continue on the portal
          <ArrowUpRight size={14} aria-hidden />
        </button>
      </div>
    </form>
  )
}
```

New request: the request filled in here, then finished on the portal.

- `props: Parameters<typeof ServiceDeskPage>[0] & { connected: boolean }`: the same props as the page (`Parameters<...>[0]` is the type of `ServiceDeskPage`'s first argument), plus whether Atlassian is connected.
- `requestTypeId`: starts on the first request type in the tenant's list.
- `useState(() => firstLine(props.draft))`, `useState(props.draft.trim())`: as the comment says, the form starts from the text box: its first line as the summary, all of it as the details. They're only starting values: editing the form doesn't change the text box.
- `includePc`: **Include PC details**, on to start with. `pcDetails`: what it adds, asked for once (`serviceDesk.pcDetails()`) to show under the checkbox.
- `lookFor`, `lookingFor`: the words to look for articles about: the summary and details, up to 300 characters, once there are at least 4 and Atlassian is connected.
- The `useEffect` on `[lookFor, lookingFor]`: like the Search tab's, a pause in typing sends `atlassian.articles(lookFor)`, and a late answer for old words is ignored. `shownArticles` is empty whenever there's nothing to look for, so old articles don't linger.
- `raise()`: **Continue on the portal**: sends the form's fields (`serviceDesk.raise({ requestTypeId, summary, details, includePcDetails })`). As the comment says, on success the main process has closed the panel and the browser is opening the portal's form. A failure shows its message from `RAISE_ERRORS`.
- `canSend`: not while sending, and only with a summary or some details.
- The form: **What kind of request?** (a list of the tenant's request types), **Summary** (255 characters at most, Jira's limit) and **Details**, each in a `Field`.
- "These might help:": the articles, each opening in the browser, with the page's snippet as its tooltip.
- The screenshots, as `AttachmentChip`s (open, remove), and **Attach screenshot**, which attaches the latest one, as the button under the text box does.
- `title={pcDetails}` and `pcDetails.split('\n').join(' · ')`: the PC details on one line under the checkbox, so the user sees exactly what's added.
- At the bottom, what happens next: the user checks the request and sends it on the portal, and, with a screenshot, pastes it with Ctrl+V. `'` is how JSX text writes `'` here.

MyRequests

`src/renderer/src/components/ServiceDeskPage.tsx`, lines 272–356

```
function MyRequests(props: {
  open: boolean
  state: AtlassianState
  list: JiraFeed['requests']
  appName: string
  onOpenIssue: (key: string) => void
}) {
  const [refreshing, setRefreshing] = useState(false)
```

```
  const connected = props.state.status === 'connected'
  const { list } = props

  useEffect(() => {
    if (props.open && connected) void window.assist.atlassian.feed()
  }, [props.open, connected])

  async function refresh() {
    setRefreshing(true)
    await window.assist.atlassian.feed(true).finally(() => setRefreshing(false))
  }

  if (!connected) {
    return (
      <AtlassianConnect
        state={props.state}
        appName={props.appName}
        purpose="your IT requests, and when IT replies"
      />
    )
  }
  if (list.status === 'loading' && list.items.length === 0) {
    return <Note spinner>Getting your requests…</Note>
  }
  return (
    <div className="flex min-h-0 flex-col">
      <div className="flex shrink-0 items-center justify-between px-3.5 pb-1">
        <p className="text-[11px] text-zinc-500">Open, and closed in the last 30 days</p>
        <IconButton
          icon={RotateCw}
          label="Refresh"
          onClick={() => void refresh()}
          disabled={refreshing}
          spinning={refreshing}
        />
      </div>
      {list.status === 'error' && (
        <Note action={<TextButton onClick={() => void refresh()}>Retry</TextButton>}>
          {list.message}
        </Note>
      )}
      {list.items.length === 0 && list.status === 'ready' && (
        <Note>No requests lately. Raise one from New request.</Note>
      )}
      {list.items.length > 0 && (
        <ul className="min-h-0 overflow-y-auto px-1.5 pb-1.5" aria-label="My requests">
          {list.items.map((request) => (
            <li key={request.key}>
              <button
                type="button"
                onClick={() => props.onOpenIssue(request.key)}
                className="flex w-full items-start gap-2 rounded-xl px-2 py-2 text-left text-xs hover:bg-
zinc-100 dark:hover:bg-zinc-800"
              >
                <span className="min-w-0 flex-1">
                  <span className="flex items-center gap-2">
                    <span className="shrink-0 font-medium text-zinc-500">{request.key}</span>
                    <span className="min-w-0 flex-1 truncate">{request.summary}</span>
                  </span>
                  <span className="mt-0.5 flex items-center gap-1.5 text-[11px] text-zinc-500">
                    <StatusPill status={request.status} />
                    {request.news ? (
                      <span className="truncate font-semibold text-accent dark:text-accent-soft">
                        {request.news}
                      </span>
```

```
                    ) : (
                      <span>updated {timeAgo(request.updated)}</span>
                    )}
                  </span>
                </span>
              </button>
            </li>
          ))}
        </ul>
      )}
    </div>
  )
}
```

My requests: the user's requests, newest change first.

- The `useEffect` and `refresh()`: as on the Jira page, ask for the latest when the tab shows, and check again on Refresh.
- `if (!connected) { return (<AtlassianConnect ... />) }`: the Connect prompt until connected, with its own `purpose`.
- "Open, and closed in the last 30 days": what the list holds (`myRequestsJql()`).
- `list.status === 'error'`: the reason with Retry, above the requests as last loaded.
- Each request is a button that opens it in the card (`onOpenIssue`), which also marks its news as seen (`Atlassian.issue()`). It shows the key and summary, the status pill, then either the news in the accent colour ("Now Resolved · Mark Duong replied") or when it was last updated.

`Field` and `firstLine()`

src/renderer/src/components/ServiceDeskPage.tsx, lines 358–371

```
function Field(props: { label: string; children: ReactNode }) {
  return (
    <label className="block">
      <span className="mb-0.5 block text-[11px] font-medium text-zinc-500">{props.label}</span>
      {props.children}
    </label>
  )
}

/** The text's first line, short enough for a summary. */
function firstLine(text: string): string {
  const line = text.trim().split('\n')[0]?.trim() ?? ''
  return line.length > 120 ? `${line.slice(0, 119).trimEnd()}…` : line
}
```

- `Field`: a form field with its label above it. Wrapping both in `<label>` means clicking the label puts the cursor in the field.
- `firstLine(text)`: as the comment says, the text's first line, cut to 120 characters (with `…`) so it fits as a summary. `?.trim() ?? ''` copes with empty text.

What's tested

These components aren't unit-tested themselves. What they rely on is: the main process's side in `tests/atlassian.test.ts` and `tests/serviceDesk.test.ts` (see Jira and the service desk), `timeAgo()` in `tests/format.test.ts`, and `findIssueKeys()` (which finds the keys for the Open button) in `tests/atlassian.test.ts`.